CODERHOUSE

MATERIAL DE LECTURA

Python

Guía de lectura del curso

Contenido

MÓDULO 1 · Primeros pasos con Python y entorno de trabajo 5

- Instalar Python y VS Code para preparar el entorno 5
- Extensión oficial de Python y selección del intérprete 8
- Terminal integrada y comandos básicos para moverte en proyectos 11
- Entornos virtuales con venv para aislar el proyecto 13
- Primer script: ficha de post por consola 13
- Flashcards de repaso 15
- PRE-ENTREGA **Preentrega 1: entorno listo y primer script ejecutado** 17

MÓDULO 2 · Datos y colecciones para representar información 21

- Variables y tipos simples para describir un post 21
- Listas para organizar publicaciones del blog 38
- Diccionarios para representar autores y su perfil 52
- Tuplas para datos constantes del blog 77
- Sets: colecciones sin elementos repetidos 97
- Flashcards de repaso 113
- PRE-ENTREGA **Preentrega 2: Estructura de datos del blog lista para crecer** 116

MÓDULO 3 · Condicionales e iteraciones 128

- Decidir qué hacer con if, elif y else 128
- Recorrer posts con for y range 140
- Construir un menú interactivo con while 153
- Filtrar y buscar posts en memoria 166
- PRE-ENTREGA **Preentrega 3: sistema de consola interactivo listo para consolidar** 187

MÓDULO 4 · Funciones, validaciones y manejo de errores 200

- Funciones y parámetros: modularizar el blog paso a paso 200
- Retorno de valores y alcance de variables 212
- Validación de datos de entrada para evitar valores incorrectos 228
- Manejo de errores con try-except en funciones del blog 244
- PRE-ENTREGA **Preentrega 4: Sistema de blog modular, validado y más robusto** 252

MÓDULO 5 · Módulos, paquetes y archivos 264

- Importar módulos locales para reutilizar funciones del blog 264

Paquetes y archivo init.py en un proyecto pequeño 279
Leer y escribir archivos de texto con open() 298
Manejar rutas de archivos con pathlib 309
Organización física del proyecto de consola del blog 326
Estructura del proyecto 338
Archivos principales 338
13. Buenas prácticas de organización 339
Flashcards de repaso 344
PRE-ENTREGA **Preentrega 5: Blog por consola organizado en módulos y paquetes** 347

MÓDULO 6 · JSON y programación orientada a objetos 360
JSON: de estructuras de Python a texto intercambiable 360
Ramas y GitHub: trabajar en paralelo y publicar el repositorio 420
Instalar Django y reconocer la estructura inicial del proyecto 433
Proyecto, aplicación y configuración básica de Django 443
PRE-ENTREGA **Preentrega 7: repositorio GitHub y base inicial de Django listos** 457
PRE-ENTREGA **Entrega** 464
PRE-ENTREGA **Criterios de aceptación** 464
PRE-ENTREGA **Qué no se espera en esta entrega** 465
PRE-ENTREGA **Errores comunes a evitar** 465

MÓDULO 8 · Primer sitio Django: URLs, views, templates y static 469
Entender cómo Django decide qué página mostrar 469
Crear vistas basadas en funciones para las páginas del blog 486
Construir plantillas reutilizables con base.html e herencia 501
Cargar estilos y scripts estáticos en el sitio 517
PRE-ENTREGA **Preentrega 8: Primer sitio Django con URLs, Templates y Static** 531

MÓDULO 9 · Modelos, ORM y panel de administración 544
Modelos y campos: diseñar los datos del blog 544
Crear migraciones y aplicar cambios en la base de datos 560
Configurar el admin de Django para gestionar posts 575
Consultar posts con el ORM y enviarlos al template 589
PRE-ENTREGA **Preentrega 9: Admin configurado y posts visibles desde la base de datos** 610

MÓDULO 10 · CRUD de posts con formularios e imágenes 624
Formularios en Django: de Form a ModelForm 624

Crear posts con una vista y un formulario ligado al modelo 641
Detalle del post y rutas amigables con slug 655
Editar y eliminar posts con formularios de confirmación 674
Subir imágenes y servir archivos media en Django 693
Flashcards de repaso 720
PRE-ENTREGA Preentrega 10: CRUD de Posts con Imágenes en Django 723

MÓDULO 11 · Autenticación, usuarios y perfiles 737

Autenticación nativa de Django: usuarios, sesiones y permisos 737
Registro de usuarios con UserCreationForm 753
Login, logout y protección de sesiones 767
Perfil de usuario y avatar 782
Edición de perfil y cambio de contraseña 798
Flashcards de repaso 814
PRE-ENTREGA Preentrega 11: Autenticación, Usuarios y Perfiles en Django 817

MÓDULO 12 · Integración, Pulido y Mejora del Blog 833

Configuración segura y limpieza del proyecto 833
Class-Based Views como alternativa a Function-Based Views 843
Integración visual con templates externos 858
Mejoras de experiencia de usuario 869
Búsqueda, filtros y revisión del flujo completo 881

MÓDULO 13 · Entrega Final del Blog Web 894

ENTREGA FINAL Consolidación del blog web en GitHub 895
Material complementario 901

MÓDULO 1

Primeros pasos con Python y entorno de trabajo

Instalar herramientas de desarrollo, configurar un entorno virtual y ejecutar un script básico de Python que represente una ficha de post.

Instalar Python y VS Code para preparar el entorno

Aprende qué son Python y VS Code, instálalos correctamente siguiendo las mejores prácticas de la industria y verifica que tu entorno esté listo para empezar a programar tu primer blog web.

¡Bienvenido! Estás a punto de dar el primer paso en un viaje emocionante: aprender a programar. No importa si nunca has visto una línea de código o si la palabra "terminal" te suena a estación de autobuses. Aquí vamos a construir los cimientos de tu carrera como desarrollador, empezando desde el absoluto cero.

En esta unidad, vamos a transformar tu computadora en una verdadera estación de trabajo para el desarrollo de software. No solo vamos a instalar programas; vamos a entender qué son, por qué los elegimos y cómo se comunican entre sí para dar vida a un blog web.

1. ¿Qué es Python y por qué es el lenguaje ideal para ti?

Imagina que quieres darle instrucciones a tu computadora. El problema es que las computadoras solo entienden electricidad y números complejos (binario), y nosotros hablamos idiomas humanos. Necesitamos un **traductor**.

Python es ese traductor. Es un lenguaje de programación de alto nivel, lo que significa que su gramática y palabras se parecen mucho al inglés, haciendo que sea muy fácil de leer y escribir para nosotros.

¿Para qué se usa Python en el mundo real?

Python no es un juguete; es una de las herramientas más poderosas de la industria actual:

- **Desarrollo Web:** Gigantes como **Instagram**, **Pinterest** y **Dropbox** utilizan Python (específicamente un marco llamado Django, que aprenderás más adelante) para funcionar.
- **Inteligencia Artificial:** Es el lenguaje número uno para crear sistemas como ChatGPT o recomendaciones de Netflix.
- **Automatización:** Muchas personas lo usan para tareas aburridas, como organizar miles de archivos en carpetas o limpiar hojas de cálculo de Excel en segundos.

Concepto clave: Cuando "instalamos Python", lo que realmente estamos instalando es el **Intérprete**. El intérprete es el procesador que lee tus instrucciones escritas en archivos de texto y las convierte en acciones que la computadora puede ejecutar. Sin el intérprete, tu código es solo un texto muerto.

2. VS Code: Tu "Taller" de trabajo

Si Python es el "traductor" o el motor, **Visual Studio Code (VS Code)** es el lugar donde vas a escribir.

Mucha gente confunde el editor con el lenguaje. Una analogía útil es pensar en la escritura de una novela:

- **Python** es el idioma (español o inglés) en el que escribes.
- **VS Code** es el procesador de textos (como Microsoft Word) que usas para escribirla.

¿Por qué VS Code y no el Bloc de Notas?

Podrías escribir código en el Bloc de Notas, pero sería como intentar reparar un motor con un cuchillo de cocina. VS Code es un **Editor de Código Fuente** profesional que te ofrece:

1. **Resaltado de sintaxis:** Pone colores a las palabras para que identifiques errores visualmente (por ejemplo, los textos en un color y los números en otro).
2. **Autocompletado:** Te sugiere palabras mientras escribes, ahorrándote tiempo y errores de ortografía.
3. **Terminal integrada:** Te permite hablar con Python sin salir de la ventana donde escribes.

VS Code es actualmente el estándar de la industria. Aprender a usarlo es una habilidad que te servirá para cualquier lenguaje de programación que decidas aprender en el futuro.

3. Instalación paso a paso: Preparando el terreno

Vamos a descargar nuestras herramientas desde las **fuentes oficiales**. Esto es vital para evitar virus o versiones modificadas que no funcionen correctamente.

Paso 1: Instalar Python (El motor)

1. Ve a python.org.
2. Coloca el cursor sobre el menú **Downloads** y haz clic en el botón que indica la versión más reciente para tu sistema operativo (Windows, macOS o Linux). En Windows, se recomienda usar el "Python install manager", ya que Python está migrando hacia este nuevo instalador como formato por defecto en versiones recientes.
3. Una vez descargado ejecutar el instalador.
4. **¡MUY IMPORTANTE (Solo Windows)!:** Al terminar la instalación se abrirá la siguiente ventana en la cual deberás colocar en las respuestas marcadas con las flechas. Las 2 deberan responderse con `y`.

- *¿Por qué?* El "PATH" es como la agenda de contactos de tu computadora. Si marcas esa casilla, le estás diciendo a tu sistema operativo dónde encontrar a Python. Si no la marcas, cuando intentes usar Python más tarde, tu computadora dirá: "¿Python? No sé quién es ese".

- Una vez que coloques la segunda y presiones enter quedará esperar a que termine.

Paso 2: Instalar Visual Studio Code (El taller)

1. Ve a code.visualstudio.com.
2. Haz clic en el botón de descarga para tu sistema (ej. "Download for Windows" o "Download for macOS").
3. Ejecuta el instalador. Se recomienda marcar todas las casillas y solo queda a eleccion de cada uno la creacion del acceso directo.

4. Verificación: ¿Realmente funciona?

Un error común de principiante es asumir que porque el instalador dijo "Terminado", todo está listo. Los programadores siempre verificamos. Para esto, usaremos la **Terminal** (también llamada Consola o Línea de Comandos).

Cómo abrir la terminal:

- **Windows:** Presiona la tecla `Windows`, escribe `cmd` y presiona Enter.
- **macOS:** Presiona `Cmd + Espacio`, escribe `Terminal` y presiona Enter.

El comando de la verdad:

Escribe lo siguiente en esa ventana negra y presiona Enter:

CÓDIGO

```
python --version
```

(Nota: En algunas Mac, puede que necesites escribir `python3 --version`)

¿Qué deberías ver? Debería aparecer algo como `Python 3.12.x` (la versión exacta puede variar).

¿Qué pasa si aparece un error? Si ves un mensaje que dice "el comando no se reconoce", lo más probable es que en el paso de instalación de Python olvidaste marcar la casilla **"Add Python to PATH"**. No te preocupes, sucede a menudo. La solución más rápida es desinstalar Python y volver a instalarlo asegurándote de marcar esa casilla.

5. Errores comunes y mejores prácticas

El mito del "Python ya instalado"

Muchas computadoras (especialmente Mac y Linux) vienen con una versión de Python instalada de fábrica. Sin embargo, suele ser una versión antigua o dedicada solo al sistema operativo. **Siempre instala la versión oficial desde python.org** para asegurarte de tener las herramientas de desarrollo más recientes para tu blog.

No confundas instalar con usar

Instalar Python es como comprar un piano. Ver que está en tu sala (verificar la versión) es el siguiente paso. Pero para tocar una melodía (hacer tu blog), necesitaremos sentarnos y abrir VS Code. No te frustres si al abrir Python solo ves una ventana negra con texto; así es como se ve el motor antes de ponerle la carrocería del auto.

Mantén el orden desde el día 1

A partir de ahora, te sugiero crear una carpeta en tu computadora llamada `Proyectos_Python` o `Proyectos`. Nunca guardes tus archivos de código sueltos en el "Escritorio" o en "Descargas", ya que esto causará problemas cuando intentemos conectar tu blog con GitHub más adelante.

6. Síntesis y que sigue

En esta unidad hemos logrado:

1. Entender que **Python** es el lenguaje (el qué) y el intérprete (el motor).
2. Entender que **VS Code** es nuestra herramienta de escritura profesional.
3. Superar la barrera de la **instalación oficial**.
4. Aprender a usar la **terminal** para verificar que Python está "vivo" en nuestro sistema.

Ahora que tenemos las herramientas físicas en nuestra computadora, necesitamos que se hablen entre sí de forma más inteligente. En la siguiente unidad, aprenderemos a instalar la "traducción" de Python dentro de VS Code para que el editor se convierta en un asistente experto que nos ayude a programar.

Extensión oficial de Python y selección del intérprete

Aprende a configurar Visual Studio Code instalando la extensión oficial de Python y seleccionando el intérprete adecuado. Esta configuración es vital para contar con autocompletado, detección de errores y ejecución simplificada de tus futuros scripts del blog.

Imagina que acabas de comprar un coche último modelo (Python) y tienes un garaje increíble (Visual Studio Code). Sin embargo, el garaje está oscuro y no tienes las herramientas adecuadas para trabajar en el motor. Necesitas a un mecánico experto que te guíe, te diga qué piezas faltan y encienda las luces para que veas lo que estás haciendo.

En el mundo de la programación, la **extensión oficial de Python para VS Code** es ese mecánico experto. En esta unidad, aprenderás a equipar tu editor de código para que "entienda" Python, permitiéndote escribir código más rápido, sin errores tipográficos y ejecutándolo con un solo clic.

1. La Extensión de Python: Tu Asistente Inteligente

Es vital entender una distinción que suele confundir a quienes empiezan: **Instalar Python no es lo mismo que instalar la extensión de Python en VS Code.**

- **Python (El Motor):** Es el lenguaje y el intérprete que instalamos en la unidad anterior. Es lo que permite que tu computadora entienda las instrucciones de código.
- **VS Code (El Garaje):** Es el editor de texto donde escribes el código.
- **Extensión de Python (El Mecánico):** Es un "paquete de mejoras" creado por Microsoft que se instala **dentro** de VS Code. Sin ella, el editor trataría a tus archivos `.py` como simples archivos de texto, como si estuvieras usando el Bloc de Notas.

¿Por qué es imprescindible?

Al instalar la extensión oficial, VS Code adquiere "superpoderes":

1. **Resaltado de sintaxis:** Pinta el código de colores para que distingas fácilmente palabras reservadas, textos y números.
2. **IntelliSense:** Es el autocompletado inteligente. Empiezas a escribir una palabra y el editor te sugiere cómo terminarla.
3. **Detección de errores (Linting):** Te avisa con un subrayado rojo si te falta un paréntesis o si escribiste algo mal antes de que intentes ejecutarlo.
4. **Botón de ejecución:** Añade un pequeño icono de "Play" para que no tengas que escribir comandos complejos para ver tu programa funcionar.

2. Instalación de la Extensión Oficial

Sigue estos pasos para preparar tu entorno:

1. Abre **Visual Studio Code**.
2. En la barra lateral izquierda, busca el icono que parece cuatro cuadrados (el último o penúltimo de la lista). Al poner el ratón encima dirá **"Extensions"**.
3. En la barra de búsqueda que aparece arriba, escribe: `Python`.
4. Busca la opción que dice **Python** y que tiene una marca de verificación azul (es la publicada por **Microsoft**). Es la extensión más popular del mundo en VS Code. (Link)
5. Haz clic en el botón de **Install**.

> **Nota**
>
> **Dato importante:** Al instalar esta extensión, VS Code instalará automáticamente otras herramientas útiles como *Pylance* (para el autocompletado avanzado). No te preocupes, es normal y necesario.

3. El Intérprete: Eligiendo el "Cerebro" de tu Proyecto

Una vez instalada la extensión, debemos decirle a VS Code qué versión de Python debe usar. Esto se conoce como **Seleccionar el Intérprete**.

A veces, una computadora puede tener varias versiones de Python instaladas (por ejemplo, una que vino con el sistema y la que tú instalaste). Si VS Code no sabe cuál usar, tu código podría dar errores extraños porque el editor está "mirando" al lugar equivocado.

Cómo seleccionar el intérprete en VS Code:

1. Presiona la combinación de teclas `Ctrl + Shift + P` (en Windows/Linux) o `Cmd + Shift + P` (en Mac). Esto abre la **Paleta de Comandos**, que es como el buscador global de funciones de VS Code.
2. Escribe: `Python: Select Interpreter`.
3. Verás una lista de las versiones de Python encontradas en tu sistema.
4. Haz clic en la que instalamos anteriormente (debería decir algo como `Python 3.12.x` o similar).

Si ves una barra azul en la parte inferior derecha de VS Code que indica la versión de Python instalada, ¡felicidades! El editor y el lenguaje ya están conectados.

4. Funciones de Ayuda: Tu Primer Contacto

Ahora que todo está configurado, notarás cambios inmediatos al crear un archivo que termine en `.py`.

Para la creación de archivos no hay una unica manera:

1. Accediendo por el menu.
2. En el explorador de archivos (icono de la barra lateral) con el icono de creación de archivos.
3. También en el explorador de archivos haciendo clic derecho sobre la zona del rectángulo se abrirá el menú desplegable y seleccionaremos `New File`.

Autocompletado (IntelliSense)

Si empiezas a escribir `pri`, verás que aparece una cajita sugiriendo `print`. Si presionas `Enter` o `Tab`, VS Code completará la palabra por ti. Esto no solo ahorra tiempo, sino que evita que cometas errores de ortografía en el código (como escribir `prnit` por accidente).

Ejecución rápida

Gracias a la extensión, en la esquina superior derecha verás un icono de un triángulo (el botón de **Play**). Al presionarlo:

1. Se abrirá automáticamente una terminal en la parte inferior.
2. VS Code escribirá el comando necesario por ti.
3. Verás el resultado de tu código al instante.

5. Errores Comunes de Principiantes

Anticipemos algunos problemas para que no te detengan:

- **"No se reconoce el comando python":** Esto suele pasar si olvidaste marcar la casilla "Add Python to PATH" durante la instalación de Python. Si te sucede, VS Code te avisará que no encuentra un intérprete. La solución es volver a ejecutar el instalador de Python y asegurarte de marcar esa opción.
- **Instalar la extensión equivocada:** Hay muchas extensiones que se llaman "Python". Asegúrate de que la que instales sea la de **Microsoft**. Es la que garantiza compatibilidad total con las herramientas que usaremos para crear nuestro blog.
- **El archivo no tiene colores:** Si tu código se ve todo en blanco o gris, asegúrate de que guardaste el archivo con la extensión `.py` (por ejemplo: `prueba.py`). Sin esa extensión, VS Code no sabe que estás escribiendo en Python.

Resumen de la Configuración

- La **extensión** es la herramienta que ayuda al programador dentro del editor.
- El **intérprete** es el motor de Python que ejecuta el código.
- Usamos la **Paleta de Comandos** (`Ctrl+Shift+P`) para conectar ambos.
- El objetivo final es que VS Code nos ayude a no cometer errores básicos y nos permita ejecutar nuestro trabajo fácilmente.

En la siguiente unidad, aprenderemos a dominar la **Terminal**, ese panel que aparece en la parte inferior, para movernos por nuestras carpetas de proyecto como profesionales. Pero antes, asegúrate de que tu mecánico (la extensión) esté listo para trabajar.

Terminal integrada y comandos básicos para moverte en proyectos

Aprende a dominar la terminal de VS Code con comandos básicos de navegación y creación de archivos para organizar tus proyectos de programación de forma eficiente.

La Terminal: Tu panel de control como desarrollador

Imagina que estás construyendo una casa. Puedes usar una interfaz visual (como Windows o macOS) donde arrastras ventanas, lo cual es genial como usuario. Pero como **arquitecto y constructor (programador)**, necesitas comunicarte directamente con los cimientos del sistema. Eso es la **Terminal**.

La terminal no es más que una ventana de texto donde escribes órdenes que el sistema operativo ejecuta. En este curso, usaremos la **Terminal Integrada de Visual Studio Code**, lo que nos permite programar y usar comandos sin cambiar de aplicación.

Conceptos clave: Terminal vs. Shell

- **Terminal:** Es la "ventana" o la interfaz gráfica que acepta texto.

- **Shell:** Es el "intérprete" que vive dentro de la terminal. Es el cerebro que lee tu comando (como `mkdir`) y le dice al ordenador: "Oye, crea una carpeta ahora mismo".

Comandos de navegación de "supervivencia"

Para no perderte en tu propio ordenador, necesitas dominar tres comandos básicos. Piensa en ellos como tu GPS:

1. `**pwd**` **(Print Working Directory):** "¿Dónde estoy?". Te muestra la ruta completa de la carpeta en la que estás parado actualmente.
2. `**ls**` **(List):** "¿Qué hay aquí?". Muestra una lista de todos los archivos y carpetas dentro de tu ubicación actual.
3. `**cd**` **(Change Directory):** "Ir a...". Te permite moverte entre carpetas.

- `cd nombre_carpeta` : Entra en una carpeta.
- `cd ..` : (con dos puntos) Te saca de la carpeta actual y te sube un nivel hacia atrás.

Creación de estructura de trabajo

Como vamos a construir un blog, necesitamos aprender a crear carpetas y archivos sin usar el ratón:

- `**mkdir**` **(Make Directory):** Crea una carpeta nueva. Ejemplo: `mkdir mi_proyecto_blog` .
- `**touch**` **:** Crea un archivo vacío. Ejemplo: `touch post.py` . (Nota: En algunas versiones de Windows, si `touch` no funciona, puedes usar `echo > post.py`).

Errores comunes de principiantes

- **Escribir mal los comandos:** La terminal es muy estricta. `LS` (en mayúsculas) no es lo mismo que `ls` .
- **Olvidar los espacios:** Los comandos y sus destinos van separados por un espacio. `cd..` puede fallar, lo correcto es `cd ..` .
- **No saber dónde estás:** Siempre ejecuta `pwd` antes de crear algo para asegurarte de que lo estás poniendo en el lugar correcto.

Ejercicio Práctico: "Mapa inicial de trabajo en terminal"

Objetivo: Crear una estructura organizada para tu futuro blog usando solo la terminal.

Instrucciones paso a paso:

1. **Abre VS Code** y abre una carpeta vacía en tu computadora (File > Open Folder).
2. **Abre la terminal integrada:** Ve al menú superior `Terminal > New Terminal` o presiona las teclas `Ctrl + ` ` (la tecla al lado del 1). Tener en cuenta que las imágenes pueden no ser exactamente iguales porque depende de la terminal que tengan configurada por defecto (las imágenes muestran una terminal bash, pero podría ser PowerShell, zsh, fish, cmd u otra).
3. **Ubícate:** Escribe `pwd` y pulsa Enter para ver tu ruta actual.

4. **Crea la base:** Escribe `mkdir modulo1_python` y pulsa Enter.
5. **Entra en ella:** Escribe `cd modulo1_python` .
6. **Organiza:** Dentro, crea otra carpeta llamada `practicas` con `mkdir practicas` .
7. **Crea un archivo:** Entra en la carpeta de prácticas (`cd practicas`) y crea un archivo llamado `leeme.txt` usando el comando `touch leeme.txt` .
8. **Verifica:** Escribe `ls` para ver que tu archivo está ahí.

Entornos virtuales con venv para aislar el proyecto

Aprende a crear y gestionar entornos virtuales con venv para mantener tus proyectos de Python organizados, aislados y libres de conflictos de librerías.

Imagina que estás construyendo dos casas al mismo tiempo. En la primera casa, necesitas usar herramientas de última tecnología, muy modernas y delicadas. En la segunda, estás restaurando una estructura antigua que requiere herramientas manuales pesadas y aceitosas. Si guardaras todas las herramientas en una única caja gigante y desordenada, las herramientas modernas terminarían llenas de grasa, o podrías usar accidentalmente un martillo neumático donde necesitabas un pincel fino, arruinando el trabajo.

En la programación ocurre algo idéntico. A medida que aprendas, crearás diferentes proyectos. Uno podría ser este blog con **Django**, y otro podría ser un script para analizar datos. Cada uno necesita sus propias "herramientas" (librerías). Si instalas todo en un mismo lugar (tu instalación global de Python), tarde o temprano las versiones chocarán y tus proyectos dejarán de funcionar.

Aquí es donde entran los **entornos virtuales**. Son, literalmente, cajas de herramientas separadas y privadas para cada uno de tus proyectos.

Primer script: ficha de post por consola

Escribe y ejecuta tu primer programa real en Python creando una ficha técnica de un post para tu blog, aprendiendo los fundamentos de salida por consola y manejo de variables.

¿Qué es un Script de Python?

Imagina que quieres cocinar un pastel. Tienes una lista de pasos: "1. Mezcla harina", "2. Añade huevos", "3. Hornea". Un **script** es exactamente eso: un archivo de texto que contiene una lista de instrucciones escritas en Python que la computadora ejecutará en orden, de arriba hacia abajo.

A diferencia de escribir comandos uno a uno en la terminal, un script nos permite guardar nuestro trabajo, corregirlo y compartirlo. Todos los archivos de script de Python deben terminar con la extensión `.py` (por ejemplo: `post.py`).

La herramienta estrella: `print()`

Para que un programa sea útil, debe poder "hablar" con nosotros. La función `print()` es el "altavoz" de Python. Todo lo que pongas dentro de los paréntesis de un `print()` se mostrará en la pantalla (la consola).

CÓDIGO

```
print("Hola, este es mi blog")
```

Usando variables para organizar datos

En lugar de escribir todo el texto directamente en el `print()`, usamos **variables**. Una variable es como una caja con una etiqueta donde guardamos información para usarla después. Estas cajas pueden pasarse a un print como ele texto que le pasamos en el ejemplo anterior pero sin usar las comillas.

En nuestro blog, un post no es solo un montón de texto; tiene una estructura. Necesitamos guardar:

1. El **título**.
2. El **autor**.
3. El **resumen** o contenido.

CÓDIGO

```
titulo = "Cómo instalar VS Code"
autor = "Tu Nombre"
```

Ejecución desde la terminal

Una vez que escribes tu código en un archivo `.py`, necesitas decirle a Python que lo lea y lo ejecute. Para esto usamos la terminal integrada de VS Code que aprendimos a usar anteriormente. El comando mágico es:

`python nombre_del_archivo.py`

(Nota: En algunas instalaciones de macOS o Linux, el comando puede ser `python3`).

Errores comunes al empezar

1. **Olvidar las comillas**: Las palabras (cadenas de texto) siempre deben ir entre comillas: `"Texto"`. Si escribes `print(Hola)`, Python buscará una variable llamada `Hola` y, si no existe, te dará un error.
2. **No guardar el archivo**: Si haces un cambio en tu código en VS Code pero no guardas el archivo (`Ctrl + S`), cuando lo ejecutes en la terminal verás la versión anterior.
3. **No estar en la carpeta correcta**: Si tu archivo está en una carpeta llamada `proyecto` pero tu terminal está en `Documentos`, Python dirá que no encuentra el archivo. Usa el comando `ls` o `dir` para ver si tu archivo está ahí.

Flashcards de repaso

Tarjetas de repaso para consolidar los conocimientos esenciales sobre la configuración del entorno de desarrollo en Python, el uso de entornos virtuales y la ejecución de scripts.

Refuerza tus conocimientos: Entorno y Primeros pasos

¡Felicidades por llegar hasta aquí! Configurar el entorno de desarrollo es, a menudo, el paso más desafiante para quien comienza, pero también el más determinante para tu éxito a largo plazo. Ya has instalado Python, configurado tu editor, dominado los comandos básicos de terminal y aprendido a aislar tus proyectos.

Estas tarjetas de repaso están diseñadas para consolidar los conceptos técnicos que sustentan tu trabajo diario como desarrollador Python.

Cómo usar estas tarjetas de forma efectiva:

1. **Recuerdo Activo**: Antes de voltear la tarjeta, intenta responder la pregunta en voz alta o por escrito. No te limites a leerla.
2. **Si fallas, revisita**: Si una tarjeta te cuesta, marca el concepto y vuelve a la unidad teórica correspondiente.
3. **Sé honesto**: El objetivo no es "pasar" las tarjetas, sino identificar qué conceptos necesitas repasar más.

Recuerda: la maestría viene de la repetición consciente. ¡Vamos a ello!

01 ¿Qué es el intérprete de Python?

El intérprete de Python es el programa encargado de leer y ejecutar el código fuente (`.py`) escrito por el programador. Es el "motor" que traduce tus instrucciones a acciones que la computadora puede entender.

02 ¿Para qué sirve un entorno virtual (`venv`)?

Es una herramienta que permite crear contenedores independientes para cada proyecto. Esto garantiza que las librerías o versiones de Python de un proyecto no entren en conflicto con las de otros.

03 ¿Cuál es el comando básico para crear un entorno virtual en Python?

python -m venv nombre_del_entorno (en Windows, macOS o Linux). Esta instrucción crea una carpeta aislada con su propia copia del ejecutable de Python.

04 ¿Cuál es la función principal de la extensión de Python para VS Code?

Es una extensión oficial que añade soporte para Python en VS Code, proporcionando características como autocompletado, resaltado de sintaxis, depuración y selección automática del intérprete correcto.

05 ¿Por qué es importante seleccionar el intérprete correcto en VS Code?

Al seleccionar un intérprete en VS Code, le indicas al editor qué versión específica de Python y qué entorno (virtual o global) debe usar para ejecutar tu código y ofrecerte asistencia durante la programación.

06 ¿Qué hacen los comandos `cd` y `ls/dir` en la terminal?

`cd` (change directory) cambia el directorio actual de la terminal, mientras que `ls` (macOS/Linux) o `dir` (Windows) sirven para listar los archivos dentro del directorio en el que te encuentras.

07 ¿Cómo ejecutas un archivo o script de Python desde la terminal?

`python nombre_del_archivo.py`. Al ejecutar esto, el intérprete procesa el contenido del archivo línea por línea desde el inicio hasta el final.

08 ¿Cómo puedes imprimir un mensaje por consola en un script de Python?

La función `print()` permite mostrar información en la consola. Es la forma más básica y fundamental para ver los resultados de la ejecución de tu código. Ejemplo: `print("Hola Mundo")`.

PRE-ENTREGA DEL PROYECTO

Actividad evaluable · parte del proyecto integrador

Preentrega 1: entorno listo y primer script ejecutado

Fase de consolidación del entorno de desarrollo. El estudiante verifica que Python, VS Code y el entorno virtual funcionan correctamente mediante la ejecución de un script de prueba.

¡Felicitaciones! Has completado el primer módulo de tu camino como desarrollador/a Python. En programación, los cimientos son lo más importante. No importa qué tan complejo sea el sistema que construyas después, siempre dependerá de que tu entorno de trabajo sea sólido, aislado y funcional.

Esta preentrega representa el primer gran hito de tu proyecto: **pasar de tener una computadora común a tener una estación de trabajo para desarrollo de software.**

¿Por qué es fundamental este preentrega?

En las unidades anteriores aprendiste a instalar Python, configurar Visual Studio Code, moverte por carpetas usando la terminal y, lo más importante, aprendiste a crear **entornos virtuales**. Sin un entorno virtual, tus proyectos de Python se mezclarían entre sí, causando errores de versiones que son la pesadilla de cualquier principiante.

Este checkpoint valida que:

1. Sabes comunicarte con tu computadora mediante comandos (Terminal).
2. Entiendes que cada proyecto es una "isla" independiente (Entorno Virtual).
3. Eres capaz de escribir código que la computadora entiende y ejecuta (Script Python).

Posición en el programa

Este es el **punto de partida real**. Lo que has construido aquí (tu carpeta de proyecto y tu entorno virtual) será el hogar de nuestro blog. En el próximo módulo, vamos a empezar a organizar mejor los datos de ese blog usando variables y colecciones de Python.

¿Qué vamos a consolidar hoy?

Tu objetivo es asegurar que tienes una carpeta de proyecto organizada con la siguiente estructura y funcionamiento:

1. El corazón: El Entorno Virtual (`.venv`)

Debes tener una carpeta llamada `.venv` (o similar) dentro de tu proyecto. Este entorno debe estar **activado** cuando trabajes.

- **Señal de éxito:** En la terminal de VS Code, deberías ver el nombre de tu entorno entre paréntesis, como `(.venv)`.

2. La herramienta: Visual Studio Code

Tu editor debe estar usando el intérprete de Python que reside dentro de ese entorno virtual, no el Python global de tu sistema.

- **Señal de éxito:** En la esquina inferior derecha de VS Code, al abrir un archivo `.py`, debería aparecer el texto "Python 3.x.x ('.venv': venv)".

3. El producto: `post_test.py`

Un script sencillo que imprima en pantalla los datos de un post (título, autor, contenido sencillo).

- **Señal de éxito:** Al ejecutar `python post_test.py` en la terminal, ves los datos impresos correctamente.

Errores comunes a evitar

- **No activar el entorno:** Es el error número uno. Si instalas cosas sin el `(.venv)` activo, se instalarán en tu sistema global y luego el proyecto no funcionará en otras computadoras.
- **Estar en la carpeta equivocada:** Si tu terminal dice que estás en `C:\Users\Nombre` pero tu código está en `C:\Proyectos\Blog`, el comando `python post_test.py` dará error. Usa `pwd` para saber dónde estás y `ls` (o `dir`) para ver si el archivo está ahí.
- **Confundir el nombre del archivo:** Python es sensible a las mayúsculas y minúsculas. Si tu archivo se llama `Post.py`, ejecutar `python post.py` (en minúsculas) podría fallar en algunos sistemas.

Conexión con el futuro: El Blog Web

Este pequeño script `post_test.py` es el ancestro de lo que pronto será un **Modelo de Datos** en Django. Hoy imprimes el título en una terminal negra; en unos módulos, ese mismo título aparecerá con un diseño elegante en un navegador web para que todo el mundo lo lea.

¡Asegúrate de que todo funcione perfectamente antes de seguir adelante!

Pasos para la entrega

Para completar esta preentrega, sigue estos pasos y genera el archivo de evidencia:

Paso 1: Verificación del Entorno

1. Abre tu carpeta de proyecto en **Visual Studio Code**.
2. Abre la **Terminal Integrada** (Ctrl + `).
3. Asegúrate de que tu entorno virtual esté activo. Deberías ver `(.venv)` al inicio de la línea de comandos.

Paso 2: El Script de Prueba

Crea un archivo llamado `evidencia_post.py` (si ya tienes uno de la unidad anterior, puedes usarlo) con el siguiente contenido:

CÓDIGO

```
# Datos de prueba para mi futuro blog
titulo = "Mi primer post en Python"
autor = "Tu Nombre"
estado = "Entorno configurado correctamente"

print("--- REPORTE DE SISTEMA ---")
print(f"Post: {titulo}")
print(f"Autor: {autor}")
print(f"Estado: {estado}")
print("--------------------------")
```

Paso 3: Ejecución y Captura

1. Ejecuta el script escribiendo en la terminal: `python evidencia_post.py`
2. Selecciona y copia todo el texto que aparece en la terminal (desde que escribiste el comando hasta el resultado final).

Paso 4: Creación del Entregable

1. Crea un nuevo archivo en el Bloc de Notas (o cualquier editor de texto simple) y guárdalo como `checkpoint_modulo1.txt`.
2. Pega dentro del archivo el texto que copiaste de la terminal.
3. Debajo del resultado de la terminal, escribe una breve lista de los archivos que tienes en tu carpeta de proyecto actualmente (ejemplo: `.venv/`, `evidencia_post.py`, etc.).

Criterios de Aceptación:

- El archivo debe ser un `.txt`.
- El contenido debe mostrar claramente que el script se ejecutó sin errores.
- Debes mencionar que el entorno virtual estaba activo al momento de la prueba.

Nota: No envíes el código `.py`, envía el `.txt` con el resultado de la ejecución.

Entregable

Formato: text

Completar con la salida de la terminal al ejecutar el script del post y una breve descripción de la estructura de carpetas creada.

Objetivos de aprendizaje

- Validar la instalación correcta de Python y Visual Studio Code.
- Confirmar la creación y activación de un entorno virtual (venv).
- Ejecutar scripts de Python desde la terminal integrada de VS Code.
- Organizar la estructura básica de archivos para un proyecto de desarrollo.

Rúbrica de evaluación

Rúbrica para evaluar un checkpoint introductorio de pre-entrega en el que el estudiante demuestra que su entorno de trabajo está configurado, que puede ejecutar un script básico de Python y que documenta correctamente la evidencia en un archivo .txt con la salida de terminal y una breve descripción de la estructura del proyecto.

Criterio	Descripción	Peso
Archivo de evidencia y formato de entrega	Evalúa si el estudiante entrega el archivo solicitado en el formato correcto y con el nombre esperado, respetando la indicación de no enviar el archivo .py como entregable principal.	25%
Evidencia de ejecución del script de Python	Evalúa si la salida copiada de la terminal muestra con claridad que el script evidencia_post.py se ejecutó y produjo el resultado esperado sin errores visibles.	35%
Verificación del entorno virtual activo	Evalúa si el estudiante demuestra, en la evidencia entregada, que el entorno virtual estaba activado durante la prueba, tal como exige el checkpoint.	20%
Descripción de la estructura de la carpeta del proyecto	Evalúa si, debajo de la salida de la terminal, el estudiante añade una breve lista de los archivos o carpetas actuales del proyecto, mostrando organización básica del espacio de trabajo.	20%

Total: 100 pts | Aprobación: 70 pts

MÓDULO 2

Datos y colecciones para representar información

Aplicar variables, tipos de datos y colecciones para modelar la estructura de datos de un sistema de blog sencillo.

Variables y tipos simples para describir un post

Aprende a representar la información de un post de blog utilizando variables y los tipos de datos fundamentales en Python: cadenas de texto, números enteros y decimales.

Imagina que estás frente a la pantalla de tu blog favorito. Ves un título llamativo, el nombre del autor, la cantidad de visitas, el tiempo estimado de lectura y quizás una calificación promedio.

Aunque para el usuario todo eso se ve como una página simple, por detrás hay un programa organizando información. En Python, esa información se guarda usando **variables** y se clasifica usando **tipos de datos**.

En esta unidad vas a aprender cómo representar la información básica de un post de blog usando Python.

1. ¿Qué es una variable?

Una **variable** es un nombre que usamos para guardar un dato y poder reutilizarlo más adelante.

Podemos imaginarla como una caja con una etiqueta.

CÓDIGO

```
titulo_post = "Mi primer viaje a la montaña"
vistas = 150
```

En este ejemplo, la variable `titulo_post` guarda un texto, mientras que la variable `vistas` guarda un número.

El signo `=` en programación no se lee como “es igual a”, sino como **asignación**.

Es decir:

CÓDIGO

```
titulo_post = "Aprender Python desde cero"
```

se puede leer como:

> **Nota**
>
> Guardar el texto `"Aprender Python desde cero"` en la variable `titulo_post`.

2. Mostrar información con `print()`

La función `print()` sirve para mostrar información en la consola.

CÓDIGO

```
titulo_post = "Aprender Python desde cero"
vistas = 120

print(titulo_post)
print(vistas)
```

Salida esperada:

CÓDIGO

```
Aprender Python desde cero
120
```

También podemos combinar texto con variables:

CÓDIGO

```
titulo_post = "Aprender Python desde cero"
vistas = 120

print("Título del post:", titulo_post)
print("Cantidad de vistas:", vistas)
```

Salida esperada:

CÓDIGO

```
Título del post: Aprender Python desde cero
Cantidad de vistas: 120
```

Esta forma es muy útil al principio porque evita algunos errores comunes al mezclar texto y números.

3. Buenas prácticas para nombrar variables

En Python solemos escribir los nombres de variables usando `snake_case`.

Eso significa escribir todo en minúsculas y separar las palabras con guion bajo.

Ejemplos correctos:

CÓDIGO

```
titulo_post = "Mi primer post"
nombre_autor = "Ana Pérez"
cantidad_vistas = 350
tiempo_lectura = 4
```

Buenas prácticas:

- Usar nombres descriptivos.
- Escribir en minúsculas.
- Separar palabras con guion bajo.
- No usar espacios.
- No empezar con números.

Por ejemplo, esto es recomendable:

CÓDIGO

```
nombre_autor = "Ana Pérez"
```

Esto es poco claro:

CÓDIGO

```
n = "Ana Pérez"
```

Aunque Python lo permite, el nombre `n` no explica qué información guarda.

También hay que evitar nombres con espacios:

CÓDIGO

```
cantidad comentarios = 12
```

Ese ejemplo está mal, porque Python no permite espacios en nombres de variables.

La forma correcta sería:

CÓDIGO

```
cantidad_comentarios = 12
```

También hay que evitar empezar con números:

CÓDIGO

```
post1 = "Primer post"
```

Eso está bien.

Pero esto daría error:

CÓDIGO

```
1post = "Primer post"
```

Además, conviene evitar nombres que Python ya usa para sus propias funciones o tipos de datos:

CÓDIGO

```
print = "Hola"
input = "Texto"
str = "Algo"
```

Aunque a veces Python lo permita, puede generar errores o comportamientos confusos más adelante.

4. Tipos de datos simples

Python necesita saber qué tipo de dato estamos guardando.

No es lo mismo guardar un texto que guardar un número. Por ejemplo, Python no interpreta igual `"100"` que `100`.

Los tipos simples más importantes para esta unidad son:

- `str`: texto.
- `int`: número entero.
- `float`: número decimal.
- `bool`: verdadero o falso.

5. Texto: `str`

Un `str` es una cadena de texto. Se escribe entre comillas simples o dobles.

Ejemplos:

CÓDIGO

```
titulo_post = "Mi primer blog"
nombre_autor = "Ana Pérez"
etiqueta_principal = "Programación"
```

También podríamos usar comillas simples:

CÓDIGO

```
titulo_post = 'Mi primer blog'
```

Ambas formas son válidas. Lo importante es abrir y cerrar las comillas correctamente.

Ejemplo aplicado al blog:

CÓDIGO

```
titulo_post = "Introducción a Python"
nombre_autor = "Cristian"
resumen = "En este post aprenderemos qué son las variables."

print("Título:", titulo_post)
print("Autor:", nombre_autor)
print("Resumen:", resumen)
```

Salida esperada:

CÓDIGO

```
Título: Introducción a Python
Autor: Cristian
Resumen: En este post aprenderemos qué son las variables.
```

6. Números enteros: `int`

Un `int` representa números enteros, es decir, números sin decimales.

Ejemplos:

CÓDIGO

```
vistas = 250
likes = 42
comentarios = 8
anio_publicacion = 2026
```

Ejemplo aplicado:

CÓDIGO

```
vistas = 250
likes = 42
comentarios = 8

print("Vistas:", vistas)
print("Likes:", likes)
print("Comentarios:", comentarios)
```

También podemos hacer operaciones matemáticas:

CÓDIGO

```
vistas = 250
nuevas_vistas = 30

total_vistas = vistas + nuevas_vistas

print("Total de vistas:", total_vistas)
```

Salida esperada:

CÓDIGO

```
Total de vistas: 280
```

7. Números decimales: `float`

Un `float` representa números con decimales.

En Python, los decimales se escriben con punto, no con coma.

Correcto:

CÓDIGO

```
rating = 4.5
peso_imagen = 1.8
tiempo_lectura = 3.5
```

Incorrecto:

CÓDIGO

```
rating = 4,5
```

Ese ejemplo no representa el decimal `4.5` como esperamos.

Ejemplo aplicado al blog:

CÓDIGO

```
rating = 4.7
tiempo_lectura = 5.5

print("Calificación promedio:", rating)
print("Tiempo estimado de lectura:", tiempo_lectura, "minutos")
```

Salida esperada:

CÓDIGO

```
Calificación promedio: 4.7
Tiempo estimado de lectura: 5.5 minutos
```

Dato importante:

CÓDIGO

```
numero_entero = 5
numero_decimal = 5.0

print(type(numero_entero))
print(type(numero_decimal))
```

Salida esperada:

CÓDIGO

```
<class 'int'>
<class 'float'>
```

Aunque `5` y `5.0` parezcan iguales, Python los trata como tipos distintos.

8. Valores verdadero/falso: `bool`

Un `bool` representa un valor lógico: verdadero o falso.

En Python se escriben así:

CÓDIGO

```
destacado = True
requiere_revision = False
```

Este tipo de dato es muy útil para representar condiciones simples.

Por ejemplo, un post puede estar destacado o puede requerir revisión.

CÓDIGO

```
titulo_post = "Aprender Python"
destacado = True

print("Título:", titulo_post)
print("¿Está destacado?", destacado)
```

Salida esperada:

CÓDIGO

```
Título: Aprender Python
¿Está destacado? True
```

Más adelante, cuando trabajemos con condicionales, podremos hacer cosas como:

CÓDIGO

```
destacado = True

if destacado:
    print("El post se puede mostrar en una sección destacada.")
else:
    print("El post se muestra en el listado normal.")
```

No hace falta dominar esto ahora, pero está bueno empezar a reconocerlo.

9. Ver el tipo de dato con `type()`

Python nos permite consultar el tipo de una variable usando `type()`.

CÓDIGO

```
titulo_post = "Aprender Python"
vistas = 150
rating = 4.8
destacado = True

print(type(titulo_post))
print(type(vistas))
print(type(rating))
print(type(destacado))
```

Salida esperada:

CÓDIGO

```
<class 'str'>
<class 'int'>
<class 'float'>
<class 'bool'>
```

Esto es muy útil para entender qué está guardando realmente una variable.

10. Mejorando la salida con f-strings

Además de usar comas en `print()`, Python tiene una forma muy cómoda de combinar texto y variables: las **f-strings**.

Ejemplo:

CÓDIGO

```
titulo_post = "Aprender Python"
vistas = 150

print(f"El post '{titulo_post}' tiene {vistas} vistas.")
```

Salida esperada:

CÓDIGO

```
El post 'Aprender Python' tiene 150 vistas.
```

Las f-strings se escriben poniendo una `f` antes de las comillas:

CÓDIGO

```
f"Texto {variable}"
```

Ejemplo más completo:

CÓDIGO

```
titulo_post = "Variables en Python"
autor = "Ana"
vistas = 320
rating = 4.6

print(f"El post '{titulo_post}' fue escrito por {autor}.")
print(f"Tiene {vistas} vistas y una calificación de {rating}.")
```

Salida esperada:

CÓDIGO

```
El post 'Variables en Python' fue escrito por Ana.
Tiene 320 vistas y una calificación de 4.6.
```

11. Entrada de datos con `input()`

Hasta ahora, los valores los escribimos directamente en el código. Pero muchas veces queremos que el usuario ingrese información.

Para eso usamos `input()`.

CÓDIGO

```
titulo_post = input("Escribe el título del post: ")

print("El título ingresado es:", titulo_post)
```

Cuando el programa llega a `input()`, se detiene y espera que el usuario escriba algo.

Ejemplo:

CÓDIGO

```
nombre_autor = input("Escribe tu nombre: ")
titulo_post = input("Escribe el título del post: ")

print(f"El post '{titulo_post}' fue creado por {nombre_autor}.")
```

Ejemplo de ejecución:

CÓDIGO

```
Escribe tu nombre: Cristian
Escribe el título del post: Aprender Python
El post 'Aprender Python' fue creado por Cristian.
```

12. La trampa de `input()`

Hay algo muy importante:

`input()` siempre devuelve texto.

Aunque el usuario escriba un número, Python lo recibe como `str`.

Ejemplo:

CÓDIGO

```
comentarios = input("¿Cuántos comentarios tiene el post? ")

print(type(comentarios))
```

Si el usuario escribe:

CÓDIGO

```
10
```

Python lo interpreta como:

CÓDIGO

```
"10"
```

No como:

CÓDIGO

```
10
```

Esto puede generar errores.

Por ejemplo:

CÓDIGO

```
comentarios = input("¿Cuántos comentarios tiene el post? ")
total = comentarios + 1

print(total)
```

Ese código falla porque Python no puede sumar texto con números.

13. Conversión de tipos: casting

Para convertir un dato de un tipo a otro usamos funciones como:

CÓDIGO

```
int()
float()
str()
```

Ejemplo con `int()`:

CÓDIGO

```
comentarios_texto = input("¿Cuántos comentarios tiene el post? ")
comentarios_numero = int(comentarios_texto)

total_comentarios = comentarios_numero + 1

print("Si llega un comentario más, habrá:", total_comentarios)
```

También se puede escribir en una sola línea:

CÓDIGO

```
comentarios = int(input("¿Cuántos comentarios tiene el post? "))

total_comentarios = comentarios + 1

print("Total:", total_comentarios)
```

Ejemplo con `float()`:

CÓDIGO

```
rating = float(input("Ingresa la calificación del post: "))

print(f"La calificación ingresada fue {rating}.")
```

Si el usuario escribe:

CÓDIGO

```
4.8
```

Python lo convierte en un número decimal.

También podemos convertir un número a texto con `str()`:

CÓDIGO

```
vistas = 150

mensaje = "Cantidad de vistas: " + str(vistas)

print(mensaje)
```

Aunque en la mayoría de los casos será más cómodo usar f-strings:

CÓDIGO

```
vistas = 150

print(f"Cantidad de vistas: {vistas}")
```

14. Ejemplo completo: crear un resumen de post

Veamos un pequeño programa que reúne todo lo aprendido.

CÓDIGO

```
titulo_post = input("Título del post: ")
autor = input("Autor del post: ")
etiqueta_principal = input("Etiqueta principal: ")

vistas = int(input("Cantidad de vistas: "))
rating = float(input("Calificación promedio: "))
destacado = True

print("----- RESUMEN DEL POST -----")
print(f"Título: {titulo_post}")
print(f"Autor: {autor}")
print(f"Etiqueta principal: {etiqueta_principal}")
print(f"Vistas: {vistas}")
print(f"Rating: {rating}")
print(f"Destacado: {destacado}")
```

Ejemplo de ejecución:

CÓDIGO

```
Título del post: Aprender Python
Autor del post: Cristian
Etiqueta principal: Programación
Cantidad de vistas: 250
Calificación promedio: 4.7
```

Salida esperada:

CÓDIGO

```
----- RESUMEN DEL POST -----
Título: Aprender Python
Autor: Cristian
Etiqueta principal: Programación
Vistas: 250
Rating: 4.7
Destacado: True
```

15. Errores comunes

Error 1: olvidar cerrar comillas

Incorrecto:

CÓDIGO

```
titulo = "Mi primer post
```

Correcto:

CÓDIGO

```
titulo = "Mi primer post"
```

Error 2: mezclar texto y números con +

Incorrecto:

CÓDIGO

```
vistas = 100

print("Vistas: " + vistas)
```

Esto da error porque `"Vistas: "` es texto y `vistas` es un número.

Correcto:

CÓDIGO

```
vistas = 100

print("Vistas:", vistas)
```

También correcto:

CÓDIGO

```
vistas = 100

print(f"Vistas: {vistas}")
```

Error 3: olvidar convertir un `input()`

Incorrecto:

CÓDIGO

```
vistas = input("Cantidad de vistas: ")
vistas_totales = vistas + 10

print(vistas_totales)
```

Correcto:

CÓDIGO

```
vistas = int(input("Cantidad de vistas: "))
vistas_totales = vistas + 10

print(vistas_totales)
```

Error 4: usar coma en números decimales

Incorrecto:

CÓDIGO

```
rating = 4,5
```

Correcto:

CÓDIGO

```
rating = 4.5
```

Error 5: creer que `int()` redondea

`int()` no redondea. Simplemente corta la parte decimal.

CÓDIGO

```
nota = 4.9

print(int(nota))
```

Salida esperada:

CÓDIGO

```
4
```

Si queremos redondear, usamos `round()`:

CÓDIGO

```
nota = 4.9

print(round(nota))
```

Salida esperada:

CÓDIGO

```
5
```

16. Mini práctica guiada

Crea un programa llamado `post_basico.py`.

El programa debe pedir al usuario:

- título del post,

- nombre del autor,
- etiqueta principal,
- cantidad de vistas,
- calificación promedio,
- tiempo estimado de lectura.

Luego debe mostrar un resumen usando `print()` y f-strings.

Ejemplo posible:

CÓDIGO

```
titulo = input("Título del post: ")
autor = input("Autor: ")
etiqueta_principal = input("Etiqueta principal: ")
vistas = int(input("Cantidad de vistas: "))
rating = float(input("Calificación promedio: "))
tiempo_lectura = float(input("Tiempo estimado de lectura en minutos: "))

print("----- POST CREADO -----")
print(f"Título: {titulo}")
print(f"Autor: {autor}")
print(f"Etiqueta principal: {etiqueta_principal}")
print(f"Vistas: {vistas}")
print(f"Rating: {rating}")
print(f"Tiempo de lectura: {tiempo_lectura} minutos")
```

17. Desafío extra

Agrega una variable llamada `likes` y otra llamada `comentarios`.

Luego calcula cuántas interacciones tiene el post sumando likes y comentarios.

CÓDIGO

```
likes = int(input("Cantidad de likes: "))
comentarios = int(input("Cantidad de comentarios: "))

interacciones = likes + comentarios

print(f"El post tiene {interacciones} interacciones.")
```

También podrías integrarlo al programa anterior:

CÓDIGO

```
titulo = input("Título del post: ")
autor = input("Autor: ")
etiqueta_principal = input("Etiqueta principal: ")

vistas = int(input("Cantidad de vistas: "))
likes = int(input("Cantidad de likes: "))
comentarios = int(input("Cantidad de comentarios: "))
rating = float(input("Calificación promedio: "))

interacciones = likes + comentarios

print("----- RESUMEN DEL POST -----")
print(f"Título: {titulo}")
print(f"Etiqueta principal: {etiqueta_principal}")
print(f"Vistas: {vistas}")
print(f"Likes: {likes}")
print(f"Comentarios: {comentarios}")
print(f"Interacciones totales: {interacciones}")
print(f"Rating: {rating}")
```

18. Síntesis de la unidad

En esta unidad aprendiste que Python puede guardar información usando variables.

También vimos que cada dato tiene un tipo:

- `str`: para textos.
- `int`: para números enteros.
- `float`: para números decimales.
- `bool`: para valores verdadero/falso.

Usamos `print()` para mostrar información y `input()` para pedir datos al usuario.

Además, aprendimos que los datos ingresados con `input()` siempre llegan como texto, por lo que debemos convertirlos usando `int()` o `float()` cuando queremos hacer cálculos.

Estos conceptos son la base para construir nuestro blog. Más adelante, cuando trabajemos con Django, cada dato de un post también tendrá un tipo: el título será texto, la cantidad de vistas será un número, una calificación podrá ser decimal y el estado editorial podrá representarse con opciones como `borrador`, `publicado` o `archivado`.

Antes de crear una web, necesitamos entender cómo representar sus datos. Y eso empieza acá: con variables y tipos simples.

Listas para organizar publicaciones del blog

Aprende a gestionar colecciones de datos usando listas de Python para organizar las publicaciones de un blog.

Listas para organizar varios posts del blog

En la unidad anterior, representamos un post individual usando variables simples. Por ejemplo:

CÓDIGO

```
titulo_post = "Mi primer post"
autor = "Ana"
vistas = 120
```

Esto funciona bien cuando tenemos un solo post. Pero un blog real no tiene una sola publicación. Puede tener decenas, cientos o incluso miles.

Entonces aparece una pregunta importante:

¿Cómo podemos manejar muchos posts sin crear variables como `post1`, `post2`, `post3`, `post4` y así sucesivamente?

Para resolver este problema, Python nos ofrece una estructura muy útil: las **listas**.

1. ¿Qué es una lista?

Una **lista** es una colección de elementos ordenados.

Podemos imaginarla como una estantería: cada estante guarda un dato y cada dato tiene una posición.

En Python, las listas se escriben usando corchetes `[]` y separando los elementos con comas.

CÓDIGO

```
mis_posts = ["Bienvenida", "Mi primer viaje", "Receta de pasta"]
```

En este ejemplo, la variable `mis_posts` guarda una lista con tres títulos de posts.

Cada elemento de la lista es un texto, es decir, un `str`.

Podemos mostrar la lista completa usando `print()`:

CÓDIGO

```
mis_posts = ["Bienvenida", "Mi primer viaje", "Receta de pasta"]

print(mis_posts)
```

Salida esperada:

CÓDIGO

```
["Bienvenida", "Mi primer viaje", "Receta de pasta"]
```

2. Una lista puede guardar varios tipos de datos

Una lista puede guardar textos, números, valores booleanos o incluso una mezcla de datos.

Por ejemplo:

CÓDIGO

```
post = ["Bienvenida al blog", 120, 4.5, True]
```

En este caso tenemos:

- `"Bienvenida al blog"` : título del post.
- `120` : cantidad de vistas.
- `4.5` : calificación promedio.
- `True` : indica si está publicado.

Sin embargo, al principio conviene usar listas simples y claras.

Por ejemplo, una lista de títulos:

CÓDIGO

```
titulos_posts = ["Bienvenida", "Aprender Python", "Mi experiencia programando"]
```

O una lista de cantidades de vistas:

CÓDIGO

```
vistas_posts = [120, 80, 300]
```

Más adelante vamos a aprender formas mejores de representar información más compleja, como diccionarios y objetos.

3. Acceder a elementos de una lista

Cada elemento de una lista tiene una posición.

A esa posición la llamamos **índice**.

Python empieza a contar desde `0` , no desde `1` .

CÓDIGO

```
mis_posts = ["Bienvenida", "Mi primer viaje", "Receta de pasta"]
```

Las posiciones serían:

CÓDIGO

```
Índice 0 -> "Bienvenida"
Índice 1 -> "Mi primer viaje"
Índice 2 -> "Receta de pasta"
```

Entonces, para acceder al primer elemento usamos el índice `0`:

CÓDIGO

```
mis_posts = ["Bienvenida", "Mi primer viaje", "Receta de pasta"]

print(mis_posts[0])
```

Salida esperada:

CÓDIGO

```
Bienvenida
```

Para acceder al segundo elemento:

CÓDIGO

```
print(mis_posts[1])
```

Salida esperada:

CÓDIGO

```
Mi primer viaje
```

Para acceder al tercer elemento:

CÓDIGO

```
print(mis_posts[2])
```

Salida esperada:

CÓDIGO

```
Receta de pasta
```

4. Obtener el último elemento con `-1`

Python también permite acceder a los elementos desde el final de la lista usando índices negativos.

El índice `-1` representa el último elemento.

CÓDIGO

```
mis_posts = ["Bienvenida", "Mi primer viaje", "Receta de pasta"]

print(mis_posts[-1])
```

Salida esperada:

CÓDIGO

```
Receta de pasta
```

Esto es muy útil porque muchas veces no sabemos cuántos elementos tiene una lista.

También podemos usar otros índices negativos:

CÓDIGO

```
mis_posts = ["Bienvenida", "Mi primer viaje", "Receta de pasta"]

print(mis_posts[-2])
```

Salida esperada:

CÓDIGO

```
Mi primer viaje
```

5. Modificar un elemento de la lista

Las listas se pueden modificar.

Por ejemplo, si queremos cambiar el título del primer post:

CÓDIGO

```
mis_posts = ["Bienvenida", "Mi primer viaje", "Receta de pasta"]

mis_posts[0] = "Bienvenida al blog"

print(mis_posts)
```

Salida esperada:

CÓDIGO

```
["Bienvenida al blog", "Mi primer viaje", "Receta de pasta"]
```

En este caso, reemplazamos el elemento que estaba en la posición `0`.

6. Agregar elementos con `append()`

Para agregar un nuevo elemento al final de una lista usamos `append()`.

Podemos imaginarlo como publicar un nuevo post en el blog.

CÓDIGO

```
mis_posts = ["Bienvenida", "Mi primer viaje", "Receta de pasta"]

mis_posts.append("Consejos para aprender Python")

print(mis_posts)
```

Salida esperada:

CÓDIGO

```
["Bienvenida", "Mi primer viaje", "Receta de pasta", "Consejos para aprender
Python"]
```

`append()` agrega un solo elemento al final de la lista.

Ejemplo aplicado:

CÓDIGO

```
posts_publicados = []

posts_publicados.append("Bienvenida al blog")
posts_publicados.append("Variables en Python")
posts_publicados.append("Listas en Python")

print(posts_publicados)
```

Salida esperada:

CÓDIGO

```
["Bienvenida al blog", "Variables en Python", "Listas en Python"]
```

7. Contar elementos con `len()`

Para saber cuántos elementos tiene una lista usamos `len()`.

CÓDIGO

```
mis_posts = ["Bienvenida", "Mi primer viaje", "Receta de pasta"]

cantidad_posts = len(mis_posts)

print("Cantidad de posts:", cantidad_posts)
```

Salida esperada:

CÓDIGO

```
Cantidad de posts: 3
```

Esto nos sirve, por ejemplo, para mostrar cuántas publicaciones tiene nuestro blog.

CÓDIGO

```
titulos_posts = ["Bienvenida", "Variables en Python", "Listas en Python"]

print(f"El blog tiene {len(titulos_posts)} posts publicados.")
```

Salida esperada:

CÓDIGO

```
El blog tiene 3 posts publicados.
```

8. Eliminar elementos con `pop()`

Para quitar un elemento de una lista podemos usar `pop()`.

`pop()` elimina un elemento según su índice y además nos devuelve ese elemento.

Ejemplo:

CÓDIGO

```
mis_posts = ["Bienvenida", "Mi primer viaje", "Receta de pasta"]

post_eliminado = mis_posts.pop(1)

print("Post eliminado:", post_eliminado)
print("Lista actualizada:", mis_posts)
```

Salida esperada:

CÓDIGO

```
Post eliminado: Mi primer viaje
Lista actualizada: ["Bienvenida", "Receta de pasta"]
```

En este caso, eliminamos el elemento que estaba en la posición `1`.

Recordemos que Python empieza a contar desde `0`, por eso la posición `1` corresponde al segundo elemento.

Si usamos `pop()` sin indicar una posición, elimina el último elemento:

CÓDIGO

```
mis_posts = ["Bienvenida", "Mi primer viaje", "Receta de pasta"]

ultimo_post = mis_posts.pop()

print("Último post eliminado:", ultimo_post)
print("Lista actualizada:", mis_posts)
```

Salida esperada:

CÓDIGO

```
Último post eliminado: Receta de pasta
Lista actualizada: ["Bienvenida", "Mi primer viaje"]
```

Nota importante sobre algunos ejemplos

A partir de esta unidad pueden aparecer algunos ejemplos que utilizan la sentencia `for`.

El `for` es una herramienta de Python que nos permite recorrer colecciones de datos, como listas, tuplas o diccionarios, y realizar una acción con cada elemento.

Este tema se explicará en detalle en el siguiente módulo, por lo que no es necesario que lo domines todavía. Por ahora, alcanza con que entiendas su idea general: nos ayuda a revisar varios elementos uno por uno.

Si algún ejemplo con `for` no queda completamente claro en este momento, no te detengas demasiado. Lo importante en esta unidad es comprender la estructura de datos que estamos trabajando. En el próximo módulo vamos a estudiar el uso de `for` con más profundidad y práctica.

9. Recorrer una lista con `for`

Muchas veces no queremos mostrar un solo post, sino todos.

Para eso podemos recorrer la lista usando un bucle `for`.

CÓDIGO

```
mis_posts = ["Bienvenida", "Mi primer viaje", "Receta de pasta"]

for post in mis_posts:
    print(post)
```

Salida esperada:

CÓDIGO

```
Bienvenida
Mi primer viaje
Receta de pasta
```

En este ejemplo, la variable `post` va tomando el valor de cada elemento de la lista.

Primero vale `"Bienvenida"`, luego `"Mi primer viaje"` y finalmente `"Receta de pasta"`.

10. Recorrer una lista con `enumerate()`

Cuando mostramos una lista de posts, muchas veces queremos mostrar también el número de posición.

Para eso podemos usar `enumerate()`.

CÓDIGO

```
mis_posts = ["Bienvenida", "Mi primer viaje", "Receta de pasta"]

for indice, post in enumerate(mis_posts):
    print(indice, post)
```

Salida esperada:

CÓDIGO

```
0 Bienvenida
1 Mi primer viaje
2 Receta de pasta
```

`enumerate()` nos da dos datos al mismo tiempo:

- el índice,
- el elemento.

También podemos mostrarlo de una forma más amigable para el usuario:

CÓDIGO

```
mis_posts = ["Bienvenida", "Mi primer viaje", "Receta de pasta"]

for indice, post in enumerate(mis_posts):
    print(f"{indice + 1}. {post}")
```

Salida esperada:

CÓDIGO

```
1. Bienvenida
2. Mi primer viaje
3. Receta de pasta
```

Usamos `indice + 1` porque Python empieza desde `0`, pero para una persona suele ser más natural ver una lista que empieza desde `1`.

11. Diferencia entre índice y número visible

Este punto es muy importante.

Supongamos que mostramos esta lista al usuario:

CÓDIGO

```
1. Bienvenida
2. Mi primer viaje
3. Receta de pasta
```

El usuario ve los números `1`, `2` y `3`.

Pero Python internamente usa los índices `0`, `1` y `2`.

Entonces, si el usuario elige borrar el post número `2`, en Python necesitamos borrar el índice `1`.

Ejemplo:

CÓDIGO

```
mis_posts = ["Bienvenida", "Mi primer viaje", "Receta de pasta"]

opcion_usuario = int(input("¿Qué post quieres borrar? "))

indice_real = opcion_usuario - 1

post_eliminado = mis_posts.pop(indice_real)

print("Post eliminado:", post_eliminado)
print("Lista actualizada:", mis_posts)
```

Si el usuario escribe:

CÓDIGO

```
2
```

Se elimina:

CÓDIGO

```
Mi primer viaje
```

Porque el número visible `2` corresponde al índice real `1`.

12. Error común: `IndexError`

Uno de los errores más comunes al trabajar con listas es intentar acceder a una posición que no existe.

Por ejemplo:

CÓDIGO

```
mis_posts = ["Bienvenida", "Mi primer viaje", "Receta de pasta"]

print(mis_posts[10])
```

Esto produce un error, porque la lista solo tiene tres elementos.

Los índices válidos son:

CÓDIGO

```
0, 1, 2
```

No existe la posición `10`.

El error que aparece se llama `IndexError`.

También puede pasar con `pop()`:

CÓDIGO

```
mis_posts = ["Bienvenida", "Mi primer viaje", "Receta de pasta"]

mis_posts.pop(5)
```

Ese código falla porque no existe el índice `5`.

13. Diferencia entre `append()` y `extend()`

Al principio, para agregar un nuevo post, vamos a usar `append()`.

CÓDIGO

```
mis_posts = ["Bienvenida", "Mi primer viaje"]

mis_posts.append("Receta de pasta")

print(mis_posts)
```

Salida esperada:

CÓDIGO

```
["Bienvenida", "Mi primer viaje", "Receta de pasta"]
```

`append()` agrega un elemento completo.

En cambio, `extend()` sirve para unir una lista con otra lista.

CÓDIGO

```
posts_python = ["Variables", "Listas"]
posts_django = ["Modelos", "Templates"]

posts_python.extend(posts_django)

print(posts_python)
```

Salida esperada:

CÓDIGO

```
["Variables", "Listas", "Modelos", "Templates"]
```

Por ahora, si quieres agregar un solo post, usa `append()`.

14. Ejemplo completo: catálogo inicial del blog

Veamos un programa que nos permita crear y mostrar un catálogo simple de posts.

CÓDIGO

```
posts = []

posts.append("Bienvenida al blog")
posts.append("Variables en Python")
posts.append("Listas en Python")

print("----- CATÁLOGO DEL BLOG -----")

for indice, post in enumerate(posts):
    print(f"{indice + 1}. {post}")

print(f"Total de posts: {len(posts)}")
```

Salida esperada:

CÓDIGO

```
----- CATÁLOGO DEL BLOG -----
1. Bienvenida al blog
2. Variables en Python
3. Listas en Python
Total de posts: 3
```

15. Ejemplo con interacción del usuario

Ahora veamos un ejemplo donde el usuario puede agregar un nuevo post.

CÓDIGO

```
posts = ["Bienvenida al blog", "Variables en Python"]

nuevo_post = input("Escribe el título del nuevo post: ")

posts.append(nuevo_post)

print("----- POSTS ACTUALES -----")

for indice, post in enumerate(posts):
    print(f"{indice + 1}. {post}")
```

Ejemplo de ejecución:

CÓDIGO

```
Escribe el título del nuevo post: Listas en Python
```

Salida esperada:

CÓDIGO

```
----- POSTS ACTUALES -----
1. Bienvenida al blog
2. Variables en Python
3. Listas en Python
```

16. Ejemplo: eliminar un post elegido por el usuario

También podemos permitir que el usuario elija qué post eliminar.

CÓDIGO

```
posts = ["Bienvenida al blog", "Variables en Python", "Listas en Python"]

print("----- POSTS DISPONIBLES -----")

for indice, post in enumerate(posts):
    print(f"{indice + 1}. {post}")

opcion = int(input("Ingresa el número del post que quieres eliminar: "))

indice_real = opcion - 1

post_eliminado = posts.pop(indice_real)

print(f"El post eliminado fue: {post_eliminado}")

print("----- POSTS ACTUALIZADOS -----")

for indice, post in enumerate(posts):
    print(f"{indice + 1}. {post}")
```

Ejemplo de ejecución:

CÓDIGO

```
----- POSTS DISPONIBLES -----
1. Bienvenida al blog
2. Variables en Python
3. Listas en Python
Ingresa el número del post que quieres eliminar: 2
```

Salida esperada:

CÓDIGO

```
El post eliminado fue: Variables en Python
----- POSTS ACTUALIZADOS -----
1. Bienvenida al blog
2. Listas en Python
```

Este ejemplo todavía puede fallar si el usuario escribe una opción inválida, pero más adelante aprenderemos a validar esos casos.

17. Mini práctica guiada

Crea un archivo llamado `catalogo_blog.py`.

Dentro del archivo:

1. Crea una lista llamada `posts`.
2. Agrega al menos tres títulos usando `append()`.
3. Muestra todos los posts usando `for` y `enumerate()`.
4. Muestra la cantidad total de posts usando `len()`.
5. Elimina el último post usando `pop()`.
6. Muestra qué post fue eliminado.
7. Vuelve a mostrar la lista actualizada.

Ejemplo posible:

CÓDIGO

```
posts = []

posts.append("Bienvenida al blog")
posts.append("Variables en Python")
posts.append("Listas en Python")

print("----- POSTS DEL BLOG -----")

for indice, post in enumerate(posts):
    print(f"{indice + 1}. {post}")

print(f"Total de posts: {len(posts)}")

post_eliminado = posts.pop()

print(f"Post eliminado: {post_eliminado}")

print("----- POSTS ACTUALIZADOS -----")

for indice, post in enumerate(posts):
    print(f"{indice + 1}. {post}")

print(f"Total de posts: {len(posts)}")
```

18. Desafío extra

Mejora el programa anterior para que el usuario pueda escribir el título de un nuevo post.

Luego agrega ese post a la lista y muestra el catálogo actualizado.

CÓDIGO

```
posts = ["Bienvenida al blog", "Variables en Python", "Listas en Python"]

nuevo_post = input("Escribe el título del nuevo post: ")

posts.append(nuevo_post)

print("----- CATÁLOGO ACTUALIZADO -----")

for indice, post in enumerate(posts):
    print(f"{indice + 1}. {post}")

print(f"Total de posts: {len(posts)}")
```

19. Síntesis de la unidad

En esta unidad aprendiste a usar listas para organizar varios datos relacionados.

Una lista nos permite guardar muchos elementos dentro de una sola variable.

Vimos cómo:

- Crear listas usando corchetes `[]`.
- Acceder a elementos usando índices.
- Recordar que Python empieza a contar desde `0`.
- Obtener el último elemento usando `-1`.
- Agregar elementos con `append()`.
- Eliminar elementos con `pop()`.
- Contar elementos con `len()`.
- Recorrer listas usando `for`.
- Mostrar índices y valores usando `enumerate()`.

Estos conceptos son fundamentales para nuestro proyecto de blog.

Primero representamos un solo post con variables. Ahora podemos representar varios posts dentro de una lista. Más adelante vamos a mejorar esta estructura para guardar no solo títulos, sino también autores, vistas, etiquetas y estados importantes de cada publicación.

Diccionarios para representar autores y su perfil

Aprende a estructurar información compleja mediante diccionarios en Python, modelando perfiles de autores y vinculándolos con sus publicaciones de forma organizada.

Diccionarios para representar autores y perfiles

Imagina que estás leyendo un blog. Al final de cada artículo aparece una pequeña sección que dice: “Sobre el autor”.

Allí puedes encontrar datos como:

- nombre del autor,
- una breve bio,
- especialidad,
- redes sociales,
- cantidad de artículos publicados.

Con variables simples podríamos hacer algo así:

CÓDIGO

```
nombre_autor = "Ana Pérez"
bio_autor = "Desarrolladora backend y docente de Python"
especialidad_autor = "Django"
twitter_autor = "@anapython"
```

Esto funciona si tenemos un solo autor. Pero si tenemos muchos autores, rápidamente se vuelve difícil de manejar.

También podríamos usar una lista:

CÓDIGO

```
autor = ["Ana Pérez", "Desarrolladora backend y docente de Python", "Django",
"@anapython"]
```

Pero aparece otro problema: necesitamos recordar que la posición `0` es el nombre, la posición `1` es la bio, la posición `2` es la especialidad y la posición `3` es Twitter.

Eso puede volverse confuso.

Para resolver este problema, Python nos ofrece una estructura muy útil: los **diccionarios**.

1. ¿Qué es un diccionario?

Un **diccionario** es una estructura de datos que permite guardar información usando pares de **clave** y **valor**.

Podemos pensarlo como una ficha de datos.

Por ejemplo:

CÓDIGO

```
autor = {
    "nombre": "Ana Pérez",
    "bio": "Desarrolladora backend y docente de Python",
    "especialidad": "Django",
    "twitter": "@anapython"
}
```

En este ejemplo, tenemos un diccionario llamado `autor`.

Cada dato está guardado con una etiqueta:

CÓDIGO

```
"nombre"
"bio"
"especialidad"
"twitter"
```

A esas etiquetas las llamamos **claves**.

Y a la información asociada a cada clave la llamamos **valor**.

Por ejemplo:

CÓDIGO

```
"nombre": "Ana Pérez"
```

La clave es:

CÓDIGO

```
"nombre"
```

Y el valor es:

CÓDIGO

```
"Ana Pérez"
```

2. Sintaxis básica de un diccionario

Los diccionarios se escriben usando llaves `{}`.

Cada elemento tiene esta forma:

CÓDIGO

```
"clave": valor
```

Los distintos pares clave-valor se separan con comas.

Ejemplo:

CÓDIGO

```
autor = {
    "nombre": "Beatriz Código",
    "especialidad": "Backend con Django",
    "articulos_publicados": 12,
    "es_admin": True
}
```

En este diccionario tenemos distintos tipos de datos:

CÓDIGO

```
"nombre": "Beatriz Código"
```

Es un texto, es decir, un `str`.

CÓDIGO

```
"articulos_publicados": 12
```

Es un número entero, es decir, un `int`.

CÓDIGO

```
"es_admin": True
```

Es un valor booleano, es decir, un `bool`.

Los diccionarios pueden guardar valores de diferentes tipos.

3. Claves y valores

En un diccionario tenemos dos conceptos importantes:

Clave

La **clave** es el nombre del dato.

Ejemplos:

CÓDIGO

```
"nombre"
"especialidad"
"articulos_publicados"
"es_admin"
```

En general, vamos a usar textos como claves.

Valor

El **valor** es el dato que guardamos dentro de esa clave.

Ejemplos:

CÓDIGO

```
"Beatriz Código"
"Backend con Django"
12
True
```

Entonces, este par:

CÓDIGO

```
"especialidad": "Backend con Django"
```

se puede leer como:

> **Nota**
>
> En la clave `"especialidad"` guardamos el valor `"Backend con Django"`.

4. Diferencia entre listas y diccionarios

Las listas y los diccionarios sirven para organizar datos, pero no se usan exactamente para lo mismo.

Una lista accede a los datos por posición:

CÓDIGO

```
autor = ["Ana Pérez", "Django", "@anapython"]

print(autor[0])
```

Salida esperada:

CÓDIGO

```
Ana Pérez
```

El problema es que tenemos que recordar qué significa cada posición.

En cambio, un diccionario accede a los datos por nombre:

CÓDIGO

```
autor = {
    "nombre": "Ana Pérez",
    "especialidad": "Django",
    "twitter": "@anapython"
}

print(autor["nombre"])
```

Salida esperada:

CÓDIGO

```
Ana Pérez
```

Este código es más claro, porque la clave `"nombre"` nos dice exactamente qué dato estamos buscando.

5. Crear un perfil de autor

Veamos un ejemplo aplicado al proyecto del blog.

CÓDIGO

```
perfil_autor = {
    "nombre": "Carlos Dev",
    "bio": "Programador Python y creador de contenido técnico.",
    "especialidad": "Django",
    "articulos_publicados": 8,
    "activo": True
}
```

Podemos mostrar el diccionario completo:

CÓDIGO

```
print(perfil_autor)
```

Pero también podemos mostrar datos específicos.

CÓDIGO

```
print("Nombre:", perfil_autor["nombre"])
print("Especialidad:", perfil_autor["especialidad"])
print("Artículos publicados:", perfil_autor["articulos_publicados"])
```

Salida esperada:

CÓDIGO

```
Nombre: Carlos Dev
Especialidad: Django
Artículos publicados: 8
```

6. Acceder a un valor usando una clave

Para leer un dato de un diccionario usamos corchetes `[]` y escribimos el nombre de la clave.

CÓDIGO

```
autor = {
    "nombre": "Beatriz Código",
    "especialidad": "Backend con Django",
    "articulos_publicados": 12
}

print(autor["nombre"])
```

Salida esperada:

CÓDIGO

```
Beatriz Código
```

También podemos guardar ese valor en una variable:

CÓDIGO

```
nombre_autor = autor["nombre"]

print(nombre_autor)
```

Salida esperada:

CÓDIGO

```
Beatriz Código
```

Ejemplo aplicado:

CÓDIGO

```
autor = {
    "nombre": "Ana Python",
    "bio": "Docente y desarrolladora backend.",
    "especialidad": "Python"
}

print(f"{autor['nombre']} se especializa en {autor['especialidad']}.")
```

Salida esperada:

CÓDIGO

```
Ana Python se especializa en Python.
```

En este caso usamos comillas simples dentro de la f-string para acceder a las claves:

CÓDIGO

```
autor['nombre']
autor['especialidad']
```

7. Actualizar un valor existente

Los diccionarios son **mutables**.

Eso significa que podemos modificarlos después de crearlos.

Por ejemplo, si queremos actualizar la cantidad de artículos publicados:

CÓDIGO

```
autor = {
    "nombre": "Beatriz Código",
    "especialidad": "Backend con Django",
    "articulos_publicados": 12
}

autor["articulos_publicados"] = 13

print(autor)
```

Salida esperada:

CÓDIGO

```
{'nombre': 'Beatriz Código', 'especialidad': 'Backend con Django',
'articulos_publicados': 13}
```

Como la clave `"articulos_publicados"` ya existía, Python reemplazó el valor anterior por el nuevo.

8. Agregar una nueva clave

También podemos agregar información nueva a un diccionario.

Si asignamos un valor a una clave que no existe, Python la crea automáticamente.

CÓDIGO

```
autor = {
    "nombre": "Beatriz Código",
    "especialidad": "Backend con Django"
}

autor["twitter"] = "@beacodigo"

print(autor)
```

Salida esperada:

CÓDIGO

```
{'nombre': 'Beatriz Código', 'especialidad': 'Backend con Django', 'twitter':
'@beacodigo'}
```

Ahora el diccionario tiene una nueva clave llamada `"twitter"`.

Ejemplo más completo:

CÓDIGO

```
autor = {
    "nombre": "Carlos Dev",
    "bio": "Programador Python."
}

autor["especialidad"] = "Django"
autor["articulos_publicados"] = 5
autor["activo"] = True

print(autor)
```

9. Eliminar una clave con `pop()`

Al igual que en listas, también podemos usar `pop()` en diccionarios.

Pero en diccionarios no eliminamos por índice, sino por clave.

CÓDIGO

```
autor = {
    "nombre": "Ana Python",
    "especialidad": "Django",
    "twitter": "@anapython"
}

twitter_eliminado = autor.pop("twitter")

print("Twitter eliminado:", twitter_eliminado)
print("Autor actualizado:", autor)
```

Salida esperada:

CÓDIGO

```
Twitter eliminado: @anapython
Autor actualizado: {'nombre': 'Ana Python', 'especialidad': 'Django'}
```

`pop()` elimina la clave indicada y devuelve el valor eliminado.

10. Error común: `KeyError`

Uno de los errores más comunes al trabajar con diccionarios es intentar acceder a una clave que no existe.

Por ejemplo:

CÓDIGO

```
autor = {
    "nombre": "Ana Python",
    "especialidad": "Django"
}

print(autor["telefono"])
```

Este código falla porque el diccionario no tiene una clave llamada `"telefono"`.

Python muestra un error llamado `KeyError`.

Este error significa:

> **Nota**
>
> Estás intentando acceder a una clave que no existe en el diccionario.

11. Acceder de forma segura con `get()`

Para evitar un `KeyError`, podemos usar el método `get()`.

CÓDIGO

```
autor = {
    "nombre": "Ana Python",
    "especialidad": "Django"
}

telefono = autor.get("telefono")

print(telefono)
```

Salida esperada:

CÓDIGO

```
None
```

`None` significa que no hay valor disponible.

También podemos indicar un valor por defecto:

CÓDIGO

```
autor = {
    "nombre": "Ana Python",
    "especialidad": "Django"
}

telefono = autor.get("telefono", "No disponible")

print(telefono)
```

Salida esperada:

CÓDIGO

```
No disponible
```

Esto es muy útil cuando no estamos seguros de si una clave existe.

Ejemplo aplicado:

CÓDIGO

```
autor = {
    "nombre": "Carlos Dev",
    "especialidad": "Python"
}

twitter = autor.get("twitter", "El autor no cargó Twitter")

print(twitter)
```

Salida esperada:

CÓDIGO

```
El autor no cargó Twitter
```

12. Recorrer un diccionario

Podemos recorrer un diccionario para mostrar sus claves y valores.

Para eso usamos `.items()`.

CÓDIGO

```
autor = {
    "nombre": "Ana Python",
    "especialidad": "Django",
    "articulos_publicados": 10
}

for clave, valor in autor.items():
    print(clave, ":", valor)
```

Salida esperada:

CÓDIGO

```
nombre : Ana Python
especialidad : Django
articulos_publicados : 10
```

También podemos hacerlo con f-strings:

CÓDIGO

```
for clave, valor in autor.items():
    print(f"{clave}: {valor}")
```

Salida esperada:

CÓDIGO

```
nombre: Ana Python
especialidad: Django
articulos_publicados: 10
```

13. Diccionarios con listas dentro

Un diccionario puede tener una lista como valor.

Esto nos permite representar información más completa.

Por ejemplo, un autor puede tener varias redes sociales:

CÓDIGO

```
perfil_autor = {
    "nombre": "Carlos Dev",
    "especialidad": "Django",
    "redes_sociales": ["@carlosdev", "linkedin.com/in/carlosdev"]
}
```

Para acceder a la lista completa:

CÓDIGO

```
print(perfil_autor["redes_sociales"])
```

Salida esperada:

CÓDIGO

```
['@carlosdev', 'linkedin.com/in/carlosdev']
```

Para acceder a la primera red social:

CÓDIGO

```
print(perfil_autor["redes_sociales"][0])
```

Salida esperada:

CÓDIGO

```
@carlosdev
```

Aquí estamos combinando dos accesos:

CÓDIGO

```
perfil_autor["redes_sociales"]
```

nos devuelve la lista.

Y luego:

CÓDIGO

```
[0]
```

nos devuelve el primer elemento de esa lista.

14. Lista de posts dentro de un autor

También podemos guardar una lista de posts dentro del perfil del autor.

CÓDIGO

```
perfil_autor = {
    "nombre": "Carlos Dev",
    "posts": ["Mi primer post en Django", "Aprendiendo variables", "Guía básica de
Git"]
}

print(perfil_autor["posts"])
```

Salida esperada:

CÓDIGO

```
['Mi primer post en Django', 'Aprendiendo variables', 'Guía básica de Git']
```

Para acceder al primer post:

CÓDIGO

```
print(perfil_autor["posts"][0])
```

Salida esperada:

CÓDIGO

```
Mi primer post en Django
```

Para recorrer todos los posts del autor:

CÓDIGO

```
for post in perfil_autor["posts"]:
    print(post)
```

Salida esperada:

CÓDIGO

```
Mi primer post en Django
Aprendiendo variables
Guía básica de Git
```

También podemos mostrarlo numerado:

CÓDIGO

```
for indice, post in enumerate(perfil_autor["posts"]):
    print(f"{indice + 1}. {post}")
```

Salida esperada:

CÓDIGO

```
1. Mi primer post en Django
2. Aprendiendo variables
3. Guía básica de Git
```

15. Diccionarios dentro de diccionarios

Un diccionario también puede tener otro diccionario como valor.

Por ejemplo, un post puede tener un autor completo dentro:

CÓDIGO

```
post_reciente = {
    "titulo": "Diccionarios en Python",
    "fecha": "2026-06-03",
    "autor": {
        "nombre": "Ana Python",
        "especialidad": "Backend",
        "redes": ["@anapy", "@anacode"]
    }
}
```

Para acceder al título del post:

CÓDIGO

```
print(post_reciente["titulo"])
```

Salida esperada:

CÓDIGO

```
Diccionarios en Python
```

Para acceder al nombre del autor:

CÓDIGO

```
print(post_reciente["autor"]["nombre"])
```

Salida esperada:

CÓDIGO

```
Ana Python
```

Este acceso se lee de izquierda a derecha:

CÓDIGO

```
post_reciente["autor"]
```

nos devuelve el diccionario del autor.

Luego:

CÓDIGO

```
["nombre"]
```

nos devuelve el nombre dentro de ese diccionario.

Para acceder a la primera red social del autor:

CÓDIGO

```
print(post_reciente["autor"]["redes"][0])
```

Salida esperada:

CÓDIGO

```
@anapy
```

Aquí combinamos:

- diccionario,
- otro diccionario,
- lista,
- índice.

16. El patrón lista de diccionarios

Uno de los patrones más usados en programación es tener una **lista de diccionarios**.

Esto nos permite representar muchos objetos similares.

Por ejemplo, varios posts del blog:

CÓDIGO

```
blog = [
    {
        "titulo": "Variables en Python",
        "autor": "Ana",
        "vistas": 120
    },
    {
        "titulo": "Listas en Python",
        "autor": "Beto",
        "vistas": 95
    },
    {
        "titulo": "Diccionarios en Python",
        "autor": "Ana",
        "vistas": 180
    }
]
```

En este ejemplo:

- `blog` es una lista.
- Cada elemento de la lista es un diccionario.
- Cada diccionario representa un post.

Podemos recorrer la lista y mostrar los títulos:

CÓDIGO

```
for post in blog:
    print(post["titulo"])
```

Salida esperada:

CÓDIGO

```
Variables en Python
Listas en Python
Diccionarios en Python
```

También podemos mostrar más información:

CÓDIGO

```
for post in blog:
    print(f"{post['titulo']} - Autor: {post['autor']} - Vistas: {post['vistas']}")
```

Salida esperada:

CÓDIGO

```
Variables en Python - Autor: Ana - Vistas: 120
Listas en Python - Autor: Beto - Vistas: 95
Diccionarios en Python - Autor: Ana - Vistas: 180
```

17. Relación con JSON

Los diccionarios de Python se parecen mucho a un formato muy usado en internet llamado **JSON**.

JSON significa **JavaScript Object Notation**.

Es un formato que se usa para intercambiar información entre sistemas.

Por ejemplo, cuando una página web le pide datos a un servidor, muchas veces esos datos viajan con una estructura parecida a esta:

CÓDIGO

```
{
    "titulo": "Diccionarios en Python",
    "autor": "Ana Python",
    "vistas": 180
}
```

Por eso, entender diccionarios en Python es un paso muy importante para entender cómo viajan los datos en aplicaciones web.

Más adelante, cuando trabajemos con Django, vamos a ver que muchas estructuras de datos se parecen a lo que estamos aprendiendo ahora.

18. Ejemplo completo: perfil de autor

Veamos un ejemplo completo usando diccionarios, listas y acceso a datos.

CÓDIGO

```
perfil_autor = {
    "nombre": "Lucía Dev",
    "bio": "Programadora Python y creadora de contenido sobre desarrollo web.",
    "especialidad": "Django",
    "redes_sociales": ["@luciadev", "linkedin.com/in/luciadev"],
    "articulos_publicados": 4,
    "activo": True
}

print("----- PERFIL DEL AUTOR -----")
print(f"Nombre: {perfil_autor['nombre']}")
print(f"Bio: {perfil_autor['bio']}")
print(f"Especialidad: {perfil_autor['especialidad']}")
print(f"Primera red social: {perfil_autor['redes_sociales'][0]}")
print(f"Artículos publicados: {perfil_autor['articulos_publicados']}")
print(f"Activo: {perfil_autor['activo']}")
```

Salida esperada:

CÓDIGO

```
----- PERFIL DEL AUTOR -----
Nombre: Lucía Dev
Bio: Programadora Python y creadora de contenido sobre desarrollo web.
Especialidad: Django
Primera red social: @luciadev
Artículos publicados: 4
Activo: True
```

19. Ejemplo completo: post con autor anidado

Ahora representemos un post que contiene el perfil del autor.

CÓDIGO

```
post = {
    "titulo": "Introducción a diccionarios",
    "vistas": 250,
    "rating": 4.8,
    "estado": "publicado",
    "autor": {
        "nombre": "Lucía Dev",
        "especialidad": "Django",
        "redes_sociales": ["@luciadev", "linkedin.com/in/luciadev"]
    }
}

print("----- DETALLE DEL POST -----")
print(f"Título: {post['titulo']}")
print(f"Estado: {post['estado']}")
print(f"Vistas: {post['vistas']}")
print(f"Rating: {post['rating']}")
print(f"Autor: {post['autor']['nombre']}")
print(f"Especialidad del autor: {post['autor']['especialidad']}")
print(f"Red principal: {post['autor']['redes_sociales'][0]}")
```

Salida esperada:

CÓDIGO

```
----- DETALLE DEL POST -----
Título: Introducción a diccionarios
Estado: publicado
Vistas: 250
Rating: 4.8
Autor: Lucía Dev
Especialidad del autor: Django
Red principal: @luciadev
```

20. Errores comunes

Error 1: olvidar las comas

Incorrecto:

CÓDIGO

```
autor = {
    "nombre": "Ana"
    "especialidad": "Django"
}
```

Ese código falla porque falta una coma después de `"Ana"`.

Correcto:

CÓDIGO

```
autor = {
    "nombre": "Ana",
    "especialidad": "Django"
}
```

Error 2: usar = en lugar de :

Incorrecto:

CÓDIGO

```
autor = {
    "nombre" = "Ana",
    "especialidad" = "Django"
}
```

Correcto:

CÓDIGO

```
autor = {
    "nombre": "Ana",
    "especialidad": "Django"
}
```

En los diccionarios usamos dos puntos `:` para separar la clave del valor.

Error 3: buscar una clave que no existe

Incorrecto:

CÓDIGO

```
autor = {
    "nombre": "Ana"
}

print(autor["twitter"])
```

Ese código produce un `KeyError`.

Una forma más segura sería:

CÓDIGO

```
autor = {
    "nombre": "Ana"
}

twitter = autor.get("twitter", "No disponible")

print(twitter)
```

Salida esperada:

CÓDIGO

```
No disponible
```

Error 4: confundir índice con clave

En una lista accedemos por índice:

CÓDIGO

```
posts = ["Variables", "Listas", "Diccionarios"]

print(posts[0])
```

En un diccionario accedemos por clave:

CÓDIGO

```
autor = {
    "nombre": "Ana",
    "especialidad": "Django"
}

print(autor["nombre"])
```

Esto estaría mal:

CÓDIGO

```
print(autor[0])
```

Porque el diccionario no tiene una clave llamada `0`.

Error 5: claves poco claras

Evita hacer esto:

CÓDIGO

```
autor = {
    "dato1": "Ana",
    "dato2": "Django",
    "dato3": 10
}
```

Es mejor hacer esto:

CÓDIGO

```
autor = {
    "nombre": "Ana",
    "especialidad": "Django",
    "articulos_publicados": 10
}
```

Los nombres claros hacen que el código sea más fácil de leer.

21. Mini práctica guiada

Crea un archivo llamado `perfil_autor.py`.

Dentro del archivo:

1. Crea un diccionario llamado `perfil_autor`.
2. Debe tener las siguientes claves:
   - `"nombre"`
   - `"bio"`
   - `"especialidad"`
   - `"redes_sociales"`
   - `"articulos_publicados"`
   - `"activo"`
3. La clave `"redes_sociales"` debe guardar una lista con al menos dos redes.
4. Muestra en consola:
   - el nombre del autor,
   - su especialidad,
   - su primera red social,
   - la cantidad de artículos publicados.

Ejemplo posible:

CÓDIGO

```
perfil_autor = {
    "nombre": "Martina Code",
    "bio": "Desarrolladora Python interesada en educación y tecnología.",
    "especialidad": "Backend",
    "redes_sociales": ["@martinacode", "linkedin.com/in/martinacode"],
    "articulos_publicados": 6,
    "activo": True
}

print("----- PERFIL DEL AUTOR -----")
print(f"Nombre: {perfil_autor['nombre']}")
print(f"Especialidad: {perfil_autor['especialidad']}")
print(f"Primera red social: {perfil_autor['redes_sociales'][0]}")
print(f"Artículos publicados: {perfil_autor['articulos_publicados']}")
```

22. Desafío extra

Crea una lista llamada `blog`.

Dentro de esa lista, agrega tres diccionarios. Cada diccionario debe representar un post con:

- título,
- autor,
- estado,
- vistas.

Luego recorre la lista y muestra cada post en consola.

Ejemplo:

CÓDIGO

```
blog = [
    {
        "titulo": "Variables en Python",
        "autor": "Ana",
        "estado": "publicado",
        "vistas": 120
    },
    {
        "titulo": "Listas en Python",
        "autor": "Beto",
        "estado": "borrador",
        "vistas": 95
    },
    {
        "titulo": "Diccionarios en Python",
        "autor": "Carla",
        "estado": "publicado",
        "vistas": 180
    }
]

print("----- POSTS DEL BLOG -----")

for post in blog:
    print(f"{post['titulo']} - Autor: {post['autor']} - Estado: {post['estado']} -
Vistas: {post['vistas']}")
```

Salida esperada:

CÓDIGO

```
----- POSTS DEL BLOG -----
Variables en Python - Autor: Ana - Estado: publicado - Vistas: 120
Listas en Python - Autor: Beto - Estado: borrador - Vistas: 95
Diccionarios en Python - Autor: Carla - Estado: publicado - Vistas: 180
```

23. Síntesis de la unidad

En esta unidad aprendiste a usar diccionarios para representar información más organizada.

Un diccionario guarda datos usando pares de clave y valor.

Vimos cómo:

- Crear diccionarios usando llaves `{}`.
- Guardar información mediante claves descriptivas.
- Acceder a valores usando una clave.
- Actualizar valores existentes.
- Agregar nuevas claves.

- Eliminar claves usando `pop()`.
- Evitar errores usando `get()`.
- Recorrer diccionarios con `.items()`.
- Combinar diccionarios con listas.
- Crear estructuras anidadas.
- Usar el patrón lista de diccionarios.

Este concepto es fundamental para nuestro blog.

Con las variables representamos datos sueltos. Con las listas organizamos varios elementos. Y con los diccionarios empezamos a representar entidades más completas, como autores, perfiles y posts con múltiples datos relacionados.

Más adelante, cuando trabajemos con Django, esta forma de pensar será muy útil para entender cómo se organizan los modelos, los formularios y la información que viaja entre el backend y el frontend.

Tuplas para datos constantes del blog

Aprende a usar tuplas para proteger datos que no deben cambiar en tu blog y descubre la potencia del desempaquetado de variables.

Tuplas para proteger datos que no deberían cambiar

En programación, no todos los datos están hechos para cambiar.

En unidades anteriores trabajamos con listas y diccionarios. Esas estructuras son muy útiles porque nos permiten agregar, quitar y modificar información.

Por ejemplo, en un blog podríamos tener una lista de posts:

CÓDIGO

```
posts = ["Bienvenida", "Variables en Python", "Listas en Python"]

posts.append("Diccionarios en Python")
```

Eso tiene sentido, porque los posts pueden crecer, cambiar o eliminarse.

Pero hay otros datos que deberían mantenerse estables. Por ejemplo:

- el nombre del sitio,
- la versión del sistema,
- las categorías base del blog,
- el idioma por defecto,
- las dimensiones de una imagen de portada.

Para esos casos, Python nos ofrece una estructura llamada **tupla**.

1. ¿Qué es una tupla?

Una **tupla** es una colección de elementos ordenados e inmutables.

Esto significa que una vez creada, no podemos cambiar sus elementos.

Podemos pensarla así:

- Una lista es como una libreta: podemos borrar, agregar y modificar.
- Una tupla es como un documento impreso: una vez creado, queda fijo.

Ejemplo de tupla:

CÓDIGO

```
categorias = ("Backend", "Frontend", "DevOps")
```

En este caso, `categorias` guarda tres elementos.

La diferencia principal con una lista es que no podemos usar métodos como `append()` para agregar una nueva categoría.

2. Crear una tupla

Las tuplas normalmente se escriben usando paréntesis `()`.

CÓDIGO

```
categorias = ("Python", "Django", "Programación")
```

También pueden guardar distintos tipos de datos:

CÓDIGO

```
configuracion_blog = ("Mi Blog Python", "es", 1.0, True)
```

En este ejemplo tenemos:

- `"Mi Blog Python"`: nombre del blog.
- `"es"`: idioma.
- `1.0`: versión.
- `True`: indica si el blog está activo.

Aunque Python permite mezclar tipos, al principio conviene usar tuplas claras y simples.

3. Acceder a elementos de una tupla

Las tuplas son ordenadas, igual que las listas.

Eso significa que cada elemento tiene una posición, también llamada **índice**.

Python empieza a contar desde `0`.

CÓDIGO

```
categorias = ("Backend", "Frontend", "DevOps")

print(categorias[0])
print(categorias[1])
print(categorias[2])
```

Salida esperada:

CÓDIGO

```
Backend
Frontend
DevOps
```

En este ejemplo:

CÓDIGO

```
Índice 0 -> Backend
Índice 1 -> Frontend
Índice 2 -> DevOps
```

4. Obtener el último elemento con `-1`

Igual que con las listas, podemos usar índices negativos.

El índice `-1` representa el último elemento.

CÓDIGO

```
categorias = ("Backend", "Frontend", "DevOps")

print(categorias[-1])
```

Salida esperada:

CÓDIGO

```
DevOps
```

También podemos usar `-2` para obtener el anteúltimo elemento:

CÓDIGO

```
categorias = ("Backend", "Frontend", "DevOps")

print(categorias[-2])
```

Salida esperada:

CÓDIGO

```
Frontend
```

5. Las tuplas no se pueden modificar

La característica más importante de una tupla es que es **inmutable**.

Eso significa que no podemos cambiar sus elementos después de crearla.

Por ejemplo, esto daría error:

CÓDIGO

```
categorias = ("Backend", "Frontend", "DevOps")

categorias[0] = "Python"
```

Python mostraría un error porque no se puede modificar una posición de la tupla.

En cambio, con una lista sí podríamos hacerlo:

CÓDIGO

```
categorias = ["Backend", "Frontend", "DevOps"]

categorias[0] = "Python"

print(categorias)
```

Salida esperada:

CÓDIGO

```
['Python', 'Frontend', 'DevOps']
```

Esta diferencia es clave.

Las listas se pueden modificar.

Las tuplas no.

6. Intentar agregar elementos a una tupla

En una lista podemos usar `append()`:

CÓDIGO

```
posts = ["Variables", "Listas"]

posts.append("Diccionarios")

print(posts)
```

Salida esperada:

CÓDIGO

```
['Variables', 'Listas', 'Diccionarios']
```

Pero en una tupla no podemos hacerlo:

CÓDIGO

```
categorias = ("Python", "Django")

categorias.append("Frontend")
```

Ese código produce error porque las tuplas no tienen el método `append()`.

Esto es justamente lo que nos ayuda a proteger datos que no queremos modificar por accidente.

7. ¿Para qué usar tuplas en un blog?

En un blog, hay datos que pueden cambiar todo el tiempo.

Por ejemplo:

CÓDIGO

```
posts = ["Bienvenida", "Variables en Python", "Listas en Python"]
```

Esa lista puede crecer cuando agregamos nuevos posts.

Pero también hay datos que deberían mantenerse más estables:

CÓDIGO

```
configuracion_blog = ("Blog de Python", "es", "1.0.0")
```

Podemos leerlo así:

CÓDIGO

```
Nombre del blog -> Blog de Python
Idioma -> es
Versión -> 1.0.0
```

Ejemplo:

CÓDIGO

```
configuracion_blog = ("Blog de Python", "es", "1.0.0")

print("Nombre del blog:", configuracion_blog[0])
print("Idioma:", configuracion_blog[1])
print("Versión:", configuracion_blog[2])
```

Salida esperada:

CÓDIGO

```
Nombre del blog: Blog de Python
Idioma: es
Versión: 1.0.0
```

8. Tuplas para categorías fijas

Supongamos que nuestro blog tiene categorías iniciales que no queremos modificar durante la ejecución del programa.

CÓDIGO

```
categorias_base = ("Python", "Django", "Programación Web")
```

Podemos recorrer la tupla con un `for`:

CÓDIGO

```
categorias_base = ("Python", "Django", "Programación Web")

for categoria in categorias_base:
    print(categoria)
```

Salida esperada:

CÓDIGO

```
Python
Django
Programación Web
```

También podemos mostrar las categorías numeradas usando `enumerate()`:

CÓDIGO

```
categorias_base = ("Python", "Django", "Programación Web")

for indice, categoria in enumerate(categorias_base):
    print(f"{indice + 1}. {categoria}")
```

Salida esperada:

CÓDIGO

```
1. Python
2. Django
3. Programación Web
```

9. Tuplas para dimensiones de imágenes

Otro ejemplo útil son las dimensiones de una imagen.

Una imagen de portada puede tener ancho y alto.

CÓDIGO

```
dimensiones_portada = (1200, 600)
```

El primer valor representa el ancho.

El segundo valor representa el alto.

CÓDIGO

```
dimensiones_portada = (1200, 600)

print("Ancho:", dimensiones_portada[0])
print("Alto:", dimensiones_portada[1])
```

Salida esperada:

CÓDIGO

```
Ancho: 1200
Alto: 600
```

Este es un buen caso para usar tuplas porque las dimensiones suelen tratarse como un grupo fijo de datos.

10. Empaquetado de tuplas

Cuando creamos una tupla, estamos agrupando varios valores dentro de una sola variable.

A esto se lo suele llamar **empaquetado**.

CÓDIGO

```
coordenadas = (10.5, 20.8)
```

En este caso, los valores `10.5` y `20.8` quedan empaquetados dentro de la variable `coordenadas`.

Ejemplo aplicado al blog:

CÓDIGO

```
configuracion_blog = ("Blog de Python", "es", "1.0.0")
```

Estamos guardando tres datos juntos dentro de una misma tupla.

11. Desempaquetado de tuplas

El **desempaquetado** nos permite tomar los valores de una tupla y guardarlos en variables individuales.

Ejemplo:

CÓDIGO

```
coordenadas = (10.5, 20.8)

latitud, longitud = coordenadas

print(latitud)
print(longitud)
```

Salida esperada:

CÓDIGO

```
10.5
20.8
```

Esto significa que Python asigna:

CÓDIGO

```
latitud -> 10.5
longitud -> 20.8
```

Ejemplo aplicado al blog:

CÓDIGO

```
configuracion_blog = ("Blog de Python", "es", "1.0.0")

nombre_blog, idioma, version = configuracion_blog

print("Nombre:", nombre_blog)
print("Idioma:", idioma)
print("Versión:", version)
```

Salida esperada:

CÓDIGO

```
Nombre: Blog de Python
Idioma: es
Versión: 1.0.0
```

El desempaquetado hace que el código sea más claro cuando sabemos exactamente cuántos valores tiene la tupla.

12. Cuidado con el desempaquetado

La cantidad de variables debe coincidir con la cantidad de elementos de la tupla.

Esto funciona:

CÓDIGO

```
dimensiones = (1200, 600)

ancho, alto = dimensiones
```

Pero esto daría error:

CÓDIGO

```
dimensiones = (1200, 600)

ancho, alto, peso = dimensiones
```

El error ocurre porque la tupla tiene dos valores, pero estamos intentando guardarlos en tres variables.

También daría error hacer esto:

CÓDIGO

```
configuracion = ("Blog de Python", "es", "1.0.0")

nombre, idioma = configuracion
```

En ese caso sobraría un valor.

13. Tupla de un solo elemento

Este es uno de los errores más comunes al aprender tuplas.

Si queremos crear una tupla con un solo elemento, necesitamos poner una coma al final.

Correcto:

CÓDIGO

```
categoria = ("Python",)

print(type(categoria))
```

Salida esperada:

CÓDIGO

```
<class 'tuple'>
```

Incorrecto:

CÓDIGO

```
categoria = ("Python")

print(type(categoria))
```

Salida esperada:

CÓDIGO

```
<class 'str'>
```

¿Por qué pasa esto?

Porque para Python, esto:

CÓDIGO

```
("Python")
```

es simplemente un texto entre paréntesis.

La coma es lo que convierte ese valor en una tupla de un solo elemento:

CÓDIGO

```
("Python",)
```

14. Contar elementos con `len()`

Al igual que con listas, podemos usar `len()` para saber cuántos elementos tiene una tupla.

CÓDIGO

```
categorias = ("Python", "Django", "Web")

print(len(categorias))
```

Salida esperada:

CÓDIGO

```
3
```

Ejemplo aplicado:

CÓDIGO

```
categorias_base = ("Python", "Django", "Programación Web")

print(f"El blog tiene {len(categorias_base)} categorías base.")
```

Salida esperada:

CÓDIGO

```
El blog tiene 3 categorías base.
```

15. Buscar elementos con `in`

Podemos comprobar si un valor existe dentro de una tupla usando `in`.

CÓDIGO

```
categorias = ("Python", "Django", "Web")

print("Django" in categorias)
print("JavaScript" in categorias)
```

Salida esperada:

CÓDIGO

```
True
False
```

Ejemplo aplicado:

CÓDIGO

```
categorias_permitidas = ("Python", "Django", "Web")

categoria_post = "Django"

if categoria_post in categorias_permitidas:
    print("Categoría válida")
else:
    print("Categoría no permitida")
```

Salida esperada:

CÓDIGO

```
Categoría válida
```

Este ejemplo conecta las tuplas con validaciones simples.

16. Métodos útiles: `count()` e `index()`

Las tuplas tienen menos métodos que las listas porque no se pueden modificar.

Pero tienen algunos métodos útiles.

`count()`

Cuenta cuántas veces aparece un valor.

CÓDIGO

```
categorias = ("Python", "Django", "Python", "Web")

print(categorias.count("Python"))
```

Salida esperada:

CÓDIGO

```
2
```

`index()`

Devuelve la posición donde aparece un valor por primera vez.

CÓDIGO

```
categorias = ("Python", "Django", "Web")

print(categorias.index("Django"))
```

Salida esperada:

CÓDIGO

```
1
```

Recordemos que Python empieza a contar desde `0`.

17. Tuplas dentro de diccionarios

También podemos combinar tuplas con diccionarios.

Por ejemplo, podemos tener un diccionario de configuración del blog y guardar algunas opciones fijas en tuplas.

CÓDIGO

```
blog_config = {
    "nombre": "Blog de Python",
    "idioma": "es",
    "version": "1.0.0",
    "categorias_base": ("Python", "Django", "Web")
}
```

Podemos acceder a las categorías:

CÓDIGO

```
print(blog_config["categorias_base"])
```

Salida esperada:

CÓDIGO

```
('Python', 'Django', 'Web')
```

Y también recorrerlas:

CÓDIGO

```
for categoria in blog_config["categorias_base"]:
    print(categoria)
```

Salida esperada:

CÓDIGO

```
Python
Django
Web
```

18. Listas vs tuplas

Las listas y las tuplas se parecen porque ambas pueden guardar varios elementos ordenados.

Pero tienen una diferencia fundamental:

- Las listas son mutables.
- Las tuplas son inmutables.

Usamos listas cuando necesitamos modificar la colección.

Por ejemplo:

CÓDIGO

```
comentarios = ["Muy bueno", "Gracias por compartir"]

comentarios.append("Excelente explicación")
```

Tiene sentido usar una lista porque los comentarios pueden crecer.

Usamos tuplas cuando queremos representar datos que no deberían cambiar.

Por ejemplo:

CÓDIGO

```
dimensiones_portada = (1200, 600)
```

Tiene sentido usar una tupla porque el ancho y alto representan una configuración fija.

19. ¿Cuándo usar lista y cuándo usar tupla?

Usa una **lista** cuando necesites:

- agregar elementos,
- eliminar elementos,
- modificar elementos,
- representar una colección que cambia con el tiempo.

Ejemplos:

CÓDIGO

```
posts = ["Bienvenida", "Variables en Python"]
comentarios = ["Muy bueno", "Me sirvió"]
usuarios_registrados = ["ana", "beto", "carla"]
```

Usa una **tupla** cuando necesites:

- proteger datos de cambios accidentales,
- representar una configuración fija,
- agrupar datos que van juntos,
- trabajar con información que no debería modificarse.

Ejemplos:

CÓDIGO

```
categorias_base = ("Python", "Django", "Web")
dimensiones_portada = (1200, 600)
version_sistema = ("1", "0", "0")
```

20. Ejemplo completo: configuración del blog

Veamos un ejemplo completo usando tuplas para guardar datos estables de nuestro blog.

CÓDIGO

```
configuracion_blog = ("Blog de Python", "es", "1.0.0")
categorias_base = ("Python", "Django", "Programación Web")
dimensiones_portada = (1200, 600)

nombre_blog, idioma, version = configuracion_blog
ancho_portada, alto_portada = dimensiones_portada

print("----- CONFIGURACIÓN DEL BLOG -----")
print(f"Nombre: {nombre_blog}")
print(f"Idioma: {idioma}")
print(f"Versión: {version}")

print("----- CATEGORÍAS BASE -----")

for indice, categoria in enumerate(categorias_base):
    print(f"{indice + 1}. {categoria}")

print("----- IMAGEN DE PORTADA -----")
print(f"Ancho: {ancho_portada}px")
print(f"Alto: {alto_portada}px")
```

Salida esperada:

CÓDIGO

```
----- CONFIGURACIÓN DEL BLOG -----
Nombre: Blog de Python
Idioma: es
Versión: 1.0.0
----- CATEGORÍAS BASE -----
1. Python
2. Django
3. Programación Web
----- IMAGEN DE PORTADA -----
Ancho: 1200px
Alto: 600px
```

21. Ejemplo con validación de categoría

Podemos usar una tupla para definir cuáles son las categorías permitidas.

CÓDIGO

```
categorias_permitidas = ("Python", "Django", "Web")

categoria = input("Ingresa la categoría del post: ")

if categoria in categorias_permitidas:
    print("Categoría válida")
else:
    print("Categoría no permitida")
```

Ejemplo de ejecución:

CÓDIGO

```
Ingresa la categoría del post: Django
Categoría válida
```

Si el usuario escribe una categoría que no está en la tupla:

CÓDIGO

```
Ingresa la categoría del post: Cocina
Categoría no permitida
```

Este ejemplo muestra un uso muy práctico: definir valores permitidos que no deberían cambiar durante el programa.

22. Errores comunes

Error 1: intentar modificar una tupla

Incorrecto:

CÓDIGO

```
categorias = ("Python", "Django", "Web")

categorias[0] = "Backend"
```

Ese código falla porque las tuplas son inmutables.

Si necesitas modificar la colección, probablemente deberías usar una lista.

Error 2: intentar usar `append()`

Incorrecto:

CÓDIGO

```
categorias = ("Python", "Django")

categorias.append("Web")
```

Las tuplas no tienen `append()`.

Si necesitas agregar elementos, usa una lista:

CÓDIGO

```
categorias = ["Python", "Django"]

categorias.append("Web")

print(categorias)
```

Error 3: crear mal una tupla de un solo elemento

Incorrecto:

CÓDIGO

```
categoria = ("Python")
```

Eso no crea una tupla, crea un `str`.

Correcto:

CÓDIGO

```
categoria = ("Python",)
```

La coma final es obligatoria cuando la tupla tiene un solo elemento.

Error 4: confundir paréntesis con tupla

No todos los paréntesis crean tuplas.

CÓDIGO

```
nombre = ("Ana")
```

Esto es un texto.

CÓDIGO

```
nombre = ("Ana",)
```

Esto es una tupla.

Podemos comprobarlo con `type()`:

CÓDIGO

```
nombre = ("Ana")
nombre_tupla = ("Ana",)

print(type(nombre))
print(type(nombre_tupla))
```

Salida esperada:

CÓDIGO

```
<class 'str'>
<class 'tuple'>
```

Error 5: usar tuplas para datos que deberían cambiar

No conviene usar tuplas para datos que van a crecer o modificarse.

Por ejemplo, una lista de comentarios debería ser una lista:

CÓDIGO

```
comentarios = ["Buen post", "Muy claro"]
```

Porque después podríamos hacer:

CÓDIGO

```
comentarios.append("Gracias por compartir")
```

En cambio, si usáramos una tupla, no podríamos agregar nuevos comentarios.

23. Mini práctica guiada

Crea un archivo llamado `configuracion_blog.py`.

Dentro del archivo:

1. Crea una tupla llamada `configuracion_blog` con:
    - nombre del blog,
    - idioma,
    - versión.
2. Crea una tupla llamada `categorias_base` con al menos tres categorías.
3. Crea una tupla llamada `dimensiones_portada` con ancho y alto.
4. Usa desempaquetado para guardar los valores de `configuracion_blog` en variables individuales.
5. Muestra toda la información en consola.
6. Recorre las categorías usando `for` y `enumerate()`.

Ejemplo posible:

CÓDIGO

```
configuracion_blog = ("Blog de Python", "es", "1.0.0")
categorias_base = ("Python", "Django", "Web")
dimensiones_portada = (1200, 600)

nombre_blog, idioma, version = configuracion_blog
ancho, alto = dimensiones_portada

print("----- CONFIGURACIÓN DEL BLOG -----")
print(f"Nombre: {nombre_blog}")
print(f"Idioma: {idioma}")
print(f"Versión: {version}")

print("----- CATEGORÍAS -----")

for indice, categoria in enumerate(categorias_base):
    print(f"{indice + 1}. {categoria}")

print("----- PORTADA -----")
print(f"Dimensiones: {ancho}px x {alto}px")
```

24. Desafío extra

Crea una tupla llamada `categorias_permitidas`.

Luego pide al usuario que ingrese la categoría de un post.

Si la categoría existe dentro de la tupla, muestra:

CÓDIGO

```
La categoría es válida.
```

Si no existe, muestra:

CÓDIGO

```
La categoría no está permitida.
```

Ejemplo posible:

CÓDIGO

```
categorias_permitidas = ("Python", "Django", "Web")

categoria = input("Ingresa la categoría del post: ")

if categoria in categorias_permitidas:
    print("La categoría es válida.")
else:
    print("La categoría no está permitida.")
```

25. Síntesis de la unidad

En esta unidad aprendiste a usar tuplas para representar datos que no deberían cambiar.

Una tupla es una colección:

- ordenada,
- accesible por índice,
- inmutable.

Vimos cómo:

- Crear tuplas usando paréntesis `()`.
- Acceder a elementos con índices.
- Usar índices negativos.
- Recorrer tuplas con `for`.
- Usar `enumerate()`.
- Contar elementos con `len()`.
- Verificar si un elemento existe usando `in`.
- Usar desempaquetado.
- Crear correctamente una tupla de un solo elemento.
- Diferenciar cuándo usar listas y cuándo usar tuplas.

Las listas son útiles para datos que cambian, como posts, comentarios o usuarios registrados.

Las tuplas son útiles para datos más estables, como configuraciones, categorías base, versiones o dimensiones.

En el proyecto del blog, las tuplas nos ayudan a proteger información que debería mantenerse fija durante la ejecución del programa.

En la próxima unidad seguiremos organizando mejor los datos del blog y veremos cómo elegir la estructura correcta según el tipo de información que queremos representar.

Sets: colecciones sin elementos repetidos

En esta unidad se introduce el uso de sets en Python como una colección útil para guardar datos únicos. A través del contexto de un blog, se trabajará con etiquetas, categorías y usuarios sin duplicados, aprendiendo a crear sets, agregar elementos, eliminarlos y verificar si un dato existe dentro de la colección.

Sets para guardar datos únicos

En programación, muchas veces necesitamos guardar datos sin repetirlos.

Imagina que en un blog un usuario carga varias etiquetas para un post y escribe dos veces `"Python"`.

No tendría mucho sentido guardar esa etiqueta duplicada. El post habla de Python una vez, no dos veces solo porque la palabra fue cargada dos veces.

Para estos casos, Python nos ofrece una estructura llamada **set**.

1. ¿Qué es un set?

Un **set** es una colección de elementos que no permite valores repetidos.

Podemos pensarlo así:

- Una lista es como una fila donde puede haber elementos repetidos.
- Un set es como una bolsa de elementos únicos: si algo ya existe, no se agrega otra vez.

Ejemplo:

CÓDIGO

```
etiquetas = {"Python", "Django", "Blog", "Python"}

print(etiquetas)
```

Salida posible:

CÓDIGO

```
{'Django', 'Blog', 'Python'}
```

Aunque `"Python"` aparece dos veces en la creación del set, Python lo guarda una sola vez.

La palabra “posible” es importante, porque los sets no garantizan un orden fijo al mostrarse en pantalla.

2. ¿Para qué usar sets en un blog?

En un blog, los sets pueden servir para manejar información donde no queremos duplicados.

Por ejemplo:

- etiquetas únicas de una publicación,
- categorías relacionadas con un post,
- usuarios únicos que dieron like,
- temas que se repiten entre varios artículos,
- palabras clave de búsqueda,
- autores únicos que participaron en una sección.

Ejemplo:

CÓDIGO

```
etiquetas_post = {"Python", "Django", "Backend"}

print(etiquetas_post)
```

Salida posible:

CÓDIGO

```
{'Python', 'Backend', 'Django'}
```

3. Crear un set

Los sets se crean usando llaves `{}`.

CÓDIGO

```
categorias = {"Backend", "Frontend", "DevOps"}

print(categorias)
```

Salida posible:

CÓDIGO

```
{'Frontend', 'Backend', 'DevOps'}
```

También podemos crear sets con números:

CÓDIGO

```
usuarios_que_dieron_like = {101, 205, 318}

print(usuarios_que_dieron_like)
```

Salida posible:

CÓDIGO

```
{205, 318, 101}
```

4. Importante: crear un set vacío

Para crear un set vacío usamos `set()`.

CÓDIGO

```
etiquetas = set()

print(etiquetas)
```

Salida esperada:

CÓDIGO

```
set()
```

Este punto es importante porque `{}` no crea un set vacío.

CÓDIGO

```
datos = {}

print(type(datos))
```

Salida esperada:

CÓDIGO

```
<class 'dict'>
```

Es decir, `{}` crea un diccionario vacío, no un set vacío.

La forma correcta de crear un set vacío es:

CÓDIGO

```
etiquetas = set()
```

5. Agregar elementos con `add()`

Para agregar un elemento a un set usamos `.add()`.

CÓDIGO

```
etiquetas = {"Python", "Django"}

etiquetas.add("Backend")

print(etiquetas)
```

Salida posible:

CÓDIGO

```
{'Python', 'Backend', 'Django'}
```

Ahora el set tiene una nueva etiqueta.

Pero si intentamos agregar una etiqueta que ya existe, no se duplica.

CÓDIGO

```
etiquetas = {"Python", "Django"}

etiquetas.add("Python")

print(etiquetas)
```

Salida posible:

CÓDIGO

```
{'Python', 'Django'}
```

Aunque usamos `add("Python")`, el set sigue teniendo `"Python"` una sola vez.

6. Ejemplo aplicado: etiquetas de un post

Supongamos que estamos cargando etiquetas para un post del blog.

CÓDIGO

```
etiquetas_post = set()

etiquetas_post.add("Python")
etiquetas_post.add("Django")
etiquetas_post.add("Backend")
etiquetas_post.add("Python")

print(etiquetas_post)
```

Salida posible:

CÓDIGO

```
{'Python', 'Django', 'Backend'}
```

Aunque `"Python"` fue agregado dos veces, el set lo conserva una sola vez.

Esto nos ayuda a mantener la información más limpia.

7. Eliminar elementos con `discard()`

Para eliminar un elemento de un set podemos usar `.discard()`.

CÓDIGO

```
etiquetas = {"Python", "Django", "Blog"}

etiquetas.discard("Blog")

print(etiquetas)
```

Salida posible:

CÓDIGO

```
{'Python', 'Django'}
```

`discard()` elimina el elemento indicado si existe.

Si el elemento no existe, no genera error.

CÓDIGO

```
etiquetas = {"Python", "Django", "Blog"}

etiquetas.discard("Frontend")

print(etiquetas)
```

Salida posible:

CÓDIGO

```
{'Python', 'Django', 'Blog'}
```

Como `"Frontend"` no estaba en el set, no se eliminó nada. Pero el programa tampoco se rompió.

8. Diferencia entre `discard()` y `remove()`

También existe `.remove()`.

La diferencia es que `remove()` genera error si el elemento no existe.

CÓDIGO

```
etiquetas = {"Python", "Django"}

etiquetas.remove("Blog")
```

Ese código falla porque `"Blog"` no está dentro del set.

En cambio, con `discard()` no habría error:

CÓDIGO

```
etiquetas = {"Python", "Django"}

etiquetas.discard("Blog")

print(etiquetas)
```

Salida posible:

CÓDIGO

```
{'Python', 'Django'}
```

Por eso, al principio suele ser más cómodo y seguro usar `discard()`.

9. Verificar si un elemento existe con `in`

Podemos saber si un elemento está dentro de un set usando `in`.

CÓDIGO

```
etiquetas = {"Python", "Django", "Backend"}

if "Django" in etiquetas:
    print("Este post habla sobre Django")
```

Salida esperada:

CÓDIGO

```
Este post habla sobre Django
```

También podemos usarlo para validar etiquetas.

CÓDIGO

```
etiquetas_permitidas = {"Python", "Django", "Backend", "Frontend"}

etiqueta = input("Ingresa una etiqueta: ")

if etiqueta in etiquetas_permitidas:
    print("Etiqueta válida")
else:
    print("Etiqueta no permitida")
```

Ejemplo de ejecución:

CÓDIGO

```
Ingresa una etiqueta: Django
Etiqueta válida
```

10. Contar elementos con `len()`

Al igual que con listas y tuplas, podemos usar `len()` para saber cuántos elementos tiene un set.

CÓDIGO

```
etiquetas = {"Python", "Django", "Backend"}

print(len(etiquetas))
```

Salida esperada:

CÓDIGO

```
3
```

Ejemplo aplicado:

CÓDIGO

```
etiquetas_post = {"Python", "Django", "Backend", "Python"}

print(f"El post tiene {len(etiquetas_post)} etiquetas únicas.")
```

Salida esperada:

CÓDIGO

```
El post tiene 3 etiquetas únicas.
```

Aunque `"Python"` aparezca repetido al crear el set, solo cuenta una vez.

11. Recorrer un set

Podemos recorrer un set usando `for`.

CÓDIGO

```
etiquetas = {"Python", "Django", "Backend"}

for etiqueta in etiquetas:
    print(etiqueta)
```

Salida posible:

CÓDIGO

```
Backend
Python
Django
```

El orden puede variar, porque los sets no mantienen un orden fijo como las listas.

Por eso, los sets sirven muy bien para evitar duplicados, pero no son la mejor opción cuando necesitamos mostrar elementos en un orden específico.

12. Los sets no usan índices

Un error común es intentar acceder a un elemento del set usando una posición.

Por ejemplo:

CÓDIGO

```
etiquetas = {"Python", "Django", "Blog"}

print(etiquetas[0])
```

Ese código genera error.

Los sets no permiten acceder por índice.

En una lista sí podemos hacer esto:

CÓDIGO

```
etiquetas = ["Python", "Django", "Blog"]

print(etiquetas[0])
```

Salida esperada:

CÓDIGO

```
Python
```

Pero en un set no, porque no está pensado para trabajar por posiciones.

Si necesitas acceder por posición, probablemente necesitas una lista.

13. Convertir una lista en set para eliminar duplicados

Un uso muy común de los sets es quitar duplicados de una lista.

Supongamos que tenemos esta lista de etiquetas:

CÓDIGO

```
etiquetas = ["Python", "Django", "Python", "Blog", "Django"]

etiquetas_unicas = set(etiquetas)

print(etiquetas_unicas)
```

Salida posible:

CÓDIGO

```
{'Python', 'Blog', 'Django'}
```

La lista tenía valores repetidos, pero al convertirla en set, los duplicados desaparecieron.

Si luego necesitamos volver a tener una lista, podemos usar `list()`.

CÓDIGO

```
etiquetas = ["Python", "Django", "Python", "Blog", "Django"]

etiquetas_unicas = list(set(etiquetas))

print(etiquetas_unicas)
```

Salida posible:

CÓDIGO

```
['Python', 'Blog', 'Django']
```

Importante: al convertir a set y volver a lista, el orden puede cambiar.

14. Ejemplo aplicado: limpiar etiquetas repetidas

Supongamos que un usuario carga etiquetas para un post.

CÓDIGO

```
etiquetas_cargadas = ["Python", "Django", "Python", "Backend", "Django"]

etiquetas_unicas = set(etiquetas_cargadas)

print("Etiquetas cargadas:", etiquetas_cargadas)
print("Etiquetas únicas:", etiquetas_unicas)
```

Salida posible:

CÓDIGO

```
Etiquetas cargadas: ['Python', 'Django', 'Python', 'Backend', 'Django']
Etiquetas únicas: {'Python', 'Backend', 'Django'}
```

Esto es muy útil para mantener datos limpios.

15. Comparar sets

Los sets también permiten comparar colecciones de datos.

Esto es muy útil cuando queremos saber qué elementos se repiten o qué elementos son diferentes.

Por ejemplo, imaginemos dos posts con distintas etiquetas:

CÓDIGO

```
etiquetas_post_1 = {"Python", "Django", "Backend"}
etiquetas_post_2 = {"Python", "Flask", "Backend"}
```

Podemos saber qué etiquetas tienen en común.

CÓDIGO

```
etiquetas_comunes = etiquetas_post_1 & etiquetas_post_2

print(etiquetas_comunes)
```

Salida posible:

CÓDIGO

```
{'Python', 'Backend'}
```

El operador `&` nos da la intersección, es decir, los elementos que aparecen en ambos sets.

16. Unión de sets

La unión nos permite combinar dos sets sin repetir elementos.

CÓDIGO

```
etiquetas_post_1 = {"Python", "Django", "Backend"}
etiquetas_post_2 = {"Python", "Flask", "Backend"}

todas_las_etiquetas = etiquetas_post_1 | etiquetas_post_2

print(todas_las_etiquetas)
```

Salida posible:

CÓDIGO

```
{'Python', 'Django', 'Backend', 'Flask'}
```

El operador `|` combina ambos sets y elimina duplicados automáticamente.

17. Diferencia entre sets

También podemos saber qué elementos están en un set pero no en otro.

CÓDIGO

```
etiquetas_post_1 = {"Python", "Django", "Backend"}
etiquetas_post_2 = {"Python", "Flask", "Backend"}

solo_post_1 = etiquetas_post_1 - etiquetas_post_2

print(solo_post_1)
```

Salida posible:

CÓDIGO

```
{'Django'}
```

Esto significa que `"Django"` está en el primer post, pero no en el segundo.

18. Ejemplo completo: etiquetas del blog

Veamos un ejemplo más completo.

CÓDIGO

```
etiquetas_post_1 = {"Python", "Django", "Backend"}
etiquetas_post_2 = {"Python", "Flask", "Backend"}
etiquetas_post_3 = {"Django", "Web", "Python"}

todas_las_etiquetas = etiquetas_post_1 | etiquetas_post_2 | etiquetas_post_3

print("----- ETIQUETAS DEL BLOG -----")

for etiqueta in todas_las_etiquetas:
    print(etiqueta)

print(f"Total de etiquetas únicas: {len(todas_las_etiquetas)}")
```

Salida posible:

CÓDIGO

```
----- ETIQUETAS DEL BLOG -----
Python
Flask
Backend
Web
Django
Total de etiquetas únicas: 5
```

Este ejemplo muestra cómo podemos combinar etiquetas de varios posts y obtener una colección final sin duplicados.

19. ¿Cuándo usar lista y cuándo usar set?

Usa una **lista** cuando necesites:

- mantener el orden,
- acceder por posición,
- permitir elementos repetidos,
- agregar elementos en una secuencia específica.

Ejemplos:

CÓDIGO

```
comentarios = ["Muy bueno", "Gracias", "Muy bueno"]
posts = ["Bienvenida", "Variables", "Listas"]
pasos_tutorial = ["Instalar Python", "Crear archivo", "Ejecutar programa"]
```

En una lista, tiene sentido que pueda haber elementos repetidos.

Por ejemplo, dos usuarios podrían escribir el mismo comentario.

Usa un **set** cuando necesites:

- evitar duplicados,
- verificar rápidamente si un elemento existe,
- combinar colecciones sin repetir datos,
- obtener elementos únicos.

Ejemplos:

CÓDIGO

```
etiquetas = {"Python", "Django", "Backend"}
usuarios_que_dieron_like = {101, 205, 318}
categorias_relacionadas = {"Web", "Backend", "Python"}
```

20. Errores comunes

Error 1: intentar acceder por índice

Incorrecto:

CÓDIGO

```
etiquetas = {"Python", "Django", "Blog"}

print(etiquetas[0])
```

Los sets no se acceden por posición.

Si necesitas usar posiciones, usa una lista.

Error 2: creer que los sets mantienen el orden

CÓDIGO

```
etiquetas = {"Python", "Django", "Blog"}

print(etiquetas)
```

Salida posible:

CÓDIGO

```
{'Blog', 'Python', 'Django'}
```

Otra ejecución podría mostrar otro orden.

Por eso, no conviene usar sets cuando el orden sea importante.

Error 3: crear un set vacío usando `{}`

Incorrecto si queremos un set:

CÓDIGO

```
etiquetas = {}

print(type(etiquetas))
```

Salida esperada:

CÓDIGO

```
<class 'dict'>
```

Correcto:

CÓDIGO

```
etiquetas = set()

print(type(etiquetas))
```

Salida esperada:

CÓDIGO

```
<class 'set'>
```

Error 4: usar `remove()` cuando no sabemos si el dato existe

Este código puede fallar:

CÓDIGO

```
etiquetas = {"Python", "Django"}

etiquetas.remove("Blog")
```

Una opción más segura es usar `discard()`:

CÓDIGO

```
etiquetas = {"Python", "Django"}

etiquetas.discard("Blog")

print(etiquetas)
```

Error 5: intentar guardar listas dentro de un set

Los sets solo pueden guardar elementos que no cambian internamente.

Por eso, esto da error:

CÓDIGO

```
datos = {["Python", "Django"], ["Backend", "Web"]}
```

Las listas son mutables, entonces no pueden estar dentro de un set.

En este curso, vamos a usar sets principalmente con textos y números simples.

21. Mini práctica guiada

Crea un archivo llamado `etiquetas_blog.py`.

Dentro del archivo:

1. Crea un set vacío llamado `etiquetas`.
2. Agrega al menos cuatro etiquetas usando `.add()`.
3. Intenta agregar una etiqueta repetida.
4. Muestra el set en consola.
5. Muestra la cantidad de etiquetas únicas usando `len()`.
6. Verifica si `"Python"` está dentro del set usando `in`.
7. Elimina una etiqueta usando `.discard()`.

Ejemplo posible:

CÓDIGO

```
etiquetas = set()

etiquetas.add("Python")
etiquetas.add("Django")
etiquetas.add("Backend")
etiquetas.add("Blog")
etiquetas.add("Python")

print("Etiquetas:", etiquetas)
print(f"Cantidad de etiquetas únicas: {len(etiquetas)}")

if "Python" in etiquetas:
    print("El post tiene la etiqueta Python")

etiquetas.discard("Blog")

print("Etiquetas actualizadas:", etiquetas)
```

Salida posible:

CÓDIGO

```
Etiquetas: {'Python', 'Backend', 'Django', 'Blog'}
Cantidad de etiquetas únicas: 4
El post tiene la etiqueta Python
Etiquetas actualizadas: {'Python', 'Backend', 'Django'}
```

22. Desafío extra

Crea dos sets:

- `etiquetas_post_1`
- `etiquetas_post_2`

Cada uno debe tener etiquetas de un post distinto.

Luego muestra:

- todas las etiquetas combinadas,
- las etiquetas que tienen en común,
- las etiquetas que están solo en el primer post.

Ejemplo posible:

CÓDIGO

```
etiquetas_post_1 = {"Python", "Django", "Backend"}
etiquetas_post_2 = {"Python", "Flask", "Backend"}

todas = etiquetas_post_1 | etiquetas_post_2
comunes = etiquetas_post_1 & etiquetas_post_2
solo_post_1 = etiquetas_post_1 - etiquetas_post_2

print("Todas las etiquetas:", todas)
print("Etiquetas en común:", comunes)
print("Solo en el post 1:", solo_post_1)
```

Salida posible:

CÓDIGO

```
Todas las etiquetas: {'Python', 'Django', 'Backend', 'Flask'}
Etiquetas en común: {'Python', 'Backend'}
Solo en el post 1: {'Django'}
```

23. Síntesis de la unidad

En esta unidad aprendiste a usar sets para guardar datos únicos.

Un set es una colección que:

- no permite elementos repetidos,

- no garantiza un orden fijo,
- no permite acceder por índice,
- permite agregar elementos con `.add()`,
- permite eliminar elementos con `.discard()`,
- permite verificar existencia con `in`.

También vimos que los sets son útiles para:

- etiquetas de posts,
- categorías relacionadas,
- usuarios únicos que dieron like,
- limpiar listas con datos repetidos,
- comparar colecciones de información.

Las listas son mejores cuando necesitamos orden, posiciones o elementos repetidos.

Los sets son mejores cuando necesitamos evitar duplicados y trabajar con elementos únicos.

En el proyecto del blog, los sets nos ayudan a mantener más limpia la información, especialmente en casos como etiquetas, categorías o usuarios que interactuaron con una publicación.

Flashcards de repaso

Unidad de repaso mediante tarjetas para consolidar el uso de colecciones (listas, diccionarios, tuplas, sets) en el contexto de un blog desarrollado con Python.

Repaso: Estructurando información en nuestro blog

A lo largo de este módulo, hemos construido las herramientas fundamentales para organizar los datos de nuestro blog. Hemos pasado de almacenar simples valores aislados a crear estructuras complejas que representan entidades reales, como posts, autores y sus perfiles.

Representar información correctamente no es solo una cuestión técnica; es la base para que un programa sea escalable y fácil de mantener. Si aprendes a elegir la estructura de datos adecuada —ya sea una lista para ordenar posts, un diccionario para detallar a un autor, o un set para gestionar etiquetas únicas—, el resto del desarrollo con Django será mucho más intuitivo.

Cómo utilizar estas tarjetas

Estas tarjetas están diseñadas para forzar el **recuerdo activo (active recall)**. Para obtener el máximo beneficio:

1. **Lee el frente** de la tarjeta y detente.
2. **Formula una respuesta mental** o en voz alta antes de hacer clic para ver el reverso.
3. No busques solo memorizar palabras; intenta visualizar **qué problema resuelve** esa estructura de datos específica en el contexto de nuestro blog.

Si fallas en una, no te preocupes: marca la tarjeta y vuelve a ella más tarde. La repetición espaciada es tu mejor aliada para fijar estos conceptos en la memoria a largo plazo.

01 ¿Qué es una lista en Python y cuál es su utilidad principal?

Una **lista** es una colección ordenada y mutable donde los elementos pueden repetirse. Es ideal para organizar elementos secuenciales, como la lista de publicaciones de un blog.

02 ¿Qué es un diccionario y para qué se usa principalmente?

Un **diccionario** es una estructura de datos basada en pares de **clave-valor**. Se utiliza para agrupar información relacionada de una entidad, como el perfil de un autor (ej: `{'nombre': 'Ana', 'email': 'ana@blog.com'}`).

03 ¿Qué caracteriza a una tupla y cuándo deberías preferirla sobre una lista?

Una **tupla** es una colección **inmutable** (no se puede modificar una vez creada). Se utiliza para almacenar datos que no deben cambiar durante la ejecución del programa, como la configuración de constantes del blog o coordenadas de ubicación.

04 ¿Qué propiedad hace únicos a los sets frente a las listas?

Un **set** es una colección desordenada de elementos **únicos** (no permite duplicados). Es perfecto para gestionar categorías de posts donde no quieres repetir etiquetas, asegurando que cada una sea única.

05 ¿Cuándo es suficiente utilizar variables de tipos simples (int, str, bool) en lugar de colecciones?

Para representar atributos individuales y cambiantes, como el título de un artículo (`titulo_post = "Hola Mundo"`) o la cantidad de visitas (`vistas = 150`).

06 ¿Cómo decidir si necesito una lista o un diccionario para modelar información?

Si necesitas mantener un orden específico y los datos pueden cambiar o repetirse, usa una **lista**. Si necesitas buscar datos rápidamente mediante un identificador único (la clave), usa un **diccionario**.

07 ¿Qué significa que una tupla sea inmutable y por qué es útil?

La **inmutabilidad** significa que no puedes agregar, eliminar o modificar los elementos de la colección tras su creación. Esto ayuda a prevenir errores accidentales en datos que deben ser constantes.

08 ¿Qué sucede si intentas añadir un elemento duplicado a un set?

Debido a que el set no permite duplicados, al intentar añadir una etiqueta que ya existe, Python simplemente ignorará la operación, manteniendo el set intacto.

PRE-ENTREGA DEL PROYECTO

Actividad evaluable · parte del proyecto integrador

Preentrega 2: Estructura de datos del blog lista para crecer

Crear una estructura de datos en Python para representar un blog en memoria, usando diccionarios, listas, tuplas y sets.

Preentrega 2: Modelando la base de datos de tu Blog con Python

¡Es hora de poner manos a la obra!

En esta preentrega, vas a construir la **columna vertebral de tu futuro blog web**: su estructura de datos.

Hasta ahora, aprendiste a usar distintas colecciones de Python para organizar información:

- Diccionarios
- Listas
- Tuplas
- Sets

Ahora vas a combinar todas esas estructuras en un mismo script para representar la información principal de un blog en memoria.

Contexto de la cadena de entregables

En el **Módulo 1**, configuraste tu entorno de trabajo, ejecutaste tus primeros scripts y creaste una ficha básica de un post usando variables simples.

En este **Módulo 2**, el desafío sube de nivel: ya no vamos a guardar la información del blog en variables sueltas, sino en estructuras organizadas.

Esto te permitirá representar:

- Un autor con información detallada.
- Una lista de posts.
- Estados posibles para las publicaciones.
- Etiquetas únicas del blog.
- Datos relacionados entre sí usando estructuras anidadas.

Lo que construyas en este checkpoint será la **base de datos en memoria** de tu blog. En los próximos módulos, vas a empezar a recorrer, buscar, filtrar y validar esta información.

Objetivo de aprendizaje de la preentrega

El objetivo de este checkpoint es consolidar el uso de colecciones en Python para modelar información realista.

Vas a practicar cómo:

- Usar un **diccionario** para representar el perfil de un autor.
- Incluir una **lista dentro de un diccionario** para guardar redes sociales.
- Usar una **tupla** para definir valores fijos, como los estados posibles de un post.
- Usar un **set** para evitar etiquetas repetidas.
- Usar una **lista de diccionarios** para representar varios posts.
- Acceder a datos anidados usando claves e índices.

¿Qué construir?

Debés crear un script de Python que modele la estructura de datos principal de un blog.

Podés trabajar el código en tu editor local, como **VS Code**, y luego entregar el código completo según el formato solicitado por la plataforma.

El archivo puede llamarse:

CÓDIGO

```
estructura_blog.py
```

El programa debe incluir

1. Un diccionario llamado `perfil_autor`

Este diccionario debe representar los datos principales de un autor del blog.

Debe incluir, como mínimo:

- `nombre`
- `bio`
- `especialidad`
- `redes_sociales`

La clave `redes_sociales` debe contener una lista.

Ejemplo:

CÓDIGO

```
perfil_autor = {
    "nombre": "Ana López",
    "bio": "Desarrolladora web y creadora de contenido sobre programación.",
    "especialidad": "Python y Django",
    "redes_sociales": ["@ana_dev", "@ana_python"]
}
```

2. Una tupla llamada `estados_post`

Esta tupla debe contener los estados posibles de una publicación.

Los estados deben ser:

CÓDIGO

```
estados_post = ("borrador", "publicado", "archivado")
```

Usamos una tupla porque estos valores funcionan como datos fijos del sistema. Es decir, no deberían cambiar constantemente mientras el programa se ejecuta.

3. Un set llamado `etiquetas_blog`

Este set debe guardar etiquetas generales relacionadas al blog.

Por ejemplo:

CÓDIGO

```
etiquetas_blog = {"Python", "Django", "Web", "Backend", "Python"}
```

Aunque `"Python"` aparece dos veces, el set lo guarda una sola vez.

Esto sirve para entender que los sets son útiles cuando necesitamos evitar valores duplicados.

4. Una lista llamada `posts`

Esta lista debe contener al menos **3 diccionarios**.

Cada diccionario representa un post del blog.

Cada post debe tener, como mínimo:

- `id`
- `titulo`
- `autor`
- `tags`
- `estado`

Importante: cada post debe usar la variable `perfil_autor` como valor de la clave `"autor"`.

Ejemplo:

CÓDIGO

```
posts = [
    {
        "id": 1,
        "titulo": "Primeros pasos con Python",
        "autor": perfil_autor,
        "tags": ["Python", "Principiantes"],
        "estado": "publicado"
    },
    {
        "id": 2,
        "titulo": "Qué es Django",
        "autor": perfil_autor,
        "tags": ["Python", "Django", "Web"],
        "estado": "borrador"
    },
    {
        "id": 3,
        "titulo": "Organizando datos con diccionarios",
        "autor": perfil_autor,
        "tags": ["Python", "Diccionarios"],
        "estado": "archivado"
    }
]
```

Verificación del acceso a datos

Al final del archivo, usá `print()` para comprobar que podés acceder correctamente a la información guardada.

Debés mostrar en consola:

- El nombre del autor del segundo post.
- La lista de todos los posts guardados.

Ejemplo:

CÓDIGO

```
print("Autor del segundo post:", posts[1]["autor"]["nombre"])
print("Lista de posts guardados:")
print(posts)
```

Recordá que las listas empiezan a contar desde `0`.

Por eso:

CÓDIGO

```
posts[0]
```

representa el primer post.

Y:

CÓDIGO

```
posts[1]
```

representa el segundo post.

Ejemplo completo orientativo

CÓDIGO

```
perfil_autor = {
    "nombre": "Ana López",
    "bio": "Desarrolladora web y creadora de contenido sobre programación.",
    "especialidad": "Python y Django",
    "redes_sociales": ["@ana_dev", "@ana_python"]
}

estados_post = ("borrador", "publicado", "archivado")

etiquetas_blog = {"Python", "Django", "Web", "Backend", "Python"}

posts = [
    {
        "id": 1,
        "titulo": "Primeros pasos con Python",
        "autor": perfil_autor,
        "tags": ["Python", "Principiantes"],
        "estado": "publicado"
    },
    {
        "id": 2,
        "titulo": "Qué es Django",
        "autor": perfil_autor,
        "tags": ["Python", "Django", "Web"],
        "estado": "borrador"
    },
    {
        "id": 3,
        "titulo": "Organizando datos con diccionarios",
        "autor": perfil_autor,
        "tags": ["Python", "Diccionarios"],
        "estado": "archivado"
    }
]

print("Autor del segundo post:", posts[1]["autor"]["nombre"])
print("Lista de posts guardados:")
print(posts)
```

Resultado esperado

Al ejecutar el archivo, deberías ver una salida similar a esta:

CÓDIGO

```
Autor del segundo post: Ana López
Lista de posts guardados:
[{'id': 1, 'titulo': 'Primeros pasos con Python', 'autor': {'nombre': 'Ana López',
'bio': 'Desarrolladora web y creadora de contenido sobre programación.',
'especialidad': 'Python y Django', 'redes_sociales': ['@ana_dev', '@ana_python']},
'tags': ['Python', 'Principiantes'], 'estado': 'publicado'}, {'id': 2, 'titulo':
'Qué es Django', 'autor': {'nombre': 'Ana López', 'bio': 'Desarrolladora web y
creadora de contenido sobre programación.', 'especialidad': 'Python y Django',
'redes_sociales': ['@ana_dev', '@ana_python']}, 'tags': ['Python', 'Django', 'Web'],
'estado': 'borrador'}, {'id': 3, 'titulo': 'Organizando datos con diccionarios',
'autor': {'nombre': 'Ana López', 'bio': 'Desarrolladora web y creadora de contenido
sobre programación.', 'especialidad': 'Python y Django', 'redes_sociales':
['@ana_dev', '@ana_python']}, 'tags': ['Python', 'Diccionarios'], 'estado':
'archivado'}]
```

No hace falta que la salida sea exactamente igual, pero sí debe mostrar correctamente el autor del segundo post y la lista de posts cargados.

Criterios de aceptación

Para considerar este checkpoint como completado, tu código debe cumplir con los siguientes puntos:

- Crear un diccionario llamado `perfil_autor`.
- Incluir dentro de `perfil_autor` una lista de redes sociales.
- Crear una tupla llamada `estados_post`.
- La tupla debe contener los valores `"borrador"`, `"publicado"` y `"archivado"`.
- Crear un set llamado `etiquetas_blog`.
- Incluir etiquetas relacionadas a tecnología.
- Repetir al menos una etiqueta para comprobar que el set elimina duplicados.
- Crear una lista llamada `posts`.
- Incluir al menos 3 diccionarios dentro de la lista `posts`.
- Cada post debe usar `perfil_autor` como valor de la clave `"autor"`.
- Usar `print()` para mostrar el nombre del autor del segundo post.
- Usar `print()` para mostrar la lista completa de posts guardados.

Pasos sugeridos

1. Prepará el archivo

Creá un archivo de Python para escribir tu solución.

Podés llamarlo:

CÓDIGO

```
estructura_blog.py
```

Trabajá el código en tu editor local, como VS Code, y luego entregá el contenido según indique la plataforma.

2. Definí el perfil del autor

Creá el diccionario `perfil_autor`.

Recordá incluir una lista dentro de este diccionario para guardar las redes sociales.

3. Establecé las constantes del sistema

Creá la tupla `estados_post`.

Esta tupla debe contener:

CÓDIGO

```
("borrador", "publicado", "archivado")
```

4. Gestioná las etiquetas

Creá el set `etiquetas_blog`.

Agregá varias etiquetas relacionadas a tecnología e intentá repetir una para verificar el comportamiento de los sets.

5. Construí el catálogo de contenido

Creá la lista `posts`.

Dentro de ella, agregá al menos 3 diccionarios.

Cada diccionario debe representar un post y debe usar la variable `perfil_autor` en la clave `"autor"`.

6. Verificá el acceso a datos

Usá `print()` para mostrar:

CÓDIGO

```
posts[1]["autor"]["nombre"]
```

Ese acceso permite llegar al nombre del autor del segundo post.

Luego, imprimí la lista completa:

CÓDIGO

```
print(posts)
```

Errores comunes a evitar

Olvidar las comas

En listas, diccionarios, tuplas y sets, los elementos deben separarse con comas.

Incorrecto:

CÓDIGO

```
redes_sociales = ["@ana_dev" "@ana_python"]
```

Correcto:

CÓDIGO

```
redes_sociales = ["@ana_dev", "@ana_python"]
```

Confundir paréntesis, corchetes y llaves

Cada estructura usa una sintaxis distinta:

CÓDIGO

```
lista = ["Python", "Django"]
tupla = ("borrador", "publicado", "archivado")
set_etiquetas = {"Python", "Django", "Web"}
diccionario = {"titulo": "Qué es Django"}
```

Intentar acceder a una clave que no existe

Si definiste la clave como `"titulo"`, tenés que acceder a ella exactamente igual.

Correcto:

CÓDIGO

```
posts[0]["titulo"]
```

Incorrecto:

CÓDIGO

```
posts[0]["título"]
```

Python distingue entre `"titulo"` y `"título"`.

Confundir índices de listas

Recordá que las listas empiezan en `0`.

CÓDIGO

```
posts[0]  # Primer post
posts[1]  # Segundo post
posts[2]  # Tercer post
```

Si querés acceder al segundo post, tenés que usar `posts[1]`, no `posts[2]`.

Olvidar que el autor es un diccionario anidado

Como cada post tiene dentro la variable `perfil_autor`, para llegar al nombre del autor necesitás acceder en dos pasos:

CÓDIGO

```
posts[1]["autor"]["nombre"]
```

Primero accedés al segundo post:

CÓDIGO

```
posts[1]
```

Luego al autor:

CÓDIGO

```
posts[1]["autor"]
```

Y finalmente al nombre:

CÓDIGO

```
posts[1]["autor"]["nombre"]
```

Cierre del checkpoint

Este checkpoint no busca construir todavía un sistema interactivo ni un menú de consola.

Su objetivo principal es que puedas **modelar datos correctamente** usando las colecciones principales de Python.

Si esta estructura está bien armada, en el próximo módulo vas a poder recorrer los posts, buscar publicaciones, filtrar por etiquetas y construir un sistema de consola más dinámico.

En otras palabras: en este checkpoint estás creando la **memoria interna del blog**. Más adelante, esa memoria se va a convertir en la lógica principal de tu aplicación.

Pasos para la entrega

¡Es hora de poner manos a la obra! En esta preentrega, vas a construir la columna vertebral de tu futuro blog web: la estructura de datos. Tu objetivo es crear un script que organice toda la información necesaria para que el blog funcione en memoria.

Pasos sugeridos:

1. **Prepara el archivo**: Escribí el código necesario para modelar la estructura de datos del blog. Podés trabajarlo en tu editor local, como VS Code, y luego entregar el código completo según el formato solicitado por la plataforma.
2. **Define los perfiles**: Crea el diccionario `perfil_autor`. Recuerda incluir una lista dentro de este diccionario para las redes sociales.
3. **Establece las constantes**: Crea una tupla llamada `estados_post` que contenga los estados inmutables: `"borrador"`, `"publicado"` y `"archivado"`.
4. **Gestiona etiquetas**: Declara un set llamado `etiquetas_blog`. Agrega varias etiquetas relacionadas a tecnología e intenta repetir una para verificar el comportamiento de los sets.
5. **Construye el catálogo de contenido**: Declara una lista llamada `posts`. Dentro de ella, crea al menos 3 diccionarios. Importante: Cada post debe usar la variable `perfil_autor` como valor para la clave `"autor"`.
6. **Verifica el acceso a datos**: Utiliza `print()` para mostrar en consola el nombre del autor del segundo post y la lista de todos los posts guardados.

Errores comunes a evitar:

- Olvidar las comas entre los elementos de una lista o diccionario.
- Confundir los paréntesis `()` de las tuplas con los corchetes `[]` de las listas.
- Intentar acceder a una clave de diccionario que no existe (ej. escribir `"título"` con tilde si lo definiste como `"titulo"`).

Entregable

Formato: text

Completar con el código generado para la entrega.

Objetivos de aprendizaje

- Implementar diccionarios para modelar entidades complejas como posts y autores.
- Utilizar listas para organizar colecciones de objetos similares.
- Aplicar tuplas y sets para manejar datos inmutables y valores únicos respectivamente.
- Acceder a datos anidados dentro de estructuras de datos complejas.

Criterios de aceptación

- El archivo se llama exactamente estructura_blog.py.
- Se incluye un diccionario para el autor que contiene al menos una lista anidada para redes sociales.
- Se utiliza una tupla para definir los estados posibles del post.
- Se utiliza un set para almacenar las etiquetas únicas del blog.
- La lista 'posts' contiene al menos 3 diccionarios, cada uno con las claves: id, titulo, autor, categoria, tags y estado.
- El script se ejecuta sin errores de sintaxis en el intérprete de Python.
- Se incluyen prints que demuestran el acceso exitoso a datos anidados (ej. nombre del autor dentro de un post).

Rúbrica de evaluación

Evalúa la capacidad del estudiante para modelar en Python la estructura de datos básica de un blog usando variables, tipos de datos y colecciones (diccionarios, listas, tuplas y sets), organizando la información de forma coherente en un script funcional entregado como texto según la consigna.

Criterio	Descripción	Peso
Modelado de la estructura de datos del blog	Evalúa si el archivo construye correctamente las colecciones solicitadas y si representan con claridad la información del blog: perfil del autor, estados de publicación, etiquetas y lista de posts.	35%
Coherencia y reutilización de datos entre posts	Evalúa si los posts están modelados de forma consistente y si reutilizan correctamente el perfil_autor como valor de la clave autor en cada diccionario, manteniendo relaciones de datos claras y evitando duplicaciones innecesarias.	20%
Uso correcto de tipos de datos y sintaxis básica de Python	Evalúa el manejo adecuado de listas, diccionarios, tuplas, sets, claves de acceso y sintaxis básica necesaria para que el script sea correcto y ejecutable.	25%
Salida por consola y verificación de acceso a datos	Evalúa si el script muestra en consola lo pedido: el nombre del autor del segundo post y la lista completa de posts, verificando que el estudiante sabe acceder a elementos anidados y visualizar la estructura creada.	20%

Total: 100 pts | Aprobación: 70 pts

MÓDULO 3

Condicionales e iteraciones

Construir la lógica de control del blog por consola usando estructuras condicionales y ciclos de repetición.

Decidir qué hacer con if, elif y else

Aprende a dotar de lógica a tus programas en Python utilizando estructuras condicionales. Descubre cómo el programa puede tomar diferentes caminos según los datos, validando entradas del usuario y controlando el acceso a funciones del blog.

En programación, muchas veces necesitamos que nuestro programa tome decisiones.

Por ejemplo, en un blog:

- Si el usuario inició sesión, puede escribir un post.
- Si no inició sesión, debe ver un mensaje para ingresar.
- Si el usuario es el autor de un post, puede editarlo.
- Si no es el autor, no debería tener permiso.

Para resolver este tipo de situaciones usamos estructuras condicionales.

En Python, las principales son:

CÓDIGO

```
if
elif
else
```

Estas estructuras nos permiten ejecutar distintos bloques de código según se cumpla o no una condición.

1. Valores booleanos: `True` y `False`

Antes de tomar decisiones, Python necesita evaluar si algo es verdadero o falso.

Para eso existen los valores booleanos:

CÓDIGO

```
True
False
```

En Python se escriben siempre con la primera letra en mayúscula.

Ejemplo:

CÓDIGO

```
usuario_logueado = True

print(usuario_logueado)
```

Salida:

CÓDIGO

```
True
```

Un booleano puede servir para representar estados simples dentro de nuestro programa.

Por ejemplo:

CÓDIGO

```
post_visible = False
es_autor = True
tiene_permiso = True
```

2. Comparadores

Muchas veces obtenemos `True` o `False` comparando valores.

Operador	Significado	Ejemplo	Resultado
==	Igual a	5 == 5	True
!=	Distinto de	5 != 3	True
>	Mayor que	10 > 20	False
\<	Menor que	8 \< 12	True
>=	Mayor o igual que	10 >= 10	True
\<=	Menor o igual que	5 \<= 2	False

Ejemplo aplicado al blog:

CÓDIGO

```
likes = 25

print(likes > 10)
print(likes == 0)
print(likes != 100)
```

Salida:

CÓDIGO

```
True
False
True
```

3. Cuidado con = y ==

Este es uno de los errores más comunes al empezar.

Un solo igual `=` sirve para asignar un valor:

CÓDIGO

```
nombre = "Ana"
```

Doble igual == sirve para comparar:

CÓDIGO

```
nombre == "Ana"
```

Ejemplo:

CÓDIGO

```
autor = "Ana"

print(autor == "Ana")
print(autor == "Juan")
```

Salida:

CÓDIGO

```
True
False
```

4. Usar `if`

La estructura `if` permite ejecutar código solo si una condición se cumple.

CÓDIGO

```
usuario_logueado = True

if usuario_logueado:
    print("Bienvenido al panel del blog.")
    print("Ya puedes escribir un nuevo post.")
```

Como `usuario_logueado` vale `True`, el bloque se ejecuta.

Salida:

CÓDIGO

```
Bienvenido al panel del blog.
Ya puedes escribir un nuevo post.
```

Ahora veamos qué pasa si vale `False`:

CÓDIGO

```
usuario_logueado = False

if usuario_logueado:
    print("Bienvenido al panel del blog.")
```

En este caso no se imprime nada, porque la condición no se cumple.

5. La importancia de la indentación

En Python, la indentación no es decorativa: define qué instrucciones pertenecen a cada bloque.

Correcto:

CÓDIGO

```
usuario_logueado = True

if usuario_logueado:
    print("Usuario autorizado.")
    print("Mostrando panel del blog.")
```

Incorrecto:

CÓDIGO

```
usuario_logueado = True

if usuario_logueado:
print("Usuario autorizado.")
```

Ese código produce un error, porque Python espera que el contenido del `if` esté indentado.

6. Usar `if` con comparaciones

Podemos usar `if` junto con comparadores.

CÓDIGO

```
edad = 20

if edad >= 18:
    print("Puedes publicar comentarios.")
```

Salida:

CÓDIGO

```
Puedes publicar comentarios.
```

Otro ejemplo:

CÓDIGO

```
cantidad_posts = 0

if cantidad_posts == 0:
    print("Todavía no hay posts publicados.")
```

Salida:

CÓDIGO

```
Todavía no hay posts publicados.
```

7. Usar `if-else`

A veces queremos hacer una cosa si la condición se cumple y otra distinta si no se cumple.

Para eso usamos `else`.

CÓDIGO

```
edad = 15

if edad >= 18:
    print("Puedes publicar comentarios.")
else:
    print("Debes ser mayor de edad para comentar.")
```

Salida:

CÓDIGO

```
Debes ser mayor de edad para comentar.
```

Ejemplo aplicado a un blog:

CÓDIGO

```
usuario_logueado = False

if usuario_logueado:
    print("Mostrar botón: Escribir post")
else:
    print("Mostrar botón: Iniciar sesión")
```

Salida:

CÓDIGO

```
Mostrar botón: Iniciar sesión
```

8. Usar `if-elif-else`

Cuando tenemos más de dos opciones, usamos `elif`.

`elif` significa “si no se cumplió lo anterior, probá esta otra condición”.

Ejemplo:

CÓDIGO

```
accion = "buscar"

if accion == "listar":
    print("Mostrando todos los posts...")
elif accion == "buscar":
    print("Buscando posts...")
elif accion == "salir":
    print("Cerrando el programa.")
else:
    print("Opción no reconocida.")
```

Salida:

CÓDIGO

```
Buscando posts...
```

Este tipo de estructura es muy útil para armar menús por consola.

9. Ejemplo: menú simple del blog

CÓDIGO

```
print("Menú del blog")
print("1 - Listar posts")
print("2 - Crear post")
print("3 - Salir")

opcion = input("Elegí una opción: ")

if opcion == "1":
    print("Mostrando posts...")
elif opcion == "2":
    print("Creando nuevo post...")
elif opcion == "3":
    print("Saliendo del programa...")
else:
    print("Opción inválida.")
```

Ejemplo de ejecución:

CÓDIGO

```
Menú del blog
1 - Listar posts
2 - Crear post
3 - Salir
Elegí una opción: 2
Creando nuevo post...
```

10. El orden de las condiciones importa

Python evalúa las condiciones de arriba hacia abajo.

Cuando encuentra una condición verdadera, ejecuta ese bloque y se salta el resto.

Ejemplo incorrecto:

CÓDIGO

```
likes = 150

if likes > 10:
    print("Post popular")
elif likes > 100:
    print("Post viral")
```

Salida:

CÓDIGO

```
Post popular
```

Aunque el post tiene más de 100 likes, Python nunca llega al segundo caso porque `likes > 10` ya era verdadero.

La versión correcta sería:

CÓDIGO

```
likes = 150

if likes > 100:
    print("Post viral")
elif likes > 10:
    print("Post popular")
else:
    print("Post nuevo")
```

Salida:

CÓDIGO

```
Post viral
```

Regla práctica: cuando varias condiciones pueden superponerse, conviene poner primero la condición más específica.

11. Operadores lógicos: `and`, `or` y `not`

A veces una decisión depende de más de una condición.

Para eso usamos operadores lógicos.

`and`

Con `and`, todas las condiciones deben ser verdaderas.

CÓDIGO

```
usuario_logueado = True
es_autor = True

if usuario_logueado and es_autor:
    print("Puedes editar este post.")
else:
    print("No tienes permiso para editar este post.")
```

Salida:

CÓDIGO

```
Puedes editar este post.
```

`or`

Con `or`, alcanza con que una condición sea verdadera.

CÓDIGO

```
es_autor = False
es_admin = True

if es_autor or es_admin:
    print("Puedes eliminar este post.")
else:
    print("No tienes permiso para eliminar este post.")
```

Salida:

CÓDIGO

```
Puedes eliminar este post.
```

`not`

Con `not`, invertimos el valor de una condición.

CÓDIGO

```
post_visible = False

if not post_visible:
    print("Este post todavía no está visible.")
```

Salida:

CÓDIGO

```
Este post todavía no está visible.
```

12. Normalizar texto con `lower()` y `strip()`

Cuando usamos `input()`, el usuario puede escribir de muchas formas:

CÓDIGO

```
"LISTAR"
"Listar"
"listar"
" listar "
```

Para Python, esos textos no son iguales.

Por eso conviene normalizar la entrada.

CÓDIGO

```
accion = input("¿Qué querés hacer? ")

accion = accion.lower().strip()

if accion == "listar":
    print("Mostrando posts...")
elif accion == "crear":
    print("Creando post...")
elif accion == "salir":
    print("Saliendo...")
else:
    print("Acción no válida.")
```

`lower()` convierte el texto a minúsculas.

`strip()` elimina espacios al principio y al final.

Entonces, si el usuario escribe:

CÓDIGO

```
" LISTAR "
```

Python lo transforma en:

CÓDIGO

```
"listar"
```

Esto hace que nuestro programa sea más flexible y menos frágil.

13. Ejemplo completo: permisos en un blog

CÓDIGO

```
usuario_logueado = True
es_autor = False
es_admin = True
post_visible = True

if not usuario_logueado:
    print("Debes iniciar sesión.")
elif es_autor or es_admin:
    print("Puedes editar el post.")
elif post_visible:
    print("Puedes leer el post, pero no editarlo.")
else:
    print("No tienes permisos sobre este post.")
```

Salida:

CÓDIGO

```
Puedes editar el post.
```

En este ejemplo, el usuario no es el autor, pero sí es administrador. Por eso tiene permiso para editar.

14. Ejemplo completo: estado de un post según sus likes

CÓDIGO

```
likes = int(input("Ingresá la cantidad de likes del post: "))

if likes >= 100:
    print("Este post es viral.")
elif likes >= 10:
    print("Este post es popular.")
elif likes > 0:
    print("Este post está empezando a recibir atención.")
else:
    print("Este post todavía no tiene likes.")
```

Ejemplo de ejecución:

CÓDIGO

```
Ingresá la cantidad de likes del post: 35
Este post es popular.
```

15. Errores comunes

Confundir asignación con comparación

Incorrecto:

CÓDIGO

```
nombre = "Ana"

if nombre = "Ana":
    print("Es la autora.")
```

Correcto:

CÓDIGO

```
nombre = "Ana"

if nombre == "Ana":
    print("Es la autora.")
```

Olvidar la indentación

Incorrecto:

CÓDIGO

```
esta_logueado = True

if esta_logueado:
print("Bienvenido.")
```

Correcto:

CÓDIGO

```
esta_logueado = True

if esta_logueado:
    print("Bienvenido.")
```

No usar `else`

Si no usamos `else`, el programa puede quedarse sin responder ante opciones inválidas.

Menos recomendable:

CÓDIGO

```
opcion = input("Elegí una opción: ")

if opcion == "1":
    print("Listando posts...")
elif opcion == "2":
    print("Creando post...")
```

Mejor:

CÓDIGO

```
opcion = input("Elegí una opción: ")

if opcion == "1":
    print("Listando posts...")
elif opcion == "2":
    print("Creando post...")
else:
    print("Opción inválida.")
```

Resumen de la unidad

En esta unidad aprendiste que:

- Los programas pueden tomar decisiones usando condiciones.
- Los valores booleanos son `True` y `False`.
- Los comparadores permiten crear condiciones.
- `if` ejecuta código si una condición se cumple.
- `else` permite definir qué hacer si la condición no se cumple.
- `elif` permite evaluar varias alternativas.
- La indentación define qué código pertenece a cada bloque.
- `and`, `or` y `not` permiten combinar condiciones.
- `lower()` y `strip()` ayudan a comparar texto ingresado por usuarios.
- El orden de las condiciones puede cambiar el resultado del programa.

Estas estructuras son fundamentales para cualquier programa real. Más adelante, en Django, vas a usar esta misma lógica para validar formularios, controlar permisos, mostrar botones según el usuario y decidir qué contenido debe ver cada persona.

Recorrer posts con for y range

Aprende a usar bucles for para procesar listas de posts, numerarlos automáticamente y gestionar colecciones de datos sin repetir código.

En el mundo real, un blog no tiene solamente 2 o 3 publicaciones fijas. Puede tener decenas, cientos o miles de posts.

Imagina que quieres mostrar en pantalla todos los títulos de un blog. ¿Escribirías un `print()` por cada publicación?

CÓDIGO

```
print("Mi primer post")
print("Guía de Python")
print("Django básico")
```

Eso podría funcionar con pocos datos, pero no sería práctico si la lista crece.

Para repetir una acción sobre varios elementos usamos la iteración.

¿Qué es iterar?

Iterar significa repetir una acción varias veces.

En programación, muchas veces iteramos sobre colecciones de datos, como listas, diccionarios, tuplas o sets.

Por ejemplo, si tenemos una lista de posts, podemos recorrerla y mostrar cada título sin escribir un `print()` manual para cada uno.

El bucle `for`

En Python, una de las formas más comunes de iterar es usando el bucle `for`.

Se puede leer casi como una frase:

> **Nota**
> Para cada post en la lista de posts, hacer algo.

Ejemplo:

CÓDIGO

```
posts = ["Mi primer post", "Guía de Python", "Django básico"]

for post in posts:
    print(f"Leyendo: {post}")
```

Salida:

CÓDIGO

```
Leyendo: Mi primer post
Leyendo: Guía de Python
Leyendo: Django básico
```

En este ejemplo:

- `posts` es la lista completa.
- `post` es una variable temporal.
- En cada vuelta, `post` toma el valor de un elemento distinto de la lista.

Primera vuelta:

CÓDIGO

```
post = "Mi primer post"
```

Segunda vuelta:

CÓDIGO

```
post = "Guía de Python"
```

Tercera vuelta:

CÓDIGO

```
post = "Django básico"
```

Cuando ya no quedan elementos, el bucle termina.

La importancia de la indentación

Al igual que con `if`, la indentación también es obligatoria en los bucles.

Correcto:

CÓDIGO

```
posts = ["Python", "Django", "HTML"]

for post in posts:
    print(post)
```

Incorrecto:

CÓDIGO

```
posts = ["Python", "Django", "HTML"]

for post in posts:
print(post)
```

El segundo ejemplo produce un error porque Python espera que el contenido del `for` esté indentado.

Todo lo que esté dentro del bloque indentado se repetirá.

CÓDIGO

```
posts = ["Python", "Django", "HTML"]

for post in posts:
    print("Título del post:")
    print(post)
```

Salida:

CÓDIGO

```
Título del post:
Python
Título del post:
Django
Título del post:
HTML
```

Recorrer una lista de posts

Supongamos que tenemos varios títulos guardados en una lista:

CÓDIGO

```
titulos = [
    "Introducción a Python",
    "Variables y tipos de datos",
    "Listas en Python",
    "Condicionales con if"
]

for titulo in titulos:
    print(titulo)
```

Salida:

CÓDIGO

```
Introducción a Python
Variables y tipos de datos
Listas en Python
Condicionales con if
```

Este patrón será muy común en nuestros programas: tener una colección de datos y recorrerla para mostrar información.

Usar `range()`

La función `range()` sirve para generar una secuencia de números.

Por ejemplo:

CÓDIGO

```
for numero in range(5):
    print(numero)
```

Salida:

CÓDIGO

```
0
1
2
3
4
```

Aunque escribimos `range(5)`, la secuencia no llega hasta 5. Empieza en `0` y termina en `4`.

Esto ocurre porque en Python muchas estructuras empiezan a contar desde `0`.

Usar `range()` para repetir una acción

`range()` es útil cuando queremos repetir algo una cantidad exacta de veces.

CÓDIGO

```
for vuelta in range(3):
    print("Bienvenido al blog")
```

Salida:

CÓDIGO

```
Bienvenido al blog
Bienvenido al blog
Bienvenido al blog
```

En este caso, el mensaje se imprime 3 veces.

`range()` con inicio y fin

También podemos indicar desde qué número empezar y dónde terminar.

CÓDIGO

```
for numero in range(1, 6):
    print(numero)
```

Salida:

CÓDIGO

```
1
2
3
4
5
```

El primer valor indica el inicio.

El segundo valor indica hasta dónde llegar, pero sin incluir ese número.

Por eso `range(1, 6)` genera los números del 1 al 5.

Ejemplo aplicado al blog

Podemos usar `range()` para mostrar una numeración simple:

CÓDIGO

```
for numero in range(1, 4):
    print(f"Post número {numero}")
```

Salida:

CÓDIGO

```
Post número 1
Post número 2
Post número 3
```

Esto nos permite repetir una acción varias veces sin tener todavía una lista real de posts.

Recorrer una lista usando índices

Las listas tienen posiciones. La primera posición es `0`, la segunda es `1`, la tercera es `2`, y así sucesivamente.

CÓDIGO

```
posts = ["Mi primer post", "Guía de Python", "Django básico"]

print(posts[0])
print(posts[1])
print(posts[2])
```

Salida:

CÓDIGO

```
Mi primer post
Guía de Python
Django básico
```

También podríamos recorrer la lista usando `range()` y los índices:

CÓDIGO

```
posts = ["Mi primer post", "Guía de Python", "Django básico"]

for indice in range(len(posts)):
    print(posts[indice])
```

Salida:

CÓDIGO

```
Mi primer post
Guía de Python
Django básico
```

En este ejemplo:

CÓDIGO

```
len(posts)
```

devuelve la cantidad de elementos de la lista.

Como hay 3 posts, `range(len(posts))` equivale a:

CÓDIGO

```
range(3)
```

Es decir:

CÓDIGO

```
0, 1, 2
```

Luego usamos esos números como índices para acceder a cada elemento.

Mejor opción: usar `enumerate()`

Aunque se puede recorrer una lista con `range(len(lista))`, en Python suele ser más claro usar `enumerate()` cuando necesitamos el elemento y su número.

Ejemplo:

CÓDIGO

```
posts = ["Mi primer post", "Guía de Python", "Django básico"]

for indice, post in enumerate(posts):
    print(f"{indice}: {post}")
```

Salida:

CÓDIGO

```
0: Mi primer post
1: Guía de Python
2: Django básico
```

Si queremos que la numeración empiece en 1, podemos usar `start=1`.

CÓDIGO

```
posts = ["Mi primer post", "Guía de Python", "Django básico"]

for numero, post in enumerate(posts, start=1):
    print(f"Post número {numero}: {post}")
```

Salida:

CÓDIGO

```
Post número 1: Mi primer post
Post número 2: Guía de Python
Post número 3: Django básico
```

Para un blog, esta versión es más amigable porque los usuarios no suelen contar desde cero.

Recorrer una lista de diccionarios

Hasta ahora recorrimos listas de textos, pero en un blog real un post tiene más información: título, autor, cantidad de likes, estado, etc.

Podemos representar cada post con un diccionario.

CÓDIGO

```
lista_posts = [
    {"titulo": "Post A", "autor": "Ana", "likes": 25},
    {"titulo": "Post B", "autor": "Beto", "likes": 8},
    {"titulo": "Post C", "autor": "Carla", "likes": 40}
]

for post in lista_posts:
    print(post["titulo"])
```

Salida:

CÓDIGO

```
Post A
Post B
Post C
```

En cada vuelta, la variable `post` contiene un diccionario completo.

Por eso podemos acceder a sus claves:

CÓDIGO

```
post["titulo"]
post["autor"]
post["likes"]
```

Mostrar información completa de cada post

CÓDIGO

```
lista_posts = [
    {"titulo": "Post A", "autor": "Ana", "likes": 25},
    {"titulo": "Post B", "autor": "Beto", "likes": 8},
    {"titulo": "Post C", "autor": "Carla", "likes": 40}
]

for post in lista_posts:
    print(f"Título: {post['titulo']}")
    print(f"Autor: {post['autor']}")
    print(f"Likes: {post['likes']}")
    print("---------------")
```

Salida:

CÓDIGO

```
Título: Post A
Autor: Ana
Likes: 25
---------------
Título: Post B
Autor: Beto
Likes: 8
---------------
Título: Post C
Autor: Carla
Likes: 40
---------------
```

Este tipo de recorrido se parece mucho a lo que más adelante haremos en Django cuando mostremos publicaciones en una página HTML.

Combinar `for` con `if`

También podemos usar condicionales dentro de un bucle.

Por ejemplo, mostrar solo los posts que tienen más de 10 likes:

CÓDIGO

```
lista_posts = [
    {"titulo": "Post A", "autor": "Ana", "likes": 25},
    {"titulo": "Post B", "autor": "Beto", "likes": 8},
    {"titulo": "Post C", "autor": "Carla", "likes": 40}
]

for post in lista_posts:
    if post["likes"] > 10:
        print(f"{post['titulo']} es un post popular.")
```

Salida:

CÓDIGO

```
Post A es un post popular.
Post C es un post popular.
```

El `for` recorre todos los posts.

El `if` decide qué hacer con cada uno.

Ejemplo completo: listado de posts del blog

CÓDIGO

```
posts = [
    {"titulo": "Introducción a Python", "autor": "Ana", "estado": "publicado"},
    {"titulo": "Django básico", "autor": "Beto", "estado": "publicado"},
    {"titulo": "Mi borrador secreto", "autor": "Carla", "estado": "borrador"}
]

print("Listado de posts publicados")
print("---------------------------")

for numero, post in enumerate(posts, start=1):
    if post["estado"] == "publicado":
        print(f"{numero}. {post['titulo']} - Autor: {post['autor']}")
```

Salida:

CÓDIGO

```
Listado de posts publicados
---------------------------
1. Introducción a Python - Autor: Ana
2. Django básico - Autor: Beto
```

El tercer post no se muestra porque su estado es `"borrador"`.

Errores comunes

Confundir el elemento con el índice

Si hacemos esto:

CÓDIGO

```
posts = ["Python", "Django", "HTML"]

for post in posts:
    print(post)
```

La variable `post` representa el elemento actual, no su posición.

Es decir, `post` vale `"Python"`, luego `"Django"` y luego `"HTML"`.

No vale `0`, `1` ni `2`.

Si necesitas la posición, usa `enumerate()`:

CÓDIGO

```
for indice, post in enumerate(posts):
    print(indice, post)
```

Olvidar los dos puntos

Incorrecto:

CÓDIGO

```
for post in posts
    print(post)
```

Correcto:

CÓDIGO

```
for post in posts:
    print(post)
```

Después de la línea del `for`, siempre van dos puntos `:`.

Olvidar la indentación

Incorrecto:

CÓDIGO

```
for post in posts:
print(post)
```

Correcto:

CÓDIGO

```
for post in posts:
    print(post)
```

Todo lo que queremos repetir debe estar indentado.

Usar `range()` cuando no hace falta

Esto funciona:

CÓDIGO

```
posts = ["Python", "Django", "HTML"]

for indice in range(len(posts)):
    print(posts[indice])
```

Pero esta versión es más simple:

CÓDIGO

```
posts = ["Python", "Django", "HTML"]

for post in posts:
    print(post)
```

Regla práctica:

- Si solo necesitas el contenido, usa `for elemento in lista`.
- Si necesitas contenido y número, usa `enumerate()`.
- Si solo necesitas repetir una acción varias veces, usa `range()`.

Resumen de la unidad

En esta unidad aprendiste que:

- Iterar significa repetir una acción sobre varios elementos.
- El bucle `for` permite recorrer colecciones.
- `for elemento in coleccion:` es la estructura básica.
- La variable del `for` cambia de valor en cada vuelta.
- `range()` permite generar secuencias de números.
- `range()` es útil para repetir acciones una cantidad determinada de veces.
- `len()` permite saber cuántos elementos tiene una lista.
- `enumerate()` permite obtener el número y el elemento al mismo tiempo.

- Podemos recorrer listas de textos y listas de diccionarios.
- También podemos combinar `for` con `if` para filtrar datos.

En un blog, los bucles nos permiten mostrar listas de publicaciones, recorrer autores, numerar posts, filtrar contenido publicado y preparar la base de lo que más adelante veremos en Django al generar páginas dinámicas.

Construir un menú interactivo con while

Aprende a diseñar y programar un menú interactivo por consola utilizando el bucle while en Python. Descubrirás cómo mantener tu programa en ejecución, capturar decisiones del usuario y validar opciones para crear una experiencia de usuario robusta en tu blog.

Hasta ahora, muchos de nuestros programas se ejecutaban de arriba hacia abajo y terminaban al llegar a la última línea.

Pero una aplicación real no suele funcionar así.

Imagina un blog por consola. El usuario debería poder elegir qué hacer:

- Ver posts.
- Crear un post.
- Buscar un post.
- Salir del programa.

Después de elegir una opción, el programa no debería cerrarse automáticamente. Debería volver a mostrar el menú hasta que el usuario decida salir.

Para lograr eso usamos el bucle `while`.

¿Qué es un bucle `while`?

La palabra `while` significa “mientras”.

En Python, un bucle `while` repite un bloque de código mientras una condición sea verdadera.

Ejemplo básico:

CÓDIGO

```
contador = 1

while contador <= 3:
    print("Vuelta número", contador)
    contador = contador + 1
```

Salida:

CÓDIGO

```
Vuelta número 1
Vuelta número 2
Vuelta número 3
```

En este ejemplo:

- `contador` empieza valiendo `1`.
- El `while` se repite mientras `contador <= 3`.
- En cada vuelta, aumentamos el valor de `contador`.
- Cuando `contador` llega a `4`, la condición deja de cumplirse y el bucle termina.

Diferencia entre `for` y `while`

Ya vimos que el bucle `for` sirve para recorrer colecciones.

Por ejemplo:

CÓDIGO

```
posts = ["Post A", "Post B", "Post C"]

for post in posts:
    print(post)
```

Usamos `for` cuando sabemos que queremos recorrer una colección completa.

En cambio, usamos `while` cuando no sabemos exactamente cuántas veces se va a repetir algo.

Por ejemplo:

- No sabemos cuántas veces el usuario va a usar el menú.
- No sabemos cuándo va a elegir salir.
- No sabemos cuántas acciones va a realizar antes de cerrar el programa.

Por eso, `while` es ideal para construir menús interactivos.

Cuidado con los bucles infinitos

Un error común es crear un bucle cuya condición nunca deja de ser verdadera.

Ejemplo problemático:

CÓDIGO

```
contador = 1

while contador <= 3:
    print("Esto se repite...")
```

Este programa no termina nunca, porque `contador` siempre vale `1`.

La condición `contador <= 3` siempre será verdadera.

Para evitarlo, necesitamos modificar algo dentro del bucle:

CÓDIGO

```
contador = 1

while contador <= 3:
    print("Esto se repite...")
    contador = contador + 1
```

Ahora el bucle sí termina.

Si alguna vez tu programa queda atrapado en un bucle infinito, normalmente puedes detenerlo desde la terminal con:

CÓDIGO

```
Ctrl + C
```

Usar `while` con una opción de salida

Podemos usar una variable para controlar si el programa sigue funcionando.

CÓDIGO

```
programa_activo = True

while programa_activo:
    print("Menú del blog")
    print("1 - Listar posts")
    print("2 - Salir")

    opcion = input("Elegí una opción: ")

    if opcion == "1":
        print("Mostrando posts...")
    elif opcion == "2":
        print("Saliendo del programa...")
        programa_activo = False
    else:
        print("Opción inválida.")
```

En este caso, el bucle se repite mientras `programa_activo` sea `True`.

Cuando el usuario elige `"2"`, cambiamos el valor de la variable a `False`.

Entonces el `while` deja de repetirse.

El bucle `while True`

Otra forma muy común de crear menús es usar:

CÓDIGO

```
while True:
```

Esto crea un bucle que, en principio, se repite para siempre.

Pero podemos cortarlo desde adentro usando `break`.

Ejemplo:

CÓDIGO

```
while True:
    print("Hola, soy un bucle.")
    break

print("Ya salí del bucle.")
```

Salida:

CÓDIGO

```
Hola, soy un bucle.
Ya salí del bucle.
```

La palabra `break` corta el bucle inmediatamente.

¿Cuándo usar `break`?

`break` se usa cuando queremos salir de un bucle antes de que continúe repitiéndose.

En un menú, suele usarse cuando el usuario elige la opción “Salir”.

CÓDIGO

```
while True:
    opcion = input("Escribí salir para terminar: ")

    if opcion == "salir":
        print("Programa finalizado.")
        break

    print("El programa sigue funcionando.")
```

Ejemplo de ejecución:

CÓDIGO

```
Escribí salir para terminar: hola
El programa sigue funcionando.
Escribí salir para terminar: probar
El programa sigue funcionando.
Escribí salir para terminar: salir
Programa finalizado.
```

Estructura de un menú interactivo

Un menú interactivo suele tener estos pasos:

1. Mostrar las opciones.
2. Pedir una opción al usuario.
3. Evaluar la opción con `if`, `elif` y `else`.
4. Ejecutar la acción correspondiente.
5. Repetir el menú hasta que el usuario elija salir.

Ejemplo inicial:

CÓDIGO

```
while True:
    print("\n--- MENÚ PRINCIPAL DEL BLOG ---")
    print("1 - Listar posts")
    print("2 - Buscar un post")
    print("3 - Salir")

    opcion = input("Elegí una opción: ")

    if opcion == "1":
        print("Mostrando posts...")
    elif opcion == "2":
        print("Buscando post...")
    elif opcion == "3":
        print("Saliendo del programa. ¡Hasta pronto!")
        break
    else:
        print("Opción inválida. Intentá nuevamente.")
```

La línea:

CÓDIGO

```
print("\n--- MENÚ PRINCIPAL DEL BLOG ---")
```

usa `\n` para dejar una línea en blanco antes del menú. Esto ayuda a que la salida en la terminal se vea más ordenada.

Recordatorio importante sobre `input()`

La función `input()` siempre devuelve texto.

Aunque el usuario escriba un número, Python lo recibe como string.

Ejemplo:

CÓDIGO

```
opcion = input("Elegí una opción: ")

print(type(opcion))
```

Si el usuario escribe:

CÓDIGO

```
1
```

Python guarda:

CÓDIGO

```
"1"
```

No guarda:

CÓDIGO

```
1
```

Por eso, en menús simples conviene comparar contra texto:

CÓDIGO

```
if opcion == "1":
    print("Elegiste la opción 1")
```

Y no contra número:

CÓDIGO

```
if opcion == 1:
    print("Esto no funcionará como esperás")
```

Normalizar la entrada del usuario

A veces el usuario puede escribir espacios sin querer.

Por ejemplo:

CÓDIGO

```
 salir
```

También puede escribir con mayúsculas:

CÓDIGO

```
SALIR
```

Para evitar problemas, podemos usar `strip()` y `lower()`.

CÓDIGO

```
opcion = input("Escribí salir para terminar: ")
opcion = opcion.strip().lower()

if opcion == "salir":
    print("Programa finalizado.")
```

`strip()` elimina espacios al principio y al final.

`lower()` convierte el texto a minúsculas.

Entonces:

CÓDIGO

```
 SALIR
```

se transforma en:

CÓDIGO

```
salir
```

Menú usando palabras

No siempre hace falta usar números. También podemos permitir comandos escritos.

CÓDIGO

```
while True:
    print("\n--- MENÚ DEL BLOG ---")
    print("listar - Ver posts")
    print("crear - Crear post")
    print("salir - Finalizar")

    opcion = input("Elegí una opción: ").strip().lower()

    if opcion == "listar":
        print("Mostrando posts...")
    elif opcion == "crear":
        print("Creando un nuevo post...")
    elif opcion == "salir":
        print("Saliendo del programa...")
        break
    else:
        print("Opción inválida.")
```

Este menú acepta opciones como:

CÓDIGO

```
listar
crear
salir
```

Y gracias a `strip().lower()`, también puede entender:

CÓDIGO

```
 LISTAR
Crear
SALIR
```

Ejemplo completo: menú del blog con posts

Ahora podemos combinar `while`, `if`, `for` y listas.

CÓDIGO

```
posts = [
    "Introducción a Python",
    "Variables y tipos de datos",
    "Condicionales con if",
    "Bucles con for"
]

while True:
    print("\n--- MENÚ PRINCIPAL DEL BLOG ---")
    print("1 - Listar posts")
    print("2 - Crear post")
    print("3 - Salir")

    opcion = input("Elegí una opción: ").strip()

    if opcion == "1":
        print("\nListado de posts:")

        for numero, post in enumerate(posts, start=1):
            print(f"{numero}. {post}")

    elif opcion == "2":
        nuevo_post = input("Ingresá el título del nuevo post: ").strip()

        if nuevo_post:
            posts.append(nuevo_post)
            print("Post creado correctamente.")
        else:
            print("El título no puede estar vacío.")

    elif opcion == "3":
        print("Saliendo del programa. ¡Hasta pronto!")
        break

    else:
        print("Opción inválida. Intentá nuevamente.")
```

Este programa permite:

- Mostrar los posts actuales.
- Agregar un nuevo post a la lista.
- Volver al menú después de cada acción.
- Salir cuando el usuario elige la opción `"3"`.

Ejemplo de ejecución

CÓDIGO

```
--- MENÚ PRINCIPAL DEL BLOG ---
1 - Listar posts
2 - Crear post
3 - Salir
Elegí una opción: 1

Listado de posts:
1. Introducción a Python
2. Variables y tipos de datos
3. Condicionales con if
4. Bucles con for

--- MENÚ PRINCIPAL DEL BLOG ---
1 - Listar posts
2 - Crear post
3 - Salir
Elegí una opción: 2
Ingresá el título del nuevo post: Bucles con while
Post creado correctamente.

--- MENÚ PRINCIPAL DEL BLOG ---
1 - Listar posts
2 - Crear post
3 - Salir
Elegí una opción: 1

Listado de posts:
1. Introducción a Python
2. Variables y tipos de datos
3. Condicionales con if
4. Bucles con for
5. Bucles con while
```

Este ejemplo muestra cómo el programa mantiene la información en memoria mientras está funcionando.

Cuando el programa termina, esa información se pierde. Más adelante veremos cómo guardar datos en archivos.

Validar opciones incorrectas

Un buen menú no debería romperse si el usuario escribe algo inesperado.

Por ejemplo:

CÓDIGO

```
hola
9
crear post
```

Para eso usamos el `else`.

CÓDIGO

```
if opcion == "1":
    print("Listando posts...")
elif opcion == "2":
    print("Creando post...")
elif opcion == "3":
    print("Saliendo...")
    break
else:
    print("Opción inválida.")
```

El `else` funciona como una red de seguridad.

Si ninguna opción coincide, el programa muestra un mensaje y vuelve a empezar el bucle.

Error común: olvidar el `break`

Este código parece correcto, pero tiene un problema:

CÓDIGO

```
while True:
    opcion = input("Elegí una opción: ")

    if opcion == "salir":
        print("Hasta luego.")
```

El programa muestra `"Hasta luego."`, pero no termina.

Para salir realmente del bucle necesitamos `break`.

CÓDIGO

```
while True:
    opcion = input("Elegí una opción: ")

    if opcion == "salir":
        print("Hasta luego.")
        break
```

Error común: comparar texto con números

Incorrecto:

CÓDIGO

```
opcion = input("Elegí una opción: ")

if opcion == 1:
    print("Elegiste listar posts.")
```

Correcto:

CÓDIGO

```
opcion = input("Elegí una opción: ")

if opcion == "1":
    print("Elegiste listar posts.")
```

Recordá: `input()` siempre devuelve texto.

Error común: no cambiar la condición del `while`

Incorrecto:

CÓDIGO

```
seguir = True

while seguir:
    print("El programa sigue funcionando.")
```

Ese bucle no termina nunca porque `seguir` siempre vale `True`.

Correcto:

CÓDIGO

```
seguir = True

while seguir:
    opcion = input("¿Querés salir? ")

    if opcion == "si":
        seguir = False
```

Mejor práctica: mantener el menú legible

Cuando el menú se repite muchas veces, la terminal puede verse desordenada.

Podemos usar separadores:

CÓDIGO

```
print("\n----------------------")
print("MENÚ DEL BLOG")
print("----------------------")
```

Ejemplo:

CÓDIGO

```
while True:
    print("\n----------------------")
    print("MENÚ DEL BLOG")
    print("----------------------")
    print("1 - Listar posts")
    print("2 - Crear post")
    print("3 - Salir")

    opcion = input("Elegí una opción: ").strip()

    if opcion == "1":
        print("Listando posts...")
    elif opcion == "2":
        print("Creando post...")
    elif opcion == "3":
        print("Saliendo...")
        break
    else:
        print("Opción inválida.")
```

No cambia la lógica del programa, pero mejora la experiencia de uso.

Aplicación en el mundo real

Aunque este menú funciona en la terminal, la idea detrás aparece en muchas aplicaciones reales.

Por ejemplo:

- Un cajero automático repite opciones hasta que el usuario finaliza la operación.
- Un videojuego mantiene un ciclo principal mientras la partida está activa.
- Una herramienta de consola espera comandos hasta que el usuario decide salir.
- Una aplicación web evalúa constantemente qué acción debe realizar según lo que el usuario hace.

En nuestro blog, el `while` nos permite crear un primer prototipo interactivo antes de pasar a una interfaz web con Django.

Resumen de la unidad

En esta unidad aprendiste que:

- `while` repite un bloque mientras una condición sea verdadera.
- Se usa cuando no sabemos cuántas veces se repetirá una acción.
- Un bucle infinito ocurre cuando la condición nunca deja de cumplirse.
- `while True` permite crear un bucle permanente controlado desde adentro.
- `break` permite salir de un bucle.
- `input()` siempre devuelve texto.
- En menús simples conviene comparar opciones como strings: `"1"`, `"2"`, `"3"`.
- `strip()` y `lower()` ayudan a limpiar la entrada del usuario.
- Podemos combinar `while`, `if`, `elif`, `else` y `for` para crear un menú funcional.

Con esta estructura, nuestro blog por consola empieza a parecerse más a una aplicación real: el usuario puede elegir acciones, repetir operaciones y salir cuando lo desee.

Filtrar y buscar posts en memoria

Aprende a localizar información específica en tu colección de posts usando lógica de búsqueda por texto y filtros por atributos.

Imagina que tu blog empieza a crecer.

Al principio tal vez tenías pocos posts:

CÓDIGO

```
posts = ["Python básico", "Django inicial", "Mi primer blog"]
```

Pero con el tiempo podrías tener muchos más. Si un usuario quiere encontrar publicaciones sobre `"Django"`, no tendría sentido revisar una por una manualmente.

Para resolver ese problema necesitamos aprender a buscar y filtrar información.

En esta unidad vamos a trabajar con posts guardados en memoria, usando listas y diccionarios.

Buscar vs filtrar

Aunque a veces usamos estas palabras como si fueran lo mismo, en programación tienen una pequeña diferencia.

Buscar significa encontrar datos que coincidan con un texto.

Por ejemplo:

> **Nota**
>
> Buscar posts que tengan la palabra `"Django"` en el título.

Filtrar significa reducir una lista según una condición.

Por ejemplo:

> **Nota**
>
> Mostrar solamente los posts publicados.

O también:

> **Nota**
>
> Mostrar solamente los posts escritos por una autora específica.

En ambos casos vamos a recorrer una lista, evaluar condiciones y guardar los resultados que nos interesan.

Lista inicial de posts

Vamos a usar esta lista como base:

CÓDIGO

```
posts = [
    {
        "titulo": "Aprendiendo Python desde cero",
        "autor": "Ana",
        "estado": "publicado",
        "likes": 25
    },
    {
        "titulo": "Introducción a Django",
        "autor": "Beto",
        "estado": "publicado",
        "likes": 40
    },
    {
        "titulo": "Ideas para mi próximo post",
        "autor": "Ana",
        "estado": "borrador",
        "likes": 0
    },
    {
        "titulo": "Cómo crear un blog con Python",
        "autor": "Carla",
        "estado": "publicado",
        "likes": 15
    }
]
```

Cada post es un diccionario.

La lista completa contiene varios diccionarios.

Buscar una palabra en el título

Para saber si un texto está dentro de otro, usamos el operador `in`.

CÓDIGO

```
titulo = "Aprendiendo Python desde cero"

if "Python" in titulo:
    print("Encontrado")
```

Salida:

CÓDIGO

```
Encontrado
```

Esto funciona porque la palabra `"Python"` está dentro del texto `"Aprendiendo Python desde cero"`.

Buscar posts por palabra clave

Ahora podemos aplicar esa idea a una lista de posts.

CÓDIGO

```
posts = [
    {"titulo": "Aprendiendo Python desde cero", "autor": "Ana", "estado":
"publicado"},
    {"titulo": "Introducción a Django", "autor": "Beto", "estado": "publicado"},
    {"titulo": "Cómo crear un blog con Python", "autor": "Carla", "estado":
"publicado"}
]

busqueda = "Python"

for post in posts:
    if busqueda in post["titulo"]:
        print(post["titulo"])
```

Salida:

CÓDIGO

```
Aprendiendo Python desde cero
Cómo crear un blog con Python
```

El `for` recorre todos los posts.

El `if` revisa si la palabra buscada aparece dentro del título.

Mejorar la búsqueda con `lower()`

El ejemplo anterior funciona, pero tiene un problema.

Para Python, estos textos no son iguales:

CÓDIGO

```
"Python"
"python"
"PYTHON"
```

Entonces, si el usuario escribe `"python"` en minúscula, tal vez no encuentre un título que tenga `"Python"` con mayúscula.

Para evitar eso, convertimos ambos textos a minúsculas usando `lower()`.

CÓDIGO

```
titulo = "Aprendiendo Python desde cero"
busqueda = "python"

if busqueda.lower() in titulo.lower():
    print("Encontrado")
```

Salida:

CÓDIGO

```
Encontrado
```

Ahora la comparación es más flexible.

Usar `strip()` para limpiar la búsqueda

Cuando usamos `input()`, el usuario puede escribir espacios sin querer.

Por ejemplo:

CÓDIGO

```
 python
```

Para limpiar esos espacios usamos `strip()`.

CÓDIGO

```
busqueda = input("Buscar post: ").strip().lower()
```

Con eso, si el usuario escribe:

CÓDIGO

```
 Python
```

Python lo transforma en:

CÓDIGO

```
python
```

Buscar posts usando `input()`

CÓDIGO

```
posts = [
    {"titulo": "Aprendiendo Python desde cero", "autor": "Ana", "estado":
"publicado"},
    {"titulo": "Introducción a Django", "autor": "Beto", "estado": "publicado"},
    {"titulo": "Cómo crear un blog con Python", "autor": "Carla", "estado":
"publicado"}
]

busqueda = input("¿Qué querés buscar? ").strip().lower()

for post in posts:
    titulo = post["titulo"].lower()

    if busqueda in titulo:
        print(post["titulo"])
```

Ejemplo de ejecución:

CÓDIGO

```
¿Qué querés buscar? python
Aprendiendo Python desde cero
Cómo crear un blog con Python
```

Guardar resultados en una nueva lista

Hasta ahora solo mostramos los resultados con `print()`.

Pero muchas veces queremos guardar los resultados encontrados para usarlos después.

CÓDIGO

```
posts = [
    {"titulo": "Aprendiendo Python desde cero", "autor": "Ana", "estado":
"publicado"},
    {"titulo": "Introducción a Django", "autor": "Beto", "estado": "publicado"},
    {"titulo": "Cómo crear un blog con Python", "autor": "Carla", "estado":
"publicado"}
]

busqueda = input("¿Qué querés buscar? ").strip().lower()

resultados = []

for post in posts:
    titulo = post["titulo"].lower()

    if busqueda in titulo:
        resultados.append(post)

print("Resultados encontrados:")

for post in resultados:
    print(post["titulo"])
```

La lista `resultados` empieza vacía.

Cada vez que encontramos un post que coincide, lo agregamos con `append()`.

Validar si no hubo resultados

¿Qué pasa si el usuario busca algo que no existe?

Podemos revisar si la lista `resultados` quedó vacía.

CÓDIGO

```
if len(resultados) == 0:
    print("No se encontraron posts.")
else:
    print("Resultados encontrados:")

    for post in resultados:
        print(post["titulo"])
```

También podemos escribirlo así:

CÓDIGO

```
if not resultados:
    print("No se encontraron posts.")
else:
    print("Resultados encontrados:")

    for post in resultados:
        print(post["titulo"])
```

En Python, una lista vacía se considera como `False`.

Por eso `if not resultados:` significa:

> **Nota**
>
> Si la lista de resultados está vacía.

Ejemplo completo: buscar posts por título

CÓDIGO

```
posts = [
    {
        "titulo": "Aprendiendo Python desde cero",
        "autor": "Ana",
        "estado": "publicado",
        "likes": 25
    },
    {
        "titulo": "Introducción a Django",
        "autor": "Beto",
        "estado": "publicado",
        "likes": 40
    },
    {
        "titulo": "Ideas para mi próximo post",
        "autor": "Ana",
        "estado": "borrador",
        "likes": 0
    },
    {
        "titulo": "Cómo crear un blog con Python",
        "autor": "Carla",
        "estado": "publicado",
        "likes": 15
    }
]

busqueda = input("Buscar por título: ").strip().lower()

resultados = []

for post in posts:
    titulo = post["titulo"].lower()

    if busqueda in titulo:
        resultados.append(post)

if not resultados:
    print("No se encontraron posts.")
else:
    print("Resultados encontrados:")

    for post in resultados:
        print(f"- {post['titulo']} | Autor: {post['autor']}")
```

Ejemplo de ejecución:

CÓDIGO

```
Buscar por título: python
Resultados encontrados:
- Aprendiendo Python desde cero | Autor: Ana
- Cómo crear un blog con Python | Autor: Carla
```

Filtrar posts publicados

Buscar por texto es útil, pero a veces queremos filtrar por una regla.

Por ejemplo:

> **Nota**
>
> Mostrar solo los posts publicados.

CÓDIGO

```
posts = [
    {"titulo": "Post A", "estado": "publicado"},
    {"titulo": "Post B", "estado": "borrador"},
    {"titulo": "Post C", "estado": "publicado"}
]

for post in posts:
    if post["estado"] == "publicado":
        print(post["titulo"])
```

Salida:

CÓDIGO

```
Post A
Post C
```

El post que está en estado `"borrador"` no se muestra.

Guardar los posts filtrados

También podemos guardar los posts publicados en una nueva lista.

CÓDIGO

```
posts_publicados = []

for post in posts:
    if post["estado"] == "publicado":
        posts_publicados.append(post)

print(posts_publicados)
```

Esto nos permite separar los datos originales de los datos filtrados.

La lista `posts` sigue teniendo todos los posts.

La lista `posts_publicados` contiene solo los que cumplen la condición.

Filtrar por autor

También podemos filtrar posts escritos por una persona específica.

CÓDIGO

```
posts = [
    {"titulo": "Post A", "autor": "Ana", "estado": "publicado"},
    {"titulo": "Post B", "autor": "Beto", "estado": "publicado"},
    {"titulo": "Post C", "autor": "Ana", "estado": "borrador"}
]

autor_buscado = input("Buscar posts por autor: ").strip().lower()

resultados = []

for post in posts:
    autor = post["autor"].lower()

    if autor == autor_buscado:
        resultados.append(post)

if not resultados:
    print("No se encontraron posts de ese autor.")
else:
    print("Posts encontrados:")

    for post in resultados:
        print(post["titulo"])
```

Ejemplo de ejecución:

CÓDIGO

```
Buscar posts por autor: ana
Posts encontrados:
Post A
Post C
```

Combinar condiciones con `and`

A veces queremos aplicar más de una condición al mismo tiempo.

Por ejemplo:

> **Nota**
>
> Mostrar solo los posts de Ana que además estén publicados.

Para eso usamos `and`.

CÓDIGO

```
posts = [
    {"titulo": "Post A", "autor": "Ana", "estado": "publicado"},
    {"titulo": "Post B", "autor": "Beto", "estado": "publicado"},
    {"titulo": "Post C", "autor": "Ana", "estado": "borrador"}
]

for post in posts:
    if post["autor"] == "Ana" and post["estado"] == "publicado":
        print(post["titulo"])
```

Salida:

CÓDIGO

```
Post A
```

El `and` exige que las dos condiciones sean verdaderas.

El post debe ser de Ana y también debe estar publicado.

Combinar condiciones con `or`

Con `or`, alcanza con que se cumpla una de las condiciones.

Por ejemplo:

> **Nota**
>
> Mostrar posts escritos por Ana o por Beto.

CÓDIGO

```
posts = [
    {"titulo": "Post A", "autor": "Ana"},
    {"titulo": "Post B", "autor": "Beto"},
    {"titulo": "Post C", "autor": "Carla"}
]

for post in posts:
    if post["autor"] == "Ana" or post["autor"] == "Beto":
        print(post["titulo"])
```

Salida:

CÓDIGO

```
Post A
Post B
```

El post de Carla no se muestra porque no cumple ninguna de las dos condiciones.

Filtrar posts populares

También podemos filtrar usando números.

Por ejemplo:

> **Nota**
>
> Mostrar posts con más de 20 likes.

CÓDIGO

```
posts = [
    {"titulo": "Post A", "likes": 10},
    {"titulo": "Post B", "likes": 35},
    {"titulo": "Post C", "likes": 50}
]

for post in posts:
    if post["likes"] > 20:
        print(post["titulo"])
```

Salida:

CÓDIGO

```
Post B
Post C
```

Búsqueda parcial en varios campos

También podemos buscar una palabra en más de un campo.

Por ejemplo, buscar en el título o en el autor.

CÓDIGO

```
posts = [
    {"titulo": "Aprendiendo Python desde cero", "autor": "Ana"},
    {"titulo": "Introducción a Django", "autor": "Beto"},
    {"titulo": "Cómo crear un blog con Python", "autor": "Carla"}
]

busqueda = input("Buscar: ").strip().lower()

resultados = []

for post in posts:
    titulo = post["titulo"].lower()
    autor = post["autor"].lower()

    if busqueda in titulo or busqueda in autor:
        resultados.append(post)

if not resultados:
    print("No se encontraron resultados.")
else:
    print("Resultados encontrados:")

    for post in resultados:
        print(f"- {post['titulo']} | Autor: {post['autor']}")
```

Ejemplo:

CÓDIGO

```
Buscar: ana
Resultados encontrados:
- Aprendiendo Python desde cero | Autor: Ana
```

Otro ejemplo:

CÓDIGO

```
Buscar: python
Resultados encontrados:
- Aprendiendo Python desde cero | Autor: Ana
- Cómo crear un blog con Python | Autor: Carla
```

Comprensión de listas

Python tiene una forma más corta de crear listas filtradas.

Se llama comprensión de listas o `list comprehension`.

Primero definimos una lista de posts con la clave `estado`:

CÓDIGO

```
posts = [
    {"titulo": "Post A", "estado": "publicado"},
    {"titulo": "Post B", "estado": "borrador"},
    {"titulo": "Post C", "estado": "publicado"}
]
```

Ahora veamos la versión con `for` tradicional:

CÓDIGO

```
posts_publicados = []

for post in posts:
    if post["estado"] == "publicado":
        posts_publicados.append(post)
```

Y la misma idea usando comprensión de listas:

CÓDIGO

```
posts_publicados = [post for post in posts if post["estado"] == "publicado"]
```

La estructura general es:

CÓDIGO

```
[nuevo_elemento for elemento in coleccion if condicion]
```

En este caso:

CÓDIGO

```
[post for post in posts if post["estado"] == "publicado"]
```

significa:

> **Nota**
>
> Crear una nueva lista con cada post de posts, pero solo si su estado es publicado.

Ejemplo con comprensión de listas

CÓDIGO

```
posts = [
    {"titulo": "Post A", "estado": "publicado"},
    {"titulo": "Post B", "estado": "borrador"},
    {"titulo": "Post C", "estado": "publicado"}
]

posts_publicados = [post for post in posts if post["estado"] == "publicado"]

for post in posts_publicados:
    print(post["titulo"])
```

Salida:

CÓDIGO

```
Post A
Post C
```

Esta forma es más breve, pero al principio puede parecer menos clara.

Por eso conviene entender primero la versión con `for`, `if` y `append()`.

Buscar con comprensión de listas

También podemos buscar por texto usando comprensión de listas.

CÓDIGO

```
posts = [
    {"titulo": "Aprendiendo Python desde cero"},
    {"titulo": "Introducción a Django"},
    {"titulo": "Cómo crear un blog con Python"}
]

busqueda = input("Buscar: ").strip().lower()

resultados = [
    post
    for post in posts
    if busqueda in post["titulo"].lower()
]

for post in resultados:
    print(post["titulo"])
```

Salida posible:

CÓDIGO

```
Aprendiendo Python desde cero
Cómo crear un blog con Python
```

Esta versión es compacta y muy usada en Python, pero no es obligatorio usarla desde el principio.

Evitar errores con claves inexistentes

Un error común al trabajar con diccionarios es intentar acceder a una clave que no existe.

Ejemplo:

CÓDIGO

```
post = {
    "titulo": "Aprendiendo Python",
    "autor": "Ana"
}

print(post["estado"])
```

Ese código produce un error porque el diccionario no tiene la clave `"estado"`.

Para evitarlo, podemos usar `.get()`.

CÓDIGO

```
post = {
    "titulo": "Aprendiendo Python",
    "autor": "Ana"
}

estado = post.get("estado", "sin estado")

print(estado)
```

Salida:

CÓDIGO

```
sin estado
```

La estructura es:

CÓDIGO

```
diccionario.get("clave", valor_por_defecto)
```

Si la clave existe, devuelve su valor.

Si no existe, devuelve el valor por defecto.

Ejemplo usando `.get()`

CÓDIGO

```
posts = [
    {"titulo": "Post A", "estado": "publicado"},
    {"titulo": "Post B"},
    {"titulo": "Post C", "estado": "borrador"}
]

for post in posts:
    estado = post.get("estado", "sin estado")

    if estado == "publicado":
        print(post["titulo"])
```

Salida:

CÓDIGO

```
Post A
```

El segundo post no rompe el programa, aunque no tenga la clave `"estado"`.

Error común: confundir `and` con `or`

Supongamos que queremos mostrar posts de Ana que además estén publicados.

Correcto:

CÓDIGO

```
if post["autor"] == "Ana" and post["estado"] == "publicado":
    print(post["titulo"])
```

Esto exige que ambas condiciones se cumplan.

Incorrecto para este caso:

CÓDIGO

```
if post["autor"] == "Ana" or post["estado"] == "publicado":
    print(post["titulo"])
```

Con `or`, aparecerían:

- Los posts de Ana.
- Los posts publicados.

- Incluso posts publicados de otros autores.

Por eso:

- `and` suele reducir más los resultados.
- `or` suele ampliar más los resultados.

Ejemplo completo: buscador y filtro de posts

CÓDIGO

```
posts = [
    {
        "titulo": "Aprendiendo Python desde cero",
        "autor": "Ana",
        "estado": "publicado",
        "likes": 25
    },
    {
        "titulo": "Introducción a Django",
        "autor": "Beto",
        "estado": "publicado",
        "likes": 40
    },
    {
        "titulo": "Ideas para mi próximo post",
        "autor": "Ana",
        "estado": "borrador",
        "likes": 0
    },
    {
        "titulo": "Cómo crear un blog con Python",
        "autor": "Carla",
        "estado": "publicado",
        "likes": 15
    }
]

busqueda = input("Buscar por título: ").strip().lower()

resultados = []

for post in posts:
    titulo = post["titulo"].lower()
    estado = post["estado"]

    if busqueda in titulo and estado == "publicado":
        resultados.append(post)

if not resultados:
    print("No se encontraron posts publicados con esa búsqueda.")
else:
    print("Resultados encontrados:")

    for post in resultados:
        print(f"- {post['titulo']} | Autor: {post['autor']} | Likes:
{post['likes']}")
```

Ejemplo de ejecución:

CÓDIGO

```
Buscar por título: python
Resultados encontrados:
- Aprendiendo Python desde cero | Autor: Ana | Likes: 25
- Cómo crear un blog con Python | Autor: Carla | Likes: 15
```

En este ejemplo, el programa busca posts que cumplan dos condiciones:

- Que el título contenga la palabra buscada.
- Que el estado sea `"publicado"`.

Resumen de la unidad

En esta unidad aprendiste que:

- Buscar significa encontrar coincidencias de texto.
- Filtrar significa reducir una colección según una condición.
- El operador `in` permite saber si un texto está dentro de otro.
- `lower()` ayuda a comparar textos sin importar mayúsculas o minúsculas.
- `strip()` elimina espacios innecesarios ingresados por el usuario.
- Podemos guardar resultados en una nueva lista usando `append()`.
- Podemos validar si una búsqueda no encontró resultados.
- `and` permite exigir varias condiciones al mismo tiempo.
- `or` permite aceptar más de una posibilidad.
- Las comprensiones de listas permiten filtrar datos de forma más compacta.
- `.get()` ayuda a evitar errores cuando una clave no existe en un diccionario.

Con estas herramientas, nuestro blog por consola ya puede comportarse como una aplicación más útil: no solo muestra datos, también permite consultarlos, buscarlos y filtrarlos según lo que necesita el usuario.

PRE-ENTREGA DEL PROYECTO

Actividad evaluable · parte del proyecto integrador

Preentrega 3: sistema de consola interactivo listo para consolidar

Checkpoint de integración de lógica de control para consolidar un menú interactivo capaz de listar, buscar y filtrar posts de un blog en memoria.

Preentrega 3: Sistema de consola interactivo para tu blog

¡Felicidades por llegar al final del módulo!

Hasta ahora, aprendiste a tomar decisiones con `if`, a repetir tareas con `for` y a crear programas que pueden mantenerse en ejecución usando `while`.

En el módulo anterior, modelaste la estructura de datos de tu blog usando listas, diccionarios, tuplas y sets. Ahora vas a tomar esa base y darle interacción: el usuario podrá elegir opciones desde la consola para ver, buscar y filtrar publicaciones.

Contexto de la cadena de entregables

En el **Módulo 1**, configuraste tu entorno y ejecutaste tus primeros scripts en Python.

En el **Módulo 2**, construiste la estructura de datos inicial del blog. Creaste un perfil de autor con un diccionario, definiste estados posibles con una tupla, usaste un set para etiquetas únicas y organizaste los posts en una lista de diccionarios.

Ahora, en el **Módulo 3**, vas a continuar sobre esa misma idea: usar una lista de posts guardados en memoria, pero agregando un menú interactivo que permita consultar esa información.

Lo que construyas aquí será la primera versión interactiva de tu futuro blog. Todavía no tendrá interfaz web ni base de datos, pero sí tendrá una lógica central: mostrar opciones, recibir una decisión del usuario y ejecutar una acción según esa elección.

Objetivo de aprendizaje de la preentrega

El objetivo de esta preentrega es consolidar la capacidad de construir un flujo de programa interactivo usando estructuras de control.

Vas a practicar:

Interactividad

Crear un menú que se muestre en consola y permita al usuario elegir una opción.

Condicionales

Usar `if`, `elif` y `else` para decidir qué acción ejecutar.

Iteraciones

Usar `for` para recorrer una lista de posts.

Bucles controlados

Usar `while` para mantener el programa activo hasta que el usuario elija salir.

Búsqueda y filtrado

Buscar posts por título y filtrar publicaciones según sus tags.

Acceso a datos anidados

Acceder a información dentro de estructuras combinadas, por ejemplo:

CÓDIGO

```
post["autor"]["nombre"]
```

¿Qué construir?

Debés crear el código de un programa llamado:

CÓDIGO

```
sistema_blog.py
```

Este programa debe representar un pequeño sistema de consola para consultar publicaciones de un blog.

La entrega no requiere adjuntar el archivo `.py`. Deberás entregar el código completo en formato texto, como si fuera el contenido del archivo `sistema_blog.py`.

Datos iniciales

El programa debe continuar la estructura trabajada en el **Módulo 2**.

Debe incluir:

- Un diccionario llamado `perfil_autor`.
- Una tupla llamada `estados_post`.
- Un set llamado `etiquetas_blog`.
- Una lista llamada `posts` con al menos 3 diccionarios.

Cada post debe tener, como mínimo, las siguientes claves:

CÓDIGO

```
"id"
"titulo"
"contenido"
"autor"
"tags"
"estado"
```

Importante: la clave `"autor"` no debe ser un texto simple, sino que debe usar el diccionario `perfil_autor`.

Ejemplo:

CÓDIGO

```
perfil_autor = {
    "nombre": "Ana López",
    "bio": "Desarrolladora web y creadora de contenido sobre programación.",
    "especialidad": "Python y Django",
    "redes_sociales": ["@ana_dev", "@ana_python"]
}

estados_post = ("borrador", "publicado", "archivado")

etiquetas_blog = {"Python", "Django", "Web", "Backend", "Python"}

posts = [
    {
        "id": 1,
        "titulo": "Primeros pasos con Python",
        "contenido": "En este post veremos cómo empezar a programar con Python.",
        "autor": perfil_autor,
        "tags": ["Python", "Principiantes"],
        "estado": "publicado"
    },
    {
        "id": 2,
        "titulo": "Qué es Django",
        "contenido": "Django es un framework web creado con Python.",
        "autor": perfil_autor,
        "tags": ["Python", "Django", "Web"],
        "estado": "borrador"
    },
    {
        "id": 3,
        "titulo": "Organizando datos con listas",
        "contenido": "Las listas permiten guardar varios elementos en una sola
variable.",
        "autor": perfil_autor,
        "tags": ["Python", "Listas"],
        "estado": "archivado"
    }
]
```

Menú del programa

El programa debe mostrar el siguiente menú en pantalla:

CÓDIGO

```
--- MENU DEL BLOG ---
1. Ver todos los posts
2. Buscar por titulo
3. Filtrar por tag
4. Salir
```

El menú debe volver a mostrarse después de cada acción, excepto cuando el usuario elija la opción `"4"` para salir.

Funcionalidades requeridas

Opción 1: Ver todos los posts

Si el usuario elige `"1"`, el programa debe recorrer la lista de posts e imprimir el título de cada publicación con un formato legible.

Como ahora el autor es un diccionario, para mostrar su nombre deberás acceder así:

CÓDIGO

```
post["autor"]["nombre"]
```

Ejemplo de salida:

CÓDIGO

```
Posts disponibles:
- Primeros pasos con Python | Autor: Ana López
- Qué es Django | Autor: Ana López
- Organizando datos con listas | Autor: Ana López
```

Opción 2: Buscar por titulo

Si el usuario elige `"2"`, el programa debe pedir una palabra por consola.

Luego debe mostrar los posts cuyo título contenga esa palabra.

La búsqueda debe funcionar aunque el usuario escriba en mayúsculas o minúsculas. Para eso, se debe usar `.lower()`.

Ejemplo:

CÓDIGO

```
Buscar por titulo: python
```

Debe poder encontrar un post titulado:

CÓDIGO

```
Primeros pasos con Python
```

Opción 3: Filtrar por tag

Si el usuario elige `"3"`, el programa debe pedir un tag, por ejemplo:

CÓDIGO

```
Python
```

Luego debe recorrer la lista de posts y mostrar solo aquellos que tengan ese tag dentro de su lista de `tags`.

El filtrado también debe ignorar mayúsculas y minúsculas.

Ejemplo de salida:

CÓDIGO

```
Posts con el tag Python:
- Primeros pasos con Python
- Qué es Django
- Organizando datos con listas
```

Opción 4: Salir

Si el usuario elige `"4"`, el programa debe mostrar un mensaje de despedida y terminar su ejecución.

Ejemplo:

CÓDIGO

```
Gracias por usar el sistema del blog. ¡Hasta luego!
```

Para finalizar el bucle, se debe usar `break`.

Manejo de errores

Si el usuario ingresa una opción que no sea `"1"`, `"2"`, `"3"` o `"4"`, el programa debe mostrar:

CÓDIGO

```
Opción inválida, intenta de nuevo
```

Luego debe volver a mostrar el menú sin cerrarse.

Criterios de aceptación

Para considerar este checkpoint como completado, tu código debe cumplir con los siguientes puntos:

- Crear un diccionario llamado `perfil_autor`.
- Crear una tupla llamada `estados_post`.
- Crear un set llamado `etiquetas_blog`.
- Crear una lista llamada `posts`.
- Incluir al menos 3 diccionarios dentro de `posts`.

- Cada post debe tener las claves `id`, `titulo`, `contenido`, `autor`, `tags` y `estado`.
- La clave `"autor"` de cada post debe usar el diccionario `perfil_autor`.
- Mostrar el menú solicitado en consola.
- Usar un bucle `while` para mantener el programa activo.
- Usar `input()` para capturar la opción elegida por el usuario.
- Usar `if`, `elif` y `else` para manejar las opciones del menú.
- Usar `for` para recorrer la lista de posts.
- La opción `"1"` debe mostrar los títulos de todos los posts y el nombre del autor accediendo al diccionario anidado.
- La opción `"2"` debe buscar posts por título usando `.lower()`.
- La opción `"3"` debe filtrar posts por tag.
- La opción `"4"` debe despedirse y cerrar el programa.
- Las opciones inválidas deben mostrar el mensaje `"Opción inválida, intenta de nuevo"`.
- El programa no debe cerrarse inesperadamente ante una opción incorrecta.
- El código debe estar comentado brevemente para explicar cada sección.
- El código debe poder copiarse, pegarse y ejecutarse con:

CÓDIGO

```
python sistema_blog.py
```

Pasos sugeridos

1. Preparar la entrega

Creá el código completo de un programa llamado `sistema_blog.py`.

No es necesario subirlo a GitHub ni adjuntar un archivo `.py`. La entrega debe realizarse pegando el código completo en formato texto, según lo solicitado por la plataforma.

2. Recuperar la estructura del Módulo 2

Partí de la estructura de datos trabajada en el módulo anterior.

El programa debe conservar:

- El diccionario del autor.
- La tupla de estados posibles.
- El set de etiquetas.
- La lista de posts.

Recordá que cada post debe usar el diccionario del autor como dato anidado, no un texto simple.

3. Crear el menú principal

Diseñá un menú que se muestre en consola y le permita al usuario elegir entre cuatro opciones:

- Ver todos los posts.
- Buscar por título.
- Filtrar por tag.
- Salir.

El menú debe volver a aparecer después de cada acción, excepto cuando el usuario elija salir.

4. Capturar la opción del usuario

Usá una entrada por consola para guardar la opción elegida.

Recordá que toda entrada recibida desde consola se interpreta inicialmente como texto.

5. Resolver cada opción con condicionales

Usá una estructura condicional para decidir qué acción ejecutar según la opción ingresada.

Cada opción del menú debe tener su propia lógica:

- Una opción para listar posts.
- Una opción para buscar por título.
- Una opción para filtrar por tag.
- Una opción para salir.
- Un caso final para manejar opciones inválidas.

6. Listar los posts

Para la opción de ver todos los posts, recorré la lista de publicaciones y mostrá el título de cada una.

También podés mostrar el nombre del autor, accediendo al dato anidado dentro de cada post.

7. Implementar la búsqueda por título

Para la opción de búsqueda, pedile al usuario una palabra o fragmento de texto.

Luego, recorré los posts y verificá si esa palabra aparece dentro del título.

La búsqueda debe ignorar diferencias entre mayúsculas y minúsculas.

8. Implementar el filtrado por tag

Para la opción de filtrado, pedile al usuario una etiqueta.

Luego, recorré los posts y revisá si esa etiqueta existe dentro de la lista de tags de cada publicación.

El filtrado también debe ignorar diferencias entre mayúsculas y minúsculas.

9. Implementar la salida del programa

Para la opción de salir, mostrá un mensaje de despedida y finalizá el bucle principal.

El programa no debe cerrarse de forma inesperada: solo debe terminar cuando el usuario elija la opción correspondiente.

10. Probar el programa

Antes de entregar, ejecutá el código y verificá que:

- El menú se muestra correctamente.
- La opción de listar posts funciona.
- La búsqueda por título encuentra coincidencias.
- El filtrado por tag muestra los posts correctos.
- La opción de salir termina el programa.
- Las opciones inválidas muestran un mensaje de error y vuelven al menú.

Errores comunes a evitar

Volver a convertir el autor en un string

En este checkpoint, el autor debe continuar siendo un diccionario, como en el Módulo 2.

Incorrecto:

CÓDIGO

```
"autor": "Ana López"
```

Correcto:

CÓDIGO

```
"autor": perfil_autor
```

Acceder mal al nombre del autor

Como el autor es un diccionario anidado, no alcanza con hacer:

CÓDIGO

```
post["autor"]
```

Eso devuelve todo el diccionario del autor.

Para obtener solo el nombre, usá:

CÓDIGO

```
post["autor"]["nombre"]
```

Confundir = con ==

`=` sirve para asignar valores.

`==` sirve para comparar valores.

Incorrecto:

CÓDIGO

```
if opcion = "1":
```

Correcto:

CÓDIGO

```
if opcion == "1":
```

Comparar texto con números

`input()` siempre devuelve texto.

Incorrecto:

CÓDIGO

```
if opcion == 1:
```

Correcto:

CÓDIGO

```
if opcion == "1":
```

Olvidar el `break`

Si no usás `break` en la opción `"4"`, el programa no va a cerrar correctamente el menú.

CÓDIGO

```
elif opcion == "4":
    print("Saliendo...")
    break
```

No usar `.lower()` en la búsqueda

Si comparás directamente el texto ingresado con el título, puede que no encuentre coincidencias por diferencias entre mayúsculas y minúsculas.

CÓDIGO

```
busqueda = input("Buscar por titulo: ").lower()
```

Recorrer mal la lista de tags

Cada post tiene una lista de tags. Para revisar si existe un tag específico, necesitás comparar contra los elementos de esa lista.

CÓDIGO

```
for tag in post["tags"]:
    print(tag)
```

Formato de entrega

Entregá el código completo en formato texto, como si fuera el contenido del archivo:

CÓDIGO

```
sistema_blog.py
```

No es necesario adjuntar un archivo `.py`.

Asegurate de que:

- El código esté comentado brevemente.
- El programa no tenga errores de sintaxis.
- Se pueda copiar y ejecutar desde la terminal usando:

CÓDIGO

```
python sistema_blog.py
```

Conexión con el futuro

Este sistema de consola es una continuación directa de la estructura creada en el Módulo 2.

En el próximo módulo, vas a aprender a ordenar esta lógica usando funciones, mejorar las validaciones y manejar errores de forma más clara.

Más adelante, la idea de listar, buscar y filtrar posts se transformará en vistas, formularios y páginas dentro de una aplicación web con Django.

Pasos para la entrega

En esta preentrega, consolidarás la lógica de control del blog creando un programa funcional en Python. El objetivo es que tu blog deje de ser una estructura estática y pase a ser un sistema capaz de responder a las interacciones del usuario.

Pasos a realizar:

1. **Preparación de la entrega:** Creá el código completo de un programa llamado `sistema_blog.py`. No es necesario subirlo a GitHub ni adjuntar un archivo `.py`; deberás entregar el código en formato texto, según lo solicitado por la plataforma.
2. **Preparación de datos:** Dentro del código de `sistema_blog.py`, copiá y adaptá las estructuras de datos trabajadas en la unidad anterior: el diccionario de autor, la tupla de estados, el set de etiquetas y la lista de posts. Esta será la base en memoria sobre la que funcionará el sistema interactivo.
3. **Implementación del Bucle Principal**: Utiliza una estructura `while` para que el programa se mantenga en ejecución. Dentro, muestra las 4 opciones del menú mencionadas en la lección.
4. **Lógica de Control**: Implementa una estructura `if-elif-else` para capturar la entrada del usuario (`input()`).
   - **Opción 1**: Usa un bucle `for` para imprimir los títulos y el nombre del autor anidado.
   - **Opción 2**: Solicita un término de búsqueda y filtra la lista usando `.lower()` y el operador `in`.
   - **Opción 3**: Pide un tag y verifica si existe dentro de la lista de `tags` de cada post.
   - **Opción 4**: Rompe el bucle y muestra un mensaje de despedida.

Entregable

Formato: text

Completar con el código de tu solución para la entrega.

Objetivos de aprendizaje

- Implementar un menú interactivo mediante bucles while y la función input.
- Aplicar estructuras condicionales (if/elif/else) para dirigir el flujo del programa.
- Realizar búsquedas y filtrados de datos mediante iteraciones y comparaciones de strings.
- Acceder correctamente a valores anidados dentro de una lista de diccionarios.

Criterios de aceptación

- El programa debe mantenerse ejecutándose hasta que el usuario elija la opción '4'.
- La opción 1 debe mostrar correctamente el nombre del autor accediendo a la clave 'autor' del diccionario del post.
- Las búsquedas por título (Opción 2) y por tag (Opción 3) deben ser insensibles a mayúsculas/minúsculas.
- El código debe incluir al menos 3 posts con la estructura de campos completa detallada en la unidad.
- El repositorio debe ser público y contener el script principal funcional sin errores de sintaxis.

Rúbrica de evaluación

Rúbrica para evaluar un código de Python entregado en formato texto que implementa un sistema interactivo de blog por consola. Se valora la preparación de datos, el control de flujo con bucles y condicionales, la funcionalidad de búsqueda y filtrado, y la claridad mínima para ejecutar y comprender el programa.

Criterio	Descripción	Peso
Preparación de datos y estructura base del programa	Evalúa que el texto contenga las estructuras de datos solicitadas y que la base del programa esté organizada de forma coherente para trabajar con el menú interactivo.	25%
Bucle principal e interacción por menú	Evalúa el uso de un bucle while para mantener el programa en ejecución y la presentación constante de las cuatro opciones del menú antes de cada interacción.	25%
Lógica condicional y salida del programa	Evalúa el uso de if-elif-else para decidir la acción según la entrada del usuario, incluyendo la opción de salir con un mensaje de despedida.	20%
Funcionalidad de las opciones 1, 2 y 3	Evalúa la implementación concreta de las funcionalidades pedidas: listar posts con datos anidados, buscar por término usando lower() e in, y filtrar por tag existente en cada post.	25%
Claridad del código y cierre del ejercicio	Evalúa que el código sea legible, con nombres entendibles, comentarios breves y una organización que facilite revisar y ejecutar el programa.	5%

Total: 100 pts | Aprobación: 70 pts

MÓDULO 4

Funciones, validaciones y manejo de errores

Modularizar el código del blog mediante funciones y asegurar la robustez con validaciones y manejo de excepciones.

Funciones y parámetros: modularizar el blog paso a paso

Aprende a identificar lógica repetida y a transformarla en funciones reutilizables con parámetros en Python, sentando las bases para construir un blog modular y profesional.

Hasta ahora, escribimos instrucciones una debajo de la otra. Eso funciona cuando el programa es pequeño, pero a medida que nuestro blog por consola crece, empieza a aparecer un problema: repetimos lógica.

Por ejemplo, podríamos repetir varias veces código para:

- Mostrar un mensaje de bienvenida.
- Mostrar el menú del blog.
- Formatear el título de un post.
- Validar que un título no esté vacío.
- Mostrar los datos de una publicación.

Si copiamos y pegamos el mismo código en muchas partes, después será más difícil mantenerlo. Si queremos cambiar algo, tendremos que buscarlo y modificarlo en varios lugares.

Para evitar eso, Python nos permite usar funciones.

Una función nos ayuda a guardar un bloque de código con un nombre, para poder reutilizarlo cuando lo necesitemos.

1. ¿Qué es una función?

Una función es un bloque de código reutilizable que realiza una tarea específica.

Podemos pensarla como una pequeña herramienta dentro de nuestro programa:

- Tiene un nombre.
- Puede recibir datos.
- Ejecuta instrucciones.
- Puede mostrar información o devolver un resultado.

En Python ya usamos funciones desde el principio, aunque quizás no las llamamos así.

Por ejemplo:

CÓDIGO

```
print("Hola, mundo")
```

`print()` es una función incluida en Python. Su tarea es mostrar información en pantalla.

También usamos:

CÓDIGO

```
len("Python")
```

`len()` es una función que devuelve la cantidad de elementos de algo. En este caso, devuelve la cantidad de caracteres del texto `"Python"`.

Ahora vamos a aprender a crear nuestras propias funciones.

2. ¿Por qué usar funciones?

Usar funciones nos permite escribir código más claro, organizado y fácil de mantener.

En el contexto de nuestro blog, podríamos tener funciones como:

CÓDIGO

```
mostrar_menu()
crear_post()
listar_posts()
mostrar_post()
validar_titulo()
```

Leer esos nombres ya nos da una idea de lo que hace el programa.

Las funciones nos ayudan a:

Evitar repetir código

Si una lógica se usa muchas veces, conviene convertirla en una función.

Organizar el programa

Un programa grande se vuelve más fácil de entender si está dividido en partes pequeñas.

Corregir errores más rápido

Si hay un problema en una lógica, lo corregimos en un solo lugar.

Mejorar la lectura del código

Un código con funciones bien nombradas es más fácil de leer para otras personas y también para nuestro “yo del futuro”.

3. Cómo definir una función

Para crear una función en Python usamos la palabra clave `def`.

La estructura básica es:

CÓDIGO

```
def nombre_de_la_funcion():
    instrucciones
```

Ejemplo:

CÓDIGO

```
def saludar_visitante():
    print("¡Bienvenido a nuestro blog de Python!")
    print("Esperamos que disfrutes la lectura.")
```

En este ejemplo:

- `def` indica que estamos definiendo una función.
- `saludar_visitante` es el nombre de la función.
- Los paréntesis `()` forman parte de la estructura.
- Los dos puntos `:` indican que comienza el bloque de código.
- Las líneas indentadas pertenecen a la función.

Importante: definir una función no la ejecuta.

Si escribimos solamente esto:

CÓDIGO

```
def saludar_visitante():
    print("¡Bienvenido a nuestro blog de Python!")
```

No veremos nada en pantalla. Python solo guarda la función para usarla después.

4. Cómo llamar a una función

Para ejecutar una función, tenemos que llamarla por su nombre y agregar paréntesis.

CÓDIGO

```
def saludar_visitante():
    print("¡Bienvenido a nuestro blog de Python!")


saludar_visitante()
```

Resultado:

CÓDIGO

```
¡Bienvenido a nuestro blog de Python!
```

Podemos llamar a una función todas las veces que queramos:

CÓDIGO

```
def mostrar_header():
    print("=== MI BLOG DE PYTHON ===")


mostrar_header()
mostrar_header()
mostrar_header()
```

Resultado:

CÓDIGO

```
=== MI BLOG DE PYTHON ===
=== MI BLOG DE PYTHON ===
=== MI BLOG DE PYTHON ===
```

Esto nos permite reutilizar código sin tener que escribirlo de nuevo.

5. Ejemplo aplicado: mostrar el menú del blog

En un blog por consola, es común mostrar un menú de opciones.

Sin funciones, podríamos tener algo así repetido varias veces:

CÓDIGO

```
print("=== MENÚ DEL BLOG ===")
print("1. Ver posts")
print("2. Crear post")
print("3. Buscar post")
print("4. Salir")
```

Con una función, podemos guardar esa lógica:

CÓDIGO

```
def mostrar_menu():
    print("=== MENÚ DEL BLOG ===")
    print("1. Ver posts")
    print("2. Crear post")
    print("3. Buscar post")
    print("4. Salir")


mostrar_menu()
```

Ahora, cada vez que necesitemos mostrar el menú, solo llamamos a la función:

CÓDIGO

```
mostrar_menu()
```

6. Parámetros y argumentos

A veces necesitamos que una función trabaje con datos diferentes.

Por ejemplo, no queremos una función que salude siempre a la misma persona. Queremos que pueda saludar a distintos usuarios.

Para eso usamos parámetros.

CÓDIGO

```
def saludar_usuario(nombre):
    print(f"Hola, {nombre}. Bienvenido al blog.")
```

En este caso, `nombre` es un parámetro.

Un parámetro es una variable que la función espera recibir para poder trabajar.

Para ejecutar la función, le pasamos un valor:

CÓDIGO

```
saludar_usuario("Ana")
saludar_usuario("Pedro")
saludar_usuario("Lucía")
```

Resultado:

CÓDIGO

```
Hola, Ana. Bienvenido al blog.
Hola, Pedro. Bienvenido al blog.
Hola, Lucía. Bienvenido al blog.
```

Los valores `"Ana"`, `"Pedro"` y `"Lucía"` son argumentos.

Entonces:

- Parámetro: el nombre que usamos al definir la función.
- Argumento: el valor real que enviamos cuando llamamos a la función.

7. Ejemplo aplicado: formatear el título de un post

Supongamos que queremos mostrar los títulos del blog siempre con un mismo formato.

Podemos crear una función:

CÓDIGO

```
def mostrar_titulo_post(titulo):
    print(f"--- {titulo.upper()} ---")
```

Luego la usamos con distintos títulos:

CÓDIGO

```
mostrar_titulo_post("mi primer post")
mostrar_titulo_post("aprendiendo funciones")
mostrar_titulo_post("introducción a django")
```

Resultado:

CÓDIGO

```
--- MI PRIMER POST ---
--- APRENDIENDO FUNCIONES ---
--- INTRODUCCIÓN A DJANGO ---
```

La función es la misma, pero el dato cambia en cada llamada.

8. Funciones con más de un parámetro

Una función puede recibir más de un parámetro.

Por ejemplo, podemos mostrar un post con su título y su estado:

CÓDIGO

```
def mostrar_post(titulo, estado):
    print(f"[{estado}] {titulo}")
```

Llamamos a la función:

CÓDIGO

```
mostrar_post("Mi primer post", "publicado")
mostrar_post("Cómo instalar Django", "borrador")
mostrar_post("Aprendiendo Git", "archivado")
```

Resultado:

CÓDIGO

```
[publicado] Mi primer post
[borrador] Cómo instalar Django
[archivado] Aprendiendo Git
```

En la primera llamada:

CÓDIGO

```
mostrar_post("Mi primer post", "publicado")
```

Python asigna los valores así:

CÓDIGO

```
titulo = "Mi primer post"
estado = "publicado"
```

El orden importa. El primer argumento va al primer parámetro, el segundo argumento va al segundo parámetro, y así sucesivamente.

9. Argumentos con nombre

También podemos indicar explícitamente qué valor corresponde a cada parámetro.

CÓDIGO

```
def mostrar_post(titulo, estado):
    print(f"[{estado}] {titulo}")


mostrar_post(titulo="Mi primer post", estado="publicado")
```

Esto puede mejorar la claridad del código.

También permite cambiar el orden:

CÓDIGO

```
mostrar_post(estado="borrador", titulo="Vistas y templates")
```

Resultado:

CÓDIGO

```
[borrador] Vistas y templates
```

Usar argumentos con nombre puede ser útil cuando una función recibe varios datos.

10. Ejemplo aplicado: listar posts con una función

Ahora veamos una función que recibe una lista de posts y los muestra en pantalla.

CÓDIGO

```
posts = [
    {"id": 1, "titulo": "Mi primer post", "estado": "publicado"},
    {"id": 2, "titulo": "Aprendiendo funciones", "estado": "borrador"},
    {"id": 3, "titulo": "Introducción a Django", "estado": "archivado"}
]


def listar_posts(lista_posts):
    print("=== LISTADO DE POSTS ===")

    for post in lista_posts:
        print(f"{post['id']}. [{post['estado']}] {post['titulo']}")


listar_posts(posts)
```

Resultado:

CÓDIGO

```
=== LISTADO DE POSTS ===
1. [publicado] Mi primer post
2. [borrador] Aprendiendo funciones
3. [archivado] Introducción a Django
```

En este ejemplo:

- `lista_posts` es el parámetro.
- `posts` es el argumento.
- La función recorre la lista y muestra cada publicación.

La ventaja es que, si más adelante cambia la forma en que queremos mostrar los posts, solo modificamos la función `listar_posts()`.

11. Ejemplo aplicado: validar un título

También podemos usar funciones para validar datos.

CÓDIGO

```
def validar_titulo(titulo):
    if titulo == "":
        print("Error: el título no puede estar vacío.")
    else:
        print("Título válido.")
```

Probamos la función:

CÓDIGO

```
validar_titulo("Aprendiendo Python")
validar_titulo("")
```

Resultado:

CÓDIGO

```
Título válido.
Error: el título no puede estar vacío.
```

Más adelante veremos cómo hacer que una función devuelva un resultado usando `return`. Por ahora, nos alcanza con entender que una función puede recibir datos y ejecutar una acción.

12. Error común: olvidarse de llamar a la función

Un error frecuente es definir la función, pero nunca ejecutarla.

CÓDIGO

```
def mostrar_menu():
    print("1. Ver posts")
    print("2. Crear post")
    print("3. Salir")
```

Este código no muestra nada en pantalla porque la función solo fue definida.

Para ejecutarla, hay que llamarla:

CÓDIGO

```
mostrar_menu()
```

13. Error común: pasar menos argumentos de los necesarios

Si una función espera un dato, tenemos que enviarlo al llamarla.

CÓDIGO

```
def saludar_usuario(nombre):
    print(f"Hola, {nombre}")
```

Esto funciona:

CÓDIGO

```
saludar_usuario("Ana")
```

Pero esto genera un error:

CÓDIGO

```
saludar_usuario()
```

Python mostrará un error del tipo `TypeError`, porque falta un argumento obligatorio.

14. Error común: confundir variables locales

Las variables creadas dentro de una función solo existen dentro de esa función.

CÓDIGO

```
def crear_autor():
    autor = "Guido van Rossum"
    print(autor)


crear_autor()
```

Esto funciona porque `autor` se usa dentro de la función.

Pero esto genera un error:

CÓDIGO

```
def crear_autor():
    autor = "Guido van Rossum"


crear_autor()
print(autor)
```

La variable `autor` fue creada dentro de la función, por lo tanto no existe fuera de ella.

A esto se lo conoce como alcance de una variable.

15. Buenas prácticas para crear funciones

Usar nombres claros

El nombre de una función debería explicar qué hace.

No recomendado:

CÓDIGO

```
def x():
    print("Hola")
```

Mejor:

CÓDIGO

```
def mostrar_bienvenida():
    print("Bienvenido al blog")
```

Usar `snake_case`

En Python, los nombres de funciones se escriben en minúscula y con guiones bajos.

CÓDIGO

```
def mostrar_menu():
    print("Menú")
```

CÓDIGO

```
def crear_post():
    print("Crear post")
```

CÓDIGO

```
def listar_posts():
    print("Listado de posts")
```

Hacer que cada función tenga una tarea clara

Una función debería encargarse de una tarea específica.

Por ejemplo, estas funciones están bien separadas:

CÓDIGO

```
def mostrar_menu():
    print("1. Ver posts")
    print("2. Crear post")
    print("3. Salir")


def mostrar_bienvenida():
    print("Bienvenido al blog")


def mostrar_despedida():
    print("Gracias por usar el blog")
```

En cambio, una función llamada `hacer_todo()` no sería clara.

16. Ejemplo integrador

Veamos un ejemplo pequeño usando varias funciones.

CÓDIGO

```
def mostrar_bienvenida():
    print("Bienvenido al blog de Python")


def mostrar_menu():
    print("=== MENÚ ===")
    print("1. Ver posts")
    print("2. Crear post")
    print("3. Salir")



def mostrar_post(titulo, estado):
    print(f"[{estado}] {titulo}")


mostrar_bienvenida()
mostrar_menu()
mostrar_post("Mi primer post", "publicado")
mostrar_post("Funciones y parámetros", "borrador")
```

Resultado:

CÓDIGO

```
Bienvenido al blog de Python
=== MENÚ ===
1. Ver posts
2. Crear post
3. Salir
[publicado] Mi primer post
[borrador] Funciones y parámetros
```

Este programa todavía es simple, pero ya está mejor organizado que si escribiéramos todas las instrucciones sueltas.

Síntesis de la unidad

Una función es un bloque de código reutilizable que realiza una tarea específica.

En Python, las funciones se definen con `def`.

Definir una función no la ejecuta. Para usarla, hay que llamarla por su nombre.

Los parámetros son los datos que la función espera recibir.

Los argumentos son los valores reales que le pasamos al llamar a la función.

Las funciones ayudan a evitar repetición, organizar mejor el código y hacer que el programa sea más fácil de mantener.

En nuestro blog por consola, las funciones nos permiten separar tareas como mostrar el menú, listar posts, validar datos y mostrar información.

Retorno de valores y alcance de variables

Aprende a devolver datos desde tus funciones y a gestionar dónde viven tus variables (alcance local vs global) para construir programas robustos.

Funciones con `return` y alcance de variables

En la unidad anterior aprendimos a crear funciones para no repetir código. Vimos que una función puede agrupar instrucciones y ejecutarlas cuando la llamamos.

Hasta ahora, muchas de nuestras funciones hacían algo en pantalla, por ejemplo mostrar un menú o imprimir un mensaje. Pero en programación no siempre queremos mostrar datos. Muchas veces necesitamos que una función calcule, valide o prepare información para usarla después en otra parte del programa.

Para eso usamos `return`.

1. La diferencia entre `print` y `return`

Antes de avanzar, es importante diferenciar dos ideas:

- `print()` muestra información en pantalla.
- `return` devuelve un valor desde una función.

Veamos un ejemplo:

CÓDIGO

```
def mostrar_titulo(titulo):
    print(titulo)


mostrar_titulo("Mi primer post")
```

Esta función muestra el título en la consola. Pero el valor no queda disponible para usarlo después.

En cambio:

CÓDIGO

```
def obtener_titulo(titulo):
    return titulo


resultado = obtener_titulo("Mi primer post")
print(resultado)
```

En este caso, la función devuelve el valor `"Mi primer post"` y lo guardamos en la variable `resultado`.

Dicho de forma simple:

CÓDIGO

```
print()
```

sirve para mostrar.

CÓDIGO

```
return
```

sirve para entregar un resultado.

2. ¿Qué hace `return`?

La instrucción `return` permite que una función devuelva un valor.

Ejemplo:

CÓDIGO

```
def sumar(a, b):
    resultado = a + b
    return resultado


total = sumar(5, 3)

print(total)
```

Resultado:

CÓDIGO

```
8
```

En este ejemplo:

- La función recibe dos números.
- Los suma.
- Devuelve el resultado.
- Ese resultado se guarda en la variable `total`.

También podríamos escribirlo más corto:

CÓDIGO

```
def sumar(a, b):
    return a + b


total = sumar(5, 3)
print(total)
```

3. Ejemplo aplicado al blog: calcular tiempo de lectura

Imaginemos que queremos estimar el tiempo de lectura de un post.

Supongamos que una persona lee aproximadamente 200 palabras por minuto.

CÓDIGO

```
def calcular_tiempo_lectura(cantidad_palabras):
    tiempo = cantidad_palabras / 200
    return tiempo


tiempo_post = calcular_tiempo_lectura(600)

print(f"Tiempo estimado de lectura: {tiempo_post} minutos")
```

Resultado:

CÓDIGO

```
Tiempo estimado de lectura: 3.0 minutos
```

La función no imprime directamente el resultado. Solo lo calcula y lo devuelve.

Después, el programa decide qué hacer con ese valor.

4. Capturar el valor que devuelve una función

Para usar el valor que devuelve una función, tenemos que guardarlo en una variable.

CÓDIGO

```
def formatear_titulo(titulo):
    return titulo.upper()


titulo_formateado = formatear_titulo("aprendiendo funciones")

print(titulo_formateado)
```

Resultado:

CÓDIGO

```
APRENDIENDO FUNCIONES
```

Si llamamos a la función sin guardar el resultado, el valor se pierde:

CÓDIGO

```
def formatear_titulo(titulo):
    return titulo.upper()


formatear_titulo("aprendiendo funciones")
```

Este código se ejecuta, pero no veremos nada en pantalla porque no usamos `print()` ni guardamos el resultado.

Lo correcto sería:

CÓDIGO

```
titulo_formateado = formatear_titulo("aprendiendo funciones")
print(titulo_formateado)
```

5. `return` termina la función

Cuando Python encuentra un `return`, la función termina inmediatamente.

Cualquier código que esté después del `return`, dentro de la función, no se ejecuta.

CÓDIGO

```
def saludar():
    return "Hola"
    print("Este mensaje nunca se muestra")


mensaje = saludar()
print(mensaje)
```

Resultado:

CÓDIGO

```
Hola
```

La línea del `print()` dentro de la función no se ejecuta porque está después del `return`.

6. Salida anticipada con `return`

El `return` también puede usarse para cortar la ejecución de una función cuando algo no cumple una condición.

Esto es muy útil para validaciones.

CÓDIGO

```
def validar_titulo(titulo):
    if titulo == "":
        return "Error: el título no puede estar vacío."

    return "Título válido."


resultado = validar_titulo("")
print(resultado)
```

Resultado:

CÓDIGO

```
Error: el título no puede estar vacío.
```

Si el título está vacío, la función devuelve el error y termina.

Si el título no está vacío, continúa y devuelve `"Título válido."`.

CÓDIGO

```
resultado = validar_titulo("Mi primer post")
print(resultado)
```

Resultado:

CÓDIGO

```
Título válido.
```

7. Ejemplo aplicado: validar un post

Podemos crear una función que valide los datos básicos de un post.

CÓDIGO

```
def validar_post(titulo, contenido):
    if titulo == "":
        return "Error: el título no puede estar vacío."

    if contenido == "":
        return "Error: el contenido no puede estar vacío."

    return "Post válido."


resultado = validar_post("Aprendiendo Python", "Contenido del post")
print(resultado)
```

Resultado:

CÓDIGO

```
Post válido.
```

Probemos con un error:

CÓDIGO

```
resultado = validar_post("", "Contenido del post")
print(resultado)
```

Resultado:

CÓDIGO

```
Error: el título no puede estar vacío.
```

Este tipo de lógica será muy útil más adelante cuando trabajemos con formularios en Django.

8. Devolver valores booleanos

Muchas veces una función de validación no devuelve un mensaje, sino un valor booleano: `True` o `False`.

CÓDIGO

```
def titulo_es_valido(titulo):
    if titulo == "":
        return False

    return True
```

Podemos usar esa función dentro de un `if`:

CÓDIGO

```
titulo = "Mi primer post"

if titulo_es_valido(titulo):
    print("El post puede publicarse.")
else:
    print("El título no es válido.")
```

Resultado:

CÓDIGO

```
El post puede publicarse.
```

También se puede escribir de forma más directa:

CÓDIGO

```
def titulo_es_valido(titulo):
    return titulo != ""
```

Esta función devuelve `True` si el título no está vacío, y `False` si está vacío.

9. Devolver múltiples valores

Python permite devolver más de un valor desde una función.

Por ejemplo, podemos calcular dos datos de un post: cantidad de palabras y tiempo estimado de lectura.

CÓDIGO

```
def analizar_contenido(contenido):
    palabras = contenido.split()
    cantidad_palabras = len(palabras)
    tiempo_lectura = cantidad_palabras / 200

    return cantidad_palabras, tiempo_lectura
```

Llamamos a la función:

CÓDIGO

```
contenido = "Python es un lenguaje muy usado para crear aplicaciones web."

cantidad, tiempo = analizar_contenido(contenido)

print(f"Cantidad de palabras: {cantidad}")
print(f"Tiempo estimado: {tiempo} minutos")
```

Resultado:

CÓDIGO

```
Cantidad de palabras: 10
Tiempo estimado: 0.05 minutos
```

La función devuelve dos valores:

CÓDIGO

```
return cantidad_palabras, tiempo_lectura
```

Y al llamarla, los guardamos en dos variables:

CÓDIGO

```
cantidad, tiempo = analizar_contenido(contenido)
```

10. Ejemplo integrador con `return`

Veamos un ejemplo más completo usando funciones que devuelven valores.

CÓDIGO

```
def formatear_titulo(titulo):
    return titulo.strip().title()


def calcular_tiempo_lectura(contenido):
    palabras = contenido.split()
    cantidad_palabras = len(palabras)
    tiempo = cantidad_palabras / 200
    return tiempo


def crear_resumen_post(titulo, contenido):
    titulo_formateado = formatear_titulo(titulo)
    tiempo = calcular_tiempo_lectura(contenido)

    resumen = f"{titulo_formateado} - Tiempo de lectura: {tiempo} minutos"
    return resumen


resultado = crear_resumen_post(
    "   aprendiendo funciones en python   ",
    "En esta unidad aprendemos a usar funciones con return."
)

print(resultado)
```

Resultado:

CÓDIGO

```
Aprendiendo Funciones En Python - Tiempo de lectura: 0.045 minutos
```

En este ejemplo:

- Una función formatea el título.
- Otra calcula el tiempo de lectura.
- Otra arma un resumen usando los resultados anteriores.

Esto muestra una idea importante: las funciones pueden trabajar juntas.

Alcance de variables

Además de entender `return`, necesitamos entender dónde existen las variables.

A esto se lo llama alcance o `scope`.

El alcance define desde qué parte del programa podemos usar una variable.

11. Variables globales

Una variable global es una variable creada fuera de una función.

CÓDIGO

```
nombre_blog = "Blog de Python"


def mostrar_blog():
    print(nombre_blog)


mostrar_blog()
```

Resultado:

CÓDIGO

```
Blog de Python
```

La variable `nombre_blog` fue creada fuera de la función, pero la función puede leerla.

12. Variables locales

Una variable local es una variable creada dentro de una función.

CÓDIGO

```
def crear_autor():
    autor = "Guido van Rossum"
    print(autor)


crear_autor()
```

Resultado:

CÓDIGO

```
Guido van Rossum
```

Pero si intentamos usar esa variable fuera de la función, tendremos un error:

CÓDIGO

```
def crear_autor():
    autor = "Guido van Rossum"


crear_autor()
print(autor)
```

Python mostrará un error porque `autor` solo existe dentro de la función.

La variable local nace dentro de la función y desaparece cuando la función termina.

13. Ejemplo visual de alcance

CÓDIGO

```
mensaje = "Variable global"


def mostrar_mensaje():
    mensaje_local = "Variable local"
    print(mensaje)
    print(mensaje_local)


mostrar_mensaje()
```

Resultado:

CÓDIGO

```
Variable global
Variable local
```

Pero esto no funciona:

CÓDIGO

```
mensaje = "Variable global"


def mostrar_mensaje():
    mensaje_local = "Variable local"


mostrar_mensaje()
print(mensaje_local)
```

`mensaje_local` fue creada dentro de la función, por lo tanto no existe fuera de ella.

14. Sombreado de variables

Puede pasar que tengamos una variable global y una variable local con el mismo nombre.

CÓDIGO

```
usuario = "Ana"


def mostrar_usuario():
    usuario = "Pedro"
    print(usuario)


mostrar_usuario()
print(usuario)
```

Resultado:

CÓDIGO

```
Pedro
Ana
```

Dentro de la función, Python usa la variable local `usuario`.

Fuera de la función, Python usa la variable global `usuario`.

A esto se lo conoce como sombreado o `shadowing`: una variable local “tapa” a una variable global con el mismo nombre dentro de la función.

15. La palabra clave `global`

Existe una palabra clave llamada `global` que permite modificar una variable global desde dentro de una función.

CÓDIGO

```
contador_posts = 0


def sumar_post():
    global contador_posts
    contador_posts = contador_posts + 1


sumar_post()
sumar_post()

print(contador_posts)
```

Resultado:

CÓDIGO

```
2
```

En este caso, `global contador_posts` le indica a Python que queremos modificar la variable global, no crear una variable local nueva.

Sin embargo, conviene usar `global` con mucho cuidado.

En general, es mejor evitar modificar variables globales dentro de funciones, porque puede volver el programa más difícil de entender.

16. Mejor alternativa: usar parámetros y `return`

En lugar de modificar una variable global directamente, suele ser mejor pasar el valor como argumento y devolver el nuevo resultado.

No recomendado para principiantes:

CÓDIGO

```
contador_posts = 0


def sumar_post():
    global contador_posts
    contador_posts = contador_posts + 1
```

Mejor:

CÓDIGO

```
def sumar_post(contador_actual):
    nuevo_contador = contador_actual + 1
    return nuevo_contador


contador_posts = 0

contador_posts = sumar_post(contador_posts)
contador_posts = sumar_post(contador_posts)

print(contador_posts)
```

Resultado:

CÓDIGO

```
2
```

Esta versión es más clara porque la función recibe un dato, trabaja con él y devuelve un resultado.

17. Error común: confundir `print` con `return`

Este es uno de los errores más frecuentes.

CÓDIGO

```
def calcular_total_posts(posts):
    print(len(posts))


posts = ["Post 1", "Post 2", "Post 3"]

total = calcular_total_posts(posts)

print(total)
```

Resultado:

CÓDIGO

```
3
None
```

¿Por qué aparece `None`?

Porque la función imprime la cantidad de posts, pero no devuelve nada.

Cuando una función no tiene `return`, Python devuelve automáticamente `None`.

La versión correcta sería:

CÓDIGO

```
def calcular_total_posts(posts):
    return len(posts)


posts = ["Post 1", "Post 2", "Post 3"]

total = calcular_total_posts(posts)

print(total)
```

Resultado:

CÓDIGO

```
3
```

18. Error común: no guardar el retorno

Otro error común es llamar a la función, pero no guardar el resultado.

CÓDIGO

```
def formatear_titulo(titulo):
    return titulo.upper()


formatear_titulo("mi primer post")
```

La función devuelve el título en mayúsculas, pero no hacemos nada con ese resultado.

Lo correcto sería:

CÓDIGO

```
def formatear_titulo(titulo):
    return titulo.upper()


titulo = formatear_titulo("mi primer post")

print(titulo)
```

Resultado:

CÓDIGO

```
MI PRIMER POST
```

19. Error común: modificar variables globales sin necesidad

Las variables globales pueden parecer cómodas, pero si muchas funciones las modifican, el programa se vuelve difícil de seguir.

No recomendado:

CÓDIGO

```
posts = []


def agregar_post(titulo):
    posts.append(titulo)


agregar_post("Mi primer post")
agregar_post("Funciones en Python")

print(posts)
```

Funciona, pero la función modifica directamente una lista que vive afuera.

Una alternativa más clara es recibir la lista como parámetro:

CÓDIGO

```
def agregar_post(lista_posts, titulo):
    lista_posts.append(titulo)
    return lista_posts


posts = []

posts = agregar_post(posts, "Mi primer post")
posts = agregar_post(posts, "Funciones en Python")

print(posts)
```

Resultado:

CÓDIGO

```
['Mi primer post', 'Funciones en Python']
```

Síntesis de la unidad

`return` permite que una función devuelva un valor.

`print()` muestra información en pantalla, pero no devuelve un dato para seguir trabajando.

Para usar el valor retornado por una función, normalmente lo guardamos en una variable.

Cuando Python encuentra un `return`, la función termina.

Una función puede devolver un valor, varios valores o valores booleanos como `True` y `False`.

El alcance de una variable define en qué parte del programa puede usarse.

Las variables globales se crean fuera de las funciones.

Las variables locales se crean dentro de una función y solo existen allí.

La palabra clave `global` permite modificar variables globales dentro de una función, pero conviene usarla con cuidado.

En general, es mejor trabajar con parámetros y `return`, porque hace que el código sea más claro, ordenado y fácil de mantener.

Desafío de Código

En este ejercicio, vamos a modularizar el procesamiento del blog. Crearemos una función que limpie los títulos de los posts y nos diga cuántos hay, y otra que actualice el contador global del sistema.

Validación de datos de entrada para evitar valores incorrectos

Aprende a diseñar reglas lógicas para proteger tu blog de datos incorrectos, asegurando que cada post tenga el formato, tipo y contenido adecuado antes de ser procesado.

Validación de datos de entrada

En las unidades anteriores aprendimos a crear funciones, usar parámetros y devolver resultados con `return`.

Ahora vamos a sumar una idea fundamental: validar los datos antes de usarlos.

Cuando un usuario interactúa con nuestro programa, puede ingresar datos incorrectos. Por ejemplo:

- Un título vacío.
- Un estado de publicación que no existe.
- Un número negativo donde esperamos una cantidad positiva.
- Texto donde esperamos un número.
- Un contenido demasiado corto para ser un post real.

Si no validamos esos datos, nuestro programa puede fallar o guardar información incorrecta.

Validar significa revisar que un dato cumpla ciertas reglas antes de procesarlo o guardarlo.

En nuestro blog por consola, validar datos nos ayuda a crear posts más consistentes y a evitar errores más adelante.

1. ¿Qué es la validación de datos?

La validación de datos es el proceso de comprobar que la información ingresada cumple con las reglas esperadas por nuestro programa.

Por ejemplo, si queremos crear un post, podríamos definir estas reglas:

- El título no puede estar vacío.
- El título debe tener al menos 3 caracteres.
- El contenido no puede estar vacío.
- El estado debe estar dentro de una lista de estados permitidos.
- La calificación debe ser un número entre 1 y 5.

Si los datos cumplen las reglas, el programa puede continuar.

Si no las cumplen, debemos mostrar un mensaje claro y evitar que el dato incorrecto se guarde.

2. Validar no es lo mismo que limpiar datos

Antes de seguir, conviene diferenciar dos ideas:

Validar es decidir si un dato es aceptable o no.

Limpiar datos es modificar un dato para dejarlo en un formato más prolijo.

Por ejemplo:

CÓDIGO

```
titulo = "   mi primer post   "

titulo_limpio = titulo.strip()
print(titulo_limpio)
```

Resultado:

CÓDIGO

```
mi primer post
```

En este caso usamos `.strip()` para eliminar espacios al inicio y al final.

Eso es limpieza.

Luego podríamos validar:

CÓDIGO

```
if titulo_limpio == "":
    print("Error: el título no puede estar vacío.")
```

En muchos programas reales hacemos ambas cosas:

1. Primero limpiamos un poco el dato.
2. Después validamos si es aceptable.

3. Validar campos obligatorios

Un campo obligatorio es un dato que no puede faltar.

En un blog, el título de un post es obligatorio. Un post sin título no tendría mucho sentido.

Podemos validar un título así:

CÓDIGO

```
def validar_titulo(titulo):
    titulo = titulo.strip()

    if titulo == "":
        return False

    return True
```

Probamos la función:

CÓDIGO

```
print(validar_titulo("Mi primer post"))
print(validar_titulo(""))
print(validar_titulo("   "))
```

Resultado:

CÓDIGO

```
True
False
False
```

La función devuelve:

- `True` si el título es válido.
- `False` si el título está vacío.

El uso de `.strip()` es importante porque un texto con espacios también debería considerarse vacío.

CÓDIGO

```
"   ".strip()
```

devuelve:

CÓDIGO

```
""
```

4. Validar longitud de texto

A veces un dato existe, pero no cumple con la longitud esperada.

Por ejemplo:

- Un título de 1 letra puede ser demasiado corto.
- Un título de 300 caracteres puede ser demasiado largo.

Podemos definir una regla:

CÓDIGO

```
def validar_largo_titulo(titulo):
    titulo = titulo.strip()

    if len(titulo) < 3:
        return False

    if len(titulo) > 100:
        return False

    return True
```

Probamos:

CÓDIGO

```
print(validar_largo_titulo("A"))
print(validar_largo_titulo("Funciones en Python"))
```

Resultado:

CÓDIGO

```
False
True
```

También podemos devolver un mensaje más claro:

CÓDIGO

```
def validar_largo_titulo(titulo):
    titulo = titulo.strip()

    if len(titulo) < 3:
        return "Error: el título debe tener al menos 3 caracteres."

    if len(titulo) > 100:
        return "Error: el título no puede superar los 100 caracteres."

    return "Título válido."
```

Uso:

CÓDIGO

```
resultado = validar_largo_titulo("A")
print(resultado)
```

Resultado:

CÓDIGO

```
Error: el título debe tener al menos 3 caracteres.
```

5. Validar tipos de datos

El tipo de dato también importa.

Python distingue entre distintos tipos, como:

CÓDIGO

```
str
int
float
bool
list
dict
```

Si una función espera un número, no debería recibir un texto.

Por ejemplo, imaginemos una función para registrar la cantidad de minutos de lectura de un post:

CÓDIGO

```
def validar_tiempo_lectura(minutos):
    if not isinstance(minutos, int):
        return False

    if minutos <= 0:
        return False

    return True
```

Probamos:

CÓDIGO

```
print(validar_tiempo_lectura(5))
print(validar_tiempo_lectura("cinco"))
print(validar_tiempo_lectura(-2))
```

Resultado:

CÓDIGO

```
True
False
False
```

En este ejemplo validamos dos cosas:

- Que `minutos` sea un número entero.
- Que sea mayor que cero.

6. Importante: `input()` siempre devuelve texto

Cuando usamos `input()`, Python siempre nos devuelve un string.

Aunque el usuario escriba un número, primero llega como texto.

CÓDIGO

```
edad = input("Ingresá tu edad: ")

print(type(edad))
```

Si el usuario escribe `25`, Python lo guarda como:

CÓDIGO

```
<class 'str'>
```

Por eso, si necesitamos un número, debemos convertirlo:

CÓDIGO

```
edad = int(edad)
```

Pero esa conversión puede fallar si el usuario escribe algo como `"hola"`.

Por eso podemos combinar validación con `try-except`.

CÓDIGO

```
def pedir_calificacion():
    try:
        calificacion = int(input("Ingresá una calificación del 1 al 5: "))
    except ValueError:
        print("Error: la calificación debe ser un número.")
        return None

    return calificacion
```

Luego podemos usar esa función:

CÓDIGO

```
calificacion = pedir_calificacion()

if calificacion is not None:
    print(f"Calificación ingresada: {calificacion}")
```

7. Validar rangos numéricos

A veces un dato tiene el tipo correcto, pero está fuera del rango permitido.

Por ejemplo, si la calificación de un post debe ir del 1 al 5, el número `10` no es válido.

CÓDIGO

```
def validar_calificacion(calificacion):
    if calificacion < 1 or calificacion > 5:
        return False

    return True
```

Probamos:

CÓDIGO

```
print(validar_calificacion(5))
print(validar_calificacion(1))
print(validar_calificacion(10))
print(validar_calificacion(0))
```

Resultado:

CÓDIGO

```
True
True
False
False
```

También podemos devolver mensajes:

CÓDIGO

```
def validar_calificacion(calificacion):
    if calificacion < 1 or calificacion > 5:
        return "Error: la calificación debe estar entre 1 y 5."

    return "Calificación válida."
```

8. Validar opciones permitidas

En un blog, puede ser útil limitar los estados disponibles de una publicación.

Por ejemplo:

CÓDIGO

```
estados_permitidos = ["borrador", "publicado", "archivado"]
```

Si el usuario intenta crear un post con un estado que no está en esa lista, deberíamos rechazarla.

CÓDIGO

```
def validar_estado(estado):
    estados_permitidos = ["borrador", "publicado", "archivado"]

    if estado not in estados_permitidos:
        return False

    return True
```

Probamos:

CÓDIGO

```
print(validar_estado("publicado"))
print(validar_estado("pendiente"))
```

Resultado:

CÓDIGO

```
True
False
```

También podríamos devolver un mensaje más útil:

CÓDIGO

```
def validar_estado(estado):
    estados_permitidos = ["borrador", "publicado", "archivado"]

    if estado not in estados_permitidos:
        return f"Error: el estado '{estado}' no está permitido."

    return "Estado válido."
```

9. Validar formato de datos

Algunos datos deben tener un formato específico.

Por ejemplo, un email debería tener al menos un `@`.

Una validación básica podría ser:

CÓDIGO

```
def validar_email(email):
    email = email.strip()

    if "@" not in email:
        return False

    return True
```

Probamos:

CÓDIGO

```
print(validar_email("usuario@email.com"))
print(validar_email("usuarioemail.com"))
```

Resultado:

CÓDIGO

```
True
False
```

Esta validación es simple. No detecta todos los casos posibles, pero sirve como primer acercamiento.

Más adelante, en proyectos reales, podemos usar validaciones más completas. Pero para empezar, alcanza con entender la idea: revisar si el dato cumple un patrón esperado.

10. Mensajes de error útiles

Un mensaje de error debe ayudar al usuario a entender qué pasó.

No es muy útil mostrar esto:

CÓDIGO

```
Entrada inválida.
```

Ese mensaje no explica qué dato está mal ni cómo corregirlo.

Es mejor mostrar algo más específico:

CÓDIGO

```
Error: el título debe tener al menos 3 caracteres.
```

Un buen mensaje de error debería indicar:

- Qué campo falló.
- Qué regla no se cumplió.
- Cómo puede corregirse.

Ejemplos:

CÓDIGO

```
Error: el título no puede estar vacío.
```

CÓDIGO

```
Error: la calificación debe estar entre 1 y 5.
```

CÓDIGO

```
Error: el estado debe ser borrador, publicado o archivado.
```

11. Ejemplo completo: validar un post

Ahora vamos a crear una función que valide un post completo.

El post tendrá:

- `titulo`
- `contenido`
- `estado`
- `calificacion`

CÓDIGO

```
def validar_post(titulo, contenido, estado, calificacion):
    estados_permitidos = ["borrador", "publicado", "archivado"]

    titulo = titulo.strip()
    contenido = contenido.strip()
    estado = estado.strip()

    if titulo == "":
        return False, "Error: el título no puede estar vacío."

    if len(titulo) < 3:
        return False, "Error: el título debe tener al menos 3 caracteres."

    if len(titulo) > 100:
        return False, "Error: el título no puede superar los 100 caracteres."

    if contenido == "":
        return False, "Error: el contenido no puede estar vacío."

    if len(contenido) < 10:
        return False, "Error: el contenido debe tener al menos 10 caracteres."

    if estado not in estados_permitidos:
        return False, "Error: el estado no está permitido."

    if not isinstance(calificacion, int):
        return False, "Error: la calificación debe ser un número entero."

    if calificacion < 1 or calificacion > 5:
        return False, "Error: la calificación debe estar entre 1 y 5."

    return True, "Post válido."
```

Probamos con datos correctos:

CÓDIGO

```
es_valido, mensaje = validar_post(
    "Funciones en Python",
    "En esta unidad aprendemos a validar datos.",
    "publicado",
    5
)

print(es_valido)
print(mensaje)
```

Resultado:

CÓDIGO

```
True
Post válido.
```

Probamos con un error:

CÓDIGO

```
es_valido, mensaje = validar_post(
    "",
    "Contenido de prueba",
    "publicado",
    5
)

print(es_valido)
print(mensaje)
```

Resultado:

CÓDIGO

```
False
Error: el título no puede estar vacío.
```

En este ejemplo devolvemos dos valores:

CÓDIGO

```
return False, "Error: el título no puede estar vacío."
```

El primer valor indica si el post es válido.

El segundo valor contiene un mensaje para mostrar al usuario.

12. Crear un post solo si los datos son válidos

Podemos usar la función `validar_post()` antes de crear el post.

CÓDIGO

```
def crear_post(titulo, contenido, estado, calificacion):
    es_valido, mensaje = validar_post(titulo, contenido, estado, calificacion)

    if not es_valido:
        print(mensaje)
        return None

    nuevo_post = {
        "titulo": titulo.strip(),
        "contenido": contenido.strip(),
        "estado": estado.strip(),
        "calificacion": calificacion
    }

    return nuevo_post
```

Uso:

CÓDIGO

```
post = crear_post(
    "Aprendiendo validaciones",
    "Validar datos nos ayuda a evitar errores.",
    "publicado",
    4
)

print(post)
```

Resultado:

CÓDIGO

```
{
    'titulo': 'Aprendiendo validaciones',
    'contenido': 'Validar datos nos ayuda a evitar errores.',
    'estado': 'publicado',
    'calificacion': 4
}
```

Si el dato no es válido:

CÓDIGO

```
post = crear_post(
    "",
    "Validar datos nos ayuda a evitar errores.",
    "publicado",
    4
)

print(post)
```

Resultado:

CÓDIGO

```
Error: el título no puede estar vacío.
None
```

Como el título estaba vacío, la función no creó el post.

13. Ejemplo con entrada del usuario

Ahora podemos combinar `input()`, `try-except` y validación.

CÓDIGO

```
def pedir_calificacion():
    try:
        calificacion = int(input("Ingresá una calificación del 1 al 5: "))
    except ValueError:
        print("Error: la calificación debe ser un número.")
        return None

    return calificacion


titulo = input("Ingresá el título del post: ")
contenido = input("Ingresá el contenido del post: ")
estado = input("Ingresá el estado del post: ")
calificacion = pedir_calificacion()



if calificacion is not None:
    post = crear_post(titulo, contenido, estado, calificacion)

    if post is not None:
        print("Post creado correctamente:")
        print(post)
```

Este ejemplo todavía es simple, pero ya tiene una estructura más segura:

- Pide datos.
- Convierte la calificación a número.
- Valida los datos.
- Crea el post solo si todo es válido.

14. Aplicación en Django

Aunque ahora estamos trabajando con Python por consola, este tema será muy importante cuando lleguemos a Django.

En Django, los formularios también validan datos.

Por ejemplo, cuando un usuario completa un formulario para crear un post, Django puede revisar:

- Si el título está vacío.
- Si el email tiene formato válido.
- Si un campo supera la longitud máxima.
- Si una opción pertenece a una lista permitida.

Si los datos son correctos, Django permite guardarlos.

Si hay errores, Django devuelve el formulario con mensajes para que el usuario pueda corregirlos.

Por eso es importante aprender validaciones ahora. Más adelante, Django hará muchas de estas tareas por nosotros, pero entender la lógica nos ayuda a saber qué está pasando.

15. Errores comunes al validar datos

Validar demasiado tarde

No conviene usar los datos primero y validarlos después.

No recomendado:

CÓDIGO

```
titulo = ""

print(f"Creando post: {titulo}")

if titulo == "":
    print("Error: el título no puede estar vacío.")
```

Mejor:

CÓDIGO

```
titulo = ""

if titulo == "":
    print("Error: el título no puede estar vacío.")
else:
    print(f"Creando post: {titulo}")
```

Mostrar mensajes poco claros

No recomendado:

CÓDIGO

```
print("Error.")
```

Mejor:

CÓDIGO

```
print("Error: el título no puede estar vacío.")
```

Confiar en que el usuario siempre escribe bien

No recomendado:

CÓDIGO

```
calificacion = int(input("Ingresá una calificación: "))
```

Si el usuario escribe `"cinco"`, el programa falla.

Mejor:

CÓDIGO

```
try:
    calificacion = int(input("Ingresá una calificación: "))
except ValueError:
    print("Error: la calificación debe ser un número.")
```

Repetir validaciones en muchos lugares

Si validamos el título en varias partes del programa copiando y pegando código, después será más difícil mantenerlo.

Mejor crear una función:

CÓDIGO

```
def validar_titulo(titulo):
    titulo = titulo.strip()

    if titulo == "":
        return False

    if len(titulo) < 3:
        return False

    return True
```

Y usarla cada vez que sea necesario.

Síntesis de la unidad

Validar datos significa comprobar que la información ingresada cumple con ciertas reglas antes de usarla o guardarla.

En un blog, podemos validar títulos, contenidos, estados, calificaciones y otros datos.

Las validaciones más comunes son:

- Validar que un campo obligatorio no esté vacío.
- Validar el tipo de dato.
- Validar la longitud de un texto.
- Validar rangos numéricos.
- Validar opciones permitidas.
- Validar formatos simples, como un email con `@`.

Los mensajes de error deben ser claros y ayudar al usuario a corregir el problema.

Combinar funciones, `return`, `try-except` y validaciones nos permite escribir programas más seguros, ordenados y fáciles de mantener.

Más adelante, en Django, estas ideas aparecerán nuevamente cuando trabajemos con formularios y modelos.

Manejo de errores con try-except en funciones del blog

Aprende a proteger tu código de Python para que no se detenga ante errores inesperados, usando bloques try-except para manejar excepciones de forma profesional.

Manejo de errores con `try-except`

Hasta ahora, escribimos código asumiendo que todo va a salir bien: que el usuario va a ingresar números cuando se le pidan, que los datos van a tener el formato esperado y que siempre vamos a encontrar la información que buscamos.

Pero en el mundo real los programas pueden fallar. Por ejemplo, en nuestro blog por consola podría pasar que:

- El usuario escriba texto cuando se espera un número.
- Se intente buscar un dato que no existe.
- Se quiera dividir por cero.
- Una función reciba información incompleta.

Cuando ocurre un error durante la ejecución, Python detiene el programa. A esto normalmente lo llamamos “crash”.

Para evitar que el programa se cierre de golpe, usamos el manejo de excepciones con `try-except`.

1. ¿Qué es una excepción?

Una excepción es un error que ocurre mientras el programa se está ejecutando.

No es un error de sintaxis, como olvidarse un paréntesis o escribir mal una palabra clave. Es un problema que aparece cuando Python intenta realizar una acción y no puede completarla.

Por ejemplo:

CÓDIGO

```
edad = int("hola")
```

Este código no tiene un error de escritura, pero falla al ejecutarse porque `"hola"` no puede convertirse en un número entero.

Python mostrará un error del tipo:

CÓDIGO

```
ValueError
```

2. Estructura básica de `try-except`

La estructura `try-except` nos permite intentar ejecutar una parte del código y definir qué hacer si ocurre un error.

CÓDIGO

```
try:
    edad = int(input("Ingresá tu edad: "))
    print(f"Tu edad es {edad}")
except ValueError:
    print("Error: tenés que ingresar un número válido.")
```

En este ejemplo:

- El bloque `try` contiene el código que podría fallar.
- El bloque `except ValueError` se ejecuta solamente si ocurre un error al convertir el texto a número.

De esta manera, el programa no se rompe. En lugar de mostrar un error técnico, muestra un mensaje comprensible para el usuario.

3. Ejemplo aplicado al blog: pedir el ID de un post

Imaginemos que el usuario quiere ver un post ingresando su ID.

CÓDIGO

```
posts = [
    {"id": 1, "titulo": "Primer post", "contenido": "Bienvenidos al blog"},
    {"id": 2, "titulo": "Python básico", "contenido": "Hoy aprendemos listas"},
    {"id": 3, "titulo": "Diccionarios", "contenido": "Los diccionarios guardan datos
con claves"}
]

try:
    id_buscado = int(input("Ingresá el ID del post que querés ver: "))

    for post in posts:
        if post["id"] == id_buscado:
            print(f"Título: {post['titulo']}")
            print(f"Contenido: {post['contenido']}")
            break
    else:
        print("No existe un post con ese ID.")

except ValueError:
    print("Error: el ID debe ser un número.")
```

En este caso, el programa controla dos situaciones:

1. Si el usuario ingresa algo que no es un número, se muestra un mensaje de error.
2. Si el número no coincide con ningún post, se informa que el post no existe.

4. Captura de excepciones concretas

Es una buena práctica evitar capturar cualquier error con un `except` vacío.

No se recomienda hacer esto:

CÓDIGO

```
try:
    edad = int(input("Ingresá tu edad: "))
except:
    print("Ocurrió un error.")
```

Aunque funciona, el problema es que captura cualquier tipo de error. Eso puede ocultar problemas que no esperábamos y hacer que sea más difícil encontrar fallos en el código.

Es mejor indicar qué error esperamos:

CÓDIGO

```
try:
    edad = int(input("Ingresá tu edad: "))
except ValueError:
    print("Tenés que ingresar un número válido.")
```

Algunas excepciones comunes son:

- **ValueError**

Ocurre cuando el valor no puede convertirse al tipo esperado.

CÓDIGO

```
numero = int("hola")
```

- **KeyError**

Ocurre cuando intentamos acceder a una clave que no existe en un diccionario.

CÓDIGO

```
post = {
    "titulo": "Mi primer post",
    "contenido": "Contenido del post"
}

print(post["autor"])
```

- **ZeroDivisionError**

Ocurre cuando intentamos dividir por cero.

CÓDIGO

```
resultado = 10 / 0
```

5. Ejemplo con `KeyError`

Supongamos que tenemos un diccionario que representa un post, pero no estamos seguros de que tenga todos los datos.

CÓDIGO

```
post = {
    "titulo": "Aprendiendo Python",
    "contenido": "Hoy vemos manejo de errores"
}

try:
    print(f"Autor: {post['autor']}")
except KeyError:
    print("Error: el post no tiene autor cargado.")
```

En este caso, el programa intenta mostrar el autor. Como la clave `"autor"` no existe, se ejecuta el bloque `except`.

6. Uso de `else`

El bloque `else` se ejecuta solamente si no ocurrió ningún error dentro del `try`.

CÓDIGO

```
try:
    edad = int(input("Ingresá tu edad: "))
except ValueError:
    print("Error: tenés que ingresar un número.")
else:
    print(f"Edad guardada correctamente: {edad}")
```

Este código se puede leer así:

- Intentá convertir la edad a número.
- Si falla, mostrás un error.
- Si no falla, confirmás que la edad se guardó correctamente.

7. Uso de `finally`

El bloque `finally` se ejecuta siempre, haya ocurrido un error o no.

CÓDIGO

```
try:
    numero = int(input("Ingresá un número: "))
    print(f"El número ingresado es {numero}")
except ValueError:
    print("Error: tenés que ingresar un número válido.")
finally:
    print("Fin de la operación.")
```

El mensaje `"Fin de la operación."` se mostrará siempre.

`finally` suele usarse para tareas de cierre o limpieza, como cerrar archivos, finalizar conexiones o mostrar mensajes finales.

8. Ejemplo completo: crear un post con validación básica

Podemos usar `try-except` para controlar errores al crear un post en nuestro blog.

CÓDIGO

```
def crear_post():
    try:
        id_post = int(input("Ingresá el ID del post: "))
        titulo = input("Ingresá el título del post: ")
        contenido = input("Ingresá el contenido del post: ")

        if titulo == "":
            print("Error: el título no puede estar vacío.")
            return

        if contenido == "":
            print("Error: el contenido no puede estar vacío.")
            return

    except ValueError:
        print("Error: el ID debe ser un número.")
    else:
        nuevo_post = {
            "id": id_post,
            "titulo": titulo,
            "contenido": contenido
        }

        print("Post creado correctamente:")
        print(nuevo_post)
    finally:
        print("Operación finalizada.")
```

Llamamos a la función:

CÓDIGO

```
crear_post()
```

Este ejemplo combina:

- `try`, para intentar ejecutar código que puede fallar.
- `except`, para manejar un error concreto.
- `else`, para ejecutar código solo si todo salió bien.
- `finally`, para ejecutar una acción final siempre.

9. Errores comunes a evitar

Usar `except: pass`

No es recomendable silenciar errores de esta forma:

CÓDIGO

```
try:
    edad = int(input("Ingresá tu edad: "))
except ValueError:
    pass
```

El problema es que el programa no informa qué ocurrió. El error queda oculto y después puede ser más difícil entender por qué algo no funciona.

Es mejor mostrar un mensaje claro:

CÓDIGO

```
try:
    edad = int(input("Ingresá tu edad: "))
except ValueError:
    print("Error: tenés que ingresar un número válido.")
```

Encerrar todo el programa en un `try`

Tampoco conviene poner todo el código dentro de un único `try`.

No recomendado:

CÓDIGO

```
try:
    # muchas líneas de código
    # muchas funciones
    # muchas operaciones distintas
except ValueError:
    print("Ocurrió un error.")
```

Es mejor usar `try-except` solo en las partes que realmente pueden fallar, como conversiones de datos, búsqueda de claves o lectura de archivos.

Conclusión

El manejo de errores con `try-except` permite que nuestros programas sean más seguros y amigables para el usuario.

En lugar de dejar que Python detenga el programa ante un error, podemos anticiparnos a posibles fallos y mostrar mensajes claros.

En nuestro blog por consola, esto nos ayuda a validar datos, evitar cierres inesperados y preparar una lógica más parecida a la que luego vamos a necesitar en aplicaciones web.

Desafío

En este ejercicio, protegerás una función que actualiza las estadísticas de tu blog. Si el sistema recibe un dato corrupto (letras en vez de números), el blog debe seguir funcionando y avisar al administrador.

PRE-ENTREGA DEL PROYECTO

Actividad evaluable · parte del proyecto integrador

Preentrega 4: Sistema de blog modular, validado y más robusto

Refactorizar el sistema de blog por consola usando funciones, validaciones y manejo básico de errores para mejorar la organización y robustez del programa.

Preentrega 4: Sistema de blog modular y validado

¡Felicidades por llegar al final del módulo!

En el módulo anterior, construiste un sistema de blog por consola que podía mostrar un menú, listar posts, buscar publicaciones por título, filtrar por tag y finalizar su ejecución correctamente.

Ahora vas a mejorar ese programa aplicando uno de los conceptos más importantes de la programación: **la modularización mediante funciones**.

Esto significa que vas a tomar el código que ya funciona y reorganizarlo en partes más pequeñas, claras y reutilizables.

Contexto de la cadena de entregables

En el **Módulo 1**, configuraste tu entorno y ejecutaste tus primeros scripts en Python.

En el **Módulo 2**, modelaste la estructura de datos del blog usando listas, diccionarios, tuplas y sets.

En el **Módulo 3**, usaste esas estructuras para crear un sistema interactivo por consola con menú, búsqueda, filtrado y salida controlada.

Ahora, en el **Módulo 4**, vas a refactorizar ese sistema usando funciones, validaciones y manejo básico de errores.

La idea no es crear un sistema nuevo desde cero, sino **mejorar el sistema de blog que ya venías construyendo**.

Objetivo de aprendizaje de la preentrega

El objetivo de esta preentrega es consolidar la capacidad de organizar un programa en funciones pequeñas, claras y reutilizables.

Vas a practicar:

Modularización

Separar el código en funciones que tengan responsabilidades concretas.

Parámetros

Enviar datos a una función para que pueda trabajar con ellos.

Retorno de valores

Usar `return` cuando una función necesite devolver una opción, un resultado o una validación.

Validaciones

Comprobar que los posts tengan la estructura esperada antes de usarlos.

Manejo de errores

Evitar que el programa se cierre si el usuario ingresa un dato incorrecto.

Organización del programa

Usar el bloque `if __name__ == "__main__":` para ordenar la ejecución principal del sistema.

Qué construir

Debés crear el código de una versión refactorizada del sistema de blog por consola.

El programa puede llamarse:

CÓDIGO

```
sistema_blog_funciones.py
```

También podés mantener el nombre:

CÓDIGO

```
sistema_blog.py
```

Lo importante es que el código final esté organizado en funciones y continúe trabajando sobre la estructura de datos del blog creada en los módulos anteriores.

La entrega no requiere adjuntar un archivo `.py`. Deberás entregar el **código completo en formato texto**, como si fuera el contenido del archivo.

Datos iniciales

El programa debe mantener las estructuras de datos trabajadas en los módulos anteriores:

- `perfil_autor`
- `estados_post`
- `etiquetas_blog`
- `posts`

Cada post debe tener, como mínimo, las siguientes claves:

- `id`
- `titulo`
- `contenido`

- `autor`
- `tags`
- `estado`

La clave `"autor"` debe seguir usando el diccionario `perfil_autor`, no un texto simple.

Además, para probar las validaciones, al menos **un post debe tener datos incompletos o incorrectos**.

Por ejemplo, puede faltar una clave, tener un título vacío, tener `tags` con un tipo de dato incorrecto o tener un estado que no pertenezca a `estados_post`.

Funciones requeridas

El programa debe estructurarse usando funciones. No hace falta que todas tengan exactamente la misma implementación, pero sí deben cumplir estas responsabilidades.

Función para el menú

Debe existir una función encargada de mostrar el menú y retornar la opción elegida por el usuario.

El menú debe permitir:

CÓDIGO

```
--- MENU DEL BLOG ---
1. Ver todos los posts
2. Buscar por titulo
3. Filtrar por tag
4. Validar posts
5. Salir
```

Esta función debe usar `try-except` para evitar que el programa se cierre si el usuario ingresa texto en lugar de un número.

Función `listar_posts(lista)`

Debe existir una función llamada `listar_posts(lista)`.

Esta función debe recibir la lista de posts como parámetro, recorrerla y mostrar las publicaciones con un formato legible.

Debe mostrar, como mínimo:

- Título del post.
- Nombre del autor.

Recordá que el autor es un diccionario anidado.

Función `buscar_por_titulo(lista, termino)`

Debe existir una función llamada `buscar_por_titulo(lista, termino)`.

Esta función debe recibir:

- La lista de posts.
- Un término de búsqueda.

Debe buscar publicaciones cuyo título contenga ese término.

La búsqueda debe ignorar mayúsculas y minúsculas usando `.lower()`.

La función puede mostrar los resultados directamente o retornarlos para que otra parte del programa los muestre.

Función `filtrar_por_tag(lista, tag)`

Debe existir una función llamada `filtrar_por_tag(lista, tag)`.

Esta función debe recibir:

- La lista de posts.
- Un tag a buscar.

Debe mostrar o retornar solo los posts que tengan ese tag dentro de su lista de `tags`.

El filtrado también debe ignorar mayúsculas y minúsculas.

Función `validar_post(post)`

Debe existir una función llamada `validar_post(post)`.

Esta función debe recibir un post y verificar si cumple con las reglas esperadas.

Debe revisar, como mínimo:

- Que el post sea un diccionario.
- Que existan las claves obligatorias.
- Que `titulo` no esté vacío.
- Que `contenido` no esté vacío.
- Que `autor` sea un diccionario.
- Que el autor tenga la clave `nombre`.
- Que `tags` sea una lista.
- Que `estado` sea uno de los valores definidos en `estados_post`.

La función debe retornar `True` o `False`.

También puede retornar un mensaje de error que explique qué problema encontró.

Bucle principal

La ejecución principal del programa debe estar organizada dentro del bloque:

CÓDIGO

```
if __name__ == "__main__":
```

Dentro de ese bloque se debe:

- Mostrar el menú.
- Capturar la opción del usuario.
- Llamar a la función correspondiente según la opción elegida.
- Mantener el programa activo hasta que el usuario elija salir.

Funcionalidades requeridas

Opción 1: Ver todos los posts

Al elegir la opción `1`, el programa debe llamar a `listar_posts(lista)`.

Debe mostrar los títulos de los posts y el nombre del autor.

Opción 2: Buscar por titulo

Al elegir la opción `2`, el programa debe pedir un término de búsqueda.

Luego debe llamar a `buscar_por_titulo(lista, termino)`.

La búsqueda debe funcionar aunque el usuario escriba en mayúsculas o minúsculas.

Opción 3: Filtrar por tag

Al elegir la opción `3`, el programa debe pedir un tag.

Luego debe llamar a `filtrar_por_tag(lista, tag)`.

El filtrado debe funcionar aunque el usuario escriba en mayúsculas o minúsculas.

Opción 4: Validar posts

Al elegir la opción `4`, el programa debe recorrer la lista de posts y usar `validar_post(post)` para revisar cada publicación.

Debe mostrar cuáles posts son válidos y cuáles tienen errores.

Ejemplo de salida posible:

CÓDIGO

```
Post 1: válido
Post 2: válido
Post 3: error - falta la clave "contenido"
```

No hace falta que el mensaje sea exactamente igual, pero debe ser claro para entender qué post tiene problemas.

Opción 5: Salir

Al elegir la opción `5`, el programa debe mostrar un mensaje de despedida y terminar su ejecución.

Manejo de errores

El programa debe evitar cerrarse inesperadamente.

Debe manejar, como mínimo:

- El usuario ingresa texto en lugar de un número en el menú.
- El usuario ingresa una opción inexistente.
- El usuario deja vacía una búsqueda por título.
- El usuario deja vacío el tag a filtrar.
- Algún post tiene datos incompletos.
- Algún post tiene un tipo de dato incorrecto.
- Se intenta acceder a una clave que no existe dentro de un diccionario.

Para evitar errores al acceder a diccionarios, podés usar `.get()` o verificar si una clave existe con `in`.

Criterios de aceptación

Para considerar este checkpoint como completado, tu código debe cumplir con los siguientes puntos:

- Mantener las estructuras de datos `perfil_autor`, `estados_post`, `etiquetas_blog` y `posts`.
- Incluir al menos 3 posts.
- Mantener el autor como diccionario anidado.
- Incluir al menos un post con datos incompletos o incorrectos para probar las validaciones.
- Crear una función para mostrar el menú y retornar la opción del usuario.
- Usar `try-except` al capturar la opción del menú.
- Crear la función `listar_posts(lista)`.
- Crear la función `buscar_por_titulo(lista, termino)`.
- Crear la función `filtrar_por_tag(lista, tag)`.
- Crear la función `validar_post(post)`.
- Usar parámetros en las funciones.
- Usar al menos una función con `return`.
- Usar `.lower()` en la búsqueda por título.
- Usar `.lower()` en el filtrado por tag.
- Validar que existan las claves necesarias antes de acceder a ellas.
- Usar `.get()` o `in` para evitar errores de tipo `KeyError`.
- Organizar la ejecución principal dentro de `if __name__ == "__main__":`.
- Mantener el menú activo hasta que el usuario elija salir.

- Mostrar mensajes claros cuando no se encuentren resultados.
- Mostrar mensajes claros cuando un post tenga errores de estructura.
- El código debe estar comentado brevemente.
- El programa debe poder copiarse, pegarse y ejecutarse desde la terminal.

Pasos sugeridos

1. Partir del sistema del Módulo 3

Usá como base el sistema interactivo que ya tenía menú, búsqueda, filtrado y salida.

No empieces desde cero: la idea es mejorar lo construido.

2. Mantener las estructuras de datos anteriores

Conservá el diccionario del autor, la tupla de estados, el set de etiquetas y la lista de posts.

Agregá intencionalmente un post con algún dato incompleto o incorrecto para comprobar que las validaciones funcionan.

3. Identificar responsabilidades

Revisá qué partes del programa cumplen una tarea clara.

Por ejemplo:

- Mostrar el menú.
- Listar posts.
- Buscar por título.
- Filtrar por tag.
- Validar un post.
- Procesar la opción elegida.

Cada responsabilidad puede convertirse en una función.

4. Crear funciones pequeñas

Separá el programa en funciones simples.

Intentá que cada función tenga una responsabilidad principal.

Una función no debería encargarse de todo el sistema completo.

5. Usar parámetros

Cuando una función necesite trabajar con los posts, pasale la lista como parámetro.

Cuando una función necesite buscar una palabra o un tag, pasale ese valor como parámetro.

Esto hace que el código sea más flexible y reutilizable.

6. Usar `return` cuando corresponda

Si una función necesita devolver un resultado, usá `return`.

Por ejemplo, la función del menú puede devolver la opción elegida por el usuario.

La función de validación puede devolver si un post es válido o si tiene errores.

7. Agregar validaciones

Revisá que cada post tenga los datos necesarios antes de usarlos.

Esto ayuda a evitar errores cuando el programa intenta acceder a claves o valores que no existen.

8. Usar `try-except` en la entrada del menú

La opción del menú debe capturarse de forma segura.

Si el usuario escribe texto en lugar de un número, el programa debe avisar el problema y volver a mostrar el menú.

9. Organizar el programa con `if __name__ == "__main__":`

Colocá la ejecución principal del sistema dentro de este bloque.

Esto permite separar la definición de funciones de la parte del programa que realmente se ejecuta.

10. Probar el programa completo

Antes de entregar, probá todas las opciones del menú.

Verificá también casos incorrectos:

- Buscar una palabra que no existe.
- Filtrar por un tag inexistente.
- Ingresar texto en el menú.
- Ingresar una opción inválida.
- Dejar una búsqueda vacía.
- Validar posts con datos correctos e incorrectos.

Errores comunes a evitar

No retornar nada

Si una función debe devolver una opción, una validación o una lista de resultados, no olvides usar `return`.

Procesar datos dentro de una función no siempre alcanza si otra parte del programa necesita usar ese resultado.

Confundir `print()` con `return`

`print()` muestra información en consola.

`return` devuelve un valor para que pueda ser usado por otra parte del programa.

Ambos son útiles, pero no cumplen la misma función.

Ignorar mayúsculas y minúsculas

Si buscás `"python"` y el título contiene `"Python"`, la búsqueda debería funcionar igual.

Para eso, usá `.lower()` tanto en el texto ingresado como en el texto comparado.

Acceder directamente a claves que podrían no existir

Si un post tiene datos incompletos, acceder directamente a una clave inexistente puede generar un error.

Para evitarlo, podés usar `.get()` o verificar si la clave existe con `in`.

Crear funciones demasiado grandes

Si una función muestra el menú, valida posts, busca publicaciones y controla la salida, probablemente está haciendo demasiado.

Es mejor dividir el problema en funciones más pequeñas.

Olvidar llamar a una función

Definir una función no significa que se ejecute automáticamente.

Después de crearla, tenés que llamarla desde alguna parte del programa.

No probar el post incompleto

Si agregás un post incompleto, asegurate de usar la opción de validación para comprobar que el programa detecta el problema.

Romper la continuidad con el Módulo 2

El autor debe seguir siendo un diccionario anidado, no un texto simple.

Incorrecto:

CÓDIGO

```
"autor": "Ana López"
```

Correcto:

CÓDIGO

```
"autor": perfil_autor
```

Formato de entrega

Entregá el código completo en formato texto, como si fuera el contenido del archivo:

CÓDIGO

```
sistema_blog_funciones.py
```

También se acepta que mantengas el nombre:

CÓDIGO

```
sistema_blog.py
```

No es necesario adjuntar un archivo `.py`.

Asegurate de que:

- El código esté comentado brevemente.
- Las funciones tengan nombres claros.
- El programa no tenga errores de sintaxis.
- El menú funcione correctamente.
- Se pueda copiar y ejecutar desde la terminal usando Python.

Conexión con el futuro

Este checkpoint transforma tu sistema de consola en un programa más ordenado y mantenible.

En los próximos módulos, esta forma de organizar la lógica va a ser fundamental. Cuando trabajes con archivos, clases o Django, vas a necesitar separar responsabilidades y pensar qué parte del programa se encarga de cada tarea.

En otras palabras: en el Módulo 3 hiciste que el blog pudiera responder al usuario. En este Módulo 4, vas a hacer que ese sistema sea más claro, más robusto y más fácil de mejorar.

Pasos para la entrega

El objetivo de esta práctica es refactorizar el código de consola que has desarrollado anteriormente, aplicando la modularización mediante funciones. Esto hará que tu sistema de blog sea más fácil de leer, mantener y escalar.

Tarea

Debes estructurar tu programa en un archivo único de Python utilizando funciones para cada responsabilidad del sistema. Sigue estos pasos:

1. **Definición de Datos**: Mantén las estructuras de datos de los módulos anteriores (`perfil_autor` , `estados_post` , `etiquetas_blog` , `posts`). Asegúrate de que al menos un post tenga datos incompletos para probar tus validaciones.
2. **Implementación de Funciones**:
   - Crea una función para el menú que retorne la opción del usuario.
   - Crea una función `listar_posts(lista)` que recorra la lista y formatee la salida.
   - Implementa `buscar_por_titulo(lista, termino)` y `filtrar_por_tag(lista, tag)` usando parámetros.

- Desarrolla la función `validar_post(post)` que retorne `True` o `False` (o un mensaje de error) tras verificar las reglas de negocio (claves existentes, campos no vacíos, autor como diccionario).

3. **Manejo de Errores**: Utiliza bloques `try-except` al capturar la entrada del usuario en el menú para evitar que el programa se cierre si se ingresa un texto en lugar de un número.
4. **Bucle Principal**: Organiza la ejecución dentro de un bloque `if __name__ == "__main__":` que orqueste las llamadas a las funciones según la opción elegida por el usuario.

Errores Comunes a Evitar

- **No retornar nada**: Recuerda que las funciones de búsqueda deben retornar los resultados o mostrarlos, no simplemente procesarlos.
- **Ignorar el Case-Sensitive**: Asegúrate de usar `.lower()` tanto en la búsqueda como en el texto comparado.
- **Acceso a diccionarios**: Si vas a validar si existe una clave, usa el método `.get()` o verifica con `in` para evitar que el programa lance un `KeyError`.

Entregable

Formato: text

Completar con el código de tu solución para la entrega.

Objetivos de aprendizaje

- Refactorizar código monolítico en funciones modulares con responsabilidades únicas.
- Implementar el paso de argumentos y el retorno de valores en funciones de procesamiento de datos.
- Aplicar lógica de validación de diccionarios para asegurar la integridad de la información.
- Utilizar manejo de excepciones básico para gestionar entradas de usuario inválidas.

Criterios de aceptación

- El código debe estar organizado en al menos 5 funciones distintas (menú, listar, buscar, filtrar, validar).
- La función de búsqueda y filtrado debe funcionar correctamente independientemente de si se usa mayúsculas o minúsculas.
- La funcionalidad de 'Validar posts' debe identificar correctamente posts con claves faltantes o campos vacíos.
- El programa no debe romperse si el usuario ingresa un carácter no numérico en el menú principal.
- El repositorio debe contener un archivo README que explique brevemente cómo ejecutar el script.

Rúbrica de evaluación

Rúbrica para evaluar un código de Python entregado en formato texto que refactoriza el sistema de blog mediante funciones, aplica validaciones básicas, maneja errores de entrada y organiza la ejecución con un bloque principal. La evaluación prioriza la modularidad, la corrección funcional y la robustez del programa de consola.

Criterio	Descripción	Peso
Modularización y organización del código	Evalúa si el programa está dividido en funciones con responsabilidades claras, si la lógica principal está bien separada y si el bloque `if __name__ == "__main__":` orquesta la ejecución de forma ordenada.	20%
Funciones requeridas y uso de parámetros/retornos	Evalúa la implementación correcta de las funciones solicitadas: menú, `listar_posts(lista)`, `buscar_por_titulo(lista, termino)`, `filtrar_por_tag(lista, tag)` y `validar_post(post)`, incluyendo el uso adecuado de parámetros y retornos.	25%
Validaciones de datos del post	Evalúa si la función `validar_post(post)` verifica las reglas de negocio indicadas: claves existentes, campos no vacíos y autor como diccionario, usando accesos seguros como `.get()` o verificación con `in` para evitar errores.	20%
Búsqueda, filtrado y manejo de case-insensitive	Evalúa si la búsqueda por título y el filtrado por etiqueta funcionan correctamente, usan la lista proporcionada, comparan sin sensibilidad a mayúsculas/minúsculas y no olvidan retornar o mostrar resultados útiles.	15%
Manejo de errores en la entrada del menú	Evalúa si el programa usa `try-except` al leer la opción del usuario y evita que el programa se cierre cuando se ingresa texto en lugar de un número, manteniendo una experiencia de consola estable.	10%
Claridad de entrega y ejecución	Evalúa si el código entregado en texto está completo, puede copiarse a un archivo `.py`, ejecutarse desde la terminal y entenderse con comentarios mínimos.	10%

Total: 100 pts | Aprobación: 70 pts

MÓDULO 5

Módulos, paquetes y archivos

Organizar el proyecto en múltiples archivos y persistir datos básicos en archivos planos.

Importar módulos locales para reutilizar funciones del blog

Aprende a organizar el código de tu blog dividiéndolo en múltiples archivos. Descubrirás cómo crear módulos propios e importarlos usando diferentes sintaxis para mantener tu proyecto limpio y profesional.

Hasta ahora venimos trabajando casi todo el código del blog en un solo archivo. Eso funciona al principio, pero cuando el programa empieza a crecer, el archivo se vuelve difícil de leer y mantener.

Por ejemplo, en un mismo archivo podríamos tener:

CÓDIGO

```
posts = []

def mostrar_menu():
    pass

def listar_posts():
    pass

def validar_post():
    pass

def buscar_post():
    pass
```

Al principio parece cómodo, pero después de varias unidades el archivo puede tener muchas líneas mezcladas: datos, menú, validaciones, búsquedas y lógica principal.

Para evitar eso, Python nos permite dividir el código en **módulos locales**.

Un módulo local es simplemente otro archivo `.py` que nosotros creamos para guardar funciones relacionadas entre sí.

1. ¿Qué es un módulo en Python?

Un módulo es cualquier archivo de Python que contiene código reutilizable.

Por ejemplo, si creamos este archivo:

CÓDIGO

```
# formateador.py

def limpiar_titulo(titulo):
    return titulo.strip().title()
```

Ese archivo ya es un módulo llamado `formateador`.

Luego podemos usarlo desde otro archivo.

2. Estructura inicial del proyecto

Para esta unidad, vamos a organizar el mini blog con varios archivos:

CÓDIGO

```
blog_consola/
│
├── main.py
├── datos.py
├── formateador.py
└── validaciones.py
```

Cada archivo tendrá una responsabilidad:

Archivo	Responsabilidad
`main.py`	Ejecuta el programa principal
`datos.py`	Guarda los datos iniciales del blog
`formateador.py`	Contiene funciones para mostrar información
`validaciones.py`	Contiene funciones para validar posts

3. Crear un módulo de datos

Primero vamos a separar los datos del blog en un archivo propio.

Archivo `datos.py`:

CÓDIGO

```
perfil_autor = {
    "nombre": "Ana Pérez",
    "bio": "Desarrolladora Python y creadora de contenido.",
    "especialidad": "Django",
    "redes_sociales": ["@ana_dev", "@python_ana"]
}

estados_post = ("borrador", "publicado", "archivado")

etiquetas_blog = {"Python", "Django", "Programación", "Web"}

posts = [
    {
        "id": 1,
        "titulo": "Primeros pasos con Python",
        "autor": perfil_autor,
        "estado": "publicado",
        "tags": {"Python", "Programación"},
        "contenido": "En este post aprendemos los conceptos básicos de Python."
    },
    {
        "id": 2,
        "titulo": "Introducción a Django",
        "autor": perfil_autor,
        "estado": "borrador",
        "tags": {"Django", "Web"},
        "contenido": "Django nos permite crear aplicaciones web con Python."
    },
    {
        "id": 3,
        "titulo": "",
        "autor": perfil_autor,
        "estado": "publicado",
        "tags": set(),
        "contenido": ""
    }
]
```

El tercer post tiene datos incompletos a propósito. Esto nos servirá para probar validaciones.

4. Importar datos desde otro archivo

Ahora podemos usar esos datos desde `main.py`.

Archivo `main.py`:

CÓDIGO

```
import datos

print(datos.posts)
```

Cuando usamos:

CÓDIGO

```
import datos
```

Python busca un archivo llamado `datos.py` en la misma carpeta y nos permite acceder a su contenido usando la notación de punto.

Por ejemplo:

CÓDIGO

```
print(datos.perfil_autor["nombre"])
print(datos.estados_post)
print(datos.posts[0]["titulo"])
```

Salida posible:

CÓDIGO

```
Ana Pérez
('borrador', 'publicado', 'archivado')
Primeros pasos con Python
```

5. Crear un módulo para formatear posts

Ahora vamos a crear funciones para mostrar los posts de forma más prolija.

Archivo `formateador.py`:

CÓDIGO

```
def crear_separador():
    return "-" * 40


def formatear_post(post):
    titulo = post["titulo"]
    estado = post["estado"]
    autor = post["autor"]["nombre"]
    tags = ", ".join(post["tags"])

    return f"""
Título: {titulo}
Autor: {autor}
Estado: {estado}
Etiquetas: {tags}
"""


def listar_posts(lista_posts):
    for post in lista_posts:
        print(crear_separador())
        print(formatear_post(post))
```

Ahora usamos este módulo desde `main.py`:

CÓDIGO

```
import datos
import formateador

formateador.listar_posts(datos.posts)
```

Salida posible:

CÓDIGO

```
----------------------------------------

Título: Primeros pasos con Python
Autor: Ana Pérez
Estado: publicado
Etiquetas: Python, Programación

----------------------------------------

Título: Introducción a Django
Autor: Ana Pérez
Estado: borrador
Etiquetas: Web, Django
```

6. Entender la notación de punto

Cuando escribimos:

CÓDIGO

```
formateador.listar_posts(datos.posts)
```

estamos diciendo:

CÓDIGO

```
Del módulo formateador, usá la función listar_posts.
Del módulo datos, usá la variable posts.
```

La notación de punto ayuda a saber de dónde viene cada cosa.

Esto es más claro que tener todas las funciones mezcladas en un solo archivo.

7. Importar funciones específicas

También podemos importar una función puntual.

En lugar de escribir:

CÓDIGO

```
import formateador

formateador.listar_posts(datos.posts)
```

podemos escribir:

CÓDIGO

```
from formateador import listar_posts
import datos

listar_posts(datos.posts)
```

Esto funciona igual, pero ahora no necesitamos escribir `formateador.` antes de llamar a la función.

Esta forma es útil cuando solo necesitamos una o dos funciones de un módulo.

8. Importar varios elementos

También podemos importar varias funciones del mismo módulo.

CÓDIGO

```
from formateador import crear_separador, formatear_post
import datos

print(crear_separador())
print(formatear_post(datos.posts[0]))
```

Aunque esto es válido, no conviene abusar. Si importamos demasiadas cosas sueltas, después puede ser difícil recordar de dónde viene cada función.

9. Evitar `import *`

Python permite hacer esto:

CÓDIGO

```
from formateador import *
```

Pero no es recomendable.

El problema es que importa todo lo que encuentre dentro del módulo, y eso puede generar confusión.

Por ejemplo:

CÓDIGO

```
from formateador import *

def crear_separador():
    return "**********"
```

Ahora tenemos dos funciones con el mismo nombre. Python no nos va a avisar claramente que una pisó a la otra.

Por eso, es mejor ser explícitos:

CÓDIGO

```
import formateador
```

o:

CÓDIGO

```
from formateador import listar_posts
```

10. Crear un módulo de validaciones

Ahora vamos a separar la lógica para validar posts.

Archivo `validaciones.py`:

CÓDIGO

```
def validar_post(post):
    if post["titulo"] == "":
        return False

    if post["contenido"] == "":
        return False

    if len(post["tags"]) == 0:
        return False

    return True
```

Ahora usamos esta función desde `main.py`.

CÓDIGO

```
import datos
from validaciones import validar_post

for post in datos.posts:
    if validar_post(post):
        print(f"El post '{post['titulo']}' es válido.")
    else:
        print(f"El post con ID {post['id']} tiene datos incompletos.")
```

Salida posible:

CÓDIGO

```
El post 'Primeros pasos con Python' es válido.
El post 'Introducción a Django' es válido.
El post con ID 3 tiene datos incompletos.
```

11. Mejorar la validación con mensajes de error

La versión anterior funciona, pero solo devuelve `True` o `False`. Podemos mejorarla devolviendo una lista con los errores encontrados.

Archivo `validaciones.py`:

CÓDIGO

```
def validar_post(post):
    errores = []

    if post["titulo"] == "":
        errores.append("El título no puede estar vacío.")

    if post["contenido"] == "":
        errores.append("El contenido no puede estar vacío.")

    if len(post["tags"]) == 0:
        errores.append("El post debe tener al menos una etiqueta.")

    if post["estado"] not in ("borrador", "publicado", "archivado"):
        errores.append("El estado del post no es válido.")

    return errores
```

Ahora en `main.py`:

CÓDIGO

```
import datos
from validaciones import validar_post

for post in datos.posts:
    errores = validar_post(post)

    if len(errores) == 0:
        print(f"El post '{post['titulo']}' es válido.")
    else:
        print(f"El post con ID {post['id']} tiene errores:")

        for error in errores:
            print(f"- {error}")
```

Salida posible:

CÓDIGO

```
El post 'Primeros pasos con Python' es válido.
El post 'Introducción a Django' es válido.
El post con ID 3 tiene errores:
- El título no puede estar vacío.
- El contenido no puede estar vacío.
- El post debe tener al menos una etiqueta.
```

Esta versión es más útil porque no solo dice que algo está mal, sino también qué está mal.

12. Usar alias con `as`

A veces un módulo tiene un nombre largo. Python permite usar un alias.

CÓDIGO

```
import formateador as fmt
import datos

fmt.listar_posts(datos.posts)
```

Ahora `fmt` es un apodo para `formateador`.

Esto es muy común en librerías externas. Por ejemplo:

CÓDIGO

```
import pandas as pd
```

Pero en proyectos chicos conviene no abusar de los alias. Si el nombre del módulo es claro y corto, muchas veces es mejor dejarlo completo.

13. Errores comunes al importar módulos

Error 1: escribir mal el nombre del archivo

Si el archivo se llama:

CÓDIGO

```
formateador.py
```

esto funciona:

CÓDIGO

```
import formateador
```

Pero esto falla:

CÓDIGO

```
import formatador
```

Python mostrará un error parecido a:

CÓDIGO

```
ModuleNotFoundError: No module named 'formatador'
```

Error 2: poner `.py` en el import

Esto está mal:

CÓDIGO

```
import formateador.py
```

Esto está bien:

CÓDIGO

```
import formateador
```

En el `import` se usa el nombre del archivo, pero sin la extensión `.py`.

Error 3: tener los archivos en carpetas distintas

Si `main.py` y `formateador.py` están en la misma carpeta, esto funciona:

CÓDIGO

```
blog_consola/
│
├── main.py
└── formateador.py
```

Pero si están separados así:

CÓDIGO

```
blog_consola/
│
├── main.py
└── utilidades/
    └── formateador.py
```

entonces el import básico ya no funciona de la misma forma. Más adelante se puede resolver usando paquetes, pero por ahora vamos a mantener todos los archivos en la misma carpeta.

Error 4: llamar a un archivo igual que una librería de Python

No conviene crear archivos con nombres como:

CÓDIGO

```
random.py
datetime.py
json.py
math.py
```

Python ya tiene módulos con esos nombres. Si usamos esos nombres para nuestros archivos, podemos generar errores difíciles de entender.

Por ejemplo, si creamos un archivo llamado `random.py`, luego este código podría fallar:

CÓDIGO

```
import random

numero = random.randint(1, 10)
print(numero)
```

Python podría intentar importar nuestro archivo `random.py` en lugar del módulo original.

14. Ejemplo final integrado

La estructura final queda así:

CÓDIGO

```
blog_consola/
│
├── main.py
├── datos.py
├── formateador.py
└── validaciones.py
```

Archivo `datos.py`:

CÓDIGO

```
perfil_autor = {
    "nombre": "Ana Pérez",
    "bio": "Desarrolladora Python y creadora de contenido.",
    "especialidad": "Django",
    "redes_sociales": ["@ana_dev", "@python_ana"]
}

posts = [
    {
        "id": 1,
        "titulo": "Primeros pasos con Python",
        "autor": perfil_autor,
        "estado": "publicado",
        "tags": {"Python", "Programación"},
        "contenido": "En este post aprendemos los conceptos básicos de Python."
    },
    {
        "id": 2,
        "titulo": "",
        "autor": perfil_autor,
        "estado": "publicado",
        "tags": set(),
        "contenido": ""
    }
]
```

Archivo `formateador.py`:

CÓDIGO

```
def crear_separador():
    return "-" * 40


def formatear_post(post):
    tags = ", ".join(post["tags"])

    return f"""
Título: {post["titulo"]}
Autor: {post["autor"]["nombre"]}
Estado: {post["estado"]}
Etiquetas: {tags}
"""


def listar_posts(lista_posts):
    for post in lista_posts:
        print(crear_separador())
        print(formatear_post(post))
```

Archivo `validaciones.py`:

CÓDIGO

```
def validar_post(post):
    errores = []

    if post["titulo"] == "":
        errores.append("El título no puede estar vacío.")

    if post["contenido"] == "":
        errores.append("El contenido no puede estar vacío.")

    if len(post["tags"]) == 0:
        errores.append("El post debe tener al menos una etiqueta.")

    return errores
```

Archivo `main.py`:

CÓDIGO

```
import datos
import formateador
from validaciones import validar_post

print("LISTADO DE POSTS")
formateador.listar_posts(datos.posts)

print("\nVALIDACIÓN DE POSTS")

for post in datos.posts:
    errores = validar_post(post)

    if len(errores) == 0:
        print(f"El post '{post['titulo']}' es válido.")
    else:
        print(f"El post con ID {post['id']} tiene errores:")

        for error in errores:
            print(f"- {error}")
```

Salida posible:

CÓDIGO

```
LISTADO DE POSTS
----------------------------------------

Título: Primeros pasos con Python
Autor: Ana Pérez
Estado: publicado
Etiquetas: Programación, Python

----------------------------------------

Título:
Autor: Ana Pérez
Estado: publicado
Etiquetas:

VALIDACIÓN DE POSTS
El post 'Primeros pasos con Python' es válido.
El post con ID 2 tiene errores:
- El título no puede estar vacío.
- El contenido no puede estar vacío.
- El post debe tener al menos una etiqueta.
```

Síntesis

Los módulos locales nos permiten dividir un programa en varios archivos `.py`.

Esto ayuda a que el código sea:

- más organizado;
- más fácil de leer;
- más fácil de corregir;
- más fácil de reutilizar.

Como regla general:

CÓDIGO

```
import modulo
```

es ideal cuando queremos mantener claro de dónde viene cada función.

CÓDIGO

```
from modulo import funcion
```

es útil cuando necesitamos importar una función puntual.

Y conviene evitar:

CÓDIGO

```
from modulo import *
```

porque puede volver el código confuso y difícil de mantener.

A partir de esta unidad, el mini blog deja de ser un único archivo gigante y empieza a parecerse más a un proyecto real de Python.

Paquetes y archivo init.py en un proyecto pequeño

Aprende a organizar tu proyecto de Python utilizando paquetes. Descubre el propósito del archivo __init__.py y cómo estructurar carpetas para que tu código sea escalable, profesional y fácil de importar.

Hasta ahora aprendimos a separar nuestro blog en varios archivos `.py`.

Por ejemplo:

CÓDIGO

```
blog_consola/
│
├── main.py
├── datos.py
├── formateador.py
└── validaciones.py
```

Eso ya es una mejora enorme frente a tener todo mezclado en un solo archivo.

Pero imaginemos que el proyecto sigue creciendo. Tal vez ahora tenemos funciones para posts, usuarios, búsquedas, validaciones, reportes y estadísticas. Si dejamos todos los archivos en la misma carpeta, el proyecto puede volverse difícil de ordenar.

Para resolver esto, Python nos permite usar **paquetes**.

Un paquete es una carpeta que agrupa módulos relacionados.

1. ¿Qué es un paquete?

Un **módulo** es un archivo `.py`.

Por ejemplo:

CÓDIGO

```
formateador.py
```

Un **paquete** es una carpeta que contiene módulos.

Por ejemplo:

CÓDIGO

```
blog/
│
├── posts.py
├── usuarios.py
└── validaciones.py
```

En proyectos reales, no todo vive en la carpeta principal. Se suele organizar el código en carpetas según su responsabilidad.

2. Primera estructura con paquetes

Vamos a transformar nuestro mini blog en una estructura más ordenada:

CÓDIGO

```
blog_consola/
│
├── main.py
│
└── blog/
    ├── __init__.py
    ├── datos.py
    ├── formateador.py
    └── validaciones.py
```

Ahora tenemos una carpeta llamada `blog`.

Esa carpeta va a contener la lógica principal de nuestro mini blog.

El archivo `main.py` queda afuera y se encarga de ejecutar el programa.

3. ¿Qué es `__init__.py`?

El archivo `__init__.py` es un archivo especial de Python.

Sirve para indicar que una carpeta debe ser tratada como un **paquete**.

CÓDIGO

```
blog/
├── __init__.py
├── datos.py
├── formateador.py
└── validaciones.py
```

Muchas veces este archivo puede estar vacío:

CÓDIGO

```
# blog/__init__.py
```

Y eso está bien.

No hace falta escribir código dentro de `__init__.py` al principio.

Su presencia ya ayuda a dejar claro que esa carpeta contiene código Python organizado como paquete.

4. Crear los datos del blog dentro del paquete

Archivo `blog/datos.py`:

CÓDIGO

```
perfil_autor = {
    "nombre": "Ana Pérez",
    "bio": "Desarrolladora Python y creadora de contenido.",
    "especialidad": "Django",
    "redes_sociales": ["@ana_dev", "@python_ana"]
}

estados_post = ("borrador", "publicado", "archivado")

posts = [
    {
        "id": 1,
        "titulo": "Primeros pasos con Python",
        "autor": perfil_autor,
        "estado": "publicado",
        "tags": {"Python", "Programación"},
        "contenido": "En este post aprendemos los conceptos básicos de Python."
    },
    {
        "id": 2,
        "titulo": "Introducción a Django",
        "autor": perfil_autor,
        "estado": "borrador",
        "tags": {"Django", "Web"},
        "contenido": "Django nos permite crear aplicaciones web con Python."
    },
    {
        "id": 3,
        "titulo": "",
        "autor": perfil_autor,
        "estado": "publicado",
        "tags": set(),
        "contenido": ""
    }
]
```

El tercer post tiene datos incompletos a propósito para probar validaciones.

5. Crear un módulo de formato dentro del paquete

Archivo `blog/formateador.py`:

CÓDIGO

```
def crear_separador():
    return "-" * 40


def formatear_post(post):
    tags = ", ".join(post["tags"])

    return f"""
Título: {post["titulo"]}
Autor: {post["autor"]["nombre"]}
Estado: {post["estado"]}
Etiquetas: {tags}
"""


def listar_posts(lista_posts):
    for post in lista_posts:
        print(crear_separador())
        print(formatear_post(post))
```

Este archivo se ocupa únicamente de mostrar posts de forma prolija.

6. Crear un módulo de validaciones

Archivo `blog/validaciones.py`:

CÓDIGO

```
def validar_post(post):
    errores = []

    if post["titulo"] == "":
        errores.append("El título no puede estar vacío.")

    if post["contenido"] == "":
        errores.append("El contenido no puede estar vacío.")

    if len(post["tags"]) == 0:
        errores.append("El post debe tener al menos una etiqueta.")

    if post["estado"] not in ("borrador", "publicado", "archivado"):
        errores.append("El estado del post no es válido.")

    return errores
```

Esta función devuelve una lista de errores.

Si la lista está vacía, el post es válido.

7. Importar desde un paquete

Ahora podemos usar estos módulos desde `main.py`.

Archivo `main.py`:

CÓDIGO

```
from blog.datos import posts
from blog.formateador import listar_posts
from blog.validaciones import validar_post

print("LISTADO DE POSTS")
listar_posts(posts)

print("\nVALIDACIÓN DE POSTS")

for post in posts:
    errores = validar_post(post)

    if len(errores) == 0:
        print(f"El post '{post['titulo']}' es válido.")
    else:
        print(f"El post con ID {post['id']} tiene errores:")

        for error in errores:
            print(f"- {error}")
```

En este ejemplo aparecen imports como estos:

CÓDIGO

```
from blog.datos import posts
from blog.formateador import listar_posts
from blog.validaciones import validar_post
```

Esto significa:

CÓDIGO

```
Desde el paquete blog,
entrar al módulo datos,
e importar la variable posts.
```

O también:

CÓDIGO

```
Desde el paquete blog,
entrar al módulo formateador,
e importar la función listar_posts.
```

8. Ejecutar el proyecto correctamente

La estructura completa queda así:

CÓDIGO

```
blog_consola/
│
├── main.py
│
└── blog/
    ├── __init__.py
    ├── datos.py
    ├── formateador.py
    └── validaciones.py
```

Para ejecutarlo, debemos estar parados en la carpeta raíz del proyecto:

CÓDIGO

```
blog_consola/
```

Y correr:

CÓDIGO

```
python main.py
```

No conviene entrar a la carpeta `blog` y ejecutar los archivos internos directamente.

Es decir, no hacemos esto:

CÓDIGO

```
cd blog
python formateador.py
```

En esta etapa, los archivos de adentro del paquete funcionan como piezas reutilizables. El punto de entrada del programa es `main.py`.

9. Salida posible

Al ejecutar `python main.py`, podríamos ver algo así:

CÓDIGO

```
LISTADO DE POSTS
----------------------------------------

Título: Primeros pasos con Python
Autor: Ana Pérez
Estado: publicado
Etiquetas: Python, Programación

----------------------------------------

Título: Introducción a Django
Autor: Ana Pérez
Estado: borrador
Etiquetas: Django, Web

----------------------------------------

Título:
Autor: Ana Pérez
Estado: publicado
Etiquetas:


VALIDACIÓN DE POSTS
El post 'Primeros pasos con Python' es válido.
El post 'Introducción a Django' es válido.
El post con ID 3 tiene errores:
- El título no puede estar vacío.
- El contenido no puede estar vacío.
- El post debe tener al menos una etiqueta.
```

10. Importar el módulo completo

También podemos importar el módulo completo en lugar de importar una función específica.

Archivo `main.py`:

CÓDIGO

```
from blog import datos
from blog import formateador
from blog import validaciones

formateador.listar_posts(datos.posts)

for post in datos.posts:
    errores = validaciones.validar_post(post)

    if len(errores) == 0:
        print("Post válido")
    else:
        print("Post con errores")
```

Esta forma es un poco más larga, pero tiene una ventaja: deja muy claro de dónde viene cada cosa.

Cuando vemos:

CÓDIGO

```
formateador.listar_posts(datos.posts)
```

sabemos que `listar_posts` viene del módulo `formateador`, y que `posts` viene del módulo `datos`.

11. Usar `__init__.py` como puerta de entrada

El archivo `__init__.py` también puede servir para simplificar imports.

Por ejemplo, podemos escribir esto:

Archivo `blog/__init__.py`:

CÓDIGO

```
from .datos import posts
from .formateador import listar_posts
from .validaciones import validar_post
```

El punto antes de `datos`, `formateador` y `validaciones` significa:

CÓDIGO

```
Importar desde esta misma carpeta.
```

Ahora, desde `main.py`, podemos hacer esto:

CÓDIGO

```
from blog import posts, listar_posts, validar_post

print("LISTADO DE POSTS")
listar_posts(posts)

print("\nVALIDACIÓN DE POSTS")

for post in posts:
    errores = validar_post(post)

    if len(errores) == 0:
        print(f"El post '{post['titulo']}' es válido.")
    else:
        print(f"El post con ID {post['id']} tiene errores:")

        for error in errores:
            print(f"- {error}")
```

Esto funciona porque `__init__.py` está actuando como una puerta de entrada al paquete.

En lugar de escribir:

CÓDIGO

```
from blog.formateador import listar_posts
from blog.validaciones import validar_post
from blog.datos import posts
```

podemos escribir:

CÓDIGO

```
from blog import posts, listar_posts, validar_post
```

Mucho más limpio.

12. ¿Conviene poner todo en `__init__.py`?

No.

El archivo `__init__.py` no debería convertirse en un archivo gigante.

No conviene hacer esto:

CÓDIGO

```
# blog/__init__.py

perfil_autor = {
    "nombre": "Ana Pérez"
}

posts = []

def listar_posts():
    pass

def validar_post():
    pass

def buscar_post():
    pass
```

Eso vuelve a mezclar todo.

Mejor usamos archivos separados:

CÓDIGO

```
blog/
├── __init__.py
├── datos.py
├── formateador.py
└── validaciones.py
```

Y dejamos `__init__.py` vacío o con imports simples:

CÓDIGO

```
from .datos import posts
from .formateador import listar_posts
from .validaciones import validar_post
```

13. Subpaquetes: carpetas dentro de carpetas

Si el proyecto sigue creciendo, podemos crear carpetas dentro del paquete.

Por ejemplo:

CÓDIGO

```
blog_consola/
│
├── main.py
│
└── blog/
    ├── __init__.py
    │
    ├── posts/
    │   ├── __init__.py
    │   ├── datos.py
    │   ├── formateador.py
    │   └── validaciones.py
    │
    └── usuarios/
        ├── __init__.py
        └── datos.py
```

Ahora tenemos:

CÓDIGO

```
blog.posts
blog.usuarios
```

Podemos importar así:

CÓDIGO

```
from blog.posts.datos import posts
from blog.posts.formateador import listar_posts
from blog.posts.validaciones import validar_post
```

Esta estructura se parece más a la forma en que se organizan proyectos más grandes.

También nos empieza a preparar para Django, donde cada app suele tener varios archivos organizados dentro de carpetas.

14. Ejemplo con subpaquetes

Archivo `blog/posts/datos.py` :

CÓDIGO

```
posts = [
    {
        "id": 1,
        "titulo": "Primer post",
        "estado": "publicado",
        "tags": {"Python"},
        "contenido": "Contenido del primer post."
    },
    {
        "id": 2,
        "titulo": "",
        "estado": "borrador",
        "tags": set(),
        "contenido": ""
    }
]
```

Archivo `blog/posts/validaciones.py`:

CÓDIGO

```
def validar_post(post):
    errores = []

    if post["titulo"] == "":
        errores.append("El título no puede estar vacío.")

    if post["contenido"] == "":
        errores.append("El contenido no puede estar vacío.")

    if len(post["tags"]) == 0:
        errores.append("El post debe tener al menos una etiqueta.")

    return errores
```

Archivo `blog/posts/formateador.py`:

CÓDIGO

```
def mostrar_post(post):
    print(f"ID: {post['id']}")
    print(f"Título: {post['titulo']}")
    print(f"Estado: {post['estado']}")
    print(f"Etiquetas: {', '.join(post['tags'])}")


def listar_posts(posts):
    for post in posts:
        print("-" * 30)
        mostrar_post(post)
```

Archivo `main.py`:

CÓDIGO

```
from blog.posts.datos import posts
from blog.posts.formateador import listar_posts
from blog.posts.validaciones import validar_post

listar_posts(posts)

print("\nVALIDACIONES")

for post in posts:
    errores = validar_post(post)

    if len(errores) == 0:
        print(f"El post {post['id']} es válido.")
    else:
        print(f"El post {post['id']} tiene errores:")

        for error in errores:
            print(f"- {error}")
```

15. Imports relativos dentro del paquete

Cuando estamos dentro de un paquete, también podemos usar imports relativos.

Por ejemplo, dentro de `blog/posts/__init__.py`:

CÓDIGO

```
from .datos import posts
from .formateador import listar_posts
from .validaciones import validar_post
```

El punto `.` significa:

CÓDIGO

```
Desde esta misma carpeta.
```

Gracias a eso, desde `main.py` podemos importar directamente:

CÓDIGO

```
from blog.posts import posts, listar_posts, validar_post
```

Y usarlo así:

CÓDIGO

```
listar_posts(posts)

for post in posts:
    errores = validar_post(post)

    if len(errores) == 0:
        print("Post válido")
    else:
        print("Post con errores")
```

Esta técnica ayuda a que el código de `main.py` sea más limpio.

16. Errores comunes

Error 1: olvidarse de crear `__init__.py`

Aunque en versiones modernas de Python no siempre es obligatorio, para este curso lo vamos a usar siempre.

Esto ayuda a que la estructura sea más clara:

CÓDIGO

```
blog/
├── __init__.py
├── datos.py
└── validaciones.py
```

Aunque el archivo esté vacío, cumple una función importante: marca la carpeta como paquete.

Error 2: poner `.py` en el import

Esto está mal:

CÓDIGO

```
from blog.datos.py import posts
```

Esto está bien:

CÓDIGO

```
from blog.datos import posts
```

En los imports nunca escribimos la extensión `.py`.

Error 3: ejecutar desde la carpeta incorrecta

Si tenemos esta estructura:

CÓDIGO

```
blog_consola/
│
├── main.py
└── blog/
    ├── __init__.py
    └── datos.py
```

Debemos ejecutar desde `blog_consola`:

CÓDIGO

```
python main.py
```

No desde adentro de `blog`.

Error 4: llamar a las carpetas con nombres poco claros

Evitemos nombres como:

CÓDIGO

```
cosas/
archivos/
varios/
pruebas/
```

Mejor usar nombres que expliquen la responsabilidad:

CÓDIGO

```
blog/
posts/
usuarios/
validaciones/
```

El nombre de una carpeta también comunica intención.

Error 5: meter demasiada lógica en `__init__.py`

Esto no conviene:

CÓDIGO

```
# blog/__init__.py

def validar_post(post):
    pass

def listar_posts(posts):
    pass

def buscar_post(posts, termino):
    pass
```

Mejor repartir responsabilidades:

CÓDIGO

```
blog/
├── __init__.py
├── validaciones.py
├── formateador.py
└── busquedas.py
```

`__init__.py` puede estar vacío o usarse solamente para imports simples.

17. Diferencia entre módulo y paquete

Una forma sencilla de recordarlo:

Concepto	Qué es	Ejemplo
Módulo	Un archivo `.py`	`validaciones.py`
Paquete	Una carpeta con módulos	`blog/`
Subpaquete	Una carpeta dentro de un paquete	`blog/posts/`
`__init__.py`	Archivo que identifica o configura el paquete	`blog/__init__.py`

18. Ejemplo final recomendado

Para este punto del curso, una buena estructura sería:

CÓDIGO

```
blog_consola/
│
├── main.py
│
└── blog/
    ├── __init__.py
    ├── datos.py
    ├── formateador.py
    ├── validaciones.py
    └── busquedas.py
```

Archivo `blog/busquedas.py`:

CÓDIGO

```
def buscar_por_titulo(posts, termino):
    resultados = []

    for post in posts:
        titulo = post["titulo"].lower()

        if termino.lower() in titulo:
            resultados.append(post)

    return resultados


def filtrar_por_tag(posts, tag):
    resultados = []

    for post in posts:
        if tag in post["tags"]:
            resultados.append(post)

    return resultados
```

Archivo `blog/__init__.py`:

CÓDIGO

```
from .datos import posts
from .formateador import listar_posts
from .validaciones import validar_post
from .busquedas import buscar_por_titulo, filtrar_por_tag
```

Archivo `main.py`:

CÓDIGO

```
from blog import (
    posts,
    listar_posts,
    validar_post,
    buscar_por_titulo,
    filtrar_por_tag
)

print("TODOS LOS POSTS")
listar_posts(posts)

print("\nBUSCAR POSTS POR TÍTULO")
resultados = buscar_por_titulo(posts, "python")
listar_posts(resultados)

print("\nFILTRAR POSTS POR TAG")
posts_python = filtrar_por_tag(posts, "Python")
listar_posts(posts_python)

print("\nVALIDAR POSTS")
for post in posts:
    errores = validar_post(post)

    if len(errores) == 0:
        print(f"El post '{post['titulo']}' es válido.")
    else:
        print(f"El post con ID {post['id']} tiene errores:")

        for error in errores:
            print(f"- {error}")
```

Este ejemplo muestra cómo un paquete puede reunir muchas piezas del blog, pero mantener cada responsabilidad separada.

19. Actividad práctica sugerida

Tomá el mini blog que ya venías trabajando y reorganizalo usando paquetes.

La estructura mínima debería ser:

CÓDIGO

```
blog_consola/
│
├── main.py
│
└── blog/
    ├── __init__.py
    ├── datos.py
    ├── formateador.py
    ├── validaciones.py
    └── busquedas.py
```

El proyecto debe incluir:

- una lista de posts en `datos.py`;
- funciones para listar posts en `formateador.py`;
- funciones para validar posts en `validaciones.py`;
- funciones para buscar y filtrar posts en `busquedas.py`;
- un archivo `main.py` que importe y use esas funciones.

Síntesis

Un módulo es un archivo `.py`.

Un paquete es una carpeta que agrupa módulos relacionados.

El archivo `__init__.py` permite identificar una carpeta como paquete y también puede funcionar como una puerta de entrada para simplificar imports.

Por ejemplo:

CÓDIGO

```
from blog import posts, listar_posts, validar_post
```

puede ser más limpio que escribir:

CÓDIGO

```
from blog.datos import posts
from blog.formateador import listar_posts
from blog.validaciones import validar_post
```

A medida que el proyecto crece, los paquetes nos ayudan a mantener el código más ordenado, más fácil de leer y más parecido a un proyecto real.

Esta organización también prepara el camino para Django, donde cada aplicación del proyecto tendrá una estructura de carpetas y archivos con responsabilidades separadas.

Leer y escribir archivos de texto con open()

Aprende a guardar y recuperar información de tu blog utilizando archivos de texto plano con la función open() y el gestor de contexto with.

Hasta ahora, cada vez que cerrabas tu programa de Python, todos los datos que habías creado se perdían.

Por ejemplo, si teníamos una lista de posts:

CÓDIGO

```
posts = [
    "Primer post del blog",
    "Aprendiendo Python",
    "Introducción a Django"
]
```

esa información vivía solamente mientras el programa estaba abierto. Al cerrar la consola, los datos desaparecían.

Esto ocurre porque las variables se guardan en la **memoria RAM**, y esa memoria es temporal.

Para construir programas más útiles, necesitamos aprender a guardar información en archivos. A esto lo llamamos **persistencia de datos**.

En esta unidad vamos a trabajar con archivos de texto `.txt`, que son una forma simple de guardar información en la computadora.

1. ¿Qué es un archivo de texto?

Un archivo de texto es un archivo que guarda caracteres legibles por humanos.

Por ejemplo, un archivo llamado `posts.txt` podría contener esto:

CÓDIGO

```
Primer post del blog
Aprendiendo Python
Introducción a Django
```

A diferencia de un archivo de Word o un PDF, un `.txt` no guarda estilos, colores, imágenes ni tamaños de letra. Solo guarda texto plano.

Esto lo vuelve ideal para empezar a practicar lectura y escritura de archivos.

2. La función `open()`

Para trabajar con archivos en Python usamos la función `open()`.

La estructura básica es:

CÓDIGO

```
open("nombre_del_archivo.txt", "modo")
```

El primer valor es el nombre del archivo.

El segundo valor indica qué queremos hacer con ese archivo.

3. Modos principales de apertura

Modo	Nombre	Qué hace
`"r"`	read	Abre el archivo para leer. Da error si no existe.
`"w"`	write	Abre el archivo para escribir. Si ya existe, borra su contenido.
`"a"`	append	Abre el archivo para agregar contenido al final. No borra lo anterior.
`"x"`	create	Crea un archivo nuevo. Da error si ya existe.

Los modos más importantes para empezar son:

CÓDIGO

```
"r"  # leer
"w"  # escribir desde cero
"a"  # agregar al final
```

4. Escribir en un archivo con `"w"`

El modo `"w"` permite escribir contenido en un archivo.

Si el archivo no existe, Python lo crea.

CÓDIGO

```
with open("posts.txt", "w", encoding="utf-8") as archivo:
    archivo.write("Mi primer post del blog")
```

Después de ejecutar ese código, se crea un archivo llamado `posts.txt` con este contenido:

CÓDIGO

```
Mi primer post del blog
```

Pero hay que tener cuidado.

El modo `"w"` borra el contenido anterior del archivo.

Por ejemplo:

CÓDIGO

```
with open("posts.txt", "w", encoding="utf-8") as archivo:
    archivo.write("Nuevo contenido")
```

Si antes el archivo tenía otros posts, se perderán.

Podemos pensar el modo `"w"` como borrar una pizarra antes de escribir de nuevo.

5. Agregar contenido con `"a"`

Para un blog, muchas veces no queremos borrar los posts anteriores. Queremos agregar uno nuevo al final.

Para eso usamos el modo `"a"`.

CÓDIGO

```
with open("posts.txt", "a", encoding="utf-8") as archivo:
    archivo.write("Nuevo post del blog\n")
```

El modo `"a"` viene de *append*, que significa agregar.

Si ejecutamos este código varias veces, cada nuevo post se suma al final del archivo.

6. El salto de línea `\n`

Cuando usamos `write()`, Python escribe exactamente lo que le indicamos.

Por eso, si queremos que cada post quede en una línea diferente, necesitamos usar `\n`.

CÓDIGO

```
with open("posts.txt", "a", encoding="utf-8") as archivo:
    archivo.write("Primer post\n")
    archivo.write("Segundo post\n")
    archivo.write("Tercer post\n")
```

El archivo quedaría así:

CÓDIGO

```
Primer post
Segundo post
Tercer post
```

Sin `\n`, el texto quedaría pegado:

CÓDIGO

```
Primer postSegundo postTercer post
```

7. El bloque `with`

Cuando abrimos un archivo, también deberíamos cerrarlo.

Antes se hacía así:

CÓDIGO

```
archivo = open("posts.txt", "w", encoding="utf-8")
archivo.write("Hola blog")
archivo.close()
```

El problema es que podemos olvidarnos de cerrar el archivo.

Por eso, en Python usamos `with`.

CÓDIGO

```
with open("posts.txt", "w", encoding="utf-8") as archivo:
    archivo.write("Hola blog")
```

El bloque `with` se encarga de cerrar el archivo automáticamente cuando termina el bloque con sangría.

Esta es la forma recomendada de trabajar con archivos.

8. Leer un archivo completo

Para leer un archivo usamos el modo `"r"`.

CÓDIGO

```
with open("posts.txt", "r", encoding="utf-8") as archivo:
    contenido = archivo.read()

print(contenido)
```

Si el archivo contiene:

CÓDIGO

```
Primer post
Segundo post
Tercer post
```

la salida será:

CÓDIGO

```
Primer post
Segundo post
Tercer post
```

El método `read()` lee todo el contenido del archivo de una sola vez.

9. Leer un archivo línea por línea

Para nuestro blog, muchas veces nos conviene leer línea por línea.

CÓDIGO

```
with open("posts.txt", "r", encoding="utf-8") as archivo:
    for linea in archivo:
        print(linea)
```

Esto recorre cada línea del archivo.

Sin embargo, puede aparecer una línea en blanco extra entre cada post, porque cada línea ya trae su propio salto de línea `\n`.

Para limpiar eso, podemos usar `.strip()`:

CÓDIGO

```
with open("posts.txt", "r", encoding="utf-8") as archivo:
    for linea in archivo:
        post = linea.strip()
        print(post)
```

El método `.strip()` quita espacios y saltos de línea al principio y al final del texto.

10. Guardar posts del blog en un archivo

Ahora vamos a aplicar esto al mini blog.

Supongamos que queremos guardar títulos de posts en un archivo llamado `posts.txt`.

CÓDIGO

```
def guardar_post(titulo):
    with open("posts.txt", "a", encoding="utf-8") as archivo:
        archivo.write(titulo + "\n")


guardar_post("Primeros pasos con Python")
guardar_post("Introducción a Django")
guardar_post("Funciones en Python")
```

El archivo `posts.txt` quedaría así:

CÓDIGO

```
Primeros pasos con Python
Introducción a Django
Funciones en Python
```

11. Leer todos los posts guardados

Ahora podemos crear una función para leer los posts del archivo.

CÓDIGO

```
def leer_posts():
    with open("posts.txt", "r", encoding="utf-8") as archivo:
        for linea in archivo:
            titulo = linea.strip()
            print(f"- {titulo}")


leer_posts()
```

Salida posible:

CÓDIGO

```
- Primeros pasos con Python
- Introducción a Django
- Funciones en Python
```

12. Convertir las líneas en una lista

También podemos leer el archivo y devolver una lista de posts.

CÓDIGO

```
def obtener_posts():
    posts = []

    with open("posts.txt", "r", encoding="utf-8") as archivo:
        for linea in archivo:
            titulo = linea.strip()
            posts.append(titulo)

    return posts


posts_guardados = obtener_posts()

print(posts_guardados)
```

Salida posible:

CÓDIGO

```
['Primeros pasos con Python', 'Introducción a Django', 'Funciones en Python']
```

Esto es útil porque después podemos recorrer esa lista, buscar posts o filtrarlos.

13. Buscar posts dentro del archivo

Podemos combinar archivos con lógica de búsqueda.

CÓDIGO

```
def buscar_post(termino):
    resultados = []

    with open("posts.txt", "r", encoding="utf-8") as archivo:
        for linea in archivo:
            titulo = linea.strip()

            if termino.lower() in titulo.lower():
                resultados.append(titulo)

    return resultados


resultados = buscar_post("python")

for post in resultados:
    print(post)
```

Salida posible:

CÓDIGO

```
Primeros pasos con Python
Funciones en Python
```

14. Manejar el error si el archivo no existe

Si intentamos leer un archivo que todavía no fue creado, Python mostrará un error.

Por ejemplo:

CÓDIGO

```
with open("posts.txt", "r", encoding="utf-8") as archivo:
    contenido = archivo.read()
```

Si `posts.txt` no existe, aparece un error parecido a:

CÓDIGO

```
FileNotFoundError: [Errno 2] No such file or directory: 'posts.txt'
```

Podemos manejar ese caso con `try` y `except`.

CÓDIGO

```
def leer_posts():
    try:
        with open("posts.txt", "r", encoding="utf-8") as archivo:
            for linea in archivo:
                print(linea.strip())
    except FileNotFoundError:
        print("Todavía no hay posts guardados.")
```

Ahora el programa no se rompe si el archivo no existe.

15. Ejemplo completo aplicado al blog

Vamos a crear un pequeño programa que permita:

- guardar un post;
- listar los posts guardados;
- buscar posts por título.

CÓDIGO

```
ARCHIVO_POSTS = "posts.txt"


def guardar_post(titulo):
    with open(ARCHIVO_POSTS, "a", encoding="utf-8") as archivo:
        archivo.write(titulo + "\n")


def listar_posts():
    try:
        with open(ARCHIVO_POSTS, "r", encoding="utf-8") as archivo:
            print("Posts guardados:")

            for linea in archivo:
                titulo = linea.strip()
                print(f"- {titulo}")

    except FileNotFoundError:
        print("Todavía no hay posts guardados.")


def buscar_post(termino):
    resultados = []

    try:
        with open(ARCHIVO_POSTS, "r", encoding="utf-8") as archivo:
            for linea in archivo:
                titulo = linea.strip()

                if termino.lower() in titulo.lower():
                    resultados.append(titulo)

    except FileNotFoundError:
        print("Todavía no hay posts guardados.")

    return resultados


guardar_post("Primeros pasos con Python")
guardar_post("Introducción a Django")
guardar_post("Funciones en Python")

listar_posts()

print("\nResultados de búsqueda:")

resultados = buscar_post("python")

for post in resultados:
    print(f"- {post}")
```

Salida posible:

CÓDIGO

```
Posts guardados:
- Primeros pasos con Python
- Introducción a Django
- Funciones en Python

Resultados de búsqueda:
- Primeros pasos con Python
- Funciones en Python
```

16. Guardar más información en una línea

Hasta ahora guardamos solamente títulos. Pero un post podría tener más datos.

Por ejemplo:

CÓDIGO

```
post = {
    "titulo": "Introducción a Django",
    "autor": "Ana Pérez",
    "estado": "borrador"
}
```

Como estamos usando `.txt`, una forma simple de guardar estos datos es separarlos con algún símbolo.

Por ejemplo:

CÓDIGO

```
Introducción a Django|Ana Pérez|borrador
```

Podemos guardar un post así:

CÓDIGO

```
def guardar_post_completo(titulo, autor, estado):
    with open("posts.txt", "a", encoding="utf-8") as archivo:
        linea = f"{titulo}|{autor}|{estado}\n"
        archivo.write(linea)


guardar_post_completo("Introducción a Django", "Ana Pérez", "borrador")
```

Y luego leerlo separando los datos con `.split("|")`:

CÓDIGO

```
with open("posts.txt", "r", encoding="utf-8") as archivo:
    for linea in archivo:
        datos = linea.strip().split("|")

        titulo = datos[0]
        autor = datos[1]
        estado = datos[2]

        print(f"Título: {titulo}")
        print(f"Autor: {autor}")
        print(f"Estado: {estado}")
```

Salida posible:

CÓDIGO

```
Título: Introducción a Django
Autor: Ana Pérez
Estado: borrador
```

Esta técnica funciona para practicar, pero tiene limitaciones. Si el texto del post contiene el símbolo `|`, podría generar problemas.

Más adelante vamos a usar formatos más adecuados para datos estructurados, como JSON.

17. Buenas prácticas al trabajar con archivos

Cuando trabajamos con archivos, conviene recordar estas reglas:

- Usar `with open(...)` para que el archivo se cierre automáticamente.
- Agregar `encoding="utf-8"` para evitar problemas con tildes y `ñ`.
- Usar `"a"` si queremos agregar contenido sin borrar lo anterior.
- Usar `"w"` solamente si realmente queremos reemplazar todo el contenido.
- Usar `try` y `except FileNotFoundError` cuando vamos a leer archivos que podrían no existir.
- Usar `\n` para separar líneas.
- Usar `.strip()` al leer líneas para limpiar saltos de línea.

18. Actividad práctica sugerida

Crear un programa de consola que permita guardar y leer posts desde un archivo de texto.

El programa debe incluir:

1. Una función `guardar_post(titulo)` que agregue un título al archivo `posts.txt`.
2. Una función `listar_posts()` que muestre todos los posts guardados.
3. Una función `buscar_post(termino)` que busque posts por título.

4. Manejo de error si el archivo todavía no existe.
5. Uso de `with open(...)`.
6. Uso de `encoding="utf-8"`.

Ejemplo de uso esperado:

CÓDIGO

```
guardar_post("Primeros pasos con Python")
guardar_post("Introducción a Django")
guardar_post("Archivos en Python")

listar_posts()

resultados = buscar_post("python")
print(resultados)
```

Síntesis

Los archivos de texto nos permiten guardar información más allá de la ejecución del programa.

Con `open()` podemos abrir archivos en distintos modos:

CÓDIGO

```
"r"  # leer
"w"  # escribir y reemplazar
"a"  # agregar al final
"x"  # crear un archivo nuevo
```

El bloque `with` nos ayuda a trabajar de forma segura, porque cierra el archivo automáticamente.

Para nuestro blog, esto nos permite dar un primer paso hacia la persistencia de datos: guardar posts en un archivo y volver a leerlos cuando el programa se ejecute otra vez.

Más adelante veremos formatos más adecuados para guardar estructuras completas, como listas y diccionarios, usando JSON.

Manejar rutas de archivos con pathlib

Aprende a localizar y gestionar archivos de forma profesional usando pathlib, asegurando que tu proyecto funcione en cualquier computadora sin errores de rutas.

Hasta ahora aprendimos a leer y escribir archivos de texto usando `open()`.

Por ejemplo:

CÓDIGO

```
with open("posts.txt", "a", encoding="utf-8") as archivo:
    archivo.write("Primer post del blog\n")
```

Este código funciona si `posts.txt` está en el lugar correcto. Pero aparece una pregunta importante:

¿Dónde está exactamente ese archivo?

Cuando un proyecto crece, no queremos tener todos los archivos mezclados en la misma carpeta. Lo más ordenado sería guardar los datos del blog dentro de una carpeta especial, por ejemplo:

CÓDIGO

```
blog_consola/
│
├── main.py
│
└── data/
    └── posts.txt
```

Para manejar rutas de archivos de forma segura, Python nos ofrece una herramienta muy útil: `pathlib`.

1. ¿Qué es una ruta?

Una ruta es la ubicación de un archivo o carpeta dentro de la computadora.

Por ejemplo:

CÓDIGO

```
data/posts.txt
```

Esa ruta significa:

CÓDIGO

```
Entrar a la carpeta data
y buscar el archivo posts.txt
```

En nuestro blog, las rutas nos van a servir para encontrar archivos donde guardamos información, como posts, autores o configuraciones.

2. Rutas relativas y absolutas

Existen dos formas comunes de escribir rutas: relativas y absolutas.

Ruta relativa

Una ruta relativa indica dónde está un archivo tomando como referencia la carpeta desde donde se ejecuta el programa.

Por ejemplo:

CÓDIGO

```
"data/posts.txt"
```

Esta ruta significa:

CÓDIGO

```
Desde donde estoy parado,
entrar a data
y abrir posts.txt
```

Es cómoda, pero puede fallar si ejecutamos el programa desde otra carpeta.

Ruta absoluta

Una ruta absoluta indica la ubicación completa del archivo en la computadora.

En Windows podría verse así:

CÓDIGO

```
C:\Users\Ana\Proyectos\blog_consola\data\posts.txt
```

En Linux o macOS podría verse así:

CÓDIGO

```
/home/ana/proyectos/blog_consola/data/posts.txt
```

El problema es que estas rutas dependen de la computadora de cada persona.

Si compartimos el proyecto con otro estudiante, probablemente esa ruta no exista en su equipo.

Por eso, no conviene escribir rutas absolutas personales dentro del código.

3. El problema de escribir rutas como texto

En Windows se suelen ver rutas con barra invertida:

CÓDIGO

```
data\posts.txt
```

En Linux y macOS se suelen ver rutas con barra normal:

CÓDIGO

```
data/posts.txt
```

Si escribimos rutas manualmente como texto, podemos generar errores o hacer que nuestro programa funcione bien en una computadora, pero falle en otra.

Para evitar esto usamos `pathlib`.

4. ¿Qué es `pathlib`?

`pathlib` es un módulo de Python que nos permite trabajar con rutas de archivos y carpetas de forma más clara y segura.

Primero importamos `Path`:

CÓDIGO

```
from pathlib import Path
```

Luego podemos crear rutas como objetos:

CÓDIGO

```
ruta_data = Path("data")
ruta_posts = ruta_data / "posts.txt"

print(ruta_posts)
```

Salida posible:

CÓDIGO

```
data/posts.txt
```

Lo importante es esta línea:

CÓDIGO

```
ruta_posts = ruta_data / "posts.txt"
```

En `pathlib`, usamos `/` para unir carpetas y archivos.

Aunque parezca una división, no lo es. Python entiende que estamos construyendo una ruta.

5. Crear una ruta para el archivo de posts

Supongamos esta estructura:

CÓDIGO

```
blog_consola/
│
├── main.py
│
└── data/
    └── posts.txt
```

Podemos representar esa ruta así:

CÓDIGO

```
from pathlib import Path

ruta_archivo = Path("data") / "posts.txt"

print(ruta_archivo)
```

Esto construye una ruta hacia:

CÓDIGO

```
data/posts.txt
```

6. Crear carpetas si no existen

Si intentamos guardar un archivo dentro de una carpeta que no existe, Python puede mostrar un error.

Por ejemplo, si `data/` no existe, esto puede fallar:

CÓDIGO

```
with open("data/posts.txt", "a", encoding="utf-8") as archivo:
    archivo.write("Nuevo post\n")
```

Con `pathlib`, podemos crear la carpeta antes de usarla:

CÓDIGO

```
from pathlib import Path

carpeta_data = Path("data")

carpeta_data.mkdir(exist_ok=True)
```

El método `.mkdir()` crea una carpeta.

El argumento `exist_ok=True` significa:

CÓDIGO

```
Si la carpeta ya existe, no generar error.
```

También podemos crear carpetas intermedias:

CÓDIGO

```
carpeta_data.mkdir(parents=True, exist_ok=True)
```

`parents=True` permite crear varias carpetas si hacen falta.

7. Guardar posts usando `pathlib`

Ahora podemos combinar `pathlib` con `open()`.

CÓDIGO

```
from pathlib import Path

carpeta_data = Path("data")
archivo_posts = carpeta_data / "posts.txt"

carpeta_data.mkdir(exist_ok=True)

with open(archivo_posts, "a", encoding="utf-8") as archivo:
    archivo.write("Primer post del blog\n")
```

Este código hace tres cosas:

1. Define la carpeta `data`.
2. Define el archivo `posts.txt` dentro de esa carpeta.
3. Guarda un post en ese archivo.

8. Leer posts usando `pathlib`

También podemos leer el archivo usando la ruta creada con `Path`.

CÓDIGO

```
from pathlib import Path

archivo_posts = Path("data") / "posts.txt"

with open(archivo_posts, "r", encoding="utf-8") as archivo:
    for linea in archivo:
        print(linea.strip())
```

Salida posible:

CÓDIGO

```
Primer post del blog
```

Aunque `archivo_posts` no sea un texto común, `open()` puede usarlo sin problema.

9. Verificar si un archivo existe

Antes de leer un archivo, conviene verificar si existe.

CÓDIGO

```
from pathlib import Path

archivo_posts = Path("data") / "posts.txt"

if archivo_posts.exists():
    print("El archivo existe.")
else:
    print("Todavía no hay archivo de posts.")
```

También podemos preguntar si es un archivo:

CÓDIGO

```
if archivo_posts.is_file():
    print("Es un archivo.")
```

O si una ruta es una carpeta:

CÓDIGO

```
carpeta_data = Path("data")

if carpeta_data.is_dir():
    print("Es una carpeta.")
```

10. Ejemplo aplicado al blog

Vamos a crear funciones para guardar y listar posts usando `pathlib`.

CÓDIGO

```
from pathlib import Path

CARPETA_DATA = Path("data")
ARCHIVO_POSTS = CARPETA_DATA / "posts.txt"


def preparar_archivos():
    CARPETA_DATA.mkdir(exist_ok=True)


def guardar_post(titulo):
    preparar_archivos()

    with open(ARCHIVO_POSTS, "a", encoding="utf-8") as archivo:
        archivo.write(titulo + "\n")


def listar_posts():
    if not ARCHIVO_POSTS.exists():
        print("Todavía no hay posts guardados.")
        return

    with open(ARCHIVO_POSTS, "r", encoding="utf-8") as archivo:
        print("Posts guardados:")

        for linea in archivo:
            titulo = linea.strip()
            print(f"- {titulo}")


guardar_post("Primeros pasos con Python")
guardar_post("Introducción a Django")
guardar_post("Archivos y rutas en Python")

listar_posts()
```

Salida posible:

CÓDIGO

```
Posts guardados:
- Primeros pasos con Python
- Introducción a Django
- Archivos y rutas en Python
```

11. Usar `Path(__file__)` para ubicar el proyecto

Hay un detalle importante.

Cuando usamos:

CÓDIGO

```
Path("data") / "posts.txt"
```

Python interpreta esa ruta desde la carpeta donde estamos ejecutando el programa.

Pero a veces eso puede traer problemas si ejecutamos el archivo desde otro lugar.

Una forma más segura es construir la ruta tomando como referencia la ubicación del archivo actual.

CÓDIGO

```
from pathlib import Path

BASE_DIR = Path(__file__).parent
CARPETA_DATA = BASE_DIR / "data"
ARCHIVO_POSTS = CARPETA_DATA / "posts.txt"

print(BASE_DIR)
print(ARCHIVO_POSTS)
```

`__file__` representa la ruta del archivo `.py` actual.

`Path(__file__).parent` obtiene la carpeta donde está ese archivo.

Entonces, si `main.py` está acá:

CÓDIGO

```
blog_consola/main.py
```

`BASE_DIR` representa:

CÓDIGO

```
blog_consola/
```

Y podemos construir rutas desde ahí.

12. Versión más segura del ejemplo del blog

CÓDIGO

```
from pathlib import Path

BASE_DIR = Path(__file__).parent
CARPETA_DATA = BASE_DIR / "data"
ARCHIVO_POSTS = CARPETA_DATA / "posts.txt"


def preparar_archivos():
    CARPETA_DATA.mkdir(exist_ok=True)


def guardar_post(titulo):
    preparar_archivos()

    with open(ARCHIVO_POSTS, "a", encoding="utf-8") as archivo:
        archivo.write(titulo + "\n")


def listar_posts():
    if not ARCHIVO_POSTS.exists():
        print("Todavía no hay posts guardados.")
        return

    with open(ARCHIVO_POSTS, "r", encoding="utf-8") as archivo:
        print("Posts guardados:")

        for linea in archivo:
            titulo = linea.strip()
            print(f"- {titulo}")


def buscar_post(termino):
    resultados = []

    if not ARCHIVO_POSTS.exists():
        return resultados

    with open(ARCHIVO_POSTS, "r", encoding="utf-8") as archivo:
        for linea in archivo:
            titulo = linea.strip()

            if termino.lower() in titulo.lower():
                resultados.append(titulo)

    return resultados


guardar_post("Primeros pasos con Python")
guardar_post("Introducción a Django")
guardar_post("Manejo de rutas con pathlib")
```

```
listar_posts()

print("\nResultados de búsqueda:")
resultados = buscar_post("python")

for resultado in resultados:
    print(f"- {resultado}")
```

Salida posible:

CÓDIGO

```
Posts guardados:
- Primeros pasos con Python
- Introducción a Django
- Manejo de rutas con pathlib

Resultados de búsqueda:
- Primeros pasos con Python
```

13. Diferencia entre `Path.cwd()` y `Path(__file__).parent`

Python también nos permite ver desde dónde se está ejecutando el programa:

CÓDIGO

```
from pathlib import Path

print(Path.cwd())
```

`Path.cwd()` significa:

CÓDIGO

```
Current Working Directory
```

Es decir, la carpeta actual desde donde se lanzó el programa.

En cambio:

CÓDIGO

```
Path(__file__).parent
```

representa la carpeta donde está guardado el archivo `.py`.

La diferencia puede ser importante.

Si el proyecto está en:

CÓDIGO

```
blog_consola/
```

pero ejecutamos el programa desde otra ubicación, `Path.cwd()` puede cambiar.

Por eso, para ubicar archivos del proyecto, suele ser más seguro usar:

CÓDIGO

```
BASE_DIR = Path(__file__).parent
```

14. Métodos útiles de `Path`

`pathlib` tiene varios métodos útiles.

CÓDIGO

```
from pathlib import Path

ruta = Path("data") / "posts.txt"

print(ruta.exists())
print(ruta.is_file())
print(ruta.is_dir())
print(ruta.name)
print(ruta.suffix)
print(ruta.parent)
```

Posibles resultados:

CÓDIGO

```
True
True
False
posts.txt
.txt
data
```

Algunos métodos importantes:

Método	Para qué sirve
`.exists()`	Verifica si la ruta existe
`.is_file()`	Verifica si es un archivo
`.is_dir()`	Verifica si es una carpeta
`.mkdir()`	Crea una carpeta
`.name`	Devuelve el nombre del archivo o carpeta
`.suffix`	Devuelve la extensión del archivo
`.parent`	Devuelve la carpeta contenedora

15. Escribir y leer texto directamente con `Path`

Además de usar `open()`, `Path` también tiene métodos propios para leer y escribir texto.

Por ejemplo:

CÓDIGO

```
from pathlib import Path

archivo = Path("data") / "mensaje.txt"

archivo.parent.mkdir(exist_ok=True)

archivo.write_text("Hola desde pathlib", encoding="utf-8")
```

Para leer:

CÓDIGO

```
contenido = archivo.read_text(encoding="utf-8")

print(contenido)
```

Esto es muy cómodo para archivos simples.

Pero para practicar y entender bien el flujo, en este curso vamos a seguir usando bastante `with open(...)`, especialmente cuando necesitemos recorrer línea por línea.

16. Usar `Path.home()`

A veces queremos acceder a la carpeta personal del usuario.

Para eso usamos:

CÓDIGO

```
from pathlib import Path

home = Path.home()

print(home)
```

En Windows podría devolver algo parecido a:

CÓDIGO

```
C:\Users\Ana
```

En Linux o macOS:

CÓDIGO

```
/home/ana
```

Por ejemplo, podríamos crear una ruta hacia la carpeta Documentos:

CÓDIGO

```
from pathlib import Path

documentos = Path.home() / "Documents"

print(documentos)
```

Para nuestro mini blog no necesitamos guardar archivos ahí todavía, pero es útil conocerlo porque muchas aplicaciones guardan configuraciones o archivos del usuario en su carpeta personal.

17. Errores comunes

Error 1: escribir rutas absolutas personales

Evitemos esto:

CÓDIGO

```
archivo = "C:\\Users\\Ana\\Desktop\\blog\\posts.txt"
```

Ese código solo funciona en la computadora de Ana.

Mejor:

CÓDIGO

```
from pathlib import Path

BASE_DIR = Path(__file__).parent
archivo = BASE_DIR / "data" / "posts.txt"
```

Error 2: olvidarse de crear la carpeta

Esto puede fallar si `data/` no existe:

CÓDIGO

```
archivo = Path("data") / "posts.txt"

with open(archivo, "a", encoding="utf-8") as f:
    f.write("Nuevo post\n")
```

Mejor:

CÓDIGO

```
carpeta = Path("data")
archivo = carpeta / "posts.txt"

carpeta.mkdir(exist_ok=True)

with open(archivo, "a", encoding="utf-8") as f:
    f.write("Nuevo post\n")
```

Error 3: confundir ruta de archivo con URL

Una ruta de archivo apunta a algo en la computadora:

CÓDIGO

```
data/posts.txt
```

Una URL apunta a una dirección web:

CÓDIGO

```
/blog/posts/1
```

Más adelante, en Django, vamos a trabajar con URLs. No son lo mismo que las rutas de archivos.

Error 4: pensar que `Path` es solo texto

Cuando creamos esto:

CÓDIGO

```
archivo = Path("data") / "posts.txt"
```

`archivo` no es exactamente un string común. Es un objeto `Path`.

La ventaja es que tiene métodos útiles:

CÓDIGO

```
archivo.exists()
archivo.name
archivo.suffix
archivo.parent
```

Y además puede usarse con `open()` sin problema.

18. Actividad práctica sugerida

Actualizar el mini blog para que guarde los posts dentro de una carpeta `data/`.

La estructura esperada es:

CÓDIGO

```
blog_consola/
│
├── main.py
│
└── data/
    └── posts.txt
```

El programa debe:

1. Usar `Path` desde `pathlib`.
2. Crear la carpeta `data/` si no existe.
3. Guardar nuevos posts en `data/posts.txt`.
4. Leer los posts desde `data/posts.txt`.
5. Verificar si el archivo existe antes de leerlo.
6. Usar `encoding="utf-8"` al abrir archivos.

Ejemplo esperado:

CÓDIGO

```
guardar_post("Primeros pasos con Python")
guardar_post("Introducción a Django")
guardar_post("Pathlib para manejar archivos")

listar_posts()
```

Salida posible:

CÓDIGO

```
Posts guardados:
- Primeros pasos con Python
- Introducción a Django
- Pathlib para manejar archivos
```

Síntesis

`pathlib` nos permite manejar rutas de archivos y carpetas de forma más segura y clara.

En lugar de escribir rutas manualmente como texto, podemos construirlas así:

CÓDIGO

```
from pathlib import Path

archivo = Path("data") / "posts.txt"
```

También podemos crear rutas basadas en la ubicación del proyecto:

CÓDIGO

```
BASE_DIR = Path(__file__).parent
archivo = BASE_DIR / "data" / "posts.txt"
```

Esto ayuda a que el programa funcione mejor en distintas computadoras y sistemas operativos.

Para nuestro blog, `pathlib` nos permite guardar los datos en una carpeta ordenada, como `data/`, y preparar el proyecto para una estructura más realista.

Organización física del proyecto de consola del blog

Aprende a estructurar las carpetas y archivos de tu proyecto de Python de forma profesional, comparando la organización del código con el diseño físico de una consola de videojuegos.

Llegamos a la última unidad del módulo. Hasta ahora aprendimos a dividir el código en módulos, agrupar archivos en paquetes, manejar rutas con `pathlib` y guardar información en archivos de texto usando `open()`.

Ahora vamos a juntar todo.

La pregunta principal de esta unidad es:

¿Cómo debería estar organizado físicamente nuestro proyecto de blog en la computadora?

Cuando un proyecto es pequeño, es común empezar con un único archivo:

CÓDIGO

```
main.py
```

Pero cuando agregamos más funcionalidades, ese archivo puede crecer demasiado.

Por ejemplo, podríamos terminar mezclando:

CÓDIGO

```
posts = []

def mostrar_menu():
    pass

def crear_post():
    pass

def listar_posts():
    pass

def guardar_post():
    pass

def leer_posts():
    pass

def validar_titulo():
    pass
```

Eso funciona al principio, pero después se vuelve difícil de leer, corregir y mantener.

Por eso, en esta unidad vamos a organizar el proyecto como una pequeña aplicación real de consola.

1. ¿Qué significa organizar físicamente un proyecto?

Organizar físicamente un proyecto significa decidir cómo se van a distribuir sus carpetas y archivos.

No se trata solo de “ordenar por ordenar”. Se trata de que cada parte tenga una responsabilidad clara.

Por ejemplo:

- el archivo principal ejecuta el programa;
- una carpeta guarda los datos;

- un módulo se encarga de leer y escribir archivos;
- otro módulo se encarga de validar datos;
- otro módulo contiene la lógica de posts.

Esto ayuda a que el proyecto sea más fácil de entender.

2. Estructura propuesta para el mini blog

Vamos a usar esta estructura:

CÓDIGO

```
mi_blog_proyecto/
│
├── main.py
├── README.md
│
├── data/
│   └── posts.txt
│
├── blog/
│   ├── __init__.py
│   ├── manager.py
│   └── storage.py
│
└── utils/
    ├── __init__.py
    └── validators.py
```

Cada parte tiene una responsabilidad.

Archivo o carpeta	Responsabilidad
`main.py`	Punto de entrada del programa
`README.md`	Explica cómo usar el proyecto
`data/`	Guarda archivos de datos
`data/posts.txt`	Guarda los posts
`blog/`	Paquete principal del blog
`blog/manager.py`	Lógica para crear, listar y buscar posts
`blog/storage.py`	Lectura y escritura de archivos
`utils/`	Funciones auxiliares
`utils/validators.py`	Validaciones de entrada del usuario

3. Crear la estructura del proyecto

Primero creamos una carpeta principal:

CÓDIGO

```
mi_blog_proyecto/
```

Dentro de esa carpeta vamos a crear los archivos y subcarpetas.

La estructura inicial puede verse así:

CÓDIGO

```
mi_blog_proyecto/
│
├── main.py
├── README.md
│
├── data/
│
├── blog/
│   ├── __init__.py
│   ├── manager.py
│   └── storage.py
│
└── utils/
    ├── __init__.py
    └── validators.py
```

El archivo `posts.txt` puede crearse automáticamente desde Python, así que no hace falta escribirlo a mano desde el principio.

4. ¿Por qué usamos `__init__.py`?

Los archivos `__init__.py` indican que una carpeta debe ser tratada como un paquete de Python.

Por ejemplo:

CÓDIGO

```
blog/
├── __init__.py
├── manager.py
└── storage.py
```

Gracias a eso, podemos importar código desde esa carpeta:

CÓDIGO

```
from blog.manager import crear_post
```

En este proyecto, los archivos `__init__.py` pueden quedar vacíos:

CÓDIGO

```
# blog/__init__.py
```

CÓDIGO

```
# utils/__init__.py
```

No hay problema con que estén vacíos. Su función principal es marcar esas carpetas como paquetes.

5. Archivo `utils/validators.py`

Este archivo tendrá funciones de validación.

Por ejemplo, validar que un título no esté vacío.

CÓDIGO

```
def validar_texto_no_vacio(texto):
    return texto.strip() != ""


def validar_opcion_menu(opcion, opciones_validas):
    return opcion in opciones_validas
```

Estas funciones son generales. Podrían usarse en varias partes del proyecto.

Por eso las guardamos en `utils/`.

6. Archivo `blog/storage.py`

Este archivo se encarga de guardar y leer información desde archivos.

Acá usamos `pathlib` y `open()`.

CÓDIGO

```
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent
DATA_DIR = BASE_DIR / "data"
POSTS_FILE = DATA_DIR / "posts.txt"


def preparar_storage():
    DATA_DIR.mkdir(exist_ok=True)


def guardar_linea(linea):
    preparar_storage()

    with open(POSTS_FILE, "a", encoding="utf-8") as archivo:
        archivo.write(linea + "\n")


def leer_lineas():
    if not POSTS_FILE.exists():
        return []

    with open(POSTS_FILE, "r", encoding="utf-8") as archivo:
        lineas = []

        for linea in archivo:
            lineas.append(linea.strip())

    return lineas
```

Veamos algunos puntos importantes:

CÓDIGO

```
BASE_DIR = Path(__file__).resolve().parent.parent
```

Esto obtiene la carpeta principal del proyecto.

Como `storage.py` está dentro de `blog/`, usamos `.parent.parent` para subir dos niveles:

CÓDIGO

```
mi_blog_proyecto/
└── blog/
    └── storage.py
```

Desde `storage.py`:

CÓDIGO

```
Path(__file__).resolve()      -> ruta completa de storage.py
.parent                       -> carpeta blog/
.parent.parent                -> carpeta mi_blog_proyecto/
```

Luego construimos la ruta al archivo de posts:

CÓDIGO

```
DATA_DIR = BASE_DIR / "data"
POSTS_FILE = DATA_DIR / "posts.txt"
```

Así evitamos escribir rutas manuales como:

CÓDIGO

```
"C:\\Users\\Ana\\Desktop\\mi_blog_proyecto\\data\\posts.txt"
```

Ese tipo de ruta solo funcionaría en una computadora.

7. Archivo `blog/manager.py`

Este archivo contiene la lógica principal relacionada con posts.

No escribe archivos directamente. Para eso usa las funciones de `storage.py`.

CÓDIGO

```
from blog.storage import guardar_linea, leer_lineas
from utils.validators import validar_texto_no_vacio


def crear_post(titulo):
    if not validar_texto_no_vacio(titulo):
        return False

    guardar_linea(titulo.strip())
    return True


def listar_posts():
    posts = leer_lineas()
    return posts


def buscar_posts(termino):
    posts = leer_lineas()
    resultados = []

    for post in posts:
        if termino.lower() in post.lower():
            resultados.append(post)

    return resultados
```

Esta separación es importante.

`manager.py` sabe trabajar con posts.

`storage.py` sabe trabajar con archivos.

`validators.py` sabe validar datos.

Cada archivo tiene una tarea clara.

8. Archivo `main.py`

Este es el punto de entrada del programa.

Es el archivo que ejecutamos desde la terminal.

CÓDIGO

```
from blog.manager import crear_post, listar_posts, buscar_posts
from utils.validators import validar_opcion_menu


def mostrar_menu():
    print("\n=== Mini Blog de Consola ===")
    print("1. Crear post")
    print("2. Listar posts")
    print("3. Buscar posts")
    print("4. Salir")

    return input("Elegí una opción: ")


def ejecutar_programa():
    continuar = True

    while continuar:
        opcion = mostrar_menu()

        if not validar_opcion_menu(opcion, ["1", "2", "3", "4"]):
            print("Opción inválida.")
            continue

        if opcion == "1":
            titulo = input("Ingresá el título del post: ")

            creado = crear_post(titulo)

            if creado:
                print("Post guardado correctamente.")
            else:
                print("El título no puede estar vacío.")

        elif opcion == "2":
            posts = listar_posts()

            if len(posts) == 0:
                print("Todavía no hay posts guardados.")
            else:
                print("\nPosts guardados:")

                for post in posts:
                    print(f"- {post}")

        elif opcion == "3":
            termino = input("Buscar por título: ")
            resultados = buscar_posts(termino)

            if len(resultados) == 0:
                print("No se encontraron posts.")
```

```
            else:
                print("\nResultados:")

                for post in resultados:
                    print(f"- {post}")

        elif opcion == "4":
            print("Saliendo del blog...")
            continuar = False


ejecutar_programa()
```

Este archivo coordina el programa, pero no hace todo por su cuenta.

Usa imports:

CÓDIGO

```
from blog.manager import crear_post, listar_posts, buscar_posts
from utils.validators import validar_opcion_menu
```

Eso conecta `main.py` con los demás módulos.

9. Cómo ejecutar el proyecto

Para ejecutar correctamente el proyecto, debemos estar parados en la carpeta raíz:

CÓDIGO

```
mi_blog_proyecto/
```

Desde ahí ejecutamos:

CÓDIGO

```
python main.py
```

No conviene ejecutar archivos internos directamente, por ejemplo:

CÓDIGO

```
python blog/manager.py
```

En este proyecto, el archivo principal es `main.py`.

10. Flujo de funcionamiento

Cuando el usuario crea un post, ocurre este recorrido:

CÓDIGO

```
main.py
  ↓
blog/manager.py
  ↓
utils/validators.py
  ↓
blog/storage.py
  ↓
data/posts.txt
```

Traducido:

1. `main.py` pide el título.
2. `manager.py` intenta crear el post.
3. `validators.py` valida que el título no esté vacío.
4. `storage.py` guarda el post en el archivo.
5. `posts.txt` almacena el dato.

Este flujo muestra cómo cada archivo participa sin mezclarse con los demás.

11. Ejemplo de uso

Al ejecutar:

CÓDIGO

```
python main.py
```

podríamos ver:

CÓDIGO

```
=== Mini Blog de Consola ===
1. Crear post
2. Listar posts
3. Buscar posts
4. Salir
Elegí una opción: 1
Ingresá el título del post: Primeros pasos con Python
Post guardado correctamente.
```

Luego:

CÓDIGO

```
=== Mini Blog de Consola ===
1. Crear post
2. Listar posts
3. Buscar posts
4. Salir
Elegí una opción: 2

Posts guardados:
- Primeros pasos con Python
```

Y si buscamos:

CÓDIGO

```
=== Mini Blog de Consola ===
1. Crear post
2. Listar posts
3. Buscar posts
4. Salir
Elegí una opción: 3
Buscar por título: python

Resultados:
- Primeros pasos con Python
```

12. Archivo `README.md`

El `README.md` funciona como la presentación del proyecto.

Es el archivo que explica qué hace el programa, cómo ejecutarlo y cómo está organizado.

Un ejemplo simple podría ser:

CÓDIGO

```
# Mini Blog de Consola

Este proyecto es un blog simple creado con Python.

Permite:

- crear posts;
- listar posts guardados;
- buscar posts por título;
- guardar la información en un archivo de texto.

## Cómo ejecutar el proyecto

Desde la carpeta principal del proyecto, ejecutar:

```bash
python main.py
```
```

Estructura del proyecto

CÓDIGO

```
mi_blog_proyecto/
├── main.py
├── README.md
├── data/
│   └── posts.txt
├── blog/
│   ├── __init__.py
│   ├── manager.py
│   └── storage.py
└── utils/
    ├── __init__.py
    └── validators.py
```

Archivos principales

- `main.py` : ejecuta el menú principal.
- `blog/manager.py` : contiene la lógica de posts.
- `blog/storage.py` : lee y escribe el archivo de posts.
- `utils/validators.py` : contiene funciones de validación.
- `data/posts.txt` : guarda los posts creados.

13. Buenas prácticas de organización

Separar responsabilidades

CÓDIGO

```
main.py          -> menú y ejecución
manager.py       -> lógica del blog
storage.py       -> archivos
validators.py    -> validaciones
```

Usar nombres claros

Evitar nombres como:

CÓDIGO

```
cosas.py
funciones.py
codigo.py
prueba.py
```

Preferir nombres que indiquen responsabilidad:

CÓDIGO

```
manager.py
storage.py
validators.py
posts.py
```

Mantener una estructura simple

No crear demasiadas carpetas si el proyecto todavía es chico.

Esto sería demasiado para este momento:

CÓDIGO

```
proyecto/
└── src/
    └── app/
        └── core/
            └── logic/
                └── handlers/
                    └── blog/
                        └── posts/
                            └── manager.py
```

Para este curso alcanza con algo más claro:

CÓDIGO

```
mi_blog_proyecto/
├── main.py
├── blog/
├── utils/
└── data/
```

Importar desde la raíz del proyecto

En `main.py`, los imports deberían verse así:

CÓDIGO

```
from blog.manager import crear_post
from utils.validators import validar_opcion_menu
```

Si movemos archivos, también debemos actualizar los imports.

Por ejemplo, si `validators.py` está dentro de `utils/`, no importamos así:

CÓDIGO

```
import validators
```

Importamos así:

CÓDIGO

```
from utils.validators import validar_opcion_menu
```

14. Errores comunes

Error 1: olvidar `__init__.py`

Si usamos carpetas como paquetes, conviene incluir `__init__.py`.

CÓDIGO

```
blog/
├── __init__.py
├── manager.py
└── storage.py
```

Aunque esté vacío, ayuda a que la estructura sea clara.

Error 2: mezclar datos y código

No conviene guardar `posts.txt` junto a los archivos `.py` internos:

CÓDIGO

```
blog/
├── manager.py
├── storage.py
└── posts.txt
```

Mejor separar datos en `data/`:

CÓDIGO

```
data/
└── posts.txt
```

Error 3: poner toda la lógica en `main.py`

Esto no es ideal:

CÓDIGO

```
# main.py

def crear_post():
    pass

def validar_post():
    pass

def guardar_archivo():
    pass

def buscar_post():
    pass

def mostrar_menu():
    pass
```

Mejor distribuir:

CÓDIGO

```
main.py
blog/manager.py
blog/storage.py
utils/validators.py
```

Error 4: usar rutas absolutas personales

Evitar:

CÓDIGO

```
archivo = "C:\\Users\\Ana\\Desktop\\mi_blog_proyecto\\data\\posts.txt"
```

Mejor:

CÓDIGO

```
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent
POSTS_FILE = BASE_DIR / "data" / "posts.txt"
```

15. Actividad práctica sugerida

Reorganizar el mini blog de consola usando la siguiente estructura:

CÓDIGO

```
mi_blog_proyecto/
│
├── main.py
├── README.md
│
├── data/
│
├── blog/
│   ├── __init__.py
│   ├── manager.py
│   └── storage.py
│
└── utils/
    ├── __init__.py
    └── validators.py
```

El proyecto debe cumplir con estos puntos:

1. `main.py` debe mostrar un menú con opciones.
2. `blog/manager.py` debe tener funciones para crear, listar y buscar posts.
3. `blog/storage.py` debe guardar y leer posts desde `data/posts.txt`.
4. `utils/validators.py` debe incluir al menos una función de validación.
5. El proyecto debe usar `pathlib`.
6. El proyecto debe usar `open()` con `encoding="utf-8"`.
7. El proyecto debe incluir un archivo `README.md`.

16. Entrega sugerida

Entregar un archivo de texto con:

1. La estructura de carpetas del proyecto.
2. El contenido de cada archivo principal.
3. Una breve explicación de qué responsabilidad cumple cada archivo.
4. Una captura o copia de una ejecución del programa en consola.

Ejemplo de explicación esperada:

CÓDIGO

```
main.py:
Es el punto de entrada del programa. Muestra el menú y llama a las funciones
principales.

blog/manager.py:
Contiene la lógica para crear, listar y buscar posts.

blog/storage.py:
Se encarga de leer y escribir el archivo data/posts.txt.

utils/validators.py:
Contiene funciones para validar entradas del usuario.

data/posts.txt:
Guarda los posts creados desde la consola.
```

Síntesis

Organizar físicamente un proyecto significa decidir dónde vive cada parte del programa.

En este mini blog separamos:

CÓDIGO

```
main.py       -> ejecución e interfaz de consola
blog/         -> lógica principal del blog
utils/        -> funciones auxiliares
data/         -> archivos de datos
README.md     -> documentación
```

Esta estructura ayuda a que el proyecto sea más claro, más fácil de corregir y más fácil de ampliar.

También prepara el camino para Django, donde trabajaremos con carpetas, archivos, paquetes, imports y responsabilidades separadas constantemente.

Con esta unidad, el blog de consola deja de ser un conjunto de scripts sueltos y empieza a parecerse a un proyecto real de Python.

Flashcards de repaso

Unidad de repaso mediante tarjetas sobre la organización de proyectos Python usando módulos, paquetes y manejo de archivos.

Para convertirte en un programador eficiente, no basta con saber escribir código; debes saber organizar ese código. A medida que tu proyecto de blog crece, tener todo en un solo archivo se vuelve insostenible. Los conceptos de **módulos**, **paquetes** y **manejo de archivos** son los pilares que te permitirán escalar tu aplicación, reutilizar lógica y persistir datos.

¿Por qué repasar estos conceptos?

- **Módulos:** Permiten dividir tu código en piezas pequeñas y manejables (archivos `.py`).
- **Paquetes:** Son carpetas que organizan módulos relacionados bajo un espacio de nombres común.
- **Manejo de archivos:** Permite que la información de tu blog (como artículos o configuración) no desaparezca cuando cierras el programa.

Estrategia de estudio con Flashcards

Para sacar el máximo provecho, sigue estos pasos: 1. **Recuerdo Activo:** Antes de mirar el reverso de la tarjeta, intenta responder la pregunta en voz alta o por escrito. Este esfuerzo mental es lo que realmente consolida la memoria. 2. **Sinceridad:** Si tardas más de unos segundos en recordar, marca la tarjeta como "difícil" y repásala pronto. 3. **Contextualización:** Trata de visualizar cómo estos conceptos se aplican específicamente a la estructura de tu blog.

¡Comencemos a consolidar los fundamentos de la organización de proyectos en Python!

01 ¿Qué es un módulo en Python?

Un módulo es cualquier archivo con extensión `.py` que contiene código ejecutable, funciones o clases. Permite organizar el código en archivos separados para mejorar la legibilidad y facilitar la reutilización.

02 ¿Qué es un paquete en Python?

Un paquete es una carpeta que contiene uno o más módulos. Se utiliza para agrupar módulos relacionados jerárquicamente, facilitando la organización de proyectos grandes.

03 ¿Cuál es la función del archivo `__init__.py` dentro de una carpeta?

El archivo `__init__.py` indica a Python que la carpeta donde se encuentra debe ser tratada como un paquete. Permite realizar inicializaciones en el paquete cuando este es importado.

04 ¿Cuál es la diferencia entre usar `import` y `from ... import ...`?

La sentencia `import nombre_modulo` carga todo el módulo, mientras que `from nombre_modulo import funcion` permite importar solo una función o clase específica, evitando cargar elementos innecesarios al espacio de nombres actual.

05 En la función `open()`, ¿qué diferencia hay entre los modos `'r'` y `'w'`?

El modo `'r'` abre el archivo para lectura, mientras que el modo `'w'` abre el archivo para escritura (sobreescribiéndolo). Otros modos incluyen `'a'` para añadir contenido al final del archivo.

06 ¿Por qué es recomendable utilizar la sentencia `with` al trabajar con archivos?

Es una buena práctica porque asegura que el archivo se cierre automáticamente al terminar el bloque de código, liberando los recursos del sistema incluso si ocurre un error durante la ejecución.

07 ¿Por qué usar `pathlib` para manejar rutas en lugar de cadenas de texto (strings)?

`pathlib` proporciona una interfaz orientada a objetos para manejar rutas del sistema de archivos, haciendo el código más legible y multiplataforma que el uso de strings simples.

08 ¿Qué ventaja principal aporta la modularización a un proyecto a largo plazo?

Mantiene el código ordenado, permite separar la lógica de negocio de la interfaz de usuario, y facilita la colaboración al permitir que diferentes personas trabajen en módulos distintos simultáneamente.

PRE-ENTREGA DEL PROYECTO

Actividad evaluable · parte del proyecto integrador

Preentrega 5: Blog por consola organizado en módulos y paquetes

Organizar el sistema de blog por consola en módulos y paquetes, separando datos, menú, operaciones y validaciones en distintos archivos para mejorar la estructura del proyecto.

Preentrega 5: Organización Modular del Blog

¡Llegó el momento de profesionalizar tu estructura de código!

En esta preentrega, vas a transformar el script único que venías trabajando en un sistema modular y organizado. Hasta ahora, tu blog por consola podía listar posts, buscar por título, filtrar por tag y validar información. Ahora vas a separar ese código en distintos archivos, como se hace en proyectos reales de software.

Propósito

El objetivo de esta práctica es separar las responsabilidades de tu programa para que sea más fácil de leer, mantener y escalar.

Vas a dividir tu sistema en partes más claras:

- Datos.
- Menú e interacción con el usuario.
- Operaciones del blog.
- Validaciones.
- Archivo principal de ejecución.

Esto te permitirá dejar atrás el archivo único y empezar a trabajar con una estructura más parecida a la de un proyecto real.

Contexto de la cadena de entregables

En el **Módulo 1**, configuraste tu entorno y ejecutaste tus primeros scripts en Python.

En el **Módulo 2**, modelaste la estructura de datos del blog usando listas, diccionarios, tuplas y sets.

En el **Módulo 3**, construiste un sistema interactivo por consola con menú, búsqueda, filtrado y salida controlada.

En el **Módulo 4**, refactorizaste ese sistema usando funciones, parámetros, retornos, validaciones y manejo básico de errores.

Ahora, en el **Módulo 5**, vas a reorganizar ese mismo sistema en varios archivos Python, usando módulos, paquetes e imports.

La idea no es crear un sistema nuevo desde cero, sino **ordenar mejor el sistema que ya venías construyendo**.

Objetivo de aprendizaje de la preentrega:

El objetivo de esta preentrega es consolidar la capacidad de organizar un programa en varios archivos.

Vas a practicar:

Módulos

Separar el código en distintos archivos `.py` según su responsabilidad.

Paquetes

Organizar varios módulos dentro de una carpeta con `__init__.py`.

Imports

Usar `from blog.modulo import funcion` para conectar el archivo principal con los módulos del paquete.

Separación de responsabilidades

Evitar que todo el programa quede concentrado en un único archivo.

Documentación básica

Crear un `README.md` que explique cómo ejecutar el sistema y cómo está organizado el proyecto.

Integración

Verificar que las partes separadas sigan funcionando juntas al ejecutar `python main.py`.

Qué construir

Debés crear una nueva versión del sistema de blog por consola, organizada en módulos y paquetes.

El proyecto debe tener una estructura similar a esta:

CÓDIGO

```
blog_consola/
│
├── main.py
├── README.md
│
└── blog/
    ├── __init__.py
    ├── datos.py
    ├── menu.py
    ├── operaciones.py
    └── validaciones.py
```

Archivos esperados

`main.py`

Debe ser el archivo principal del programa.

Desde este archivo se debe:

- Importar las funciones y datos necesarios.
- Iniciar el sistema.
- Coordinar el flujo del menú.
- Llamar a las funciones correspondientes según la opción elegida.
- Proteger la ejecución principal con `if __name__ == "__main__":`.

Este archivo no debería contener toda la lógica del sistema. Su responsabilidad principal es **orquestar** las partes.

`blog/__init__.py`

Debe existir dentro de la carpeta `blog/`.

Este archivo permite que Python reconozca la carpeta como un paquete.

Puede estar vacío.

`blog/datos.py`

Debe contener las estructuras de datos base del sistema, por ejemplo:

- `perfil_autor`
- `estados_post`
- `etiquetas_blog`
- `posts`

Debe conservar la continuidad de los módulos anteriores.

El autor debe seguir siendo un diccionario anidado dentro de cada post.

`blog/menu.py`

Debe contener la función encargada del menú y del manejo de `input()`.

Por ejemplo, una función que:

- Muestre las opciones disponibles.
- Solicite una opción al usuario.
- Retorne la opción elegida.

El menú sugerido es:

CÓDIGO

```
--- MENU DEL BLOG ---
1. Ver todos los posts
2. Buscar por titulo
3. Filtrar por tag
4. Validar posts
5. Salir
```

blog/operaciones.py

Debe contener las funciones principales del blog, especialmente las relacionadas con búsqueda y filtrado.

Por ejemplo:

- `listar_posts(lista)`
- `buscar_por_titulo(lista, termino)`
- `filtrar_por_tag(lista, tag)`

Estas funciones deben recibir datos por parámetro y no depender completamente de variables globales.

blog/validaciones.py

Debe contener las reglas lógicas del sistema.

Como mínimo, debe incluir una función para validar posts.

La validación puede revisar:

- Que el post sea un diccionario.
- Que tenga las claves necesarias.
- Que el título no esté vacío.
- Que el contenido no esté vacío.
- Que el autor sea un diccionario.
- Que el autor tenga nombre.
- Que los tags estén guardados como lista.
- Que el estado sea válido.

README.md

Debe explicar brevemente:

- Qué hace el programa.
- Cómo ejecutar el sistema.
- Cómo está organizada la carpeta.
- Qué responsabilidad cumple cada módulo.

- Qué archivo se debe ejecutar.

Funcionalidades requeridas

El sistema debe conservar las funcionalidades principales trabajadas en módulos anteriores.

Opción 1: Ver todos los posts

El programa debe mostrar todos los posts disponibles.

Debe mostrar, como mínimo:

- Título.
- Nombre del autor.
- Estado del post.

Opción 2: Buscar por título

El programa debe pedir un término de búsqueda y mostrar los posts cuyo título coincida total o parcialmente.

La búsqueda debe ignorar mayúsculas y minúsculas.

Opción 3: Filtrar por tag

El programa debe pedir un tag y mostrar solo los posts que tengan esa etiqueta.

El filtrado también debe ignorar mayúsculas y minúsculas.

Opción 4: Validar posts

El programa debe recorrer los posts cargados en memoria y validar su estructura.

Debe mostrar cuáles posts son válidos y cuáles tienen errores.

Opción 5: Salir

El programa debe mostrar un mensaje de despedida y terminar su ejecución.

Implementación de imports

En `main.py`, debés conectar las piezas usando imports desde el paquete `blog`.

La forma esperada es:

CÓDIGO

```
from blog.modulo import funcion
```

Por ejemplo:

CÓDIGO

```
from blog.datos import posts
from blog.menu import mostrar_menu
from blog.operaciones import listar_posts, buscar_por_titulo, filtrar_por_tag
from blog.validaciones import validar_post
```

No es obligatorio que uses exactamente esos nombres de funciones, pero sí debe quedar claro que el archivo principal importa y utiliza funciones desde los módulos del paquete.

Prueba de integración

Después de separar el código en módulos, debés probar que el sistema sigue funcionando correctamente.

Desde la carpeta raíz del proyecto, ejecutá:

CÓDIGO

```
python main.py
```

Verificá que:

- El programa inicia correctamente.
- El menú se muestra.
- Las opciones llaman a las funciones correspondientes.
- La búsqueda funciona.
- El filtrado funciona.
- Las validaciones funcionan.
- La opción de salir finaliza el programa.
- No aparecen errores de importación.

Criterios de aceptación

Para considerar este checkpoint como completado, tu proyecto debe cumplir con los siguientes puntos:

- Crear una carpeta raíz para el proyecto.
- Crear un paquete llamado `blog/`.
- Incluir un archivo `__init__.py` dentro de `blog/`.
- Crear un archivo principal llamado `main.py`.
- Crear un archivo `README.md`.
- Separar los datos base en `blog/datos.py`.
- Separar el menú y el manejo de `input()` en `blog/menu.py`.
- Separar las funciones de búsqueda y filtrado en `blog/operaciones.py`.

- Separar las reglas de validación en `blog/validaciones.py`.
- Usar imports desde `main.py`, por ejemplo con la forma `from blog.modulo import funcion`.
- Mantener las estructuras trabajadas en módulos anteriores.
- Mantener el autor como diccionario anidado.
- Conservar el menú interactivo del blog.
- Permitir listar posts, buscar por título, filtrar por tag y validar posts.
- Usar `.lower()` en búsquedas y filtros.
- Organizar la ejecución principal dentro de `if __name__ == "__main__":`.
- Verificar que el programa funcione ejecutando `python main.py`.
- Subir el código a un repositorio público de GitHub.
- El repositorio debe ser accesible para revisión.

Pasos sugeridos

1. Crear la estructura de directorios

Creá una carpeta raíz para el proyecto.

Dentro de esa carpeta, creá una carpeta llamada `blog/`.

Dentro de `blog/`, agregá el archivo `__init__.py`.

También creá los archivos principales del proyecto:

- `main.py`
- `README.md`
- `blog/datos.py`
- `blog/menu.py`
- `blog/operaciones.py`
- `blog/validaciones.py`

2. Distribuir el código

Tomá el código que ya venías trabajando y separalo según su responsabilidad.

Una distribución posible es:

- Las estructuras de datos base van en `datos.py`.
- Las funciones de búsqueda y filtrado van en `operaciones.py`.
- El manejo del menú y del `input()` va en `menu.py`.
- Las reglas lógicas y validaciones van en `validaciones.py`.
- La coordinación general del programa queda en `main.py`.

3. Implementar los imports

En `main.py`, importá las funciones y datos que necesites desde los módulos del paquete `blog`.

Usá imports claros y evitá copiar funciones entre archivos.

4. Proteger la ejecución principal

En `main.py`, asegurate de que la lógica inicial del programa esté dentro del bloque:

CÓDIGO

```
if __name__ == "__main__":
```

Esto permite que el archivo principal controle la ejecución del sistema sin mezclar la definición de funciones con el arranque del programa.

5. Probar la integración

Ejecutá el programa desde la terminal usando:

CÓDIGO

```
python main.py
```

Si aparece un error de importación, revisá:

- Que la carpeta `blog/` exista.
- Que `blog/` tenga el archivo `__init__.py`.
- Que los nombres de los archivos coincidan con los imports.
- Que estés ejecutando el comando desde la carpeta raíz del proyecto.

6. Documentar el proyecto

Redactá un `README.md` breve.

Debe explicar:

- Qué hace el sistema.
- Cómo ejecutarlo.
- Qué contiene cada archivo.
- Qué funcionalidades tiene el menú.

7. Subir el proyecto a GitHub

Creá un repositorio público en GitHub y subí el código del proyecto.

Podés hacerlo usando Git desde la terminal o cargando los archivos manualmente desde la página de GitHub.

El enlace del repositorio será la entrega del checkpoint.

Subida manual a GitHub sin usar Git

Si todavía no usás Git desde la terminal, podés subir el proyecto desde el navegador.

Pasos generales:

1. Entrá a GitHub e iniciá sesión.
2. Creá un repositorio público como muestran las imágenes. (de la segunda imagen solo completar el nombre y crear, las demas opciones se veran despues)
3. Elegí **Uploading an existing file**.
4. Arrastrá los archivos y carpetas del proyecto, o seleccionalos desde tu computadora.
5. Confirmá la subida (clickeando en "Commit changes") con un mensaje breve, por ejemplo: `Entrega módulo 5`.
6. Copiá el enlace del repositorio y entregalo en la plataforma.

Importante: esta forma sirve para entregar el proyecto sin usar comandos de Git. Más adelante, aprenderemos Git porque es la herramienta estándar para trabajar con repositorios de código.

Errores comunes a evitar

Circular imports

Evitá que dos módulos se importen entre sí.

Por ejemplo, si `operaciones.py` importa datos desde `datos.py`, no conviene que `datos.py` también importe funciones desde `operaciones.py`.

Esto puede generar errores difíciles de entender.

Scripts sueltos

Asegurate de que toda la lógica de ejecución inicial esté protegida por el bloque:

CÓDIGO

```
if __name__ == "__main__":
```

en `main.py`.

Poner todo el código en `main.py`

El archivo `main.py` debe coordinar el programa, pero no debería contener toda la lógica.

La búsqueda, el filtrado, los datos y las validaciones deben estar separados en sus módulos correspondientes.

Copiar funciones en varios archivos

Si necesitás usar una función desde otro archivo, importala.

No dupliques el mismo código en distintos módulos.

Olvidar el archivo `__init__.py`

Para que una carpeta funcione como paquete, debe incluir un archivo `__init__.py`.

Aunque esté vacío, debe estar presente.

Usar imports incorrectos

Si un import falla, revisá la ubicación de los archivos y cómo estás haciendo los imports.

La estructura de carpetas debe coincidir con la forma en que importás.

Ejemplo de import esperado:

CÓDIGO

```
from blog.operaciones import buscar_por_titulo
```

Ejecutar el programa desde una ubicación incorrecta

Si ejecutás `main.py` desde una carpeta distinta a la raíz del proyecto, pueden aparecer errores de importación.

Ubicate primero en la carpeta donde está `main.py` y luego ejecutá:

CÓDIGO

```
python main.py
```

Romper la continuidad de los módulos anteriores

El proyecto debe seguir usando la estructura de datos del blog trabajada previamente.

El autor debe seguir siendo un diccionario anidado, no un texto simple.

Incorrecto:

CÓDIGO

```
"autor": "Ana López"
```

Correcto:

CÓDIGO

```
"autor": perfil_autor
```

Agregar funcionalidades fuera del alcance

Este checkpoint no busca trabajar con archivos JSON, bases de datos ni persistencia.

El foco está en organizar el código usando módulos, paquetes e imports.

Formato de entrega

La entrega debe realizarse mediante un repositorio público de GitHub.

El repositorio debe contener la estructura completa del proyecto:

CÓDIGO

```
blog_consola/
│
├── main.py
├── README.md
│
└── blog/
    ├── __init__.py
    ├── datos.py
    ├── menu.py
    ├── operaciones.py
    └── validaciones.py
```

Asegurate de que:

- El repositorio sea público.
- El código esté organizado en los archivos solicitados.
- El `README.md` explique cómo ejecutar el sistema.
- El programa funcione con `python main.py`.
- El enlace del repositorio sea accesible para revisión.

Conexión con el futuro

Este checkpoint es un paso importante entre el blog de consola y una aplicación más grande.

Separar el código en módulos y paquetes te prepara para trabajar con proyectos reales, donde cada archivo tiene una responsabilidad específica.

Más adelante, cuando trabajes con archivos, persistencia de datos o Django, esta organización te va a ayudar a entender mejor cómo se estructuran las aplicaciones web.

En otras palabras: en el Módulo 4 hiciste que el sistema fuera más claro usando funciones. En el Módulo 5, vas a convertirlo en un proyecto mejor organizado.

Pasos para la entrega

Propósito

El objetivo es separar las responsabilidades de tu programa (datos, lógica, interfaz y validaciones) para que sea escalable y fácil de mantener, tal como se trabaja en proyectos reales de software.

Pasos sugeridos

1. **Estructura de Directorios**: Crea una carpeta raíz para tu proyecto y dentro crea la carpeta `blog/`. No olvides el archivo `__init__.py` dentro del paquete para que Python lo reconozca como tal.
2. **Distribución de Código**: Mueve las funciones de búsqueda y filtrado a `operaciones.py`, los diccionarios base a `datos.py`, el manejo del `input()` a `menu.py` y las reglas lógicas a `validaciones.py`.

3. **Implementación de Imports**: En tu archivo `main.py`, utiliza `from blog.modulo import funcion` para conectar todas las piezas.
4. **Prueba de Integración**: Ejecuta `python main.py` y asegúrate de que el flujo del menú sigue funcionando correctamente.
5. **Documentación**: Redacta un `README.md` que explique cómo ejecutar el sistema y cómo está organizado.
6. **Subida**: Sube el código a un repositorio público en GitHub.

Errores comunes

- **Circular Imports**: Evita que `datos.py` importe de `operaciones.py` si este último ya importa de `datos.py`.
- **Scripts Sueltos**: Asegúrate de que toda la lógica de ejecución inicial esté protegida por el bloque `if __name__ == "__main__":` en `main.py`.

Entregable

Formato: repositorio

Un repositorio público en GitHub que contenga la carpeta del paquete blog/, el archivo main.py y un README informativo.

Objetivos de aprendizaje

- Estructurar un proyecto de Python utilizando paquetes y módulos.
- Implementar la comunicación entre archivos mediante sentencias de importación (import/from).
- Aplicar el principio de separación de responsabilidades en funciones de consola.
- Gestionar la ejecución principal del programa mediante el uso de if **name** == '**main**'.

Criterios de aceptación

- El proyecto cuenta con un paquete llamado 'blog' que contiene un archivo **init**.py.
- El archivo main.py importa y ejecuta correctamente las funciones residentes en el paquete blog.
- La función de validación comprueba la existencia de las claves requeridas y el tipo de dato del autor (diccionario).
- El sistema permite buscar posts e ignorar mayúsculas/minúsculas utilizando .lower().
- El repositorio incluye un archivo README.md con instrucciones claras de ejecución.

Rúbrica de evaluación

Esta rúbrica evalúa un repositorio público de GitHub con un proyecto de blog por consola reorganizado en módulos y paquetes. Se valora la correcta separación de responsabilidades, la integración funcional entre archivos, el uso adecuado de imports y estructura de paquete, la organización de los datos base del proyecto y la documentación. La rúbrica está pensada para un nivel principiante, pero con criterios propios de un proyecto real.

Criterio	Descripción	Peso
Estructura del proyecto y organización en módulos/ paquetes	Evalúa si el repositorio presenta una arquitectura clara y coherente con la consigna: carpeta raíz del proyecto, paquete blog/ con init.py, separación de archivos por responsabilidad y presencia de main.py en la raíz.	30%
Uso correcto de imports y evitación de dependencias problemáticas	Evalúa si el proyecto conecta correctamente sus módulos mediante imports explícitos, sin errores de importación, sin circular imports y con una integración coherente entre main.py y el paquete blog/.	20%
Funcionamiento integrado del blog por consola	Evalúa si el sistema sigue funcionando al ejecutar python main.py: navegación por el menú, acceso a funciones básicas, uso correcto de la lógica movida a distintos módulos y conservación del comportamiento esperado del blog por consola.	25%
Organización de los datos base del proyecto	Evalúa si los datos base del blog están correctamente ubicados en blog/datos.py y separados de la lógica, el menú y la validación, manteniendo una división de responsabilidades clara entre los módulos.	10%
Documentación del repositorio y guía de ejecución	Evalúa si el README.md explica cómo ejecutar el sistema, cómo está organizado el proyecto y brinda suficiente información para que otra persona pueda entender y probar el repositorio.	8%
Calidad de entrega del repositorio y cumplimiento de la consigna	Evalúa si la entrega final corresponde a un repositorio público en GitHub, con los archivos solicitados y una presentación general completa y usable como entregable real.	7%

Total: 100 pts | Aprobación: 70 pts

MÓDULO 6

JSON y programación orientada a objetos

Implementar persistencia estructurada con JSON y modelar entidades de dominio mediante Programación Orientada a Objetos.

JSON: de estructuras de Python a texto intercambiable

Explora los fundamentos de JSON, aprendiendo a transformar estructuras de datos de Python en un formato de texto universal para el intercambio y almacenamiento de información.

JSON: guardar y recuperar datos del blog

CÓDIGO

```
posts = [
    {
        "id": 1,
        "titulo": "Mi primer post",
        "autor": "Ana",
        "estado": "publicado"
    }
]
```

Mientras el programa está abierto, podemos usar esa lista sin problema. Pero si cerramos Python, esos datos desaparecen.

Para que la información pueda guardarse en un archivo y volver a usarse después, necesitamos convertir nuestras estructuras de Python a un formato de texto. Uno de los formatos más usados para eso es **JSON**.

1. ¿Qué es JSON?

JSON significa **JavaScript Object Notation**.

Aunque viene del mundo de JavaScript, hoy se usa en casi todos los lenguajes de programación. Es un formato de texto que permite representar datos de una forma simple, ordenada y fácil de leer.

Un JSON se parece mucho a un diccionario de Python:

CÓDIGO

```
{
    "titulo": "Mi primer post",
    "autor": "Ana",
    "estado": "publicado"
}
```

La diferencia importante es que JSON tiene sus propias reglas.

Por ejemplo:

- Usa `true` y `false` en minúscula.
- Usa `null` en lugar de `None`.
- Las claves y los textos van siempre con comillas dobles.
- No permite comas sobrantes al final.

2. ¿Por qué usar JSON en nuestro blog?

En nuestro proyecto de blog, JSON puede servir para guardar información como:

- Posts.
- Autores.
- Categorías.
- Etiquetas.
- Configuraciones simples.

Por ejemplo, podríamos guardar los posts del blog en un archivo llamado `posts.json` y recuperarlos cuando volvamos a ejecutar el programa.

Esto nos permite dar un paso importante: pasar de tener datos temporales en memoria a tener datos persistentes en archivos.

3. Comparación entre Python y JSON

Python	JSON	Ejemplo en JSON
`dict`	object	`{"titulo": "Python"}`
`list`	array	`["python", "django"]`
`str`	string	`"Hola"`
`int` / `float`	number	`10`, `3.14`
`True`	true	`true`
`False`	false	`false`
`None`	null	`null`

Ejemplo en Python:

CÓDIGO

```
post = {
    "titulo": "Aprendiendo JSON",
    "vistas": 120,
    "estado": "publicado",
    "comentarios": None,
    "etiquetas": ["python", "archivos", "blog"]
}
```

El mismo dato representado como JSON:

CÓDIGO

```
{
    "titulo": "Aprendiendo JSON",
    "vistas": 120,
    "estado": "publicado",
    "comentarios": null,
    "etiquetas": ["python", "archivos", "blog"]
}
```

4. El módulo `json` de Python

Python incluye un módulo llamado `json` que nos permite trabajar con este formato.

Para usarlo, primero debemos importarlo:

CÓDIGO

```
import json
```

Con este módulo podemos hacer dos cosas principales:

- Convertir datos de Python a JSON.
- Convertir JSON a datos de Python.

5. Convertir un diccionario de Python a texto JSON

A este proceso se lo llama **serialización**.

Significa tomar una estructura de Python y convertirla en texto JSON.

CÓDIGO

```
import json

post = {
    "titulo": "Mi primer post",
    "autor": "Ana",
    "estado": "publicado",
    "etiquetas": ["python", "blog"]
}

post_json = json.dumps(post)

print(post_json)
```

Salida posible:

CÓDIGO

```
{"titulo": "Mi primer post", "autor": "Ana", "estado": "publicado", "etiquetas":
["python", "blog"]}
```

Observa que Python transformó el diccionario en una cadena con formato JSON. Si el dato incluyera booleanos, también convertiría los valores de Python `True` y `False` a los valores JSON `true` y `false`.

6. Mejorar la lectura del JSON con `indent`

El JSON anterior funciona, pero se ve todo en una sola línea.

Para que sea más fácil de leer, podemos usar `indent`.

CÓDIGO

```
import json

post = {
    "titulo": "Mi primer post",
    "autor": "Ana",
    "estado": "publicado",
    "etiquetas": ["python", "blog"]
}

post_json = json.dumps(post, indent=4)

print(post_json)
```

Salida:

CÓDIGO

```
{
    "titulo": "Mi primer post",
    "autor": "Ana",
    "estado": "publicado",
    "etiquetas": [
        "python",
        "blog"
    ]
}
```

`indent=4` indica que queremos una sangría de 4 espacios.

Esto es útil cuando queremos que el archivo sea legible para una persona.

7. Guardar datos en un archivo JSON

Para guardar datos en un archivo, usamos `json.dump()`.

La diferencia es importante:

- `json.dumps()` convierte a texto JSON.
- `json.dump()` guarda directamente en un archivo.

Ejemplo:

CÓDIGO

```
import json

posts = [
    {
        "id": 1,
        "titulo": "Introducción a Python",
        "autor": "Ana",
        "estado": "publicado"
    },
    {
        "id": 2,
        "titulo": "Listas y diccionarios",
        "autor": "Luis",
        "estado": "borrador"
    }
]

with open("posts.json", "w", encoding="utf-8") as archivo:
    json.dump(posts, archivo, indent=4, ensure_ascii=False)

print("Posts guardados correctamente.")
```

Después de ejecutar este código, se crea un archivo llamado `posts.json`.

El contenido del archivo será:

CÓDIGO

```
[
    {
        "id": 1,
        "titulo": "Introducción a Python",
        "autor": "Ana",
        "estado": "publicado"
    },
    {
        "id": 2,
        "titulo": "Listas y diccionarios",
        "autor": "Luis",
        "estado": "borrador"

    }
]
```

Usamos `ensure_ascii=False` para que Python respete caracteres como tildes y ñ.

8. Leer datos desde un archivo JSON

Para leer un archivo JSON y convertirlo nuevamente en una estructura de Python, usamos `json.load()`.

CÓDIGO

```
import json

with open("posts.json", "r", encoding="utf-8") as archivo:
    posts = json.load(archivo)

print(posts)
print(type(posts))
```

Salida posible:

CÓDIGO

```
[
    {
        "id": 1,
        "titulo": "Introducción a Python",
        "autor": "Ana",
        "estado": "publicado"
    },
    {
        "id": 2,
        "titulo": "Listas y diccionarios",
        "autor": "Luis",
        "estado": "borrador"
    }
]
```

Y el tipo de dato será:

CÓDIGO

```
<class 'list'>
```

Esto significa que Python leyó el archivo JSON y reconstruyó la lista original.

9. Acceder a los datos después de leer el JSON

Una vez que cargamos el archivo, podemos trabajar con los datos como si fueran listas y diccionarios normales de Python.

CÓDIGO

```
import json

with open("posts.json", "r", encoding="utf-8") as archivo:
    posts = json.load(archivo)

for post in posts:
    print("Título:", post["titulo"])
    print("Autor:", post["autor"])
    print("Estado:", post["estado"])
    print("---")
```

Salida posible:

CÓDIGO

```
Título: Introducción a Python
Autor: Ana
Estado: publicado

## Clases y objetos: la base de un modelo de dominio

_Aprende a modelar las entidades de tu blog (Posts y Usuarios) usando Programación
Orientada a Objetos en Python para crear código organizado y escalable._

### Introducción a la Programación Orientada a Objetos

En módulos anteriores aprendimos a guardar información usando variables, listas y
diccionarios.

Por ejemplo, podríamos representar un post de un blog así:

```python
post = {
    "titulo": "Mi primer post",
    "autor": "Ana",
    "contenido": "Estoy aprendiendo Python.",
    "estado": "borrador"
}
```

Esto funciona, pero a medida que el programa crece puede volverse difícil de mantener.

Imaginemos que tenemos muchos posts:

CÓDIGO

```
posts = [
    {
        "titulo": "Mi primer post",
        "autor": "Ana",
        "contenido": "Estoy aprendiendo Python.",
        "estado": "borrador"
    },
    {
        "titulo": "Listas en Python",
        "autor": "Luis",
        "contenido": "Las listas permiten guardar varios datos.",
        "estado": "publicado"
    }
]
```

El problema es que dependemos de que todos los diccionarios tengan las mismas claves. Si en uno escribimos `"titulo"` y en otro `"title"`, el programa puede empezar a fallar.

Además, si queremos agregar acciones como publicar, editar o mostrar un resumen del post, esas acciones quedan separadas de los datos.
```

Para organizar mejor esta información, Python nos ofrece la **Programación Orientada a Objetos**, también conocida como **POO**.

1. ¿Qué es la Programación Orientada a Objetos?

La Programación Orientada a Objetos es una forma de organizar el código agrupando datos y acciones relacionadas dentro de una misma estructura.

En lugar de tener por separado:

- Datos del post.
- Funciones para publicar.
- Funciones para editar.
- Funciones para mostrar información.

Podemos crear un objeto `Post` que tenga sus propios datos y sus propias acciones.

2. ¿Qué es una clase?

Una **clase** es una plantilla o molde para crear objetos.

Podemos imaginar una clase como el molde para fabricar galletas.

El molde define la forma general, pero no es una galleta real. Sirve para crear muchas galletas con la misma estructura.

En programación, una clase define qué datos tendrá un objeto y qué acciones podrá realizar.

Ejemplo:

CÓDIGO

```
class Post:
    def __init__(self, titulo, autor):
        self.titulo = titulo
        self.autor = autor
```

En este caso, `Post` es una clase.

Define que cada post tendrá:

- Un título.
- Un autor.

3. ¿Qué es un objeto?

Un **objeto** es un elemento creado a partir de una clase.

Si la clase es el molde, el objeto es la galleta real.

Por ejemplo:

CÓDIGO

```
class Post:
    def __init__(self, titulo, autor):
        self.titulo = titulo
        self.autor = autor


mi_post = Post("Mi primer post", "Ana")
```

En este código:

CÓDIGO

```
Post
```

es la clase.

Y:

CÓDIGO

```
mi_post
```

es un objeto creado a partir de esa clase.

También podemos decir que `mi_post` es una **instancia** de la clase `Post`.

4. El método `__init__`

El método `__init__` se ejecuta automáticamente cuando creamos un nuevo objeto.

Sirve para definir los datos iniciales del objeto.

CÓDIGO

```
class Post:
    def __init__(self, titulo, autor):
        self.titulo = titulo
        self.autor = autor
```

Cuando escribimos:

CÓDIGO

```
mi_post = Post("Mi primer post", "Ana")
```

Python ejecuta automáticamente el método `__init__`.

Entonces:

CÓDIGO

```
self.titulo = "Mi primer post"
self.autor = "Ana"
```

El objeto queda creado con esos datos.

5. ¿Qué es `self`?

`self` representa al objeto actual.

Puede parecer raro al principio, pero podemos pensarlo como una forma de decir:

> [!nota] “Este dato pertenece a este objeto específico”.

Ejemplo:

CÓDIGO

```
class Post:
    def __init__(self, titulo, autor):
        self.titulo = titulo
        self.autor = autor
```

En esta línea:

CÓDIGO

```
self.titulo = titulo
```

Estamos diciendo:

> [!nota] El título de este post será el valor que recibimos en el parámetro `titulo`.

Gracias a `self`, cada objeto puede guardar sus propios datos.

Ejemplo:

CÓDIGO

```
class Post:
    def __init__(self, titulo, autor):
        self.titulo = titulo
        self.autor = autor


post_1 = Post("Introducción a Python", "Ana")
post_2 = Post("Listas y diccionarios", "Luis")

print(post_1.titulo)
print(post_2.titulo)
```

Salida:

CÓDIGO

```
Introducción a Python
Listas y diccionarios
```

Aunque ambos objetos fueron creados con la misma clase, cada uno tiene sus propios datos.

6. Atributos: lo que el objeto tiene

Los **atributos** son los datos que pertenecen a un objeto.

En nuestro blog, un post puede tener:

- `titulo`
- `autor`
- `contenido`
- `estado`

Ejemplo:

CÓDIGO

```
class Post:
    def __init__(self, titulo, autor, contenido):
        self.titulo = titulo
        self.autor = autor
        self.contenido = contenido
        self.estado = "borrador"
```

Creamos un objeto:

CÓDIGO

```
post = Post(
    "Aprendiendo POO",
    "Ana",
    "La programación orientada a objetos ayuda a organizar mejor el código."
)
```

Accedemos a sus atributos usando punto:

CÓDIGO

```
print(post.titulo)
print(post.autor)
print(post.estado)
```

Salida:

CÓDIGO

```
Aprendiendo POO
Ana
borrador
```

7. Métodos: lo que el objeto hace

Los **métodos** son funciones que pertenecen a una clase.

Representan acciones que puede realizar un objeto.

Por ejemplo, un post podría:

- Publicarse.
- Editar su título.
- Mostrar un resumen.

Ejemplo:

CÓDIGO

```
class Post:
    def __init__(self, titulo, autor, contenido):
        self.titulo = titulo
        self.autor = autor
        self.contenido = contenido
        self.estado = "borrador"

    def publicar(self):
        self.estado = "publicado"
        print(f"El post '{self.titulo}' fue publicado.")
```

Usamos el método:

CÓDIGO

```
post = Post("Aprendiendo POO", "Ana", "Contenido del post...")

print(post.estado)

post.publicar()

print(post.estado)
```

Salida:

CÓDIGO

```
borrador
El post 'Aprendiendo POO' fue publicado.
publicado
```

El método `publicar()` modificó el estado interno del objeto.

Antes, `estado` era `"borrador"`.

Después de ejecutar el método, pasó a ser `"publicado"`.

8. Agregar más métodos a la clase

Podemos agregar más acciones.

Por ejemplo, editar el título:

CÓDIGO

```
class Post:
    def __init__(self, titulo, autor, contenido):
        self.titulo = titulo
        self.autor = autor
        self.contenido = contenido
        self.estado = "borrador"

    def publicar(self):
        self.estado = "publicado"
        print(f"El post '{self.titulo}' fue publicado.")

    def editar_titulo(self, nuevo_titulo):
        self.titulo = nuevo_titulo
        print("El título fue actualizado.")
```

Probamos el código:

CÓDIGO

```
post = Post("Titulo viejo", "Ana", "Contenido del post...")

print(post.titulo)

post.editar_titulo("Título nuevo")

print(post.titulo)
```

Salida:

CÓDIGO

```
Titulo viejo
El título fue actualizado.
Título nuevo
```

9. Método para mostrar información del post

También podemos crear un método que muestre la información del objeto de forma ordenada.

CÓDIGO

```
class Post:
    def __init__(self, titulo, autor, contenido):
        self.titulo = titulo
        self.autor = autor
        self.contenido = contenido
        self.estado = "borrador"

    def publicar(self):
        self.estado = "publicado"

    def mostrar_info(self):
        print("Título:", self.titulo)
        print("Autor:", self.autor)
        print("Estado:", self.estado)
```

Uso:

CÓDIGO

```
post = Post("Introducción a Python", "Ana", "Contenido del post...")

post.mostrar_info()
```

Salida:

CÓDIGO

```
Título: Introducción a Python
Autor: Ana
Estado: borrador
```

10. Crear varios objetos

Una clase nos permite crear muchos objetos con la misma estructura.

CÓDIGO

```
class Post:
    def __init__(self, titulo, autor, contenido):
        self.titulo = titulo
        self.autor = autor
        self.contenido = contenido
        self.estado = "borrador"

    def publicar(self):
        self.estado = "publicado"

    def mostrar_info(self):
        print("Título:", self.titulo)
        print("Autor:", self.autor)
        print("Estado:", self.estado)
        print("---")


post_1 = Post("Introducción a Python", "Ana", "Contenido del primer post")
post_2 = Post("Listas en Python", "Luis", "Contenido del segundo post")
post_3 = Post("Diccionarios en Python", "Marta", "Contenido del tercer post")

post_1.publicar()
post_3.publicar()

post_1.mostrar_info()
post_2.mostrar_info()
post_3.mostrar_info()
```

Salida:

CÓDIGO

````
Título: Introducción a Python
Autor: Ana
Estado: publicado

## Modelado de Post y Usuario para un blog más claro

_En esta unidad aprenderás a organizar la información de tu blog definiendo
responsabilidades claras para las clases Post y Usuario, evitando la mezcla de datos
y preparando la estructura para un desarrollo profesional con Django._

### Modelado de `Post` y `Usuario` en un blog

En la unidad anterior vimos los conceptos básicos de Programación Orientada a
Objetos:

* Clase.
* Objeto.
* Atributos.
* Métodos.
* `__init__`.
* `self`.

Ahora vamos a aplicar esos conceptos al proyecto del curso: nuestro blog.

Hasta ahora, podríamos representar un post usando un diccionario:

```python
post = {
    "titulo": "Mi primer post",
    "autor": "Ana",
    "contenido": "Estoy aprendiendo Python.",
    "estado": "borrador"
}
````

Esto funciona para ejemplos simples, pero en un blog real vamos a tener muchos posts y varios usuarios.

Si seguimos usando solo diccionarios, pueden aparecer problemas:
````

CÓDIGO

```
post_1 = {
    "titulo": "Mi primer post",
    "autor": "Ana",
    "contenido": "Estoy aprendiendo Python."
}

post_2 = {
    "title": "Otro post",
    "author": "Luis",
    "texto": "Contenido del segundo post."
}
```

Los dos diccionarios representan posts, pero usan claves distintas:

- `titulo` / `title`
- `autor` / `author`
- `contenido` / `texto`

Esto vuelve el código más difícil de mantener.

La Programación Orientada a Objetos nos permite crear estructuras más claras y ordenadas.

En esta unidad vamos a modelar dos entidades principales:

- `Usuario`
- `Post`

1. ¿Qué significa modelar?

Modelar significa decidir qué elementos importantes existen en nuestro sistema y qué información necesita cada uno.

En un blog, podemos identificar rápidamente dos elementos importantes:

- La persona que escribe.
- La publicación que se muestra.

En código, esos elementos pueden convertirse en clases.

CÓDIGO

```
class Usuario:
    pass


class Post:
    pass
```

Por ahora están vacías, pero ya estamos diciendo algo importante:

> **Nota**
>
> En nuestro sistema existen usuarios y posts.

2. Separar responsabilidades

Una buena clase debe tener una responsabilidad clara.

La clase `Usuario` debe representar a una persona.

La clase `Post` debe representar una publicación.

No conviene mezclar todo en una sola clase.

Por ejemplo, esto sería poco claro:

CÓDIGO

```
class Blog:
    def __init__(self, titulo_post, contenido_post, nombre_autor, email_autor):
        self.titulo_post = titulo_post
        self.contenido_post = contenido_post
        self.nombre_autor = nombre_autor
        self.email_autor = email_autor
```

El problema es que esta clase mezcla datos del post con datos del usuario.

¿Qué representa este objeto?

¿Un blog? ¿Un post? ¿Un usuario?

La idea de la POO es separar mejor las responsabilidades.

3. Clase `Usuario`

La clase `Usuario` representa a la persona que escribe posts en el blog.

Podemos darle estos atributos:

- `username`
- `email`
- `nombre`

Ejemplo:

CÓDIGO

```
class Usuario:
    def __init__(self, username, email, nombre):
        self.username = username
        self.email = email
        self.nombre = nombre
```

Ahora podemos crear un usuario:

CÓDIGO

```
usuario_1 = Usuario("ana_dev", "ana@email.com", "Ana")
```

Y acceder a sus datos:

CÓDIGO

```
print(usuario_1.username)
print(usuario_1.email)
print(usuario_1.nombre)
```

Salida:

CÓDIGO

```
ana_dev
ana@email.com
Ana
```

4. Agregar métodos a `Usuario`

Además de guardar datos, una clase también puede tener métodos.

Por ejemplo, podemos crear un método para mostrar el perfil del usuario.

CÓDIGO

```
class Usuario:
    def __init__(self, username, email, nombre):
        self.username = username
        self.email = email
        self.nombre = nombre

    def mostrar_perfil(self):
        print("Usuario:", self.username)
        print("Nombre:", self.nombre)
        print("Email:", self.email)
```

Uso:

CÓDIGO

```
usuario_1 = Usuario("ana_dev", "ana@email.com", "Ana")

usuario_1.mostrar_perfil()
```

Salida:

CÓDIGO

```
Usuario: ana_dev
Nombre: Ana
Email: ana@email.com
```

5. Clase `Post`

La clase `Post` representa una publicación del blog.

Un post puede tener:

- `titulo`
- `contenido`
- `autor`
- `estado`

Ejemplo inicial:

CÓDIGO

```
class Post:
    def __init__(self, titulo, contenido, autor):
        self.titulo = titulo
        self.contenido = contenido
        self.autor = autor
        self.estado = "borrador"
```

Observa algo importante:

CÓDIGO

```
self.autor = autor
```

El autor no será simplemente un texto como `"Ana"`.

El autor será un objeto de la clase `Usuario`.

6. Relacionar `Post` con `Usuario`

Primero creamos un usuario:

CÓDIGO

```
class Usuario:
    def __init__(self, username, email, nombre):
        self.username = username
        self.email = email
        self.nombre = nombre


usuario_1 = Usuario("ana_dev", "ana@email.com", "Ana")
```

Después creamos un post usando ese usuario como autor:

CÓDIGO

```
class Post:
    def __init__(self, titulo, contenido, autor):
        self.titulo = titulo
        self.contenido = contenido
        self.autor = autor
        self.estado = "borrador"


post_1 = Post(
    "Introducción a Python",
    "En este post vamos a aprender los primeros pasos con Python.",
    usuario_1
)
```

Ahora `post_1` tiene un autor conectado.

Podemos acceder al nombre del autor así:

CÓDIGO

```
print(post_1.autor.nombre)
```

Salida:

CÓDIGO

```
Ana
```

La lectura sería:

> **Nota**
>
> Del post 1, quiero acceder a su autor, y de ese autor quiero obtener su nombre.

7. ¿Por qué usar un objeto como autor?

Podríamos haber hecho esto:

CÓDIGO

```
post = {
    "titulo": "Introducción a Python",
    "autor": "Ana"
}
```

Pero eso guarda solamente el nombre.

En cambio, si usamos un objeto `Usuario`, tenemos más información disponible:

CÓDIGO

```
print(post_1.autor.nombre)
print(post_1.autor.email)
print(post_1.autor.username)
```

Salida:

CÓDIGO

```
Ana
ana@email.com
ana_dev
```

Esto es mucho más flexible.

Si en el futuro el usuario tiene más datos, como una biografía o una foto de perfil, el post puede acceder a esa información a través del objeto autor.

8. Agregar métodos a `Post`

Un post puede tener acciones propias.

Por ejemplo:

- Publicarse.
- Mostrar un resumen.
- Mostrar la información completa.

CÓDIGO

```
class Post:
    def __init__(self, titulo, contenido, autor):
        self.titulo = titulo
        self.contenido = contenido
        self.autor = autor
        self.estado = "borrador"

    def publicar(self):
        self.estado = "publicado"
        print(f"El post '{self.titulo}' fue publicado.")

    def mostrar_resumen(self):
        print("Título:", self.titulo)
        print("Autor:", self.autor.nombre)
        print("Estado:", self.estado)
```

Uso:

CÓDIGO

```
usuario_1 = Usuario("ana_dev", "ana@email.com", "Ana")

post_1 = Post(
    "Introducción a Python",
    "En este post vamos a aprender los primeros pasos con Python.",
    usuario_1
)

post_1.mostrar_resumen()

post_1.publicar()

post_1.mostrar_resumen()
```

Salida:

CÓDIGO

```
Título: Introducción a Python
Autor: Ana
Estado: borrador
El post 'Introducción a Python' fue publicado.
Título: Introducción a Python
Autor: Ana
Estado: publicado
```

9. Relación uno a muchos

En un blog, un usuario puede escribir muchos posts.

Pero cada post tiene un solo autor principal.

A esto se lo conoce como una relación **uno a muchos**.

En nuestro caso:

- Un `Usuario` puede tener muchos `Post`.
- Un `Post` pertenece a un `Usuario`.

Ejemplo:

CÓDIGO

```
usuario_1 = Usuario("ana_dev", "ana@email.com", "Ana")

post_1 = Post("Introducción a Python", "Contenido del primer post", usuario_1)
post_2 = Post("Listas en Python", "Contenido del segundo post", usuario_1)
post_3 = Post("Diccionarios en Python", "Contenido del tercer post", usuario_1)
```

Los tres posts tienen el mismo autor:

CÓDIGO

```
print(post_1.autor.nombre)
print(post_2.autor.nombre)
print(post_3.autor.nombre)
```

Salida:

CÓDIGO

```
Ana
Ana
Ana
```

10. Guardar posts en una lista

Podemos guardar los posts en una lista para recorrerlos.

CÓDIGO

```
posts = [post_1, post_2, post_3]

for post in posts:
    post.mostrar_resumen()
```

Salida posible:

CÓDIGO

```
Título: Introducción a Python
Autor: Ana
Estado: borrador
Título: Listas en Python
Autor: Ana
Estado: borrador
Título: Diccionarios en Python
Autor: Ana
Estado: borrador
```

11. Filtrar posts por autor

Como cada post tiene un autor, podemos buscar posts escritos por determinado usuario.

CÓDIGO

```
usuario_1 = Usuario("ana_dev", "ana@email.com", "Ana")
usuario_2 = Usuario("luis_py", "luis@email.com", "Luis")

post_1 = Post("Introducción a Python", "Contenido del primer post", usuario_1)
post_2 = Post("Listas en Python", "Contenido del segundo post", usuario_1)
post_3 = Post("HTML básico", "Contenido del tercer post", usuario_2)

posts = [post_1, post_2, post_3]

print("Posts escritos por Ana:")

for post in posts:
    if post.autor.username == "ana_dev":
        print("-", post.titulo)
```

Salida:

CÓDIGO

```
Posts escritos por Ana:
- Introducción a Python
- Listas en Python
```

12. Publicar solo algunos posts

También podemos trabajar con el estado de publicación.

CÓDIGO

```
post_1.publicar()
post_3.publicar()

print("Posts publicados:")

for post in posts:
    if post.estado == "publicado":
        print("-", post.titulo)
```

Salida:

CÓDIGO

```
El post 'Introducción a Python' fue publicado.
El post 'HTML básico' fue publicado.
Posts publicados:
- Introducción a Python
- HTML básico
```

13. Diagrama simple de clases

Podemos representar nuestras clases de forma visual.

CÓDIGO

```
Usuario
-------------------------
username
email
nombre
-------------------------
mostrar_perfil()


Post
-------------------------
titulo
contenido
autor
publicado
-------------------------
publicar()
mostrar_resumen()
```

La relación entre ambas sería:

CÓDIGO

```
Usuario 1  --------  muchos Post
```

Dicho de otra forma:

CÓDIGO

```
Un usuario puede escribir muchos posts.
Un post tiene un autor.
```

Este tipo de relación aparecerá más adelante en Django cuando trabajemos con modelos y bases de datos.

14. Ejemplo completo

CÓDIGO

```
class Usuario:
    def __init__(self, username, email, nombre):
        self.username = username
        self.email = email
        self.nombre = nombre

    def mostrar_perfil(self):
        print("Usuario:", self.username)
        print("Nombre:", self.nombre)
        print("Email:", self.email)


class Post:
    def __init__(self, titulo, contenido, autor):
        self.titulo = titulo
        self.contenido = contenido
        self.autor = autor
        self.estado = "borrador"

    def publicar(self):
        self.estado = "publicado"
        print(f"El post '{self.titulo}' fue publicado.")

    def mostrar_resumen(self):
        print("Título:", self.titulo)
        print("Autor:", self.autor.nombre)
        print("Estado:", self.estado)
        print("---")


usuario_1 = Usuario("ana_dev", "ana@email.com", "Ana")
usuario_2 = Usuario("luis_py", "luis@email.com", "Luis")

post_1 = Post("Introducción a Python", "Contenido del primer post", usuario_1)
post_2 = Post("Listas en Python", "Contenido del segundo post", usuario_1)
post_3 = Post("HTML básico", "Contenido del tercer post", usuario_2)

post_1.publicar()
post_3.publicar()

posts = [post_1, post_2, post_3]

print("Todos los posts:")
for post in posts:
    post.mostrar_resumen()

print("Posts publicados:")
for post in posts:
    if post.estado == "publicado":
        print("-", post.titulo)
```

```
print("Posts escritos por Ana:")
for post in posts:
    if post.autor.username == "ana_dev":
        print("-", post.titulo)
```

Salida esperada:

CÓDIGO

```
El post 'Introducción a Python' fue publicado.
El post 'HTML básico' fue publicado.

Todos los posts:
Título: Introducción a Python
Autor: Ana
Estado: publicado

## Integrar objetos y JSON en el blog por consola

_Aprende a conectar la Programación Orientada a Objetos con archivos JSON para que
los datos de tu blog se guarden permanentemente en el disco._

### Integrar objetos y JSON en el blog

Hasta ahora, nuestro blog por consola puede crear usuarios y publicaciones usando
Programación Orientada a Objetos.

Por ejemplo, podemos tener una clase `Usuario` y una clase `Post`.

Eso nos permite trabajar con objetos más claros y ordenados.

El problema es que, si cerramos la terminal, todos esos objetos desaparecen.

Para evitarlo, necesitamos guardar la información en un archivo.

En esta unidad vamos a usar JSON como archivo de datos para que nuestro blog pueda
conservar la información entre ejecuciones.

#### 1. El problema: los objetos no se guardan solos

Imaginemos que tenemos esta clase:

```python
class Post:
    def __init__(self, titulo, contenido, autor):
        self.titulo = titulo
        self.contenido = contenido
        self.autor = autor
        self.estado = "borrador"
```

Y creamos un objeto:
```

CÓDIGO

```
post = Post("Mi primer post", "Estoy aprendiendo Python.", "Ana")
```

Python entiende perfectamente qué es `post`.

Pero un archivo `.json` no entiende qué es un objeto de la clase `Post`.

JSON solo puede guardar datos simples como:

- textos
- números
- booleanos
- listas
- diccionarios
- `null`

Por eso, antes de guardar un objeto en JSON, tenemos que convertirlo en un diccionario.

2. El camino de ida y vuelta

Cuando trabajamos con objetos y JSON, normalmente hacemos dos procesos.

Guardar datos

Para guardar un objeto:

CÓDIGO

```
Objeto -> Diccionario -> JSON
```

Primero convertimos el objeto en un diccionario.

Después guardamos ese diccionario en un archivo JSON.

Cargar datos

Para recuperar un objeto:

CÓDIGO

```
JSON -> Diccionario -> Objeto
```

Primero leemos el archivo JSON.

Python nos devuelve diccionarios.

Después usamos esos diccionarios para crear nuevamente objetos.

3. Crear una clase `Post`

Empecemos con una versión simple de la clase `Post`.

CÓDIGO

```
class Post:
    def __init__(self, titulo, contenido, autor):
        self.titulo = titulo
        self.contenido = contenido
        self.autor = autor
        self.estado = "borrador"

    def publicar(self):
        self.estado = "publicado"

    def mostrar_resumen(self):
        print("Título:", self.titulo)
        print("Autor:", self.autor)
        print("Estado:", self.estado)
        print("---")
```

Creamos un post:

CÓDIGO

```
post = Post("Introducción a Python", "Contenido del post...", "Ana")

post.mostrar_resumen()
```

Salida:

CÓDIGO

```
Título: Introducción a Python
Autor: Ana
Estado: borrador

<section class="preentrega pre" markdown="1">
<span class="badge">PRE-ENTREGA DEL PROYECTO</span>
<p class="bajada">Actividad evaluable · parte del proyecto integrador</p>

## Preentrega 6: blog por consola con POO y JSON

_Guía de pre-entrega para consolidar el blog por consola utilizando Programación
Orientada a Objetos y persistencia de datos en formato JSON._

### Preentrega 6: Refactorización a POO y Persistencia JSON

¡Llegó el momento de dar un salto importante en el diseño de tu proyecto!

Hasta ahora, tu blog por consola fue creciendo paso a paso: primero como datos
sueltos, luego como estructura de colecciones, después como menú interactivo y más
adelante como proyecto modular organizado en varios archivos.

En esta preentrega, vas a transformar ese sistema basado principalmente en
diccionarios y funciones hacia un modelo de **Programación Orientada a Objetos**,
incorporando además **carga y guardado de datos en archivos JSON**.

#### Propósito

El objetivo de esta práctica es refactorizar el blog de consola para que use clases
y objetos, representando mejor las entidades principales del sistema.

Además, vas a incorporar persistencia usando archivos JSON, para que los posts
puedan guardarse y recuperarse entre ejecuciones del programa.

Vas a trabajar sobre el mismo repositorio del módulo anterior, actualizando el
proyecto que ya venías construyendo.

#### Contexto de la cadena de entregables

En el **Módulo 1**, configuraste tu entorno y ejecutaste tus primeros scripts en
Python.

En el **Módulo 2**, modelaste la estructura de datos del blog usando listas,
diccionarios, tuplas y sets.

En el **Módulo 3**, construiste un sistema interactivo por consola con menú,
búsqueda, filtrado y salida controlada.

En el **Módulo 4**, refactorizaste el sistema usando funciones, parámetros,
retornos, validaciones y manejo básico de errores.

En el **Módulo 5**, organizaste el proyecto en módulos y paquetes, separando datos,
```

```
menú, operaciones y validaciones.

Ahora, vas a continuar sobre ese mismo proyecto para aplicar dos conceptos nuevos:

* Programación Orientada a Objetos.
* Persistencia de datos con JSON.

La idea no es crear un sistema desde cero, sino mejorar el proyecto modular que ya
venías trabajando.

#### Objetivo de aprendizaje de la preentrega

El objetivo de esta preentrega es consolidar la capacidad de modelar un sistema
usando clases, objetos y archivos externos.

Vas a practicar:

##### Clases

Crear clases para representar entidades del sistema, como autores, posts y el blog.

##### Objetos

Instanciar objetos a partir de esas clases y usarlos dentro del programa.

##### Atributos

Guardar información dentro de objetos mediante atributos.

##### Métodos

Mover parte de la lógica del sistema a métodos de clase.

##### Composición

Hacer que un objeto `Post` tenga como autor una instancia de la clase `Autor`.

##### Persistencia JSON

Guardar y cargar información desde un archivo `posts.json`.

##### Serialización

Convertir objetos a diccionarios para poder guardarlos en JSON.

##### Integración

Conectar el menú del programa con la nueva clase `Blog`.

#### Qué construir

Debés continuar trabajando sobre el repositorio público de GitHub utilizado en el
checkpoint anterior.
```

```
No crees un repositorio nuevo.

En este checkpoint vas a actualizar el proyecto existente para incorporar:

* Clases.
* Objetos.
* Métodos.
* Persistencia con JSON.
* Integración del menú con la clase `Blog`.

#### Estructura sugerida del proyecto

El proyecto puede mantener una estructura similar a esta:
```

blog_consola/ | ├── main.py ├── README.md ├── posts.json | └── blog/ ├── **init**.py ├── datos.py ├── menu.py ├── modelos.py ├── operaciones.py └── validaciones.py

CÓDIGO

```
En este checkpoint se agrega o toma especial importancia el archivo:
```

blog/modelos.py

CÓDIGO

```
También se modifica el archivo:
```

blog/datos.py

CÓDIGO

```
para manejar la carga y guardado desde JSON.

### Requisitos técnicos

#### 1. Preparación del repositorio

Debés continuar trabajando en el mismo repositorio de GitHub usado en el módulo
anterior.

El proyecto debe actualizarse con los cambios de este checkpoint y luego subirse
nuevamente al repositorio.

El `README.md` también debe actualizarse para explicar los cambios realizados.

#### 2. Modelado de clases en `blog/modelos.py`

Debés crear un archivo llamado:
```

blog/modelos.py

CÓDIGO

```
Dentro de ese archivo, definí las clases principales del sistema.

Como mínimo, el proyecto debe incluir:

* `Autor`
* `Post`
* `Blog`

#### Clase `Autor`

La clase `Autor` debe representar la información de una persona autora del blog.

Puede incluir atributos como:

* `nombre`
* `bio`

También puede incluir otros atributos si ya los venías usando, por ejemplo:

* `especialidad`
* `redes_sociales`

Esta clase reemplaza el uso directo de un diccionario para representar autores
dentro del funcionamiento principal del sistema.

#### Clase `Post`

La clase `Post` debe representar una publicación del blog.

Puede incluir atributos como:

* `id`
* `titulo`
* `contenido`
* `autor`
* `tags`
* `estado`

El atributo `autor` debe recibir una instancia de la clase `Autor`, no un string
simple.

Es decir, un post debe estar relacionado con un objeto autor.

#### Clase `Blog`

La clase `Blog` debe centralizar la lógica principal del sistema.

Debe tener un atributo que sea una lista de objetos `Post`.

Esa clase puede incluir métodos para:
```

```
* Listar posts.
* Buscar posts por título.
* Filtrar posts por tag.
* Crear un nuevo post.
* Validar posts.
* Obtener todos los posts cargados.

La idea es que la clase `Blog` sea el punto principal desde donde se maneja la
colección de publicaciones.

#### 3. Conversión entre objetos y diccionarios

Como JSON no puede guardar objetos personalizados directamente, vas a necesitar
convertir los objetos a estructuras compatibles.

Para guardar datos en JSON, los objetos deben convertirse a diccionarios.

Por ejemplo:

* Un objeto `Autor` debe poder representarse como diccionario.
* Un objeto `Post` debe poder representarse como diccionario.
* La lista de posts del blog debe poder convertirse en una lista de diccionarios.

También debe existir alguna forma de reconstruir objetos a partir de los
diccionarios cargados desde JSON.

No hace falta que uses un nombre obligatorio, pero pueden ser métodos o funciones
con responsabilidades como:

* Convertir un autor a diccionario.
* Convertir un post a diccionario.
* Crear un autor desde un diccionario.
* Crear un post desde un diccionario.

#### 4. Persistencia con JSON en `blog/datos.py`

El archivo `blog/datos.py` debe encargarse de la carga y guardado de datos.

Debés modificarlo o ampliarlo para trabajar con un archivo llamado:
```

posts.json

CÓDIGO

```
Al iniciar el programa, se deben cargar los posts desde ese archivo.

Si el archivo no existe, está vacío o tiene datos inválidos, el programa debe
manejar la situación sin cerrarse inesperadamente.

También debe existir una función para guardar los posts actuales en `posts.json`.

#### 5. Crear nuevos posts

El menú debe permitir crear un nuevo post desde consola.

Al crear un nuevo post:

* Se deben pedir los datos necesarios al usuario.
* Se debe crear una instancia de `Post`.
* Esa instancia debe agregarse al objeto `Blog`.
* El post debe poder convertirse a diccionario para guardarse en `posts.json`.

Esto permite verificar que la persistencia JSON realmente funcione con objetos
nuevos creados durante la ejecución.

#### 6. Integración del menú

El sistema debe seguir funcionando desde consola.

En `main.py`, se debe instanciar la clase `Blog`.

El menú debe interactuar con esa instancia de `Blog`, llamando a sus métodos para
listar, buscar, filtrar, crear y validar posts.

El menú sugerido es:
```

--- MENU DEL BLOG --- 1. Ver todos los posts 2. Buscar por titulo 3. Filtrar por tag 4. Crear nuevo post 5. Validar posts 6. Guardar posts en JSON 7. Salir

CÓDIGO

```
No es obligatorio que el menú sea exactamente igual, pero debe permitir probar las
funcionalidades principales del checkpoint.

#### 7. Integración en `main.py`

El archivo `main.py` debe ser el punto de entrada del programa.

Debe encargarse de:

* Importar las clases y funciones necesarias.
* Cargar los datos desde `posts.json`.
* Crear una instancia de `Blog`.
* Mostrar el menú.
* Llamar a los métodos correspondientes según la opción elegida.
* Guardar los datos cuando corresponda.
* Proteger la ejecución principal con `if __name__ == "__main__":`.

La lógica principal del sistema no debería quedar toda escrita en `main.py`.

#### 8. Actualización del README

El archivo `README.md` debe actualizarse.

Debe explicar:

* Qué hace el sistema.
* Cómo ejecutar el programa.
* Qué clases principales existen.
* Qué responsabilidad tiene cada clase.
* Cómo interactúan las clases con el archivo `posts.json`.
* Cómo se guarda y carga la información.
* Qué cambió respecto al checkpoint anterior.

### Funcionalidades requeridas

#### Opción 1: Ver todos los posts

El programa debe mostrar los posts cargados actualmente en el blog.

Debe mostrar, como mínimo:

* Título.
* Autor.
* Estado.

Esta funcionalidad debe usar la instancia de la clase `Blog`.

#### Opción 2: Buscar por título

El programa debe permitir buscar publicaciones por título.
```

```
La búsqueda debe ignorar mayúsculas y minúsculas.

Esta funcionalidad debe resolverse mediante un método de la clase `Blog`.

#### Opción 3: Filtrar por tag

El programa debe permitir filtrar publicaciones según una etiqueta.

El filtrado debe ignorar mayúsculas y minúsculas.

Esta funcionalidad debe resolverse mediante un método de la clase `Blog`.

#### Opción 4: Crear nuevo post

El programa debe permitir crear un nuevo post desde consola.

El nuevo post debe crearse como una instancia de la clase `Post`.

Luego debe agregarse a la lista de posts del objeto `Blog`.

#### Opción 5: Validar posts

El programa debe poder validar que los posts tengan datos correctos.

Puede hacerlo mediante métodos de la clase `Blog`, funciones del módulo
`validaciones.py` o una combinación de ambos.

Lo importante es que la validación siga existiendo y funcione con la nueva
estructura orientada a objetos.

#### Opción 6: Guardar posts en JSON

El programa debe guardar los posts actuales en el archivo `posts.json`.

Para eso, los objetos deben convertirse a diccionarios compatibles con JSON.

#### Opción 7: Salir

El programa debe mostrar un mensaje de despedida y finalizar correctamente.

Podés decidir si el programa guarda automáticamente antes de salir o si el guardado
queda como opción manual del menú.

### Manejo de errores

El programa debe evitar cerrarse inesperadamente.

Debe manejar, como mínimo:

* El archivo `posts.json` no existe.
* El archivo `posts.json` está vacío.
* El archivo `posts.json` tiene contenido inválido.
```

```
* El usuario ingresa una opción incorrecta en el menú.
* El usuario deja campos vacíos al crear un post.
* Se intenta guardar un objeto sin convertirlo antes a diccionario.
* Se intenta reconstruir un objeto desde un diccionario incompleto.

Los mensajes de error deben ser claros y ayudar a entender qué ocurrió.

### Criterios de aceptación

Para considerar este checkpoint como completado, tu proyecto debe cumplir con los
siguientes puntos:

* Continuar sobre el repositorio de GitHub del módulo anterior.
* Mantener una estructura modular.
* Incluir el archivo `blog/modelos.py`.
* Definir la clase `Autor`.
* Definir la clase `Post`.
* Definir la clase `Blog`.
* La clase `Post` debe recibir una instancia de `Autor`.
* La clase `Blog` debe tener una lista de objetos `Post`.
* La clase `Blog` debe centralizar la lógica principal del sistema.
* El programa debe cargar datos desde `posts.json` al iniciar.
* El programa debe poder guardar datos en `posts.json`.
* Los objetos deben convertirse a diccionarios antes de guardarse en JSON.
* Los diccionarios cargados desde JSON deben convertirse nuevamente en objetos.
* El menú debe usar una instancia de `Blog`.
* El programa debe permitir listar posts.
* El programa debe permitir buscar posts por título.
* El programa debe permitir filtrar posts por tag.
* El programa debe permitir crear un nuevo post.
* El programa debe permitir guardar posts en JSON.
* La búsqueda y el filtrado deben ignorar mayúsculas y minúsculas.
* El programa debe manejar errores de archivo con claridad.
* `main.py` debe usar `if __name__ == "__main__":`.
* El `README.md` debe estar actualizado.
* Los cambios deben subirse al repositorio de GitHub.

### Pasos sugeridos

#### 1. Continuar desde el repositorio anterior

Usá el mismo repositorio del checkpoint anterior.

No crees un proyecto nuevo: este checkpoint es una mejora sobre el sistema modular
que ya venías trabajando.

#### 2. Crear el archivo `modelos.py`

Dentro del paquete `blog`, creá el archivo `modelos.py`.

Ese archivo debe contener las clases principales del sistema.

#### 3. Modelar la clase `Autor`
```

```
Identificá qué datos del autor venías usando en los módulos anteriores.

Luego, transformá esos datos en atributos de una clase.

#### 4. Modelar la clase `Post`

Tomá la estructura anterior de cada post y convertí esos datos en atributos de una
clase.

Recordá que el autor del post debe ser un objeto de la clase `Autor`.

#### 5. Modelar la clase `Blog`

Creá una clase que represente el blog completo.

Esta clase debe guardar una lista de posts y ofrecer métodos para trabajar con esa
lista.

#### 6. Adaptar la lógica anterior

Revisá las funciones que antes listaban, buscaban y filtraban posts.

Decidí cuáles de esas funciones conviene transformar en métodos de la clase `Blog`.

#### 7. Preparar la conversión a JSON

Antes de guardar objetos en JSON, pensá cómo convertirlos a diccionarios.

También necesitás poder hacer el camino inverso: leer diccionarios desde JSON y
convertirlos en objetos.

#### 8. Modificar `datos.py`

Adaptá el módulo de datos para que pueda cargar y guardar posts usando el módulo
`json`.

Este módulo debe ocuparse de la persistencia del sistema.

#### 9. Actualizar `main.py`

Instanciá la clase `Blog`.

Luego, conectá las opciones del menú con los métodos de ese objeto.

#### 10. Probar el sistema completo

Probá especialmente estos casos:

* Ejecutar el programa con `posts.json` existente.
* Ejecutar el programa sin `posts.json`.
* Crear un nuevo post.
* Guardar los posts.
```

```
* Cerrar y volver a abrir el programa.
* Verificar que el post creado siga estando disponible.
* Buscar por título.
* Filtrar por tag.
* Ingresar opciones inválidas en el menú.

#### 11. Actualizar el README

Documentá los cambios principales.

Explicá cómo se usan las clases y cómo funciona la persistencia JSON.

#### 12. Subir los cambios a GitHub

Subí los cambios al mismo repositorio de GitHub que venías usando.

Asegurate de que el repositorio sea accesible para revisión.

### Errores comunes a evitar

#### Crear clases que no se usan

No alcanza con definir `Autor`, `Post` y `Blog` si después el programa sigue usando
únicamente diccionarios.

Las clases deben participar en el funcionamiento real del sistema.

#### Intentar guardar objetos directamente en JSON

JSON no sabe guardar objetos personalizados de Python directamente.

Antes de guardar, convertí los objetos a diccionarios, listas, strings, números o
booleanos.

#### No reconstruir objetos al cargar JSON

Si cargás datos desde JSON y seguís trabajando solo con diccionarios, no estás
completando la refactorización a POO.

Los datos cargados deben convertirse nuevamente en objetos del sistema.

#### Dejar toda la lógica en `main.py`

El archivo `main.py` debe coordinar el programa, no contener toda la lógica.

La lógica principal debe estar en las clases y módulos correspondientes.

#### Romper la estructura modular

Agregar POO no significa volver a un único archivo.

El proyecto debe seguir organizado en módulos.
```

```
#### Olvidar actualizar el README

Si el proyecto cambió su estructura interna, el README debe reflejarlo.

#### No probar la persistencia

No alcanza con guardar una vez.

Probá cerrar el programa, volver a ejecutarlo y verificar que los datos guardados se
carguen correctamente.

### Formato de entrega

La entrega debe realizarse mediante el mismo repositorio público de GitHub utilizado
en el checkpoint anterior.

Asegurate de que:

* El repositorio siga siendo público.
* Los cambios estén subidos correctamente.
* El código esté organizado en módulos.
* Exista el archivo `blog/modelos.py`.
* Exista o se genere el archivo `posts.json`.
* El `README.md` esté actualizado.
* El programa funcione ejecutando:
```

python main.py

CÓDIGO

```
El enlace del repositorio será la entrega del checkpoint.

### Conexión con el futuro

Este checkpoint marca un cambio importante en la forma de pensar el sistema.

Hasta ahora, el blog estaba construido principalmente con datos en diccionarios y
funciones que operaban sobre ellos. Ahora, empezás a modelar el problema usando
objetos: autores, posts y un blog que centraliza el comportamiento principal.

Además, al incorporar JSON, los datos del sistema pueden persistir más allá de la
ejecución del programa.

Más adelante, cuando trabajes con Django, esta forma de pensar te va a resultar
familiar: Django también trabaja con modelos, objetos, métodos y persistencia de
datos.

### Pasos para la entrega

1. **Preparación del Repositorio:**
    * Continúa trabajando en el repositorio de GitHub que utilizaste en el módulo
anterior.
2. **Modelado de Clases (`blog/modelos.py`):**
    * Define la clase `Autor` con atributos como nombre y bio.
    * Define la clase `Post` que reciba una instancia de `Autor` y contenga los
datos de la publicación.
    * Define la clase `Blog`, la cual debe centralizar la lógica. Esta clase debe
tener un atributo que sea una lista de objetos `Post`.
3. **Persistencia con JSON (`blog/datos.py`):**
    * Modifica o crea funciones para que, al iniciar el programa, se lean los datos
desde un archivo `posts.json`.
    * Asegúrate de que al crear un nuevo post, este se convierta a un formato
compatible (diccionario) para guardarse en el archivo JSON mediante el módulo `json`
de Python.
4. **Integración del Menú:**
    * En `main.py`, instancia la clase `Blog`.
    * Actualiza el menú para que los métodos de búsqueda y listado llamen a las
funciones de tu nueva clase `Blog`.
5. **Documentación:**
    * Actualiza el `README.md` explicando cómo las clases interactúan con el archivo
JSON y cómo ejecutar el proyecto.

### Entregable

**Formato:** repositorio

Enlace al repositorio de GitHub actualizado con la estructura modular, clases POO y
persistencia JSON.

### Objetivos de aprendizaje
```

```
- Implementar clases y objetos para modelar entidades de dominio (Autor, Post,
Blog).
- Gestionar la persistencia de datos utilizando el módulo json de Python.
- Organizar la lógica de negocio dentro de métodos de clase para mejorar la
modularidad.
- Vincular objetos de distintas clases mediante composición (un Post tiene un
Autor).

### Criterios de aceptación

- El repositorio contiene el archivo `blog/modelos.py` con las clases Autor, Post y
Blog definidas.
- El programa carga y guarda la información en un archivo `.json` de forma exitosa.
- El archivo `main.py` utiliza una instancia de la clase `Blog` para gestionar las
operaciones.
- El menú de consola permite listar y buscar posts consumiendo los datos procesados
por las clases.
- El código sigue una estructura de paquetes y módulos (uso de `__init__.py`).

<div class="rubrica" markdown="1">

### Rúbrica de evaluación

Rúbrica para evaluar un repositorio de GitHub con refactorización de un blog de
consola hacia Programación Orientada a Objetos, persistencia en JSON, integración
del menú y documentación. Se valora tanto la corrección técnica como la organización
del proyecto y la claridad para mantenerlo y ejecutarlo.

| Criterio | Descripción | Peso |
|----------|-------------|------|
| Modelado orientado a objetos del dominio | Evalúa si el proyecto representa
correctamente las entidades del blog mediante clases y relaciones entre objetos, de
acuerdo con el enunciado. | 35% |
| Persistencia y transformación a JSON | Evalúa la carga inicial desde posts.json,
la conversión de objetos a una forma serializable y el guardado de datos usando el
módulo json. | 30% |
| Integración del menú y flujo de la aplicación | Evalúa si main.py instancia Blog y
conecta correctamente el menú con las operaciones de listado, búsqueda y creación
usando la nueva arquitectura. | 20% |
| Estructura del repositorio y organización modular | Evalúa la calidad de la
estructura del proyecto, el uso de archivos separados según su responsabilidad y la
claridad general del repositorio como entregable real. | 10% |
| Documentación y guía de ejecución | Evalúa si el README explica cómo funciona la
solución, cómo interactúan las clases con JSON y cómo ejecutar el proyecto. | 5% |

Total: 100 pts | Aprobación: 70 pts
</div>
</section>

<div class="module-head"><h1 class="module" data-eyebrow="MÓDULO 7">Git, GitHub e
introducción a Django</h1><hr class="module-underline"><hr class="module-
hairline"></div>
```

```
Inicializar el control de versiones con Git, publicar en GitHub y desplegar el
andamiaje inicial de un proyecto Django.

## Git desde cero: crear repositorios y guardar cambios

_Aprende los fundamentos de Git, el sistema de control de versiones líder de la
industria. Descubrirás cómo transformar una carpeta común en un repositorio, cómo
rastrear cambios y cómo crear "puntos de control" seguros para tu código del blog._

### Git desde cero: instalación, configuración y primeros commits

Hasta ahora, probablemente trabajaste con archivos sueltos de Python. Tal vez, si
querías guardar una versión anterior, creabas copias como:
```

blog.py blog_v2.py blog_final.py blog_final_ahora_si.py

CÓDIGO

```
Este método puede funcionar al principio, pero rápidamente se vuelve desordenado y peligroso.

Imagina que estás construyendo tu blog y, después de varias horas de trabajo, borras una parte importante del código o rompes algo que antes funcionaba. Sin una forma clara de volver atrás, podrías perder mucho tiempo.

Para resolver este problema usamos **Git**.

#### 1. ¿Qué es Git?

Git es un sistema de control de versiones.

Dicho de forma simple: Git permite guardar el historial de cambios de un proyecto.

Funciona como una especie de “máquina del tiempo” para tu código. Te permite:

* Guardar puntos de control.
* Ver qué archivos cambiaron.
* Volver a una versión anterior si algo sale mal.
* Trabajar con más orden a medida que el proyecto crece.

Git no guarda simplemente “archivos finales”. Guarda la historia del proyecto.

#### 2. Git no es lo mismo que GitHub

Es muy común confundir Git con GitHub, pero no son lo mismo.

**Git** es la herramienta que se instala en tu computadora para controlar versiones de tus archivos.

**GitHub** es una plataforma web donde se pueden subir repositorios para guardarlos en la nube, compartirlos o trabajar con otras personas.

En esta unidad vamos a trabajar solo con **Git local**, es decir, dentro de nuestra computadora.

El trabajo con GitHub, repositorios remotos y ramas se verá más adelante.

#### 3. Instalación de Git

Antes de usar Git, necesitamos tenerlo instalado.

La instalación depende del sistema operativo.

##### En Windows

La forma más común es instalar Git desde el instalador oficial de Git para Windows. Durante la instalación, se pueden dejar las opciones recomendadas para comenzar.
```

```
Una vez instalado, vas a poder usar Git desde:

* PowerShell
* CMD
* Git Bash
* La terminal integrada de VS Code

Git mantiene instaladores oficiales para Windows, macOS y Linux desde su sitio
oficial. ([Git](https://git-scm.com/install/?utm_source=chatgpt.com))

##### En macOS

En macOS, una forma simple de comprobar si Git está disponible es abrir la terminal
y ejecutar:

```shell
git --version
```

Si Git no está instalado, el sistema puede pedirte instalar las herramientas de línea de comandos de Xcode.

También puede instalarse usando Homebrew:

CÓDIGO

```
brew install git
```

La documentación oficial de Git menciona varias formas de instalación según el sistema operativo. (Git)

En Linux

En distribuciones basadas en Ubuntu o Debian, se puede instalar con:

CÓDIGO

```
sudo apt update
sudo apt install git
```

En Fedora, se puede usar:

CÓDIGO

```
sudo dnf install git
```

4. Verificar que Git funciona

Después de instalar Git, abrí una terminal y ejecutá:
```

CÓDIGO

```
git --version
```

Si todo está correcto, deberías ver una respuesta similar a:

CÓDIGO

```
git version 2.x.x
```

El número exacto puede variar según la versión instalada.

Lo importante es que la terminal reconozca el comando `git`.

5. Configuración inicial de Git

Antes de hacer commits, Git necesita saber quién sos.

Esto no significa iniciar sesión en GitHub. Es simplemente una configuración local para que cada cambio guardado tenga un autor.

Ejecutá estos comandos reemplazando los datos por los tuyos:

CÓDIGO

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu@email.com"
```

Por ejemplo:

CÓDIGO

```
git config --global user.name "Ana Pérez"
git config --global user.email "ana@email.com"
```

La herramienta `git config` permite configurar variables que controlan cómo funciona Git, incluyendo el nombre y el email asociados a los commits. (Git)

6. Ver la configuración actual

Para verificar que los datos se guardaron correctamente, podés ejecutar:

CÓDIGO

```
git config --global --list
```

Deberías ver algo parecido a:

CÓDIGO

```
user.name=Ana Pérez
user.email=ana@email.com
```

También se pueden consultar valores individuales:

CÓDIGO

```
git config --global user.name
git config --global user.email
```

7. Configuración opcional recomendada

También podemos indicar que el editor por defecto sea Visual Studio Code:

CÓDIGO

```
git config --global core.editor "code --wait"
```

Esto sirve para que, si Git necesita abrir un editor de texto, use VS Code en lugar de abrir otro editor menos amigable para principiantes.

También podemos configurar el nombre inicial usado por Git al crear repositorios nuevos:

CÓDIGO

```
git config --global init.defaultBranch main
```

No vamos a profundizar todavía en ramas, pero esta configuración evita diferencias entre proyectos que podrían iniciar con nombres distintos.

8. ¿Qué es un repositorio local?

Un repositorio es una carpeta controlada por Git.

Cuando convertimos una carpeta común en un repositorio, Git empieza a observar los cambios que ocurren dentro de ella.

Para lograrlo, Git crea una carpeta oculta llamada:

CÓDIGO

```
.git
```

Esa carpeta guarda la información interna del repositorio: historial, configuración y cambios registrados.

Regla importante:

> **Nota**
> No borres ni modifiques manualmente la carpeta `.git`.

Si la borrás, los archivos del proyecto seguirán existiendo, pero Git perderá el historial.

9. Crear un repositorio con `git init`

Supongamos que tenemos una carpeta llamada `mi_blog`.

Primero entramos a esa carpeta desde la terminal:

CÓDIGO

```
cd mi_blog
```

Luego inicializamos Git:

CÓDIGO

```
git init
```

Este comando convierte la carpeta actual en un repositorio local.

Git crea una carpeta `.git` dentro del proyecto. Según la documentación oficial, `git init` crea un repositorio vacío, incluyendo la estructura interna necesaria dentro de `.git`. (Git)

A partir de este momento, Git puede empezar a registrar los cambios del proyecto.

10. Revisar el estado con `git status`

Uno de los comandos más importantes de Git es:

CÓDIGO

```
git status
```

Este comando muestra el estado actual del repositorio.

Puede indicarnos:

- Qué archivos son nuevos.
- Qué archivos fueron modificados.
- Qué archivos están listos para guardarse.
- Si no hay cambios pendientes.

Es recomendable usar `git status` muchas veces mientras trabajamos.

Por ejemplo:

CÓDIGO

```
git status
```

Podríamos ver algo como:

CÓDIGO

```
Untracked files:
  blog.py
```

Esto significa que Git detectó un archivo nuevo, pero todavía no lo está siguiendo.

11. Los tres estados básicos de Git

Para guardar cambios en Git, los archivos suelen pasar por tres estados.

1. Working Directory

Es la carpeta donde estamos trabajando.

Acá están los archivos nuevos o modificados.

Ejemplo:

CÓDIGO

```
Modifiqué blog.py
```

Pero Git todavía no preparó ese cambio para guardarlo.

2. Staging Area

Es una zona intermedia.

Acá colocamos los archivos que queremos incluir en el próximo commit.

Es como decir:

> **Nota**
> “Estos cambios son los que quiero guardar ahora”.

3. Commit

Es el punto de control guardado en el historial.

Un commit representa una versión concreta del proyecto en un momento determinado.

12. Preparar archivos con `git add`

Para pasar archivos al área de preparación usamos:

CÓDIGO

```
git add
```

Si queremos preparar un archivo específico:

CÓDIGO

```
git add blog.py
```

Si queremos preparar todos los cambios del proyecto:

CÓDIGO

```
git add .
```

El punto significa “todo lo que cambió dentro de esta carpeta y sus subcarpetas”.

Después de ejecutar `git add`, podemos revisar otra vez:

CÓDIGO

```
git statuss
```

Ahora Git debería mostrar esos archivos como preparados para commit.

13. Guardar cambios con `git commit`

Una vez que los archivos están preparados, podemos crear un commit:

CÓDIGO

```
git commit -m "Crear estructura inicial del blog"
```

Un commit necesita un mensaje que explique qué se guardó.

El mensaje debería ser claro y breve.

Buenos ejemplos:

CÓDIGO

```
git commit -m "Crear archivo principal del blog"
git commit -m "Agregar lista inicial de posts"
git commit -m "Corregir validación de título vacío"
```

Malos ejemplos:

CÓDIGO

```
git commit -m "cambios"
git commit -m "asdf"
git commit -m "cosas"
```

En un proyecto real, los mensajes claros ayudan a entender la historia del código.

14. Flujo básico de trabajo

El flujo básico que vamos a repetir muchas veces es:

CÓDIGO

```
git status
git add .
git commit -m "Mensaje claro del cambio"
git status
```

Ejemplo completo:

CÓDIGO

```
git status
git add .
git commit -m "Crear primer archivo del blog"
git status
```

Si todo salió bien, al final Git puede mostrar algo como:

CÓDIGO

```
nothing to commit, working tree clean
```

Esto significa que no hay cambios pendientes por guardar.

15. Crear un archivo `.gitignore`

No todos los archivos de un proyecto deben guardarse con Git.

Por ejemplo, normalmente no queremos guardar:

- Carpetas de entornos virtuales.
- Archivos temporales.
- Configuraciones privadas.
- Archivos con contraseñas.
- Cachés generadas automáticamente.

Para ignorar archivos o carpetas, creamos un archivo llamado:

CÓDIGO

```
.gitignore
```

Dentro podemos escribir reglas como:

CÓDIGO

```
venv/
env/
__pycache__/
*.pyc
.env
```

Ejemplo:

CÓDIGO

```
mi_blog/
├── blog.py
├── .gitignore
└── venv/
```

Si dentro de `.gitignore` escribimos:

CÓDIGO

```
venv/
```

Git dejará de mostrar la carpeta `venv` como archivo pendiente.

Esto es especialmente útil cuando trabajamos con Python, porque los entornos virtuales no deberían guardarse dentro del historial del proyecto.

16. Error común: hacer commit sin configurar Git

Si intentás hacer un commit sin haber configurado tu nombre y email, Git puede mostrar un mensaje de error pidiendo que configures tu identidad.

Se soluciona con:

CÓDIGO

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu@email.com"
```

Después podés intentar el commit nuevamente.

17. Error común: olvidar `git add`

A veces modificamos archivos y ejecutamos:

CÓDIGO

```
git commit -m "Actualizar blog"
```

Pero Git responde que no hay cambios preparados.

Eso pasa porque antes faltó ejecutar:

CÓDIGO

```
git add .
```

Recordá este orden:

CÓDIGO

```
git add .
git commit -m "Mensaje del cambio"
```

Primero se prepara. Después se confirma.

18. Error común: cerrar mal el mensaje del commit

Si escribís:

CÓDIGO

```
git commit -m Crear blog
```

Git puede interpretar mal el comando porque el mensaje no está entre comillas.

Lo correcto es:

CÓDIGO

```
git commit -m "Crear blog"
```

El mensaje del commit debería ir entre comillas.

19. Buenas prácticas iniciales

Al comenzar con Git, conviene seguir estas reglas:

- Hacer commits pequeños y frecuentes.
- Usar mensajes claros.
- Ejecutar `git status` antes y después de cada commit.
- No guardar entornos virtuales en Git.
- No modificar manualmente la carpeta `.git`.
- No esperar al final del proyecto para hacer el primer commit.

Git sirve más cuando se usa como hábito, no solo al final.

20. Ejemplo completo desde cero

Supongamos que queremos crear un proyecto llamado `mi_blog`.

Creamos la carpeta:

CÓDIGO

```
mkdir mi_blog
cd mi_blog
```

Creamos un archivo:

CÓDIGO

```
touch blog.py
```

En Windows, si `touch` no funciona, podemos crear el archivo desde VS Code o usar:

CÓDIGO

```
New-Item blog.py
```

Inicializamos Git:

CÓDIGO

```
git init
```

Revisamos el estado:

CÓDIGO

```
git status
```

Preparamos el archivo:

CÓDIGO

```
git add blog.py
```

Creamos el primer commit:

CÓDIGO

```
git commit -m "Crear archivo inicial del blog"
```

Volvemos a revisar:

CÓDIGO

```
git status
```

Si aparece:

CÓDIGO

```
nothing to commit, working tree clean
```

significa que el proyecto está limpio y el cambio quedó guardado en el historial local.

Síntesis

En esta unidad instalaste Git, verificaste que funcione y configuraste tu nombre y email con `git config`.

También aprendiste a crear un repositorio local con `git init`, revisar cambios con `git status`, preparar archivos con `git add` y guardar puntos de control con `git commit`.

Todavía no trabajamos con GitHub, ramas ni repositorios remotos. Por ahora, el objetivo es dominar el flujo local básico de Git para guardar avances del proyecto de forma segura y ordenada.

Ramas y GitHub: trabajar en paralelo y publicar el repositorio

Aprende a gestionar el crecimiento de tu proyecto usando ramas para trabajar de forma segura y a sincronizar tu progreso con GitHub, la plataforma líder para alojar código.

En la unidad anterior aprendimos a usar Git de forma local: inicializamos un repositorio, revisamos cambios, preparamos archivos y guardamos versiones usando commits.

Ahora vamos a dar un paso más.

Hasta este momento, nuestro trabajo avanzaba en una sola línea. Modificábamos archivos y guardábamos commits uno detrás de otro.

Pero en proyectos reales suele pasar algo como esto:

- Queremos probar una funcionalidad nueva.
- Queremos corregir un error sin romper lo que ya funciona.
- Queremos subir el proyecto a internet.
- Queremos compartir el código con otras personas.
- Queremos dejar una copia de seguridad fuera de nuestra computadora.

Para eso vamos a aprender dos conceptos importantes:

- **Ramas**, para trabajar en paralelo sin romper la versión principal.

- **GitHub**, para publicar nuestro repositorio en la nube.

1. ¿Qué es una rama en Git?

Una rama, o `branch`, es una línea de trabajo paralela dentro del proyecto.

Podemos imaginar el proyecto como un árbol.

La rama principal es el tronco. Desde ese tronco podemos crear ramas para probar ideas, agregar funciones o corregir errores.

Por ejemplo, nuestro blog ya tiene una estructura básica funcionando. Ahora queremos probar una mejora en el menú.

En lugar de modificar directamente la versión principal, podemos crear una rama llamada:

CÓDIGO

```
mejorar-menu
```

Dentro de esa rama podemos trabajar tranquilos.

Si la mejora funciona, después podemos integrarla a la rama principal.

Si sale mal, podemos descartar esa rama y el proyecto principal sigue intacto.

2. La rama principal

Cuando creamos un repositorio, Git trabaja sobre una rama principal.

Hoy en día, lo más común es que esa rama se llame:

CÓDIGO

```
main
```

En algunos proyectos antiguos puede llamarse:

CÓDIGO

```
master
```

En este curso vamos a usar `main` como rama principal.

La idea es que `main` represente la versión más estable del proyecto.

3. Ver en qué rama estamos

Antes de crear o cambiar ramas, conviene revisar el estado del repositorio:

CÓDIGO

```
git status
```

Git puede mostrar algo como:

CÓDIGO

```
On branch main
nothing to commit, working tree clean
```

Esto significa que estamos en la rama `main` y no hay cambios pendientes.

También podemos listar las ramas disponibles con:

CÓDIGO

```
git branch
```

La rama actual aparece marcada con un asterisco:

CÓDIGO

```
* main
```

4. Crear una rama nueva

Para crear una rama nueva usamos:

CÓDIGO

```
git branch nombre-de-la-rama
```

Por ejemplo:

CÓDIGO

```
git branch mejorar-menu
```

Esto crea la rama, pero todavía no nos mueve a ella.

Seguimos parados en `main`.

5. Cambiar de rama

Para movernos a otra rama podemos usar:

CÓDIGO

```
git switch mejorar-menu
```

También existe esta forma clásica:

CÓDIGO

```
git checkout mejorar-menu
```

Ambas permiten cambiar de rama.

Para principiantes, `git switch` suele ser más claro porque expresa directamente la acción: cambiar de rama.

6. Crear y cambiar de rama en un solo paso

Muchas veces queremos crear una rama y movernos a ella inmediatamente.

Podemos hacerlo con:

CÓDIGO

```
git switch -c mejorar-menu
```

También se puede ver esta variante más tradicional:

CÓDIGO

```
git checkout -b mejorar-menu
```

Ambas hacen lo mismo:

1. crean la rama;
2. nos mueven a esa rama.

7. Nombres recomendados para ramas

Los nombres de ramas deberían ser descriptivos.

No conviene usar nombres como:

CÓDIGO

```
prueba
cambios
rama1
cosas
```

Es mejor usar nombres que expliquen qué estamos haciendo:

CÓDIGO

```
agregar-menu-blog
corregir-error-login
crear-listado-posts
mejorar-estilos-home
```

Una buena práctica es escribir los nombres en minúscula y separar palabras con guiones.

8. Trabajar dentro de una rama

Supongamos que creamos una rama para mejorar el menú del blog:

CÓDIGO

```
git switch -c mejorar-menu
```

Ahora modificamos nuestro archivo `blog.py`.

Después revisamos el estado:

CÓDIGO

```
git status
```

Preparamos los cambios:

CÓDIGO

```
git add .
```

Y creamos un commit:

CÓDIGO

```
git commit -m "Mejorar menú principal del blog"
```

Ese commit queda guardado en la rama `mejorar-menu`.

La rama `main` todavía no fue modificada.

9. Volver a la rama principal

Para volver a `main`, usamos:

CÓDIGO

```
git switch main
```

O, si estamos usando la forma clásica:

CÓDIGO

```
git checkout main
```

Al volver a `main`, puede pasar que algunos cambios que vimos en la rama anterior ya no aparezcan.

Eso es normal.

Significa que esos cambios existen en la otra rama, pero todavía no fueron integrados a la principal.

10. Integrar cambios con `merge`

Cuando terminamos el trabajo en una rama y queremos llevarlo a `main`, usamos `merge`.

Primero volvemos a la rama principal:

CÓDIGO

```
git switch main
```

Luego integramos la rama que contiene los cambios:

CÓDIGO

```
git merge mejorar-menu
```

Esto le dice a Git:

> **Nota**
>
> Tomá los cambios de `mejorar-menu` e incorporalos en `main`.

Después podemos revisar:

CÓDIGO

```
git status
```

Y también podemos ver el historial con:

CÓDIGO

```
git log --oneline
```

11. ¿Qué es GitHub?

GitHub es una plataforma web que permite alojar repositorios Git en la nube.

Git trabaja en nuestra computadora.

GitHub guarda una copia remota del repositorio.

Dicho de forma simple:

- **Git** controla las versiones del proyecto.
- **GitHub** permite guardar y compartir ese proyecto en internet.

Esto nos sirve para:

- Tener una copia de seguridad.
- Entregar trabajos.
- Compartir código.
- Trabajar en equipo.
- Mostrar proyectos en un portfolio.

12. Crear un repositorio en GitHub

Para publicar nuestro proyecto, primero necesitamos crear un repositorio en GitHub.

Pasos generales:

1. Entrar a GitHub.
2. Iniciar sesión.
3. Crear un repositorio nuevo.
4. Elegir un nombre para el repositorio.
5. Dejarlo público o privado según corresponda.
6. No agregar archivos extra si ya tenemos un proyecto local creado.
7. Copiar la URL del repositorio.

La URL suele tener una forma parecida a esta:

CÓDIGO

```
https://github.com/usuario/nombre-del-repositorio.git
```

Por ejemplo:

CÓDIGO

```
https://github.com/ana/mi-blog-python.git
```

13. Conectar el repositorio local con GitHub

Nuestro proyecto local todavía no sabe que existe un repositorio en GitHub.

Para conectarlo, usamos:

CÓDIGO

```
git remote add origin URL_DEL_REPOSITORIO
```

Ejemplo:

CÓDIGO

```
git remote add origin https://github.com/ana/mi-blog-python.git
```

Veamos las partes del comando:

CÓDIGO

```
git remote add origin https://github.com/ana/mi-blog-python.git
```

- `git remote add`: agrega un repositorio remoto.
- `origin`: es el nombre que normalmente usamos para el remoto principal.
- la URL: indica dónde está el repositorio en GitHub.

14. Verificar la conexión remota

Para revisar si el remoto se agregó correctamente, usamos:

CÓDIGO

```
git remote -v
```

Podríamos ver algo como:

CÓDIGO

```
origin  https://github.com/ana/mi-blog-python.git (fetch)
origin  https://github.com/ana/mi-blog-python.git (push)
```

Esto significa que el repositorio local ya está conectado con GitHub.

15. Subir el proyecto por primera vez

Para subir la rama principal a GitHub usamos:

CÓDIGO

```
git push -u origin main
```

Este comando sube los commits locales al repositorio remoto.

Veamos sus partes:

- `git push`: sube cambios al remoto.
- `origin`: nombre del repositorio remoto.
- `main`: rama que queremos subir.
- `-u`: deja vinculada la rama local con la rama remota.

Después de hacerlo una vez, en general podremos usar simplemente:

CÓDIGO

```
git push
```

16. Diferencia entre `commit` y `push`

Este punto es muy importante.

`commit` guarda cambios en tu computadora.

CÓDIGO

```
git commit -m "Crear estructura inicial del blog"
```

`push` sube esos commits a GitHub.

CÓDIGO

```
git push
```

Si hacés commits pero nunca hacés push, los cambios existen solo en tu computadora.

GitHub no los va a mostrar hasta que los subas.

17. Subir una rama nueva a GitHub

Supongamos que creamos una rama llamada:

CÓDIGO

```
crear-listado-posts
```

Y trabajamos allí:

CÓDIGO

```
git switch -c crear-listado-posts
```

Luego hacemos cambios, los preparamos y creamos un commit:

CÓDIGO

```
git add .
git commit -m "Crear listado inicial de posts"
```

Para subir esa rama a GitHub por primera vez:

CÓDIGO

```
git push -u origin crear-listado-posts
```

Después de eso, la rama también existirá en GitHub.

18. ¿Qué es un Pull Request?

Un Pull Request, o PR, es una solicitud para integrar cambios de una rama en otra.

Normalmente se usa para decir:

> **Nota**
>
> Terminé este trabajo en una rama. Quiero revisar los cambios y luego unirlos a `main`.

Por ejemplo:

- Rama principal: `main`
- Rama de trabajo: `crear-listado-posts`

Creamos un Pull Request para pedir que los cambios de `crear-listado-posts` se integren en `main`.

El Pull Request permite:

- Revisar qué archivos cambiaron.
- Leer los commits.
- Dejar comentarios.
- Detectar errores antes de integrar.
- Mantener más ordenado el proyecto.

19. Flujo básico con rama y Pull Request

Un flujo de trabajo típico sería:

CÓDIGO

```
git switch main
git switch -c crear-listado-posts
```

Trabajamos en el código.

Luego:

CÓDIGO

```
git status
git add .
git commit -m "Crear listado inicial de posts"
git push -u origin crear-listado-posts
```

Después vamos a GitHub y creamos un Pull Request desde:

CÓDIGO

```
crear-listado-posts
```

hacia:

CÓDIGO

```
main
```

Si todo está bien, hacemos merge del Pull Request desde GitHub.

20. Buenas prácticas

Al trabajar con ramas y GitHub, conviene seguir estas reglas:

- No trabajar todo directamente en `main`.
- Crear una rama por cada mejora o corrección.
- Usar nombres descriptivos para las ramas.
- Hacer commits pequeños y claros.
- Subir los cambios con `git push`.
- Revisar el Pull Request antes de hacer merge.
- Usar `git status` frecuentemente para saber dónde estamos parados.

21. Errores comunes

Hacer cambios en la rama equivocada

Puede pasar que querías trabajar en una rama nueva, pero modificaste archivos estando en `main`.

Para evitarlo, revisá siempre:

CÓDIGO

```
git status
```

Confundir `commit` con `push`

Recordá:

CÓDIGO

```
git commit
```

guarda localmente.

CÓDIGO

```
git push
```

sube a GitHub.

Crear la rama pero olvidarse de cambiarse a ella

Este comando solo crea la rama:

CÓDIGO

```
git branch mejorar-menu
```

Pero este crea la rama y además te mueve a ella:

CÓDIGO

```
git switch -c mejorar-menu
```

Subir una rama sin commits

Si creás una rama pero no hacés ningún commit nuevo, no hay nada importante que subir.

Primero trabajá, después hacé:

CÓDIGO

```
git add .
git commit -m "Mensaje claro"
```

Y recién ahí:

CÓDIGO

```
git push
```

22. Ejemplo completo

Supongamos que tenemos un blog por consola y queremos agregar un listado de posts.

Primero nos aseguramos de estar en `main`:

CÓDIGO

```
git switch main
git status
```

Creamos una rama nueva:

CÓDIGO

```
git switch -c crear-listado-posts
```

Modificamos nuestro código.

Después revisamos los cambios:

CÓDIGO

```
git status
```

Preparamos los archivos:

CÓDIGO

```
git add .
```

Creamos el commit:

CÓDIGO

```
git commit -m "Crear listado inicial de posts"
```

Subimos la rama a GitHub:

CÓDIGO

```
git push -u origin crear-listado-posts
```

Luego entramos a GitHub y creamos un Pull Request para integrar esa rama en `main`.

Síntesis

En esta unidad aprendiste a trabajar con ramas para separar cambios y evitar modificar directamente la versión principal del proyecto.

También aprendiste a conectar tu repositorio local con GitHub usando `git remote add origin`, verificar la conexión con `git remote -v` y subir cambios con `git push`.

Por último, vimos qué es un Pull Request y por qué ayuda a revisar los cambios antes de integrarlos a la rama principal.

Con este flujo, tu proyecto empieza a parecerse mucho más a una forma profesional de trabajo: una rama para cada mejora, commits claros, repositorio remoto y revisión antes de integrar cambios.

Instalar Django y reconocer la estructura inicial del proyecto

Aprende a instalar el framework Django, crear la estructura base de tu proyecto de blog y reconocer los archivos fundamentales que lo hacen funcionar.

Instalando el motor del blog: Django

Hasta ahora preparaste varias herramientas importantes: Python, VS Code, entornos virtuales, Git y GitHub.

Ya podés escribir código, organizar archivos y guardar versiones de tu proyecto. Ahora vamos a sumar la herramienta principal para construir nuestra aplicación web: **Django**.

Django es un **framework de Python** para crear aplicaciones web. Un framework es un conjunto de herramientas, reglas y estructuras que nos ayudan a desarrollar más rápido. En lugar de construir todo desde cero, Django ya nos ofrece muchas piezas listas para usar: rutas, configuración del proyecto, conexión con base de datos, panel de administración, sistema de usuarios y más.

En este curso lo vamos a usar para transformar nuestro blog por consola en un **blog web**.

1. Antes de instalar Django

Antes de instalar Django, necesitamos estar dentro de la carpeta de nuestro proyecto y tener activo el entorno virtual.

El entorno virtual nos permite instalar paquetes solo para este proyecto, sin afectar otros proyectos de Python.

En la terminal deberías ver algo parecido a esto:

CÓDIGO

```
(venv) usuario@equipo:~/mi_blog$
```

O en Windows:

CÓDIGO

```
(venv) PS C:\Users\Usuario\mi_blog>
```

El nombre `(venv)` al inicio indica que el entorno virtual está activo.

Si no está activo, podés activarlo con el comando correspondiente según tu sistema operativo.

En Windows con PowerShell:

CÓDIGO

```
.\venv\Scripts\Activate.ps1
```

En Windows con CMD:

CÓDIGO

```
venv\Scripts\activate
```

En Linux o macOS:

CÓDIGO

```
source venv/bin/activate
```

Una vez activo, ya podemos instalar paquetes de forma segura dentro del proyecto.

2. Instalar Django

Para instalar Django usamos `pip`, el gestor de paquetes de Python.

Un gestor de paquetes nos permite descargar e instalar herramientas creadas por otras personas para usarlas en nuestros proyectos.

Ejecutamos:

CÓDIGO

```
pip install Django
```

En algunos sistemas Windows también puede usarse:

CÓDIGO

```
py -m pip install Django
```

Este comando descarga Django y lo instala dentro del entorno virtual activo.

Para verificar que la instalación funcionó correctamente, ejecutamos:

CÓDIGO

```
django-admin --version
```

Si Django se instaló bien, deberías ver un número de versión, por ejemplo:

CÓDIGO

```
5.2.x
```

o:

CÓDIGO

```
6.0.x
```

El número exacto puede variar. Lo importante es que la terminal reconozca el comando `django-admin`.

3. Crear el proyecto Django

Django nos ofrece una herramienta llamada `django-admin`, que permite crear la estructura inicial de un proyecto.

Para crear nuestro proyecto, ejecutamos:

CÓDIGO

```
django-admin startproject blog_project .
```

Este comando crea el proyecto dentro de la carpeta actual. El punto final indica que `manage.py` debe quedar en la raíz y que la configuración principal debe quedar dentro de `blog_project/`.

4. Estructura creada por Django

Después de ejecutar el comando anterior, Django genera una estructura similar a esta:

CÓDIGO

```
mi_blog/
├── manage.py
└── blog_project/
    ├── __init__.py
    ├── settings.py
    ├── urls.py
    ├── asgi.py
    └── wsgi.py
```

Al principio puede parecer mucha información, pero cada archivo tiene una función concreta.

`manage.py`

El archivo `manage.py` es una herramienta de administración del proyecto.

No solemos editarlo directamente, pero lo usamos mucho desde la terminal.

Con `manage.py` podemos:

- iniciar el servidor de desarrollo;
- crear aplicaciones internas;
- ejecutar migraciones;
- crear usuarios administradores;
- trabajar con la base de datos;
- ejecutar comandos propios de Django.

Por ahora, lo vamos a usar para levantar el servidor local.

Carpeta interna del proyecto

Dentro de la carpeta de trabajo aparece una carpeta llamada `blog_project`:

CÓDIGO

```
blog_project/
└── blog_project/
```

Esa carpeta contiene la configuración principal del proyecto. Podemos pensarla como el "centro de control" de Django.

`settings.py`

El archivo `settings.py` guarda la configuración general del proyecto.

Allí se definen cosas como:

- aplicaciones instaladas;
- configuración de base de datos;
- idioma;
- zona horaria;
- archivos estáticos;
- configuraciones de seguridad;
- templates;
- middleware.

Todavía no vamos a modificar demasiado este archivo, pero será uno de los más importantes durante el curso.

`urls.py`

El archivo `urls.py` funciona como el mapa de rutas del proyecto.

Una ruta es una dirección que el usuario puede visitar en el navegador.

Por ejemplo:

CÓDIGO

```
/
```

CÓDIGO

```
/posts/
```

CÓDIGO

```
/contacto/
```

En `urls.py` le indicamos a Django qué debe responder cuando alguien visita una URL determinada.

Más adelante vamos a conectar las rutas de nuestro blog desde este archivo.

`asgi.py` y `wsgi.py`

Los archivos `asgi.py` y `wsgi.py` sirven para conectar nuestro proyecto Django con servidores web reales.

Por ahora no vamos a modificarlos. Es suficiente con saber que forman parte de la estructura interna que Django necesita para funcionar en distintos entornos.

5. Ubicarse en la carpeta del proyecto

Para ejecutar comandos de Django, necesitamos estar en la carpeta donde se encuentra `manage.py`.

Si creamos el proyecto con:

CÓDIGO

```
django-admin startproject blog_project .
```

no necesitamos entrar a otra carpeta. Ya estamos ubicados en la carpeta donde está `manage.py`.

En Linux o macOS:

CÓDIGO

```
ls
```

En Windows:

CÓDIGO

```
dir
```

Deberíamos ver el archivo:

CÓDIGO

```
manage.py
```

Si `manage.py` aparece en la lista, estamos en la carpeta correcta.

6. Levantar el servidor de desarrollo

Django incluye un servidor de desarrollo para probar nuestro proyecto localmente.

Para iniciarlo, ejecutamos:

CÓDIGO

```
python manage.py runserver
```

Si todo está bien, veremos un mensaje indicando que el servidor está funcionando.

La dirección suele ser:

CÓDIGO

```
http://127.0.0.1:8000/
```

También puede aparecer como:

CÓDIGO

```
http://localhost:8000/
```

Abrimos esa dirección en el navegador y deberíamos ver la pantalla de bienvenida de Django.

Ese es nuestro primer sitio Django funcionando localmente.

Mientras el servidor esté activo, la terminal quedará ocupada mostrando información. Para detenerlo, podemos presionar:

CÓDIGO

```
Ctrl + C
```

Esto corta el servidor de desarrollo y nos devuelve el control de la terminal.

7. Errores comunes

Error: `manage.py` no encontrado

Un error frecuente es ejecutar:

CÓDIGO

```
python manage.py runserver
```

desde la carpeta incorrecta.

Si aparece un error indicando que no se encuentra `manage.py`, significa que no estamos ubicados en la carpeta correcta.

Para revisarlo:

En Linux o macOS:

CÓDIGO

```
ls
```

En Windows:

CÓDIGO

```
dir
```

Si no vemos `manage.py`, tenemos que movernos a la carpeta correspondiente.

Por ejemplo:

CÓDIGO

```
cd blog_project
```

Y luego volver a ejecutar:

CÓDIGO

```
python manage.py runserver
```

Error: Django no está instalado

Si ejecutamos:

CÓDIGO

```
django-admin --version
```

o:

CÓDIGO

```
python manage.py runserver
```

y aparece un error relacionado con Django, puede ser que:

- el entorno virtual no esté activo;
- Django no se haya instalado;
- estemos usando otro Python distinto;
- la instalación haya fallado.

Una forma simple de revisarlo es ejecutar:

CÓDIGO

```
pip show Django
```

Si Django está instalado, veremos información del paquete.

Si no aparece nada, debemos instalarlo nuevamente con el entorno virtual activo:

CÓDIGO

```
pip install Django
```

Error: el puerto ya está en uso

A veces, al ejecutar:

CÓDIGO

```
python manage.py runserver
```

puede aparecer un error indicando que el puerto `8000` ya está ocupado.

Esto puede pasar si dejamos otro servidor Django abierto.

Una solución es detener el servidor anterior con:

CÓDIGO

```
Ctrl + C
```

Otra opción es iniciar el servidor en otro puerto:

CÓDIGO

```
python manage.py runserver 8001
```

Luego entraríamos desde el navegador a:

CÓDIGO

```
http://127.0.0.1:8001/
```

8. Buenas prácticas iniciales

Al empezar un proyecto Django, conviene seguir estos pasos:

1. Crear y activar el entorno virtual.
2. Instalar Django dentro del entorno virtual.
3. Verificar la instalación.
4. Crear el proyecto con `django-admin startproject`.
5. Entrar a la carpeta donde está `manage.py`.
6. Ejecutar el servidor con `python manage.py runserver`.
7. Confirmar que la pantalla de bienvenida se ve en el navegador.

También es recomendable hacer un commit después de crear la estructura inicial del proyecto:

CÓDIGO

```
git status
git add .
git commit -m "Crear proyecto inicial con Django"
```

De esa forma, dejamos guardado el primer punto estable del proyecto.

9. Ejemplo completo

Desde la carpeta donde queremos crear el proyecto:

Creamos el entorno virtual con

python -m venv venv

Activamos el entorno virtual.

En Windows PowerShell:

CÓDIGO

```
.\venv\Scripts\Activate.ps1
```

En Linux o macOS:

CÓDIGO

```
source venv/bin/activate
```

Instalamos Django:

CÓDIGO

```
pip install Django
```

Verificamos la instalación:

CÓDIGO

```
django-admin --version
```

Creamos el proyecto:

CÓDIGO

```
django-admin startproject blog_project .
```

Ejecutamos el servidor:

CÓDIGO

```
python manage.py runserver
```

Abrimos en el navegador:

CÓDIGO

```
http://127.0.0.1:8000/
```

Si vemos la página de bienvenida de Django, el proyecto fue creado correctamente.

Síntesis

En esta unidad instalamos Django dentro de un entorno virtual y creamos nuestro primer proyecto web.

Aprendimos que `django-admin` sirve para crear la estructura inicial del proyecto y que `manage.py` es la herramienta que vamos a usar frecuentemente desde la terminal.

También vimos la función general de archivos importantes como `settings.py`, `urls.py`, `asgi.py` y `wsgi.py`.

Finalmente, levantamos el servidor de desarrollo con:

CÓDIGO

```
python manage.py runserver
```

y comprobamos que nuestro proyecto Django funciona en el navegador.

A partir de ahora, ya tenemos la base para empezar a construir nuestro blog web.

Proyecto, aplicación y configuración básica de Django

Aprende a organizar un proyecto Django distinguiendo entre el proyecto general y sus aplicaciones modulares, creando tu primera app y vinculándola correctamente en la configuración.

Proyecto, aplicaciones y configuración básica en Django

Ya instalaste Django, creaste tu primer proyecto y levantaste el servidor de desarrollo.

Eso significa que tu computadora ya puede ejecutar una aplicación web local. Pero todavía falta entender algo fundamental: **cómo organiza Django un proyecto por dentro**.

Django no nos invita a escribir código en cualquier archivo al azar. Tiene una estructura pensada para que el proyecto sea ordenado, fácil de mantener y preparado para crecer.

En esta unidad vamos a aprender tres cosas importantes:

- qué diferencia hay entre un proyecto y una aplicación;
- cómo crear nuestra primera app del blog;
- cómo registrar esa app dentro de la configuración del proyecto.

1. Proyecto y aplicación no son lo mismo

Cuando empezamos con Django, es común pensar que “proyecto” y “aplicación” significan lo mismo.

Pero en Django son conceptos distintos.

2. ¿Qué es un proyecto Django?

El proyecto es la configuración general del sitio web.

Podemos pensarlo como el contenedor principal de toda la aplicación.

El proyecto define cosas como:

- qué base de datos se usa;
- qué aplicaciones están activadas;
- qué idioma usa el sitio;
- qué zona horaria tiene;
- cuáles son las rutas principales;
- qué configuraciones generales necesita Django para funcionar.

Cuando ejecutamos:

CÓDIGO

```
django-admin startproject blog_project .
```

Django crea el proyecto principal dentro de la carpeta actual.

Dentro de la carpeta `blog_project/` aparecen archivos como:

CÓDIGO

```
settings.py
urls.py
asgi.py
wsgi.py
```

Estos archivos no representan una funcionalidad específica del blog, sino la configuración general del sitio.

3. ¿Qué es una aplicación Django?

Una aplicación, o app, es una parte específica del proyecto.

Cada app debería encargarse de una funcionalidad concreta.

En nuestro caso, vamos a crear una app para manejar las publicaciones del blog.

Por ejemplo, en un proyecto más grande podríamos tener:

CÓDIGO

```
posts       → publicaciones del blog
usuarios    → registro, login y perfiles
comentarios → comentarios de lectores
paginas     → páginas informativas como "Acerca de"
```

La idea es que cada app tenga una responsabilidad clara.

Esto ayuda a mantener el código más ordenado.

4. Ejemplo simple: proyecto vs app

Podemos imaginarlo así:

CÓDIGO

```
Proyecto Django
│
├── configuración general
├── rutas principales
├── base de datos
├── idioma y zona horaria
│
├── app posts
├── app usuarios
└── app comentarios
```

El proyecto contiene a las aplicaciones.

Las aplicaciones contienen la lógica específica de cada parte del sitio.

5. Archivos principales del proyecto

Dentro de la carpeta interna del proyecto encontramos varios archivos importantes.

Por ejemplo:

CÓDIGO

```
mi_blog/
├── manage.py
└── blog_project/
    ├── __init__.py
    ├── settings.py
    ├── urls.py
    ├── asgi.py
    └── wsgi.py
```

Veamos qué hace cada uno.

6. `manage.py`

El archivo `manage.py` es una herramienta de administración.

No solemos modificarlo directamente, pero lo usamos mucho desde la terminal.

Por ejemplo:

CÓDIGO

```
python manage.py runserver
```

Más adelante también lo usaremos para:

- crear apps;
- crear migraciones;
- aplicar cambios en la base de datos;
- crear usuarios administradores;
- ejecutar comandos propios de Django.

7. `settings.py`

El archivo `settings.py` contiene la configuración principal del proyecto.

Allí se definen muchas reglas importantes, por ejemplo:

- aplicaciones instaladas;
- base de datos;
- idioma;
- zona horaria;

- archivos estáticos;
- configuraciones de seguridad;
- templates;
- middleware.

Es uno de los archivos más importantes de cualquier proyecto Django.

8. `urls.py`

El archivo `urls.py` funciona como el mapa de rutas principal del proyecto.

Una ruta es una dirección que el usuario puede visitar desde el navegador.

Por ejemplo:

CÓDIGO

```
/
```

CÓDIGO

```
/posts/
```

CÓDIGO

```
/admin/
```

En `urls.py` le decimos a Django qué parte del código debe responder cuando alguien visita una dirección determinada.

9. `asgi.py` y `wsgi.py`

Los archivos `asgi.py` y `wsgi.py` sirven para conectar Django con servidores web reales.

Por ahora no vamos a modificarlos.

Es suficiente con saber que forman parte de la estructura técnica que Django necesita para poder ejecutarse en distintos entornos.

10. Crear nuestra primera aplicación

Ahora vamos a crear la app que manejará las publicaciones del blog.

Antes de ejecutar el comando, asegurate de cumplir dos condiciones:

1. El entorno virtual debe estar activo.
2. La terminal debe estar ubicada en la carpeta donde está `manage.py`.

Podés verificarlo con:

CÓDIGO

```
ls
```

En Windows también podés usar:

CÓDIGO

```
dir
```

Si ves el archivo `manage.py`, estás en el lugar correcto.

Ahora creamos la app:

CÓDIGO

```
python manage.py startapp posts
```

Usamos el nombre `posts` porque esta app va a manejar las publicaciones del blog.

11. Estructura creada por `startapp`

Después de ejecutar el comando, Django crea una carpeta llamada `posts`.

La estructura será parecida a esta:

CÓDIGO

```
posts/
├── __init__.py
├── admin.py
├── apps.py
├── migrations/
│   └── __init__.py
├── models.py
├── tests.py
└── views.py
```

Cada archivo tiene una función.

No vamos a usar todos al mismo tiempo, pero conviene empezar a reconocerlos.

12. `models.py`

En `models.py` vamos a definir los modelos de datos.

Un modelo representa una estructura que luego se guardará en la base de datos.

Por ejemplo, más adelante podríamos crear un modelo `Post` con campos como:

CÓDIGO

```
título
contenido
fecha de publicación
autor
estado
```

Por ahora no vamos a crear el modelo todavía, pero este archivo será clave para darle “memoria” a nuestro blog.

13. `views.py`

En `views.py` escribimos la lógica que responde a una petición del usuario.

Por ejemplo, cuando alguien entra a:

CÓDIGO

```
/posts/
```

una vista podría encargarse de buscar todos los posts y preparar una respuesta para mostrar en pantalla.

Más adelante vamos a conectar vistas con templates para mostrar HTML.

14. `admin.py`

El archivo `admin.py` se usa para registrar modelos en el panel de administración de Django.

Django incluye un administrador automático muy útil.

Más adelante, cuando creemos el modelo `Post`, podremos registrarlo en este archivo para administrarlo desde el navegador.

15. `apps.py`

El archivo `apps.py` contiene la configuración de la app.

Si lo abrimos, veremos algo parecido a esto:

CÓDIGO

```
from django.apps import AppConfig


class PostsConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'posts'
```

Esta clase representa la configuración de la aplicación `posts`.

16. `migrations/`

La carpeta `migrations` guardará archivos relacionados con cambios en la base de datos.

Todavía no vamos a trabajar con migraciones en profundidad, pero más adelante serán muy importantes.

Cada vez que creemos o modifiquemos modelos, Django podrá generar migraciones para reflejar esos cambios en la base de datos.

17. Crear la app no alcanza

Este es un punto muy importante.

Cuando ejecutamos:

CÓDIGO

```
python manage.py startapp posts
```

Django crea la carpeta de la app, pero el proyecto todavía no la está usando.

Es decir, la app existe, pero todavía no está conectada al proyecto.

Para conectarla, tenemos que registrarla en `settings.py`.

18. Registrar la app en `INSTALLED_APPS`

Abrimos el archivo:

CÓDIGO

```
blog_project/settings.py
```

Buscamos la lista llamada `INSTALLED_APPS`.

Por defecto, se ve parecida a esto:

CÓDIGO

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
]
```

Al final de la lista agregamos nuestra app:

CÓDIGO

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',

    # Aplicaciones propias
    'posts',
]
```

Con esto, Django ya sabe que debe incluir la app `posts` dentro del proyecto.

19. Registrar la app usando `AppConfig`

También podemos registrar la app usando la clase de configuración definida en `apps.py`.

En ese caso, en lugar de escribir:

CÓDIGO

```
'posts',
```

podemos escribir:

CÓDIGO

```
'posts.apps.PostsConfig',
```

La lista quedaría así:

CÓDIGO

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',

    # Aplicaciones propias
    'posts.apps.PostsConfig',
]
```

Ambas formas pueden funcionar.

En este curso vamos a usar la forma con `AppConfig`, porque es más explícita y nos ayuda a reconocer mejor cómo Django conecta cada app con el proyecto.

20. Cuidado con las comas

`INSTALLED_APPS` es una lista de Python.

Por eso, cada elemento debe estar separado por una coma.

Correcto:

CÓDIGO

```
'posts.apps.PostsConfig',
```

Incorrecto:

CÓDIGO

```
'posts.apps.PostsConfig'
'django.contrib.staticfiles',
```

Si falta una coma, Python puede interpretar mal la lista y Django puede mostrar errores al iniciar.

21. Configurar idioma

Ya que estamos en `settings.py`, podemos ajustar el idioma del proyecto.

Buscamos esta línea:

CÓDIGO

```
LANGUAGE_CODE = 'en-us'
```

Y la cambiamos por:

CÓDIGO

```
LANGUAGE_CODE = 'es-ar'
```

Esto ayuda a que algunas partes automáticas de Django, como mensajes del panel de administración, usen español cuando sea posible.

También podrías ver otros códigos según el país, por ejemplo:

CÓDIGO

```
LANGUAGE_CODE = 'es-es'
```

22. Configurar zona horaria

También podemos configurar la zona horaria.

Buscamos:

CÓDIGO

```
TIME_ZONE = 'UTC'
```

Y la cambiamos por una zona horaria más cercana a nuestro contexto.

Por ejemplo, para Argentina:

CÓDIGO

```
TIME_ZONE = 'America/Argentina/Buenos_Aires'
```

Esta configuración será importante cuando empecemos a trabajar con fechas de publicación.

23. Verificar que todo siga funcionando

Después de crear y registrar la app, ejecutamos nuevamente el servidor:

CÓDIGO

```
python manage.py runserver
```

Si no aparece ningún error, significa que Django pudo cargar la configuración correctamente.

Luego abrimos en el navegador:

CÓDIGO

```
http://127.0.0.1:8000/
```

Si la página sigue funcionando, la app quedó registrada correctamente.

24. Error común: crear la app en la carpeta equivocada

Para crear una app con:

CÓDIGO

```
python manage.py startapp posts
```

tenemos que estar en la carpeta donde está `manage.py`.

Si no vemos `manage.py`, probablemente estemos en una carpeta incorrecta.

Podemos movernos con:

CÓDIGO

```
cd nombre_de_la_carpeta
```

Y revisar con:

CÓDIGO

```
ls
```

o en Windows:

CÓDIGO

```
dir
```

25. Error común: olvidar registrar la app

Otro error frecuente es crear la app, escribir código dentro de ella y olvidarse de agregarla en `INSTALLED_APPS`.

Si la app no está registrada, Django no la tiene en cuenta como parte del proyecto.

Por eso, después de crear una app, el paso siguiente casi siempre será abrir `settings.py` y agregarla en `INSTALLED_APPS`.

26. Error común: usar nombres poco recomendados

Para los nombres de apps conviene usar:

- minúsculas;
- nombres cortos;
- nombres descriptivos;
- guiones bajos si hace falta separar palabras.

Buenos nombres:

CÓDIGO

```
posts
usuarios
comentarios
blog
```

Nombres poco recomendados:

CÓDIGO

```
Mis Posts
mis-posts
AppDeBlog
```

Django trabaja mejor con nombres simples y válidos como identificadores de Python.

27. Buenas prácticas iniciales

Al crear una app en Django, conviene seguir este flujo:

CÓDIGO

```
python manage.py startapp posts
```

Luego:

1. revisar que la carpeta se haya creado;
2. abrir `settings.py`;
3. agregar la app en `INSTALLED_APPS`;
4. configurar idioma y zona horaria si corresponde;
5. ejecutar `python manage.py runserver`;
6. comprobar que no haya errores.

También conviene hacer un commit después de este avance:

CÓDIGO

```
git status
git add .
git commit -m "Crear app posts y configurar proyecto Django"
```

De esta forma dejamos guardado un punto estable del proyecto.

28. Ejemplo completo

Desde la carpeta donde está `manage.py`, creamos la app:

CÓDIGO

```
python manage.py startapp posts
```

Abrimos `blog_project/settings.py` y registramos la app:

CÓDIGO

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',

    # Aplicaciones propias
    'posts.apps.PostsConfig',
]
```

Configuramos idioma y zona horaria:

CÓDIGO

```
LANGUAGE_CODE = 'es-ar'

TIME_ZONE = 'America/Argentina/Buenos_Aires'
```

Probamos que el servidor siga funcionando:

CÓDIGO

```
python manage.py runserver
```

Y abrimos:

CÓDIGO

```
http://127.0.0.1:8000/
```

Si no aparece ningún error, la app quedó correctamente conectada al proyecto.

Síntesis

En esta unidad aprendiste que en Django un proyecto y una aplicación no son lo mismo.

El proyecto contiene la configuración general del sitio. Las aplicaciones contienen funcionalidades específicas.

También creaste tu primera app con:

CÓDIGO

```
python manage.py startapp posts
```

Después la registraste en `INSTALLED_APPS` dentro de `settings.py`, configuraste idioma y zona horaria, y verificaste que el servidor siguiera funcionando.

Con esto, tu proyecto Django ya tiene una primera app llamada `posts`, que será la base para construir las publicaciones del blog.

PRE-ENTREGA DEL PROYECTO

Actividad evaluable · parte del proyecto integrador

Preentrega 7: repositorio GitHub y base inicial de Django listos

Instrucciones detalladas para completar la preentrega: inicialización del repositorio GitHub y configuración base del proyecto Django.

Preentrega 7: Repositorio base de Django

Propósito

En este checkpoint vas a consolidar el punto de partida técnico de tu blog web con Django.

Hasta ahora venías trabajando con Python desde consola. En esta etapa, el proyecto empieza a transformarse en una aplicación web. Para eso, necesitamos asegurarnos de que la estructura inicial de Django esté correctamente creada, configurada y subida a GitHub.

El objetivo de esta entrega no es crear todavía modelos, vistas, templates ni funcionalidades del blog. El foco está en dejar preparada una base ordenada para continuar trabajando en las próximas unidades.

Contexto

En las unidades anteriores aprendiste a:

- instalar y verificar Django;
- crear un proyecto con `django-admin startproject`;
- entender la diferencia entre proyecto y aplicación;
- crear una app con `python manage.py startapp`;
- registrar una app en `INSTALLED_APPS`;
- modificar configuraciones básicas en `settings.py`;
- usar Git para versionar cambios;
- publicar un repositorio en GitHub.

Este checkpoint reúne todos esos pasos en una entrega concreta.

Objetivo

Crear y publicar un repositorio base de Django que incluya:

- un proyecto Django llamado `blog_project`;
- una aplicación llamada `posts`;
- configuración básica de idioma y zona horaria;
- archivo `.gitignore`;

- archivo `requirements.txt`;
- archivo `README.md`;
- historial de commits claro;
- repositorio público en GitHub.

Pasos a seguir

1. Crear la estructura del proyecto Django

Dentro de tu carpeta de trabajo, asegurate de haber creado el proyecto con el siguiente comando:

CÓDIGO

```
django-admin startproject blog_project .
```

El punto final `.` es importante porque permite que el archivo `manage.py` quede en la raíz de la carpeta de trabajo.

La estructura esperada debería ser similar a esta:

CÓDIGO

```
blog_django/
├── manage.py
├── blog_project/
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
```

2. Crear la aplicación `posts`

Desde la carpeta donde está el archivo `manage.py`, creá la aplicación principal del blog:

CÓDIGO

```
python manage.py startapp posts
```

Esto generará una carpeta llamada `posts` con la estructura inicial de una app Django.

La estructura esperada será similar a esta:

CÓDIGO

```
posts/
├── __init__.py
├── admin.py
├── apps.py
├── migrations/
│   └── __init__.py
├── models.py
├── tests.py
└── views.py
```

3. Registrar la app en `settings.py`

Abrí el archivo:

CÓDIGO

```
blog_project/settings.py
```

Buscá la lista `INSTALLED_APPS` y agregá la app `posts` usando su configuración:

CÓDIGO

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',

    # Aplicaciones propias
    'posts.apps.PostsConfig',
]
```

Este paso es fundamental. Si la app no está registrada, Django no la reconocerá correctamente en las próximas etapas.

4. Configurar idioma y zona horaria

En el mismo archivo `settings.py` , buscá estas variables:

CÓDIGO

```
LANGUAGE_CODE = 'en-us'

TIME_ZONE = 'UTC'
```

Y modificalas por una configuración regional en español.

Por ejemplo:

CÓDIGO

```
LANGUAGE_CODE = 'es-ar'

TIME_ZONE = 'America/Argentina/Buenos_Aires'
```

También podés usar otra variante regional si corresponde a tu país.

5. Verificar que el servidor funcione

Ejecutá el servidor de desarrollo:

CÓDIGO

```
python manage.py runserver
```

Luego abrí en el navegador:

CÓDIGO

```
http://127.0.0.1:8000/
```

Si ves la pantalla inicial de Django y no aparecen errores en la terminal, la configuración base está funcionando correctamente.

Para detener el servidor, presioná:

CÓDIGO

```
Ctrl + C
```

6. Crear el archivo `.gitignore`

Creá un archivo llamado:

CÓDIGO

```
.gitignore
```

Incluí como mínimo:

CÓDIGO

```
venv/
__pycache__/
*.pyc
.env
db.sqlite3
```

Este archivo le indica a Git qué archivos o carpetas no debe subir al repositorio.

Es importante no subir:

- el entorno virtual;
- archivos temporales de Python;
- archivos de variables privadas;
- la base de datos local.

Aunque todavía no trabajemos en profundidad con base de datos, `db.sqlite3` debe quedar fuera del repositorio porque es un archivo local generado por Django.

7. Generar el archivo `requirements.txt`

Con el entorno virtual activo, ejecutá:

CÓDIGO

```
pip freeze > requirements.txt
```

Este archivo guarda las dependencias instaladas en el proyecto.

Debe incluir Django.

Otra persona podrá usar este archivo para instalar las mismas dependencias en su computadora.

8. Crear el archivo `README.md`

Creá un archivo llamado:

CÓDIGO

```
README.md
```

El README debe explicar cómo ejecutar el proyecto.

Debe incluir, como mínimo:

- nombre del proyecto;
- breve descripción;
- pasos para clonar el repositorio;
- cómo crear y activar el entorno virtual;
- cómo instalar dependencias con `requirements.txt`;
- cómo levantar el servidor de desarrollo;
- nombre de la app principal creada.

Ejemplo orientativo:

CÓDIGO

````
# Blog Django

Proyecto base de un blog web desarrollado con Django.

## Descripción

Este repositorio contiene la estructura inicial de un proyecto Django para construir
un blog.
Incluye la configuración base del proyecto y una aplicación llamada `posts`.

## Instalación

Clonar el repositorio:

```bash
git clone URL_DEL_REPOSITORIO
```

Entrar a la carpeta del proyecto:
```
cd NOMBRE_DEL_REPOSITORIO
```

Crear entorno virtual:
python -m venv venv

Activar entorno virtual:
En Windows PowerShell:
```
.\venv\Scripts\Activate.ps1
```

En Linux o macOS:
```
source venv/bin/activate
```

Instalar dependencias:
```
pip install -r requirements.txt
```

Ejecutar el servidor:
```
python manage.py runserver
```

Abrir en el navegador:
http://127.0.0.1:8000/
````

```
Aplicaciones
- posts: aplicación inicial para manejar las publicaciones del blog.
```

---

9. Versionar los cambios con Git

Revisá el estado del repositorio:

CÓDIGO

```
git status
```

Agregá los archivos:

CÓDIGO

```
git add .
```

Creá un commit descriptivo:

CÓDIGO

```
git commit -m "Inicializar proyecto Django y app posts"
```

El mensaje del commit debe explicar claramente qué cambio realizaste.

Evitá mensajes como:

CÓDIGO

```
cambios
cosas
update
```

---

10. Publicar el repositorio en GitHub

Subí el proyecto a un repositorio público de GitHub.

El repositorio debe contener la estructura base del proyecto Django, pero no debe incluir la carpeta `venv/` ni archivos ignorados por `.gitignore`.

Después de subirlo, verificá desde GitHub que estén presentes archivos como:

CÓDIGO

```
manage.py
blog_project/
posts/
.gitignore
README.md
requirements.txt
```

Entrega

Deberás entregar el enlace al repositorio público de GitHub.

Criterios de aceptación

La entrega se considera correcta si:

- El repositorio está publicado en GitHub.
- El archivo `manage.py` está en la raíz del proyecto.
- Existe la carpeta `blog_project`.
- Existe la app `posts`.
- La app `posts` está registrada en `INSTALLED_APPS` como:

CÓDIGO

```
'posts.apps.PostsConfig',
```

- El idioma está configurado en español.
- La zona horaria está configurada correctamente.
- Existe un archivo `.gitignore`.
- El `.gitignore` incluye `venv/` y `db.sqlite3`.
- La carpeta `venv/` no fue subida al repositorio.
- Existe el archivo `requirements.txt`.
- El archivo `requirements.txt` incluye Django.
- Existe un archivo `README.md` con instrucciones básicas para ejecutar el proyecto.
- El servidor puede levantarse con:

CÓDIGO

```
python manage.py runserver
```

Qué no se espera en esta entrega

En este checkpoint todavía no hace falta crear:

- modelos propios;
- vistas;
- templates;
- formularios;
- rutas de la app `posts`;
- CRUD;
- usuarios;
- login;
- estilos personalizados;
- carga de imágenes.

Todo eso se trabajará en futuras unidades.

Errores comunes a evitar

Olvidar registrar la app `posts`

No alcanza con crear la app usando:

CÓDIGO

```
python manage.py startapp posts
```

También debe estar agregada en `INSTALLED_APPS`.

Subir la carpeta `venv/`

La carpeta `venv/` no debe subirse a GitHub.

Si aparece en el repositorio, significa que el `.gitignore` no se configuró correctamente o se agregó tarde.

Olvidar actualizar `requirements.txt`

Después de instalar Django, recordá ejecutar:

CÓDIGO

```
pip freeze > requirements.txt
```

Si no lo hacés, otra persona no sabrá qué dependencias instalar para ejecutar tu proyecto.

Crear el proyecto con una carpeta extra innecesaria

Para este checkpoint se espera que uses:

CÓDIGO

```
django-admin startproject blog_project .
```

El punto final evita que `manage.py` quede dentro de una carpeta adicional.

Hacer commits poco claros

Usá mensajes descriptivos.

Ejemplo recomendado:

CÓDIGO

```
git commit -m "Inicializar proyecto Django y app posts"
```

Síntesis

En este checkpoint dejaste preparado el repositorio base del blog web.

Creaste el proyecto Django, agregaste la app `posts`, configuraste idioma y zona horaria, preparaste archivos importantes como `.gitignore`, `requirements.txt` y `README.md`, y publicaste el resultado en GitHub.

Esta base será el punto de partida para las próximas etapas del desarrollo del blog.

Tambien te quedaran 2 cheatsheet para comandos de git y django, tenelos como ayuda (algunos ya se vieron, otros estan por verlos).

Pasos para la entrega

Pasos a seguir:

1. **Estructura del Proyecto**: Asegúrate de haber ejecutado `django-admin startproject blog_project .` para que el archivo `manage.py` quede en la raíz de tu carpeta de trabajo.

2. **Configuración Interna**: En `settings.py`, verifica que `posts.apps.PostsConfig` esté incluida en `INSTALLED_APPS`. Ajusta el idioma a español (`es-ar` o tu variante regional) y la zona horaria.
3. **Control de Versiones**: Es crucial que el archivo `.gitignore` incluya la carpeta `venv/` y el archivo `db.sqlite3`. Nunca subas el entorno virtual ni la base de datos local al repositorio público.
4. **Documentación**: Tu archivo `README.md` es la cara de tu proyecto. Debe explicarle a otra persona cómo clonar tu repositorio, instalar las dependencias (usando el archivo `requirements.txt` que generaste) y levantar el servidor.
5. **Publicación**: Sube tus cambios a un repositorio público de GitHub. Verifica que el historial de commits sea descriptivo (ej: 'Inicialización de proyecto Django y app posts').

Errores comunes:

- Olvidar agregar la app `posts` en `settings.py`, lo que impedirá que Django reconozca tus futuros modelos y vistas.
- Subir la carpeta `venv/` al repositorio (ocupa mucho espacio y no es portable). Si esto sucede, bórrala de GitHub, agrégala al `.gitignore` y vuelve a subir.
- No actualizar el `requirements.txt` tras instalar Django.

Entregable

Formato: repositorio

Repositorio público en GitHub conteniendo la estructura base del proyecto Django, la aplicación 'posts' configurada y el archivo README.md.

Objetivos de aprendizaje

- Inicializar un entorno de desarrollo profesional con Django y entornos virtuales.
- Configurar la estructura modular de Django mediante la creación y registro de aplicaciones.
- Gestionar el control de versiones utilizando Git y GitHub, aplicando buenas prácticas de exclusión de archivos con .gitignore.
- Documentar técnicamente un proyecto de software para facilitar su instalación y ejecución por terceros.

Criterios de aceptación

- El repositorio contiene el archivo manage.py en la raíz y la carpeta blog_project con settings.py.
- La aplicación 'posts' está creada y presente como una carpeta en el repositorio.
- El archivo blog_project/settings.py tiene la app 'posts' correctamente registrada en INSTALLED_APPS.
- El archivo .gitignore está presente y excluye correctamente la carpeta venv/ y el archivo db.sqlite3.
- Existe un archivo requirements.txt que incluye la dependencia de Django.
- El archivo README.md incluye el nombre del proyecto y los comandos para instalar dependencias y ejecutar el servidor.

Rúbrica de evaluación

Esta rúbrica evalúa un repositorio público de GitHub con la estructura inicial de un proyecto Django correctamente creada, configurada y documentada para servir como base del blog web. Se valora la organización técnica del proyecto, la configuración de Django, el uso adecuado de Git/GitHub y la calidad mínima de la documentación para que otra persona pueda reproducir el entorno y continuar el desarrollo.

Criterio	Descripción	Peso
Estructura inicial del proyecto Django	Evalúa si el proyecto fue creado con la estructura esperada para este checkpoint, incluyendo la ubicación correcta de manage.py, la presencia de la carpeta del proyecto Django y la app posts lista para continuar el desarrollo.	30%
Configuración de Django y app posts	Evalúa la configuración interna del proyecto, especialmente la inclusión de posts.apps.PostsConfig en INSTALLED_APPS, el idioma en español y la zona horaria ajustada, ya que estas decisiones permiten que Django reconozca la app y que el proyecto quede contextualizado correctamente.	25%
Control de versiones y archivos ignorados	Evalúa el uso correcto de Git y la limpieza del repositorio público: presencia de commits útiles, historial descriptivo y un .gitignore que evite subir venv/ y db.sqlite3. Este criterio es clave para trabajar de forma profesional y segura en GitHub.	25%
Documentación y reproducibilidad del entorno	Evalúa la calidad del README.md como guía para otra persona: debe explicar cómo clonar el repositorio, instalar dependencias desde requirements.txt y ejecutar el servidor. Se valora que la documentación permita reproducir el entorno con pasos claros.	20%

Total: 100 pts | Aprobación: 70 pts

MÓDULO 8

Primer sitio Django: URLs, views, templates y static

Construir la interfaz web estática del blog utilizando el sistema de plantillas de Django y herencia.

Entender cómo Django decide qué página mostrar

Aprende cómo funciona el sistema de navegación de Django: desde que un usuario escribe una URL en el navegador hasta que el servidor decide qué lógica ejecutar mediante las vistas.

Cuando un usuario entra a una web, por ejemplo:

CÓDIGO

```
http://localhost:8000/posts/
```

Django necesita saber qué código ejecutar para responder a esa dirección.

No lo hace “por magia”. Django usa un sistema de rutas que funciona como un mapa: recibe una URL, busca si esa URL existe en sus archivos de configuración y, si la encuentra, ejecuta una función encargada de preparar la respuesta.

En esta unidad vamos a trabajar con dos piezas fundamentales:

- `urls.py` : define las rutas disponibles.
- `views.py` : contiene las funciones que responden a esas rutas.

1. El ciclo de vida de una petición

Cuando una persona entra a una página de nuestro blog, sucede este recorrido:

1. El usuario escribe una URL en el navegador.

CÓDIGO

```
http://localhost:8000/posts/
```

1. Django recibe esa petición.
2. Django revisa sus archivos `urls.py` para encontrar una ruta que coincida con `/posts/` .
3. Si encuentra una coincidencia, llama a una función ubicada en `views.py` .
4. Esa función prepara una respuesta.
5. Django devuelve esa respuesta al navegador.

Si Django no encuentra ninguna ruta que coincida, aparece el conocido error:

CÓDIGO

```
404 Not Found
```

2. ¿Qué es `urls.py`?

El archivo `urls.py` es el mapa de rutas del proyecto.

Allí definimos qué dirección web ejecuta qué vista.

Un ejemplo simple sería:

CÓDIGO

```
from django.urls import path
from . import views

urlpatterns = [
    path("inicio/", views.inicio, name="inicio"),
]
```

Analicemos esta ruta:

CÓDIGO

```
path("inicio/", views.inicio, name="inicio")
```

- `"inicio/"`: es la URL que el usuario puede visitar.
- `views.inicio`: es la función que Django va a ejecutar.
- `name="inicio"`: es un nombre interno para identificar esa ruta.

Entonces, si el proyecto está corriendo en:

CÓDIGO

```
http://localhost:8000/
```

la ruta completa sería:

CÓDIGO

```
http://localhost:8000/inicio/
```

3. ¿Qué es una vista?

Una vista es una función de Python que recibe una petición y devuelve una respuesta.

Por ejemplo, en el archivo `views.py` podríamos tener esto:

CÓDIGO

```
from django.http import HttpResponse

def inicio(request):
    return HttpResponse("Bienvenido a mi blog")
```

Esta vista recibe un parámetro llamado `request`.

CÓDIGO

```
def inicio(request):
```

Ese `request` representa la petición que hizo el usuario. Por ahora no necesitamos usarlo demasiado, pero Django siempre lo envía porque puede contener información importante, como:

- qué URL visitó el usuario,
- qué método HTTP usó,
- si el usuario está autenticado,
- qué datos envió desde un formulario.

La respuesta se devuelve con:

CÓDIGO

```
return HttpResponse("Bienvenido a mi blog")
```

`HttpResponse` permite devolver texto simple al navegador.

4. Conectar una URL con una vista

Supongamos que tenemos una app llamada `posts`.

Dentro de esa app podríamos tener esta estructura:

CÓDIGO

```
posts/
├── __init__.py
├── admin.py
├── apps.py
├── models.py
├── urls.py
├── views.py
```

En `posts/views.py` escribimos:

CÓDIGO

```
from django.http import HttpResponse

def lista_posts(request):
    return HttpResponse("Listado de posts del blog")
```

Luego, en `posts/urls.py`, conectamos esa vista con una ruta:

CÓDIGO

```
from django.urls import path
from . import views

urlpatterns = [
    path("posts/", views.lista_posts, name="lista_posts"),
]
```

Ahora, cuando el usuario visite:

CÓDIGO

```
http://localhost:8000/posts/
```

Django ejecutará esta función:

CÓDIGO

```
def lista_posts(request):
    return HttpResponse("Listado de posts del blog")
```

5. Separar URLs del proyecto y URLs de la app

En Django existe una diferencia importante entre proyecto y aplicación.

El proyecto es la configuración general.

La app es una parte específica del sistema.

Por ejemplo:

CÓDIGO

```
blog_project/
├── settings.py
├── urls.py
posts/
├── urls.py
├── views.py
```

El archivo principal del proyecto podría estar en:

CÓDIGO

```
blog_project/urls.py
```

Y el archivo de rutas de la app podría estar en:

CÓDIGO

```
posts/urls.py
```

La buena práctica es no poner todas las rutas en el `urls.py` principal. Lo ideal es que cada app tenga sus propias rutas.

En `blog_project/urls.py`:

CÓDIGO

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("posts.urls")),
]
```

Con esta línea:

CÓDIGO

```
path("", include("posts.urls"))
```

le decimos a Django:

> **Nota**
> “Para las rutas principales del sitio, revisá también el archivo `urls.py` de la app `posts`”.

Entonces ahora Django también tendrá en cuenta las rutas definidas en:

CÓDIGO

```
posts/urls.py
```

6. Ejemplo completo con varias páginas

Ahora vamos a crear tres páginas para nuestro blog:

- Inicio
- Lista de posts

- Contacto

En `posts/views.py`:

CÓDIGO

```
from django.http import HttpResponse

def inicio(request):
    return HttpResponse("Bienvenido a mi blog")

def lista_posts(request):
    return HttpResponse("Estos son los posts del blog")

def contacto(request):
    return HttpResponse("Página de contacto")
```

En `posts/urls.py`:

CÓDIGO

```
from django.urls import path
from . import views

urlpatterns = [
    path("", views.inicio, name="inicio"),
    path("posts/", views.lista_posts, name="lista_posts"),
    path("contacto/", views.contacto, name="contacto"),
]
```

Con esta configuración, las URLs disponibles serían:

CÓDIGO

```
http://localhost:8000/
http://localhost:8000/posts/
http://localhost:8000/contacto/
```

Cada una ejecuta una vista distinta.

7. Rutas dinámicas

No todas las URLs son fijas.

En un blog, normalmente queremos ver un post específico. Por ejemplo:

CÓDIGO

```
http://localhost:8000/posts/1/
http://localhost:8000/posts/2/
http://localhost:8000/posts/3/
```

Para eso usamos rutas dinámicas.

En `posts/urls.py`:

CÓDIGO

```
from django.urls import path
from . import views

urlpatterns = [
    path("posts/", views.lista_posts, name="lista_posts"),
    path("posts/<int:post_id>/", views.detalle_post, name="detalle_post"),
]
```

La parte importante es esta:

CÓDIGO

```
<int:post_id>
```

Eso significa:

> **Nota**
> "Django espera recibir un número entero en esta parte de la URL, y lo va a guardar en una variable llamada `post_id`".

Ahora en `posts/views.py`:

CÓDIGO

```
from django.http import HttpResponse

def lista_posts(request):
    return HttpResponse("Listado de posts")

def detalle_post(request, post_id):
    return HttpResponse(f"Estás viendo el post número {post_id}")
```

Si el usuario visita:

CÓDIGO

```
http://localhost:8000/posts/5/
```

Django ejecuta:

CÓDIGO

```
detalle_post(request, post_id=5)
```

Y la respuesta será:

CÓDIGO

```
Estás viendo el post número 5
```

8. Devolver HTML usando `render`

Hasta ahora usamos `HttpResponse`, que sirve para ejemplos simples. Pero en una web real no queremos devolver solo texto plano.

Queremos devolver una página HTML.

Para eso usamos `render`.

Primero, imaginemos esta estructura:

CÓDIGO

```
posts/
├── templates/
│   └── posts/
│       ├── inicio.html
│       ├── lista_posts.html
│       └── detalle_post.html
├── urls.py
├── views.py
```

En `posts/views.py`:

CÓDIGO

```
from django.shortcuts import render

def inicio(request):
    return render(request, "posts/inicio.html")

def lista_posts(request):
    return render(request, "posts/lista_posts.html")

def contacto(request):
    return render(request, "posts/contacto.html")
```

Ahora la vista ya no devuelve texto directo.

En lugar de esto:

CÓDIGO

```
return HttpResponse("Bienvenido a mi blog")
```

usamos esto:

CÓDIGO

```
return render(request, "posts/inicio.html")
```

Eso le dice a Django:

> **Nota**
>
> “Buscá esta plantilla HTML y devolvela como respuesta”.

9. Enviar datos desde la vista hacia el template

Una vista no solo puede mostrar un archivo HTML. También puede enviarle datos.

Por ejemplo, en `posts/views.py`:

CÓDIGO

```
from django.shortcuts import render

def lista_posts(request):
    posts = [
        {"id": 1, "titulo": "Mi primer post", "autor": "Ana"},
        {"id": 2, "titulo": "Aprendiendo Django", "autor": "Luis"},
        {"id": 3, "titulo": "Rutas y vistas", "autor": "Marta"},
    ]

    contexto = {
        "posts": posts
    }

    return render(request, "posts/lista_posts.html", contexto)
```

Acá estamos creando una lista de posts simulada.

Luego armamos un diccionario llamado `contexto`:

CÓDIGO

```
contexto = {
    "posts": posts
}
```

Ese diccionario se envía al template.

En `posts/templates/posts/lista_posts.html`:

CÓDIGO

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Lista de posts</title>
</head>
<body>
    <h1>Posts del blog</h1>

    <ul>
        {% for post in posts %}
            <li>
                {{ post.titulo }} - {{ post.autor }}
            </li>
        {% endfor %}
    </ul>
</body>
</html>
```

Django recibe la lista `posts` desde la vista y permite recorrerla dentro del HTML con:

CÓDIGO

```
{% for post in posts %}
```

Y mostrar datos con:

CÓDIGO

```
{{ post.titulo }}
{{ post.autor }}
```

10. Usar el nombre de una ruta en un template

Cuando definimos una ruta, usamos `name`.

Por ejemplo:

CÓDIGO

```
path("posts/", views.lista_posts, name="lista_posts")
```

Ese nombre se puede usar dentro de un template para crear enlaces.

En vez de escribir la URL a mano:

CÓDIGO

```
<a href="/posts/">Ver posts</a>
```

podemos escribir:

CÓDIGO

```
<a href="{% url 'lista_posts' %}">Ver posts</a>
```

Esto es mejor porque si más adelante cambiamos la ruta de:

CÓDIGO

```
path("posts/", views.lista_posts, name="lista_posts")
```

a:

CÓDIGO

```
path("publicaciones/", views.lista_posts, name="lista_posts")
```

el template seguirá funcionando, porque usa el nombre interno de la ruta.

11. Ejemplo completo con enlace al detalle de cada post

En `posts/views.py`:

CÓDIGO

```
from django.shortcuts import render

def lista_posts(request):
    posts = [
        {"id": 1, "titulo": "Mi primer post", "autor": "Ana"},
        {"id": 2, "titulo": "Aprendiendo Django", "autor": "Luis"},
        {"id": 3, "titulo": "Rutas y vistas", "autor": "Marta"},
    ]

    return render(request, "posts/lista_posts.html", {"posts": posts})


def detalle_post(request, post_id):
    post = {
        "id": post_id,
        "titulo": f"Post número {post_id}",
        "contenido": "Este es el contenido de ejemplo del post.",
        "autor": "Ana",
    }

    return render(request, "posts/detalle_post.html", {"post": post})
```

En `posts/urls.py`:

CÓDIGO

```
from django.urls import path
from . import views

urlpatterns = [
    path("posts/", views.lista_posts, name="lista_posts"),
    path("posts/<int:post_id>/", views.detalle_post, name="detalle_post"),
]
```

En `posts/templates/posts/lista_posts.html`:

CÓDIGO

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Posts</title>
</head>
<body>
    <h1>Posts del blog</h1>

    <ul>
        {% for post in posts %}
            <li>
                <a href="{% url 'detalle_post' post.id %}">
                    {{ post.titulo }}
                </a>
                - {{ post.autor }}
            </li>
        {% endfor %}
    </ul>
</body>
</html>
```

En `posts/templates/posts/detalle_post.html`:

CÓDIGO

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>{{ post.titulo }}</title>
</head>
<body>
    <h1>{{ post.titulo }}</h1>

    <p><strong>Autor:</strong> {{ post.autor }}</p>

    <p>{{ post.contenido }}</p>

    <a href="{% url 'lista_posts' %}">Volver al listado</a>
</body>
</html>
```

La parte más importante del enlace es esta:

CÓDIGO

```
{% url 'detalle_post' post.id %}
```

Eso genera una URL como:

CÓDIGO

```
/posts/1/
/posts/2/
/posts/3/
```

según el `id` de cada post.

12. Vista vs Template

Es importante separar responsabilidades.

La vista se encarga de la lógica.

Por ejemplo:

CÓDIGO

```
def lista_posts(request):
    posts = [
        {"id": 1, "titulo": "Mi primer post", "autor": "Ana"},
        {"id": 2, "titulo": "Aprendiendo Django", "autor": "Luis"},
    ]

    return render(request, "posts/lista_posts.html", {"posts": posts})
```

El template se encarga de la presentación.

Por ejemplo:

CÓDIGO

```
<h1>Posts del blog</h1>

<ul>
    {% for post in posts %}
        <li>{{ post.titulo }} - {{ post.autor }}</li>
    {% endfor %}
</ul>
```

La vista decide qué datos se muestran.

El template decide cómo se ven esos datos.

13. Errores comunes

Error 1: olvidar importar la vista

Incorrecto:

CÓDIGO

```
from django.urls import path

urlpatterns = [
    path("posts/", views.lista_posts, name="lista_posts"),
]
```

Esto falla porque `views` no fue importado.

Correcto:

CÓDIGO

```
from django.urls import path
from . import views

urlpatterns = [
    path("posts/", views.lista_posts, name="lista_posts"),
]
```

Error 2: olvidar conectar las URLs de la app

Si definimos rutas en `posts/urls.py`, pero no las incluimos en `blog_project/urls.py`, Django no las va a encontrar.

Correcto:

CÓDIGO

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("posts.urls")),
]
```

Error 3: escribir mal el nombre de la ruta

Si en `urls.py` tenemos:

CÓDIGO

```
path("posts/", views.lista_posts, name="lista_posts")
```

en el template debemos escribir exactamente:

CÓDIGO

```
{% url 'lista_posts' %}
```

Si escribimos otro nombre, por ejemplo:

CÓDIGO

```
{% url 'posts' %}
```

Django mostrará un error porque no encuentra una ruta con ese nombre.

Error 4: olvidar devolver una respuesta

Incorrecto:

CÓDIGO

```
def inicio(request):
    mensaje = "Bienvenido"
```

Esta vista no devuelve nada.

Correcto:

CÓDIGO

```
from django.http import HttpResponse

def inicio(request):
    mensaje = "Bienvenido"
    return HttpResponse(mensaje)
```

O usando template:

CÓDIGO

```
from django.shortcuts import render

def inicio(request):
    return render(request, "posts/inicio.html")
```

14. Resumen

En esta unidad aprendimos que:

- Django usa archivos `urls.py` para decidir qué vista ejecutar.
- Una ruta se define con `path()`.
- Una vista es una función de Python que recibe `request` y devuelve una respuesta.
- `HttpResponse` sirve para devolver texto simple.
- `render` sirve para devolver una plantilla HTML.
- Es buena práctica separar las URLs del proyecto y las URLs de cada app usando `include`.
- Las rutas pueden ser fijas, como `"contacto/"`, o dinámicas, como `"posts/<int: post_id>/"`.

- El atributo `name` permite referenciar rutas sin escribir URLs a mano.
- La vista maneja lógica y datos; el template maneja la presentación.

Actividad práctica

Crear tres páginas para el blog:

1. Página de inicio.
2. Página de listado de posts.
3. Página “Acerca de mí”.

El proyecto debe tener:

CÓDIGO

```
posts/views.py
posts/urls.py
posts/templates/posts/inicio.html
posts/templates/posts/lista_posts.html
posts/templates/posts/acerca_de.html
```

En `posts/views.py`, crear tres vistas:

CÓDIGO

```
from django.shortcuts import render

def inicio(request):
    return render(request, "posts/inicio.html")

def lista_posts(request):
    posts = [
        {"id": 1, "titulo": "Primer post", "autor": "Ana"},
        {"id": 2, "titulo": "Segundo post", "autor": "Luis"},
    ]

    return render(request, "posts/lista_posts.html", {"posts": posts})

def acerca_de(request):
    return render(request, "posts/acerca_de.html")
```

En `posts/urls.py`, crear las rutas:

CÓDIGO

```
from django.urls import path
from . import views

urlpatterns = [
    path("", views.inicio, name="inicio"),
    path("posts/", views.lista_posts, name="lista_posts"),
    path("acerca-de/", views.acerca_de, name="acerca_de"),
]
```

Y en `blog_project/urls.py`, incluir las rutas de la app:

CÓDIGO

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("posts.urls")),
]
```

Pregunta para cerrar:

Si quisieras agregar una página de “Contacto”, ¿qué archivos tendrías que modificar y qué ruta crearías?

Crear vistas basadas en funciones para las páginas del blog

Aprende a crear funciones en Python que actúan como vistas en Django, procesando peticiones del usuario y devolviendo respuestas HTML a través del sistema de rutas.

Vistas basadas en funciones en Django

En la unidad anterior vimos cómo Django recibe una petición, busca una ruta en `urls.py` y decide qué función ejecutar.

Ahora vamos a enfocarnos en esa función: la **vista**.

Una vista es una función de Python que recibe una petición del usuario y devuelve una respuesta. En otras palabras, es el lugar donde Django decide qué mostrar.

1. ¿Qué es una Vista Basada en Función?

En Django, una Vista Basada en Función, también llamada **FBV** por *Function-Based View*, es una función común de Python que cumple dos reglas importantes:

1. Recibe un parámetro llamado `request`.
2. Devuelve una respuesta.

Por ejemplo:

CÓDIGO

```
from django.http import HttpResponse

def inicio(request):
    return HttpResponse("Bienvenido a mi blog")
```

Esta es una vista mínima.

Cuando el usuario entra a la URL conectada con esta vista, Django ejecuta la función `inicio` y devuelve el texto:

CÓDIGO

```
Bienvenido a mi blog
```

2. El parámetro `request`

Toda vista recibe un objeto llamado `request`.

CÓDIGO

```
def inicio(request):
    return HttpResponse("Bienvenido a mi blog")
```

Ese objeto contiene información sobre la petición que hizo el usuario.

Por ahora no necesitamos usarlo demasiado, pero más adelante puede servir para saber cosas como:

- qué usuario está navegando,
- qué datos envió desde un formulario,
- qué método HTTP usó,
- qué información viene desde el navegador.

Aunque no lo usemos dentro de la función, Django espera que la vista lo reciba.

Esto sería incorrecto:

CÓDIGO

```
def inicio():
    return HttpResponse("Bienvenido a mi blog")
```

La vista debe recibir `request`:

CÓDIGO

```
def inicio(request):
    return HttpResponse("Bienvenido a mi blog")
```

3. Devolver una respuesta con `HttpResponse`

La forma más simple de responder desde una vista es usando `HttpResponse`.

CÓDIGO

```
from django.http import HttpResponse

def inicio(request):
    return HttpResponse("Página de inicio")
```

También podemos crear varias vistas en el mismo archivo `views.py`.

CÓDIGO

```
from django.http import HttpResponse

def inicio(request):
    return HttpResponse("Bienvenido a mi blog")

def lista_posts(request):
    return HttpResponse("Listado de posts")

def contacto(request):
    return HttpResponse("Página de contacto")
```

Cada vista representa una página distinta del sitio.

4. Conectar las vistas con URLs

Para que Django pueda ejecutar esas vistas, debemos conectarlas en el archivo `urls.py` de la app.

Por ejemplo, en `posts/urls.py`:

CÓDIGO

```
from django.urls import path
from . import views

urlpatterns = [
    path("", views.inicio, name="inicio"),
    path("posts/", views.lista_posts, name="lista_posts"),
    path("contacto/", views.contacto, name="contacto"),
]
```

Con esta configuración, tendríamos estas páginas:

CÓDIGO

```
http://localhost:8000/
http://localhost:8000/posts/
http://localhost:8000/contacto/
```

Cada URL ejecuta una vista diferente.

5. Usar `render` para devolver HTML

`HttpResponse` sirve para ejemplos simples, pero en una web real normalmente queremos devolver archivos HTML.

Para eso usamos `render`.

CÓDIGO

```
from django.shortcuts import render

def inicio(request):
    return render(request, "posts/inicio.html")
```

La función `render` recibe, como mínimo, dos elementos:

CÓDIGO

```
render(request, "posts/inicio.html")
```

- `request`: la petición del usuario.
- `"posts/inicio.html"`: el template HTML que queremos mostrar.

Entonces, en vez de devolver texto plano, Django busca un archivo HTML y lo devuelve como respuesta.

6. Estructura recomendada de templates

Para que Django encuentre los templates, podemos usar esta estructura dentro de la app:

CÓDIGO

```
posts/
├── templates/
│   └── posts/
│       ├── inicio.html
│       ├── lista_posts.html
│       └── contacto.html
├── urls.py
├── views.py
```

Es común repetir el nombre de la app dentro de `templates`.

Por eso usamos:

CÓDIGO

```
templates/posts/inicio.html
```

Y desde la vista lo llamamos así:

CÓDIGO

```
return render(request, "posts/inicio.html")
```

7. Ejemplo: vista de inicio con template

En `posts/views.py`:

CÓDIGO

```
from django.shortcuts import render

def inicio(request):
    return render(request, "posts/inicio.html")
```

En `posts/templates/posts/inicio.html`:

CÓDIGO

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8" />
    <title>Inicio</title>
</head>
<body>
    <h1>Bienvenido a mi blog</h1>
    <p>Este es el inicio del proyecto.</p>
</body>
</html>
```

Cuando el usuario entre a:

CÓDIGO

```
http://localhost:8000/
```

Django ejecutará la vista `inicio` y mostrará el template `inicio.html`.

8. Enviar datos desde la vista al template

Una vista puede enviar información al HTML usando un diccionario.

Ese diccionario se llama comúnmente `contexto`.

Por ejemplo:

CÓDIGO

```
from django.shortcuts import render

def lista_posts(request):
    posts = [
        {"titulo": "Mi primer post", "autor": "Ana"},
        {"titulo": "Aprendiendo Django", "autor": "Luis"},
        {"titulo": "Vistas y rutas", "autor": "Marta"},
    ]

    contexto = {
        "posts": posts
    }

    return render(request, "posts/lista_posts.html", contexto)
```

En este caso, la vista crea una lista de posts y se la envía al template.

La clave `"posts"` del diccionario será el nombre que podremos usar dentro del HTML.

9. Mostrar datos en el template

En `posts/templates/posts/lista_posts.html`:

CÓDIGO

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8" />
    <title>Posts</title>
</head>
<body>
    <h1>Listado de posts</h1>

    <ul>
        {% for post in posts %}
            <li>{{ post.titulo }} - {{ post.autor }}</li>
        {% endfor %}
    </ul>
</body>
</html>
```

La línea:

CÓDIGO

```
{% for post in posts %}
```

recorre la lista de posts que enviamos desde la vista.

Y estas líneas:

CÓDIGO

```
{{ post.titulo }}
{{ post.autor }}
```

muestran los datos de cada post.

10. Vista con un dato individual

También podemos enviar un solo dato al template.

En `posts/views.py`:

CÓDIGO

```
from django.shortcuts import render

def acerca_de(request):
    autor = {
        "nombre": "Ana",
        "bio": "Desarrolladora Python y autora del blog.",
        "especialidad": "Django"
    }

    return render(request, "posts/acerca_de.html", {"autor": autor})
```

En `posts/templates/posts/acerca_de.html`:

CÓDIGO

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8" />
    <title>Acerca de mí</title>
</head>
<body>
    <h1>Acerca de {{ autor.nombre }}</h1>

    <p>{{ autor.bio }}</p>

    <p>Especialidad: {{ autor.especialidad }}</p>
</body>
</html>
```

La vista decide qué datos enviar.

El template decide cómo mostrarlos.

11. Vista con parámetro desde la URL

A veces una vista necesita recibir un dato desde la URL.

Por ejemplo:

CÓDIGO

```
http://localhost:8000/posts/1/
http://localhost:8000/posts/2/
```

Para eso, en `posts/urls.py` podemos escribir:

CÓDIGO

```
from django.urls import path
from . import views

urlpatterns = [
    path("posts/<int:post_id>/", views.detalle_post, name="detalle_post"),
]
```

La parte:

CÓDIGO

```
<int:post_id>
```

significa que Django espera recibir un número entero y lo va a guardar en una variable llamada `post_id`.

Entonces la vista debe recibir ese parámetro:

CÓDIGO

```
from django.shortcuts import render

def detalle_post(request, post_id):
    post = {
        "id": post_id,
        "titulo": f"Post número {post_id}",
        "contenido": "Este es el contenido del post.",
        "autor": "Ana"
    }

    return render(request, "posts/detalle_post.html", {"post": post})
```

En el template `posts/templates/posts/detalle_post.html`:

CÓDIGO

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8" />
    <title>{{ post.titulo }}</title>
</head>
<body>
    <h1>{{ post.titulo }}</h1>

    <p>{{ post.contenido }}</p>

    <p>Autor: {{ post.autor }}</p>
</body>
</html>
```

Si el usuario entra a:

CÓDIGO

```
http://localhost:8000/posts/3/
```

Django ejecutará la vista con:

CÓDIGO

```
post_id = 3
```

12. Usar enlaces entre páginas

Los nombres de las rutas nos permiten crear enlaces dentro de los templates sin escribir la URL manualmente.

Si en `urls.py` tenemos:

CÓDIGO

```
path("posts/", views.lista_posts, name="lista_posts")
```

en un template podemos escribir:

CÓDIGO

```
<a href="{% url 'lista_posts' %}">Ver posts</a>
```

Esto es mejor que escribir:

CÓDIGO

```
<a href="/posts/">Ver posts</a>
```

Porque si luego cambiamos la ruta, el enlace seguirá funcionando mientras mantengamos el mismo `name`.

13. Ejemplo completo de vistas para el blog

En `posts/views.py`:

CÓDIGO

```
from django.shortcuts import render

def inicio(request):
    return render(request, "posts/inicio.html")


def lista_posts(request):
    posts = [
        {"id": 1, "titulo": "Mi primer post", "autor": "Ana"},
        {"id": 2, "titulo": "Aprendiendo Django", "autor": "Luis"},
        {"id": 3, "titulo": "Vistas basadas en funciones", "autor": "Marta"},
    ]

    return render(request, "posts/lista_posts.html", {"posts": posts})


def detalle_post(request, post_id):
    post = {
        "id": post_id,
        "titulo": f"Post número {post_id}",
        "contenido": "Este es el contenido de ejemplo del post.",
        "autor": "Ana",
    }

    return render(request, "posts/detalle_post.html", {"post": post})


def contacto(request):
    return render(request, "posts/contacto.html")
```

En `posts/urls.py`:

CÓDIGO

```
from django.urls import path
from . import views

urlpatterns = [
    path("", views.inicio, name="inicio"),
    path("posts/", views.lista_posts, name="lista_posts"),
    path("posts/<int:post_id>/", views.detalle_post, name="detalle_post"),
    path("contacto/", views.contacto, name="contacto"),
]
```

14. Template de listado con enlaces al detalle

En `posts/templates/posts/lista_posts.html`:

CÓDIGO

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8" />
    <title>Posts</title>
</head>
<body>
    <h1>Listado de posts</h1>

    <ul>
        {% for post in posts %}
            <li>
                <a href="{% url 'detalle_post' post.id %}">
                    {{ post.titulo }}
                </a>
                - {{ post.autor }}
            </li>
        {% endfor %}
    </ul>

    <a href="{% url 'lista_posts' %}">Volver al inicio</a>
</body>
</html>
```

La parte más importante es:

CÓDIGO

```
{% url 'detalle_post' post.id %}
```

Como la ruta `detalle_post` necesita un `post_id`, le pasamos `post.id`.

15. Errores comunes de principiantes

Olvidar el parámetro `request`

Incorrecto:

CÓDIGO

```
def inicio():
    return render("posts/inicio.html")
```

Correcto:

CÓDIGO

```
def inicio(request):
    return render(request, "posts/inicio.html")
```

Olvidar el `return`

Incorrecto:

CÓDIGO

```
def inicio(request):
    render(request, "posts/inicio.html")
```

Correcto:

CÓDIGO

```
def inicio(request):
    return render(request, "posts/inicio.html")
```

Escribir mal el nombre del template

Si el archivo está en:

CÓDIGO

```
posts/templates/posts/inicio.html
```

la vista debe usar:

CÓDIGO

```
return render(request, "posts/inicio.html")
```

No:

CÓDIGO

```
return render(request, "inicio.html")
```

Escribir mal el nombre de la ruta

Si en `urls.py` tenemos:

CÓDIGO

```
path("posts/", views.lista_posts, name="lista_posts")
```

en el template debemos usar:

CÓDIGO

```
{% url 'lista_posts' %}
```

No:

CÓDIGO

```
{% url 'posts' %}
```

Olvidar importar las vistas

Incorrecto:

CÓDIGO

```
from django.urls import path

urlpatterns = [
    path("", views.inicio, name="inicio"),
]
```

Correcto:

CÓDIGO

```
from django.urls import path
from . import views

urlpatterns = [
    path("", views.inicio, name="inicio"),
]
```

16. Resumen

En esta unidad aprendimos que:

- Una Vista Basada en Función es una función de Python.
- Toda vista recibe un parámetro `request`.
- Toda vista debe devolver una respuesta.
- `HttpResponse` permite devolver texto simple.
- `render` permite devolver un template HTML.
- Las vistas pueden enviar datos al template usando un diccionario.
- Las URLs pueden enviar parámetros a las vistas.
- El template usa esos datos para mostrar contenido dinámico.

Las vistas son una pieza central de Django: conectan la petición del usuario con la lógica del proyecto y la respuesta final que se muestra en el navegador.

Actividad práctica

Crear las vistas principales del blog usando FBV.

El proyecto debe tener estas páginas:

1. Inicio.
2. Listado de posts.
3. Detalle de un post.
4. Contacto.

Crear las vistas en `posts/views.py`:

CÓDIGO

```
from django.shortcuts import render

def inicio(request):
    return render(request, "posts/inicio.html")


def lista_posts(request):
    posts = [
        {"id": 1, "titulo": "Primer post", "autor": "Ana"},
        {"id": 2, "titulo": "Segundo post", "autor": "Luis"},
    ]

    return render(request, "posts/lista_posts.html", {"posts": posts})


def detalle_post(request, post_id):
    post = {
        "id": post_id,
        "titulo": f"Post número {post_id}",
        "contenido": "Contenido de ejemplo del post.",
        "autor": "Ana",
    }

    return render(request, "posts/detalle_post.html", {"post": post})


def contacto(request):
    return render(request, "posts/contacto.html")
```

Crear las rutas en `posts/urls.py`:

CÓDIGO

```
from django.urls import path
from . import views

urlpatterns = [
    path("", views.inicio, name="inicio"),
    path("posts/", views.lista_posts, name="lista_posts"),
    path("posts/<int:post_id>/", views.detalle_post, name="detalle_post"),
    path("contacto/", views.contacto, name="contacto"),
]
```

Crear los templates correspondientes:

CÓDIGO

```
posts/templates/posts/inicio.html
posts/templates/posts/lista_posts.html
posts/templates/posts/detalle_post.html
posts/templates/posts/contacto.html
```

Pregunta para reflexionar:

¿Qué diferencia hay entre escribir la lógica en `views.py` y escribir el contenido visual en un archivo `.html`?

Construir plantillas reutilizables con base.html e herencia

Aprende a crear una estructura HTML maestra y reutilizable para tu blog, evitando la duplicidad de código mediante el potente sistema de herencia de Django.

Hasta ahora, cada página de nuestro blog podía tener su propio archivo HTML completo.

Por ejemplo:

CÓDIGO

```
inicio.html
lista_posts.html
detalle_post.html
contacto.html
```

El problema aparece cuando todas esas páginas repiten la misma estructura:

CÓDIGO

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Mi Blog</title>
</head>
<body>
    <header>
        <h1>Mi Blog</h1>
    </header>

    <main>
        Contenido de la página
    </main>

    <footer>
        <p>Creado con Django</p>
    </footer>
</body>
</html>
```

Si tenemos 4 páginas, quizá no parece grave. Pero si el blog crece y tenemos 50 páginas, repetir esa estructura se vuelve un problema.

Si queremos cambiar el menú, el título del sitio o el pie de página, tendríamos que modificar muchos archivos.

Para evitar eso, Django nos permite crear una plantilla base llamada normalmente `base.html`.

La idea es simple:

- `base.html` contiene la estructura común del sitio.
- Las demás plantillas heredan de `base.html`.
- Cada página solo completa la parte que cambia.

Esto nos permite aplicar una idea muy importante en programación:

CÓDIGO

```
DRY: Don't Repeat Yourself
```

Es decir:

CÓDIGO

```
No te repitas.
```

1. ¿Qué es una plantilla base?

Una plantilla base es un archivo HTML principal que funciona como esqueleto del sitio.

En nuestro blog, ese archivo puede tener:

- estructura general del HTML,
- título de la página,
- menú de navegación,
- contenido principal,
- pie de página.

Por ejemplo:

CÓDIGO

```
base.html
```

sería el molde general.

Luego, las demás páginas usan ese molde y reemplazan solamente algunas partes.

2. Estructura recomendada

Dentro de nuestra app `posts`, podemos organizar los templates así:

CÓDIGO

```
posts/
├── templates/
│   └── posts/
│       ├── base.html
│       ├── inicio.html
│       ├── lista_posts.html
│       ├── detalle_post.html
│       └── contacto.html
├── urls.py
├── views.py
```

El archivo más importante de esta unidad será:

CÓDIGO

```
posts/templates/posts/base.html
```

3. Crear el archivo `base.html`

En `posts/templates/posts/base.html`:

CÓDIGO

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>{% block title %}Mi Blog{% endblock %}</title>
</head>
<body>
    <header>
        <h1>Mi Blog</h1>

        <nav>
            <a href="{% url 'inicio' %}">Inicio</a>
            <a href="{% url 'lista_posts' %}">Posts</a>
            <a href="{% url 'contacto' %}">Contacto</a>
        </nav>
    </header>

    <main>
        {% block content %}
        {% endblock %}
    </main>

    <footer>
        <p>Creado con Django</p>
    </footer>
</body>
</html>
```

Este archivo tiene HTML normal, pero también tiene etiquetas especiales de Django.

Las más importantes son:

CÓDIGO

```
{% block title %}
{% endblock %}
```

y:

CÓDIGO

```
{% block content %}
{% endblock %}
```

4. ¿Qué es un bloque?

Un bloque es un espacio reservado que las plantillas hijas pueden completar.

Por ejemplo:

CÓDIGO

```
{% block content %}
{% endblock %}
```

significa:

CÓDIGO

```
Acá va a ir el contenido específico de cada página.
```

Entonces `base.html` define la estructura general, pero deja huecos para que cada página coloque su propio contenido.

5. Crear una plantilla hija con `{% extends %}`

Ahora podemos crear una página que herede de `base.html`.

Por ejemplo, en `posts/templates/posts/inicio.html`:

CÓDIGO

```
{% extends "posts/base.html" %}

{% block title %}Inicio | Mi Blog{% endblock %}

{% block content %}
    <h2>Bienvenido a mi blog</h2>
    <p>Este es el inicio del proyecto.</p>
{% endblock %}
```

La primera línea es clave:

CÓDIGO

```
{% extends "posts/base.html" %}
```

Eso le dice a Django:

CÓDIGO

```
Esta plantilla usa como base el archivo posts/base.html.
```

Luego completamos los bloques definidos en `base.html`.

6. Cómo se combina todo

Django toma esta plantilla hija:

CÓDIGO

```
{% extends "posts/base.html" %}

{% block title %}Inicio | Mi Blog{% endblock %}

{% block content %}
    <h2>Bienvenido a mi blog</h2>
    <p>Este es el inicio del proyecto.</p>
{% endblock %}
```

y la inserta dentro de `base.html`.

El resultado final sería algo parecido a esto:

CÓDIGO

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Inicio | Mi Blog</title>
</head>
<body>
    <header>
        <h1>Mi Blog</h1>

        <nav>
            <a href="/">Inicio</a>
            <a href="/posts/">Posts</a>
            <a href="/contacto/">Contacto</a>
        </nav>
    </header>

    <main>
        <h2>Bienvenido a mi blog</h2>
        <p>Este es el inicio del proyecto.</p>
    </main>

    <footer>
        <p>Creado con Django</p>
    </footer>
</body>
</html>
```

El navegador no ve los bloques de Django. El navegador recibe HTML terminado.

7. Crear la plantilla de listado de posts

En `posts/templates/posts/lista_posts.html`:

CÓDIGO

```
{% extends "posts/base.html" %}

{% block title %}Posts | Mi Blog{% endblock %}

{% block content %}
    <h2>Listado de posts</h2>

    <ul>
        {% for post in posts %}
            <li>
                <a href="{% url 'detalle_post' post.id %}">
                    {{ post.titulo }}
                </a>
                - {{ post.autor }}
            </li>
        {% endfor %}
    </ul>
{% endblock %}
```

Esta plantilla no necesita repetir:

CÓDIGO

```
<!DOCTYPE html>
<html>
<head>
<body>
<header>
<footer>
```

Todo eso ya está en `base.html`.

La plantilla hija solo se ocupa de mostrar el contenido específico de esta página.

8. Crear la plantilla de detalle de post

En `posts/templates/posts/detalle_post.html`:

CÓDIGO

```
{% extends "posts/base.html" %}

{% block title %}{{ post.titulo }} | Mi Blog{% endblock %}

{% block content %}
    <article>
        <h2>{{ post.titulo }}</h2>

        <p><strong>Autor:</strong> {{ post.autor }}</p>

        <p>{{ post.contenido }}</p>
    </article>

    <a href="{% url 'lista_posts' %}">Volver al listado</a>
{% endblock %}
```

Acá usamos datos enviados desde la vista:

CÓDIGO

```
{{ post.titulo }}
{{ post.autor }}
{{ post.contenido }}
```

Y también usamos el bloque `title` para que el título del navegador cambie según el post.

9. Crear la plantilla de contacto

En `posts/templates/posts/contacto.html`:

CÓDIGO

```
{% extends "posts/base.html" %}

{% block title %}Contacto | Mi Blog{% endblock %}

{% block content %}
    <h2>Contacto</h2>

    <p>Podés escribirme a:</p>

    <p>contacto@miblog.com</p>
{% endblock %}
```

Nuevamente, la página solo completa los bloques necesarios.

10. Conectar las vistas

En `posts/views.py`:

CÓDIGO

```
from django.shortcuts import render

def inicio(request):
    return render(request, "posts/inicio.html")


def lista_posts(request):
    posts = [
        {
            "id": 1,
            "titulo": "Mi primer post",
            "autor": "Ana",
            "contenido": "Contenido del primer post.",
        },
        {
            "id": 2,
            "titulo": "Aprendiendo Django",
            "autor": "Luis",
            "contenido": "Contenido del segundo post.",
        },
        {
            "id": 3,
            "titulo": "Templates reutilizables",
            "autor": "Marta",
            "contenido": "Contenido del tercer post.",
        },
    ]

    return render(request, "posts/lista_posts.html", {"posts": posts})


def detalle_post(request, post_id):
    post = {
        "id": post_id,
        "titulo": f"Post número {post_id}",
        "autor": "Ana",
        "contenido": "Este es el contenido de ejemplo del post.",
    }

    return render(request, "posts/detalle_post.html", {"post": post})


def contacto(request):
    return render(request, "posts/contacto.html")
```

11. Conectar las rutas

En `posts/urls.py`:

CÓDIGO

```
from django.urls import path
from . import views

urlpatterns = [
    path("", views.inicio, name="inicio"),
    path("posts/", views.lista_posts, name="lista_posts"),
    path("posts/<int:post_id>/", views.detalle_post, name="detalle_post"),
    path("contacto/", views.contacto, name="contacto"),
]
```

Cada ruta ejecuta una vista.

Cada vista devuelve un template.

Cada template hereda de `base.html`.

12. El uso de `{% url %}` en `base.html`

Dentro del menú de navegación usamos:

CÓDIGO

```
<nav>
    <a href="{% url 'inicio' %}">Inicio</a>
    <a href="{% url 'lista_posts' %}">Posts</a>
    <a href="{% url 'contacto' %}">Contacto</a>
</nav>
```

Esto es mejor que escribir las rutas manualmente:

CÓDIGO

```
<a href="/">Inicio</a>
<a href="/posts/">Posts</a>
<a href="/contacto/">Contacto</a>
```

Porque si más adelante cambiamos una URL en `urls.py`, los enlaces pueden seguir funcionando mientras mantengamos el mismo `name`.

Por ejemplo, si cambiamos:

CÓDIGO

```
path("posts/", views.lista_posts, name="lista_posts")
```

por:

CÓDIGO

```
path("publicaciones/", views.lista_posts, name="lista_posts")
```

el enlace:

CÓDIGO

```
{% url 'lista_posts' %}
```

seguirá funcionando.

13. Agregar estilos básicos en `base.html`

También podemos agregar CSS directamente en `base.html` para que todas las páginas compartan el mismo estilo.

Por ejemplo:

CÓDIGO

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>{% block title %}Mi Blog{% endblock %}</title>

    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 0;
            background-color: #f4f4f4;
        }

        header {
            background-color: #222;
            color: white;
            padding: 20px;
        }

        nav a {
            color: white;
            margin-right: 15px;
            text-decoration: none;
        }

        main {
            padding: 20px;
            background-color: white;
            max-width: 800px;
            margin: 20px auto;
        }

        footer {
            text-align: center;
            padding: 15px;
            background-color: #222;
            color: white;
        }
    </style>
</head>
<body>
    <header>
        <h1>Mi Blog</h1>

        <nav>
            <a href="{% url 'inicio' %}">Inicio</a>
            <a href="{% url 'lista_posts' %}">Posts</a>
            <a href="{% url 'contacto' %}">Contacto</a>
        </nav>
    </header>
```

```
    <main>
        {% block content %}
        {% endblock %}
    </main>

    <footer>
        <p>Creado con Django</p>
    </footer>
</body>
</html>
```

Más adelante, lo ideal será mover estos estilos a un archivo CSS separado. Pero para este primer ejemplo, tenerlos dentro de `base.html` nos ayuda a entender cómo la estructura se reutiliza en todas las páginas.

14. Ventajas de usar herencia de templates

Usar `base.html` tiene varias ventajas:

Evita repetir código

No necesitamos copiar el mismo menú y footer en cada página.

Facilita los cambios

Si queremos cambiar el menú, lo hacemos una sola vez en `base.html`.

Mejora la organización

Las plantillas hijas quedan más cortas y más fáciles de leer.

Mantiene consistencia visual

Todas las páginas comparten la misma estructura.

15. Errores comunes

Olvidar `{% extends %}`

Incorrecto:

CÓDIGO

```
{% block content %}
    <h2>Inicio</h2>
{% endblock %}
```

Si no usamos `{% extends "posts/base.html" %}`, Django no sabe de qué plantilla base debe heredar.

Correcto:

CÓDIGO

```
{% extends "posts/base.html" %}

{% block content %}
    <h2>Inicio</h2>
{% endblock %}
```

Escribir mal el nombre del bloque

Si en `base.html` tenemos:

CÓDIGO

```
{% block content %}
{% endblock %}
```

en la plantilla hija debemos usar el mismo nombre:

CÓDIGO

```
{% block content %}
    <p>Contenido de la página</p>
{% endblock %}
```

Si escribimos otro nombre, por ejemplo:

CÓDIGO

```
{% block contenido %}
    <p>Contenido de la página</p>
{% endblock %}
```

Django no lo insertará en el lugar esperado.

Escribir mal la ruta del template base

Incorrecto:

CÓDIGO

```
{% extends "base.html" %}
```

Correcto, si el archivo está dentro de `templates/posts/` :

CÓDIGO

```
{% extends "posts/base.html" %}
```

Olvidar cerrar el bloque

Incorrecto:

CÓDIGO

```
{% block content %}
    <h2>Inicio</h2>
```

Correcto:

CÓDIGO

```
{% block content %}
    <h2>Inicio</h2>
{% endblock %}
```

Usar un nombre de ruta que no existe

Si en `base.html` usamos:

CÓDIGO

```
{% url 'contacto' %}
```

debe existir una ruta con ese nombre:

CÓDIGO

```
path("contacto/", views.contacto, name="contacto")
```

Si el nombre no coincide, Django mostrará un error.

16. Resumen

En esta unidad aprendimos que:

- `base.html` funciona como plantilla principal del sitio.
- Las plantillas hijas heredan de `base.html` usando `{% extends %}`.
- Los bloques `{% block %}` y `{% endblock %}` permiten definir partes reemplazables.
- El bloque `title` permite cambiar el título de cada página.
- El bloque `content` permite insertar el contenido principal.
- La etiqueta `{% url %}` permite crear enlaces usando el nombre de las rutas.
- La herencia de templates evita repetir código y facilita el mantenimiento del proyecto.

Gracias a este sistema, nuestro blog empieza a parecer una web más organizada y profesional.

Actividad práctica

Crear una estructura de templates reutilizable para el blog.

El proyecto debe incluir:

CÓDIGO

```
posts/templates/posts/base.html
posts/templates/posts/inicio.html
posts/templates/posts/lista_posts.html
posts/templates/posts/detalle_post.html
posts/templates/posts/contacto.html
```

El archivo `base.html` debe contener:

- estructura HTML general,
- título dinámico con bloque `title`,
- menú de navegación,
- bloque `content`,
- footer.

Las plantillas hijas deben:

- heredar de `base.html`,
- completar el bloque `title`,
- completar el bloque `content`.

Ejemplo mínimo de plantilla hija:

CÓDIGO

```
{% extends "posts/base.html" %}

{% block title %}Inicio | Mi Blog{% endblock %}

{% block content %}
    <h2>Bienvenido a mi blog</h2>
    <p>Esta página reutiliza la estructura de base.html.</p>
{% endblock %}
```

Pregunta para reflexionar:

Si quisieras agregar una nueva página llamada “Acerca de mí”, ¿qué archivos deberías crear o modificar para que también use `base.html`?

Cargar estilos y scripts estáticos en el sitio

Aprende a organizar y enlazar archivos CSS y JavaScript en tu proyecto Django para darle estilo y funcionalidad a tu blog.

Hasta ahora, nuestro blog ya tiene páginas, vistas y plantillas reutilizables con `base.html`.

Pero todavía se ve bastante simple.

Para que una web tenga diseño, colores, tipografías, espacios, botones e interacción, necesitamos usar archivos externos como:

- CSS
- JavaScript
- imágenes

En Django, este tipo de archivos se conocen como **archivos estáticos**.

1. ¿Qué es un archivo estático?

Un archivo estático es un archivo que Django entrega al navegador tal como está guardado.

A diferencia de un template HTML, que Django puede completar con datos dinámicos, un archivo estático no cambia según el usuario.

Por ejemplo:

CÓDIGO

```
style.css
main.js
logo.png
fondo.jpg
```

Estos archivos se envían directamente al navegador.

Los archivos estáticos más comunes son:

CSS

Sirve para definir el diseño visual del sitio.

Por ejemplo:

CÓDIGO

```
body {
    background-color: #f4f4f4;
    font-family: Arial, sans-serif;
}
```

JavaScript

Sirve para agregar interacción.

Por ejemplo:

CÓDIGO

```
console.log("El sitio cargó correctamente");
```

Imágenes

Sirven para mostrar logos, fondos, íconos o imágenes decorativas.

Por ejemplo:

CÓDIGO

```
logo.png
```

2. Diferencia entre templates y archivos estáticos

En Django, es importante no mezclar carpetas.

Los archivos HTML van en:

CÓDIGO

```
templates/
```

Los archivos CSS, JavaScript e imágenes van en:

CÓDIGO

```
static/
```

Por ejemplo:

CÓDIGO

```
posts/
├── templates/
│   └── posts/
│       ├── base.html
│       ├── inicio.html
│       └── lista_posts.html
│
├── static/
│   └── posts/
│       ├── css/
│       │   └── style.css
│       ├── js/
│       │   └── main.js
│       └── img/
│           └── logo.png
│
├── views.py
├── urls.py
```

Esta separación ayuda a mantener el proyecto ordenado.

3. ¿Por qué repetimos el nombre de la app?

Dentro de `static`, solemos crear otra carpeta con el nombre de la app.

Por ejemplo:

CÓDIGO

```
posts/static/posts/css/style.css
```

Esto puede parecer repetido, pero es una buena práctica.

La razón es que un proyecto Django puede tener muchas apps.

Por ejemplo:

CÓDIGO

```
posts/
usuarios/
comentarios/
```

Cada app podría tener su propio archivo llamado:

CÓDIGO

```
style.css
```

Si todas usaran directamente:

CÓDIGO

```
static/css/style.css
```

podrían aparecer conflictos.

Por eso usamos:

CÓDIGO

```
static/posts/css/style.css
static/usuarios/css/style.css
static/comentarios/css/style.css
```

Así Django puede distinguir claramente a qué app pertenece cada archivo.

4. Crear la carpeta `static`

Dentro de la app `posts`, creamos esta estructura:

CÓDIGO

```
posts/
└── static/
    └── posts/
        ├── css/
        │   └── style.css
        ├── js/
        │   └── main.js
        └── img/
            └── logo.png
```

El archivo principal de estilos será:

CÓDIGO

```
posts/static/posts/css/style.css
```

El archivo principal de JavaScript será:

CÓDIGO

```
posts/static/posts/js/main.js
```

5. Cargar archivos estáticos en un template

Para usar archivos estáticos dentro de un template, primero debemos cargar las herramientas de Django.

Esto se hace con:

CÓDIGO

```
{% load static %}
```

Normalmente lo colocamos al comienzo del archivo HTML.

Por ejemplo, en `base.html`:

CÓDIGO

```
{% load static %}

<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>{% block title %}Mi Blog{% endblock %}</title>

    <link rel="stylesheet" href="{% static 'posts/css/style.css' %}">
</head>
<body>
    <h1>Mi Blog</h1>
</body>
</html>
```

La línea clave es:

CÓDIGO

```
<link rel="stylesheet" href="{% static 'posts/css/style.css' %}">
```

Eso le dice a Django:

CÓDIGO

```
Buscá el archivo posts/css/style.css dentro de las carpetas static.
```

6. ¿Por qué no escribir la ruta manualmente?

Podríamos pensar en escribir algo así:

CÓDIGO

```
<link rel="stylesheet" href="/static/posts/css/style.css">
```

Pero no es lo ideal.

La forma recomendada es usar:

CÓDIGO

```
<link rel="stylesheet" href="{% static 'posts/css/style.css' %}">
```

¿Por qué?

Porque Django se encarga de generar la ruta correcta.

Si en el futuro el proyecto cambia de servidor, configuración o estructura, la etiqueta `{% static %}` ayuda a que los enlaces a archivos estáticos sigan funcionando correctamente.

7. Ejemplo completo de `base.html` con CSS

En `posts/templates/posts/base.html`:

CÓDIGO

```
{% load static %}

<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>{% block title %}Mi Blog{% endblock %}</title>

    <link rel="stylesheet" href="{% static 'posts/css/style.css' %}">
</head>
<body>
    <header class="site-header">
        <h1>Mi Blog</h1>

        <nav class="site-nav">
            <a href="{% url 'inicio' %}">Inicio</a>
            <a href="{% url 'lista_posts' %}">Posts</a>
            <a href="{% url 'contacto' %}">Contacto</a>
        </nav>
    </header>

    <main class="site-main">
        {% block content %}
        {% endblock %}
    </main>

    <footer class="site-footer">
        <p>Creado con Django</p>
    </footer>
</body>
</html>
```

Ahora el archivo `base.html` carga el CSS desde:

CÓDIGO

```
posts/static/posts/css/style.css
```

8. Crear estilos en `style.css`

En `posts/static/posts/css/style.css`:

CÓDIGO

```
body {
    margin: 0;
    font-family: Arial, sans-serif;
    background-color: #f4f4f4;
    color: #222;
}

.site-header {
    background-color: #222;
    color: white;
    padding: 20px;
}

.site-header h1 {
    margin: 0 0 10px 0;
}

.site-nav a {
    color: white;
    text-decoration: none;
    margin-right: 15px;
}

.site-nav a:hover {
    text-decoration: underline;
}

.site-main {
    max-width: 800px;
    margin: 30px auto;
    padding: 20px;
    background-color: white;
}

.site-footer {
    text-align: center;
    padding: 20px;
    background-color: #222;
    color: white;
}
```

Con esto, todas las páginas que heredan de `base.html` van a compartir el mismo diseño.

9. Usar una imagen estática

Supongamos que tenemos un logo en esta ubicación:

CÓDIGO

```
posts/static/posts/img/logo.png
```

Podemos mostrarlo en `base.html` de esta forma:

CÓDIGO

```
<img src="{% static 'posts/img/logo.png' %}" alt="Logo del blog">
```

Por ejemplo:

CÓDIGO

```
<header class="site-header">
    <img src="{% static 'posts/img/logo.png' %}" alt="Logo del blog" class="logo">

    <h1>Mi Blog</h1>

    <nav class="site-nav">
        <a href="{% url 'inicio' %}">Inicio</a>
        <a href="{% url 'lista_posts' %}">Posts</a>
        <a href="{% url 'contacto' %}">Contacto</a>
    </nav>
</header>
```

Y en `style.css`:

CÓDIGO

```
.logo {
    width: 80px;
    height: auto;
}
```

10. Cargar JavaScript

También podemos cargar archivos JavaScript.

Creamos el archivo:

CÓDIGO

```
posts/static/posts/js/main.js
```

Dentro escribimos:

CÓDIGO

```
console.log("El blog cargó correctamente");

const titulo = document.querySelector("h1");

if (titulo) {
    titulo.addEventListener("click", function () {
        alert("Hiciste clic en el título del blog");
    });
}
```

Ahora lo conectamos en `base.html`, antes de cerrar el `body`:

CÓDIGO

```
<script src="{% static 'posts/js/main.js' %}"></script>
```

Ejemplo:

CÓDIGO

```
<body>
    <header class="site-header">
        <h1>Mi Blog</h1>

        <nav class="site-nav">
            <a href="{% url 'inicio' %}">Inicio</a>
            <a href="{% url 'lista_posts' %}">Posts</a>
            <a href="{% url 'contacto' %}">Contacto</a>
        </nav>
    </header>

    <main class="site-main">
        {% block content %}
        {% endblock %}
    </main>

    <footer class="site-footer">
        <p>Creado con Django</p>
    </footer>

    <script src="{% static 'posts/js/main.js' %}"></script>
</body>
```

De esta manera, todas las páginas que heredan de `base.html` también cargarán el mismo JavaScript.

11. Ejemplo completo de `base.html` con CSS, imagen y JavaScript

CÓDIGO

```
{% load static %}

<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>{% block title %}Mi Blog{% endblock %}</title>

    <link rel="stylesheet" href="{% static 'posts/css/style.css' %}">
</head>
<body>
    <header class="site-header">
        <img src="{% static 'posts/img/logo.png' %}" alt="Logo del blog"
class="logo">

        <h1>Mi Blog</h1>

        <nav class="site-nav">
            <a href="{% url 'inicio' %}">Inicio</a>
            <a href="{% url 'lista_posts' %}">Posts</a>
            <a href="{% url 'contacto' %}">Contacto</a>
        </nav>
    </header>

    <main class="site-main">
        {% block content %}
        {% endblock %}
    </main>

    <footer class="site-footer">
        <p>Creado con Django</p>
    </footer>

    <script src="{% static 'posts/js/main.js' %}"></script>
</body>
</html>
```

12. Verificar que `django.contrib.staticfiles` esté activo

Para trabajar con archivos estáticos en desarrollo, Django usa una app interna llamada:

CÓDIGO

```
django.contrib.staticfiles
```

Normalmente ya viene incluida cuando creamos el proyecto.

Podemos verificarlo en `settings.py`:

CÓDIGO

```
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",

    "posts",
]
```

También debemos tener configurado:

CÓDIGO

```
STATIC_URL = "static/"
```

En proyectos nuevos de Django, esta configuración suele venir creada por defecto.

13. Errores comunes

Olvidar `{% load static %}`

Incorrecto:

CÓDIGO

```
<link rel="stylesheet" href="{% static 'posts/css/style.css' %}">
```

Correcto:

CÓDIGO

```
{% load static %}

<link rel="stylesheet" href="{% static 'posts/css/style.css' %}">
```

Si no cargamos `static`, Django no entiende la etiqueta.

Poner el CSS dentro de `templates`

Incorrecto:

CÓDIGO

```
posts/templates/posts/css/style.css
```

Correcto:

CÓDIGO

```
posts/static/posts/css/style.css
```

Los templates son para HTML.

Los archivos CSS, JS e imágenes van en `static`.

Escribir mal la ruta del archivo

Si el archivo está en:

CÓDIGO

```
posts/static/posts/css/style.css
```

en el template debemos escribir:

CÓDIGO

```
<link rel="stylesheet" href="{% static 'posts/css/style.css' %}">
```

No debemos incluir la primera carpeta `static` en la etiqueta.

Incorrecto:

CÓDIGO

```
<link rel="stylesheet" href="{% static 'static/posts/css/style.css' %}">
```

Correcto:

CÓDIGO

```
<link rel="stylesheet" href="{% static 'posts/css/style.css' %}">
```

No guardar los cambios

Puede parecer obvio, pero es un error común: modificar `style.css`, volver al navegador y no ver cambios porque el archivo no fue guardado.

El navegador muestra una versión vieja del CSS

A veces el navegador guarda una copia anterior del CSS.

Si hicimos cambios y no se ven, podemos probar recargar la página con una recarga forzada:

CÓDIGO

```
Ctrl + F5
```

No reiniciar el servidor

Si agregamos carpetas nuevas y Django no parece detectarlas, podemos detener el servidor y volver a iniciarlo.

Esto ayuda a que Django reconozca mejor la nueva estructura del proyecto.

14. Resumen

En esta unidad aprendimos que:

- Los archivos estáticos son archivos que Django entrega sin modificar.
- Los archivos CSS, JavaScript e imágenes van dentro de una carpeta `static`.
- Es buena práctica usar una estructura como `static/nombre_app/`.
- Para usar archivos estáticos en un template debemos escribir `{% load static %}`.
- Para referenciar un archivo usamos `{% static 'ruta/del/archivo' %}`.
- El CSS se carga con la etiqueta `<link>`.
- El JavaScript se carga con la etiqueta `<script>`.
- Las imágenes se cargan con la etiqueta `<img>`.
- `base.html` es el mejor lugar para cargar los archivos comunes del sitio.

Los archivos estáticos nos permiten transformar una página básica en una interfaz más clara, atractiva y fácil de usar.

Actividad práctica

Agregar archivos estáticos al blog.

Crear la siguiente estructura dentro de la app `posts`:

CÓDIGO

```
posts/
└── static/
    └── posts/
        ├── css/
        │   └── style.css
        ├── js/
        │   └── main.js
        └── img/
            └── logo.png
```

Modificar `base.html` para cargar:

- el archivo CSS,
- el archivo JavaScript,
- una imagen de logo.

Ejemplo mínimo:

CÓDIGO

```
{% load static %}

<link rel="stylesheet" href="{% static 'posts/css/style.css' %}">

<img src="{% static 'posts/img/logo.png' %}" alt="Logo del blog">

<script src="{% static 'posts/js/main.js' %}"></script>
```

Luego, en `style.css`, agregar estilos para:

- el color de fondo,
- el encabezado,
- el menú de navegación,
- el contenido principal,
- el pie de página.

Pregunta para reflexionar:

¿Por qué es mejor cargar un archivo CSS desde `static` en lugar de escribir todos los estilos directamente dentro de cada template?

PRE-ENTREGA DEL PROYECTO

Actividad evaluable · parte del proyecto integrador

Preentrega 8: Primer sitio Django con URLs, Templates y Static

Implementar el flujo Ruta → Vista → Template → Estático en Django, creando páginas HTML con herencia de templates y estilos CSS para transformar el blog en un sitio visual y navegable.

Preentrega 8: Transformando tu Proyecto Django en un Sitio Real

¡Llegó el momento de que tu blog empiece a verse como una página web real!

En el módulo anterior, creaste la base de tu proyecto Django: configuraste el entorno, instalaste Django, creaste el proyecto, creaste la app `posts` y ejecutaste el servidor local.

Ahora vas a avanzar un paso más: dejar atrás las respuestas simples de texto plano en el navegador para construir páginas HTML organizadas, conectadas con rutas, vistas y archivos estáticos.

Propósito

El objetivo de esta práctica es implementar el flujo completo de Django:

CÓDIGO

```
Ruta -> Vista -> Template -> Estático
```

Vas a construir una primera interfaz visual para tu blog usando:

- URLs del proyecto.
- URLs de la app.
- Views basadas en funciones.
- Templates HTML.
- Herencia de templates.
- Archivos estáticos.
- CSS personalizado.

Este checkpoint no busca trabajar todavía con base de datos, modelos ni formularios. El foco está en entender cómo Django recibe una URL, ejecuta una view, renderiza un template y carga archivos estáticos como CSS.

Contexto de la cadena de entregables

En el **Módulo 1**, configuraste tu entorno y ejecutaste tus primeros scripts en Python.

En el **Módulo 2**, modelaste la estructura de datos del blog usando colecciones.

En el **Módulo 3**, construiste un sistema interactivo por consola.

En el **Módulo 4**, refactorizaste el sistema usando funciones y validaciones.

En el **Módulo 5**, organizaste el proyecto usando módulos y paquetes.

En el **Módulo 6**, aplicaste Programación Orientada a Objetos y persistencia con JSON.

En el **Módulo 7**, creaste la base inicial del proyecto Django.

Ahora, en el **Módulo 8**, vas a transformar esa base Django en un sitio navegable con HTML y CSS.

Objetivo de aprendizaje de la preentrega

El objetivo de esta preentrega es consolidar el flujo básico de una aplicación web en Django.

Vas a practicar:

Rutas

Definir URLs para acceder a distintas páginas del sitio.

Views

Crear funciones en `views.py` que reciban una petición y devuelvan una respuesta usando `render()`.

Templates

Crear archivos HTML separados del código Python.

Herencia de templates

Crear un layout base reutilizable para evitar repetir la estructura general del sitio.

Archivos estáticos

Cargar CSS desde la carpeta `static/`.

Integración

Conectar proyecto, app, views, templates y archivos estáticos en un flujo completo.

¿Qué construir?

Debés continuar trabajando sobre el proyecto Django creado en el módulo anterior.

No crees un proyecto nuevo.

El objetivo es construir una primera versión visual del blog con al menos dos páginas:

- Página de inicio.
- Página “Acerca de”.

La estructura sugerida es:

CÓDIGO

```
blog_django/
│
├── manage.py
├── README.md
├── requirements.txt
│
├── blog_project/
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
│
└── posts/
    ├── __init__.py
    ├── admin.py
    ├── apps.py
    ├── models.py
    ├── tests.py
    ├── urls.py
    ├── views.py
    │
    ├── templates/
    │   └── posts/
    │       ├── base.html
    │       ├── inicio.html
    │       ├── lista_posts.html
    │       └── acerca.html
    │
    └── static/
        └── posts/
            └── css/
                └── estilos.css
```

Requisitos técnicos

1. Continuidad del proyecto

Debés continuar trabajando sobre el proyecto Django creado en el Módulo 7.

El proyecto debe seguir funcionando con:

CÓDIGO

```
python manage.py runserver
```

La app `posts` debe continuar registrada en `INSTALLED_APPS`.

2. Preparación de estructuras

Dentro de la app `posts`, creá las siguientes carpetas:

CÓDIGO

```
posts/templates/posts/
```

y:

CÓDIGO

```
posts/static/posts/css/
```

Esta estructura es importante para que Django pueda localizar correctamente los templates y los archivos estáticos de la app.

3. Creación del layout base

Creá un archivo llamado:

CÓDIGO

```
posts/templates/posts/base.html
```

Este archivo debe funcionar como plantilla principal del sitio.

Debe incluir:

- Estructura HTML5.
- Etiqueta `{% load static %}` al inicio.
- Enlace al archivo CSS.
- Un menú de navegación.
- Un bloque `{% block content %}` para que las páginas hijas puedan insertar su contenido.

El objetivo de `base.html` es evitar repetir el mismo encabezado, menú y estructura en todas las páginas.

4. Implementación de herencia

Creá los siguientes templates:

CÓDIGO

```
posts/templates/posts/inicio.html
```

CÓDIGO

```
posts/templates/posts/acerca.html
```

Ambos deben extender de la plantilla base usando:

CÓDIGO

```
{% extends 'posts/base.html' %}
```

En `inicio.html`, agregá:

- Un título principal.
- Una descripción de bienvenida al blog.

En `acerca.html`, agregá:

- Una breve reseña sobre el autor del blog.
- Una explicación simple sobre el propósito del sitio.

5. Lógica en views

En el archivo:

CÓDIGO

```
posts/views.py
```

definí dos funciones basadas en views:

- Una view para la página de inicio.
- Una view para la página acerca.

Cada función debe usar `render()` para devolver el template correspondiente.

Por ejemplo, la view de inicio debe renderizar:

CÓDIGO

```
posts/inicio.html
```

Y la view de acerca debe renderizar:

CÓDIGO

```
posts/acerca.html
```

6. Configuración de rutas de la app

Creá o actualizá el archivo:

CÓDIGO

```
posts/urls.py
```

Dentro de ese archivo, definí los `path` necesarios para ambas vistas.

Debe existir una ruta para:

- Inicio.
- Acerca de.

7. Configuración de rutas del proyecto

En el archivo:

CÓDIGO

```
blog_project/urls.py
```

usá `include()` para conectar las rutas de la app `posts` con el proyecto principal.

Esto permite que las URLs de la app se mantengan organizadas dentro de `posts/urls.py`.

8. Estilizado con CSS

Creá un archivo llamado:

CÓDIGO

```
posts/static/posts/css/estilos.css
```

Agregá reglas básicas para mejorar la presentación visual del sitio.

Por ejemplo:

- Fuente general.
- Colores.
- Márgenes.
- Estilos para el menú de navegación.
- Estilos para títulos y secciones.

Luego, vinculá este archivo en el `<head>` de `base.html` usando:

CÓDIGO

```
{% static 'posts/css/estilos.css' %}
```

El sitio debe mostrar cambios visuales visibles al cargar la página.

9. Sincronización con GitHub

Realizá un commit con un mensaje descriptivo, por ejemplo:

CÓDIGO

```
Implementación de templates y CSS
```

Luego, subí los cambios al repositorio actual de GitHub.

No crees un repositorio nuevo.

Funcionalidades requeridas

Página de inicio

El sitio debe tener una página de inicio visible desde el navegador.

Esta página debe mostrar:

- Un título del blog.
- Un mensaje de bienvenida.
- Una breve descripción del proyecto.

Página “Acerca de”

El sitio debe tener una página “Acerca de”.

Esta página debe incluir:

- Una breve reseña sobre el autor del blog.
- Una explicación del objetivo del sitio.

Navegación

El template `base.html` debe incluir un menú de navegación que permita moverse entre:

- Inicio.
- Acerca de.

Los enlaces deben funcionar correctamente usando las rutas configuradas en Django.

Estilos

El sitio debe cargar el archivo `estilos.css` desde la carpeta `static`.

Al abrir el sitio en el navegador, debe notarse que el CSS está aplicado.

Criterios de aceptación

Para considerar este checkpoint como completado, tu proyecto debe cumplir con los siguientes puntos:

- Continuar sobre el proyecto Django del Módulo 7.
- Mantener la app `posts` registrada en `INSTALLED_APPS`.

- Crear la carpeta `posts/templates/posts/`.
- Crear la carpeta `posts/static/posts/css/`.
- Crear el archivo `base.html`.
- Crear el archivo `inicio.html`.
- Crear el archivo `acerca.html`.
- Usar `{% load static %}` en el template base.
- Usar `{% block content %}` en `base.html`.
- Usar herencia con `{% extends 'posts/base.html' %}` en las páginas hijas.
- Crear dos views en `posts/views.py`.
- Usar `render()` en ambas views.
- Crear o actualizar `posts/urls.py`.
- Conectar las URLs de la app desde `blog_project/urls.py` usando `include()`.
- Crear el archivo `estilos.css`.
- Vincular el CSS usando `{% static 'posts/css/estilos.css' %}`.
- Ver estilos aplicados en el navegador.
- El proyecto debe ejecutarse con `python manage.py runserver`.
- El menú de navegación debe funcionar.
- Subir los cambios al repositorio actual de GitHub.

Pasos sugeridos

1. Revisar el proyecto anterior

Antes de agregar templates y static, ejecutá el servidor y verificá que el proyecto Django del módulo anterior siga funcionando correctamente.

2. Crear las carpetas necesarias

Dentro de la app `posts`, creá la estructura para templates y static.

La organización debe respetar el nombre de la app para evitar conflictos cuando el proyecto crezca.

3. Crear el template base

Armá un archivo `base.html` con la estructura general del sitio.

Incluí el menú de navegación y un bloque de contenido reutilizable.

4. Crear los templates hijos

Creá `inicio.html` y `acerca.html`.

Ambos deben extender de `base.html` y definir su contenido específico.

5. Crear las views

En `posts/views.py`, definí las funciones necesarias para mostrar cada página.

Cada función debe devolver su template usando `render()`.

6. Configurar las URLs

Definí las rutas de la app en `posts/urls.py`.

Luego conectá esas rutas desde `blog_project/urls.py` usando `include()`.

7. Crear y vincular el CSS

Creá el archivo `estilos.css`.

Luego enlazalo en `base.html` usando el sistema de archivos estáticos de Django.

8. Probar en el navegador

Ejecutá el servidor local y abrí las rutas del sitio.

Verificá que:

- La página de inicio carga.
- La página acerca carga.
- El menú navega correctamente.
- Los estilos CSS se aplican.

9. Actualizar GitHub

Cuando todo funcione, realizá un commit con un mensaje claro y subí los cambios al repositorio.

Errores comunes a evitar

Olvidar `{% load static %}`

Si no cargás static en el template, Django no podrá resolver correctamente la ruta del archivo CSS.

Ubicar mal los templates

La estructura recomendada es:

CÓDIGO

```
posts/templates/posts/
```

No alcanza con crear una carpeta `templates` en cualquier lugar si Django no puede encontrarla correctamente.

Ubicar mal los archivos estáticos

La estructura recomendada es:

CÓDIGO

```
posts/static/posts/css/
```

Esto evita conflictos si más adelante el proyecto tiene varias apps con archivos CSS propios.

No usar `render()`

En este módulo, las views deben devolver templates HTML.

No alcanza con responder solamente texto plano usando `HttpResponse`.

Olvidar conectar `posts/urls.py`

Crear rutas dentro de la app no alcanza si después no las incluís desde `blog_project/urls.py`.

Escribir mal la ruta del template

Si el template está en:

CÓDIGO

```
posts/templates/posts/inicio.html
```

entonces desde la view debe llamarse como:

CÓDIGO

```
posts/inicio.html
```

Escribir mal la ruta del CSS

Si el CSS está en:

CÓDIGO

```
posts/static/posts/css/estilos.css
```

entonces en el template debe enlazarse como:

CÓDIGO

```
{% static 'posts/css/estilos.css' %}
```

No verificar visualmente el resultado

Que el servidor no muestre errores no significa que el CSS esté cargado.

Abrí el navegador y confirmá que los estilos se apliquen.

Subir el entorno virtual a GitHub

No subas la carpeta `venv/`.

El entorno virtual debe estar ignorado en `.gitignore`.

Formato de entrega

La entrega debe realizarse mediante el mismo repositorio público de GitHub utilizado en los módulos anteriores.

Asegurate de que:

- El repositorio siga siendo público.
- Los cambios estén subidos correctamente.
- El proyecto Django esté incluido en el repositorio.
- La app `posts` tenga sus templates.
- La app `posts` tenga su carpeta static.
- El archivo `estilos.css` esté incluido.
- El archivo `README.md` esté actualizado si agregaste instrucciones nuevas.
- El entorno virtual no esté subido al repositorio.
- El proyecto pueda ejecutarse con:

CÓDIGO

```
python manage.py runserver
```

El enlace del repositorio será la entrega del checkpoint.

Conexión con el futuro

Este checkpoint convierte tu proyecto Django inicial en un sitio web visible y navegable.

Hasta ahora, Django podía responder con texto simple. A partir de este módulo, el blog empieza a tener estructura visual: páginas HTML, herencia de templates, navegación y estilos.

En los próximos módulos, estas páginas podrán conectarse con modelos, base de datos, formularios y vistas más completas.

En otras palabras: en el Módulo 7 encendiste el motor de Django. En el Módulo 8, empezás a construir la primera versión visual del blog.

Pasos para la entrega

1. **Preparación de Estructuras**: Dentro de tu app `posts`, crea las carpetas `templates/posts/` y `static/posts/css/`. Es vital mantener esta arquitectura para que Django localice los archivos automáticamente.
2. **Creación del Layout Base**: Crea un archivo `base.html` que defina el esqueleto HTML5. Usa el tag `{% load static %}` al inicio y configura un `{% block content %}` donde se inyectará el contenido de las páginas hijas.

3. **Implementación de Herencia**: Crea `inicio.html` y `acerca.html`. Ambos deben empezar con `{% extends 'posts/base.html' %}`. En el bloque de contenido de `inicio.html`, agrega un título y una descripción de bienvenida. En `acerca.html`, escribe una breve reseña sobre el autor del blog.
4. **Lógica en Views**: En `posts/views.py`, define dos funciones que utilicen `render()`. Asegúrate de pasar el nombre correcto del template (ej: `'posts/inicio.html'`).
5. **Configuración de Rutas (URLs)**:
   - Crea `posts/urls.py` y define los `path` para ambas vistas.
   - En `blog_project/urls.py`, usa `include()` para conectar las rutas de tu app al proyecto principal.
6. **Estilizado**: Crea un archivo `estilos.css` en la carpeta static. Agrega reglas básicas (fuentes, colores, márgenes para el menú de navegación) y vincúlalo en el `head` de tu `base.html` usando `{% static 'posts/css/estilos.css' %}`.
7. **Sincronización con GitHub**: Realiza un commit con un mensaje descriptivo (ej: "Implementación de templates y CSS") y sube los cambios a tu repositorio actual.

Entregable

Formato: repositorio

Repositorio de GitHub actualizado con la arquitectura de carpetas templates/ y static/, archivos HTML con herencia y el archivo CSS activo.

Objetivos de aprendizaje

- Configurar el sistema de ruteo de Django mediante el uso de include() e urlpatterns.
- Implementar la herencia de plantillas para optimizar la reutilización de código HTML.
- Vincular archivos estáticos (CSS) a templates utilizando etiquetas de plantilla de Django.
- Organizar la estructura interna de una aplicación Django siguiendo las convenciones de 'templates' y 'static'.

Criterios de aceptación

- El servidor de desarrollo (`runserver`) se ejecuta sin errores de importación o configuración de URLs.
- El archivo base.html contiene la estructura HTML básica y es extendido correctamente por al menos otros dos archivos HTML.
- El archivo estilos.css se carga correctamente en el navegador devolviendo un código 200 en la consola (verificable con herramientas de desarrollador).
- La navegación entre la página de inicio y 'Acerca de' funciona mediante rutas de Django configuradas en posts/urls.py.
- Se utiliza correctamente la etiqueta {% load static %} y la función render() en las vistas.

Rúbrica de evaluación

Esta rúbrica evalúa la construcción de una interfaz web básica en Django mediante la implementación correcta de templates, herencia, vistas, rutas y archivos estáticos, además de la organización del repositorio y la sincronización del trabajo en GitHub. Está pensada para un nivel principiante y prioriza la coherencia técnica, la estructura del proyecto y la evidencia de funcionamiento real del sitio.

Criterio	Descripción	Peso
Estructura del proyecto y organización de carpetas	Evalúa si el repositorio contiene la arquitectura solicitada para templates y archivos estáticos dentro de la app posts, con una organización clara y compatible con Django.	20%
Templates y herencia con base.html	Evalúa la creación de base.html, el uso de {% load static %}, la definición de bloques y la correcta herencia en inicio.html y acerca.html.	30%
Vistas Django y renderizado de páginas	Evalúa la creación de las funciones en posts/views.py y el uso correcto de render() para devolver los templates solicitados.	15%
Rutas y conexión entre app y proyecto	Evalúa la definición de posts/urls.py, el uso de path() para ambas vistas y la integración con include() en blog_project/urls.py.	15%
Uso de archivos estáticos y CSS	Evalúa la creación del archivo estilos.css, su ubicación correcta, la aplicación de reglas básicas y su vinculación en base.html mediante la etiqueta static.	15%
Entrega del repositorio y sincronización en GitHub	Evalúa que el repositorio esté actualizado, que el commit sea descriptivo y que la evidencia de subida a GitHub muestre un trabajo completo y verificable.	5%

Total: 100 pts | Aprobación: 70 pts

MÓDULO 9

Modelos, ORM y panel de administración

Definir el modelo de datos de posts en Django, gestionar migraciones y utilizar el panel administrativo.

Modelos y campos: diseñar los datos del blog

Aprende los fundamentos del modelado de datos en Django para estructurar la información de un blog, identificando entidades, atributos y los tipos de campos necesarios para representar posts.

Imagina que decides abrir una librería física. Antes de comprar el primer estante o cargar el primer libro, necesitas responder una pregunta fundamental: **¿cómo voy a organizar la información?**

Necesitas saber, por ejemplo:

- El título del libro.
- El autor.
- El género.
- Si está disponible o prestado.
- Cuándo ingresó a la librería.

Si simplemente lanzas todos los libros en un montón, no tienes una librería: tienes un caos.

En el mundo del desarrollo web pasa algo parecido. Antes de crear pantallas, formularios o botones, necesitamos definir **qué datos vamos a guardar** y **cómo se van a organizar**.

En Django, ese diseño se realiza mediante **modelos**.

Un modelo nos permite decirle a Django:

> **Nota**
> “Cada publicación de mi blog va a tener un título, un contenido, una fecha de creación y un estado que indique si está publicada o no”.

1. ¿Qué es una base de datos y para qué sirve?

Hasta ahora, en tus ejercicios de Python, probablemente guardaste información en variables, listas o diccionarios.

Por ejemplo:

CÓDIGO

```
post = {
    "titulo": "Mi primer post",
    "contenido": "Estoy aprendiendo Django",
    "estado": "publicado"
}
```

Esto funciona mientras el programa está ejecutándose. El problema es que, cuando cierras el programa o apagas la computadora, esa información se pierde.

Para que un blog real funcione, necesitamos que los artículos sigan guardados mañana, la semana que viene y el mes que viene.

Ahí aparece la **base de datos**.

Una base de datos es un sistema que permite **almacenar, organizar y recuperar información de forma persistente**.

Persistente significa que los datos no desaparecen cuando el programa termina.

2. La analogía del archivo ordenado

Podemos imaginar una base de datos como un gran mueble de archivos en una oficina.

- El mueble completo es la **base de datos**.
- Cada cajón es una **tabla**.
- Cada carpeta dentro del cajón es un **registro** o **fila**.
- Cada dato dentro de la carpeta es un **campo** o **columna**.

Para un blog, podríamos tener una tabla llamada `posts`.

Esa tabla podría guardar datos como:

titulo	contenido	fecha_creacion	estado
Mi primer post	Estoy aprendiendo Django	2026-06-16	publicado
Ideas para mi blog	Hoy pensé nuevos temas	2026-06-17	borrador

Cada fila representa un post distinto.

Cada columna representa una característica de ese post.

3. ¿Qué es un modelo de datos en Django?

En Django, no necesitamos escribir SQL desde el primer día para crear tablas manualmente.

Django usa una herramienta llamada **ORM**, que significa **Object-Relational Mapper**.

Dicho de forma simple, el ORM permite trabajar con la base de datos usando clases y objetos de Python.

Un **modelo** en Django es una clase de Python que representa una tabla de la base de datos.

Por ejemplo, si queremos guardar publicaciones de un blog, podemos crear un modelo llamado `Post`.

Ese modelo será el plano que le dice a Django qué información tendrá cada publicación.

Importante:

> **Nota**
> El modelo no es un post específico. El modelo es la estructura que van a seguir todos los posts.

Es como una receta.

La receta dice qué ingredientes lleva un pastel. Pero la receta no es el pastel.

De la misma manera:

- El modelo `Post` define qué datos debe tener una publicación.
- Cada post guardado en la base de datos será un objeto real creado a partir de ese modelo.

4. ¿Dónde se escriben los modelos?

Dentro de una aplicación de Django, los modelos se escriben en el archivo `models.py`.

Si nuestra app se llama `posts`, la ruta sería:

CÓDIGO

```
posts/models.py
```

Por defecto, Django ya crea ese archivo cuando creamos la app.

Un archivo inicial podría verse así:

CÓDIGO

```
from django.db import models


class Post(models.Model):
    ESTADOS = [
        ("borrador", "Borrador"),
        ("publicado", "Publicado"),
        ("archivado", "Archivado"),
    ]

    titulo = models.CharField(max_length=200)
    contenido = models.TextField()
    fecha_creacion = models.DateTimeField(auto_now_add=True)
    estado = models.CharField(max_length=20, choices=ESTADOS, default="borrador")
```

Aunque es poco código, acá estamos diciendo muchísimo.

Le estamos indicando a Django:

- Que existirá un modelo llamado `Post`.
- Que cada post tendrá un título.
- Que cada post tendrá contenido.
- Que cada post tendrá una fecha de creación.
- Que cada post podrá estar publicado o no.

5. Entidades, atributos y campos

Para diseñar correctamente los datos de una aplicación, es útil pensar en tres conceptos: **entidades**, **atributos** y **campos**.

Entidades

Una entidad es una “cosa” del mundo real sobre la que queremos guardar información.

En un blog, algunas entidades podrían ser:

- Post
- Autor
- Categoría
- Comentario
- Etiqueta

Por ahora, nos vamos a concentrar en la entidad principal: el `Post`.

Atributos

Los atributos son las características de una entidad.

Si la entidad es un post, sus atributos podrían ser:

- Título
- Contenido
- Autor
- Fecha de creación
- Estado de publicación

Antes de escribir código, conviene pensar qué información necesita realmente nuestro modelo.

Por ejemplo:

CÓDIGO

```
Entidad: Post

Atributos:
- titulo
- contenido
- autor
- fecha_creacion
- estado
```

Campos

En Django, esos atributos se transforman en **campos de modelo**.

Cada campo tiene un tipo de dato específico.

Por ejemplo:

CÓDIGO

```
titulo = models.CharField(max_length=200)
```

Esto significa:

- `titulo` es el nombre del campo.
- `models.CharField` indica que será un texto corto.
- `max_length=200` indica que el texto podrá tener hasta 200 caracteres.

6. Tipos de campos comunes en Django

Django trae muchos tipos de campos. En esta unidad vamos a trabajar con algunos de los más comunes.

CharField

Se usa para textos cortos.

Ejemplos:

- Un título.
- Un nombre.
- Un estado editorial.
- Un nombre corto.

Siempre necesita un `max_length`.

Ejemplo:

CÓDIGO

```
titulo = models.CharField(max_length=200)
```

Otro ejemplo:

CÓDIGO

```
autor = models.CharField(max_length=100)
```

TextField

Se usa para textos largos.

Es ideal para el contenido completo de un post.

Ejemplo:

CÓDIGO

```
contenido = models.TextField()
```

Usamos `TextField` cuando no queremos limitar el contenido a una frase corta.

DateTimeField

Se usa para guardar fecha y hora.

Ejemplo:

CÓDIGO

```
fecha_creacion = models.DateTimeField(auto_now_add=True)
```

El parámetro `auto_now_add=True` le dice a Django que guarde automáticamente la fecha y hora cuando se crea el objeto por primera vez.

Esto es útil para saber cuándo fue creado un post.

CharField con `choices`

Cuando un texto solo puede tomar ciertos valores, podemos usar `choices`.

Ejemplo:

CÓDIGO

```
estado = models.CharField(max_length=20, choices=ESTADOS, default="borrador")
```

Este campo puede guardar únicamente valores definidos en la lista `ESTADOS`, por ejemplo:

CÓDIGO

```
ESTADOS = [
        ("borrador", "Borrador"),
        ("publicado", "Publicado"),
        ("archivado", "Archivado"),
    ]
```

En nuestro blog, podemos usarlo para indicar si el post está publicado, todavía es un borrador o se encuentra archivado.

Por defecto usamos `borrador`, porque no queremos que un post no aparezca publicado automáticamente apenas se crea.

7. Diseñando el modelo Post

Ahora vamos a diseñar el modelo principal de nuestro blog.

Queremos que cada post tenga:

- Un título.
- Un contenido.
- El nombre del autor.
- Una fecha de creación.
- Un estado que indique si está publicado.

El modelo podría quedar así:

CÓDIGO

```
from django.db import models


class Post(models.Model):
    ESTADOS = [
        ("borrador", "Borrador"),
        ("publicado", "Publicado"),
        ("archivado", "Archivado"),
    ]

    titulo = models.CharField(max_length=200)
    contenido = models.TextField()
    autor = models.CharField(max_length=100)
    fecha_creacion = models.DateTimeField(auto_now_add=True)
    estado = models.CharField(max_length=20, choices=ESTADOS, default="borrador")
```

Este código todavía no crea la tabla en la base de datos.

Por ahora, solo estamos definiendo el modelo en Python.

Más adelante, usaremos **migraciones** para que Django cree la tabla real en la base de datos.

8. ¿Por qué usamos nombres claros y sin tildes?

En este curso vamos a usar nombres en español para que el proyecto sea más claro y cercano durante el aprendizaje. Sin embargo, en proyectos profesionales es muy frecuente usar inglés para modelos, campos, funciones y variables, ya que es la convención más extendida en la industria y facilita la colaboración con equipos internacionales.

Más allá del idioma elegido, lo importante es mantener nombres consistentes, descriptivos, sin espacios, sin tildes y escritos en `snake_case`.

9. Agregando el método `__str__`

Cuando trabajamos con modelos en Django, es muy común agregar un método llamado `__str__`.

Este método define cómo se mostrará un objeto cuando Django necesite representarlo como texto.

Por ejemplo:

CÓDIGO

```
from django.db import models


class Post(models.Model):
    ESTADOS = [
        ("borrador", "Borrador"),
        ("publicado", "Publicado"),
        ("archivado", "Archivado"),
    ]

    titulo = models.CharField(max_length=200)
    contenido = models.TextField()
    autor = models.CharField(max_length=100)
    fecha_creacion = models.DateTimeField(auto_now_add=True)
    estado = models.CharField(max_length=20, choices=ESTADOS, default="borrador")



    def __str__(self):
        return self.titulo
```

Con esto, si tenemos un post llamado `"Aprendiendo Django"`, Django mostrará ese título en lugar de algo poco claro como:

CÓDIGO

```
Post object (1)
```

Este método no crea un campo en la base de datos. Solo mejora la forma en la que se representa el objeto.

10. ¿Por qué definimos tipos de datos?

Definir tipos de datos ayuda a que la información se guarde correctamente.

Imagina que permitimos guardar una fecha escribiendo cualquier texto:

CÓDIGO

```
"El martes pasado"
```

El problema aparece cuando queremos ordenar los posts del más nuevo al más antiguo.

La computadora no puede comparar fácilmente frases como:

CÓDIGO

```
"El martes pasado"
"Antes de ayer"
"El mes que viene"
```

En cambio, si usamos un `DateTimeField`, Django guarda una fecha real que puede ordenar, filtrar y comparar.

Lo mismo pasa con otros datos.

Para un título usamos texto corto:

CÓDIGO

```
titulo = models.CharField(max_length=200)
```

Para el contenido usamos texto largo:

CÓDIGO

```
contenido = models.TextField()
```

Para saber en qué estado editorial está el post usamos un campo de texto controlado por `choices`:

CÓDIGO

```
estado = models.CharField(max_length=20, choices=ESTADOS, default="borrador")
```

Cada tipo de campo ayuda a mantener la información ordenada y consistente.

11. Modelo inicial recomendado

Para este momento del curso, un buen modelo inicial para nuestro blog podría ser:

CÓDIGO

```
from django.db import models


class Post(models.Model):
    ESTADOS = [
        ("borrador", "Borrador"),
        ("publicado", "Publicado"),
        ("archivado", "Archivado"),
    ]

    titulo = models.CharField(max_length=200)
    contenido = models.TextField()
    autor = models.CharField(max_length=100)
    fecha_creacion = models.DateTimeField(auto_now_add=True)
    estado = models.CharField(max_length=20, choices=ESTADOS, default="borrador")

    def __str__(self):
        return self.titulo
```

Este modelo es simple, pero ya nos permite representar publicaciones reales.

Con este diseño podríamos guardar posts como:

CÓDIGO

```
Título: Mi primer post en Django
Contenido: Hoy estoy aprendiendo modelos y campos.
Autor: Ana
Fecha de creación: generada automáticamente
Estado: Borrador
```

O también:

CÓDIGO

```
Título: Introducción a Python
Contenido: Python es un lenguaje muy usado para desarrollo web.
Autor: Luis
Fecha de creación: generada automáticamente
Estado: Publicado
```

12. ¿Qué pasa con etiquetas, categorías o usuarios?

Tal vez te preguntes:

> **Nota**
>
> En la etapa de consola usamos `tags` y hablamos de categorías. ¿Por qué no aparecen ahora como campos del modelo?

En este primer modelo de Django vamos a mantener una versión simple y estable del post:

CÓDIGO

```
titulo
contenido
autor
fecha_creacion
estado
```

Los `tags` nos sirvieron en los módulos de Python para practicar listas, sets, búsquedas y filtrados en memoria. En Django podríamos incorporarlos más adelante, pero eso abre nuevas decisiones: guardarlos como texto simple, crear un modelo separado o usar una relación entre tablas.

Para este curso inicial, la clasificación principal del post será el campo `estado`, porque ya lo venimos usando desde los módulos de consola y se conecta directamente con el flujo editorial: borrador, publicado o archivado.

Si más adelante quisiéramos sumar categorías o etiquetas reales, podríamos hacerlo como una mejora del proyecto, pero no forma parte del modelo mínimo de este checkpoint.

13. Errores comunes

Error 1: Confundir el modelo con el objeto

Un error común es pensar que el modelo es el post real.

El modelo es la receta.

El objeto es el post concreto creado a partir de esa receta.

Ejemplo de modelo:

CÓDIGO

```
class Post(models.Model):
    titulo = models.CharField(max_length=200)
    contenido = models.TextField()
```

Ejemplo de objeto real:

CÓDIGO

```
Título: Aprendiendo Django
Contenido: Hoy creé mi primer modelo.
```

El modelo sirve para crear muchos objetos con la misma estructura.

Error 2: Usar `CharField` para todo

A veces, al comenzar, se usa `CharField` para cualquier texto.

Por ejemplo:

CÓDIGO

```
contenido = models.CharField(max_length=200)
```

Esto no es ideal para el contenido de un post, porque un artículo puede tener muchos párrafos.

Para contenido largo, conviene usar:

CÓDIGO

```
contenido = models.TextField()
```

Error 3: Olvidar `max_length` en un `CharField`

Este código está incompleto:

CÓDIGO

```
titulo = models.CharField()
```

Django necesita saber cuál será la longitud máxima del campo.

La forma correcta sería:

CÓDIGO

```
titulo = models.CharField(max_length=200)
```

Error 4: Usar nombres poco claros

Este modelo es difícil de leer:

CÓDIGO

```
class Post(models.Model):
    X = [
        ("borrador", "Borrador"),
        ("publicado", "Publicado"),
        ("archivado", "Archivado"),
    ]

    t = models.CharField(max_length=200)
    c = models.TextField()
    e = models.CharField(max_length=20, choices=X, default="borrador")
```

Una versión más clara sería:

CÓDIGO

```
class Post(models.Model):
    ESTADOS = [
        ("borrador", "Borrador"),
        ("publicado", "Publicado"),
        ("archivado", "Archivado"),
    ]

    titulo = models.CharField(max_length=200)
    contenido = models.TextField()
    estado = models.CharField(max_length=20, choices=ESTADOS, default="borrador")
```

El código se escribe una vez, pero se lee muchas veces.

Por eso, los nombres claros son una parte fundamental de un buen proyecto.

14. Buena práctica: ordenar los posts

En un blog, normalmente queremos mostrar primero los posts más recientes.

Para eso, podemos agregar una clase interna llamada `Meta`.

Ejemplo:

CÓDIGO

```
from django.db import models


class Post(models.Model):
    ESTADOS = [
        ("borrador", "Borrador"),
        ("publicado", "Publicado"),
        ("archivado", "Archivado"),
    ]

    titulo = models.CharField(max_length=200)
    contenido = models.TextField()
    autor = models.CharField(max_length=100)
    fecha_creacion = models.DateTimeField(auto_now_add=True)
    estado = models.CharField(max_length=20, choices=ESTADOS, default="borrador")


    class Meta:
        ordering = ["-fecha_creacion"]


    def __str__(self):
        return self.titulo
```

La línea:

CÓDIGO

```
ordering = ["-fecha_creacion"]
```

indica que los posts se ordenarán por fecha de creación de forma descendente.

Es decir, del más nuevo al más antiguo.

El signo `-` delante de `fecha_creacion` significa “orden descendente”.

15. Código final de ejemplo

Una versión completa y recomendable para este momento del proyecto sería:

CÓDIGO

```
from django.db import models


class Post(models.Model):
    ESTADOS = [
        ("borrador", "Borrador"),
        ("publicado", "Publicado"),
        ("archivado", "Archivado"),
    ]

    titulo = models.CharField(max_length=200)
    contenido = models.TextField()
    autor = models.CharField(max_length=100)
    fecha_creacion = models.DateTimeField(auto_now_add=True)
    estado = models.CharField(max_length=20, choices=ESTADOS, default="borrador")


    class Meta:
        ordering = ["-fecha_creacion"]


    def __str__(self):
        return self.titulo
```

Este modelo representa una publicación del blog con una estructura clara y fácil de entender.

Cada post tendrá:

- Un título.
- Un contenido.
- Un autor.
- Una fecha de creación automática.

- Un estado de publicación.
- Un orden por defecto del más nuevo al más antiguo.

16. Importante: el modelo todavía no crea la tabla

Es importante recordar algo:

> **Nota**
> Escribir un modelo en `models.py` no crea automáticamente la tabla en la base de datos.

El modelo es el diseño.

Para que ese diseño se convierta en una tabla real, Django necesita un paso más: las **migraciones**.

Las migraciones son archivos que Django usa para traducir nuestros modelos de Python en cambios reales dentro de la base de datos.

Ese será el próximo paso del proyecto.

Síntesis

En esta unidad pasamos de la idea general de “quiero crear un blog” a una estructura concreta de datos.

Aprendimos que:

- Una base de datos permite guardar información de forma persistente.
- Un modelo de Django es una clase de Python que representa una tabla.
- Cada atributo del modelo se transforma en un campo.
- Cada campo tiene un tipo de dato específico.
- `CharField` se usa para textos cortos.
- `TextField` se usa para textos largos.
- `DateTimeField` se usa para fechas y horas.
- `CharField` con `choices` sirve para limitar el estado a valores permitidos.
- El método `__str__` mejora la representación de los objetos.
- La clase `Meta` permite configurar comportamientos del modelo, como el orden por defecto.

En la próxima unidad vamos a ver cómo usar migraciones para convertir este modelo en una tabla real dentro de la base de datos.

Mini actividad

Crea o modifica el archivo:

CÓDIGO

```
posts/models.py
```

Luego define un modelo `Post` con los siguientes campos:

- `titulo`: texto corto de máximo 200 caracteres.
- `contenido`: texto largo.
- `autor`: texto corto de máximo 100 caracteres.
- `fecha_creacion`: fecha de creación automática.
- `estado`: texto con opciones como `borrador`, `publicado` o `archivado`, con valor inicial `"borrador"`.

También agrega:

- Un método `__str__` que devuelva el título del post.
- Una clase `Meta` para ordenar los posts del más nuevo al más antiguo.

Preguntas para reflexionar

1. ¿Por qué no conviene usar `CharField` para guardar el contenido completo de un post?
2. ¿Qué ventaja tiene usar `DateTimeField` en lugar de guardar una fecha como texto?
3. ¿Por qué el campo `estado` debería comenzar con `default="borrador"`?
4. ¿Qué ventaja tiene definir opciones fijas para `estado` en lugar de escribir cualquier texto?
5. ¿Qué crees que pasaría si olvidamos agregar una fecha de creación y luego queremos mostrar los posts más recientes primero?

Crear migraciones y aplicar cambios en la base de datos

Aprende a usar los comandos makemigrations y migrate para sincronizar tus modelos de Python con la base de datos real de tu blog.

Imagina que estás construyendo una casa.

En la unidad anterior diseñaste el plano: definiste que tu blog tendrá una estructura para guardar publicaciones con título, contenido, autor, fecha y estado de publicación.

Ese plano, en Django, es el **modelo**.

Pero dibujar un plano no hace que la casa aparezca automáticamente. Todavía falta construirla.

En Django, ese proceso de convertir los modelos en tablas reales de la base de datos se realiza mediante **migraciones**.

1. ¿Qué es una migración?

Una migración es un archivo de Python que contiene instrucciones para modificar la estructura de la base de datos.

Por ejemplo, una migración puede indicar:

- Crear una tabla nueva.
- Agregar una columna.
- Cambiar el tipo de un campo.
- Eliminar un campo.
- Modificar una configuración del modelo.

Podemos pensar las migraciones como un historial de cambios de la base de datos.

Así como Git registra los cambios del código, las migraciones registran los cambios en la estructura de las tablas.

2. Modelo, migración y base de datos

Es importante entender la diferencia entre estas tres partes:

Elemento	¿Qué es?
Modelo	La clase de Python que define la estructura de los datos.
Migración	El archivo con instrucciones para crear o modificar tablas.
Base de datos	El lugar donde finalmente se guardan los datos reales.

Por ejemplo, este modelo define cómo debería ser un post:

CÓDIGO

```
from django.db import models


class Post(models.Model):
    ESTADOS = [
        ("borrador", "Borrador"),
        ("publicado", "Publicado"),
        ("archivado", "Archivado"),
    ]

    titulo = models.CharField(max_length=200)
    contenido = models.TextField()
    autor = models.CharField(max_length=100)
    fecha_creacion = models.DateTimeField(auto_now_add=True)
    estado = models.CharField(max_length=20, choices=ESTADOS, default="borrador")


    def __str__(self):
        return self.titulo
```

Pero escribir este código en `models.py` todavía no crea la tabla en la base de datos.

Para eso necesitamos crear y aplicar migraciones.

3. El flujo de trabajo en dos pasos

En Django, el proceso normal para llevar cambios de los modelos hacia la base de datos tiene dos comandos principales:

1. Crear la migración.
2. Aplicar la migración.

Paso A: Crear la migración

El primer comando es:

CÓDIGO

```
python manage.py makemigrations
```

Este comando revisa los modelos del proyecto y detecta si hubo cambios.

Por ejemplo, si creaste un modelo llamado `Post`, Django detectará que necesita crear una tabla nueva.

Una salida posible en la terminal sería:

CÓDIGO

```
Migrations for 'posts':
  posts/migrations/0001_initial.py
    + Create model Post
```

Esto significa que Django creó un archivo nuevo dentro de la carpeta:

CÓDIGO

```
posts/migrations/
```

El archivo podría llamarse:

CÓDIGO

```
0001_initial.py
```

Importante:

> **Nota**
>
> `makemigrations` no modifica todavía la base de datos. Solo crea el archivo con las instrucciones.

Es decir, en este paso Django prepara el plano de construcción, pero todavía no construye la tabla.

Paso B: Aplicar la migración

El segundo comando es:

CÓDIGO

```
python manage.py migrate
```

Este comando toma las migraciones pendientes y las ejecuta en la base de datos.

Una salida posible podría ser:

CÓDIGO

```
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, posts, sessions

Running migrations:
  Applying posts.0001_initial... OK
```

Esto significa que la migración `0001_initial` de la app `posts` fue aplicada correctamente.

Ahora sí, la tabla existe físicamente en la base de datos.

4. Resumen del flujo completo

El proceso completo sería:

CÓDIGO

```
python manage.py makemigrations
python manage.py migrate
```

Primero generamos las instrucciones.

Después las aplicamos.

Una forma simple de recordarlo:

CÓDIGO

```
models.py  →  makemigrations  →  archivo de migración  →  migrate  →  base de datos
```

5. ¿Dónde se guardan las migraciones?

Cada app de Django tiene una carpeta llamada `migrations`.

Por ejemplo:

CÓDIGO

```
posts/
├── migrations/
│   ├── __init__.py
│   └── 0001_initial.py
├── models.py
├── views.py
└── admin.py
```

El archivo `0001_initial.py` fue creado automáticamente por Django.

Ese archivo contiene instrucciones para crear la tabla relacionada con el modelo `Post`.

Una versión simplificada de ese archivo podría verse así:

CÓDIGO

```
# Generated by Django

from django.db import migrations, models


class Migration(migrations.Migration):

    initial = True

    dependencies = []

    operations = [
        migrations.CreateModel(
            name="Post",
            fields=[
                ("id", models.BigAutoField(auto_created=True, primary_key=True)),
                ("titulo", models.CharField(max_length=200)),
                ("contenido", models.TextField()),
                ("autor", models.CharField(max_length=100)),
                ("fecha_creacion", models.DateTimeField(auto_now_add=True)),
                ("estado", models.CharField(max_length=20, default="borrador")),
            ],
        ),
    ]
```

No hace falta escribir este archivo a mano.

Django lo genera por nosotros cuando ejecutamos:

CÓDIGO

```
python manage.py makemigrations
```

6. Ver el estado de las migraciones

Django también nos permite revisar qué migraciones existen y cuáles ya fueron aplicadas.

Para eso usamos:

CÓDIGO

```
python manage.py showmigrations
```

Una salida posible sería:

CÓDIGO

```
posts
 [X] 0001_initial
```

El significado es:

- `[X]` : la migración ya fue aplicada.
- `[ ]` : la migración existe, pero todavía está pendiente.

Por ejemplo:

CÓDIGO

```
posts
 [ ] 0001_initial
```

Esto indica que la migración fue creada, pero todavía no se aplicó a la base de datos.

En ese caso, faltaría ejecutar:

CÓDIGO

```
python manage.py migrate
```

7. Ejemplo práctico completo

Supongamos que tenemos una app llamada `posts` .

En el archivo:

CÓDIGO

```
posts/models.py
```

escribimos el siguiente modelo:

CÓDIGO

```
from django.db import models


class Post(models.Model):
    ESTADOS = [
        ("borrador", "Borrador"),
        ("publicado", "Publicado"),
        ("archivado", "Archivado"),
    ]

    titulo = models.CharField(max_length=200)
    contenido = models.TextField()
    autor = models.CharField(max_length=100)
    fecha_creacion = models.DateTimeField(auto_now_add=True)
    estado = models.CharField(max_length=20, choices=ESTADOS, default="borrador")


    class Meta:
        ordering = ["-fecha_creacion"]


    def __str__(self):
        return self.titulo
```

Luego, en la terminal, desde la carpeta donde está el archivo `manage.py`, ejecutamos:

CÓDIGO

```
python manage.py makemigrations
```

Django detecta el nuevo modelo y crea una migración.

Después ejecutamos:

CÓDIGO

```
python manage.py migrate
```

Django aplica la migración y crea la tabla en la base de datos.

Finalmente, podemos verificar el estado con:

CÓDIGO

```
python manage.py showmigrations
```

Si todo salió bien, deberíamos ver la migración marcada con una `X`.

8. ¿Qué pasa si modifico el modelo después?

Los modelos pueden cambiar durante el desarrollo.

Por ejemplo, imaginemos que queremos agregar un resumen breve al post.

Modificamos el modelo:

CÓDIGO

```
from django.db import models


class Post(models.Model):
    ESTADOS = [
        ("borrador", "Borrador"),
        ("publicado", "Publicado"),
        ("archivado", "Archivado"),
    ]

    titulo = models.CharField(max_length=200)
    contenido = models.TextField()
    autor = models.CharField(max_length=100)
    fecha_creacion = models.DateTimeField(auto_now_add=True)
    estado = models.CharField(max_length=20, choices=ESTADOS, default="borrador")

    resumen = models.CharField(max_length=250, blank=True)


    class Meta:
        ordering = ["-fecha_creacion"]


    def __str__(self):
        return self.titulo
```

Ahora agregamos este campo:

CÓDIGO

```
resumen = models.CharField(max_length=250, blank=True)
```

Como cambió el modelo, debemos volver a ejecutar:

CÓDIGO

```
python manage.py makemigrations
```

Django podría mostrar algo parecido a esto:

CÓDIGO

```
Migrations for 'posts':
  posts/migrations/0002_post_resumen.py
    + Add field resumen to post
```

Luego aplicamos el cambio:

CÓDIGO

```
python manage.py migrate
```

De esta manera, Django actualiza la tabla existente agregando la nueva columna.

9. ¿Tengo que ejecutar migraciones cada vez?

No cada vez que abres el proyecto.

Solo debes crear y aplicar migraciones cuando cambias algo en los modelos.

Por ejemplo, debes usar migraciones si:

- Creas un modelo nuevo.
- Agregas un campo.
- Eliminas un campo.
- Cambias el tipo de un campo.
- Modificas alguna configuración importante del modelo.

No necesitas crear migraciones si solo cambias:

- Una vista.
- Una plantilla HTML.
- Un archivo CSS.
- Un archivo JavaScript.
- Un texto dentro de una página.

Las migraciones están relacionadas con cambios en la estructura de la base de datos.

10. Errores comunes

Error 1: Ejecutar solo `makemigrations`

Muchos principiantes ejecutan:

CÓDIGO

```
python manage.py makemigrations
```

Ven que se creó el archivo y creen que ya terminaron.

Pero si no ejecutan:

CÓDIGO

```
python manage.py migrate
```

la tabla todavía no existe en la base de datos.

Esto puede provocar errores cuando intenten guardar o consultar datos.

Recuerda:

CÓDIGO

```
makemigrations crea las instrucciones.
migrate ejecuta las instrucciones.
```

Error 2: Ejecutar los comandos desde una carpeta incorrecta

Los comandos deben ejecutarse desde la carpeta donde está el archivo:

CÓDIGO

```
manage.py
```

Si estás en otra carpeta, podrías ver un error como:

CÓDIGO

```
can't open file 'manage.py': [Errno 2] No such file or directory
```

En ese caso, debes moverte a la carpeta correcta del proyecto.

Error 3: Olvidar registrar la app en `INSTALLED_APPS`

Para que Django detecte los modelos de una app, la app debe estar registrada en `settings.py`.

Por ejemplo, si la app se llama `posts`, en el archivo:

CÓDIGO

```
blog_project/settings.py
```

deberías tener algo como:

CÓDIGO

```
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    "posts",
]
```

También puede aparecer usando la clase de configuración de la app:

CÓDIGO

```
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    "posts.apps.PostsConfig",
]
```

Ambas formas pueden funcionar.

Si la app no está registrada, Django no tendrá en cuenta sus modelos al crear migraciones.

Error 4: Editar archivos de migración manualmente

Por regla general, no conviene editar manualmente los archivos dentro de la carpeta `migrations`.

Si te equivocaste en un campo, lo mejor es corregirlo en `models.py` y volver a ejecutar:

CÓDIGO

```
python manage.py makemigrations
python manage.py migrate
```

Los archivos de migración deben tratarse como parte del historial del proyecto.

Error 5: Borrar la base de datos sin entender las consecuencias

Durante las primeras prácticas puede parecer tentador borrar el archivo `db.sqlite3` si algo sale mal.

En proyectos de aprendizaje, a veces se puede hacer, pero hay que entender que eso elimina los datos guardados.

En proyectos reales, borrar la base de datos sería un problema grave.

Por eso, es mejor acostumbrarse a trabajar correctamente con migraciones desde el principio.

11. Buena práctica: revisar antes de aplicar

Antes de aplicar migraciones, puedes revisar qué migraciones están pendientes con:

CÓDIGO

```
python manage.py showmigrations
```

Si ves una migración sin marcar:

CÓDIGO

```
posts
 [ ] 0002_post_resumen
```

significa que todavía no fue aplicada.

Luego puedes ejecutar:

CÓDIGO

```
python manage.py migrate
```

Y volver a revisar:

CÓDIGO

```
python manage.py showmigrations
```

Ahora debería aparecer:

CÓDIGO

```
posts
 [X] 0002_post_resumen
```

12. Síntesis

En esta unidad aprendimos que los modelos no crean tablas automáticamente.

El flujo correcto en Django es:

CÓDIGO

```
Crear o modificar modelos
        ↓
Ejecutar makemigrations
        ↓
Django crea archivos de migración
        ↓
Ejecutar migrate
        ↓
Django actualiza la base de datos
```

También vimos que:

- Una migración es un archivo con instrucciones para cambiar la base de datos.
- `makemigrations` crea las migraciones.
- `migrate` aplica las migraciones.
- `showmigrations` permite ver cuáles están aplicadas.
- Los archivos de migración no deberían editarse manualmente.
- La app debe estar registrada en `INSTALLED_APPS` para que Django detecte sus modelos.

Ejercicio práctico

Vas a implementar el modelo `Post` y preparar su migración.

Paso 1: Revisar la app

Verifica que exista una app llamada `posts`.

También revisa que esté registrada en `INSTALLED_APPS` dentro de `settings.py`.

Puedes usar esta forma:

CÓDIGO

```
"posts.apps.PostsConfig",
```

Paso 2: Crear el modelo

En el archivo:

CÓDIGO

```
posts/models.py
```

escribe el siguiente código:

CÓDIGO

```
from django.db import models


class Post(models.Model):
    ESTADOS = [
        ("borrador", "Borrador"),
        ("publicado", "Publicado"),
        ("archivado", "Archivado"),
    ]

    titulo = models.CharField(max_length=200)
    contenido = models.TextField()
    autor = models.CharField(max_length=100)
    fecha_creacion = models.DateTimeField(auto_now_add=True)
    estado = models.CharField(max_length=20, choices=ESTADOS, default="borrador")


    class Meta:
        ordering = ["-fecha_creacion"]


    def __str__(self):
        return self.titulo
```

Paso 3: Crear la migración

Desde la terminal, ubicado en la carpeta donde está `manage.py`, ejecuta:

CÓDIGO

```
python manage.py makemigrations
```

Deberías obtener una salida similar a:

CÓDIGO

```
Migrations for 'posts':
  posts/migrations/0001_initial.py
    + Create model Post
```

Paso 4: Aplicar la migración

Luego ejecuta:

CÓDIGO

```
python manage.py migrate
```

Si todo salió bien, deberías ver algo parecido a:

CÓDIGO

```
Applying posts.0001_initial... OK
```

Paso 5: Verificar el estado

Finalmente, ejecuta:

CÓDIGO

```
python manage.py showmigrations
```

Busca la app `posts`.

Deberías ver algo similar a:

CÓDIGO

```
posts
 [X] 0001_initial
```

Eso significa que la migración fue aplicada correctamente.

Preguntas para reflexionar

1. ¿Cuál es la diferencia entre `makemigrations` y `migrate`?
2. ¿Por qué escribir un modelo en `models.py` no alcanza para crear una tabla?
3. ¿Qué significa que una migración aparezca con `[X]` en `showmigrations`?
4. ¿Qué podría pasar si olvidas registrar la app `posts` en `INSTALLED_APPS`?
5. Si agregas un nuevo campo al modelo `Post`, ¿qué comandos deberías ejecutar después?

Configurar el admin de Django para gestionar posts

Aprende a activar y personalizar el potente panel de administración de Django. Descubrirás cómo crear un superusuario para gestionar tus contenidos y cómo registrar el modelo Post para que puedas crear artículos sin tocar código, optimizando la visualización con filtros y buscadores.

Ya tienes tu modelo de datos listo y la tabla creada en la base de datos mediante migraciones.

Pero ahora aparece una nueva pregunta:

> **Nota**
> ¿Cómo cargamos información en esa tabla?

Hasta ahora, Django ya sabe que existe un modelo llamado `Post`, pero no sería práctico escribir código cada vez que queremos crear una nueva publicación.

Imagina que eres el editor de un blog o de un periódico digital. No querrías abrir archivos de Python cada vez que necesitas publicar una noticia. Necesitas una pantalla con formularios, botones y listas donde puedas escribir un título, cargar el contenido y hacer clic en “Guardar”.

Para eso existe una de las herramientas más útiles de Django: el **Django Admin**.

El admin es un panel de administración que Django genera automáticamente para gestionar los datos de nuestros modelos.

1. ¿Qué es Django Admin?

Django Admin es una interfaz web privada que permite crear, editar, listar y eliminar datos de los modelos del proyecto.

En otras palabras, es una especie de panel interno de gestión.

También se lo puede pensar como un pequeño **CMS** o **back-office**.

Un CMS, o Content Management System, es un sistema que permite gestionar contenido sin escribir código cada vez.

En nuestro blog, el admin nos permitirá:

- Crear posts.
- Editar posts existentes.
- Eliminar posts.
- Buscar publicaciones.
- Filtrar contenidos.
- Marcar posts como publicados o borradores.

2. Sitio público vs sitio administrativo

En una aplicación web suelen existir dos partes diferentes.

Por un lado, está el sitio público: lo que ven los usuarios o lectores.

Por ejemplo:

CÓDIGO

```
http://127.0.0.1:8000/
```

Ahí se mostrarían los posts publicados del blog.

Por otro lado, está el sitio administrativo: lo que usa el dueño del proyecto para gestionar la información.

En Django, ese panel suele estar disponible en:

CÓDIGO

```
http://127.0.0.1:8000/admin/
```

Es importante entender esta diferencia:

> **Nota**
> El admin no es para los lectores del blog. El admin es para administradores, editores o personas autorizadas.

Para los usuarios finales construiremos vistas, templates y formularios propios.

3. Antes de entrar al admin

Antes de usar el panel, necesitamos cumplir algunas condiciones:

- Tener el proyecto Django creado.
- Tener una app, por ejemplo `posts`.
- Tener el modelo `Post` definido.
- Haber creado y aplicado las migraciones.
- Tener un usuario administrador.

Si todavía no aplicaste las migraciones, deberías ejecutar:

CÓDIGO

```
python manage.py makemigrations
python manage.py migrate
```

Esto es importante porque Django necesita tener creadas las tablas de la base de datos, incluyendo las tablas internas de usuarios, permisos y sesiones.

4. Crear un superusuario

Para entrar al panel de administración necesitamos una cuenta con permisos especiales.

En Django, esa cuenta se llama **superusuario**.

Un superusuario tiene permiso para ver, crear, editar y borrar información dentro del admin.

Para crearlo, usamos este comando desde la terminal, ubicados en la carpeta donde está el archivo `manage.py`:

CÓDIGO

```
python manage.py createsuperuser
```

La terminal hará algunas preguntas:

CÓDIGO

```
Username:
Email address:
Password:
Password (again):
```

Ejemplo:

CÓDIGO

```
Username: admin
Email address: admin@example.com
Password:
Password (again):
Superuser created successfully.
```

Cuando escribas la contraseña, es normal que la terminal no muestre letras, puntos ni asteriscos.

Parece que no está escribiendo nada, pero sí lo está haciendo. Es una medida de seguridad.

5. Ejecutar el servidor

Una vez creado el superusuario, podemos levantar el servidor de desarrollo:

CÓDIGO

```
python manage.py runserver
```

Luego abrimos el navegador y entramos a:

CÓDIGO

```
http://127.0.0.1:8000/admin/
```

Django mostrará una pantalla de inicio de sesión.

Allí ingresamos el usuario y la contraseña creados con `createsuperuser`.

6. ¿Por qué no aparece mi modelo Post?

Si entras al admin después de crear el superusuario, puede que veas secciones como:

- Users
- Groups

Pero tal vez no veas el modelo `Post`.

Esto es normal.

Django no muestra todos los modelos automáticamente en el admin. Por seguridad y organización, debemos decirle explícitamente qué modelos queremos administrar desde ese panel.

Para eso usamos el archivo:

CÓDIGO

```
posts/admin.py
```

7. Registrar el modelo Post en el admin

Dentro de la app `posts`, abrimos el archivo `admin.py`.

Al principio puede verse así:

CÓDIGO

```
from django.contrib import admin
```

Para registrar el modelo `Post`, debemos importarlo y luego usar `admin.site.register()`.

CÓDIGO

```
from django.contrib import admin
from .models import Post


admin.site.register(Post)
```

La línea:

CÓDIGO

```
from .models import Post
```

significa que estamos importando el modelo `Post` desde el archivo `models.py` de la misma carpeta.

El punto `.` indica una importación relativa.

Luego, esta línea:

CÓDIGO

```
admin.site.register(Post)
```

le dice a Django:

> **Nota**
> “Quiero que el modelo Post se pueda gestionar desde el panel de administración”.

Ahora, si refrescas el admin en el navegador, deberías ver una nueva sección llamada `Posts`.

8. Crear un post desde el admin

Una vez registrado el modelo, entra al panel:

CÓDIGO

```
http://127.0.0.1:8000/admin/
```

Luego sigue estos pasos:

1. Haz clic en `Posts`.
2. Haz clic en `Add Post`.
3. Completa el formulario.
4. Guarda los cambios.

Si usamos el modelo que venimos trabajando, el formulario mostrará campos como:

- `titulo`
- `contenido`
- `autor`
- `fecha_creacion`
- `estado`

El campo `fecha_creacion` puede no aparecer como editable si fue definido con:

CÓDIGO

```
fecha_creacion = models.DateTimeField(auto_now_add=True)
```

Esto sucede porque Django completa esa fecha automáticamente cuando se crea el post.

9. Mejorar cómo se muestran los posts

Si creas varios posts, puede que Django los muestre de una forma poco descriptiva, por ejemplo:

CÓDIGO

```
Post object (1)
Post object (2)
Post object (3)
```

Esto ocurre cuando el modelo no tiene definido el método `__str__`.

Para mejorar eso, en `posts/models.py` conviene tener algo así:

CÓDIGO

```
from django.db import models


class Post(models.Model):
    ESTADOS = [
        ("borrador", "Borrador"),
        ("publicado", "Publicado"),
        ("archivado", "Archivado"),
    ]

    titulo = models.CharField(max_length=200)
    contenido = models.TextField()
    autor = models.CharField(max_length=100)
    fecha_creacion = models.DateTimeField(auto_now_add=True)
    estado = models.CharField(max_length=20, choices=ESTADOS, default="borrador")


    class Meta:
        ordering = ["-fecha_creacion"]


    def __str__(self):
        return self.titulo
```

El método:

CÓDIGO

```
def __str__(self):
    return self.title
```

hace que Django muestre el título del post en lugar de un texto genérico.

Ahora, en el admin, los objetos se verán mejor:

CÓDIGO

```
Mi primer post en Django
Aprendiendo modelos
Ideas para mi blog
```

10. Personalizar el listado con ModelAdmin

El registro simple funciona, pero podemos mejorar mucho la experiencia.

Por ejemplo, sería útil que el admin muestre una tabla con:

- Título.
- Autor.

- Fecha de creación.
- Si está publicado o no.

Para eso usamos una clase llamada `ModelAdmin`.

En `posts/admin.py`, podemos reemplazar el registro simple por esto:

CÓDIGO

```
from django.contrib import admin
from .models import Post


class PostAdmin(admin.ModelAdmin):
    list_display = ("titulo", "autor", "fecha_creacion", "estado")
    search_fields = ("titulo", "contenido", "autor")
    list_filter = ("estado", "fecha_creacion")


admin.site.register(Post, PostAdmin)
```

Ahora el admin mostrará una lista más útil.

11. ¿Qué hace cada configuración?

La propiedad `list_display` define qué columnas queremos ver en la tabla principal del admin.

CÓDIGO

```
list_display = ("titulo", "autor", "fecha_creacion", "estado")
```

Esto permite ver rápidamente información importante sin entrar post por post.

La propiedad `search_fields` agrega una barra de búsqueda.

CÓDIGO

```
search_fields = ("titulo", "contenido", "autor")
```

Esto permite buscar posts por título, contenido o autor.

La propiedad `list_filter` agrega filtros laterales.

CÓDIGO

```
list_filter = ("estado", "fecha_creacion")
```

Esto permite filtrar, por ejemplo, posts publicados o borradores.

12. Usar el decorador admin.register

Otra forma muy usada de registrar modelos en el admin es mediante el decorador `@admin.register`.

El resultado es equivalente, pero el código queda un poco más ordenado.

CÓDIGO

```
from django.contrib import admin
from .models import Post


@admin.register(Post)
class PostAdmin(admin.ModelAdmin):
    list_display = ("titulo", "autor", "fecha_creacion", "estado")
    search_fields = ("titulo", "contenido", "autor")
    list_filter = ("estado", "fecha_creacion")
```

Esta forma dice:

> **Nota**
> “Registra el modelo Post usando esta configuración de administración”.

Puedes usar cualquiera de las dos formas.

Forma 1:

CÓDIGO

```
admin.site.register(Post, PostAdmin)
```

Forma 2:

CÓDIGO

```
@admin.register(Post)
```

En este curso usaremos la forma que resulte más clara para cada etapa.

13. Agregar edición rápida

También podemos permitir modificar algunos campos directamente desde el listado.

Por ejemplo, podríamos cambiar si un post está publicado o no sin abrir el formulario completo.

CÓDIGO

```
from django.contrib import admin
from .models import Post


@admin.register(Post)
class PostAdmin(admin.ModelAdmin):
    list_display = ("titulo", "autor", "fecha_creacion", "estado")
    search_fields = ("titulo", "contenido", "autor")
    list_filter = ("estado", "fecha_creacion")
    list_editable = ("estado",)
```

La opción:

CÓDIGO

```
list_editable = ("estado",)
```

permite editar ese campo desde la lista.

Importante: el campo incluido en `list_editable` también debe estar en `list_display`.

14. Configuración final recomendada

Para este momento del proyecto, una buena configuración de `posts/admin.py` sería:

CÓDIGO

```
from django.contrib import admin
from .models import Post


@admin.register(Post)
class PostAdmin(admin.ModelAdmin):
    list_display = ("titulo", "autor", "fecha_creacion", "estado")
    search_fields = ("titulo", "contenido", "autor")
    list_filter = ("estado", "fecha_creacion")
    list_editable = ("estado",)
```

Con esto, el panel de administración permite:

- Ver los posts en una tabla clara.
- Buscar por título, contenido o autor.
- Filtrar por estado de publicación.
- Filtrar por fecha de creación.
- Cambiar rápidamente si un post está publicado.

15. Personalizar textos del admin

Por defecto, Django muestra títulos genéricos como:

CÓDIGO

```
Django administration
```

Podemos personalizar esos textos desde `admin.py`.

Por ejemplo:

CÓDIGO

```
from django.contrib import admin
from .models import Post


admin.site.site_header = "Administración de Mi Blog"
admin.site.site_title = "Panel del Blog"
admin.site.index_title = "Gestión de contenidos"


@admin.register(Post)
class PostAdmin(admin.ModelAdmin):
    list_display = ("titulo", "autor", "fecha_creacion", "estado")
    search_fields = ("titulo", "contenido", "autor")
    list_filter = ("estado", "fecha_creacion")
    list_editable = ("estado",)
```

Estas opciones modifican textos visuales del panel.

No cambian los modelos ni la base de datos.

16. Errores comunes

Error 1: No registrar el modelo

Crear el modelo y aplicar migraciones no alcanza para verlo en el admin.

Si no registras el modelo en `admin.py`, Django no lo mostrará.

Código necesario:

CÓDIGO

```
from django.contrib import admin
from .models import Post


admin.site.register(Post)
```

O usando decorador:

CÓDIGO

```
from django.contrib import admin
from .models import Post


@admin.register(Post)
class PostAdmin(admin.ModelAdmin):
    pass
```

Error 2: Usar nombres de campos que no existen

Si tu modelo tiene este campo:

CÓDIGO

```
autor = models.CharField(max_length=100)
```

No puedes usar en el admin:

CÓDIGO

```
list_display = ("titulo", "author")
```

Porque `author` no existe.

La versión correcta sería:

CÓDIGO

```
list_display = ("titulo", "autor")
```

Siempre revisa que los nombres usados en `list_display`, `search_fields` y `list_filter` existan realmente en el modelo.

Error 3: Entrar con un usuario sin permisos

Para entrar al admin, el usuario debe tener permisos de administración.

La forma más simple en esta etapa es usar:

CÓDIGO

```
python manage.py createsuperuser
```

Si creas un usuario común, puede que no tenga permiso para acceder al panel.

Error 4: Olvidar ejecutar migraciones antes de crear el superusuario

Si el proyecto todavía no tiene aplicadas las migraciones, puede fallar la creación del superusuario o el inicio del admin.

Antes de usar el panel, ejecuta:

CÓDIGO

```
python manage.py migrate
```

Error 5: Usar el admin como sitio para usuarios finales

El admin es una herramienta interna.

No conviene dar acceso al `/admin/` a usuarios comunes del sitio.

Para lectores, autores externos o usuarios registrados, más adelante se pueden crear vistas y formularios personalizados.

17. Síntesis

En esta unidad aprendimos a usar Django Admin para gestionar los posts del blog.

Vimos que:

- Django Admin es una interfaz privada de administración.
- El admin permite crear, editar, listar y borrar datos.
- Para entrar necesitamos un superusuario.
- El superusuario se crea con `createsuperuser`.
- Para que un modelo aparezca en el admin, debemos registrarlo en `admin.py`.
- `ModelAdmin` permite personalizar cómo se ve y se comporta el listado.
- `list_display` define columnas visibles.
- `search_fields` agrega búsqueda.
- `list_filter` agrega filtros laterales.
- `list_editable` permite editar campos desde la lista.

Ahora nuestro blog ya no es solo código: también tiene una herramienta visual para cargar contenido.

En la próxima unidad, usaremos el ORM de Django para consultar esos posts desde Python y mostrarlos en nuestras páginas HTML.

Ejercicio práctico

Configura el admin de Django para gestionar los posts del blog.

Paso 1: Verifica el modelo

En `posts/models.py`, asegúrate de tener un modelo similar a este:

CÓDIGO

```
from django.db import models


class Post(models.Model):
    ESTADOS = [
        ("borrador", "Borrador"),
        ("publicado", "Publicado"),
        ("archivado", "Archivado"),
    ]

    titulo = models.CharField(max_length=200)
    contenido = models.TextField()
    autor = models.CharField(max_length=100)
    fecha_creacion = models.DateTimeField(auto_now_add=True)
    estado = models.CharField(max_length=20, choices=ESTADOS, default="borrador")


    class Meta:
        ordering = ["-fecha_creacion"]


    def __str__(self):
        return self.titulo
```

Paso 2: Aplica migraciones

Desde la terminal, ejecuta:

CÓDIGO

```
python manage.py makemigrations
python manage.py migrate
```

Paso 3: Crea un superusuario

Ejecuta:

CÓDIGO

```
python manage.py createsuperuser
```

Completa los datos solicitados por la terminal.

Paso 4: Registra el modelo en el admin

En `posts/admin.py`, escribe:

CÓDIGO

```
from django.contrib import admin
from .models import Post


@admin.register(Post)
class PostAdmin(admin.ModelAdmin):
    list_display = ("titulo", "autor", "fecha_creacion", "estado")
    search_fields = ("titulo", "contenido", "autor")
    list_filter = ("estado", "fecha_creacion")
    list_editable = ("estado",)
```

Paso 5: Entra al admin

Ejecuta el servidor:

CÓDIGO

```
python manage.py runserver
```

Luego entra en el navegador a:

CÓDIGO

```
http://127.0.0.1:8000/admin/
```

Inicia sesión con el superusuario y crea algunos posts de prueba.

Preguntas para reflexionar

1. ¿Para qué sirve el Django Admin?
2. ¿Por qué el modelo `Post` no aparece automáticamente en el admin?
3. ¿Qué diferencia hay entre un usuario común y un superusuario?
4. ¿Qué hace `list_display` dentro de una clase `ModelAdmin`?
5. ¿Por qué no conviene usar el admin como interfaz para los lectores del blog?

Consultar posts con el ORM y enviarlos al template

Aprende a usar el ORM de Django para recuperar posts de la base de datos, filtrarlos y enviarlos dinámicamente a tus plantillas HTML.

Hasta ahora, nuestro blog ya tiene varias piezas importantes funcionando.

Ya hicimos lo siguiente:

- Diseñamos el modelo `Post`.
- Creamos y aplicamos migraciones.

- Registramos el modelo en el admin.
- Cargamos posts desde el panel de administración.

Pero todavía falta algo fundamental:

> **Nota**
> Mostrar esos posts en la página pública del blog.

Un blog no existe solo para quien administra el contenido. Existe para que los lectores puedan entrar, leer publicaciones y navegar el sitio.

En esta unidad aprenderás cómo consultar los posts guardados en la base de datos y enviarlos a un template HTML para mostrarlos en pantalla.

1. ¿Qué problema queremos resolver?

Cuando cargamos posts desde el admin, esos datos quedan guardados en la base de datos.

Pero el navegador no los muestra automáticamente.

Para que un usuario pueda verlos, necesitamos hacer este recorrido:

CÓDIGO

```
Base de datos → Vista de Django → Contexto → Template HTML → Navegador
```

Dicho de otra forma:

1. La vista recibe la petición del usuario.
2. La vista consulta los posts usando el ORM.
3. La vista envía esos posts al template mediante un contexto.
4. El template recorre los posts y los muestra con HTML.

2. ¿Qué es el ORM?

Django utiliza una herramienta llamada **ORM**, que significa **Object-Relational Mapper**.

El ORM nos permite trabajar con la base de datos usando Python, sin tener que escribir SQL directamente.

En lugar de escribir una consulta SQL como esta:

CÓDIGO

```
SELECT * FROM posts_post;
```

En Django podemos escribir:

CÓDIGO

```
Post.objects.all()
```

Mucho más amigable para quienes están aprendiendo.

El ORM traduce ese código Python en consultas reales a la base de datos.

3. ¿Qué es un QuerySet?

Cuando hacemos una consulta con el ORM, Django normalmente devuelve un **QuerySet**.

Un QuerySet es una colección de objetos obtenidos desde la base de datos.

Podemos pensarlo como una “lista inteligente”.

Por ejemplo:

CÓDIGO

```
posts = Post.objects.all()
```

La variable `posts` no contiene una lista común de Python, sino un QuerySet con objetos del modelo `Post`.

Ese QuerySet puede recorrerse, filtrarse, ordenarse y enviarse a un template.

4. El modelo que vamos a consultar

Para esta unidad, vamos a asumir que tenemos un modelo parecido a este:

CÓDIGO

```
from django.db import modelsy


class Post(models.Model):
    ESTADOS = [
        ("borrador", "Borrador"),
        ("publicado", "Publicado"),
        ("archivado", "Archivado"),
    ]

    titulo = models.CharField(max_length=200)
    contenido = models.TextField()
    autor = models.CharField(max_length=100)
    fecha_creacion = models.DateTimeField(auto_now_add=True)
    estado = models.CharField(max_length=20, choices=ESTADOS, default="borrador")


    class Meta:
        ordering = ["-fecha_creacion"]


    def __str__(self):
        return self.titulo
```

Los campos importantes para consultar y mostrar serán:

- `titulo`
- `contenido`
- `autor`
- `fecha_creacion`
- `estado`

5. Consultas básicas con el ORM

Para consultar información del modelo `Post`, usamos:

CÓDIGO

```
Post.objects
```

`objects` es el administrador que Django agrega a los modelos para poder consultar la base de datos.

Traer todos los posts

CÓDIGO

```
Post.objects.all()
```

Este comando trae todos los posts guardados en la tabla.

Ejemplo:

CÓDIGO

```
posts = Post.objects.all()
```

Traer solo los posts publicados

CÓDIGO

```
Post.objects.filter(estado="publicado")
```

Este comando trae únicamente los posts cuyo campo `estado` esté en `publicado`.

Ejemplo:

CÓDIGO

```
posts = Post.objects.filter(estado="publicado")
```

Esto es muy útil para un blog, porque probablemente no queremos mostrar al público los posts que todavía son borradores.

Filtrar por autor

Si queremos traer solo los posts de una persona específica, podemos filtrar por `autor`.

CÓDIGO

```
posts = Post.objects.filter(autor="Ana")
```

Esto significa:

> **Nota**
> Traer todos los posts cuyo autor sea Ana.

Ordenar los posts

En un blog, normalmente queremos mostrar primero los posts más recientes.

Para eso usamos `order_by`.

CÓDIGO

```
posts = Post.objects.order_by("-fecha_creacion")
```

El signo `-` delante de `fecha_creacion` significa orden descendente.

Es decir:

CÓDIGO

```
Más nuevo → Más antiguo
```

Sin el signo `-`, el orden sería ascendente:

CÓDIGO

```
posts = Post.objects.order_by("fecha_creacion")
```

Eso mostraría primero los posts más antiguos.

Combinar filtro y orden

Podemos combinar consultas.

Por ejemplo:

CÓDIGO

```
posts = Post.objects.filter(estado="publicado").order_by("-fecha_creacion")
```

Esto significa:

> **Nota**
> Traer solo los posts publicados y ordenarlos del más nuevo al más antiguo.

Esta será una de las consultas más útiles para la página principal del blog.

Contar posts

Podemos contar cuántos posts existen:

CÓDIGO

```
cantidad = Post.objects.count()
```

También podemos contar solo los publicados:

CÓDIGO

```
cantidad_publicados = Post.objects.filter(estado="publicado").count()
```

Saber si existen posts

Podemos verificar si hay posts publicados:

CÓDIGO

```
hay_posts = Post.objects.filter(estado="publicado").exists()
```

Esto devuelve `True` o `False`.

Es útil cuando queremos saber si hay contenido sin traer todos los datos completos.

6. La vista: consultar datos y enviarlos al template

En Django, la vista es la encargada de recibir la petición del usuario y devolver una respuesta.

En nuestro caso, la vista hará tres cosas:

1. Consultar los posts publicados.
2. Guardarlos en una variable.
3. Enviarlos al template usando un contexto.

En el archivo:

CÓDIGO

```
posts/views.py
```

podemos escribir:

CÓDIGO

```
from django.shortcuts import render
from .models import Post




def lista_posts(request):
    posts = Post.objects.filter(estado="publicado").order_by("-fecha_creacion")

    context = {
        "posts": posts,
    }

    return render(request, "posts/lista_posts.html", context)
```

Veamos qué hace cada parte.

Importar el modelo

CÓDIGO

```
from .models import Post
```

Esta línea importa el modelo `Post` desde el archivo `models.py` de la misma app.

Sin esta importación, la vista no puede consultar los posts.

Consultar los posts

CÓDIGO

```
posts = Post.objects.filter(estado="publicado").order_by("-fecha_creacion")
```

Esta línea busca en la base de datos todos los posts publicados y los ordena del más nuevo al más antiguo.

Crear el contexto

CÓDIGO

```
context = {
    "posts": posts,
}
```

El contexto es un diccionario de Python.

Sirve para enviar datos desde la vista hacia el template.

En este caso, estamos enviando una variable llamada `posts`.

Esa variable estará disponible dentro del HTML.

Renderizar el template

CÓDIGO

```
return render(request, "posts/lista_posts.html", context)
```

La función `render` recibe tres elementos principales:

- `request`: la petición del usuario.
- `"posts/lista_posts.html"`: el template que queremos mostrar.
- `context`: los datos que queremos enviar al template.

7. Configurar la URL

Para que el usuario pueda entrar a esta vista desde el navegador, necesitamos asociarla a una URL.

En el archivo:

CÓDIGO

```
posts/urls.py
```

podemos tener:

CÓDIGO

```
from django.urls import path
from . import views


urlpatterns = [
    path("posts/", views.lista_posts, name="lista_posts"),
]
```

Esto significa que cuando alguien visite `/posts/`, Django ejecutará la vista `lista_posts`.

Si todavía no existe el archivo `posts/urls.py`, puedes crearlo.

Luego, en el archivo principal de URLs del proyecto, por ejemplo:

CÓDIGO

```
blog_project/urls.py
```

debemos incluir las URLs de la app:

CÓDIGO

```
from django.contrib import admin
from django.urls import include, path


urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("posts.urls")),
]
```

Con esto, la página principal del proyecto usará las rutas definidas en la app `posts`.

8. Mostrar los posts en el template

Ahora necesitamos crear o modificar el archivo:

CÓDIGO

```
posts/templates/posts/lista_posts.html
```

Una versión simple podría ser:

CÓDIGO

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8" />
    <title>Mi Blog</title>
</head>
<body>
    <h1>Últimos posts</h1>

    {% for post in posts %}
        <article>
            <h2>{{ post.titulo }}</h2>
            <p>Escrito por {{ post.autor }}</p>
            <p>{{ post.fecha_creacion }}</p>
            <p>{{ post.contenido }}</p>
        </article>
    {% empty %}
        <p>Todavía no hay posts publicados.</p>
    {% endfor %}
</body>
</html>
```

La parte más importante es esta:

CÓDIGO

```
{% for post in posts %}
    <h2>{{ post.titulo }}</h2>
{% endfor %}
```

Esto recorre la colección de posts que enviamos desde la vista.

Por cada post, muestra su título.

9. Usar el template con herencia

Si ya tienes un archivo `base.html`, lo recomendable es que `lista_posts.html` herede de ese layout.

Por ejemplo, si tienes:

CÓDIGO

```
posts/templates/posts/base.html
```

Podrías escribir `lista_posts.html` así:

CÓDIGO

```
{% extends "posts/base.html" %}

{% block content %}
    <h1>Últimos posts</h1>

    {% for post in posts %}
        <article>
            <h2>{{ post.titulo }}</h2>
            <p>Escrito por {{ post.autor }}</p>
            <p>Publicado el {{ post.fecha_creacion }}</p>
            <p>{{ post.contenido }}</p>
        </article>
    {% empty %}
        <p>Todavía no hay posts publicados.</p>
    {% endfor %}
{% endblock %}
```

Esta opción es mejor si ya venimos trabajando con templates reutilizables.

10. ¿Qué significa `{% for %}`?

En los templates de Django podemos usar etiquetas especiales.

Una de ellas es `{% for %}`.

Sirve para recorrer colecciones.

Ejemplo:

CÓDIGO

```
{% for post in posts %}
    <h2>{{ post.titulo }}</h2>
{% endfor %}
```

Esto significa:

> **Nota**
>
> Por cada elemento dentro de `posts`, crea una variable temporal llamada `post` y muestra sus datos.

Es parecido a un `for` de Python, pero escrito con la sintaxis de templates de Django.

11. ¿Qué significa `{% empty %}`?

La etiqueta `{% empty %}` permite mostrar un mensaje cuando la colección está vacía.

Ejemplo:

CÓDIGO

```
{% for post in posts %}
    <h2>{{ post.titulo }}</h2>
{% empty %}
    <p>No hay posts disponibles.</p>
{% endfor %}
```

Si hay posts, Django los muestra.

Si no hay ninguno, muestra:

CÓDIGO

```
No hay posts disponibles.
```

Esto evita que la página quede vacía sin explicación.

12. Variables en el template

Cuando enviamos este contexto desde la vista:

CÓDIGO

```
context = {
    "posts": posts,
}
```

En el template podemos usar la variable `posts`.

Dentro del bucle, cada elemento se llama `post`.

Por eso podemos acceder a sus campos:

CÓDIGO

```
{{ post.titulo }}
{{ post.contenido }}
{{ post.autor }}
{{ post.fecha_creacion }}
{{ post.estado }}
```

Cada expresión entre dobles llaves muestra un dato en el HTML.

13. Mostrar solo una parte del contenido

A veces no queremos mostrar el contenido completo del post en la página de listado.

Podemos usar el filtro `truncatewords`.

Ejemplo:

CÓDIGO

```
<p>{{ post.contenido|truncatewords:30 }}</p>
```

Esto muestra solo las primeras 30 palabras del contenido.

Es útil para mostrar una vista previa de cada publicación.

Ejemplo completo:

CÓDIGO

```
{% for post in posts %}
    <article>
        <h2>{{ post.titulo }}</h2>
        <p>Escrito por {{ post.autor }}</p>
        <p>{{ post.contenido|truncatewords:30 }}</p>
    </article>
{% empty %}
    <p>Todavía no hay posts publicados.</p>
{% endfor %}
```

14. Dar formato a la fecha

También podemos mostrar la fecha de forma más amigable usando el filtro `date`.

Ejemplo:

CÓDIGO

```
<p>Publicado el {{ post.fecha_creacion|date:"d/m/Y" }}</p>
```

Esto podría mostrar algo como:

CÓDIGO

```
Publicado el 16/06/2026
```

También podemos incluir hora:

CÓDIGO

```
<p>Publicado el {{ post.fecha_creacion|date:"d/m/Y H:i" }}</p>
```

15. Ejemplo completo de vista y template

Archivo:

CÓDIGO

```
posts/views.py
```

CÓDIGO

```
from django.shortcuts import render
from .models import Post



def lista_posts(request):
    posts = Post.objects.filter(estado="publicado").order_by("-fecha_creacion")

    context = {
        "posts": posts,
        "page_title": "Últimos posts",
    }

    return render(request, "posts/lista_posts.html", context)
```

Archivo:

CÓDIGO

```
posts/templates/posts/lista_posts.html
```

CÓDIGO

```
{% extends "posts/base.html" %}

{% block content %}
    <h1>{{ page_title }}</h1>

    {% for post in posts %}
        <article>
            <h2>{{ post.titulo }}</h2>

            <p>
                Escrito por {{ post.autor }}
                el {{ post.fecha_creacion|date:"d/m/Y" }}
            </p>

            <p>{{ post.contenido|truncatewords:40 }}</p>
        </article>
    {% empty %}
        <p>Todavía no hay posts publicados.</p>
    {% endfor %}
{% endblock %}
```

16. El flujo completo de datos

Ahora podemos entender el recorrido completo:

CÓDIGO

```
1. Cargamos posts desde el admin.
2. Los posts quedan guardados en la base de datos.
3. La vista consulta los posts con el ORM.
4. La vista los guarda en un contexto.
5. El template recibe el contexto.
6. El template recorre los posts con {% for %}.
7. El navegador muestra el HTML final al usuario.
```

Este es uno de los flujos más importantes de Django.

17. Errores comunes

Error 1: Olvidar importar el modelo

Si en la vista escribimos:

CÓDIGO

```
posts = Post.objects.all()
```

pero no importamos `Post`, Django dará error.

La importación correcta es:

CÓDIGO

```
from .models import Post
```

Error 2: Consultar los posts pero no enviarlos al contexto

Este código consulta los posts:

CÓDIGO

```
posts = Post.objects.all()
```

Pero si luego hacemos esto:

CÓDIGO

```
return render(request, "posts/lista_posts.html")
```

el template no recibirá la variable `posts`.

La versión correcta es:

CÓDIGO

```
context = {
    "posts": posts,
}

return render(request, "posts/lista_posts.html", context)
```

Error 3: Usar en el template una variable con otro nombre

Si la vista envía esto:

CÓDIGO

```
context = {
    "posts": posts,
}
```

El template debe usar:

CÓDIGO

```
{% for post in posts %}
```

Pero si escribimos:

CÓDIGO

```
{% for post in publicaciones %}
```

no funcionará, porque `publicaciones` no existe en el contexto.

Los nombres deben coincidir.

Error 4: Confundir un QuerySet con un único objeto

Consultas como estas devuelven una colección:

CÓDIGO

```
Post.objects.all()
Post.objects.filter(estado="publicado")
```

Aunque haya un solo resultado, siguen devolviendo un QuerySet.

Por eso, en el template normalmente usamos un bucle:

CÓDIGO

```
{% for post in posts %}
    <h2>{{ post.titulo }}</h2>
{% endfor %}
```

Error 5: Mostrar posts borradores

Si usamos:

CÓDIGO

```
posts = Post.objects.all()
```

vamos a mostrar todos los posts, incluso los que tienen:

CÓDIGO

```

estado = "borrador"
```

Para una página pública, conviene usar:

CÓDIGO

```
posts = Post.objects.filter(estado="publicado")
```

Así evitamos mostrar borradores.

Error 6: No crear posts desde el admin

Si la página muestra el mensaje:

CÓDIGO

```
Todavía no hay posts publicados.
```

puede que no sea un error del código.

Tal vez todavía no creaste posts desde el admin, o los creaste pero dejaste el estado como `borrador`.

Para probar la vista, entra al admin y crea al menos un post con estado `publicado`.

18. Buena práctica: nombres claros en el contexto

Podríamos enviar el contexto así:

CÓDIGO

```
context = {
    "x": posts,
}
```

Y luego usar en el template:

CÓDIGO

```
{% for post in x %}
```

Esto funcionaría, pero sería poco claro.

Es mejor usar nombres descriptivos:

CÓDIGO

```
context = {
    "posts": posts,
}
```

Así el código se entiende mejor.

19. Síntesis

En esta unidad aprendimos a consultar datos desde la base de datos y mostrarlos en un template.

Vimos que:

- El ORM permite consultar la base de datos usando Python.
- `Post.objects.all()` trae todos los posts.

- `Post.objects.filter()` permite filtrar resultados.
- `order_by()` permite ordenar los resultados.
- Un QuerySet es una colección de objetos obtenidos desde la base de datos.
- La vista consulta los datos.
- El contexto envía los datos al template.
- El template usa `{% for %}` para recorrer los posts.
- El template usa `{{ }}` para mostrar valores.
- `{% empty %}` permite mostrar un mensaje cuando no hay resultados.

Con esto, el blog ya puede mostrar contenido real cargado desde el admin.

Ejercicio práctico

Vas a mostrar en la página principal los posts publicados del blog.

Paso 1: Verifica que tengas posts creados

Entra al admin:

CÓDIGO

```
http://127.0.0.1:8000/admin/
```

Crea algunos posts y asegúrate de marcar al menos uno como publicado.

Paso 2: Crear la vista

En `posts/views.py`, escribe:

CÓDIGO

```
from django.shortcuts import render
from .models import Post



def lista_posts(request):
    posts = Post.objects.filter(estado="publicado").order_by("-fecha_creacion")

    context = {
        "posts": posts,
        "page_title": "Últimos posts",
    }

    return render(request, "posts/lista_posts.html", context)
```

Paso 3: Configurar la URL de la app

En `posts/urls.py`, escribe:

CÓDIGO

```
from django.urls import path
from . import views


urlpatterns = [
    path("posts/", views.lista_posts, name="lista_posts"),
]
```

Paso 4: Incluir las URLs de la app en el proyecto

En `blog_project/urls.py`, asegúrate de tener:

CÓDIGO

```
from django.contrib import admin
from django.urls import include, path


urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("posts.urls")),
]
```

Paso 5: Crear el template

En `posts/templates/posts/lista_posts.html`, escribe:

CÓDIGO

```
{% extends "posts/base.html" %}

{% block content %}
    <h1>{{ page_title }}</h1>

    {% for post in posts %}
        <article>
            <h2>{{ post.titulo }}</h2>
            <p>Escrito por {{ post.autor }}</p>
            <p>Publicado el {{ post.fecha_creacion|date:"d/m/Y" }}</p>
            <p>{{ post.contenido|truncatewords:40 }}</p>
        </article>
    {% empty %}
        <p>Todavía no hay posts publicados.</p>
    {% endfor %}
{% endblock %}
```

Paso 6: Probar en el navegador

Ejecuta el servidor:

CÓDIGO

```
python manage.py runserver
```

Luego entra a:

CÓDIGO

```
http://127.0.0.1:8000/
```

Si todo está correcto, deberías ver los posts publicados cargados desde la base de datos.

Preguntas para reflexionar

1. ¿Qué es el ORM y para qué nos ayuda?
2. ¿Qué diferencia hay entre `all()` y `filter()`?
3. ¿Por qué conviene usar `filter(estado="publicado")` en la página pública?
4. ¿Qué función cumple el contexto en una vista de Django?
5. ¿Por qué necesitamos usar `{% for %}` para mostrar los posts en el template?
6. ¿Qué puede estar pasando si la página muestra “Todavía no hay posts publicados”?

PRE-ENTREGA DEL PROYECTO

Actividad evaluable · parte del proyecto integrador

Preentrega 9: Admin configurado y posts visibles desde la base de datos

Breve guía para completar el segundo checkpoint del programa: configurar el modelo de datos, el panel de administración de Django y verificar la visualización dinámica de posts en el frontend.

Preentrega 9:Modelos y Admin configurados en Django

¡Llegó el momento de conectar tu blog con una base de datos real!

En el módulo anterior, transformaste tu proyecto Django en un sitio visual utilizando rutas, vistas, templates, herencia HTML y archivos estáticos. Hasta ahora, las páginas podían mostrar contenido, pero ese contenido seguía siendo fijo o escrito directamente en el HTML.

En esta preentrega, vas a dar un paso clave: crear un modelo `Post`, sincronizarlo con la base de datos, administrarlo desde el panel de Django y mostrar publicaciones reales en el template usando el ORM.

Propósito

El objetivo de esta práctica es conectar tu aplicación de Django con una base de datos utilizando el ORM.

A partir de este checkpoint, dejarás de usar datos estáticos para mostrar publicaciones y comenzarás a trabajar con posts gestionados desde el panel administrativo de Django.

Vas a practicar:

- Creación de modelos.
- Definición de campos.
- Migraciones.
- Uso de SQLite.
- Registro de modelos en el admin.
- Creación de superusuario.
- Carga de contenido desde `/admin/`.
- Consultas con el ORM.
- Renderizado dinámico en templates.

Contexto de la cadena de entregables

En el **Módulo 7**, creaste la base inicial del proyecto Django: entorno virtual, instalación, proyecto, app y servidor local.

En el **Módulo 8**, implementaste el flujo:

CÓDIGO

```
Ruta -> Vista -> Template -> Estático
```

Creaste templates, herencia con `base.html`, páginas como `inicio.html` y `acerca.html`, y estilos con CSS.

Ahora, en el **Módulo 9**, vas a conectar esa estructura visual con una base de datos. El objetivo es que la página de listado muestre posts reales cargados desde el panel administrativo.

Objetivo de aprendizaje de la preentrega

El objetivo de esta preentrega es consolidar el uso inicial de modelos y ORM en Django.

Vas a practicar:

Modelos

Crear una clase que represente una publicación del blog.

Campos

Definir qué información guarda cada post, como título, contenido, autor, fecha y estado.

Migraciones

Sincronizar los cambios del modelo con la base de datos.

Admin

Habilitar el modelo para gestionarlo desde el panel administrativo de Django.

Superusuario

Crear un usuario administrador para acceder al panel `/admin/`.

ORM

Consultar publicaciones usando `Post.objects.all()` sin escribir SQL manualmente.

Templates dinámicos

Mostrar datos reales en HTML usando bucles de Django Template Language.

¿Qué construir?

Debés continuar trabajando sobre el proyecto Django del módulo anterior.

No crees un proyecto nuevo.

El objetivo es crear un modelo `Post`, cargar publicaciones desde el panel administrativo y mostrarlas dinámicamente en la página de listado del blog.

La estructura general del proyecto puede mantenerse así:

CÓDIGO

```
blog_django/
│
├── manage.py
├── README.md
├── requirements.txt
│
├── blog_project/
│   ├── settings.py
│   └── urls.py
│
└── posts/
    ├── admin.py
    ├── apps.py
    ├── models.py
    ├── urls.py
    ├── views.py
    │
    ├── templates/
    │   └── posts/
    │       ├── base.html
    │       ├── inicio.html
    │       ├── lista_posts.html
    │       └── acerca.html
    │
    └── static/
        └── posts/
            └── css/
                └── estilos.css
```

Requisitos técnicos

1. Continuidad del proyecto

Debés continuar trabajando sobre el mismo repositorio de GitHub utilizado en el módulo anterior.

El proyecto debe seguir funcionando con:

CÓDIGO

```
python manage.py runserver
```

La app `posts` debe continuar registrada en `INSTALLED_APPS`.

2. Definir el modelo `Post`

En el archivo:

CÓDIGO

```
posts/models.py
```

creá una clase llamada `Post`.

El modelo debe incluir, como mínimo, los siguientes campos:

- `titulo`
- `contenido`
- `autor`
- `fecha_creacion`
- `estado`

El campo `fecha_creacion` debe usar:

CÓDIGO

```
auto_now_add=True
```

Esto permite guardar automáticamente la fecha en la que se crea cada publicación.

3. Definir estados para los posts

El campo `estado` debe permitir diferenciar publicaciones según su situación.

Por ejemplo:

- Borrador.
- Publicado.
- Archivado.

Podés resolverlo con opciones predefinidas usando `choices`.

4. Implementar el método `__str__`

El modelo `Post` debe implementar el método `__str__`.

Este método debe retornar el título del post.

Esto permite que cada publicación se vea de forma clara dentro del panel de administración.

5. Sincronizar la base de datos

Después de definir el modelo, debés ejecutar las migraciones.

Primero:

CÓDIGO

```
python manage.py makemigrations
```

Luego:

CÓDIGO

```
python manage.py migrate
```

Al aplicar las migraciones, Django creará o actualizará la base de datos local.

Deberías ver el archivo:

CÓDIGO

```
db.sqlite3
```

en la raíz del proyecto.

Importante: `db.sqlite3` puede existir localmente, pero si el proyecto tiene `.gitignore`, no es necesario subirlo al repositorio.

6. Habilitar el panel admin

En el archivo:

CÓDIGO

```
posts/admin.py
```

importá el modelo `Post` y registralo en el panel administrativo.

Además, creá una clase `PostAdmin` que herede de:

CÓDIGO

```
admin.ModelAdmin
```

Esta clase debe permitir mostrar columnas útiles en la lista del admin, por ejemplo:

- Título.
- Autor.
- Estado.
- Fecha de creación.

7. Crear superusuario

Ejecutá el siguiente comando:

CÓDIGO

```
python manage.py createsuperuser
```

Luego, seguí los pasos para definir:

- Nombre de usuario.
- Email.
- Contraseña.

Este usuario te permitirá acceder al panel administrativo de Django.

8. Cargar contenido desde el admin

Iniciá el servidor con:

CÓDIGO

```
python manage.py runserver
```

Luego ingresá a:

CÓDIGO

```
http://127.0.0.1:8000/admin/
```

Accedé con el superusuario y cargá al menos **3 posts**.

Los posts deben tener distintos datos y, si es posible, diferentes estados.

9. Consultar posts con ORM

En el archivo:

CÓDIGO

```
posts/views.py
```

actualizá la view `lista_posts` para consultar los posts publicados desde la base de datos usando:

CÓDIGO

```
Post.objects.filter(estado="publicado").order_by("-fecha_creacion")
```

La view debe enviar esos posts al template mediante el contexto.

Un ejemplo recomendado sería:

CÓDIGO

```
from django.shortcuts import render
from .models import Post


def lista_posts(request):
    posts = Post.objects.filter(estado="publicado").order_by("-fecha_creacion")
    context = {
        "posts": posts
    }
    return render(request, "posts/lista_posts.html", context)
```

10. Renderizar posts en el template

En el archivo:

CÓDIGO

```
posts/templates/posts/lista_posts.html
```

utilizá un bucle de Django Template Language para recorrer las publicaciones.

Debe usarse una estructura como:

CÓDIGO

```
{% for post in posts %}
```

Dentro del bucle, mostrá dinámicamente:

- El título del post.
- El contenido del post.

También podés mostrar:

- Autor.
- Estado.
- Fecha de creación.

11. Subir cambios a GitHub

Realizá un commit con el mensaje:

CÓDIGO

```
Checkpoint: Modelos y Admin configurados
```

Luego subí los cambios al repositorio actual de GitHub.

Funcionalidades requeridas

Modelo `Post`

El proyecto debe tener un modelo `Post` definido en `posts/models.py`.

Ese modelo debe representar publicaciones del blog.

Migraciones aplicadas

El proyecto debe tener migraciones generadas y aplicadas correctamente.

La base de datos local debe quedar sincronizada con el modelo.

Admin configurado

El modelo `Post` debe estar registrado en el panel administrativo.

El admin debe mostrar columnas útiles como autor y estado.

Superusuario creado

Debe existir un superusuario local para ingresar al panel `/admin/`.

Posts cargados

Desde el panel administrativo, se deben crear al menos 3 publicaciones.

Página de listado dinámica

La página de listado debe consultar los posts desde la base de datos usando el ORM y mostrarlos en el template.

Renderizado con bucle

El template `lista_posts.html` debe usar un bucle:

CÓDIGO

```
{% for post in posts %}
```

para mostrar las publicaciones.

Criterios de aceptación

Para considerar este checkpoint como completado, tu proyecto debe cumplir con los siguientes puntos:

- Continuar sobre el proyecto Django del módulo anterior.
- Mantener la app `posts` registrada en `INSTALLED_APPS`.
- Crear el modelo `Post` en `posts/models.py`.
- Incluir los campos `titulo`, `contenido`, `autor`, `fecha_creacion` y `estado`.

- Usar `auto_now_add=True` en `fecha_creacion`.
- Implementar `__str__` para retornar el título.
- Ejecutar `python manage.py makemigrations`.
- Ejecutar `python manage.py migrate`.
- Verificar que se cree `db.sqlite3` localmente.
- Registrar el modelo `Post` en `posts/admin.py`.
- Crear una clase `PostAdmin` heredando de `admin.ModelAdmin`.
- Mostrar columnas como autor y estado en el listado del admin.
- Crear un superusuario.
- Acceder al panel `/admin/`.
- Cargar al menos 3 posts desde el admin.
- Consultar posts en la view `lista_posts` usando `Post.objects.filter(estado="publicado").order_by("-fecha_creacion")`.
- Enviar los posts al template mediante contexto.
- Usar `{% for post in posts %}` en `lista_posts.html`.
- Mostrar dinámicamente título y contenido de cada post.
- Mantener la herencia de templates y los estilos del módulo anterior.
- Realizar un commit con el mensaje `Checkpoint: Modelos y Admin configurados`.
- Subir los cambios al repositorio actual de GitHub.

Pasos sugeridos

1. Revisar el proyecto anterior

Antes de crear modelos, verificá que el proyecto del módulo anterior siga funcionando correctamente.

Ejecutá el servidor y comprobá que las páginas creadas con templates sigan cargando bien.

2. Crear el modelo `Post`

Abrí `posts/models.py` y definí la clase `Post`.

Pensá qué información necesita guardar una publicación del blog.

3. Agregar los campos necesarios

Incluí los campos mínimos solicitados:

- Título.
- Contenido.
- Autor.
- Fecha de creación.
- Estado.

La fecha de creación debe completarse automáticamente.

4. Agregar `__str__`

Definí el método `__str__` para que cada post se muestre por su título.

Esto mejora la visualización dentro del admin.

5. Crear migraciones

Ejecutá el comando para generar las migraciones.

Este paso le indica a Django que detecte los cambios realizados en el modelo.

6. Aplicar migraciones

Ejecutá el comando para aplicar las migraciones.

Este paso crea la tabla correspondiente en SQLite.

7. Configurar el admin

Importá y registrá el modelo `Post` en `posts/admin.py`.

Creá una clase `PostAdmin` para personalizar qué columnas se ven en el listado.

8. Crear superusuario

Creá un superusuario desde la terminal.

Luego ingresá al panel administrativo.

9. Cargar publicaciones

Desde `/admin/`, cargá al menos 3 posts.

Verificá que los datos se guarden correctamente.

10. Actualizar la view `lista_posts`

Modificá la view `lista_posts` para consultar los posts usando el ORM.

Enviá esos datos al template.

11. Actualizar el template

En `lista_posts.html`, reemplazá los datos estáticos por un bucle que muestre los posts recibidos desde la view.

12. Probar en el navegador

Ejecutá el servidor y verificá que la página de listado muestre las publicaciones cargadas desde el admin.

13. Subir los cambios

Realizá el commit solicitado y subí los cambios al repositorio actual.

Errores comunes a evitar

Crear el modelo pero no migrar

Definir `Post` en `models.py` no alcanza.

Después de crear o modificar modelos, siempre hay que generar y aplicar migraciones.

Olvidar registrar el modelo en admin

Si `Post` no aparece en `/admin/`, revisá `posts/admin.py`.

No crear superusuario

Para ingresar al panel administrativo necesitás un usuario administrador.

Usar datos estáticos en lugar del ORM

La página de listado debe mostrar posts obtenidos desde la base de datos.

No debería depender solamente de textos escritos a mano en el HTML.

No pasar contexto al template

Si la view consulta los posts pero no los envía al template, no se podrán mostrar en HTML.

Escribir mal el nombre de la variable

Si en la view enviás una variable llamada `posts`, en el template también debés usar `posts`.

Olvidar el bucle en el template

Para mostrar varios posts, necesitás recorrerlos con `{% for post in posts %}`.

Subir `db.sqlite3` sin querer

Si el proyecto está configurado para ignorar la base de datos local, no subas `db.sqlite3`.

La base puede crearse nuevamente ejecutando migraciones.

Romper el diseño anterior

Al agregar datos dinámicos, mantené la herencia con `base.html` y los estilos del módulo anterior.

Formato de entrega

La entrega debe realizarse mediante el mismo repositorio público de GitHub utilizado en los módulos anteriores.

Asegurate de que:

- El repositorio siga siendo público.
- Los cambios estén subidos correctamente.
- El modelo `Post` esté creado.

- Las migraciones estén incluidas.
- El modelo esté registrado en admin.
- El `README.md` esté actualizado si agregaste instrucciones nuevas.
- El entorno virtual no esté subido al repositorio.
- El proyecto pueda ejecutarse con:

CÓDIGO

```
python manage.py runserver
```

El commit solicitado debe tener el mensaje:

CÓDIGO

```
Checkpoint: Modelos y Admin configurados
```

El enlace del repositorio será la entrega del checkpoint.

Conexión con el futuro

Este checkpoint conecta por primera vez el blog visual con una base de datos real.

Hasta ahora, el sitio podía mostrar páginas HTML con estilos. A partir de este módulo, las publicaciones empiezan a vivir en SQLite y pueden administrarse desde el panel de Django.

En los próximos módulos, esta base permitirá construir vistas de detalle, formularios y operaciones CRUD para crear, editar y eliminar posts desde el propio sitio.

En otras palabras: en el Módulo 8 tu blog empezó a verse como un sitio web. En el Módulo 9, empieza a comportarse como una aplicación real con datos persistentes.

Pasos para la entrega

Conectar tu aplicación de Django con una base de datos real utilizando el ORM (Object-Relational Mapper). En este paso, dejarás de usar datos estáticos para mostrar publicaciones gestionadas desde el panel administrativo.

Pasos a realizar

1. **Definir el Modelo**: Abre `posts/models.py` y crea la clase `Post`. Asegúrate de que incluya los campos `titulo`, `contenido`, `autor`, `fecha_creacion` (usa `auto_now_add=True`) y `estado`. Implementa el método `__str__` para que retorne el título.
2. **Sincronizar la Base de Datos**: Ejecuta `python manage.py makemigrations` seguido de `python manage.py migrate`. Verifica que se haya creado el archivo `db.sqlite3` en la raíz.
3. **Habilitar el Panel Admin**: En `posts/admin.py`, importa tu modelo y regístralo. Crea una clase `PostAdmin` que herede de `admin.ModelAdmin` para mostrar las columnas de autor y estado en la lista.

4. **Generar el Superusuario**: Ejecuta `python manage.py createsuperuser` en tu terminal. Sigue los pasos para definir nombre, email y contraseña.
5. **Cargar Contenido**: Inicia el servidor, entra a `/admin` y crea al menos 3 posts con diferentes estados.
6. **Consultar y Renderizar**: En `views.py`, usa `Post.objects.filter(estado="publicado").order_by("-fecha_creacion")` dentro de la vista `lista_posts` para enviar los datos al template. En `lista_posts.html`, utiliza un bucle `{% for post in posts %}` para iterar y mostrar dinámicamente el título y contenido de cada uno.
7. **Subir cambios**: Realiza un commit con el mensaje 'Checkpoint: Modelos y Admin configurados' y sube el código a tu repositorio de GitHub.

Entregable

Formato: repositorio

Repositorio de GitHub actualizado con el modelo Post definido, migraciones aplicadas y la vista lista_posts renderizando datos desde el ORM.

Objetivos de aprendizaje

- Implementar modelos de Django para definir la estructura de datos de una aplicación.
- Realizar migraciones para sincronizar el código Python con la base de datos SQLite.
- Configurar el panel de administración de Django para la gestión de contenidos.
- Utilizar el ORM de Django para recuperar y mostrar registros en un template HTML.

Criterios de aceptación

- El archivo 'posts/models.py' contiene la clase Post con los campos requeridos.
- Existen archivos de migración en la carpeta 'posts/migrations/' correspondientes al modelo Post.
- El modelo Post está registrado en 'admin.py' y es editable desde el panel /admin.
- La página de inicio muestra dinámicamente los posts almacenados en la base de datos.
- El repositorio incluye un README.md actualizado que describe brevemente cómo entrar al panel admin.

Rúbrica de evaluación

Evalúa un repositorio de Django donde el estudiante define el modelo Post, aplica migraciones, configura el panel administrativo y renderiza publicaciones desde la base de datos usando el ORM. La rúbrica valora la corrección técnica, la integración entre archivos, la organización del repositorio y la demostración de contenido dinámico en la vista lista_posts.

Criterio	Descripción	Peso
Definición del modelo Post y campos requeridos	Evalúa si el modelo de datos está creado correctamente en posts/models.py y si refleja exactamente la estructura solicitada para posts gestionados desde la base de datos.	35%
Migraciones y base de datos aplicadas correctamente	Evalúa si el proyecto refleja el uso real del ORM mediante makemigrations, migrate y la creación de la base de datos SQLite dentro del repositorio.	20%
Configuración del panel administrativo y registro del modelo	Evalúa si posts/admin.py registra el modelo correctamente y si la clase PostAdmin mejora la visualización de la lista del admin mostrando autor y estado.	20%
Uso del ORM y renderizado dinámico en la vista lista_posts	Evalúa si la vista consulta datos desde la base de datos con Post.objects.filter(estado="publicado").order_by("-fecha_creacion") y si la plantilla lista_posts.html recorre y muestra los posts de forma dinámica.	20%
Carga de contenido, pruebas manuales y entrega del repositorio	Evalúa si el estudiante creó un superusuario, cargó al menos tres posts con estados distintos, dejó evidencia de un repositorio actualizado y presentó un commit coherente con el checkpoint.	5%

Total: 100 pts | Aprobación: 70 pts

MÓDULO 10

CRUD de posts con formularios e imágenes

Desarrollar operaciones CRUD completas y gestionar la carga y visualización de archivos multimedia.

Formularios en Django: de Form a ModelForm

Aprende la diferencia fundamental entre formularios genéricos y formularios vinculados a modelos en Django, comprendiendo su ciclo de validación y seguridad.

Imagina que estás construyendo la puerta de entrada a tu blog. No una puerta física, sino una puerta digital: el lugar donde los usuarios pueden escribir comentarios, enviar mensajes de contacto o crear contenido dentro de tu aplicación.

En el mundo web, esa puerta se llama **formulario**.

Podríamos crear formularios usando solamente HTML, pero rápidamente aparecerían varios problemas:

- Tendríamos que escribir cada campo manualmente.
- Tendríamos que recibir los datos en la vista.
- Tendríamos que validar si los campos están completos.
- Tendríamos que controlar si un email tiene formato válido.
- Tendríamos que convertir los datos recibidos a tipos que Python pueda usar.
- Tendríamos que mostrar errores cuando algo esté mal.

Django nos ayuda con todo ese proceso mediante **Django Forms**.

Un formulario de Django no es solo HTML. Es una clase de Python que se encarga de definir campos, validar datos, mostrar errores y entregar información limpia para que podamos usarla en nuestras vistas.

1. ¿Qué es un formulario en Django?

En Django, un formulario se define como una **clase de Python**.

Así como un modelo define la estructura de una tabla en la base de datos, un formulario define la estructura de los datos que esperamos recibir desde el usuario.

Por ejemplo, si queremos que alguien nos contacte desde el blog, podríamos pedir:

- Nombre
- Correo electrónico
- Mensaje

En Django, eso se puede representar así:

CÓDIGO

```
# posts/forms.py

from django import forms


class ContactoForm(forms.Form):
    nombre = forms.CharField(max_length=100, label="Tu nombre")
    correo = forms.EmailField(label="Tu correo electrónico")
    mensaje = forms.CharField(
        label="Mensaje",
        widget=forms.Textarea
    )
```

En este ejemplo:

- `CharField` representa un campo de texto.
- `EmailField` valida que el dato tenga formato de email.
- `Textarea` muestra un campo más grande para escribir mensajes largos.
- `label` permite definir el texto visible del campo.

2. ¿Por qué usar Django Forms?

Django Forms nos ayuda principalmente en tres tareas:

Generar HTML automáticamente

Django puede generar los campos del formulario sin que tengamos que escribir cada `<input />` a mano.

Por ejemplo, en un template podríamos hacer:

CÓDIGO

```
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Enviar</button>
</form>
```

Con `{{ form.as_p }}`, Django renderiza los campos dentro de etiquetas `<p>`.

También existen otras opciones como:

CÓDIGO

```
{{ form.as_table }}
{{ form.as_ul }}
```

Aunque en proyectos más reales solemos personalizar el HTML campo por campo.

Validar datos

Django puede revisar automáticamente si un campo obligatorio está vacío, si un email es válido o si un texto supera la longitud máxima permitida.

Por ejemplo:

CÓDIGO

```
correo = forms.EmailField(label="Tu correo electrónico")
```

Si el usuario escribe algo como:

CÓDIGO

```
hola
```

Django detectará que no es un correo válido.

Limpiar datos

Cuando los datos llegan desde el navegador, Django los recibe desde el request. El formulario se encarga de procesarlos y convertirlos a una forma más segura y útil para Python.

Cuando un formulario es válido, podemos acceder a los datos mediante `cleaned_data`.

Ejemplo:

CÓDIGO

```
nombre = form.cleaned_data["nombre"]
correo = form.cleaned_data["correo"]
mensaje = form.cleaned_data["mensaje"]
```

3. `Form` vs. `ModelForm`

Django nos ofrece dos formas principales de crear formularios:

- `forms.Form`
- `forms.ModelForm`

Aunque se parecen, se usan para situaciones diferentes.

A. `forms.Form`

Un `Form` es un formulario independiente. No está conectado directamente con un modelo de la base de datos.

Lo usamos cuando queremos pedir datos, validarlos y hacer algo con ellos, pero no necesariamente guardarlos en una tabla.

Casos típicos para usar `forms.Form`

- Formulario de contacto.
- Formulario de búsqueda.
- Filtros.
- Encuestas simples.
- Formularios que disparan alguna acción, pero no crean un objeto directamente.

Ejemplo de formulario de contacto:

CÓDIGO

```
# posts/forms.py

from django import forms


class ContactoForm(forms.Form):
    nombre = forms.CharField(
        max_length=100,
        label="Tu nombre"
    )

    correo = forms.EmailField(
        label="Tu correo electrónico"
    )

    mensaje = forms.CharField(
        label="Mensaje",
        min_length=10,
        widget=forms.Textarea(attrs={"rows": 5})
    )
```

En este caso, el formulario no está asociado a ningún modelo. Solo recibe datos y los valida.

Ejemplo de vista usando `forms.Form`

CÓDIGO

```
# posts/views.py

from django.shortcuts import render
from .forms import ContactoForm


def contacto(request):
    if request.method == "POST":
        form = ContactoForm(request.POST)

        if form.is_valid():
            nombre = form.cleaned_data["nombre"]
            correo = form.cleaned_data["correo"]
            mensaje = form.cleaned_data["mensaje"]

            print("Nombre:", nombre)
            print("Correo:", correo)
            print("Mensaje:", mensaje)

            return render(request, "posts/contacto_exito.html", {
                "nombre": nombre
            })

    else:
        form = ContactoForm()

    return render(request, "posts/contacto.html", {
        "form": form
    })
```

Analicemos el flujo:

CÓDIGO

```
if request.method == "POST":
```

Esto significa que el usuario envió el formulario.

CÓDIGO

```
form = ContactoForm(request.POST)
```

Creamos un formulario con los datos enviados.

CÓDIGO

```
if form.is_valid():
```

Le pedimos a Django que valide los datos.

CÓDIGO

```
form.cleaned_data
```

Si el formulario es válido, accedemos a los datos limpios.

Template para el formulario de contacto

CÓDIGO

```
{/* posts/templates/posts/contacto.html */}

<h1>Contacto</h1>

<form method="post">
    {% csrf_token %}

    {{ form.as_p }}

    <button type="submit">Enviar mensaje</button>
</form>
```

La línea más importante es:

CÓDIGO

```
{% csrf_token %}
```

Django usa este token como protección de seguridad. Si lo olvidamos, Django rechazará el envío del formulario.

Template de éxito

CÓDIGO

```
{/* posts/templates/posts/contacto_exito.html */}

<h1>Mensaje enviado</h1>

<p>Gracias por contactarte, {{ nombre }}.</p>
<p>Te responderemos a la brevedad.</p>
```

B. `forms.ModelForm`

Un `ModelForm` es un formulario conectado a un modelo.

Lo usamos cuando queremos crear o editar objetos de la base de datos.

Por ejemplo, si ya tenemos un modelo `Post`, no necesitamos volver a escribir todos los campos manualmente. Django puede leer el modelo y construir el formulario automáticamente.

Supongamos que tenemos este modelo:

CÓDIGO

```
# posts/models.py

from django.db import models
from django.contrib.auth.models import User


class Post(models.Model):
    ESTADOS = [
        ("borrador", "Borrador"),
        ("publicado", "Publicado"),
        ("archivado", "Archivado"),
    ]

    titulo = models.CharField(max_length=200)
    contenido = models.TextField()
    autor = models.CharField(max_length=100)
    fecha_creacion = models.DateTimeField(auto_now_add=True)
    estado = models.CharField(max_length=20, choices=ESTADOS, default="borrador")


    def __str__(self):
        return self.titulo
```

A partir de ese modelo, podemos crear un `ModelForm`.

CÓDIGO

```
# posts/forms.py

from django import forms
from .models import Post


class PostForm(forms.ModelForm):
    class Meta:
        model = Post
        fields = ["titulo", "contenido", "autor", "estado"]
```

Con muy poco código, Django genera un formulario basado en el modelo `Post`.

¿Qué hace la clase `Meta`?

Dentro de un `ModelForm`, la clase `Meta` le indica a Django qué modelo debe usar y qué campos deben aparecer en el formulario.

CÓDIGO

```
class Meta:
    model = Post
    fields = ["titulo", "contenido", "autor", "estado"]
```

Esto significa:

- El formulario está conectado al modelo `Post`.
- El usuario podrá completar `titulo`, `contenido`, `autor` y `estado`.
- Los demás campos no aparecerán en el formulario.

En este caso no incluimos `fecha_creacion`.

¿Por qué?

Porque `fecha_creacion` se genera automáticamente gracias a `auto_now_add=True`. En este módulo, `autor` sigue siendo texto simple y por eso lo completa el usuario en el formulario.

Ejemplo mejorado de `PostForm`

Podemos personalizar labels, widgets y mensajes de ayuda.

CÓDIGO

```
# posts/forms.py

from django import forms
from .models import Post


class PostForm(forms.ModelForm):
    class Meta:
        model = Post
        fields = ["titulo", "contenido", "autor", "estado"]

        labels = {
            "titulo": "Título del post",
            "contenido": "Contenido",
            "estado": "Estado de publicación",
        }

        widgets = {
            "titulo": forms.TextInput(attrs={
                "placeholder": "Escribe el título del post"
            }),
            "contenido": forms.Textarea(attrs={
                "rows": 6,
                "placeholder": "Escribe el contenido del post"
            }),
        }
```

Esto no cambia el modelo. Solo modifica la forma en que se muestra el formulario.

4. Crear un post usando `ModelForm`

Ahora veamos cómo usar el formulario en una vista.

CÓDIGO

```
# posts/views.py

from django.shortcuts import render, redirect
from .forms import PostForm


def crear_post(request):
    if request.method == "POST":
        form = PostForm(request.POST)

        if form.is_valid():
            form.save()

            return redirect("lista_posts")

    else:
        form = PostForm()

    return render(request, "posts/post_form.html", {
        "form": form,
        "titulo_pagina": "Crear post",
        "texto_boton": "Crear"

    })
```

En esta vista pasan varias cosas importantes.

Primero, revisamos si el formulario fue enviado:

CÓDIGO

```
if request.method == "POST":
```

Luego creamos el formulario con los datos recibidos:

CÓDIGO

```
form = PostForm(request.POST)
```

Después validamos:

CÓDIGO

```
if form.is_valid():
```

Y finalmente guardamos el post:

CÓDIGO

```
form.save()
```

Como todavía no incorporamos usuarios reales al flujo de creación de posts, el campo `autor` se completa desde el formulario.

Template para crear un post

CÓDIGO

```
{/* posts/templates/posts/post_form.html */}

<h1>{{ titulo_pagina }}</h1>

<form method="post">
    {% csrf_token %}

    {{ form.as_p }}

    <button type="submit">{{ texto_boton }}</button>
</form>
```

Este template muestra el formulario y permite enviar los datos usando `POST`.

5. El ciclo de vida de un formulario

Cuando un usuario completa y envía un formulario, Django sigue este proceso:

1. El navegador envía los datos mediante una petición `POST`.
2. La vista recibe esos datos en `request.POST`.
3. Creamos una instancia del formulario usando esos datos.
4. Ejecutamos `form.is_valid()`.
5. Si los datos son válidos, usamos `cleaned_data` o `form.save()`.
6. Si los datos no son válidos, Django vuelve a mostrar el formulario con errores.

Ejemplo:

CÓDIGO

```
if request.method == "POST":
    form = PostForm(request.POST)

    if form.is_valid():
        form.save()
        return redirect("lista_posts")
else:
    form = PostForm()
```

Este patrón se repite muchísimo en Django.

6. ¿Qué pasa cuando el formulario tiene errores?

Si el formulario no es válido, Django no entra al bloque de `if form.is_valid()`.

Por ejemplo, si el usuario deja vacío un campo obligatorio, el formulario vuelve al template con los errores cargados.

Podemos mostrar esos errores automáticamente con:

CÓDIGO

```
{{ form.as_p }}
```

O podemos mostrarlos de forma más personalizada:

CÓDIGO

```
<form method="post">
    {% csrf_token %}

    <div>
        {{ form.titulo.label_tag }}
        {{ form.titulo }}
        {{ form.titulo.errors }}
    </div>

    <div>
        {{ form.contenido.label_tag }}
        {{ form.contenido }}
        {{ form.contenido.errors }}
    </div>

    <div>
        {{ form.estado.label_tag }}
        {{ form.estado }}
        {{ form.estado.errors }}
    </div>

    <button type="submit">Guardar</button>
</form>
```

Esto nos da más control sobre el diseño del formulario.

7. Validaciones personalizadas

Además de las validaciones automáticas, podemos crear nuestras propias reglas.

Por ejemplo, podríamos evitar que un post tenga un título demasiado corto.

CÓDIGO

```
# posts/forms.py

from django import forms
from .models import Post


class PostForm(forms.ModelForm):
    class Meta:
        model = Post
        fields = ["titulo", "contenido", "autor", "estado"]

    def clean_titulo(self):
        titulo = self.cleaned_data["titulo"]

        if len(titulo) < 5:
            raise forms.ValidationError("El título debe tener al menos 5
caracteres.")

        return titulo
```

El método debe llamarse:

CÓDIGO

```
clean_nombreDelCampo
```

En este caso, como queremos validar `titulo`, escribimos:

CÓDIGO

```
clean_titulo
```

Si algo está mal, usamos:

CÓDIGO

```
raise forms.ValidationError("Mensaje de error")
```

Si todo está bien, devolvemos el valor:

CÓDIGO

```
return titulo
```

8. Validación del cliente vs. validación del servidor

Es importante entender que existen dos capas de validación.

Validación del cliente

Ocurre en el navegador.

Por ejemplo, HTML puede evitar que el usuario envíe un campo vacío si el input tiene `required`.

CÓDIGO

```
<input type="text" name="titulo" required="true" />
```

Esta validación mejora la experiencia del usuario, porque muestra errores rápido y evita envíos innecesarios.

Pero no es suficiente.

Un usuario con conocimientos técnicos puede modificar el HTML desde el navegador o enviar datos directamente al servidor.

Validación del servidor

Ocurre en Django, dentro de nuestro código Python.

Esta es la validación importante.

Aunque el navegador permita enviar cualquier cosa, Django debe volver a revisar los datos usando:

CÓDIGO

```
form.is_valid()
```

Por eso, nunca debemos guardar datos sin validar primero.

Mal ejemplo:

CÓDIGO

```
def crear_post(request):
    titulo = request.POST["titulo"]
    contenido = request.POST["contenido"]

    Post.objects.create(
        titulo=titulo,
        contenido=contenido
    )
```

Este código toma los datos directamente desde `request.POST` y los guarda sin validar.

Mejor ejemplo:

CÓDIGO

```
def crear_post(request):
    if request.method == "POST":
        form = PostForm(request.POST)

        if form.is_valid():
            form.save()
```

9. CSRF Token: el escudo de los formularios

En Django, cada formulario que envía datos con `POST` debe incluir:

CÓDIGO

```
{% csrf_token %}
```

Ejemplo:

CÓDIGO

```
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Enviar</button>
</form>
```

CSRF significa **Cross-Site Request Forgery**.

Es un tipo de ataque en el que otro sitio intenta enviar formularios en nombre de un usuario sin permiso.

Django protege nuestras vistas verificando que cada formulario tenga un token válido.

Si olvidamos escribir `{% csrf_token %}`, Django rechazará el envío del formulario.

10. Errores comunes

Error 1: confiar solo en el HTML

No alcanza con poner `required` en el template.

Siempre debemos validar en Django con:

CÓDIGO

```
form.is_valid()
```

Error 2: usar `fields = "__all__"` sin pensar

En un `ModelForm`, podemos hacer esto:

CÓDIGO

```
fields = "__all__"
```

Pero no es una buena práctica para principiantes.

El problema es que podríamos exponer campos que el usuario no debería editar.

Por ejemplo:

- `autor`
- `fecha_creacion`
- `id`
- campos internos del sistema

Es mejor indicar explícitamente qué campos se pueden editar:

CÓDIGO

```
fields = ["titulo", "contenido", "autor", "estado"]
```

Error 3: olvidar `csrf_token`

Este template fallará al enviar el formulario:

CÓDIGO

```
<form method="post">
    {{ form.as_p }}
    <button type="submit">Enviar</button>
</form>
```

La versión correcta es:

CÓDIGO

```
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Enviar</button>
</form>
```

Error 4: olvidar el `method="post"`

Si no indicamos el método, el formulario puede enviarse por `GET`.

Mal ejemplo:

CÓDIGO

```
<form>
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Enviar</button>
</form>
```

Mejor ejemplo:

CÓDIGO

```
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Enviar</button>
</form>
```

11. ¿Cuándo usar cada uno?

Usamos `forms.Form` cuando el formulario no representa directamente una tabla de la base de datos.

Ejemplos:

- Contacto
- Búsqueda
- Filtros
- Suscripción a newsletter
- Simuladores o calculadoras

Usamos `forms.ModelForm` cuando el formulario sirve para crear o editar datos de un modelo.

Ejemplos:

- Crear un post
- Editar un post
- Editar un perfil
- Cargar una imagen de post
- Actualizar datos de usuario

12. Resumen final

Los formularios son una parte central de cualquier aplicación web.

En Django, nos permiten recibir datos del usuario de forma ordenada, validarlos y trabajar con ellos dentro de nuestras vistas.

En esta unidad vimos que:

- Un formulario de Django es una clase de Python.
- `forms.Form` sirve para formularios independientes.
- `forms.ModelForm` sirve para formularios conectados a modelos.
- `form.is_valid()` valida los datos enviados.
- `cleaned_data` contiene los datos limpios.
- `form.save()` permite guardar objetos cuando usamos `ModelForm`.
- `{% csrf_token %}` protege los formularios enviados por `POST`.
- No debemos confiar únicamente en la validación del navegador.
- Es mejor listar campos manualmente antes que usar `fields = "__all__"`.

Con esto ya tienes la base para crear formularios reales dentro de tu blog. En la próxima etapa, podrás usar `PostForm` para crear y editar publicaciones desde una vista de Django.

Crear posts con una vista y un formulario ligado al modelo

Aprende a crear formularios vinculados a modelos (ModelForms) para permitir a los usuarios crear nuevos posts en el blog de forma segura y validada.

Crear posts con `ModelForm`: del navegador a la base de datos

En el módulo anterior aprendimos a mostrar información que ya existía en nuestra base de datos. Es decir, Django consultaba los posts guardados y los enviaba al template para mostrarlos en el navegador.

Pero un blog no está completo si no podemos escribir contenido nuevo.

Ahora vamos a dar el siguiente paso: permitir que el usuario complete un formulario desde el navegador y que esos datos terminen guardados como un nuevo registro en la base de datos.

Para lograrlo, usaremos una de las herramientas más útiles de Django: los `ModelForm`.

1. El problema: repetir demasiado trabajo

Imagina que tienes un modelo `Post` como este:

CÓDIGO

```
# posts/models.py

from django.db import models


class Post(models.Model):
    ESTADOS = [
        ("borrador", "Borrador"),
        ("publicado", "Publicado"),
        ("archivado", "Archivado"),
    ]

    titulo = models.CharField(max_length=200)
    contenido = models.TextField()
    autor = models.CharField(max_length=100)
    fecha_creacion = models.DateTimeField(auto_now_add=True)
    estado = models.CharField(max_length=20, choices=ESTADOS, default="borrador")

    def __str__(self):
        return self.titulo
```

Si quisiéramos crear un formulario manualmente, tendríamos que hacer muchas cosas:

- Escribir el HTML de cada campo.
- Validar que el título no esté vacío.
- Validar que el contenido tenga datos.
- Recibir los datos enviados desde el navegador.
- Crear manualmente el objeto `Post`.
- Guardarlo en la base de datos.

Algo así podría funcionar, pero sería bastante manual:

CÓDIGO

```
# posts/views.py

from django.shortcuts import render, redirect
from .models import Post


def crear_post(request):
    if request.method == "POST":
        titulo = request.POST.get("titulo")
        contenido = request.POST.get("contenido")
        autor = request.POST.get("autor")
        estado = request.POST.get("estado")

        Post.objects.create(
            titulo=titulo,
            contenido=contenido,
            autor=autor,
            estado=estado
        )

        return redirect("lista_posts")

    # Ejemplo conceptual: este template manual no será el que usemos finalmente.
    return render(request, "posts/post_form_manual.html")
```

El problema de este enfoque es que estamos tomando los datos directamente desde `request.POST` y guardándolos sin usar las validaciones de Django.

Además, si el modelo cambia, deberíamos actualizar también el formulario, las validaciones y la lógica de guardado.

Acá es donde aparece `ModelForm`.

2. La solución: usar `ModelForm`

Un `ModelForm` es un formulario que Django construye a partir de un modelo.

Esto significa que Django puede leer la estructura del modelo `Post` y generar un formulario automáticamente.

Por ejemplo:

- Si `titulo` es un `CharField`, Django genera un campo de texto.
- Si `contenido` es un `TextField`, Django puede mostrarlo como un área de texto.
- Si `estado` tiene opciones con `choices`, Django puede mostrar un selector.
- Si un campo es obligatorio en el modelo, el formulario también lo tratará como obligatorio.

3. Crear el archivo `forms.py`

Dentro de la aplicación `posts`, creamos un archivo llamado `forms.py`.

La estructura quedaría así:

CÓDIGO

```
blog_project/
│
├── manage.py
│
├── blog_project/
│   └── settings.py
│
└── posts/
    ├── models.py
    ├── views.py
    ├── urls.py
    ├── admin.py
    └── forms.py
```

Ahora definimos nuestro formulario:

CÓDIGO

```
# posts/forms.py

from django import forms
from .models import Post


class PostForm(forms.ModelForm):
    class Meta:
        model = Post
        fields = ["titulo", "contenido", "autor", "estado"]
```

Con muy poco código, ya tenemos un formulario conectado al modelo `Post`.

La clase `Meta` le indica a Django dos cosas importantes:

CÓDIGO

```
model = Post
```

Esto indica que el formulario está basado en el modelo `Post`.

CÓDIGO

```
fields = ["titulo", "contenido", "autor", "estado"]
```

Esto indica qué campos queremos mostrar en el formulario.

No incluimos `fecha_creacion` porque ese campo se completa automáticamente gracias a:

CÓDIGO

```
auto_now_add=True
```

4. Mejorar la presentación del formulario

Podemos personalizar los textos visibles y los campos HTML usando `labels` y `widgets`.

CÓDIGO

```
# posts/forms.py

from django import forms
from .models import Post


class PostForm(forms.ModelForm):
    class Meta:
        model = Post
        fields = ["titulo", "contenido", "autor", "estado"]

        labels = {
            "titulo": "Título",
            "contenido": "Contenido",
            "autor": "Autor",
            "estado": "Estado",
        }

        widgets = {
            "titulo": forms.TextInput(attrs={
                "placeholder": "Escribe el título del post"
            }),
            "contenido": forms.Textarea(attrs={
                "placeholder": "Escribe el contenido del post",
                "rows": 6
            }),
            "autor": forms.TextInput(attrs={
                "placeholder": "Nombre del autor"
            }),
        }
```

Esto no modifica el modelo. Solo cambia cómo se ve el formulario en el navegador.

5. Crear la vista para guardar posts

Ahora necesitamos una vista que muestre el formulario y procese los datos enviados.

En `posts/views.py`:

CÓDIGO

```
# posts/views.py

from django.shortcuts import render, redirect
from .forms import PostForm


def crear_post(request):
    if request.method == "POST":
        form = PostForm(request.POST)

        if form.is_valid():
            form.save()
            return redirect("lista_posts")

    else:
        form = PostForm()

    return render(request, "posts/post_form.html", {
        "form": form,
        "titulo_pagina": "Crear post",
        "texto_boton": "Crear"
    })
```

Esta vista maneja dos momentos distintos.

6. Momento GET: mostrar el formulario vacío

Cuando el usuario entra a la página por primera vez, normalmente lo hace mediante una petición `GET`.

En ese caso, queremos mostrar un formulario vacío.

CÓDIGO

```
else:
    form = PostForm()
```

Luego enviamos ese formulario al template:

CÓDIGO

```
return render(request, "posts/post_form.html", {
        "form": form,
        "titulo_pagina": "Crear post",
        "texto_boton": "Crear"
})
```

7. Momento POST: recibir y validar datos

Cuando el usuario completa el formulario y presiona el botón “Guardar”, el navegador envía una petición `POST`.

En ese caso, Django entra en este bloque:

CÓDIGO

```
if request.method == "POST":
    form = PostForm(request.POST)
```

Acá creamos una instancia de `PostForm`, pero esta vez le pasamos los datos recibidos desde el navegador.

Después validamos:

CÓDIGO

```
if form.is_valid():
```

Este paso es fundamental.

`is_valid()` revisa que los datos cumplan las reglas del formulario y del modelo.

Por ejemplo:

- Que los campos obligatorios no estén vacíos.
- Que no se superen los máximos de caracteres.
- Que las opciones elegidas sean válidas.
- Que los datos puedan convertirse correctamente.

Si todo está bien, guardamos:

CÓDIGO

```
form.save()
```

Como estamos usando `ModelForm`, Django sabe cómo crear un objeto `Post` y guardarlo en la base de datos.

8. Redirigir después de guardar

Después de guardar el post, usamos:

CÓDIGO

```
return redirect("lista_posts")
```

Esto redirige al usuario a otra página, por ejemplo, al listado de posts.

Esta práctica es muy importante.

Si después de guardar nos quedáramos en la misma página y el usuario actualizara el navegador, podría enviarse el formulario otra vez y crearse un post duplicado.

Por eso usamos el patrón:

CÓDIGO

```
POST → guardar → redirect
```

A este flujo se lo suele llamar **Post/Redirect/Get**.

9. Crear el template del formulario

Ahora necesitamos crear el archivo HTML.

Ruta sugerida:

CÓDIGO

```
posts/templates/posts/post_form.html
```

Contenido:

CÓDIGO

```
{/* posts/templates/posts/post_form.html */}

<h1>{{ titulo_pagina }}</h1>

<form method="post">
    {% csrf_token %}

    {{ form.as_p }}

    <button type="submit">{{ texto_boton }}</button>
</form>
```

La línea:

CÓDIGO

```
{{ form.as_p }}
```

le dice a Django que renderice automáticamente los campos del formulario dentro de etiquetas `<p>`.

Esto es útil para empezar rápido.

10. Mostrar errores del formulario

Si el usuario envía datos inválidos, Django vuelve a mostrar el formulario con los errores correspondientes.

Con `{{ form.as_p }}`, esos errores se muestran automáticamente.

Por ejemplo, si el usuario deja vacío el título, Django puede mostrar un mensaje indicando que ese campo es obligatorio.

También podemos mostrar los campos de forma más manual:

CÓDIGO

```
{/* posts/templates/posts/post_form.html */}

<h1>{{ titulo_pagina }}</h1>

<form method="post">
    {% csrf_token %}

    <div>
        {{ form.titulo.label_tag }}
        {{ form.titulo }}
        {{ form.titulo.errors }}
    </div>

    <div>
        {{ form.contenido.label_tag }}
        {{ form.contenido }}
        {{ form.contenido.errors }}
    </div>

    <div>
        {{ form.autor.label_tag }}
        {{ form.autor }}
        {{ form.autor.errors }}
    </div>

    <div>
        {{ form.estado.label_tag }}
        {{ form.estado }}
        {{ form.estado.errors }}
    </div>

    <button type="submit">{{ texto_boton }}</button>
</form>
```

Este formato requiere más código, pero nos da más control sobre el diseño.

11. Configurar la URL

Para poder acceder a la vista desde el navegador, necesitamos agregar una ruta.

En `posts/urls.py`:

CÓDIGO

```
# posts/urls.py

from django.urls import path
from . import views


urlpatterns = [
    path("", views.lista_posts, name="lista_posts"),
    path("crear/", views.crear_post, name="crear_post"),
]
```

Con esta URL, la página para crear posts quedaría disponible en:

CÓDIGO

```
/posts/crear/
```

Dependiendo de cómo esté configurado el archivo principal de URLs del proyecto, la ruta puede variar.

12. Agregar un enlace para crear posts

En el template donde mostramos la lista de posts, podemos agregar un enlace hacia el formulario.

Por ejemplo:

CÓDIGO

```
{/* posts/templates/posts/lista_posts.html */}

<h1>Listado de posts</h1>

<a href="{% url 'crear_post' %}">Crear nuevo post</a>

{% for post in posts %}
    <article>
        <h2>{{ post.titulo }}</h2>
        <p>{{ post.contenido }}</p>
        <p>Autor: {{ post.autor }}</p>
        <p>Estado: {{ post.estado }}</p>
    </article>
{% empty %}
    <p>No hay posts cargados.</p>
{% endfor %}
```

La línea importante es:

CÓDIGO

```
<a href="{% url 'crear_post' %}">Crear nuevo post</a>
```

Esto genera un enlace usando el nombre de la URL.

13. El token CSRF

Cada vez que enviamos un formulario con `method="post"`, debemos incluir:

CÓDIGO

```
{% csrf_token %}
```

Ejemplo:

CÓDIGO

```
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Guardar</button>
</form>
```

CSRF significa **Cross-Site Request Forgery**.

Es una protección que evita que otro sitio web intente enviar formularios en nombre de un usuario sin permiso.

Si olvidamos escribir `{% csrf_token %}`, Django rechazará el envío del formulario.

14. Código completo de referencia

forms.py

CÓDIGO

```
from django import forms
from .models import Post


class PostForm(forms.ModelForm):
    class Meta:
        model = Post
        fields = ["titulo", "contenido", "autor", "estado"]

        labels = {
            "titulo": "Título",
            "contenido": "Contenido",
            "autor": "Autor",
            "estado": "Estado",
        }

        widgets = {
            "titulo": forms.TextInput(attrs={
                "placeholder": "Escribe el título del post"
            }),
            "contenido": forms.Textarea(attrs={
                "placeholder": "Escribe el contenido del post",
                "rows": 6
            }),
            "autor": forms.TextInput(attrs={
                "placeholder": "Nombre del autor"
            }),
        }
```

views.py

CÓDIGO

```
from django.shortcuts import render, redirect
from .models import Post
from .forms import PostForm


def lista_posts(request):
    posts = Post.objects.all().order_by("-fecha_creacion")

    return render(request, "posts/lista_posts.html", {
        "posts": posts
    })


def crear_post(request):
    if request.method == "POST":
        form = PostForm(request.POST)

        if form.is_valid():
            form.save()
            return redirect("lista_posts")

    else:
        form = PostForm()

    return render(request, "posts/post_form.html", {
        "form": form,
        "titulo_pagina": "Crear post",
        "texto_boton": "Crear"
    })
```

urls.py

CÓDIGO

```
from django.urls import path
from . import views


urlpatterns = [
    path("", views.lista_posts, name="lista_posts"),
    path("crear/", views.crear_post, name="crear_post"),
]
```

post_form.html

CÓDIGO

```
<h1>{{ titulo_pagina }}</h1>

<form method="post">
    {% csrf_token %}

    {{ form.as_p }}

    <button type="submit">{{ texto_boton }}</button>
</form>

<a href="{% url 'lista_posts' %}">Volver al listado</a>
```

lista_posts.html

CÓDIGO

```
<h1>Listado de posts</h1>

<a href="{% url 'crear_post' %}">Crear nuevo post</a>

{% for post in posts %}
    <article>
        <h2>{{ post.titulo }}</h2>
        <p>{{ post.contenido }}</p>
        <p>Autor: {{ post.autor }}</p>
        <p>Estado: {{ post.estado }}</p>
        <p>Fecha: {{ post.fecha_creacion }}</p>
    </article>
{% empty %}
    <p>No hay posts cargados.</p>
{% endfor %}
```

15. Resumen del flujo completo

El circuito completo para crear un post es:

1. El usuario entra a `/posts/crear/`.
2. Django recibe una petición `GET`.
3. La vista crea un formulario vacío.
4. El template muestra el formulario.
5. El usuario completa los campos y presiona “Guardar”.
6. El navegador envía una petición `POST`.
7. Django crea un formulario con los datos recibidos.
8. La vista ejecuta `form.is_valid()`.
9. Si los datos son correctos, ejecuta `form.save()`.

10. Django guarda el nuevo post en la base de datos.
11. La vista redirige al listado de posts.
12. El usuario ve el nuevo post en pantalla.

16. Buenas prácticas

Al trabajar con formularios en Django, recuerda:

- Siempre usar `method="post"` cuando el formulario modifica datos.
- Siempre incluir `{% csrf_token %}` en formularios `POST`.
- Siempre validar con `form.is_valid()` antes de guardar.
- Usar `redirect()` después de guardar correctamente.
- Evitar tomar datos directamente desde `request.POST` si puedes usar un formulario.
- En `ModelForm`, listar los campos explícitamente en `fields`.

17. Cierre

En esta unidad aprendimos cómo conectar el navegador con la base de datos usando `ModelForm`.

Ahora nuestro blog ya no depende únicamente de posts cargados desde el panel de administración. También podemos crear publicaciones desde una página propia de nuestra aplicación.

Este es un paso clave dentro de Django: pasar de mostrar información a permitir que el usuario interactúe con el sistema y genere nuevos datos.

A partir de este punto, podemos seguir ampliando el blog con edición, eliminación, autenticación de usuarios y formularios más personalizados.

Detalle del post y rutas amigables con slug

Aprende a crear URLs legibles y profesionales para tu blog utilizando slugs y a implementar la página de detalle para visualizar cada post de forma individual.

Imagina que acabas de escribir un artículo en tu blog llamado:

CÓDIGO

```
Cómo configurar VS Code para Python
```

Quieres compartirlo con tus amigos, copias la URL del navegador y ves algo como esto:

CÓDIGO

```
http://miblog.com/posts/14/
```

Ese `14` identifica el post dentro de la base de datos, pero no dice nada sobre el contenido.

Ahora imagina que la URL fuera así:

CÓDIGO

```
http://miblog.com/posts/como-configurar-vs-code-para-python/
```

Mucho mejor.

Esa segunda URL es más clara para una persona, más fácil de compartir y más descriptiva para buscadores como Google.

En esta unidad vamos a aprender dos cosas importantes:

- Cómo crear una página de detalle para ver un post individual.
- Cómo usar `slug` para construir URLs más amigables.

1. ¿Qué es un `slug`?

Un `slug` es una versión simplificada de un texto que se usa dentro de una URL.

Por ejemplo:

CÓDIGO

```
Título: Mi primer post en Django
Slug: mi-primer-post-en-django
```

Otro ejemplo:

CÓDIGO

```
Título: Cómo configurar VS Code para Python
Slug: como-configurar-vs-code-para-python
```

El slug convierte un título común en una parte válida de una URL.

2. ¿Por qué no usar solo el ID?

Hasta ahora podríamos mostrar el detalle de un post usando su `id`.

Por ejemplo:

CÓDIGO

```
/posts/1/
/posts/2/
/posts/3/
```

Esto funciona, pero tiene algunas desventajas:

- No describe el contenido del post.
- Es menos claro para el usuario.

- Es menos amigable para buscadores.
- Hace que las URLs se vean más técnicas.

En cambio, una URL con slug es más expresiva:

CÓDIGO

```
/posts/mi-primer-post-en-django/
```

A simple vista ya podemos imaginar de qué trata la publicación.

3. Reglas básicas de un slug

Un slug suele tener estas características:

- Se escribe en minúsculas.
- No tiene espacios.
- Usa guiones `-` para separar palabras.
- Evita tildes y caracteres especiales.
- Debe ser único si lo usamos para identificar una página específica.

Ejemplo:

CÓDIGO

```
Título original:
¡Mi primer post en Django!

Slug:
mi-primer-post-en-django
```

4. Actualizar el modelo `Post`

Para usar slugs, necesitamos agregar un campo nuevo al modelo `Post`.

Supongamos que veníamos trabajando con un modelo parecido a este:

CÓDIGO

```
# posts/models.py

from django.db import models


class Post(models.Model):
    ESTADOS = [
        ("borrador", "Borrador"),
        ("publicado", "Publicado"),
        ("archivado", "Archivado"),
    ]

    titulo = models.CharField(max_length=200)
    contenido = models.TextField()
    autor = models.CharField(max_length=100)
    fecha_creacion = models.DateTimeField(auto_now_add=True)
    estado = models.CharField(max_length=20, choices=ESTADOS, default="borrador")

    def __str__(self):
        return self.titulo
```

Ahora le agregamos el campo `slug`:

CÓDIGO

```
# posts/models.py

from django.db import models


class Post(models.Model):
    ESTADOS = [
        ("borrador", "Borrador"),
        ("publicado", "Publicado"),
        ("archivado", "Archivado"),
    ]

    titulo = models.CharField(max_length=200)
    contenido = models.TextField()
    autor = models.CharField(max_length=100)
    fecha_creacion = models.DateTimeField(auto_now_add=True)
    estado = models.CharField(max_length=20, choices=ESTADOS, default="borrador")
    slug = models.SlugField(max_length=220, unique=True, blank=True, null=True)

    def __str__(self):
        return self.titulo
```

El campo nuevo es este:

CÓDIGO

```
slug = models.SlugField(max_length=220, unique=True, blank=True, null=True)
```

Veamos qué significa cada parte:

CÓDIGO

```
models.SlugField()
```

Indica que este campo está pensado para guardar textos compatibles con URLs.

CÓDIGO

```
max_length=220
```

Define la cantidad máxima de caracteres.

CÓDIGO

```
unique=True
```

Indica que no puede haber dos posts con el mismo slug.

CÓDIGO

```
blank=True
```

Permite dejar el campo vacío en formularios o en el panel de administración.

CÓDIGO

```
null=True
```

Permite que la base de datos guarde temporalmente el valor como vacío.

Esto es útil si ya teníamos posts creados antes de agregar el campo `slug`.

5. Crear y aplicar la migración

Después de modificar el modelo, debemos crear y aplicar una migración.

En la terminal:

CÓDIGO

```
python manage.py makemigrations
```

Luego:

CÓDIGO

```
python manage.py migrate
```

Esto actualizará la base de datos para agregar la nueva columna `slug` en la tabla de posts.

6. Generar el slug automáticamente

Podríamos escribir el slug manualmente desde el panel de administración, pero sería incómodo.

Django nos da una función llamada `slugify`, que convierte un texto en formato de slug.

Por ejemplo:

CÓDIGO

```
from django.utils.text import slugify

slugify("Mi primer post en Django")
```

Resultado:

CÓDIGO

```
mi-primer-post-en-django
```

Podemos usar `slugify` dentro del modelo para generar el slug automáticamente antes de guardar.

CÓDIGO

```
# posts/models.py

from django.db import models
from django.utils.text import slugify


class Post(models.Model):
    ESTADOS = [
        ("borrador", "Borrador"),
        ("publicado", "Publicado"),
        ("archivado", "Archivado"),
    ]

    titulo = models.CharField(max_length=200)
    contenido = models.TextField()
    autor = models.CharField(max_length=100)
    fecha_creacion = models.DateTimeField(auto_now_add=True)
    estado = models.CharField(max_length=20, choices=ESTADOS, default="borrador")
    slug = models.SlugField(max_length=220, unique=True, blank=True, null=True)

    def save(self, *args, **kwargs):
        if not self.slug:
            self.slug = slugify(self.titulo)

        super().save(*args, **kwargs)

    def __str__(self):
        return self.titulo
```

La parte importante es esta:

CÓDIGO

```
def save(self, *args, **kwargs):
    if not self.slug:
        self.slug = slugify(self.titulo)

    super().save(*args, **kwargs)
```

Esto significa:

- Antes de guardar el post, revisamos si no tiene slug.
- Si no tiene, lo generamos a partir del título.
- Después dejamos que Django guarde normalmente el objeto.

7. Cuidado con los slugs duplicados

Si tenemos dos posts con el mismo título, ambos intentarían generar el mismo slug.

Por ejemplo:

CÓDIGO

```
Título 1: Mi cumpleaños
Slug 1: mi-cumpleanos

Título 2: Mi cumpleaños
Slug 2: mi-cumpleanos
```

Como el campo tiene `unique=True`, Django no permitirá guardar dos posts con el mismo slug.

Para este curso, podemos resolverlo de forma simple escribiendo títulos diferentes.

En proyectos reales, se suele agregar un número al final del slug:

CÓDIGO

```
mi-cumpleanos
mi-cumpleanos-2
mi-cumpleanos-3
```

También se puede usar la fecha:

CÓDIGO

```
mi-cumpleanos-2026
```

8. Mejorar el panel de administración

Podemos hacer que el campo `slug` se complete automáticamente en el admin mientras escribimos el título.

En `posts/admin.py`:

CÓDIGO

```
# posts/admin.py

from django.contrib import admin
from .models import Post


@admin.register(Post)
class PostAdmin(admin.ModelAdmin):
    list_display = ("titulo", "autor", "estado", "fecha_creacion")
    list_filter = ("estado", "fecha_creacion")
    search_fields = ("titulo", "contenido", "autor")
    prepopulated_fields = {"slug": ("titulo",)}
```

La línea clave es:

CÓDIGO

```
prepopulated_fields = {"slug": ("titulo",)}
```

Esto le dice al admin:

“Mientras escribo el título, generá una sugerencia de slug automáticamente”.

9. ¿Qué es una página de detalle?

Hasta ahora veníamos trabajando con una lista de posts.

Una lista responde a la pregunta:

CÓDIGO

```
¿Qué posts existen?
```

Pero una página de detalle responde a esta otra pregunta:

CÓDIGO

```
¿Qué información completa tiene este post específico?
```

La lista muestra muchos objetos.

El detalle muestra uno solo.

10. Diferencia entre lista y detalle

Una vista de lista trae varios posts:

CÓDIGO

```
Post.objects.all()
```

Una vista de detalle trae un post específico:

CÓDIGO

```
get_object_or_404(Post, slug=slug)
```

La primera consulta podría devolver muchos resultados.

La segunda debe devolver un solo objeto.

11. Crear la URL de detalle

Ahora necesitamos crear una ruta dinámica.

En `posts/urls.py`:

CÓDIGO

```
# posts/urls.py

from django.urls import path
from . import views


urlpatterns = [
    path("", views.lista_posts, name="lista_posts"),
    path("crear/", views.crear_post, name="crear_post"),
    path("<slug:slug>/", views.detalle_post, name="detalle_post"),
]
```

La parte más importante es esta:

CÓDIGO

```
path("<slug:slug>/", views.detalle_post, name="detalle_post")
```

El primer `slug` indica el tipo de dato que Django espera capturar desde la URL.

El segundo `slug` es el nombre de la variable que recibirá la vista.

Por ejemplo, si el usuario entra a:

CÓDIGO

```
/posts/mi-primer-post-en-django/
```

Django interpreta que:

CÓDIGO

```
slug = "mi-primer-post-en-django"
```

Y ese valor se lo pasa a la vista `detalle_post`.

12. Crear la vista de detalle

En `posts/views.py`:

CÓDIGO

```
# posts/views.py

from django.shortcuts import render, get_object_or_404
from .models import Post


def detalle_post(request, slug):
    post = get_object_or_404(Post, slug=slug)

    return render(request, "posts/detalle_post.html", {
        "post": post
    })
```

La función recibe dos parámetros:

CÓDIGO

```
def detalle_post(request, slug):
```

`request` representa la petición del navegador.

`slug` representa el texto capturado desde la URL.

Luego buscamos el post:

CÓDIGO

```
post = get_object_or_404(Post, slug=slug)
```

Esto significa:

“Buscá un post cuyo slug coincida con el slug recibido en la URL. Si no existe, mostrá una página 404”.

13. ¿Por qué usamos `get_object_or_404`?

Podríamos buscar el post de esta manera:

CÓDIGO

```
post = Post.objects.get(slug=slug)
```

Pero si el post no existe, Django lanzaría un error.

En cambio, con:

CÓDIGO

```
post = get_object_or_404(Post, slug=slug)
```

Django muestra automáticamente una respuesta de “Página no encontrada”.

Esto es mejor para el usuario y evita que nuestra aplicación muestre errores técnicos innecesarios.

14. Crear el template de detalle

Creamos el archivo:

CÓDIGO

```
posts/templates/posts/detalle_post.html
```

Contenido:

CÓDIGO

```
{/* posts/templates/posts/detalle_post.html */}

<h1>{{ post.titulo }}</h1>

<p>Autor: {{ post.autor }}</p>
<p>Estado: {{ post.estado }}</p>
<p>Fecha: {{ post.fecha_creacion }}</p>

<hr />

<p>{{ post.contenido }}</p>

<a href="{% url 'lista_posts' %}">Volver al listado</a>
```

Este template muestra un solo post.

Usamos:

CÓDIGO

```
{{ post.titulo }}
{{ post.autor }}
{{ post.estado }}
{{ post.fecha_creacion }}
{{ post.contenido }}
```

porque la vista envió el post al template con este contexto:

CÓDIGO

```
{
    "post": post
}
```

15. Agregar enlaces desde la lista

Ahora necesitamos que cada post de la lista tenga un enlace a su página de detalle.

En `lista_posts.html`:

CÓDIGO

```
{/* posts/templates/posts/lista_posts.html */}

<h1>Listado de posts</h1>

<a href="{% url 'crear_post' %}">Crear nuevo post</a>

{% for post in posts %}
    <article>
        <h2>
            <a href="{% url 'detalle_post' post.slug %}">
                {{ post.titulo }}
            </a>
        </h2>

        <p>{{ post.contenido|truncatewords:30 }}</p>
        <p>Autor: {{ post.autor }}</p>
        <p>Estado: {{ post.estado }}</p>
        <p>Fecha: {{ post.fecha_creacion }}</p>
    </article>
{% empty %}
    <p>No hay posts cargados.</p>
{% endfor %}
```

La línea más importante es:

CÓDIGO

```
<a href="{% url 'detalle_post' post.slug %}">
```

Esto genera una URL usando:

- El nombre de la ruta: `detalle_post`
- El slug del post: `post.slug`

Por ejemplo, si el post tiene este slug:

CÓDIGO

```
mi-primer-post-en-django
```

Django genera una URL como:

CÓDIGO

```
/posts/mi-primer-post-en-django/
```

16. Código completo de referencia

models.py

CÓDIGO

```
from django.db import models
from django.utils.text import slugify


class Post(models.Model):
    ESTADOS = [
        ("borrador", "Borrador"),
        ("publicado", "Publicado"),
        ("archivado", "Archivado"),
    ]

    titulo = models.CharField(max_length=200)
    contenido = models.TextField()
    autor = models.CharField(max_length=100)
    fecha_creacion = models.DateTimeField(auto_now_add=True)
    estado = models.CharField(max_length=20, choices=ESTADOS, default="borrador")
    slug = models.SlugField(max_length=220, unique=True, blank=True, null=True)

    def save(self, *args, **kwargs):
        if not self.slug:
            self.slug = slugify(self.titulo)

        super().save(*args, **kwargs)

    def __str__(self):
        return self.titulo
```

admin.py

CÓDIGO

```
from django.contrib import admin
from .models import Post


@admin.register(Post)
class PostAdmin(admin.ModelAdmin):
    list_display = ("titulo", "autor", "estado", "fecha_creacion")
    list_filter = ("estado", "fecha_creacion")
    search_fields = ("titulo", "contenido", "autor")
    prepopulated_fields = {"slug": ("titulo",)}
```

views.py

CÓDIGO

```
from django.shortcuts import render, redirect, get_object_or_404
from .models import Post
from .forms import PostForm


def lista_posts(request):
    posts = Post.objects.all().order_by("-fecha_creacion")

    return render(request, "posts/lista_posts.html", {
        "posts": posts
    })


def crear_post(request):
    if request.method == "POST":
        form = PostForm(request.POST)

        if form.is_valid():
            form.save()
            return redirect("lista_posts")

    else:
        form = PostForm()

    return render(request, "posts/post_form.html", {
        "form": form,
        "titulo_pagina": "Crear post",
        "texto_boton": "Crear"
    })


def detalle_post(request, slug):
    post = get_object_or_404(Post, slug=slug)

    return render(request, "posts/detalle_post.html", {
        "post": post
    })
```

urls.py

CÓDIGO

```
from django.urls import path
from . import views


urlpatterns = [
    path("", views.lista_posts, name="lista_posts"),
    path("crear/", views.crear_post, name="crear_post"),
    path("<slug:slug>/", views.detalle_post, name="detalle_post"),
]
```

lista_posts.html

CÓDIGO

```
<h1>Listado de posts</h1>

<a href="{% url 'crear_post' %}">Crear nuevo post</a>

{% for post in posts %}
    <article>
        <h2>
            <a href="{% url 'detalle_post' post.slug %}">
                {{ post.titulo }}
            </a>
        </h2>

        <p>{{ post.contenido|truncatewords:30 }}</p>
        <p>Autor: {{ post.autor }}</p>
        <p>Estado: {{ post.estado }}</p>
        <p>Fecha: {{ post.fecha_creacion }}</p>
    </article>
{% empty %}
    <p>No hay posts cargados.</p>
{% endfor %}
```

detalle_post.html

CÓDIGO

```
<h1>{{ post.titulo }}</h1>

<p>Autor: {{ post.autor }}</p>
<p>Estado: {{ post.estado }}</p>
<p>Fecha: {{ post.fecha_creacion }}</p>

<hr />

<p>{{ post.contenido }}</p>

<a href="{% url 'lista_posts' %}">Volver al listado</a>
```

17. El flujo completo

Cuando el usuario entra a una URL como esta:

CÓDIGO

```
/posts/mi-primer-post-en-django/
```

Django sigue este camino:

1. Revisa las rutas definidas en `urls.py`.
2. Encuentra la ruta `"<slug: slug>/"`.
3. Captura el valor `"mi-primer-post-en-django"`.
4. Envía ese valor a la vista `detalle_post`.
5. La vista busca el post con ese slug.
6. Si lo encuentra, renderiza `detalle_post.html`.
7. Si no lo encuentra, muestra una página 404.

18. Slug dentro del ciclo CRUD

CRUD significa:

CÓDIGO

```
Create - Read - Update - Delete
```

Es decir:

CÓDIGO

```
Crear - Leer - Actualizar - Eliminar
```

El slug participa especialmente en la lectura de datos.

Create

Cuando creamos un post, Django puede generar el slug automáticamente desde el título.

CÓDIGO

```
Título: Mi primer post
Slug: mi-primer-post
```

Read

Cuando queremos leer un post específico, usamos el slug en la URL.

CÓDIGO

```
/posts/mi-primer-post/
```

Update

Si cambiamos el título de un post, podríamos cambiar también el slug.

Pero hay que tener cuidado.

Si ya compartimos el enlace, cambiar el slug puede romper la URL anterior.

Delete

Si eliminamos un post, su URL dejará de existir.

Si alguien intenta entrar, Django mostrará un error 404.

19. Buenas prácticas

Al trabajar con slugs, conviene tener en cuenta estas recomendaciones:

- Usar slugs claros y relacionados con el título.
- Evitar slugs duplicados.
- No cambiar slugs de posts ya publicados sin necesidad.
- Usar `slugify` para generar slugs válidos.
- Usar `get_object_or_404` para buscar objetos individuales.
- Crear enlaces usando `{% url %}` en lugar de escribir URLs a mano.
- Usar `unique=True` para evitar dos posts con la misma ruta.

20. Errores comunes

Error 1: escribir mal `get_object_or_404`

Incorrecto:

CÓDIGO

```
from django.shortcuts import get_object_ some_404
```

Correcto:

CÓDIGO

```
from django.shortcuts import get_object_or_404
```

Y luego:

CÓDIGO

```
post = get_object_or_404(Post, slug=slug)
```

Error 2: olvidar pasar el slug en el enlace

Incorrecto:

CÓDIGO

```
<a href="{% url 'detalle_post' %}">
    {{ post.titulo }}
</a>
```

Correcto:

CÓDIGO

```
<a href="{% url 'detalle_post' post.slug %}">
    {{ post.titulo }}
</a>
```

La URL `detalle_post` necesita recibir un slug. Si no se lo pasamos, Django no puede construir el enlace.

Error 3: no crear la migración

Si agregamos el campo `slug` al modelo pero no ejecutamos migraciones, la base de datos no tendrá esa columna.

Debemos ejecutar:

CÓDIGO

```
python manage.py makemigrations
python manage.py migrate
```

Error 4: tener posts sin slug

Si ya teníamos posts creados antes de agregar el campo `slug`, puede ocurrir que algunos no tengan slug.

En ese caso, podemos entrar al admin, editar cada post y guardar nuevamente para que se genere el slug.

Otra opción es completar el slug manualmente desde el panel de administración.

21. Resumen final

En esta unidad aprendimos a crear páginas de detalle para nuestros posts usando URLs amigables con `slug`.

Vimos que:

- Un `slug` es una versión limpia de un texto para usar en una URL.
- Las URLs con slug son más claras que las URLs con ID.
- Django ofrece `SlugField` para guardar slugs en los modelos.
- Podemos usar `slugify` para generar slugs automáticamente.
- La URL puede capturar un slug usando `<slug: slug>`.
- La vista puede recibir ese slug como parámetro.
- `get_object_or_404` permite buscar un objeto o mostrar un error 404 si no existe.
- Los templates pueden generar enlaces dinámicos usando `{% url 'detalle_post' post.slug %}`.

Con esto, nuestro blog deja de tener rutas genéricas y empieza a comportarse como un sitio más profesional, con URLs claras, compartibles y fáciles de entender.

Editar y eliminar posts con formularios de confirmación

Aprende a cerrar el ciclo CRUD implementando las funciones de actualizar y borrar posts, asegurando la integridad de los datos y una buena experiencia de usuario.

Editar y eliminar posts: operaciones `Update` y `Delete`

En las unidades anteriores aprendimos a crear posts desde cero usando `ModelForm`. Eso nos permitió enviar datos desde el navegador y guardarlos en la base de datos.

Pero en una aplicación real, crear contenido no alcanza.

Un autor puede equivocarse al escribir un título, querer corregir el contenido de una publicación o decidir que un post ya no debería estar disponible.

Ahí aparecen dos operaciones fundamentales del ciclo CRUD:

CÓDIGO

```
Update - Actualizar
Delete - Eliminar
```

En esta unidad vamos a aprender cómo editar y eliminar posts existentes usando Django.

1. Recordemos el ciclo CRUD

CRUD es una sigla que representa las cuatro operaciones básicas que podemos hacer con datos:

CÓDIGO

```
Create - Crear
Read - Leer
Update - Actualizar
Delete - Eliminar
```

En nuestro blog, esto se puede traducir así:

CÓDIGO

```
Create: crear un nuevo post.
Read: ver la lista de posts o el detalle de un post.
Update: editar un post existente.
Delete: eliminar un post.
```

Hasta ahora ya trabajamos con:

- Crear posts.
- Listar posts.
- Ver el detalle de un post.

Ahora completaremos el circuito con:

- Editar posts.
- Eliminar posts.

2. La diferencia entre crear y editar

Crear y editar se parecen bastante, porque ambas acciones usan un formulario.

Pero hay una diferencia clave: **la instancia**.

Cuando creamos un post, el formulario aparece vacío.

CÓDIGO

```
form = PostForm()
```

Cuando editamos un post, el formulario debe aparecer cargado con los datos existentes.

CÓDIGO

```
form = PostForm(instance=post)
```

El argumento `instance` le dice a Django:

CÓDIGO

```
Este formulario no es para crear un objeto nuevo.
Este formulario es para modificar este objeto que ya existe.
```

Esa es la diferencia principal entre `Create` y `Update`.

3. Modelo de referencia

Seguiremos trabajando con el modelo `Post` que ya veníamos usando.

CÓDIGO

```
# posts/models.py

from django.db import models
from django.utils.text import slugify


class Post(models.Model):
    ESTADOS = [
        ("borrador", "Borrador"),
        ("publicado", "Publicado"),
        ("archivado", "Archivado"),
    ]

    titulo = models.CharField(max_length=200)
    contenido = models.TextField()
    autor = models.CharField(max_length=100)
    fecha_creacion = models.DateTimeField(auto_now_add=True)
    estado = models.CharField(max_length=20, choices=ESTADOS, default="borrador")
    slug = models.SlugField(max_length=220, unique=True, blank=True, null=True)

    def save(self, *args, **kwargs):
        if not self.slug:
            self.slug = slugify(self.titulo)

        super().save(*args, **kwargs)

    def __str__(self):
        return self.titulo
```

En este modelo, el `slug` se genera automáticamente cuando el post todavía no tiene uno.

Esto es importante: si editamos el título de un post, el slug no cambia automáticamente. Eso ayuda a mantener estable la URL del post.

4. Formulario de referencia

Usaremos el mismo `PostForm` que ya creamos antes.

CÓDIGO

```
# posts/forms.py

from django import forms
from .models import Post


class PostForm(forms.ModelForm):
    class Meta:
        model = Post
        fields = ["titulo", "contenido", "autor", "estado"]

        labels = {
            "titulo": "Título",
            "contenido": "Contenido",
            "autor": "Autor",
            "estado": "Estado",
        }

        widgets = {
            "titulo": forms.TextInput(attrs={
                "placeholder": "Escribe el título del post"
            }),
            "contenido": forms.Textarea(attrs={
                "placeholder": "Escribe el contenido del post",
                "rows": 6
            }),
            "autor": forms.TextInput(attrs={
                "placeholder": "Nombre del autor"
            }),
        }
```

Este formulario sirve tanto para crear como para editar.

La diferencia estará en cómo lo usamos dentro de la vista.

Parte 1: Editar posts

5. Crear la URL de edición

Primero necesitamos una ruta para editar un post específico.

Como venimos usando `slug`, la URL puede ser así:

CÓDIGO

```
/posts/mi-primer-post/editar/
```

En `posts/urls.py` agregamos:

CÓDIGO

```
# posts/urls.py

from django.urls import path
from . import views


urlpatterns = [
    path("", views.lista_posts, name="lista_posts"),
    path("crear/", views.crear_post, name="crear_post"),
    path("<slug:slug>/", views.detalle_post, name="detalle_post"),
    path("<slug:slug>/editar/", views.editar_post, name="editar_post"),
    path("<slug:slug>/eliminar/", views.eliminar_post, name="eliminar_post"),
]
```

Sin embargo, conviene ordenar las rutas de esta manera:

CÓDIGO

```
# posts/urls.py

from django.urls import path
from . import views


urlpatterns = [
    path("", views.lista_posts, name="lista_posts"),
    path("crear/", views.crear_post, name="crear_post"),
    path("<slug:slug>/editar/", views.editar_post, name="editar_post"),
    path("<slug:slug>/eliminar/", views.eliminar_post, name="eliminar_post"),
    path("<slug:slug>/", views.detalle_post, name="detalle_post"),
]
```

¿Por qué dejamos el detalle al final?

Porque esta ruta:

CÓDIGO

```
path("<slug:slug>/", views.detalle_post, name="detalle_post")
```

es muy general. Puede capturar muchas URLs. Por eso es buena práctica dejar primero las rutas más específicas y después las más generales.

6. Crear la vista para editar

En `posts/views.py` agregamos la vista `editar_post`.

CÓDIGO

```
# posts/views.py

from django.shortcuts import render, redirect, get_object_or_404
from .models import Post
from .forms import PostForm


def editar_post(request, slug):
    post = get_object_or_404(Post, slug=slug)

    if request.method == "POST":
        form = PostForm(request.POST, instance=post)

        if form.is_valid():
            form.save()
            return redirect("detalle_post", slug=post.slug)

    else:
        form = PostForm(instance=post)

    return render(request, "posts/post_form.html", {
        "form": form,
        "post": post,
        "titulo_pagina": "Editar post",
        "texto_boton": "Guardar cambios"
    })
```

Analicemos la vista paso a paso.

Primero buscamos el post que queremos editar:

CÓDIGO

```
post = get_object_or_404(Post, slug=slug)
```

Si existe, Django lo guarda en la variable `post`.

Si no existe, Django muestra una página 404.

7. Mostrar el formulario con datos existentes

Cuando el usuario entra a la página de edición, la petición es `GET`.

En ese caso hacemos:

CÓDIGO

```
form = PostForm(instance=post)
```

Esto genera un formulario con los datos actuales del post.

Por ejemplo, si el post tiene:

CÓDIGO

```
Título: Mi primer post
Autor: Ana
Estado: Publicado
```

Django mostrará esos valores cargados dentro del formulario.

8. Guardar los cambios enviados

Cuando el usuario modifica los datos y presiona “Guardar cambios”, el navegador envía una petición `POST`.

En ese caso hacemos:

CÓDIGO

```
form = PostForm(request.POST, instance=post)
```

Esta línea es clave.

Si escribimos solo esto:

CÓDIGO

```
form = PostForm(request.POST)
```

Django interpretará que queremos crear un post nuevo.

Pero si escribimos esto:

CÓDIGO

```
form = PostForm(request.POST, instance=post)
```

Django entiende que queremos actualizar el post existente.

Luego validamos:

CÓDIGO

```
if form.is_valid():
```

Y guardamos:

CÓDIGO

```
form.save()
```

Finalmente redirigimos al detalle del post:

CÓDIGO

```
return redirect("detalle_post", slug=post.slug)
```

9. Template para editar un post

Creamos el archivo:

CÓDIGO

```
posts/templates/posts/post_form.html
```

Contenido:

CÓDIGO

```
{/* posts/templates/posts/post_form.html */}

<h1>{{ titulo_pagina }}</h1>

<form method="post">
    {% csrf_token %}

    {{ form.as_p }}

    <button type="submit">{{ texto_boton }}</button>
</form>

<a href="{% url 'detalle_post' post.slug %}">Cancelar</a>
```

Este template es muy parecido al de creación.

La diferencia es que el formulario llega cargado con los datos existentes gracias a `instance=post`.

Parte 2: Eliminar posts

10. ¿Por qué eliminar requiere más cuidado?

Eliminar datos es una acción sensible.

No deberíamos borrar un post simplemente porque alguien entra a una URL como esta:

CÓDIGO

```
/posts/mi-primer-post/eliminar/
```

Eso sería peligroso.

Una eliminación nunca debería ejecutarse directamente por `GET`.

Lo correcto es este flujo:

1. El usuario hace clic en “Eliminar”.
2. Django muestra una página de confirmación.
3. El usuario confirma con un botón.
4. Ese botón envía una petición `POST`.
5. Recién ahí Django elimina el post.
6. Luego redirige al listado.

Esto evita borrados accidentales.

11. Crear la vista para eliminar

En `posts/views.py` agregamos:

CÓDIGO

```
# posts/views.py

from django.shortcuts import render, redirect, get_object_or_404
from .models import Post


def eliminar_post(request, slug):
    post = get_object_or_404(Post, slug=slug)

    if request.method == "POST":
        post.delete()
        return redirect("lista_posts")

    return render(request, "posts/post_confirm_delete.html", {
        "post": post
    })
```

Veamos el flujo.

Primero buscamos el post:

CÓDIGO

```
post = get_object_or_404(Post, slug=slug)
```

Si la petición es `POST`, eliminamos:

CÓDIGO

```
post.delete()
```

Y luego redirigimos:

CÓDIGO

```
return redirect("lista_posts")
```

Si la petición es `GET`, no borramos nada. Solo mostramos una página de confirmación.

CÓDIGO

```
return render(request, "posts/post_confirm_delete.html", {
    "post": post
})
```

12. Template de confirmación

Creamos el archivo:

CÓDIGO

```
posts/templates/posts/post_confirm_delete.html
```

Contenido:

CÓDIGO

```
{/* posts/templates/posts/post_confirm_delete.html */}

<h1>Eliminar post</h1>

<p>¿Estás seguro de que quieres eliminar el siguiente post?</p>

<h2>{{ post.titulo }}</h2>

<p>{{ post.contenido|truncatewords:30 }}</p>

<form method="post">
    {% csrf_token %}

    <button type="submit">Sí, eliminar</button>
</form>

<a href="{% url 'detalle_post' post.slug %}">Cancelar</a>
```

La eliminación real ocurre solo cuando el usuario presiona el botón:

CÓDIGO

```
<button type="submit">Sí, eliminar</button>
```

Como el formulario usa:

CÓDIGO

```
<form method="post">
```

Django recibirá una petición `POST` y recién ahí ejecutará:

CÓDIGO

```
post.delete()
```

13. Agregar enlaces en el detalle del post

Ahora podemos agregar botones o enlaces desde la página de detalle.

En `detalle_post.html`:

CÓDIGO

```
{/* posts/templates/posts/detalle_post.html */}

<h1>{{ post.titulo }}</h1>

<p>Autor: {{ post.autor }}</p>
<p>Estado: {{ post.estado }}</p>
<p>Fecha: {{ post.fecha_creacion }}</p>

<hr />

<p>{{ post.contenido }}</p>

<hr />

<a href="{% url 'editar_post' post.slug %}">Editar</a>
<a href="{% url 'eliminar_post' post.slug %}">Eliminar</a>
<a href="{% url 'lista_posts' %}">Volver al listado</a>
```

La ruta de edición recibe el slug:

CÓDIGO

```
<a href="{% url 'editar_post' post.slug %}">Editar</a>
```

La ruta de eliminación también recibe el slug:

CÓDIGO

```
<a href="{% url 'eliminar_post' post.slug %}">Eliminar</a>
```

14. Código completo de referencia

urls.py

CÓDIGO

```
from django.urls import path
from . import views


urlpatterns = [
    path("", views.lista_posts, name="lista_posts"),
    path("crear/", views.crear_post, name="crear_post"),
    path("<slug:slug>/editar/", views.editar_post, name="editar_post"),
    path("<slug:slug>/eliminar/", views.eliminar_post, name="eliminar_post"),
    path("<slug:slug>/", views.detalle_post, name="detalle_post"),
]
```

views.py

CÓDIGO

```
from django.shortcuts import render, redirect, get_object_or_404
from .models import Post
from .forms import PostForm


def lista_posts(request):
    posts = Post.objects.all().order_by("-fecha_creacion")

    return render(request, "posts/lista_posts.html", {
        "posts": posts
    })


def crear_post(request):
    if request.method == "POST":
        form = PostForm(request.POST)

        if form.is_valid():
            form.save()
            return redirect("lista_posts")

    else:
        form = PostForm()

    return render(request, "posts/post_form.html", {
        "form": form,
        "titulo_pagina": "Crear post",
        "texto_boton": "Crear"
    })


def detalle_post(request, slug):
    post = get_object_or_404(Post, slug=slug)

    return render(request, "posts/detalle_post.html", {
        "post": post
    })


def editar_post(request, slug):
    post = get_object_or_404(Post, slug=slug)

    if request.method == "POST":
        form = PostForm(request.POST, instance=post)

        if form.is_valid():
            form.save()
            return redirect("detalle_post", slug=post.slug)

    else:
        form = PostForm(instance=post)
```

```
    return render(request, "posts/post_form.html", {
        "form": form,
        "post": post,
        "titulo_pagina": "Editar post",
        "texto_boton": "Guardar cambios"
    })


def eliminar_post(request, slug):
    post = get_object_or_404(Post, slug=slug)

    if request.method == "POST":
        post.delete()
        return redirect("lista_posts")

    return render(request, "posts/post_confirm_delete.html", {
        "post": post
    })
```

detalle_post.html

CÓDIGO

```
<h1>{{ post.titulo }}</h1>

<p>Autor: {{ post.autor }}</p>
<p>Estado: {{ post.estado }}</p>
<p>Fecha: {{ post.fecha_creacion }}</p>

<hr />

<p>{{ post.contenido }}</p>

<hr />

<a href="{% url 'editar_post' post.slug %}">Editar</a>
<a href="{% url 'eliminar_post' post.slug %}">Eliminar</a>
<a href="{% url 'lista_posts' %}">Volver al listado</a>
```

post_form.html

CÓDIGO

```
<h1>{{ titulo_pagina }}</h1>

<form method="post">
    {% csrf_token %}

    {{ form.as_p }}

    <button type="submit">{{ texto_boton }}</button>
</form>

<a href="{% url 'detalle_post' post.slug %}">Cancelar</a>
```

post_confirm_delete.html

CÓDIGO

```
<h1>Eliminar post</h1>

<p>¿Estás seguro de que quieres eliminar el siguiente post?</p>

<h2>{{ post.titulo }}</h2>

<p>{{ post.contenido|truncatewords:30 }}</p>

<form method="post">
    {% csrf_token %}

    <button type="submit">Sí, eliminar</button>
</form>

<a href="{% url 'detalle_post' post.slug %}">Cancelar</a>
```

15. El flujo completo de edición

Cuando el usuario quiere editar un post:

1. Entra al detalle del post.
2. Hace clic en “Editar”.
3. Django recibe una petición `GET`.
4. La vista busca el post usando el slug.
5. Django crea un formulario con `instance=post`.
6. El template muestra el formulario cargado con los datos actuales.
7. El usuario modifica los campos.
8. El navegador envía una petición `POST`.
9. Django crea el formulario con `request.POST` e `instance=post`.

10. Se ejecuta `form.is_valid()`.
11. Si los datos son válidos, se ejecuta `form.save()`.
12. Django actualiza el registro existente.
13. El usuario es redirigido al detalle del post.

16. El flujo completo de eliminación

Cuando el usuario quiere eliminar un post:

1. Entra al detalle del post.
2. Hace clic en “Eliminar”.
3. Django recibe una petición `GET`.
4. La vista busca el post usando el slug.
5. Django muestra una pantalla de confirmación.
6. El usuario presiona “Sí, eliminar”.
7. El navegador envía una petición `POST`.
8. La vista ejecuta `post.delete()`.
9. El post se elimina de la base de datos.
10. El usuario es redirigido al listado de posts.

17. Errores comunes

Error 1: olvidar `instance` al editar

Incorrecto:

CÓDIGO

```
form = PostForm(request.POST)
```

Esto puede crear un nuevo post en lugar de editar el existente.

Correcto:

CÓDIGO

```
form = PostForm(request.POST, instance=post)
```

Y para mostrar el formulario cargado:

CÓDIGO

```
form = PostForm(instance=post)
```

Error 2: borrar usando `GET`

Incorrecto:

CÓDIGO

```
def eliminar_post(request, slug):
    post = get_object_or_404(Post, slug=slug)
    post.delete()
    return redirect("lista_posts")
```

Este código borra el post apenas alguien entra a la URL.

Correcto:

CÓDIGO

```
def eliminar_post(request, slug):
    post = get_object_or_404(Post, slug=slug)

    if request.method == "POST":
        post.delete()
        return redirect("lista_posts")

    return render(request, "posts/post_confirm_delete.html", {
        "post": post
    })
```

Error 3: olvidar el `csrf_token`

Incorrecto:

CÓDIGO

```
<form method="post">
    <button type="submit">Sí, eliminar</button>
</form>
```

Correcto:

CÓDIGO

```
<form method="post">
    {% csrf_token %}
    <button type="submit">Sí, eliminar</button>
</form>
```

Error 4: no redirigir después de guardar o borrar

Después de editar o eliminar correctamente, conviene redirigir al usuario.

Después de editar:

CÓDIGO

```
return redirect("detalle_post", slug=post.slug)
```

Después de eliminar:

CÓDIGO

```
return redirect("lista_posts")
```

Esto evita pantallas confusas y mejora la navegación.

Error 5: escribir mal el nombre de la URL

Si en `urls.py` definimos:

CÓDIGO

```
path("<slug:slug>/editar/", views.editar_post, name="editar_post")
```

En el template debemos usar exactamente ese nombre:

CÓDIGO

```
<a href="{% url 'editar_post' post.slug %}">Editar</a>
```

Si escribimos otro nombre, Django no podrá construir la URL.

18. Buenas prácticas

Al trabajar con edición y eliminación en Django, recuerda:

- Usar `get_object_or_404` para buscar el post.
- Usar `instance=post` al editar.
- Validar siempre con `form.is_valid()`.
- No borrar nunca directamente por `GET`.
- Mostrar una página de confirmación antes de borrar.
- Usar `POST` para acciones que modifican datos.
- Incluir `{% csrf_token %}` en formularios `POST`.
- Redirigir después de editar o eliminar.
- Crear enlaces usando `{% url %}` en lugar de escribir rutas manualmente.

19. Resumen final

En esta unidad completamos las operaciones principales del CRUD para nuestro blog.

Aprendimos que editar y eliminar son acciones diferentes a crear.

Para editar, usamos el mismo `PostForm`, pero le pasamos una instancia existente:

CÓDIGO

```
form = PostForm(instance=post)
```

Y al guardar cambios:

CÓDIGO

```
form = PostForm(request.POST, instance=post)
```

Para eliminar, primero mostramos una pantalla de confirmación y solo borramos el post si el usuario envía una petición `POST`.

CÓDIGO

```
if request.method == "POST":
    post.delete()
```

Con esto, nuestro blog ya permite crear, leer, actualizar y eliminar publicaciones desde páginas propias de Django.

A partir de ahora, la aplicación empieza a sentirse mucho más real: ya no dependemos solamente del panel de administración, sino que construimos nuestras propias pantallas para gestionar contenido.

Subir imágenes y servir archivos media en Django

Aprende a configurar Django para permitir la subida de imágenes en tus posts, comprendiendo la diferencia entre archivos estáticos y media, y cómo servirlos correctamente en el navegador.

Hasta ahora, nuestro blog trabaja principalmente con texto: títulos, contenido, autores, estados y fechas.

Pero un blog real suele necesitar imágenes.

Una imagen de portada puede hacer que un post se vea más atractivo, más profesional y más fácil de identificar. No es lo mismo mostrar una lista de artículos solo con texto que mostrar cada publicación acompañada de una imagen destacada.

En esta unidad aprenderemos a permitir que cada post tenga su propia imagen.

Para lograrlo, necesitamos entender cómo Django maneja los archivos subidos por usuarios o administradores.

1. Archivos estáticos vs. archivos media

Antes de escribir código, necesitamos diferenciar dos conceptos importantes:

CÓDIGO

```
Archivos estáticos
Archivos media
```

Aunque ambos son archivos externos al código Python, no se usan para lo mismo.

2. Archivos estáticos

Los archivos estáticos son archivos que forman parte del diseño o funcionamiento visual del sitio.

Son archivos que el desarrollador agrega al proyecto.

Ejemplos:

CÓDIGO

```
CSS
JavaScript
Logos del sitio
Íconos
Imágenes fijas del diseño
Fuentes
```

Por ejemplo:

CÓDIGO

```
static/posts/css/styles.css
static/posts/img/logo.png
```

Estos archivos no cambian porque un usuario complete un formulario. Cambian cuando el desarrollador modifica el proyecto.

3. Archivos media

Los archivos media son archivos que se suben mientras la aplicación está funcionando.

Normalmente los sube un usuario, un administrador o alguien que carga contenido desde un formulario.

Ejemplos:

CÓDIGO

```
Imagen destacada de un post
Imagen de usuario
PDF adjunto
Foto de perfil
Imagen adicional de un post
```

Por ejemplo, si desde un formulario cargamos una imagen para un post, esa imagen no debería ir a la carpeta `static`.

Debería guardarse dentro de una carpeta especial para archivos subidos, normalmente llamada:

CÓDIGO

```
media/
```

4. Diferencia principal

La diferencia puede resumirse así:

CÓDIGO

```
STATIC = archivos del proyecto
MEDIA = archivos subidos por usuarios o administradores
```

Ejemplo práctico:

CÓDIGO

```
El logo del blog → static
La imagen de portada de un post → media
```

Esta diferencia es muy importante porque Django los configura de manera distinta.

5. Configurar `MEDIA_URL` y `MEDIA_ROOT`

Para que Django pueda guardar y mostrar imágenes subidas, debemos configurar dos variables en `settings.py`.

Abrimos:

CÓDIGO

```
blog_project/settings.py
```

Y agregamos:

CÓDIGO

```
MEDIA_URL = "/media/"
MEDIA_ROOT = BASE_DIR / "media"
```

Veamos qué significa cada una.

6. `MEDIA_ROOT`

`MEDIA_ROOT` indica la ubicación física donde Django guardará los archivos subidos.

CÓDIGO

```
MEDIA_ROOT = BASE_DIR / "media"
```

Esto significa que Django creará o usará una carpeta llamada `media` en la raíz del proyecto.

Por ejemplo:

CÓDIGO

```
blog_project/
│
├── manage.py
├── db.sqlite3
├── media/
│   └── posts/
│       └── mi-imagen.jpg
│
├── blog_project/
└── posts/
```

Esa carpeta contiene archivos reales dentro de nuestra computadora.

7. `MEDIA_URL`

`MEDIA_URL` indica la URL pública desde la cual el navegador podrá pedir esos archivos.

CÓDIGO

```
MEDIA_URL = "/media/"
```

Por ejemplo, si se sube esta imagen:

CÓDIGO

```
media/posts/python.jpg
```

El navegador podrá acceder a ella desde una URL parecida a esta:

CÓDIGO

```
http://127.0.0.1:8000/media/posts/python.jpg
```

Entonces:

CÓDIGO

```
MEDIA_ROOT = dónde se guarda físicamente el archivo
MEDIA_URL = desde dónde se accede en el navegador
```

8. Configurar las URLs del proyecto para desarrollo

Por defecto, Django no sirve archivos media automáticamente en desarrollo.

Para habilitarlo, debemos modificar el archivo `urls.py` principal del proyecto, no el de la app.

Abrimos:

CÓDIGO

```
blog_project/urls.py
```

Y agregamos esta configuración:

CÓDIGO

```
# blog_project/urls.py

from django.contrib import admin
from django.urls import path, include

from django.conf import settings
from django.conf.urls.static import static


urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("posts.urls")),
]


if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

La parte importante es esta:

CÓDIGO

```
if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

Esto le dice a Django:

CÓDIGO

```
Mientras estemos en desarrollo, si alguien pide una URL que empieza con /media/,
buscá el archivo dentro de MEDIA_ROOT.
```

9. Esta configuración es solo para desarrollo

La configuración anterior sirve para trabajar localmente con:

CÓDIGO

```
python manage.py runserver
```

En producción, normalmente no se sirven archivos media directamente desde Django.

En un servidor real se suelen usar herramientas como:

CÓDIGO

```
Nginx
Apache
Amazon S3
Cloudinary
DigitalOcean Spaces
```

Pero para este curso y para desarrollo local, la configuración con `static()` es suficiente.

10. Instalar Pillow

Para trabajar con campos de imagen, Django necesita una librería llamada `Pillow`.

Pillow permite que Python pueda procesar imágenes.

La instalamos con:

CÓDIGO

```
pip install Pillow
```

Luego, si estamos manteniendo un archivo de dependencias, podemos actualizarlo con:

CÓDIGO

```
pip freeze > requirements.txt
```

Si no instalamos Pillow, Django mostrará un error al intentar usar `ImageField`.

11. Agregar una imagen al modelo `Post`

Ahora vamos a modificar el modelo `Post` para que pueda tener una imagen destacada.

En `posts/models.py`:

CÓDIGO

```
# posts/models.py

from django.db import models
from django.utils.text import slugify


class Post(models.Model):
    ESTADOS = [
        ("borrador", "Borrador"),
        ("publicado", "Publicado"),
    ]

    titulo = models.CharField(max_length=200)
    contenido = models.TextField()
    autor = models.CharField(max_length=100)
    fecha_creacion = models.DateTimeField(auto_now_add=True)
    estado = models.CharField(max_length=20, choices=ESTADOS, default="borrador")
    slug = models.SlugField(max_length=220, unique=True, blank=True, null=True)
    imagen = models.ImageField(upload_to="posts/", null=True, blank=True)

    def save(self, *args, **kwargs):
        if not self.slug:
            self.slug = slugify(self.titulo)

        super().save(*args, **kwargs)

    def __str__(self):
        return self.titulo
```

El campo nuevo es este:

CÓDIGO

```
imagen = models.ImageField(upload_to="posts/", null=True, blank=True)
```

12. ¿Qué significa `ImageField`?

`ImageField` es un campo especial de Django para trabajar con imágenes.

CÓDIGO

```
models.ImageField()
```

No guarda la imagen completa dentro de la base de datos.

Lo que guarda en la base de datos es la ruta del archivo.

Por ejemplo:

CÓDIGO

```
posts/python.jpg
```

La imagen real se guarda en la carpeta `media`.

13. ¿Qué significa `upload_to`?

El atributo `upload_to` indica en qué subcarpeta dentro de `media` se guardará el archivo.

CÓDIGO

```
upload_to="posts/"
```

Esto significa que las imágenes de posts se guardarán en:

CÓDIGO

```
media/posts/
```

Esto nos ayuda a mantener los archivos organizados.

Más adelante podríamos tener otras carpetas:

CÓDIGO

```
media/posts/
media/imagenes/
media/documentos/
```

14. ¿Por qué usamos `null=True` y `blank=True`?

Usamos:

CÓDIGO

```
null=True
blank=True
```

para que la imagen no sea obligatoria.

Esto permite crear posts sin imagen.

CÓDIGO

```
imagen = models.ImageField(upload_to="posts/", null=True, blank=True)
```

`blank=True` permite dejar el campo vacío en formularios.

`null=True` permite que la base de datos guarde el valor como nulo.

Esto es útil especialmente si ya teníamos posts creados antes de agregar el campo de imagen.

15. Crear y aplicar migraciones

Cada vez que modificamos un modelo, debemos crear una migración.

En la terminal:

CÓDIGO

```
python manage.py makemigrations
```

Luego aplicamos la migración:

CÓDIGO

```
python manage.py migrate
```

Con esto, la base de datos queda actualizada y el modelo `Post` ya puede almacenar la referencia a una imagen.

16. Agregar la imagen al formulario

Si estamos usando un `ModelForm`, debemos incluir el campo `imagen`.

En `posts/forms.py`:

CÓDIGO

```
# posts/forms.py

from django import forms
from .models import Post


class PostForm(forms.ModelForm):
    class Meta:
        model = Post
        fields = ["titulo", "contenido", "autor", "estado", "imagen"]

        labels = {
            "titulo": "Título",
            "contenido": "Contenido",
            "autor": "Autor",
            "estado": "Estado",
            "imagen": "Imagen destacada",
        }

        widgets = {
            "titulo": forms.TextInput(attrs={
                "placeholder": "Escribe el título del post"
            }),
            "contenido": forms.Textarea(attrs={
                "placeholder": "Escribe el contenido del post",
                "rows": 6
            }),
            "autor": forms.TextInput(attrs={
                "placeholder": "Nombre del autor"
            }),
        }
```

La parte importante es que ahora agregamos `"imagen"` en `fields`:

CÓDIGO

```
fields = ["titulo", "contenido", "autor", "estado", "imagen"]
```

Si nos olvidamos de incluirlo, el formulario no mostrará el campo para subir la imagen.

17. Modificar la vista de creación

Para recibir archivos, no alcanza con usar `request.POST`.

También debemos usar:

CÓDIGO

```
request.FILES
```

En `posts/views.py`:

CÓDIGO

```
# posts/views.py

from django.shortcuts import render, redirect, get_object_or_404
from .models import Post
from .forms import PostForm


def crear_post(request):
    if request.method == "POST":
        form = PostForm(request.POST, request.FILES)

        if form.is_valid():
            form.save()
            return redirect("lista_posts")

    else:
        form = PostForm()

    return render(request, "posts/post_form.html", {
        "form": form,
        "titulo_pagina": "Crear post",
        "texto_boton": "Crear"
    })
```

La línea clave es:

CÓDIGO

```
form = PostForm(request.POST, request.FILES)
```

`request.POST` contiene los datos de texto.

Por ejemplo:

CÓDIGO

```
titulo
contenido
autor
estado
```

`request.FILES` contiene los archivos enviados.

Por ejemplo:

CÓDIGO

```
imagen
```

18. Modificar la vista de edición

También debemos actualizar la vista de edición.

Si queremos permitir cambiar la imagen de un post existente, necesitamos pasar `request.FILES`.

CÓDIGO

```
# posts/views.py

def editar_post(request, slug):
    post = get_object_or_404(Post, slug=slug)

    if request.method == "POST":
        form = PostForm(request.POST, request.FILES, instance=post)

        if form.is_valid():
            form.save()
            return redirect("detalle_post", slug=post.slug)

    else:
        form = PostForm(instance=post)

    return render(request, "posts/post_form.html", {
        "form": form,
        "post": post,
        "titulo_pagina": "Editar post",
        "texto_boton": "Guardar cambios"
    })
```

La línea importante es:

CÓDIGO

```
form = PostForm(request.POST, request.FILES, instance=post)
```

Esto le dice a Django:

CÓDIGO

```
Actualizá este post existente usando los datos de texto y los archivos enviados.
```

19. Modificar el formulario HTML

Para que el navegador pueda enviar archivos, el formulario HTML necesita un atributo especial:

CÓDIGO

```
enctype="multipart/form-data"
```

Sin este atributo, el navegador enviará los textos, pero no enviará la imagen.

20. Template para crear post con imagen

En `posts/templates/posts/post_form.html`:

CÓDIGO

```
<h1>{{ titulo_pagina }}</h1>

<form method="post" enctype="multipart/form-data">
    {% csrf_token %}

    {{ form.as_p }}

    <button type="submit">{{ texto_boton }}</button>
</form>

<a href="{% url 'lista_posts' %}">Volver al listado</a>
```

La línea clave es:

CÓDIGO

```
<form method="post" enctype="multipart/form-data">
```

21. Reutilizar `post_form.html` para editar post con imagen

En `posts/templates/posts/post_form.html` usamos el mismo archivo que para crear. La vista de edición envía `titulo_pagina` y `texto_boton` con valores distintos, y además puede enviar `post` para mostrar la imagen actual:

CÓDIGO

```
<h1>{{ titulo_pagina }}</h1>



{% if post and post.imagen %}
    <p>Imagen actual:</p>
    <img src="{{ post.imagen.url }}" alt="{{ post.titulo }}" width="300" />
{% endif %}



<form method="post" enctype="multipart/form-data">
    {% csrf_token %}

    {{ form.as_p }}

    <button type="submit">{{ texto_boton }}</button>
</form>



{% if post %}
    <a href="{% url 'detalle_post' post.slug %}">Cancelar edición</a>
{% else %}
    <a href="{% url 'lista_posts' %}">Cancelar creación</a>
{% endif %}
```

También acá usamos:

CÓDIGO

```
enctype="multipart/form-data"
```

porque el usuario podría subir una nueva imagen al editar.

22. Mostrar la imagen en el detalle del post

Ahora que el post puede tener imagen, podemos mostrarla en el template de detalle.

En `posts/templates/posts/detalle_post.html`:

CÓDIGO

```
<h1>{{ post.titulo }}</h1>

<p>Autor: {{ post.autor }}</p>
<p>Estado: {{ post.estado }}</p>
<p>Fecha: {{ post.fecha_creacion }}</p>

{% if post.imagen %}
    <img src="{{ post.imagen.url }}" alt="{{ post.titulo }}" width="600" />
{% endif %}

<hr />

<p>{{ post.contenido }}</p>

<hr />

<a href="{% url 'editar_post' post.slug %}">Editar</a>
<a href="{% url 'eliminar_post' post.slug %}">Eliminar</a>
<a href="{% url 'lista_posts' %}">Volver al listado</a>
```

La parte más importante es:

CÓDIGO

```
{% if post.imagen %}
    <img src="{{ post.imagen.url }}" alt="{{ post.titulo }}" width="600" />
{% endif %}
```

Primero verificamos si el post tiene imagen.

Después usamos:

CÓDIGO

```
{{ post.imagen.url }}
```

Esto devuelve la URL de la imagen.

Por ejemplo:

CÓDIGO

```
/media/posts/python.jpg
```

23. ¿Por qué usamos `if post.imagen`?

Si intentamos acceder a:

CÓDIGO

```
{{ post.imagen.url }}
```

pero el post no tiene imagen, Django puede mostrar un error.

Por eso siempre conviene verificar primero:

CÓDIGO

```
{% if post.imagen %}
```

Así evitamos que la página se rompa cuando un post no tiene imagen cargada.

24. Mostrar imagen en el listado de posts

También podemos mostrar una miniatura en la lista.

En `posts/templates/posts/lista_posts.html`:

CÓDIGO

```
<h1>Listado de posts</h1>

<a href="{% url 'crear_post' %}">Crear nuevo post</a>

{% for post in posts %}
    <article>
        {% if post.imagen %}
            <img src="{{ post.imagen.url }}" alt="{{ post.titulo }}" width="250" />
        {% endif %}

        <h2>
            <a href="{% url 'detalle_post' post.slug %}">
                {{ post.titulo }}
            </a>
        </h2>

        <p>{{ post.contenido|truncatewords:30 }}</p>
        <p>Autor: {{ post.autor }}</p>
        <p>Estado: {{ post.estado }}</p>
        <p>Fecha: {{ post.fecha_creacion }}</p>
    </article>
{% empty %}
    <p>No hay posts cargados.</p>
{% endfor %}
```

Esto permite que el listado se vea más visual.

25. Usar una imagen por defecto

A veces queremos mostrar una imagen por defecto cuando el post no tiene imagen cargada.

Esa imagen por defecto debería ir en la carpeta `static`, porque forma parte del diseño del sitio.

Por ejemplo:

CÓDIGO

```
posts/static/posts/img/default-post.png
```

En el template debemos cargar `static`:

CÓDIGO

```
{% load static %}

{% if post.imagen %}
    <img src="{{ post.imagen.url }}" alt="{{ post.titulo }}" width="600" />
{% else %}
    <img src="{% static 'posts/img/default-post.png' %}" alt="Imagen por defecto"
width="600" />
{% endif %}
```

Recordemos:

CÓDIGO

```
La imagen subida por el usuario va en media.
La imagen por defecto del diseño va en static.
```

26. Cuidado con `className`

En algunos ejemplos de internet puede aparecer algo como esto:

CÓDIGO

```
<img src="{{ post.imagen.url }}" alt="{{ post.titulo }}" className="img-fluid" />
```

Eso no es HTML común de Django.

`className` se usa en React.

En templates HTML de Django debemos escribir:

CÓDIGO

```
<img src="{{ post.imagen.url }}" alt="{{ post.titulo }}" class="img-fluid" />
```

Si estamos usando Bootstrap, `img-fluid` permite que la imagen se adapte mejor al ancho disponible.

27. Código completo de referencia

settings.py

CÓDIGO

```
MEDIA_URL = "/media/"
MEDIA_ROOT = BASE_DIR / "media"
```

blog_project/urls.py

CÓDIGO

```
from django.contrib import admin
from django.urls import path, include

from django.conf import settings
from django.conf.urls.static import static


urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("posts.urls")),
]


if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

models.py

CÓDIGO

```
from django.db import models
from django.utils.text import slugify


class Post(models.Model):
    ESTADOS = [
        ("borrador", "Borrador"),
        ("publicado", "Publicado"),
        ("archivado", "Archivado"),
    ]

    titulo = models.CharField(max_length=200)
    contenido = models.TextField()
    autor = models.CharField(max_length=100)
    fecha_creacion = models.DateTimeField(auto_now_add=True)
    estado = models.CharField(max_length=20, choices=ESTADOS, default="borrador")
    slug = models.SlugField(max_length=220, unique=True, blank=True, null=True)
    imagen = models.ImageField(upload_to="posts/", null=True, blank=True)

    def save(self, *args, **kwargs):
        if not self.slug:
            self.slug = slugify(self.titulo)

        super().save(*args, **kwargs)

    def __str__(self):
        return self.titulo
```

forms.py

CÓDIGO

```
from django import forms
from .models import Post


class PostForm(forms.ModelForm):
    class Meta:
        model = Post
        fields = ["titulo", "contenido", "autor", "estado", "imagen"]

        labels = {
            "titulo": "Título",
            "contenido": "Contenido",
            "autor": "Autor",
            "estado": "Estado",
            "imagen": "Imagen destacada",
        }

        widgets = {
            "titulo": forms.TextInput(attrs={
                "placeholder": "Escribe el título del post"
            }),
            "contenido": forms.Textarea(attrs={
                "placeholder": "Escribe el contenido del post",
                "rows": 6
            }),
            "autor": forms.TextInput(attrs={
                "placeholder": "Nombre del autor"
            }),
        }
```

views.py

CÓDIGO

```
from django.shortcuts import render, redirect, get_object_or_404
from .models import Post
from .forms import PostForm


def lista_posts(request):
    posts = Post.objects.all().order_by("-fecha_creacion")

    return render(request, "posts/lista_posts.html", {
        "posts": posts
    })


def crear_post(request):
    if request.method == "POST":
        form = PostForm(request.POST, request.FILES)

        if form.is_valid():
            form.save()
            return redirect("lista_posts")

    else:
        form = PostForm()

    return render(request, "posts/post_form.html", {
        "form": form,
        "titulo_pagina": "Crear post",
        "texto_boton": "Crear"
    })


def detalle_post(request, slug):
    post = get_object_or_404(Post, slug=slug)

    return render(request, "posts/detalle_post.html", {
        "post": post
    })


def editar_post(request, slug):
    post = get_object_or_404(Post, slug=slug)

    if request.method == "POST":
        form = PostForm(request.POST, request.FILES, instance=post)

        if form.is_valid():
            form.save()
            return redirect("detalle_post", slug=post.slug)

    else:
        form = PostForm(instance=post)
```

```
    return render(request, "posts/post_form.html", {
        "form": form,
        "post": post,
        "titulo_pagina": "Editar post",
        "texto_boton": "Guardar cambios"
    })


def eliminar_post(request, slug):
    post = get_object_or_404(Post, slug=slug)

    if request.method == "POST":
        post.delete()
        return redirect("lista_posts")

    return render(request, "posts/post_confirm_delete.html", {
        "post": post
    })
```

post_form.html

CÓDIGO

```
<h1>{{ titulo_pagina }}</h1>



{% if post and post.imagen %}
    <p>Imagen actual:</p>
    <img src="{{ post.imagen.url }}" alt="{{ post.titulo }}" width="300" />
{% endif %}



<form method="post" enctype="multipart/form-data">
    {% csrf_token %}

    {{ form.as_p }}

    <button type="submit">{{ texto_boton }}</button>
</form>



{% if post %}
    <a href="{% url 'detalle_post' post.slug %}">Cancelar edición</a>
{% else %}
    <a href="{% url 'lista_posts' %}">Cancelar creación</a>
{% endif %}
```

detalle_post.html

CÓDIGO

```
<h1>{{ post.titulo }}</h1>

<p>Autor: {{ post.autor }}</p>
<p>Estado: {{ post.estado }}</p>
<p>Fecha: {{ post.fecha_creacion }}</p>

{% if post.imagen %}
    <img src="{{ post.imagen.url }}" alt="{{ post.titulo }}" width="600" />
{% endif %}

<hr />

<p>{{ post.contenido }}</p>

<hr />

<a href="{% url 'editar_post' post.slug %}">Editar</a>
<a href="{% url 'eliminar_post' post.slug %}">Eliminar</a>
<a href="{% url 'lista_posts' %}">Volver al listado</a>
```

lista_posts.html

CÓDIGO

```
<h1>Listado de posts</h1>

<a href="{% url 'crear_post' %}">Crear nuevo post</a>

{% for post in posts %}
    <article>
        {% if post.imagen %}
            <img src="{{ post.imagen.url }}" alt="{{ post.titulo }}" width="250" />
        {% endif %}

        <h2>
            <a href="{% url 'detalle_post' post.slug %}">
                {{ post.titulo }}
            </a>
        </h2>

        <p>{{ post.contenido|truncatewords:30 }}</p>
        <p>Autor: {{ post.autor }}</p>
        <p>Estado: {{ post.estado }}</p>
        <p>Fecha: {{ post.fecha_creacion }}</p>
    </article>
{% empty %}
    <p>No hay posts cargados.</p>
{% endfor %}
```

28. Agregar `media/` al `.gitignore`

La carpeta `media/` contiene archivos subidos durante el uso de la aplicación.

En general, no conviene subir esa carpeta al repositorio.

Podemos agregar al archivo `.gitignore`:

CÓDIGO

```
media/
```

Esto evita subir imágenes de prueba o archivos cargados localmente.

Importante: la imagen por defecto del diseño no debería estar en `media/`, sino en `static/`, porque sí forma parte del proyecto.

29. Errores comunes

Error 1: olvidar instalar Pillow

Si usamos `ImageField`, necesitamos instalar Pillow.

CÓDIGO

```
pip install Pillow
```

Error 2: olvidar las migraciones

Después de agregar `imagen` al modelo, debemos ejecutar:

CÓDIGO

```
python manage.py makemigrations
python manage.py migrate
```

Error 3: olvidar `request.FILES`

Incorrecto:

CÓDIGO

```
form = PostForm(request.POST)
```

Correcto:

CÓDIGO

```
form = PostForm(request.POST, request.FILES)
```

Esto aplica especialmente al crear y editar posts con imagen.

Error 4: olvidar `enctype`

Incorrecto:

CÓDIGO

```
<form method="post">
```

Correcto:

CÓDIGO

```
<form method="post" enctype="multipart/form-data">
```

Sin `enctype="multipart/form-data"`, el navegador no envía el archivo.

Error 5: usar `post.imagen.url` sin verificar si existe imagen

Incorrecto:

CÓDIGO

```
<img src="{{ post.imagen.url }}" alt="{{ post.titulo }}" />
```

Correcto:

CÓDIGO

```
{% if post.imagen %}
    <img src="{{ post.imagen.url }}" alt="{{ post.titulo }}" />
{% endif %}
```

Error 6: confundir `MEDIA_ROOT` con `MEDIA_URL`

`MEDIA_ROOT` es una ruta del sistema de archivos.

Ejemplo:

CÓDIGO

```
C:/Users/usuario/proyecto/media/
```

`MEDIA_URL` es una ruta web.

Ejemplo:

CÓDIGO

```
/media/
```

No cumplen la misma función.

Error 7: configurar media en el archivo `urls.py` incorrecto

La configuración de media para desarrollo va en el `urls.py` principal del proyecto.

Por ejemplo:

CÓDIGO

```
blog_project/urls.py
```

No en:

CÓDIGO

```
posts/urls.py
```

30. Buenas prácticas

Al trabajar con imágenes en Django, conviene recordar:

- Usar `ImageField` para imágenes.
- Instalar `Pillow`.
- Configurar `MEDIA_URL` y `MEDIA_ROOT`.
- Agregar la configuración de media en el `urls.py` principal.
- Pasar `request.FILES` al formulario.
- Usar `enctype="multipart/form-data"` en el HTML.
- Verificar `{% if post.imagen %}` antes de mostrar la imagen.
- Usar el atributo `alt` en las imágenes.
- No subir la carpeta `media/` al repositorio.
- Guardar imágenes por defecto en `static/`, no en `media/`.

31. Resumen final

En esta unidad aprendimos cómo permitir que nuestro blog trabaje con imágenes subidas desde formularios.

Vimos que Django separa los archivos en dos grandes grupos:

CÓDIGO

```
STATIC: archivos propios del proyecto.
MEDIA: archivos subidos por usuarios o administradores.
```

También configuramos:

CÓDIGO

```
MEDIA_URL = "/media/"
MEDIA_ROOT = BASE_DIR / "media"
```

Agregamos el campo de imagen al modelo:

CÓDIGO

```
imagen = models.ImageField(upload_to="posts/", null=True, blank=True)
```

Actualizamos el formulario:

CÓDIGO

```
fields = ["titulo", "contenido", "autor", "estado", "imagen"]
```

Y modificamos las vistas para recibir archivos:

CÓDIGO

```
form = PostForm(request.POST, request.FILES)
```

Finalmente, mostramos la imagen en el template usando:

CÓDIGO

```
{% if post.imagen %}
    <img src="{{ post.imagen.url }}" alt="{{ post.titulo }}" />
{% endif %}
```

Con esto, nuestro blog deja de ser solamente textual y empieza a manejar contenido visual, algo fundamental para que una aplicación web se sienta más completa y profesional.

Flashcards de repaso

Unidad de repaso mediante tarjetas para consolidar la creación, edición, eliminación y subida de imágenes en aplicaciones Django.

Repaso: Consolidando el CRUD de Posts y Gestión Multimedia

Hemos recorrido un camino esencial para cualquier desarrollador web: entender cómo los usuarios interactúan con nuestros datos. Desde la creación de formularios simples hasta la gestión de archivos multimedia, hemos construido las bases de una aplicación dinámica.

¿Por qué repasar estos conceptos?

El desarrollo web suele dividirse en la interacción entre el **modelo de datos** (lo que guardamos en la base de datos), la **vista** (lo que el usuario ve) y el **controlador** (que orquesta la lógica). Al trabajar con un CRUD (*Create*, *Read*, *Update*, *Delete*) y archivos, estos componentes deben trabajar en sincronía perfecta. Si uno falla, el usuario final experimenta errores o datos corruptos.

Estas tarjetas de repaso están diseñadas para fijar los conceptos fundamentales que conectan estos mundos.

Cómo utilizar estas tarjetas

1. **Recuerdo activo:** Antes de voltear la tarjeta, intenta responder a la pregunta en voz alta o por escrito. Cuanto más esfuerzo hagas por recuperar la información, más fuerte será tu memoria.
2. **Conecta con la práctica:** Si no recuerdas una respuesta, no solo leas el reverso; piensa en el código que escribiste en las unidades anteriores. ¿Cómo se llamaba ese atributo? ¿Qué método específico usaste en la vista?
3. **No memorices mecánicamente:** Enfócate en el **porqué**. Si entiendes por qué Django necesita `request.FILES` o por qué un `ModelForm` simplifica el trabajo, no necesitarás memorizar la sintaxis, será algo natural.

¡Es momento de poner a prueba lo aprendido durante este módulo!

01 ¿Qué significan las siglas CRUD en el desarrollo web y por qué son importantes?

Significa **Create, Read, Update, Delete**. Son las cuatro operaciones básicas que permiten gestionar los datos de una aplicación web, como crear un post, leerlo, editarlo o borrarlo.

02 ¿Qué es un ModelForm en Django y cuál es su principal ventaja?

Es una clase de Django que permite crear formularios automáticamente basados en un modelo de la base de datos. Reduce la duplicidad de código al inferir automáticamente los campos y tipos de datos a partir del `models.py`.

03 ¿Qué requisito especial es necesario en la etiqueta `<form>` HTML para permitir la subida de imágenes?

Para manejar archivos (como imágenes), el formulario debe incluir `enctype="multipart/form-data"` en su etiqueta HTML. Sin esto, el navegador no enviará los datos binarios del archivo al servidor.

04 ¿Cómo se procesan los archivos enviados por el usuario en una vista de Django?

En la vista, al instanciar el formulario, debes pasar explícitamente `request.FILES` además de `request.POST`. Ejemplo: `form = PostForm(request.POST, request.FILES)`.

05 ¿Cómo se configura Django para gestionar y servir archivos multimedia (imágenes)?

Configuramos `MEDIA_URL` y `MEDIA_ROOT` en el archivo `settings.py`. Además, se deben exponer las rutas en el `urls.py` de nivel de proyecto para que el servidor pueda servir estos archivos estáticos durante el desarrollo.

06 ¿Cuál es la función del campo slug en un modelo y por qué se prefiere en las URLs?

Se utiliza para identificar un recurso (post) mediante una cadena de texto amigable en la URL, en lugar de usar el ID numérico (ej. `midominio.com/posts/mi-primer-post/`). Mejora tanto la legibilidad para el usuario como el SEO.

07 ¿Para qué sirve el método `is_valid()` en un formulario de Django?

El método `is_valid()` verifica que los datos enviados por el usuario cumplan con las reglas de validación definidas en el modelo o en el formulario (por ejemplo, que campos requeridos no estén vacíos o que el tipo de archivo sea correcto).

08 ¿Cómo se logra que un formulario muestre los datos actuales al editar un post existente?

Se usa un `ModelForm` pasando el objeto existente como argumento `instance`. Por ejemplo: `PostForm(instance=post_a_editar)`. Esto rellena automáticamente el formulario con los datos guardados actualmente.

PRE-ENTREGA DEL PROYECTO

Actividad evaluable · parte del proyecto integrador

Preentrega 10: CRUD de Posts con Imágenes en Django

Implementar un CRUD funcional de posts en Django, incorporando formularios dinámicos con `ModelForm` y carga de imágenes mediante `ImageField`, `MEDIA_URL`, `MEDIA_ROOT` y archivos media.

¡Llegó el momento de que tu blog pueda gestionar publicaciones completas desde el propio sitio!

En el módulo anterior, conectaste tu proyecto Django con una base de datos. Creaste el modelo `Post`, aplicaste migraciones, registraste el modelo en el panel de administración y mostraste publicaciones reales usando el ORM.

Ahora vas a dar un paso más: implementar un sistema CRUD funcional para crear, ver, editar y eliminar posts desde la interfaz del blog, incorporando además la posibilidad de subir imágenes para cada publicación.

Propósito

El objetivo de esta práctica es implementar un sistema CRUD en tu blog Django, integrando la capacidad de gestionar imágenes mediante formularios.

CRUD significa:

- **Create:** crear publicaciones.
- **Read:** listar y ver publicaciones.
- **Update:** editar publicaciones existentes.
- **Delete:** eliminar publicaciones.

Además, cada post podrá tener una imagen asociada, cargada desde un formulario del sitio.

Vas a trabajar con:

- `ModelForm`.
- Vistas para listar, ver detalle, crear, editar y eliminar posts.
- Templates para formularios y confirmación de borrado.
- `ImageField`.
- Configuración de archivos media.
- `MEDIA_URL` y `MEDIA_ROOT`.
- `request.FILES`.
- Migraciones.
- Pillow.

Contexto de la cadena de entregables

En el **Módulo 7**, creaste la base inicial del proyecto Django.

En el **Módulo 8**, implementaste el flujo:

CÓDIGO

```
Ruta -> Vista -> Template -> Estático
```

En el **Módulo 9**, creaste el modelo `Post`, lo conectaste con SQLite, lo registraste en el admin y mostraste posts usando el ORM.

Ahora, en el **Módulo 10**, vas a construir un CRUD completo para el modelo `Post` desde la interfaz web e incorporar carga de imágenes.

Este checkpoint no incluye autenticación, login, logout ni permisos de usuario. El foco está en formularios, CRUD e imágenes.

Objetivo de aprendizaje de la preentrega

El objetivo de esta preentrega es consolidar el uso de formularios, operaciones CRUD y archivos media en Django.

Vas a practicar:

CRUD

Crear, leer, actualizar y eliminar publicaciones desde el sitio.

Formularios dinámicos

Usar formularios conectados al modelo `Post`.

ModelForm

Crear un formulario basado en el modelo para evitar repetir campos manualmente.

Imágenes

Permitir que un post tenga una imagen asociada.

Media files

Configurar Django para guardar y mostrar archivos subidos por usuarios.

Templates

Crear páginas para listado, detalle, creación, edición y confirmación de eliminación.

Views

Implementar vistas basadas en funciones o clases para cada operación del CRUD.

Qué construir

Debés continuar trabajando sobre el proyecto Django del módulo anterior.

No crees un proyecto nuevo.

El objetivo es implementar un CRUD funcional para el modelo `Post`, agregando soporte para imágenes.

La estructura general puede mantenerse así:

CÓDIGO

```
blog_django/
│
├── manage.py
├── README.md
├── requirements.txt
│
├── blog_project/
│   ├── settings.py
│   └── urls.py
│
├── media/
│   └── posts/
│
└── posts/
    ├── admin.py
    ├── forms.py
    ├── models.py
    ├── urls.py
    ├── views.py
    │
    ├── templates/
    │   └── posts/
    │       ├── base.html
    │       ├── inicio.html
    │       ├── lista_posts.html
    │       ├── detalle_post.html
    │       ├── post_form.html
    │       └── post_confirm_delete.html
    │
    └── static/
        └── posts/
            └── css/
                └── estilos.css
```

Requisitos técnicos

1. Continuidad del proyecto

Debés continuar trabajando sobre el mismo repositorio de GitHub utilizado en los módulos anteriores.

El proyecto debe seguir funcionando con:

CÓDIGO

```
python manage.py runserver
```

La app `posts` debe continuar registrada en `INSTALLED_APPS`.

2. Preparación del modelo

En el archivo:

CÓDIGO

```
posts/models.py
```

agregá al modelo `Post` un campo de imagen:

CÓDIGO

```
imagen = models.ImageField(upload_to='posts/', null=True, blank=True)
```

Este campo permitirá asociar una imagen a cada publicación.

El parámetro `upload_to='posts/'` indica que las imágenes se guardarán dentro de una carpeta `posts/` dentro del directorio media.

Los parámetros `null=True` y `blank=True` permiten que la imagen sea opcional.

3. Instalar Pillow

Para usar `ImageField`, necesitás instalar Pillow.

Ejecutá:

CÓDIGO

```
pip install Pillow
```

Luego actualizá el archivo `requirements.txt`:

CÓDIGO

```
pip freeze > requirements.txt
```

4. Crear y aplicar migraciones

Después de agregar el campo `imagen`, ejecutá:

CÓDIGO

```
python manage.py makemigrations
```

Luego:

CÓDIGO

```
python manage.py migrate
```

Esto sincroniza el cambio del modelo con la base de datos.

5. Configuración de media

En `settings.py`, asegurate de definir:

CÓDIGO

```
MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'
```

Esto indica dónde se guardan los archivos subidos y desde qué URL se sirven durante el desarrollo.

6. Servir archivos media en desarrollo

En el archivo principal de rutas del proyecto, por ejemplo:

CÓDIGO

```
blog_project/urls.py
```

agregá la configuración necesaria para servir archivos media durante el desarrollo.

Esto permite que las imágenes cargadas desde formularios se puedan ver en el navegador.

7. Crear el formulario

Creá el archivo:

CÓDIGO

```
posts/forms.py
```

Dentro de ese archivo, definí una clase:

CÓDIGO

```
PostForm
```

Esta clase debe heredar de:

CÓDIGO

```
forms.ModelForm
```

El formulario debe estar basado en el modelo `Post` e incluir, como mínimo, los siguientes campos:

- `titulo`
- `contenido`
- `autor`
- `estado`
- `imagen`

8. Implementación de vistas

Creá vistas basadas en funciones o clases para las operaciones principales del CRUD.

El sistema debe permitir:

- Listar posts.
- Ver el detalle de un post.
- Crear un nuevo post.
- Editar un post existente.
- Borrar un post.

Podés usar vistas basadas en funciones o vistas basadas en clases. Lo importante es que el flujo CRUD esté completo y funcione.

9. Procesar imágenes en creación y edición

En las vistas de creación y edición, asegurate de procesar también los archivos enviados por el formulario.

Cuando instancies el formulario, debés contemplar tanto los datos del formulario como los archivos.

Es decir, además de los datos de texto, debe procesarse la imagen enviada.

10. Maquetación de templates

Creá o actualizá los templates necesarios para el CRUD.

Como mínimo:

CÓDIGO

```
posts/templates/posts/lista_posts.html
posts/templates/posts/detalle_post.html
posts/templates/posts/post_form.html
posts/templates/posts/post_confirm_delete.html
```

Todos los templates deben mantener la herencia desde:

CÓDIGO

```
posts/base.html
```

11. Formulario con soporte para archivos

En el archivo:

CÓDIGO

```
post_form.html
```

la etiqueta `<form>` debe incluir:

CÓDIGO

```
enctype="multipart/form-data"
```

Sin este atributo, las imágenes no se enviarán correctamente al servidor.

12. Mostrar imágenes en templates

En el listado o en el detalle de posts, mostrá la imagen asociada al post cuando exista.

Si un post no tiene imagen, el template no debe romperse.

Puede mostrarse solo el contenido textual o un mensaje alternativo.

13. Configurar URLs del CRUD

En `posts/urls.py`, agregá rutas para:

- Listar posts.
- Ver detalle.
- Crear.
- Editar.
- Eliminar.

Ejemplo de rutas posibles:

CÓDIGO

```
/

posts/<slug>/
posts/crear/
posts/<slug>/editar/
posts/<slug>/eliminar/
```

Los nombres pueden variar, pero todas las operaciones deben estar disponibles.

14. README

Actualizá el archivo `README.md`.

Debe incluir una breve explicación de:

- Cómo ejecutar el proyecto.
- Qué operaciones CRUD están disponibles.
- Cómo se configuró el manejo de imágenes.
- Qué dependencia nueva se instaló.
- Cómo probar la carga de imágenes.

Funcionalidades requeridas

Listar posts

El sitio debe mostrar publicaciones obtenidas desde la base de datos.

Si un post tiene imagen, puede mostrarse en el listado o reservarse para la vista detalle.

Ver detalle de un post

Cada post debe tener una página individual.

Esta página debe mostrar:

- Título.
- Contenido.
- Autor.
- Estado.
- Imagen, si existe.

Crear post

El sitio debe permitir crear un nuevo post desde un formulario.

El formulario debe permitir cargar una imagen.

Editar post

El sitio debe permitir editar un post existente.

También debe permitir actualizar o cambiar la imagen.

Borrar post

El sitio debe permitir borrar un post.

Antes de eliminar, debe mostrarse una página de confirmación.

Mostrar imágenes

Las imágenes cargadas deben poder verse correctamente en el navegador durante el desarrollo.

Criterios de aceptación

Para considerar este checkpoint como completado, tu proyecto debe cumplir con los siguientes puntos:

- Continuar sobre el proyecto Django del módulo anterior.
- Mantener el modelo `Post` funcionando.
- Agregar el campo `imagen = models.ImageField(upload_to='posts/', null=True, blank=True)`.
- Instalar Pillow.
- Actualizar `requirements.txt`.
- Ejecutar `python manage.py makemigrations`.
- Ejecutar `python manage.py migrate`.
- Configurar `MEDIA_URL` en `settings.py`.
- Configurar `MEDIA_ROOT` en `settings.py`.
- Configurar el archivo `urls.py` principal para servir archivos media en desarrollo.
- Crear `posts/forms.py`.
- Crear la clase `PostForm`.
- Hacer que `PostForm` herede de `forms.ModelForm`.
- Incluir los campos `titulo`, `contenido`, `autor`, `estado` e `imagen`.
- Implementar vistas para listar, ver detalle, crear, editar y borrar posts.
- Procesar `request.FILES` en las vistas de creación y edición si usás vistas basadas en funciones.
- Usar `enctype="multipart/form-data"` en `post_form.html`.
- Crear o actualizar templates para listado, detalle, formulario y confirmación de eliminación.
- Mostrar la imagen del post en el template cuando exista.
- Eliminar posts mediante una vista con confirmación previa.
- Actualizar el `README.md` explicando el manejo de imágenes.
- Subir los cambios al repositorio actual de GitHub.
- El proyecto debe poder ejecutarse con:

CÓDIGO

```
python manage.py runserver
```

Pasos sugeridos

1. Revisar el proyecto anterior

Antes de comenzar, verificá que el proyecto del módulo anterior siga funcionando.

Comprobá que:

- El modelo `Post` exista.
- Las migraciones estén aplicadas.
- El sitio muestre posts desde la base de datos.

2. Agregar el campo de imagen

Editá el modelo `Post` para incluir el campo `imagen`.

Recordá que la imagen debe ser opcional.

3. Instalar Pillow

Instalá Pillow dentro del entorno virtual.

Luego actualizá `requirements.txt`.

4. Ejecutar migraciones

Generá y aplicá migraciones para que la base de datos reconozca el nuevo campo.

5. Configurar media

Agregá `MEDIA_URL` y `MEDIA_ROOT` en `settings.py`.

Luego conectá la configuración necesaria en el `urls.py` principal para servir media durante el desarrollo.

6. Crear el formulario

Creá `posts/forms.py`.

Dentro, definí `PostForm` como un `ModelForm` basado en `Post`.

7. Crear o actualizar vistas

Implementá las vistas necesarias para listar, ver detalle, crear, editar y borrar publicaciones.

Asegurate de que las vistas que procesan formularios también puedan recibir archivos.

8. Actualizar templates

Modificá `post_form.html` para que el formulario permita enviar imágenes.

También actualizá el listado o detalle para mostrar la imagen cuando exista.

9. Configurar URLs

Agregá las rutas necesarias en `posts/urls.py`.

Cada operación del CRUD debe tener una URL asociada.

10. Probar el flujo completo

Probá los siguientes casos:

- Crear un post con imagen.
- Crear un post sin imagen.
- Editar un post y cambiar su imagen.
- Ver la imagen en el navegador.
- Listar posts.
- Entrar al detalle de un post.
- Borrar un post con confirmación.
- Confirmar que las redirecciones funcionan.

11. Actualizar README

Agregá una breve explicación sobre cómo se configuró media y cómo probar la carga de imágenes.

12. Subir cambios

Realizá un commit descriptivo y subí los cambios al repositorio actual.

Errores comunes a evitar

Olvidar el `enctype` en el formulario HTML

Si el formulario no tiene:

CÓDIGO

```
enctype="multipart/form-data"
```

la imagen llegará vacía al servidor.

No realizar `migrate` después de añadir el campo

Después de agregar `ImageField`, la base de datos necesita actualizarse mediante migraciones.

No pasar `request.FILES`

Si usás vistas basadas en funciones y no pasás `request.FILES` al formulario, la imagen no se guardará.

No instalar Pillow

Sin Pillow, Django no podrá trabajar correctamente con `ImageField`.

No configurar `MEDIA_URL` y `MEDIA_ROOT`

Sin esta configuración, Django no sabrá dónde guardar ni cómo servir las imágenes.

No servir media en desarrollo

Aunque la imagen se guarde, puede no verse en el navegador si no configuraste las URLs de media durante el desarrollo.

Intentar mostrar una imagen inexistente

Antes de acceder a la URL de una imagen, verificá que el post tenga una imagen cargada.

Eliminar sin confirmación

El borrado debe tener una página de confirmación para evitar eliminaciones accidentales.

Formato de entrega

La entrega debe realizarse mediante el mismo repositorio público de GitHub utilizado en los módulos anteriores.

Asegurate de que:

- El repositorio siga siendo público.
- Los cambios estén subidos correctamente.
- El CRUD de posts funcione.
- La carga de imágenes funcione.
- El `README.md` explique el manejo de imágenes.
- El entorno virtual no esté subido al repositorio.
- El proyecto pueda ejecutarse con:

CÓDIGO

```
python manage.py runserver
```

El enlace del repositorio será la entrega del checkpoint.

Conexión con el futuro

Este checkpoint transforma el blog en una aplicación más completa.

Hasta ahora, las publicaciones podían cargarse desde el admin y mostrarse en el sitio. Ahora, el propio sitio permite crear, editar, ver y borrar publicaciones, incluyendo imágenes destacadas.

En el próximo módulo, estas acciones podrán combinarse con autenticación, perfiles de usuario y permisos para construir una experiencia más personalizada.

En otras palabras: en el Módulo 9 conectaste el blog con la base de datos. En el Módulo 10, empezás a gestionar el contenido desde el propio sitio, incluyendo archivos subidos por usuarios.

Pasos para la entrega

El objetivo de esta práctica es implementar un sistema CRUD (Create, Read, Update, Delete) funcional en tu blog Django, integrando la capacidad de gestionar imágenes mediante un formulario dinámico.

Pasos sugeridos

1. **Preparación del Modelo**: En tu archivo `models.py`, agrega el campo `imagen = models.ImageField(upload_to='posts/', null=True, blank=True)`. Recuerda instalar Pillow (`pip install Pillow`) y realizar las migraciones correspondientes.
2. **Configuración de Media**: Asegúrate de que en `settings.py` estén definidos `MEDIA_URL` y `MEDIA_ROOT`, y que en el archivo `urls.py` de la raíz hayas incluido la configuración para servir archivos estáticos en desarrollo.
3. **Creación del Formulario**: Crea el archivo `posts/forms.py` y define una clase `PostForm` que herede de `forms.ModelForm`. Incluye los campos de título, contenido, autor, estado e imagen.
4. **Implementación de Vistas**: Crea vistas basadas en funciones o clases para listar (Index), ver detalle (Detail), crear (Create), editar (Update) y borrar (Delete) posts.
5. **Maquetación de Templates**: En el archivo `post_form.html`, asegúrate de que la etiqueta `<form>` incluya el atributo `enctype="multipart/form-data"`, de lo contrario, las imágenes no se enviarán al servidor.
6. **README**: Actualiza tu README con una breve explicación de cómo configuraste el manejo de imágenes.

Errores comunes

- Olvidar el `enctype` en el formulario HTML: esto hace que la imagen llegue vacía a la base de datos.
- No realizar el `migrate` después de añadir el campo de imagen.
- No pasar `request.FILES` al instanciar el formulario en la vista de creación/edición.

Entregable

Formato: repositorio

Repositorio de GitHub con el proyecto Django actualizado que incluya el modelo Post con imagen, el formulario ModelForm y las vistas CRUD configuradas.

Objetivos de aprendizaje

- Implementar operaciones CRUD completas utilizando Django ModelForms.
- Configurar el manejo de archivos estáticos y multimedia (media files) en un entorno de desarrollo.
- Integrar campos de imagen en modelos y procesarlos a través de peticiones POST con archivos.
- Vincular vistas y templates para la edición y eliminación de registros de base de datos.

Criterios de aceptación

- El repositorio contiene el archivo `posts/forms.py` con una clase ModelForm vinculada al modelo Post.
- El modelo Post incluye un campo `ImageField` y se incluye el archivo `requirements.txt` con la dependencia Pillow.
- Existen rutas (URLs) y vistas específicas para crear, editar, listar y eliminar publicaciones.
- El formulario de creación de posts en el template HTML incluye el atributo `enctype='multipart/form-data'`.
- Las imágenes subidas se visualizan correctamente en la vista de detalle del post.

CRUD de Posts con Imágenes en Django

Esta rúbrica evalúa un repositorio de Django que implemente un CRUD completo de posts con carga y visualización de imágenes, integración correcta del formulario ModelForm, configuración de archivos media, vistas funcionales o basadas en clases, templates operativos y documentación básica del proceso. Está diseñada para valorar tanto la funcionalidad técnica como la calidad de organización y entrega del proyecto.

Criterio	Descripción	Peso
Implementación del modelo Post con soporte de imagen	Evalúa si el modelo del blog incluye correctamente el campo de imagen y si la base de datos queda actualizada mediante migraciones. Este criterio es central porque habilita la funcionalidad multimedia del ejercicio.	35%
Formulario ModelForm y manejo de archivos enviados	Evalúa la creación de un ModelForm para Post y el uso correcto de request.FILES y enctype multipart/form-data para que las imágenes se envíen y se guarden correctamente.	20%
CRUD completo de posts	Evalúa si el repositorio implementa de forma funcional las operaciones Create, Read, Update y Delete, incluyendo listado, detalle, creación, edición y borrado de posts.	25%
Configuración de media y manejo de archivos en desarrollo	Evalúa la configuración de MEDIA_URL, MEDIA_ROOT y la exposición de archivos media en el proyecto para que las imágenes subidas puedan visualizarse durante el desarrollo.	10%
Documentación y presentación del repositorio	Evalúa si el repositorio está presentado de forma clara y útil, con README actualizado y estructura comprensible para revisar y ejecutar el proyecto.	10%

Total: 100 pts | Aprobación: 70 pts

MÓDULO 11

Autenticación, usuarios y perfiles

Implementar un sistema robusto de gestión de usuarios con perfiles personalizados y protección de rutas.

Autenticación nativa de Django: usuarios, sesiones y permisos

Explora los fundamentos del sistema de autenticación de Django: cómo identifica usuarios, gestiona permisos y utiliza sesiones para mantener el estado de navegación de forma segura.

Bienvenido al módulo de **autenticación, usuarios y perfiles**.

Hasta ahora, tu blog ya puede mostrar publicaciones y, según lo que hayas construido, también puede permitir crear, editar o borrar contenido. Pero hay un problema importante: si cualquier persona puede modificar cualquier post, el blog todavía no tiene control real sobre sus usuarios.

En una aplicación real necesitamos responder preguntas como:

- ¿Quién está usando el sitio?
- ¿Quién escribió cada publicación?
- ¿Quién puede crear contenido?
- ¿Quién puede editar o borrar un post?
- ¿Qué debería ver un visitante que todavía no inició sesión?

Para resolver esto, Django incluye un sistema de autenticación listo para usar. No tenemos que construir desde cero el manejo de usuarios, contraseñas, sesiones ni permisos.

Podemos imaginar nuestro blog como un club privado:

- La **autenticación** es la puerta de entrada: verifica quién sos.
- La **sesión** es la pulsera que demuestra que ya entraste.
- Los **permisos** son las reglas que dicen qué podés hacer dentro.
- El **usuario** es la persona que interactúa con el sitio.

1. El sistema de autenticación nativo de Django

Uno de los puntos fuertes de Django es que trae incluido el módulo:

CÓDIGO

```
django.contrib.auth
```

Este módulo permite trabajar con:

- Usuarios.
- Contraseñas seguras.
- Login y logout.
- Sesiones.
- Permisos.
- Grupos de usuarios.
- Restricciones de acceso a vistas.
- Integración con el panel de administración.

Esto es muy importante porque manejar usuarios y contraseñas manualmente es delicado. No deberíamos guardar claves como texto plano ni inventar nuestro propio sistema de seguridad.

Django ya incluye una solución preparada para la mayoría de los proyectos web iniciales.

2. Configuración necesaria

Cuando creamos un proyecto con Django, muchas partes del sistema de autenticación ya vienen configuradas en `settings.py`.

Dentro de `INSTALLED_APPS`, normalmente encontramos apps como estas:

CÓDIGO

```
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",

    "posts",
]
```

Las más importantes para esta unidad son:

CÓDIGO

```
"django.contrib.auth"
```

Maneja usuarios, grupos y permisos.

CÓDIGO

```
"django.contrib.sessions"
```

Permite que Django recuerde a un usuario mientras navega por el sitio.

También necesitamos algunos middlewares:

CÓDIGO

```
MIDDLEWARE = [
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    "django.middleware.csrf.CsrfViewMiddleware",
    "django.contrib.auth.middleware.AuthenticationMiddleware",
    "django.contrib.messages.middleware.MessageMiddleware",
]
```

Dos piezas clave son:

CÓDIGO

```
SessionMiddleware
```

Permite manejar sesiones entre una petición y otra.

CÓDIGO

```
AuthenticationMiddleware
```

Permite acceder al usuario actual mediante `request.user`.

Luego de crear el proyecto, recordá ejecutar:

CÓDIGO

```
python manage.py migrate
```

Este comando crea las tablas necesarias para usuarios, permisos, grupos y sesiones.

3. Autenticación vs autorización

Antes de escribir código, conviene separar dos ideas que suelen confundirse.

Autenticación: ¿quién sos?

La autenticación verifica la identidad del usuario.

Por ejemplo:

- Escribir usuario y contraseña.
- Django revisa si esos datos son correctos.
- Si son válidos, el usuario inicia sesión.

En términos simples:

CÓDIGO

```
"¿Sos realmente Cristian?"
```

Autorización: ¿qué podés hacer?

La autorización define qué acciones puede realizar ese usuario.

Por ejemplo:

- Un visitante puede leer posts.
- Un usuario registrado puede crear posts.
- El autor de un post puede editarlo.
- Un administrador puede borrar cualquier post.

En términos simples:

CÓDIGO

```
"Ya sé quién sos, pero... ¿tenés permiso para hacer esto?"
```

Error común

Un error muy común es pensar:

> **Nota**
> “Si el usuario inició sesión, entonces puede hacer cualquier cosa.”

Eso no es correcto.

Un usuario autenticado no debería poder borrar publicaciones de otros usuarios. Para eso necesitamos agregar reglas de autorización.

4. El modelo User

Django incluye un modelo de usuario por defecto. Este modelo representa a las personas que pueden iniciar sesión en nuestro sitio.

Algunos campos habituales son:

- `username` : nombre de usuario.
- `password` : contraseña guardada de forma segura.
- `email` : correo electrónico.
- `first_name` : nombre.
- `last_name` : apellido.
- `is_staff` : indica si puede entrar al panel de administración.

- `is_superuser`: indica si tiene permisos totales.
- `is_active`: indica si la cuenta está activa.

Desde el panel de administración podemos crear usuarios manualmente. Para eso primero necesitamos un superusuario:

CÓDIGO

```
python manage.py createsuperuser
```

Luego podemos entrar a:

CÓDIGO

```
/admin/
```

Y administrar usuarios desde la interfaz de Django.

5. Contraseñas: nunca guardarlas manualmente

Una regla fundamental:

> **Nota**
> Nunca guardes contraseñas como texto plano.

No deberíamos hacer esto:

CÓDIGO

```
user.password = "123456"
user.save()
```

Ese código guarda mal la contraseña porque no aplica el proceso de seguridad que Django necesita.

Django no guarda la contraseña original. Guarda un **hash**, que es una versión procesada de la contraseña. Esto permite verificarla más adelante sin tener que almacenar el texto original.

La forma correcta, si alguna vez necesitás asignar una contraseña desde código, es usar:

CÓDIGO

```
user.set_password("123456")
user.save()
```

También podemos crear usuarios correctamente con:

CÓDIGO

```
from django.contrib.auth.models import User

usuario = User.objects.create_user(
    username="ana",
    email="ana@example.com",
    password="clave_segura_123"
)
```

`create_user()` se encarga de aplicar el tratamiento correcto a la contraseña.

6. Sesiones: cómo Django recuerda al usuario

HTTP, el protocolo que usa la web, no tiene memoria. Cada vez que el navegador pide una página, el servidor recibe una petición nueva.

Entonces aparece una pregunta:

> **Nota**
> Si ya hice login, ¿cómo sabe Django que sigo siendo yo en la siguiente página?

La respuesta es: mediante sesiones.

El flujo simplificado es:

1. El usuario escribe su usuario y contraseña.
2. Django valida esos datos.
3. Si son correctos, Django crea una sesión.
4. El navegador recibe una cookie con un identificador.
5. En las siguientes peticiones, el navegador envía esa cookie.
6. Django usa esa cookie para reconocer al usuario.

La cookie funciona como una pulsera de un evento: no contiene necesariamente todos tus datos, pero sirve para identificar tu sesión.

Gracias a esto, el usuario no tiene que iniciar sesión en cada página.

7. `request.user`: el usuario actual en una vista

Cuando `AuthenticationMiddleware` está activo, Django agrega automáticamente el usuario actual al objeto `request`.

Eso significa que dentro de una vista podemos hacer:

CÓDIGO

```
def mi_vista(request):
    usuario_actual = request.user
```

Si el usuario inició sesión, `request.user` será un usuario real.

Si no inició sesión, `request.user` será un usuario anónimo.

Podemos comprobarlo con:

CÓDIGO

```
def mi_vista(request):
    if request.user.is_authenticated:
        print("Usuario logueado:", request.user.username)
    else:
        print("Usuario anónimo")
```

Esta propiedad es muy útil para mostrar contenido diferente según el estado del usuario.

8. Mostrar opciones distintas en el template

En los templates también podemos usar el objeto `user`.

Por ejemplo, en `base.html`:

CÓDIGO

```
<nav>
    <a href="{% url 'lista_posts' %}">Inicio</a>

    {% if user.is_authenticated %}
        <span>Hola, {{ user.username }}</span>
        <a href="{% url 'crear_post' %}">Crear post</a>
        <form action="{% url 'logout' %}" method="post" style="display: inline;">
            {% csrf_token %}
            <button type="submit">Cerrar sesión</button>
        </form>

    {% else %}
        <a href="{% url 'login' %}">Iniciar sesión</a>
        <a href="{% url 'registro' %}">Crear cuenta</a>
    {% endif %}
</nav>
```

La condición importante es:

CÓDIGO

```
{% if user.is_authenticated %}
```

Esto pregunta si el visitante actual tiene una sesión válida.

Si inició sesión, mostramos opciones para usuarios registrados.

Si no inició sesión, mostramos opciones para visitantes.

9. Proteger una vista con `login_required`

Supongamos que queremos que solo los usuarios registrados puedan crear posts.

Para eso usamos el decorador:

CÓDIGO

```
@login_required
```

Ejemplo:

CÓDIGO

```
from django.contrib.auth.decorators import login_required
from django.shortcuts import render, redirect
from .forms import PostForm

@login_required
def crear_post(request):
    if request.method == "POST":
        form = PostForm(request.POST, request.FILES)

        if form.is_valid():
            post = form.save(commit=False)
            post.autor = request.user
            post.save()
            return redirect("lista_posts")
    else:
        form = PostForm()

    return render(request, "posts/post_form.html", {
        "form": form,
        "titulo_pagina": "Crear post",
        "texto_boton": "Crear"

    })
```

Veamos las partes importantes.

Primero:

CÓDIGO

```
@login_required
```

Indica que la vista requiere un usuario autenticado.

Luego:

CÓDIGO

```
post = form.save(commit=False)
```

Crea el objeto `Post`, pero todavía no lo guarda en la base de datos.

Después:

CÓDIGO

```
post.autor = request.user
```

Asigna como autor al usuario actual.

Finalmente:

CÓDIGO

```
post.save()
```

Guarda el post completo.

Este patrón es muy común cuando un dato no viene del formulario, sino del contexto de la sesión.

Hasta ahora, el campo autor era un CharField porque todavía no teníamos usuarios reales en el sistema. A partir de este módulo, como ya incorporamos autenticación, vamos a reemplazar ese texto por una relación con el usuario logueado mediante ForeignKey.

Este cambio implica:

1. Modificar el modelo Post.
2. Ejecutar makemigrations y migrate.
3. Quitar autor del PostForm.
4. Asignar el autor desde la vista con request.user.
5. Actualizar templates para mostrar post.autor.username si queremos ver el nombre de usuario.

En `posts/forms.py`, el formulario de posts queda sin el campo `autor`:

CÓDIGO

```
from django import forms
from .models import Post


class PostForm(forms.ModelForm):
    class Meta:
        model = Post
        fields = ["titulo", "contenido", "estado", "imagen"]
```

10. Relacionar un post con su autor

Para que un post tenga dueño, necesitamos relacionarlo con un usuario.

En `models.py`:

CÓDIGO

```
from django.conf import settings
from django.db import models

class Post(models.Model):
    ESTADOS = [
        ("borrador", "Borrador"),
        ("publicado", "Publicado"),
        ("archivado", "Archivado"),
    ]

    titulo = models.CharField(max_length=200)
    contenido = models.TextField()
    autor = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE)
    fecha_creacion = models.DateTimeField(auto_now_add=True)
    estado = models.CharField(max_length=20, choices=ESTADOS, default="borrador")
    slug = models.SlugField(max_length=220, unique=True, blank=True, null=True)
    imagen = models.ImageField(upload_to="posts/", null=True, blank=True)


    def save(self, *args, **kwargs):
        if not self.slug:
            self.slug = slugify(self.titulo)
        super().save(*args, **kwargs)

    def __str__(self):
        return self.titulo
```

La parte clave es:

CÓDIGO

```
autor = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE)
```

Esto significa que cada post pertenece a un usuario.

Usamos `settings.AUTH_USER_MODEL` porque es una buena práctica en Django: permite que el proyecto siga funcionando incluso si más adelante se usa un modelo de usuario personalizado.

Después de modificar el modelo, debemos ejecutar:

CÓDIGO

```
python manage.py makemigrations
python manage.py migrate
```

11. Mostrar el autor en el template

Una vez que el post tiene autor, podemos mostrarlo fácilmente.

Por ejemplo, en `detalle_post.html`:

CÓDIGO

```
<article>
    <h1>{{ post.titulo }}</h1>

    <p>
        Publicado por {{ post.autor.username }}
        el {{ post.fecha_creacion }}
    </p>

    <div>
        {{ post.contenido }}
    </div>
</article>
```

También podríamos mostrar botones de edición solo si el usuario actual es el autor:

CÓDIGO

```
{% if user == post.autor %}
    <a href="{% url 'editar_post' post.slug %}">Editar</a>
    <a href="{% url 'eliminar_post' post.slug %}">Eliminar</a>
{% endif %}
```

Esto mejora la experiencia visual, pero no alcanza como medida de seguridad.

Aunque ocultemos el botón, un usuario podría intentar entrar manualmente a la URL de edición.

Por eso también hay que validar permisos en la vista.

12. Validar que solo el autor pueda editar

Supongamos que tenemos una vista para editar posts.

No alcanza con pedir login. También debemos controlar que el usuario sea el autor del post.

CÓDIGO

```
from django.contrib.auth.decorators import login_required
from django.http import HttpResponseForbidden
from django.shortcuts import get_object_or_404, render, redirect
from .models import Post
from .forms import PostForm

@login_required
def editar_post(request, slug):
    post = get_object_or_404(Post, slug=slug)

    if post.autor != request.user:
        return HttpResponseForbidden("No tenés permiso para editar este post.")

    if request.method == "POST":
        form = PostForm(request.POST, request.FILES, instance=post)

        if form.is_valid():
            form.save()
            return redirect("detalle_post", slug=post.slug)
    else:
        form = PostForm(instance=post)

    return render(request, "posts/post_form.html", {
        "form": form,
        "post": post,
        "titulo_pagina": "Editar post",
        "texto_boton": "Guardar cambios"
    })
```

La validación importante es:

CÓDIGO

```
if post.autor != request.user:
    return HttpResponseForbidden("No tenés permiso para editar este post.")
```

Con esto evitamos que un usuario edite publicaciones ajenas.

13. Validar que solo el autor pueda borrar

La misma lógica aplica para borrar posts.

CÓDIGO

```
from django.contrib.auth.decorators import login_required
from django.http import HttpResponseForbidden
from django.shortcuts import get_object_or_404, render, redirect
from .models import Post

@login_required
def eliminar_post(request, slug):
    post = get_object_or_404(Post, slug=slug)

    if post.autor != request.user:
        return HttpResponseForbidden("No tenés permiso para eliminar este post.")

    if request.method == "POST":
        post.delete()
        return redirect("lista_posts")

    return render(request, "posts/post_confirm_delete.html", {"post": post})
```

Este tipo de control es una forma simple de autorización basada en dueño del contenido.

14. Grupos y permisos

Django también permite trabajar con permisos más generales.

Por cada modelo, Django puede crear permisos como:

- `add`: crear.
- `change`: modificar.
- `delete`: eliminar.
- `view`: ver.

Para un modelo llamado `Post` dentro de una app llamada `posts`, los permisos podrían verse así:

CÓDIGO

```
posts.add_post
posts.change_post
posts.delete_post
posts.view_post
```

Podemos preguntar si un usuario tiene un permiso:

CÓDIGO

```
if request.user.has_perm("posts.change_post"):
    print("Puede editar posts")
```

También podríamos usar un decorador:

CÓDIGO

```
from django.contrib.auth.decorators import permission_required

@permission_required("posts.add_post")
def crear_post(request):
    ...
```

Esto es útil cuando trabajamos con roles más generales, como:

- Administradores.
- Editores.
- Redactores.
- Moderadores.

15. ¿Para qué sirven los grupos?

Los grupos permiten asignar permisos a muchas personas al mismo tiempo.

Por ejemplo, podríamos crear un grupo llamado:

CÓDIGO

```
Redactores
```

Y darle permisos para:

CÓDIGO

```
posts.add_post
posts.change_post
```

Luego agregamos usuarios a ese grupo.

Así evitamos configurar permisos usuario por usuario.

La idea sería:

CÓDIGO

```
Usuario -> pertenece a un grupo -> hereda permisos del grupo
```

Esto es muy útil cuando el blog empieza a crecer y ya no alcanza con tener un único administrador.

16. `is_staff` vs `is_superuser`

Django incluye dos campos importantes para el panel de administración.

`is_staff`

Indica si el usuario puede entrar al admin de Django.

Pero entrar al admin no significa poder hacer todo. Un usuario con `is_staff=True` solo podrá ver o modificar aquello para lo que tenga permisos.

`is_superuser`

Indica que el usuario tiene permisos totales.

Un superusuario puede crear, modificar y eliminar cualquier recurso desde el admin.

Dicho de forma simple:

CÓDIGO

```
is_staff = puede entrar al edificio del admin
is_superuser = tiene la llave de todas las puertas
```

17. Errores comunes

Guardar contraseñas manualmente

Incorrecto:

CÓDIGO

```
user.password = "123456"
user.save()
```

Correcto:

CÓDIGO

```
user.set_password("123456")
user.save()
```

O mejor:

CÓDIGO

```
User.objects.create_user(
    username="ana",
    email="ana@example.com",
    password="123456"
)
```

Proteger solo el template

Incorrecto:

CÓDIGO

```
{% if user == post.autor %}
    <a href="{% url 'post_delete' post.id %}">Eliminar</a>
{% endif %}
```

Eso oculta el botón, pero no protege la URL.

También hay que validar en la vista:

CÓDIGO

```
if post.autor != request.user:
    return HttpResponseForbidden("No tenés permiso.")
```

Pensar que login equivale a permiso total

Incorrecto:

CÓDIGO

```
@login_required
def eliminar_post(request, slug):
    post = get_object_or_404(Post, slug=slug)
    post.delete()
    return redirect("lista_posts")
```

Ese código permite que cualquier usuario logueado borre cualquier post.

Mejor:

CÓDIGO

```
@login_required
def eliminar_post(request, slug):
    post = get_object_or_404(Post, slug=slug)

    if post.autor != request.user:
        return HttpResponseForbidden("No tenés permiso.")

    post.delete()
    return redirect("lista_posts")
```

Olvidar las migraciones

Si agregamos un campo como este:

CÓDIGO

```
autor = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE)
```

Debemos ejecutar:

CÓDIGO

```
python manage.py makemigrations
python manage.py migrate
```

Si no lo hacemos, la base de datos no tendrá la nueva columna.

18. Mini resumen aplicado al blog

En este módulo empezamos a convertir nuestro blog en una aplicación con usuarios reales.

Aprendimos que:

- Django trae un sistema de autenticación integrado.
- La autenticación responde: "¿quién sos?".
- La autorización responde: "¿qué podés hacer?".
- Las sesiones permiten recordar al usuario entre páginas.
- `request.user` representa al usuario actual en una vista.
- `user.is_authenticated` permite adaptar el contenido en los templates.
- `login_required` protege vistas que requieren inicio de sesión.
- Un post puede relacionarse con su autor mediante una `ForeignKey`.
- Ocultar botones en el template no alcanza: también hay que validar permisos en la vista.
- Django maneja contraseñas de forma segura si usamos sus herramientas.

En la siguiente unidad vamos a pasar de la teoría a la práctica creando las pantallas de registro, inicio de sesión y cierre de sesión para que los visitantes puedan convertirse en usuarios del blog.

Registro de usuarios con UserCreationForm

Aprende a crear formularios de registro seguros y personalizados utilizando el UserCreationForm de Django, integrando campos como email y nombre.

En la unidad anterior vimos que Django incluye un sistema de autenticación nativo para trabajar con usuarios, sesiones y permisos.

Ahora vamos a dar el siguiente paso: permitir que una persona pueda **crear una cuenta** en nuestro blog.

Registrar usuarios no es simplemente guardar un nombre y una contraseña en la base de datos. También necesitamos validar que el nombre de usuario no exista, comprobar que las contraseñas coincidan, aplicar reglas mínimas de seguridad y guardar la contraseña correctamente.

Hacer todo eso manualmente sería riesgoso. Por suerte, Django ya trae un formulario preparado para este caso: `UserCreationForm`.

1. ¿Qué es `UserCreationForm`?

`UserCreationForm` es un formulario incluido en Django que permite crear usuarios usando el modelo `User`.

Por defecto, trae estos campos:

- `username`
- `password1`
- `password2`

El campo `password1` representa la contraseña.

El campo `password2` representa la confirmación de contraseña.

Django valida automáticamente que ambas contraseñas coincidan. También puede aplicar las reglas de seguridad configuradas en el proyecto, como longitud mínima o evitar contraseñas demasiado comunes.

Para usarlo, lo importamos desde:

CÓDIGO

```
from django.contrib.auth.forms import UserCreationForm
```

2. Registro vs login

Antes de escribir código, conviene separar dos acciones distintas.

Registro

El registro crea una cuenta nueva.

Ejemplo:

CÓDIGO

```
Crear usuario: ana
Contraseña: ********
```

Después del registro, el usuario ya existe en la base de datos.

Login

El login permite entrar con una cuenta existente.

Ejemplo:

CÓDIGO

```
Usuario: ana
Contraseña: ********
```

El login no crea una cuenta nueva. Solo verifica que las credenciales pertenezcan a un usuario existente.

Dicho simple:

CÓDIGO

```
Registro = crear la llave.
Login = usar la llave para entrar.
```

En esta unidad nos vamos a enfocar en el registro. El login y el logout los trabajaremos en la siguiente unidad.

3. Crear una app para usuarios

Podríamos poner la lógica de registro dentro de la app `posts`, pero no sería lo más ordenado.

Como el registro pertenece a la gestión de usuarios, conviene crear una app separada. Por ejemplo:

CÓDIGO

```
python manage.py startapp accounts
```

Luego agregamos la app en `settings.py`:

CÓDIGO

```
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",

    "posts",
    "accounts",
]
```

La app `accounts` puede encargarse de:

- Registro.
- Login.
- Logout.
- Perfil de usuario.
- Edición de datos personales.

En esta unidad vamos a enfocarnos solamente en el registro.

4. Crear nuestro formulario de registro

Django ya nos da `UserCreationForm`, pero muchas veces queremos agregar algún campo extra.

Por ejemplo, además del nombre de usuario y la contraseña, podemos pedir el email.

En `accounts/forms.py`:

CÓDIGO

```
from django import forms
from django.contrib.auth.forms import UserCreationForm
from django.contrib.auth.models import User


class RegistroUsuarioForm(UserCreationForm):
    email = forms.EmailField(required=True)

    class Meta:
        model = User
        fields = ["username", "email", "password1", "password2"]
```

Veamos cada parte.

Importamos `UserCreationForm`:

CÓDIGO

```
from django.contrib.auth.forms import UserCreationForm
```

Importamos el modelo de usuario por defecto:

CÓDIGO

```
from django.contrib.auth.models import User
```

Creamos nuestro formulario heredando de `UserCreationForm`:

CÓDIGO

```
class RegistroUsuarioForm(UserCreationForm):
```

Esto significa que nuestro formulario conserva el comportamiento del formulario original de Django, pero podemos personalizarlo.

Agregamos el campo `email`:

CÓDIGO

```
email = forms.EmailField(required=True)
```

Y usamos la clase `Meta` para indicarle a Django qué modelo y qué campos debe usar:

CÓDIGO

```
class Meta:
    model = User
    fields = ["username", "email", "password1", "password2"]
```

5. ¿Qué es la clase `Meta`?

La clase `Meta` es una forma de darle instrucciones internas a Django.

En este caso, le estamos diciendo:

CÓDIGO

```
model = User
```

Este formulario trabaja con el modelo `User`.

Y también:

CÓDIGO

```
fields = ["username", "email", "password1", "password2"]
```

Estos son los campos que queremos mostrar en el formulario.

No es una clase que el usuario vea en la página. Es una configuración para que Django sepa cómo construir y procesar el formulario.

6. Crear la vista de registro

Ahora necesitamos una vista para mostrar el formulario y procesar los datos enviados por el usuario.

En `accounts/views.py`:

CÓDIGO

```
from django.shortcuts import render, redirect
from .forms import RegistroUsuarioForm



def registro(request):
    if request.method == "POST":
        form = RegistroUsuarioForm(request.POST)

        if form.is_valid():
            form.save()
            return redirect("lista_posts")
    else:
        form = RegistroUsuarioForm()


    return render(request, "accounts/registro.html", {"form": form})
```

Analicemos el flujo.

Si el usuario entra por primera vez a la página, la petición será `GET`:

CÓDIGO

```
else:
    form = RegistroUsuarioForm()
```

En ese caso mostramos un formulario vacío.

Si el usuario completa el formulario y lo envía, la petición será `POST`:

CÓDIGO

```
if request.method == "POST":
    form = RegistroUsuarioForm(request.POST)
```

Ahí el formulario recibe los datos enviados.

Luego validamos:

CÓDIGO

```
if form.is_valid():
```

Esto revisa que:

- El nombre de usuario no esté repetido.
- Las contraseñas coincidan.

- La contraseña cumpla las reglas configuradas.
- El email tenga formato válido.

Si todo está bien, guardamos:

CÓDIGO

```
form.save()
```

Ese `save()` crea el usuario en la base de datos.

Finalmente redirigimos al listado de posts:

CÓDIGO

```
return redirect("lista_posts")
```

En la próxima unidad, cuando creemos la página de login, podremos cambiar esa redirección para mandar al usuario directamente a iniciar sesión.

7. Crear las URLs de `accounts`

Ahora necesitamos conectar la vista con una URL.

En `accounts/urls.py`:

CÓDIGO

```
from django.urls import path
from . import views

urlpatterns = [
    path("registro/", views.registro, name="registro"),
```

Luego incluimos esas URLs en el archivo principal del proyecto.

Por ejemplo, en `blog_project/urls.py`:

CÓDIGO

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("posts.urls")),
    path("accounts/", include("accounts.urls")),
]
```

Con esta configuración, la URL de registro será:

CÓDIGO

```
/accounts/registro/
```

8. Crear el template de registro

Ahora necesitamos la plantilla HTML.

Creamos el archivo:

CÓDIGO

```
accounts/templates/accounts/registro.html
```

Contenido básico:

CÓDIGO

```
{% extends "posts/base.html" %}

{% block content %}
    <h1>Crear cuenta</h1>

    <form method="post">
        {% csrf_token %}

        {{ form.as_p }}

        <button type="submit">Registrarme</button>
    </form>
{% endblock %}
```

La línea más importante del formulario es:

CÓDIGO

```
<form method="post">
```

Usamos `POST` porque estamos enviando datos que modifican la base de datos.

También necesitamos:

CÓDIGO

```
{% csrf_token %}
```

Este token ayuda a proteger el formulario contra ataques CSRF.

Por último:

CÓDIGO

```
{{ form.as_p }}
```

Muestra el formulario completo envuelto en etiquetas `<p>`.

Más adelante podemos personalizar el diseño campo por campo, pero para empezar esta opción es clara y funcional.

9. Mostrar errores del formulario

Si el usuario escribe una contraseña débil o las contraseñas no coinciden, Django mostrará errores.

Con `{{ form.as_p }}`, esos errores ya aparecen automáticamente.

Pero también podemos mostrar errores generales de forma más explícita:

CÓDIGO

```
{% extends "posts/base.html" %}

{% block content %}
    <h1>Crear cuenta</h1>

    <form method="post">
        {% csrf_token %}

        {% if form.errors %}
            <div>
                <p>Revisá los datos ingresados:</p>
                {{ form.errors }}
            </div>
        {% endif %}

        {{ form.as_p }}

        <button type="submit">Registrarme</button>
    </form>
{% endblock %}
```

Esto ayuda a que el estudiante vea claramente qué está pasando si el formulario falla.

10. Guardar correctamente el email

Aunque agregamos el campo `email`, conviene asegurarnos de que se guarde correctamente.

Podemos sobrescribir el método `save()` del formulario.

En `accounts/forms.py`:

CÓDIGO

```
from django import forms
from django.contrib.auth.forms import UserCreationForm
from django.contrib.auth.models import User


class RegistroUsuarioForm(UserCreationForm):
    email = forms.EmailField(required=True)

    class Meta:
        model = User
        fields = ["username", "email", "password1", "password2"]

    def save(self, commit=True):
        user = super().save(commit=False)
        user.email = self.cleaned_data["email"]

        if commit:
            user.save()

        return user
```

La línea:

CÓDIGO

```
user = super().save(commit=False)
```

Crea el usuario usando la lógica de `UserCreationForm`, pero todavía no lo guarda.

Luego asignamos el email:

CÓDIGO

```
user.email = self.cleaned_data["email"]
```

Finalmente guardamos:

CÓDIGO

```
if commit:
    user.save()
```

Este patrón es útil cuando necesitamos completar datos antes del guardado definitivo.

11. Evitar emails repetidos

Por defecto, el modelo `User` de Django no obliga a que el email sea único.

Si queremos evitar que dos usuarios se registren con el mismo email, podemos agregar una validación en el formulario.

CÓDIGO

```
from django import forms
from django.contrib.auth.forms import UserCreationForm
from django.contrib.auth.models import User


class RegistroUsuarioForm(UserCreationForm):
    email = forms.EmailField(required=True)

    class Meta:
        model = User
        fields = ["username", "email", "password1", "password2"]

    def clean_email(self):
        email = self.cleaned_data["email"]

        if User.objects.filter(email=email).exists():
            raise forms.ValidationError("Ya existe un usuario con ese email.")

        return email

    def save(self, commit=True):
        user = super().save(commit=False)
        user.email = self.cleaned_data["email"]

        if commit:
            user.save()

        return user
```

El método:

CÓDIGO

```
def clean_email(self):
```

Sirve para validar específicamente el campo `email`.

Si encontramos un usuario con ese email:

CÓDIGO

```
if User.objects.filter(email=email).exists():
```

lanzamos un error de validación:

CÓDIGO

```
raise forms.ValidationError("Ya existe un usuario con ese email.")
```

De esta manera, el formulario no se guarda y muestra el error al usuario.

12. Versión simple recomendada para empezar

Para una primera práctica, podemos usar esta versión del formulario.

En `accounts/forms.py`:

CÓDIGO

```
from django import forms
from django.contrib.auth.forms import UserCreationForm
from django.contrib.auth.models import User


class RegistroUsuarioForm(UserCreationForm):
    email = forms.EmailField(required=True)

    class Meta:
        model = User
        fields = ["username", "email", "password1", "password2"]
```

En `accounts/views.py`:

CÓDIGO

```
from django.shortcuts import render, redirect
from .forms import RegistroUsuarioForm


def registro(request):
    if request.method == "POST":
        form = RegistroUsuarioForm(request.POST)

        if form.is_valid():
            form.save()
            return redirect("lista_posts")
    else:
        form = RegistroUsuarioForm()

    return render(request, "accounts/registro.html", {"form": form})
```

En `accounts/urls.py`:

CÓDIGO

```
from django.urls import path
from . import views

urlpatterns = [
    path("registro/", views.registro, name="registro"),
]
```

Esta versión alcanza para implementar un registro funcional y entender el circuito principal.

13. Errores comunes

Escribir mal `username`

Incorrecto:

CÓDIGO

```
Usermame
```

Correcto:

CÓDIGO

```
Username
```

En el código usamos:

CÓDIGO

```
username
```

Olvidar `is_valid()`

Incorrecto:

CÓDIGO

```
form = RegistroUsuarioForm(request.POST)
form.save()
```

Correcto:

CÓDIGO

```
form = RegistroUsuarioForm(request.POST)

if form.is_valid():
    form.save()
```

Nunca deberíamos guardar un formulario sin validarlo antes.

Olvidar `method="post"`

Incorrecto:

CÓDIGO

```
<form>
```

Correcto:

CÓDIGO

```
<form method="post">
```

Si no indicamos `method="post"`, el navegador usará `GET` por defecto.

Eso puede exponer datos en la URL y además no representa correctamente una acción que modifica la base de datos.

Olvidar `{% csrf_token %}`

Incorrecto:

CÓDIGO

```
<form method="post">
    {{ form.as_p }}
    <button type="submit">Registrarme</button>
</form>
```

Correcto:

CÓDIGO

```
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Registrarme</button>
</form>
```

Django espera este token en formularios que envían datos por `POST`.

Confundir registro con login

El registro crea una cuenta nueva.

El login permite entrar con una cuenta existente.

No son la misma acción y normalmente tienen vistas, URLs y templates diferentes.

14. Síntesis

En esta unidad aprendimos a crear un registro de usuarios usando herramientas nativas de Django.

Vimos que:

- `UserCreationForm` permite crear usuarios sin programar toda la lógica desde cero.
- El formulario trae campos para usuario, contraseña y confirmación.
- Podemos personalizarlo agregando campos como `email`.
- La clase `Meta` le indica a Django qué modelo y qué campos debe usar.
- La vista de registro debe distinguir entre `GET` y `POST`.
- Antes de guardar, siempre debemos usar `form.is_valid()`.
- Los formularios enviados por `POST` deben incluir `{% csrf_token %}`.
- Registro y login son acciones distintas.

Con esto, nuestro blog ya puede recibir nuevos usuarios. En la próxima unidad vamos a conectar este registro con el inicio y cierre de sesión.

Login, logout y protección de sesiones

Aprende a configurar el inicio y cierre de sesión utilizando las herramientas nativas de Django y a proteger las rutas de tu blog para usuarios autenticados.

En la unidad anterior aprendimos a registrar usuarios. Es decir, ya podemos crear cuentas nuevas en nuestro blog.

Pero crear una cuenta no alcanza. Ahora necesitamos permitir que el usuario pueda **entrar** al sistema, mantenerse identificado mientras navega y cerrar sesión cuando termine.

A ese proceso lo llamamos autenticación en la web:

- **Login**: el usuario ingresa sus credenciales y Django inicia una sesión.
- **Sesión**: Django recuerda quién es el usuario mientras navega.
- **Logout**: Django finaliza la sesión actual.
- **Protección de vistas**: evitamos que usuarios anónimos accedan a páginas privadas.

Django ya trae herramientas listas para resolver todo esto sin tener que programar manualmente la validación de contraseñas ni el manejo de cookies.

1. ¿Qué es el login?

El login es el proceso mediante el cual un usuario demuestra quién es.

Normalmente lo hace ingresando:

- Nombre de usuario.
- Contraseña.

Si esos datos son correctos, Django crea una sesión para ese usuario.

Dicho simple:

CÓDIGO

```
El usuario muestra su llave.
Django revisa si la llave es válida.
Si es válida, lo deja entrar.
```

2. ¿Qué es el logout?

El logout es el proceso contrario.

Cuando un usuario cierra sesión, Django finaliza la sesión actual. A partir de ese momento, el navegador ya no queda identificado como ese usuario.

Dicho simple:

CÓDIGO

```
El usuario devuelve la pulsera del evento.
Django deja de reconocerlo como usuario logueado.
```

3. Vistas nativas de autenticación

Django incluye vistas listas para usar dentro de:

CÓDIGO

```
django.contrib.auth.views
```

Las dos más importantes para esta unidad son:

CÓDIGO

```
LoginView
```

Muestra el formulario de acceso, valida las credenciales y, si son correctas, inicia la sesión.

CÓDIGO

```
LogoutView
```

Finaliza la sesión actual del usuario.

No necesitamos crear toda esa lógica a mano. Solo debemos conectar estas vistas con nuestras URLs y crear los templates correspondientes.

4. Agregar login y logout a las URLs

Como venimos trabajando con una app llamada `accounts`, podemos agregar las URLs ahí.

En `accounts/urls.py`:

CÓDIGO

```
from django.urls import path
from django.contrib.auth import views as auth_views
from . import views

urlpatterns = [
    path("registro/", views.registro, name="registro"),
    path(
        "login/",
        auth_views.LoginView.as_view(template_name="accounts/login.html"),
        name="login",
    ),
    path(
        "logout/",
        auth_views.LogoutView.as_view(),
        name="logout",
    ),
]
```

La ruta:

CÓDIGO

```
path("login/", auth_views.LoginView.as_view(...), name="login")
```

crea la página de inicio de sesión.

La ruta:

CÓDIGO

```
path("logout/", auth_views.LogoutView.as_view(), name="logout")
```

crea la acción para cerrar sesión.

Si en el archivo principal del proyecto ya tenemos esto:

CÓDIGO

```
path("accounts/", include("accounts.urls"))
```

entonces las URLs finales serán:

CÓDIGO

```
/accounts/login/
```

y:

CÓDIGO

```
/accounts/logout/
```

5. Crear el template de login

Como configuramos `LoginView` con:

CÓDIGO

```
template_name="accounts/login.html"
```

tenemos que crear este archivo:

CÓDIGO

```
accounts/templates/accounts/login.html
```

Contenido básico:

CÓDIGO

```
{% extends "posts/base.html" %}

{% block content %}
    <h1>Iniciar sesión</h1>

    <form method="post">
        {% csrf_token %}

        {{ form.as_p }}

        <button type="submit">Entrar</button>
    </form>

    <p>
        ¿No tenés cuenta?
        <a href="{% url 'registro' %}">Crear cuenta</a>
    </p>
{% endblock %}
```

La vista `LoginView` ya nos entrega un formulario listo para usar.

Por eso podemos escribir:

CÓDIGO

```
{{ form.as_p }}
```

Ese formulario incluye los campos necesarios para ingresar usuario y contraseña.

6. Mostrar errores de login

Si el usuario escribe mal su nombre o contraseña, Django vuelve a mostrar el formulario con errores.

Podemos hacerlo más claro así:

CÓDIGO

```
{% extends "posts/base.html" %}

{% block content %}
    <h1>Iniciar sesión</h1>

    {% if form.errors %}
        <p>El usuario o la contraseña no son correctos.</p>
    {% endif %}

    <form method="post">
        {% csrf_token %}

        {{ form.as_p }}

        <button type="submit">Entrar</button>
    </form>

    <p>
        ¿No tenés cuenta?
        <a href="{% url 'registro' %}">Crear cuenta</a>
    </p>
{% endblock %}
```

Esto no cambia la lógica del login. Solo mejora el mensaje que ve el usuario.

7. Configurar redirecciones en `settings.py`

Ahora tenemos que decirle a Django a dónde mandar al usuario después de iniciar o cerrar sesión.

Al final de `settings.py`, agregamos:

CÓDIGO

```
LOGIN_URL = "login"
LOGIN_REDIRECT_URL = "lista_posts"
LOGOUT_REDIRECT_URL = "lista_posts"
```

Veamos cada una.

CÓDIGO

```
LOGIN_URL = "login"
```

Es la URL a la que Django enviará a un usuario anónimo cuando intente entrar a una vista protegida.

CÓDIGO

```
LOGIN_REDIRECT_URL = "lista_posts"
```

Es el destino después de un login exitoso.

CÓDIGO

```
LOGOUT_REDIRECT_URL = "lista_posts"
```

Es el destino después de cerrar sesión.

En este caso usamos `lista_posts` porque probablemente sea la página principal del blog.

8. Ajustar el registro para redirigir al login

En la unidad anterior, después de registrar al usuario, lo redirigíamos al listado de posts.

Ahora que ya existe la URL de login, podemos mejorar ese flujo.

En `accounts/views.py`:

CÓDIGO

```
from django.shortcuts import render, redirect
from .forms import RegistroUsuarioForm



def registro(request):
    if request.method == "POST":
        form = RegistroUsuarioForm(request.POST)

        if form.is_valid():
            form.save()
            return redirect("login")
    else:
        form = RegistroUsuarioForm()


    return render(request, "accounts/registro.html", {"form": form})
```

La línea clave es:

CÓDIGO

```
return redirect("login")
```

Ahora el flujo queda así:

CÓDIGO

```
1. El usuario se registra.
2. Django crea la cuenta.
3. El usuario es redirigido al login.
4. El usuario inicia sesión.
5. Django lo manda al listado de posts.
```

9. Usar `user` en los templates

Cuando el sistema de autenticación está activo, Django permite acceder al usuario actual desde los templates mediante el objeto:

CÓDIGO

```
{{ user }}
```

Pero normalmente no mostramos `{{ user }}` directamente. Lo usamos para preguntar si el visitante inició sesión.

La condición más importante es:

CÓDIGO

```
{% if user.is_authenticated %}
```

Ejemplo en `base.html`:

CÓDIGO

```
<nav>
    <a href="{% url 'lista_posts' %}">Inicio</a>

    {% if user.is_authenticated %}
        <span>Hola, {{ user.username }}</span>
    {% else %}
        <span>Bienvenido, invitado</span>
    {% endif %}
</nav>
```

Si el usuario inició sesión, mostramos su nombre.

Si no inició sesión, mostramos un mensaje genérico.

10. Agregar login y logout a la navegación

Ahora podemos mejorar la navegación principal.

En `base.html`:

CÓDIGO

```
<nav>
    <a href="{% url 'lista_posts' %}">Inicio</a>

    {% if user.is_authenticated %}
        <span>Hola, {{ user.username }}</span>

        <form action="{% url 'logout' %}" method="post" style="display: inline;">
            {% csrf_token %}
            <button type="submit">Cerrar sesión</button>
        </form>
    {% else %}
        <a href="{% url 'login' %}">Iniciar sesión</a>
        <a href="{% url 'registro' %}">Crear cuenta</a>
    {% endif %}
</nav>
```

Un detalle importante: para logout usamos un formulario con `method="post"`.

CÓDIGO

```
<form action="{% url 'logout' %}" method="post">
```

No usamos un simple enlace como este:

CÓDIGO

```
<a href="{% url 'logout' %}">Cerrar sesión</a>
```

En versiones actuales de Django, `LogoutView` espera recibir una petición `POST`. Por eso el botón de cerrar sesión debe estar dentro de un formulario.

También agregamos:

CÓDIGO

```
{% csrf_token %}
```

para proteger esa acción.

11. Proteger vistas con `login_required`

Hasta ahora, cualquier usuario podría intentar entrar manualmente a una URL.

Por ejemplo:

CÓDIGO

```
/posts/crear/
```

Aunque ocultemos el botón de “Crear post” en el HTML, una persona podría escribir la URL directamente en el navegador.

Por eso necesitamos proteger la vista desde Python.

Para eso usamos el decorador:

CÓDIGO

```
@login_required
```

Un decorador es una función que envuelve a otra función para agregarle un comportamiento extra.

En este caso, el comportamiento extra es:

CÓDIGO

```
Antes de ejecutar la vista, revisar si el usuario inició sesión.
```

12. Proteger la vista para crear posts

En `posts/views.py`:

CÓDIGO

```
from django.contrib.auth.decorators import login_required
from django.shortcuts import render, redirect
from .forms import PostForm


@login_required
def crear_post(request):
    if request.method == "POST":
        form = PostForm(request.POST, request.FILES)

        if form.is_valid():
            post = form.save(commit=False)
            post.autor = request.user
            post.save()
            return redirect("lista_posts")
    else:
        form = PostForm()

    return render(request, "posts/post_form.html", {
        "form": form,
        "titulo_pagina": "Crear post",
        "texto_boton": "Crear"

    })
```

La línea:

CÓDIGO

```
@login_required
```

hace que solo los usuarios logueados puedan ejecutar esta vista.

Si un usuario anónimo intenta entrar, Django lo manda automáticamente al login.

Además, usamos:

CÓDIGO

```
post = form.save(commit=False)
post.autor = request.user
post.save()
```

Esto nos permite guardar el post asignándole como autor al usuario actual.

13. ¿Qué pasa con `next`?

Cuando un usuario anónimo intenta entrar a una página protegida, Django lo redirige al login.

Por ejemplo, si intenta entrar a:

CÓDIGO

```
/posts/crear/
```

Django podría mandarlo a:

CÓDIGO

```
/accounts/login/?next=/posts/crear/
```

Ese parámetro `next` recuerda a qué página quería entrar originalmente.

Entonces el flujo puede ser:

CÓDIGO

```
1. Usuario anónimo intenta crear un post.
2. Django lo manda al login.
3. El usuario inicia sesión.
4. Django lo devuelve a la página que quería usar.
```

Para que esto funcione bien, el template de login puede incluir un campo oculto:

CÓDIGO

```
<input type="hidden" name="next" value="{{ next }}" />
```

Template completo:

CÓDIGO

```
{% extends "posts/base.html" %}

{% block content %}
    <h1>Iniciar sesión</h1>

    {% if form.errors %}
        <p>El usuario o la contraseña no son correctos.</p>
    {% endif %}

    <form method="post">
        {% csrf_token %}

        {{ form.as_p }}

        <input type="hidden" name="next" value="{{ next }}" />

        <button type="submit">Entrar</button>
    </form>

    <p>
        ¿No tenés cuenta?
        <a href="{% url 'registro' %}">Crear cuenta</a>
    </p>
{% endblock %}
```

14. Ocultar acciones en el template

Además de proteger la vista, podemos mejorar la interfaz.

Por ejemplo, en el listado de posts podemos mostrar el botón “Crear post” solo a usuarios logueados.

CÓDIGO

```
{% if user.is_authenticated %}
    <a href="{% url 'crear_post' %}">Crear nuevo post</a>
{% endif %}
```

Esto mejora la experiencia del usuario, pero no reemplaza la protección en la vista.

La regla es:

CÓDIGO

```
El template mejora lo que se ve.
La vista protege lo que se puede hacer.
```

15. Proteger edición y borrado

Si ya tenemos vistas para editar o borrar posts, también deberíamos protegerlas.

Ejemplo:

CÓDIGO

```
from django.contrib.auth.decorators import login_required
from django.shortcuts import get_object_or_404, render, redirect
from .models import Post
from .forms import PostForm


@login_required
def editar_post(request, slug):
    post = get_object_or_404(Post, slug=slug)

    if request.method == "POST":
        form = PostForm(request.POST, request.FILES, instance=post)

        if form.is_valid():
            form.save()
            return redirect("detalle_post", slug=post.slug)
    else:
        form = PostForm(instance=post)

    return render(request, "posts/post_form.html", {
        "form": form,
        "post": post,
        "titulo_pagina": "Editar post",
        "texto_boton": "Guardar cambios"
    })
```

Con `@login_required`, evitamos que usuarios anónimos editen posts.

Sin embargo, todavía falta una regla más: validar que el usuario sea el autor del post. Eso lo podemos trabajar en una unidad posterior sobre permisos y propiedad del contenido.

Por ahora, el primer paso es impedir que visitantes anónimos usen estas vistas.

16. Login requerido no significa permiso total

Es muy importante entender esta diferencia:

CÓDIGO

```
login_required = el usuario inició sesión.
```

Pero eso no significa automáticamente:

CÓDIGO

```
el usuario puede hacer cualquier cosa.
```

Ejemplo de problema:

CÓDIGO

```
@login_required
def eliminar_post(request, slug):
    post = get_object_or_404(Post, slug=slug)
    post.delete()
    return redirect("lista_posts")
```

Ese código impide que usuarios anónimos borren posts, pero permite que cualquier usuario logueado borre cualquier post.

Todavía necesitaríamos validar permisos o propiedad:

CÓDIGO

```
if post.autor != request.user:
    return HttpResponseForbidden("No tenés permiso para eliminar este post.")
```

Esta diferencia es clave:

- Autenticación: saber quién es el usuario.
- Autorización: decidir qué puede hacer ese usuario.

17. Error común: usar logout con un enlace

Incorrecto:

CÓDIGO

```
<a href="{% url 'logout' %}">Cerrar sesión</a>
```

Correcto:

CÓDIGO

```
<form action="{% url 'logout' %}" method="post">
    {% csrf_token %}
    <button type="submit">Cerrar sesión</button>
</form>
```

Cerrar sesión modifica el estado de la sesión. Por eso conviene hacerlo mediante `POST`.

18. Error común: ocultar el botón pero no proteger la vista

Incorrecto:

CÓDIGO

```
{% if user.is_authenticated %}
    <a href="{% url 'crear_post' %}">Crear post</a>
{% endif %}
```

Eso solo oculta el enlace.

Correcto:

CÓDIGO

```
@login_required
def crear_post(request):
    ...
```

Lo ideal es usar ambas cosas:

- En el template, ocultamos o mostramos opciones.
- En la vista, protegemos la acción real.

19. Error común: olvidar `LOGIN_URL`

Si usamos:

CÓDIGO

```
@login_required
```

pero no configuramos correctamente `LOGIN_URL`, Django puede redirigir a una URL que no existe o que no coincide con nuestro proyecto.

Por eso en `settings.py` agregamos:

CÓDIGO

```
LOGIN_URL = "login"
```

Y en `accounts/urls.py` debemos tener una URL con ese nombre:

CÓDIGO

```
path("login/", auth_views.LoginView.as_view(template_name="accounts/login.html"),
name="login")
```

20. Error común: no incluir las URLs de `accounts`

Podemos tener bien armado `accounts/urls.py`, pero si olvidamos incluirlo en las URLs principales, no funcionará.

En `blog_project/urls.py`:

CÓDIGO

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("posts.urls")),
    path("accounts/", include("accounts.urls")),
]
```

Sin esta línea:

CÓDIGO

```
path("accounts/", include("accounts.urls")),
```

Django no encontrará `/accounts/login/`, `/accounts/logout/` ni `/accounts/registro/`.

21. Síntesis

En esta unidad conectamos el registro de usuarios con el inicio y cierre de sesión.

Aprendimos que:

- `LoginView` permite iniciar sesión usando una vista nativa de Django.
- `LogoutView` permite cerrar sesión.
- El template de login recibe un formulario listo para usar.
- `LOGIN_URL` define a dónde mandar usuarios anónimos.
- `LOGIN_REDIRECT_URL` define a dónde ir después de iniciar sesión.
- `LOGOUT_REDIRECT_URL` define a dónde ir después de cerrar sesión.
- `user.is_authenticated` permite cambiar lo que se muestra en los templates.
- `login_required` protege vistas para que solo usuarios logueados puedan acceder.
- Ocultar botones en HTML no alcanza: también hay que proteger las vistas.
- Cerrar sesión debe hacerse con un formulario `POST`.

Con esto, nuestro blog ya tiene un flujo básico de autenticación: los usuarios pueden registrarse, iniciar sesión, navegar como usuarios identificados y cerrar sesión.

Perfil de usuario y avatar

Aprende a extender el modelo de usuario nativo de Django para crear perfiles personalizados con avatar, diferenciando claramente entre datos de cuenta y datos de perfil.

Hasta ahora, nuestro blog permite que los usuarios se registren, inicien sesión y creen publicaciones.

Pero todavía falta algo importante: darle identidad a cada autor.

En muchos sitios web no vemos solamente un nombre de usuario. También vemos una foto de perfil, una pequeña biografía o algún dato que nos ayuda a reconocer quién está detrás de cada publicación.

En esta unidad vamos a crear un **perfil de usuario** conectado al usuario nativo de Django. Ese perfil nos permitirá guardar información extra, como una biografía, un link personal y un avatar.

1. Usuario vs perfil

Django ya trae un modelo de usuario llamado `User`.

Ese usuario representa la información necesaria para entrar al sistema:

- Nombre de usuario.
- Contraseña.
- Email.
- Nombre.
- Apellido.
- Permisos.

Podemos pensar al usuario como la parte relacionada con el acceso.

CÓDIGO

```
User = datos para iniciar sesión
```

Pero en un blog también podemos necesitar información adicional:

- Biografía.
- Avatar.
- Link personal.
- Redes sociales.

Esa información no es estrictamente necesaria para iniciar sesión. Por eso la guardaremos en otro modelo llamado `Perfil`.

CÓDIGO

```
Perfil = datos extra del usuario
```

2. ¿Por qué crear un modelo Perfil?

Podríamos intentar guardar todos los datos directamente en el modelo `User`, pero para este curso vamos a mantener una separación simple:

CÓDIGO

```
User -> maneja el acceso al sistema
Perfil -> maneja la información adicional del usuario
```

Esto nos permite agregar datos nuevos sin modificar directamente el sistema de autenticación de Django.

Por ejemplo, si mañana queremos sumar un campo como `biografia`, `avatar` o `link_web`, lo agregamos al modelo `Perfil`.

3. Crear el modelo Perfil

Vamos a trabajar dentro de la app `accounts`.

En `accounts/models.py`:

CÓDIGO

```
from django.db import models
from django.contrib.auth.models import User



class Perfil(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE,
related_name="perfil")
    biografia = models.TextField(max_length=500, blank=True)
    link_web = models.URLField(max_length=200, blank=True)
    avatar = models.ImageField(upload_to="avatares/", null=True, blank=True)

    def __str__(self):
        return f"Perfil de {self.user.username}"
```

Veamos las partes importantes.

La relación con el usuario:

CÓDIGO

```
user = models.OneToOneField(User, on_delete=models.CASCADE, related_name="perfil")
```

Esto significa que cada usuario tendrá un solo perfil.

La opción:

CÓDIGO

```
on_delete=models.CASCADE
```

significa que, si se borra el usuario, también se borra su perfil.

4. Relación uno a uno

`OneToOneField` significa relación uno a uno.

En nuestro caso:

CÓDIGO

```
Un usuario tiene un perfil.
Un perfil pertenece a un usuario.
```

Ejemplo:

CÓDIGO

```
Usuario: ana
Perfil: perfil de ana
```

No queremos que Ana tenga muchos perfiles. Tampoco queremos que un mismo perfil pertenezca a varios usuarios.

Por eso usamos `OneToOneField`.

5. Agregar campos extra al perfil

El campo `biografia` permite guardar una presentación corta:

CÓDIGO

```
biografia = models.TextField(max_length=500, blank=True)
```

Usamos `blank=True` porque no queremos obligar al usuario a completar su biografía.

El campo `link_web` permite guardar un enlace personal:

CÓDIGO

```
link_web = models.URLField(max_length=200, blank=True)
```

También es opcional.

El campo `avatar` permite guardar un avatar de perfil:

CÓDIGO

```
avatar = models.ImageField(upload_to="avatares/", null=True, blank=True)
```

La opción:

CÓDIGO

```
upload_to="avatares/"
```

le dice a Django que guarde esas imágenes dentro de una carpeta llamada `avatares`.

La opción:

CÓDIGO

```
null=True, blank=True
```

permite que el usuario no tenga avatar todavía.

Esto es importante porque un usuario debería poder registrarse aunque no tenga una foto de perfil cargada.

6. ¿Dónde se guarda realmente el avatar?

Un avatar no se guarda completo dentro de la base de datos.

Lo que hace Django es:

CÓDIGO

```
1. Guarda el archivo de avatar en una carpeta del proyecto.
2. Guarda en la base de datos la ruta hacia ese archivo.
```

Por ejemplo, si una usuaria llamada Ana sube un avatar, Django podría guardarlo en:

CÓDIGO

```
media/avatares/foto_ana.png
```

Y en la base de datos se guarda una referencia parecida a:

CÓDIGO

```
avatares/foto_ana.png
```

Esto hace que la base de datos no se llene de archivos pesados.

7. Instalar Pillow

Para usar `ImageField`, necesitamos instalar una librería llamada `Pillow`.

En la terminal, con el entorno virtual activado:

CÓDIGO

```
pip install Pillow
```

Si el proyecto usa `requirements.txt`, podemos actualizarlo con:

CÓDIGO

```
pip freeze > requirements.txt
```

`Pillow` permite que Django trabaje correctamente con imágenes.

8. Crear y aplicar migraciones

Como agregamos un modelo nuevo, debemos crear y aplicar migraciones.

CÓDIGO

```
python manage.py makemigrations
python manage.py migrate
```

Esto crea la tabla necesaria para guardar los perfiles.

Si olvidamos este paso, Django no tendrá dónde guardar la información del modelo `Perfil`.

9. Registrar Perfil en el admin

Para ver los perfiles desde el panel de administración, registramos el modelo.

En `accounts/admin.py`:

CÓDIGO

```
from django.contrib import admin
from .models import Perfil


admin.site.register(Perfil)
```

Ahora podremos entrar al admin y ver los perfiles creados.

10. Crear un perfil cuando se registra un usuario

El modelo `Perfil` no se crea solo.

Cuando un usuario se registra, Django crea el usuario, pero nosotros también necesitamos crear su perfil.

Podemos hacerlo directamente en la vista de registro.

En `accounts/views.py`:

CÓDIGO

```
from django.shortcuts import render, redirect
from .forms import RegistroUsuarioForm
from .models import Perfil



def registro(request):
    if request.method == "POST":
        form = RegistroUsuarioForm(request.POST)

        if form.is_valid():
            user = form.save()
            Perfil.objects.create(user=user)
            return redirect("login")
    else:
        form = RegistroUsuarioForm()

    return render(request, "accounts/registro.html", {"form": form})
```

La línea clave es:

CÓDIGO

```
Perfil.objects.create(user=user)
```

Primero guardamos el usuario:

CÓDIGO

```
user = form.save()
```

Después creamos un perfil vacío para ese usuario:

CÓDIGO

```
Perfil.objects.create(user=user)
```

El flujo queda así:

CÓDIGO

```
1. El usuario completa el formulario de registro.
2. Django crea el usuario.
3. Creamos un perfil vacío conectado a ese usuario.
4. Redirigimos al login.
```

11. Cuidado con usuarios creados antes del perfil

Si ya tenías usuarios creados antes de agregar el modelo `Perfil`, esos usuarios todavía no tendrán perfil.

Por ejemplo:

- Usuarios creados desde el admin.
- Superusuarios creados antes de esta unidad.
- Usuarios registrados antes de modificar la vista `registro`.

En esos casos, si intentamos acceder a:

CÓDIGO

```
user.perfil
```

Django puede mostrar un error porque ese usuario no tiene perfil asociado.

Para este curso, una solución simple es entrar al panel de administración y crear manualmente el perfil para esos usuarios.

Más adelante, cuando editemos el perfil, también podremos usar una forma segura para obtener o crear el perfil si no existe.

12. Static vs media

Antes de mostrar el avatar, necesitamos entender una diferencia importante.

Static

Son archivos que agrega el desarrollador.

Por ejemplo:

- CSS.
- JavaScript.
- Logo del sitio.
- Imágenes fijas del diseño.

Ejemplo:

CÓDIGO

```
static/images/logo.png
```

Media

Son archivos que suben los usuarios.

Por ejemplo:

- Avatares.
- Imágenes de posts.

Ejemplo:

CÓDIGO

```
media/avatares/foto_ana.png
```

Regla simple:

CÓDIGO

```
Static = archivos del proyecto.
Media = archivos subidos por usuarios.
```

13. Configurar archivos media

En `settings.py`, agregamos:

CÓDIGO

```
MEDIA_URL = "/media/"
MEDIA_ROOT = BASE_DIR / "media"
```

`MEDIA_URL` define cómo será la URL pública para acceder a esos archivos.

Por ejemplo:

CÓDIGO

```
/media/avatares/foto_ana.png
```

`MEDIA_ROOT` define en qué carpeta real se guardarán esos archivos.

Por ejemplo:

CÓDIGO

```
media/avatares/foto_ana.png
```

14. Servir archivos media en desarrollo

Para que Django pueda mostrar las imágenes durante el desarrollo, tenemos que modificar las URLs principales del proyecto.

En `blog_project/urls.py`:

CÓDIGO

```
from django.conf import settings
from django.conf.urls.static import static
from django.contrib import admin
from django.urls import path, include


urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("posts.urls")),
    path("accounts/", include("accounts.urls")),
]

if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

Este bloque sirve para desarrollo.

La condición:

CÓDIGO

```
if settings.DEBUG:
```

indica que esto se usa mientras estamos trabajando localmente.

15. Imagen por defecto para usuarios sin avatar

¿Qué pasa si un usuario todavía no subió su avatar?

Si intentamos mostrar un avatar que no existe, podríamos ver un icono roto en el navegador.

Para evitarlo, podemos usar un avatar por defecto.

Por ejemplo, podemos guardar un avatar llamado:

CÓDIGO

```
static/images/default-user.png
```

Ese avatar será mostrado cuando el usuario no tenga avatar cargado.

16. Mostrar el avatar en un template

Una vez que el usuario tiene perfil, podemos acceder a su avatar desde el template.

Por ejemplo, en `base.html`:

CÓDIGO

```
{% load static %}

<nav>
    <a href="{% url 'lista_posts' %}">Inicio</a>

    {% if user.is_authenticated %}
        <span>Hola, {{ user.username }}</span>

        {% if user.perfil.avatar %}
            <img
                src="{{ user.perfil.avatar.url }}"
                alt="Avatar de {{ user.username }}"
                class="avatar-navbar" />
        {% else %}
            <img
                src="{% static 'images/default-user.png' %}"
                alt="Avatar por defecto"
                class="avatar-navbar" />
        {% endif %}
    {% else %}
        <a href="{% url 'login' %}">Iniciar sesión</a>
        <a href="{% url 'registro' %}">Crear cuenta</a>
    {% endif %}
</nav>
```

La condición:

CÓDIGO

```
{% if user.perfil.avatar %}
```

pregunta si el usuario tiene un avatar cargado.

Si tiene avatar, mostramos:

CÓDIGO

```
{{ user.perfil.avatar.url }}
```

Si no tiene avatar, mostramos un avatar por defecto desde `static`.

17. Estilos básicos para el avatar

Podemos agregar un estilo simple para que el avatar se vea circular.

En nuestro archivo CSS:

CÓDIGO

```
.avatar-navbar {
    width: 40px;
    height: 40px;
    border-radius: 50%;
    object-fit: cover;
}
```

La propiedad:

CÓDIGO

```
border-radius: 50%;
```

hace que la imagen se vea redonda.

La propiedad:

CÓDIGO

```
object-fit: cover;
```

evita que la imagen se deforme.

18. Mostrar avatar en el detalle de un post

Si el modelo `Post` tiene un autor conectado al usuario, podemos mostrar el avatar del autor.

Por ejemplo, en `detalle_post.html`:

CÓDIGO

```
{% load static %}

<article>
    <header>
        {% if post.autor.perfil.avatar %}
            <img
                src="{{ post.autor.perfil.avatar.url }}"
                alt="Avatar de {{ post.autor.username }}"
                class"avatar-post" />
        {% else %}
            <img
                src="{% static 'images/default-user.png' %}"
                alt="Avatar por defecto"
                class"avatar-post" />
        {% endif %}

        <p>Publicado por {{ post.autor.username }}</p>
    </header>

    <h1>{{ post.titulo }}</h1>

    <div>
        {{ post.contenido }}
    </div>
</article>
```

Así el blog empieza a verse más personal, porque cada publicación muestra quién la escribió.

19. Estilos para el avatar del post

Podemos agregar un estilo similar para el avatar dentro de una publicación.

CÓDIGO

```
.avatar-post {
    width: 64px;
    height: 64px;
    border-radius: 50%;
    object-fit: cover;
}
```

Esto hará que la imagen del autor se vea más grande que la del menú de navegación.

20. ¿Y cómo se sube o cambia el avatar?

En esta unidad dejamos preparada la estructura:

CÓDIGO

```
Modelo Perfil
Campo avatar
Configuración de media
Visualización en templates
```

Todavía no construimos el formulario para editar el perfil.

Eso lo veremos en la próxima unidad, donde aprenderemos a permitir que cada usuario pueda modificar su biografía, su link personal y su avatar.

Ahí aparecerán dos conceptos importantes:

CÓDIGO

```
request.FILES
enctype="multipart/form-data"
```

Por ahora, alcanza con entender que el avatar será un archivo media asociado al perfil del usuario.

21. Errores comunes

Confundir static con media

Incorrecto:

CÓDIGO

```
Guardar avatares en static/
```

Correcto:

CÓDIGO

```
Guardar avatares en media/
```

Recordá:

CÓDIGO

```
Static = archivos del proyecto.
Media = archivos subidos por usuarios.
```

Olvidar Pillow

Si usamos `ImageField`, necesitamos instalar Pillow:

CÓDIGO

```
pip install Pillow
```

Olvidar migraciones

Después de crear `Perfil`, debemos ejecutar:

CÓDIGO

```
python manage.py makemigrations
python manage.py migrate
```

Olvidar crear el perfil al registrar el usuario

Si en la vista de registro solo hacemos esto:

CÓDIGO

```
form.save()
```

se crea el usuario, pero no el perfil.

Para crear también el perfil:

CÓDIGO

```
user = form.save()
Perfil.objects.create(user=user)
```

Intentar mostrar un avatar sin tener perfil creado

Si un usuario no tiene perfil, esto puede fallar:

CÓDIGO

```
{{ user.perfil.avatar.url }}
```

Por eso debemos asegurarnos de crear el perfil al registrar al usuario.

Si ya existían usuarios antes de esta unidad, podemos crearles el perfil desde el admin.

Olvidar `MEDIA_URL` y `MEDIA_ROOT`

Sin esta configuración, Django no sabrá dónde guardar ni cómo mostrar los archivos subidos.

CÓDIGO

```
MEDIA_URL = "/media/"
MEDIA_ROOT = BASE_DIR / "media"
```

Olvidar servir media en desarrollo

En `blog_project/urls.py`:

CÓDIGO

```
if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

Usar `class` en templates Django

Incorrecto:

CÓDIGO

```
<img src="{{ user.perfil.avatar.url }}" className="avatar" />
```

Correcto:

CÓDIGO

```
<img src="{{ user.perfil.avatar.url }}" class="avatar" />
```

Recordemos que `className` se usa en React. En HTML normal usamos `class`.

Subir la carpeta media a GitHub

En general, no conviene subir archivos subidos por usuarios al repositorio.

Podemos agregar al `.gitignore`:

CÓDIGO

```
media/
```

El código del proyecto va a GitHub. Las imágenes subidas por usuarios son datos variables del entorno.

22. Síntesis

En esta unidad aprendimos a crear la base del perfil de usuario con avatar.

Vimos que:

- `User` representa los datos de acceso del usuario.
- `Perfil` representa información adicional del usuario.
- Usamos `OneToOneField` para conectar un usuario con su perfil.
- Usamos `ImageField` para guardar avatares.
- Para trabajar con imágenes necesitamos instalar `Pillow`.
- Los archivos subidos por usuarios se guardan como archivos media.
- `MEDIA_URL` define la URL pública de esos archivos.

- `MEDIA_ROOT` define la carpeta física donde se guardan.
- En desarrollo, debemos configurar las URLs para servir archivos media.
- Si el usuario no tiene avatar, podemos mostrar un avatar por defecto desde `static`.
- El perfil debe crearse cuando se registra el usuario.

Con esto, nuestro blog deja de mostrar usuarios genéricos y empieza a tener autores con identidad visual propia.

En la próxima unidad vamos a crear el formulario para que cada usuario pueda editar su perfil, cambiar su avatar y modificar su contraseña de forma segura.

Edición de perfil y cambio de contraseña

Aprende a crear formularios de actualización de perfil y a configurar el flujo de seguridad para el cambio de contraseñas en Django.

En la unidad anterior creamos el modelo `Perfil` para guardar información adicional del usuario, como su biografía, su link personal y su avatar.

Ahora vamos a dar el siguiente paso: permitir que cada usuario pueda **editar su propio perfil** y también **cambiar su contraseña**.

Estas dos acciones son diferentes:

- Editar perfil modifica datos públicos o visuales del usuario.
- Cambiar contraseña modifica la forma en que el usuario accede al sistema.

Por eso vamos a trabajarlas por separado.

1. ¿Qué vamos a construir?

En esta unidad vamos a crear:

- Un formulario para editar el perfil.
- Una vista para que el usuario edite su propio perfil.
- Un template con soporte para subir imágenes.
- Una URL para acceder a la edición del perfil.
- Una página para cambiar la contraseña usando herramientas nativas de Django.
- Una página de confirmación después del cambio de contraseña.

El objetivo es que el usuario pueda entrar al blog, ir a su perfil, modificar sus datos y actualizar su clave de forma segura.

2. Editar perfil no es lo mismo que cambiar contraseña

Antes de escribir código, separemos bien los conceptos.

Editar perfil

Editar perfil significa modificar datos como:

- Biografía.
- Link personal.
- Avatar.

Estos datos viven en el modelo `Perfil`.

Ejemplo:

CÓDIGO

```
Cambiar mi foto de perfil.
Actualizar mi biografía.
Agregar mi sitio web.
```

Cambiar contraseña

Cambiar contraseña significa modificar la clave que el usuario usa para iniciar sesión.

Este dato pertenece al sistema de autenticación de Django, no al modelo `Perfil`.

Ejemplo:

CÓDIGO

```
Ingresar contraseña actual.
Ingresar nueva contraseña.
Confirmar nueva contraseña.
Guardar el cambio.
```

Dicho simple:

CÓDIGO

```
Perfil = cómo me muestro.
Contraseña = cómo entro.
```

3. Crear el formulario de perfil

Para editar el perfil vamos a usar un `ModelForm`.

En `accounts/forms.py`:

CÓDIGO

```
from django import forms
from .models import Perfil




class PerfilForm(forms.ModelForm):
    class Meta:
        model = Perfil
        fields = ["biografia", "link_web", "avatar"]
```

Este formulario se basa en el modelo `Perfil`.

La clase `Meta` le indica a Django:

CÓDIGO

```
model = Perfil
```

que el formulario trabaja con el modelo `Perfil`.

Y también:

CÓDIGO

```
fields = ["biografia", "link_web", "avatar"]
```

que esos son los campos que queremos permitir editar.

No incluimos el campo `user`, porque el usuario no debería poder elegir a qué cuenta pertenece el perfil. El perfil siempre debe ser el del usuario logueado.

4. Crear la vista para editar perfil

Ahora creamos una vista para mostrar y procesar el formulario.

En `accounts/views.py`:

CÓDIGO

```
from django.contrib.auth.decorators import login_required
from django.shortcuts import render, redirect
from .forms import PerfilForm


@login_required
def editar_perfil(request):
    perfil = request.user.perfil

    if request.method == "POST":
        form = PerfilForm(request.POST, request.FILES, instance=perfil)

        if form.is_valid():
            form.save()
            return redirect("lista_posts")
    else:
        form = PerfilForm(instance=perfil)

    return render(request, "accounts/editar_perfil.html", {"form": form})
```

Veamos las partes importantes.

Primero protegemos la vista:

CÓDIGO

```
@login_required
```

Esto evita que un usuario anónimo pueda editar un perfil.

Luego obtenemos el perfil del usuario actual:

CÓDIGO

```
perfil = request.user.perfil
```

Esto es clave. No buscamos un perfil por ID en la URL, porque no queremos que un usuario pueda editar perfiles ajenos.

Después, cuando el formulario llega por `POST`, usamos:

CÓDIGO

```
form = PerfilForm(request.POST, request.FILES, instance=perfil)
```

`request.POST` trae los datos de texto.

`request.FILES` trae los archivos subidos, como el avatar.

`instance=perfil` le indica a Django que estamos editando un perfil existente, no creando uno nuevo.

5. ¿Por qué usamos `instance=perfil`?

Si escribiéramos esto:

CÓDIGO

```
form = PerfilForm(request.POST, request.FILES)
```

Django pensaría que queremos crear un perfil nuevo.

Pero nosotros queremos editar el perfil del usuario actual.

Por eso usamos:

CÓDIGO

```
form = PerfilForm(request.POST, request.FILES, instance=perfil)
```

Esto significa:

CÓDIGO

```
Usá estos datos para actualizar este perfil existente.
```

El mismo criterio se usa cuando mostramos el formulario vacío:

CÓDIGO

```
form = PerfilForm(instance=perfil)
```

Así el formulario aparece cargado con los datos actuales del usuario.

6. Cuidado con usuarios sin perfil

Si todos los usuarios se registraron después de crear el modelo `Perfil`, probablemente cada usuario tenga su perfil.

Pero puede pasar que ya existan usuarios anteriores, por ejemplo creados desde el admin.

En ese caso, esta línea podría fallar:

CÓDIGO

```
perfil= request.user.perfil
```

Para evitarlo, podemos usar:

CÓDIGO

```
perfil, creado = Perfil.objects.get_or_create(user=request.user)
```

La vista quedaría así:

CÓDIGO

```
from django.contrib.auth.decorators import login_required
from django.shortcuts import render, redirect
from .forms import PerfilForm
from .models import Perfil


@login_required
def editar_perfil(request):
    perfil, creado = Perfil.objects.get_or_create(user=request.user)

    if request.method == "POST":
        form = PerfilForm(request.POST, request.FILES, instance=perfil)

        if form.is_valid():
            form.save()
            return redirect("lista_posts")
    else:
        form = PerfilForm(instance=perfil)

    return render(request, "accounts/editar_perfil.html", {"form": form})
```

Esta versión es más segura para el curso porque evita errores si algún usuario todavía no tiene perfil.

7. Crear la URL de edición de perfil

En `accounts/urls.py`:

CÓDIGO

```
from django.urls import path
from django.contrib.auth import views as auth_views
from . import views


urlpatterns = [
    path("registro/", views.registro, name="registro"),
    path("login/", auth_views.LoginView.as_view(template_name="accounts/
login.html"), name="login"),
    path("logout/", auth_views.LogoutView.as_view(), name="logout"),
    path("perfil/editar/", views.editar_perfil, name="editar_perfil"),
]
```

La URL final será:

CÓDIGO

```
/accounts/perfil/editar/
```

8. Crear el template para editar perfil

Creamos el archivo:

CÓDIGO

```
accounts/templates/accounts/editar_perfil.html
```

Contenido:

CÓDIGO

```
{% extends "posts/base.html" %}

{% block content %}
    <h1>Editar perfil</h1>

    <form method="post" enctype="multipart/form-data">
        {% csrf_token %}

        {{ form.as_p }}


        <button type="submit">{{ texto_boton }}</button>
    </form>
{% endblock %}
```

La línea más importante es:

CÓDIGO

```
<form method="post" enctype="multipart/form-data">
```

Cuando un formulario sube archivos, necesitamos agregar:

CÓDIGO

```
enctype="multipart/form-data"
```

Si olvidamos ese atributo, la imagen no llegará correctamente a Django.

También necesitamos:

CÓDIGO

```
{% csrf_token %}
```

porque estamos enviando datos por `POST`.

9. Agregar enlace para editar perfil

En `base.html`, podemos agregar un enlace visible solo para usuarios logueados.

CÓDIGO

```
{% if user.is_authenticated %}
    <a href="{% url 'editar_perfil' %}">Editar perfil</a>
{% endif %}
```

Ejemplo dentro de la navegación:

CÓDIGO

```
<nav>
    <a href="{% url 'lista_posts' %}">Inicio</a>

    {% if user.is_authenticated %}
        <span>Hola, {{ user.username }}</span>
        <a href="{% url 'editar_perfil' %}">Editar perfil</a>

        <form action="{% url 'logout' %}" method="post" style="display: inline;">
            {% csrf_token %}
            <button type="submit">Cerrar sesión</button>
        </form>
    {% else %}
        <a href="{% url 'login' %}">Iniciar sesión</a>
        <a href="{% url 'registro' %}">Crear cuenta</a>
    {% endif %}
</nav>
```

10. Mostrar una vista simple de perfil

Además de editar el perfil, podemos crear una página simple para que el usuario vea sus datos.

En `accounts/views.py`:

CÓDIGO

```
from django.contrib.auth.decorators import login_required
from django.shortcuts import render
from .models import Perfil


@login_required
def perfil(request):
    perfil, creado = Perfil.objects.get_or_create(user=request.user)
    return render(request, "accounts/perfil.html", {"perfil": perfil})
```

En `accounts/urls.py`:

CÓDIGO

```
path("perfil/", views.perfil, name="perfil"),
```

Template:

CÓDIGO

```
accounts/templates/accounts/perfil.html
```

Contenido:

CÓDIGO

```
{% extends "posts/base.html" %}
{% load static %}

{% block content %}
    <h1>Mi perfil</h1>

    {% if perfil.avatar %}
        <img src="{{ perfil.avatar.url }}" alt="Avatar" class="avatar-post" />
    {% else %}
        <img src="{% static 'images/default-user.png' %}" alt="Avatar por defecto"
class="avatar-post" />
    {% endif %}

    <p><strong>Usuario:</strong> {{ user.username }}</p>
    <p><strong>Email:</strong> {{ user.email }}</p>
    <p><strong>Biografía:</strong> {{ perfil.biografia }}</p>

    {% if perfil.link_web %}
        <p>
            <strong>Web:</strong>
            <a href="{{ perfil.link_web }}">{{ perfil.link_web }}</a>
        </p>
    {% endif %}

    <a href="{% url 'editar_perfil' %}">Editar perfil</a>
{% endblock %}
```

Esta página no es obligatoria, pero ayuda a que el usuario tenga un lugar claro desde donde ver y editar sus datos.

11. Cambio de contraseña

Cambiar una contraseña es una acción sensible.

No deberíamos hacerlo manualmente con un formulario inventado desde cero.

Django ya trae una vista preparada llamada:

CÓDIGO

```
PasswordChangeView
```

Esta vista permite que un usuario logueado cambie su contraseña.

El formulario pide:

- Contraseña actual.
- Nueva contraseña.
- Confirmación de nueva contraseña.

Django se encarga de validar los datos y guardar la nueva contraseña correctamente.

12. Cambio vs recuperación de contraseña

Es importante no confundir dos procesos.

Cambio de contraseña

Ocurre cuando el usuario está logueado y recuerda su contraseña actual.

Ejemplo:

CÓDIGO

```
Estoy dentro de mi cuenta y quiero cambiar mi clave.
```

Recuperación o restablecimiento de contraseña

Ocurre cuando el usuario olvidó su contraseña y necesita recuperarla, normalmente mediante email.

Ejemplo:

CÓDIGO

```
No puedo entrar a mi cuenta porque olvidé mi clave.
```

En esta unidad trabajaremos solamente el cambio de contraseña.

No vamos a implementar recuperación por email.

13. Agregar URL para cambiar contraseña

En `accounts/urls.py`, agregamos `PasswordChangeView`.

CÓDIGO

```
from django.urls import path
from django.contrib.auth import views as auth_views
from . import views


urlpatterns = [
    path("registro/", views.registro, name="registro"),
    path("login/", auth_views.LoginView.as_view(template_name="accounts/
login.html"), name="login"),
    path("logout/", auth_views.LogoutView.as_view(), name="logout"),

    path("perfil/", views.perfil, name="perfil"),
    path("perfil/editar/", views.editar_perfil, name="editar_perfil"),

    path(
        "password/cambiar/",
        auth_views.PasswordChangeView.as_view(
            template_name="accounts/password_change.html",
            success_url="/accounts/password/cambiada/"
        ),
        name="password_change",
    ),
]
```

La URL final será:

CÓDIGO

```
/accounts/password/cambiar/
```

La parte:

CÓDIGO

```
template_name="accounts/password_change.html"
```

indica qué template usará la vista.

La parte:

CÓDIGO

```
success_url="/accounts/password/cambiada/"
```

indica a dónde redirigir después de cambiar la contraseña correctamente.

14. Página de confirmación

También necesitamos una página para mostrar que la contraseña fue modificada.

En `accounts/urls.py` agregamos:

CÓDIGO

```
path(
    "password/cambiada/",
    auth_views.PasswordChangeDoneView.as_view(
        template_name="accounts/password_change_done.html"
    ),
    name="password_change_done",
),
```

El archivo completo quedaría así:

CÓDIGO

```
from django.urls import path
from django.contrib.auth import views as auth_views
from . import views


urlpatterns = [
    path("registro/", views.registro, name="registro"),
    path("login/", auth_views.LoginView.as_view(template_name="accounts/
login.html"), name="login"),
    path("logout/", auth_views.LogoutView.as_view(), name="logout"),

    path("perfil/", views.perfil, name="perfil"),
    path("perfil/editar/", views.editar_perfil, name="editar_perfil"),

    path(
        "password/cambiar/",
        auth_views.PasswordChangeView.as_view(
            template_name="accounts/password_change.html",
            success_url="/accounts/password/cambiada/"
        ),
        name="password_change",
    ),
    path(
        "password/cambiada/",
        auth_views.PasswordChangeDoneView.as_view(
            template_name="accounts/password_change_done.html"
        ),
        name="password_change_done",
    ),
]
```

15. Template para cambiar contraseña

Creamos:

CÓDIGO

```
accounts/templates/accounts/password_change.html
```

Contenido:

CÓDIGO

```
{% extends "posts/base.html" %}

{% block content %}
    <h1>Cambiar contraseña</h1>

    <form method="post">
        {% csrf_token %}

        {{ form.as_p }}

        <button type="submit">Cambiar contraseña</button>
    </form>
{% endblock %}
```

En este caso no necesitamos `enctype="multipart/form-data"` porque no estamos subiendo archivos.

Solo enviamos datos de texto.

16. Template de contraseña cambiada

Creamos:

CÓDIGO

```
accounts/templates/accounts/password_change_done.html
```

Contenido:

CÓDIGO

```
{% extends "posts/base.html" %}

{% block content %}
    <h1>Contraseña actualizada</h1>

    <p>Tu contraseña fue cambiada correctamente.</p>

    <a href="{% url 'lista_posts' %}">Volver al inicio</a>
{% endblock %}
```

17. Agregar enlace para cambiar contraseña

Podemos agregar el enlace dentro de la página `perfil.html`:

CÓDIGO

```
<a href="{% url 'password_change' %}">Cambiar contraseña</a>
```

Por ejemplo:

CÓDIGO

```
<a href="{% url 'editar_perfil' %}">Editar perfil</a>
<a href="{% url 'password_change' %}">Cambiar contraseña</a>
```

Así el usuario tendrá desde su perfil dos acciones claras:

CÓDIGO

```
Editar mis datos
Cambiar mi contraseña
```

18. ¿Por qué no modificamos la contraseña manualmente?

No deberíamos hacer esto:

CÓDIGO

```
request.user.password = "nueva_clave"
request.user.save()
```

Ese código es incorrecto porque no guarda la contraseña con el tratamiento de seguridad que Django necesita.

Django debe guardar la contraseña como un hash, no como texto plano.

Por eso usamos herramientas propias de Django, como `PasswordChangeView`.

19. ¿La sesión se mantiene después del cambio?

Cuando cambiamos una contraseña, Django necesita actualizar la información de la sesión.

Si usamos `PasswordChangeView`, Django se encarga de manejar este flujo.

Esto evita que el usuario cambie su propia contraseña y quede desconectado inmediatamente.

Para este curso, la idea importante es:

CÓDIGO

```
No cambiamos contraseñas a mano.
Usamos la vista de Django.
```

20. Errores comunes

No proteger la edición de perfil

Incorrecto:

CÓDIGO

```
def editar_perfil(request):
    ...
```

Correcto:

CÓDIGO

```
@login_required
def editar_perfil(request):
    ...
```

La edición de perfil debe estar disponible solo para usuarios logueados.

Buscar el perfil por ID en la URL

Riesgoso:

CÓDIGO

```
def editar_perfil(request, perfil_id):
    perfil = Perfil.objects.get(id=perfil_id)
```

Mejor:

CÓDIGO

```
perfil, creado = Perfil.objects.get_or_create(user=request.user)
```

Así cada usuario edita únicamente su propio perfil.

Olvidar `request.FILES`

Incorrecto:

CÓDIGO

```
form = PerfilForm(request.POST, instance=perfil)
```

Correcto:

CÓDIGO

```
form = PerfilForm(request.POST, request.FILES, instance=perfil)
```

Si el formulario permite subir avatar, necesitamos `request.FILES`.

Olvidar `enctype`

Incorrecto:

CÓDIGO

```
<form method="post">
```

Correcto:

CÓDIGO

```
<form method="post" enctype="multipart/form-data">
```

Sin `enctype="multipart/form-data"`, el archivo no se envía correctamente.

Intentar guardar la contraseña como texto

Incorrecto:

CÓDIGO

```
request.user.password = "123456"
request.user.save()
```

Correcto:

CÓDIGO

```
Usar PasswordChangeView.
```

Django ya trae el flujo seguro para este caso.

Confundir cambio con recuperación

Cambio de contraseña:

CÓDIGO

```
El usuario está logueado y sabe su contraseña actual.
```

Recuperación de contraseña:

CÓDIGO

```
El usuario olvidó su contraseña y necesita recuperarla por email.
```

En esta unidad trabajamos solamente el cambio de contraseña.

21. Síntesis

En esta unidad aprendimos a completar el flujo de usuario de nuestro blog.

Vimos que:

- Editar perfil modifica datos como bio, link web y avatar.
- Cambiar contraseña modifica el acceso del usuario.
- Para editar `Perfil`, usamos un `ModelForm`.
- Para subir avatar, necesitamos `request.FILES`.
- En el HTML, el formulario debe tener `enctype="multipart/form-data"`.
- La vista de edición debe estar protegida con `login_required`.
- Cada usuario debe editar solo su propio perfil.
- Para cambiar contraseñas usamos `PasswordChangeView`.
- Después del cambio podemos mostrar una página de confirmación con `PasswordChangeDoneView`.
- No debemos guardar contraseñas manualmente como texto.

Con esto, nuestro blog ya tiene un flujo básico de usuarios completo: registro, login, logout, perfil, avatar, edición de datos y cambio de contraseña.

Flashcards de repaso

Unidad final compuesta por tarjetas de memoria para consolidar los conceptos clave aprendidos sobre gestión de usuarios, autenticación y perfiles en Django.

Repaso: Autenticación, usuarios y perfiles

Has completado el camino fundamental para gestionar identidades en tus aplicaciones Django. La autenticación es el corazón de la seguridad y la personalización en una aplicación web; sin ella, todos los usuarios tendrían las mismas capacidades y no podríamos ofrecer experiencias a medida.

¿Por qué es vital este repaso?

En este módulo, has aprendido desde cómo Django gestiona internamente los usuarios (`User model`) hasta cómo permitir que ellos mismos se registren y gestionen sus datos (`UserCreationForm`, perfiles personalizados).

Dominar estos conceptos te permite: - **Proteger:** Asegurar que solo personas autorizadas accedan a información sensible. - **Personalizar:** Ofrecer experiencias únicas (avatares, biografías, configuración

personal). - **Mantener:** Gestionar el ciclo de vida de una sesión, desde que un usuario entra hasta que sale de forma segura.

Estrategia para el uso de estas tarjetas

Para que esta sesión de repaso sea efectiva, te sugiero el siguiente enfoque:

1. **Recuerdo Activo (Active Recall):** Antes de hacer clic para voltear la tarjeta, intenta responder la pregunta en voz alta o por escrito. No te limites a decir "ya sé qué es eso", oblígate a articular la definición o el propósito.
2. **No memorices, relaciona:** Si una tarjeta dice "¿Para qué sirve X?", piensa en el último ejercicio de código donde lo utilizaste. Vincular el concepto con una experiencia práctica es la mejor forma de fijar el conocimiento.
3. **Identifica tus lagunas:** Si respondes una tarjeta incorrectamente o dudas mucho, márcala mentalmente y repasa el contenido de las unidades anteriores correspondientes antes de volver a intentar.

¡Es momento de consolidar lo aprendido! Disfruta el proceso.

01 ¿Cuál es la diferencia fundamental entre autenticación y autorización?

La **autenticación** verifica quién es el usuario (ej. mediante credenciales). La **autorización** determina qué acciones puede realizar ese usuario una vez autenticado (ej. permisos de lectura o edición).

02 ¿Qué rol cumple el modelo `User` en Django?

Es la clase base de Django que gestiona la información esencial (nombre de usuario, contraseña, email). Django permite ampliar esta funcionalidad creando un `Perfil` vinculado.

03 ¿Por qué se utiliza frecuentemente `OneToOneField` al crear perfiles de usuario?

Se utiliza para extender el modelo `User` (como en el perfil de usuario). Asegura que cada instancia de `User` tenga exactamente una instancia relacionada en el modelo `Perfil`.

04 ¿Para qué sirve `UserCreationForm`?

Es una clase preconstruida de Django (`django.contrib.auth.forms`) que simplifica la creación de un formulario para el registro seguro de nuevos usuarios, manejando automáticamente la validación y el hash de contraseñas.

05 ¿Qué es y qué función cumple una sesión en Django?

Es un mecanismo que utiliza cookies para mantener el estado de un usuario autenticado. Al hacer login, se crea una sesión; al hacer logout, se invalida, asegurando que el servidor "recuerde" al usuario entre peticiones.

06 ¿Cuál es la utilidad del decorador `@login_required`?

Es una herramienta (`@login_required`) que impide que usuarios no autenticados accedan a una vista específica. Si un usuario no logueado intenta entrar, es redirigido automáticamente a la página de inicio de sesión.

07 ¿Por qué es importante el "hashing" de contraseñas?

Es el proceso de transformar una contraseña legible en una cadena de caracteres fija e irreversible. Django lo hace automáticamente para nunca almacenar contraseñas en texto plano en la base de datos.

08 ¿Cómo afecta la eliminación de un usuario a su perfil vinculado?

Al ser una relación 1:1, si borras un `User` en Django, puedes configurar el modelo `Perfil` para que se elimine automáticamente también (`on_delete=models.CASCADE`). Esto evita datos huérfanos en la base de datos.

PRE-ENTREGA DEL PROYECTO

Actividad evaluable · parte del proyecto integrador

Preentrega 11: Autenticación, Usuarios y Perfiles en Django

Integrar un sistema de autenticación y gestión de usuarios en Django, incorporando registro, login, logout, perfil con biografía y avatar, y protección de rutas para que solo usuarios autenticados puedan crear, editar o eliminar posts.

Preentrega 11: Autenticación, Usuarios y Perfil en Django

¡Llegó el momento de que tu blog tenga usuarios reales!

En el módulo anterior, implementaste un CRUD funcional para publicaciones. Los posts podían crearse, editarse, eliminarse y mostrar imágenes desde la interfaz del sitio.

Ahora vas a sumar una capa fundamental para cualquier aplicación web: la gestión de usuarios. A partir de este checkpoint, los usuarios podrán registrarse, iniciar sesión, cerrar sesión y tener un perfil propio con biografía y avatar.

Además, vas a proteger la creación, edición y eliminación de posts para que solo puedan acceder usuarios autenticados.

Propósito

El objetivo de esta preentrega es integrar un sistema de gestión de usuarios en tu proyecto de Blog.

Vas a trabajar con:

- Sistema de autenticación de Django.
- Login.
- Logout.
- Registro de usuarios.
- Modelo `Perfil`.
- Relación `OneToOneField` con `User`.
- Biografia de usuario.
- Avatar de perfil.
- Protección de rutas.
- `@login_required`.
- `LoginRequiredMixin`.
- Navegación según usuario autenticado o anónimo.

Este checkpoint no exige signals. El perfil puede crearse explícitamente desde la vista de registro o con `get_or_create()` al editarlo.

Contexto de la cadena de entregables

En el **Módulo 7**, creaste la base inicial del proyecto Django.

En el **Módulo 8**, implementaste rutas, vistas, templates y archivos estáticos.

En el **Módulo 9**, conectaste el proyecto con una base de datos usando modelos, migraciones, admin y ORM.

En el **Módulo 10**, implementaste un CRUD de posts con formularios e imágenes.

Ahora, en el **Módulo 11**, vas a incorporar autenticación, registro, usuarios, perfiles y protección de vistas.

La idea es que el blog pueda diferenciar entre visitantes anónimos y usuarios registrados.

Objetivo de aprendizaje de la preentrega

El objetivo de esta preentrega es consolidar el uso del sistema de autenticación de Django y aprender a extender la información de los usuarios mediante un perfil.

Vas a practicar:

Autenticación

Permitir que los usuarios puedan iniciar y cerrar sesión.

Registro

Crear un formulario para que nuevos usuarios puedan darse de alta.

Perfil

Crear un modelo relacionado con el usuario para guardar información adicional.

Relación OneToOne

Vincular cada usuario con un único perfil.

Media files

Permitir que cada perfil tenga un avatar asociado.

Protección de rutas

Evitar que usuarios no autenticados puedan crear, editar o borrar publicaciones.

Navegación condicional

Mostrar opciones diferentes en el menú según si el usuario inició sesión o no.

¿Qué construir?

Debés continuar trabajando sobre el proyecto Django del módulo anterior.

No crees un proyecto nuevo.

El objetivo es sumar autenticación y perfiles al blog.

Podés crear una app nueva llamada `accounts` para organizar todo lo relacionado con usuarios.

Estructura sugerida:

CÓDIGO

```
blog_django/
│
├── manage.py
├── README.md
├── requirements.txt
│
├── blog_project/
│   ├── settings.py
│   └── urls.py
│
├── media/
│   ├── posts/
│   └── avatares/
│
├── posts/
│   ├── admin.py
│   ├── forms.py
│   ├── models.py
│   ├── urls.py
│   ├── views.py
│   └── templates/
│       └── posts/
│           ├── base.html
│           ├── lista_posts.html
│           ├── detalle_post.html
│           ├── post_form.html
│           └── post_confirm_delete.html
│
└── accounts/
    ├── __init__.py
    ├── admin.py
    ├── apps.py
    ├── forms.py
    ├── models.py
    ├── urls.py
    ├── views.py
    └── templates/
        └── accounts/
            ├── login.html
            ├── registro.html
            ├── perfil.html
            ├── editar_perfil.html
            ├── password_change.html
            └── password_change_done.html
```

La estructura puede variar según cómo esté organizado tu proyecto, pero las funcionalidades deben estar presentes.

Requisitos técnicos

1. Continuidad del proyecto

Debés continuar trabajando sobre el mismo repositorio de GitHub utilizado en los módulos anteriores.

El proyecto debe seguir funcionando con:

CÓDIGO

```
python manage.py runserver
```

El CRUD de posts con imágenes del módulo anterior debe seguir funcionando.

2. Crear una app para usuarios

Creá una app para organizar la lógica de usuarios.

Nombre sugerido:

CÓDIGO

```
accounts
```

Comando sugerido:

CÓDIGO

```
python manage.py startapp accounts
```

Luego registrá la app en `INSTALLED_APPS`.

3. Configurar login

Utilizá el sistema de autenticación de Django incluido en:

CÓDIGO

```
django.contrib.auth
```

Implementá una vista para iniciar sesión.

Podés usar la vista nativa de Django:

CÓDIGO

```
LoginView
```

La página de login debe tener un template propio y mantener el diseño general del blog.

4. Configurar logout

Implementá una vista para cerrar sesión.

Podés usar la vista nativa de Django:

CÓDIGO

```
LogoutView
```

Después de cerrar sesión, el usuario debe ser redirigido a una página coherente, por ejemplo el listado de posts.

5. Configurar redirecciones

En `settings.py`, configurá las redirecciones principales del sistema de autenticación.

Por ejemplo:

CÓDIGO

```
LOGIN_URL = 'login'
LOGIN_REDIRECT_URL = 'lista_posts'
LOGOUT_REDIRECT_URL = 'lista_posts'
```

Los nombres pueden cambiar según las URLs definidas en tu proyecto.

6. Crear formulario de registro

Creá un formulario personalizado para que los usuarios puedan registrarse.

Podés basarte en:

CÓDIGO

```
UserCreationForm
```

El formulario debe permitir crear un nuevo usuario en el sistema.

Luego, creá una vista de registro que procese ese formulario y guarde el usuario.

Después de registrarse, el usuario debe ser redirigido a una página clara, por ejemplo:

- Login.
- Inicio.
- Perfil.

7. Crear el modelo `Perfil`

Creá un modelo llamado:

CÓDIGO

```
Perfil
```

Este modelo debe estar relacionado con el modelo `User` de Django mediante una relación uno a uno.

Debe incluir, como mínimo:

- Usuario.
- Biografía.
- Avatar.

Campos sugeridos:

CÓDIGO

```
user
biografia
link_web
avatar
```

La relación con `User` debe ser de tipo:

CÓDIGO

```
OneToOneField
```

La biografía puede ser un `TextField`.

El avatar debe ser un `ImageField`.

8. Configurar imagen de perfil

El campo de avatar del perfil debe permitir subir un avatar asociado al usuario.

Podés usar una ruta como:

CÓDIGO

```
avatares/
```

para guardar las imágenes dentro de la carpeta `media`.

Recordá que, para mostrar imágenes, el proyecto debe tener configurado:

- `MEDIA_URL`
- `MEDIA_ROOT`

Si esto ya fue configurado en el módulo anterior, verificá que siga funcionando correctamente.

9. Crear y aplicar migraciones

Después de crear el modelo `Perfil`, ejecutá:

CÓDIGO

```
python manage.py makemigrations
```

Luego:

CÓDIGO

```
python manage.py migrate
```

Esto creará la tabla correspondiente en la base de datos.

10. Registrar el perfil en el admin

Registrá el modelo `Perfil` en el panel administrativo.

De esta forma, podrás ver y administrar perfiles desde `/admin/`.

11. Crear página de perfil

Creá una vista para que cada usuario autenticado pueda ver su perfil.

La página debe mostrar, como mínimo:

- Nombre de usuario.
- Email, si está disponible.
- Biografía.
- Avatar de perfil, si existe.

Esta vista debe estar protegida para usuarios autenticados.

12. Editar perfil

Creá una vista y un formulario para editar los datos del perfil.

Debe permitir modificar, como mínimo:

- Biografía.
- Avatar.

Si también se trabajó la edición de datos del usuario, podés permitir modificar:

- Nombre.
- Apellido.
- Email.

La edición del perfil debe procesar imágenes correctamente usando `request.FILES`.

13. Proteger rutas del CRUD

Las acciones sensibles sobre posts deben requerir autenticación.

Como mínimo, protegé las vistas de:

- Crear post.
- Editar post.
- Borrar post.

Si usás vistas basadas en funciones, aplicá:

CÓDIGO

```
@login_required
```

Si usás vistas basadas en clases, aplicá:

CÓDIGO

```
LoginRequiredMixin
```

El listado y el detalle de posts pueden seguir siendo públicos.

14. Actualizar navegación

Actualizá `base.html` para que el menú muestre opciones diferentes según el estado del usuario.

Si el usuario no inició sesión, debería ver:

- Iniciar sesión.
- Registrarse.

Si el usuario inició sesión, debería ver:

- Mi perfil.
- Crear post.
- Cerrar sesión.

También podés mostrar un mensaje como:

CÓDIGO

```
Hola, {{ user.username }}
```

15. Actualizar README

Actualizá el archivo `README.md` con una breve explicación de:

- Cómo funciona el registro.

- Cómo iniciar sesión.
- Cómo cerrar sesión.
- Cómo se creó el modelo `Perfil`.
- Cómo se configuran los avatares de perfil.
- Qué rutas están protegidas.
- Qué vistas requieren autenticación.

Funcionalidades requeridas

Registro de usuarios

El sitio debe permitir que una persona cree una cuenta nueva.

Login

El sitio debe permitir iniciar sesión con un usuario existente.

Logout

El sitio debe permitir cerrar sesión.

Perfil con biografía y avatar

Cada usuario debe poder tener un perfil asociado con biografía y avatar.

Página de perfil

El usuario autenticado debe poder ver su perfil.

Edición de perfil

El usuario autenticado debe poder modificar su biografía e imagen.

Protección de posts

Crear, editar y borrar posts deben ser acciones disponibles solo para usuarios autenticados.

Criterios de aceptación

Para considerar este checkpoint como completado, tu proyecto debe cumplir con los siguientes puntos:

- Continuar sobre el proyecto Django del módulo anterior.
- Mantener funcionando el CRUD de posts con imágenes.
- Crear una app para usuarios o una organización equivalente.
- Usar el sistema de autenticación de Django.
- Implementar login.
- Implementar logout.
- Crear un formulario personalizado de registro.

- Permitir registrar nuevos usuarios.
- Crear el modelo `Perfil`.
- Relacionar `Perfil` con `User` usando `OneToOneField`.
- Agregar al perfil un campo `biografia`.
- Agregar al perfil un campo `avatar`.
- Crear y aplicar migraciones para `Perfil`.
- Registrar `Perfil` en el admin.
- Crear una página de perfil.
- Permitir editar biografia y avatar del perfil.
- Procesar correctamente imágenes con `request.FILES` en la edición de perfil.
- Mantener configurado `MEDIA_URL`.
- Mantener configurado `MEDIA_ROOT`.
- Mostrar el avatar de perfil si existe.
- Proteger la creación de posts.
- Proteger la edición de posts.
- Proteger el borrado de posts.
- Mantener listado y detalle de posts visibles para visitantes anónimos.
- Actualizar la navegación según si el usuario está autenticado o no.
- Actualizar el `README.md`.
- Subir los cambios al repositorio actual de GitHub.
- El proyecto debe poder ejecutarse con:

CÓDIGO

```
python manage.py runserver
```

Pasos sugeridos

1. Revisar el proyecto anterior

Antes de comenzar, verificá que el CRUD de posts con imágenes siga funcionando correctamente.

Probá:

- Listar posts.
- Ver detalle.
- Crear post.
- Editar post.
- Borrar post.
- Subir imagen a un post.

2. Crear la app de usuarios

Creá la app `accounts` o una estructura equivalente para manejar usuarios.

Registrala en `INSTALLED_APPS`.

3. Configurar login y logout

Agregá las rutas necesarias para iniciar y cerrar sesión.

Usá las vistas nativas de Django si fueron trabajadas en clase.

4. Crear templates de autenticación

Creá templates para login y registro.

Asegurate de que extiendan del template base del sitio.

5. Crear formulario de registro

Creá un formulario personalizado basado en `UserCreationForm`.

Luego implementá la vista de registro.

6. Crear modelo `Perfil`

Definí el modelo `Perfil` con relación `OneToOneField` hacia `User`.

Agregá los campos de biografia y avatar.

7. Migrar la base de datos

Generá y aplicá migraciones para que la tabla de perfiles se cree correctamente.

8. Registrar `Perfil` en admin

Agregá el modelo al panel administrativo para poder revisar los perfiles creados.

9. Crear página de perfil

Agregá una vista protegida para mostrar el perfil del usuario autenticado.

10. Crear edición de perfil

Agregá un formulario y una vista para editar biografía e avatar.

Recordá procesar los archivos enviados por el formulario.

11. Proteger vistas de posts

Agregá protección a las vistas de crear, editar y borrar posts.

Probá acceder a esas URLs sin iniciar sesión.

12. Actualizar navegación

Modificá el menú del sitio para mostrar opciones según el estado de autenticación.

13. Probar el flujo completo

Probá los siguientes casos:

- Registrarte como usuario nuevo.
- Iniciar sesión.
- Ver el perfil.
- Editar biografía.
- Subir avatar de perfil.
- Cerrar sesión.
- Intentar crear un post sin estar logueado.
- Iniciar sesión y crear un post.
- Editar un post.
- Borrar un post.

14. Actualizar README

Documentá cómo funciona el sistema de usuarios y perfiles.

15. Subir cambios

Realizá un commit descriptivo y subí los cambios al repositorio actual.

Errores comunes a evitar

Olvidar las migraciones

Después de crear el modelo `Perfil`, debés ejecutar `makemigrations` y `migrate`.

No configurar o revisar media

Si `MEDIA_URL` y `MEDIA_ROOT` no están configurados correctamente, los avatares de perfil no se verán.

No usar `OneToOneField`

El perfil debe estar relacionado con un único usuario.

No debería usarse una relación de muchos a muchos ni una relación suelta sin vínculo claro.

No proteger la creación de posts

Aunque no se muestre el enlace en el menú, una persona podría intentar entrar escribiendo la URL directa.

La vista debe estar protegida.

Dejar editar o borrar posts sin autenticación

Editar y borrar también son acciones sensibles.

Deben estar protegidas.

Olvidar `request.FILES` en perfil

Si el formulario de perfil permite subir avatar, la vista debe procesar archivos.

Intentar mostrar un avatar inexistente

Antes de mostrar la imagen del perfil, verificá que exista.

Confundir login con registro

Login sirve para entrar con un usuario existente.

Registro sirve para crear un usuario nuevo.

Romper el CRUD anterior

Al sumar autenticación, asegurate de no romper las funcionalidades del módulo anterior.

Formato de entrega

La entrega debe realizarse mediante el mismo repositorio público de GitHub utilizado en los módulos anteriores.

Asegurate de que:

- El repositorio siga siendo público.
- Los cambios estén subidos correctamente.
- Login funcione.
- Logout funcione.
- Registro funcione.
- Perfil funcione.
- La imagen de perfil se vea correctamente.
- Las vistas protegidas redirijan al login si el usuario no está autenticado.
- El CRUD anterior siga funcionando.
- El `README.md` esté actualizado.
- El entorno virtual no esté subido al repositorio.
- El proyecto pueda ejecutarse con:

CÓDIGO

```
python manage.py runserver
```

El enlace del repositorio será la entrega del checkpoint.

Conexión con el futuro

Este checkpoint convierte el blog en una aplicación con usuarios reales.

Hasta ahora, el sitio permitía gestionar publicaciones e imágenes. A partir de este módulo, cada usuario puede registrarse, iniciar sesión, tener un perfil con biografía e avatar, y acceder a acciones protegidas.

En el próximo módulo, vas a integrar y pulir el proyecto final: revisar navegación, estilos, documentación, flujo completo, consistencia visual y preparación de la entrega final.

En otras palabras: en el Módulo 10 el blog aprendió a gestionar publicaciones con imágenes. En el Módulo 11, aprende a reconocer a sus autores y proteger las acciones importantes.

Pasos para la entrega

El objetivo de esta preentrega es integrar un sistema completo de gestión de usuarios en tu proyecto de Blog. No solo permitirás que los usuarios se registren e inicien sesión, sino que extenderás el modelo de Django para que cada autor tenga un perfil con biografía e avatar, protegiendo además la creación de artículos para que solo usuarios autenticados puedan escribir.

Pasos sugeridos

1. **Configuración de Usuarios**: Utiliza el sistema de autenticación de Django (`django.contrib.auth`) para crear las vistas de Login y Logout.
2. **Extensión del Modelo**: Crea un modelo llamado `Perfil` con una relación `OneToOneField` con el modelo `User` de Django. Debe incluir campos como `biografia` (TextField), `link_web` (URLField opcional) y `avatar` (ImageField).
3. **Creación del perfil**: Crea el objeto `Perfil` desde la vista de registro después de guardar el usuario, usando `Perfil.objects.create(user=usuario)` .
4. **Protección de Rutas**: Aplica el decorador `@login_required` o el mixin `LoginRequiredMixin` en las vistas de creación, edición y borrado de posts.
5. **Formulario de Registro**: Crea un formulario personalizado que permita a los nuevos usuarios darse de alta en el sistema.

Errores comunes

- Olvidar las migraciones tras crear el modelo `Perfil` .
- No configurar `MEDIA_URL` y `MEDIA_ROOT` en `settings.py` , lo que impedirá que los avatares de perfil se visualicen correctamente.
- Dejar las vistas de creación de posts accesibles mediante la URL directa sin estar logueado.

Entregable

Formato: repositorio

Repositorio de GitHub con el código del proyecto Django actualizado, incluyendo el modelo Perfil, creación del perfil desde la vista de registro y vistas protegidas.

Objetivos de aprendizaje

- Implementar el sistema de autenticación nativo de Django para gestionar sesiones de usuario.
- Extender el modelo de usuario estándar mediante una relación OneToOne para añadir metadatos de perfil.
- Aplicar restricciones de acceso a vistas específicas para garantizar la seguridad del blog.
- Configurar el manejo de archivos estáticos y multimedia para las imágenes de perfil.

Criterios de aceptación

- El repositorio contiene un modelo 'Profile' relacionado con 'User'.
- Existe una funcionalidad de registro, login y logout operativa.
- Al intentar acceder a '/post/nuevo/' sin estar autenticado, el sistema redirige al login.
- El perfil de usuario se crea automáticamente al registrar un nuevo usuario mediante señales (signals).
- El archivo README.md incluye instrucciones básicas sobre cómo ejecutar las migraciones.

Rúbrica de evaluación

Rúbrica para evaluar un repositorio de Django que implementa autenticación con login/logout, registro de usuarios, perfil personalizado con relación OneToOne, creación del perfil desde la vista de registro y protección de rutas para la gestión de posts. Se valora la corrección técnica, la integración entre componentes, la organización del proyecto y la preparación del repositorio como entregable real.

Criterio	Descripción	Peso
Autenticación y formulario de registro	Evalúa si el proyecto implementa correctamente el inicio de sesión, cierre de sesión y un formulario personalizado para registrar nuevos usuarios usando el sistema de autenticación de Django.	30%
Modelo Perfil y relación con User	Evalúa la creación del modelo Perfil con relación OneToOne al usuario, incluyendo los campos solicitados y su correcta integración con el ORM de Django.	25%
Protección de rutas para posts	Evalúa si las vistas de creación, edición y borrado de artículos están protegidas para que solo puedan acceder usuarios autenticados mediante @login_required o LoginRequiredMixin.	20%
Configuración del proyecto y manejo de medios	Evalúa si el repositorio incluye la configuración necesaria para que el perfil con avatar funcione correctamente, así como migraciones y ajustes del proyecto coherentes con la funcionalidad implementada.	15%
Organización del repositorio y entrega técnica	Evalúa la calidad del repositorio como entregable real: estructura de archivos, claridad del código, coherencia entre módulos y presencia de elementos mínimos que permitan revisar e instalar el proyecto.	10%

Total: 100 pts | Aprobación: 70 pts

MÓDULO 12

Integración, Pulido y Mejora del Blog

En este módulo vas a mejorar la calidad general del proyecto Django, incorporando configuración segura, limpieza del repositorio, adaptación de un template visual externo, mensajes para el usuario, estados vacíos, búsqueda de publicaciones y revisión del flujo completo del blog.

Configuración segura y limpieza del proyecto

Preparar el proyecto Django para una entrega final más prolija y segura, configurando variables de entorno, revisando el .gitignore, limpiando archivos innecesarios y dejando el repositorio listo para compartir en GitHub.

Hasta ahora, construiste un blog funcional en Django: tiene modelos, templates, formularios, imágenes, usuarios, login, logout, perfil y rutas protegidas.

Pero antes de preparar la entrega final, hay un paso muy importante: revisar que el proyecto esté ordenado, que no tenga archivos innecesarios y que no exponga información sensible.

En esta unidad vas a aprender a limpiar tu repositorio y a separar algunas configuraciones privadas usando variables de entorno.

1. ¿Por qué necesitamos revisar la configuración?

Cuando trabajamos en un proyecto Django, muchas configuraciones viven dentro del archivo `settings.py`.

Por ejemplo:

- La clave secreta del proyecto.
- Si el sitio está en modo desarrollo.
- La configuración de archivos estáticos.
- La configuración de archivos media.
- Las apps instaladas.
- La base de datos.
- Las rutas de login y logout.

Durante el desarrollo local, esto funciona bien. Pero cuando subimos el proyecto a GitHub, tenemos que tener cuidado con qué información compartimos.

No todo lo que está en nuestro equipo debería terminar en el repositorio.

2. Información que no conviene subir a GitHub

En un proyecto real, hay ciertos datos que no deberían quedar publicados.

Por ejemplo:

- Claves secretas.
- Contraseñas.
- Tokens.
- Credenciales de servicios externos.
- Variables privadas del proyecto.
- Archivos locales del entorno virtual.
- Archivos temporales generados por Python.
- Bases de datos locales, si la consigna indica no subirlas.

En nuestro proyecto de blog, el ejemplo más claro es `SECRET_KEY`.

Django crea una clave secreta dentro de `settings.py`. Esa clave se usa internamente para tareas de seguridad, como sesiones y protección de formularios.

Aunque en este curso estamos trabajando en local, es una buena práctica empezar a entender que esa clave no debería quedar escrita directamente en un repositorio público.

3. ¿Qué son las variables de entorno?

Una variable de entorno es un valor de configuración que vive fuera del código fuente.

En lugar de escribir un dato sensible directamente en `settings.py`, podemos guardarlo en un archivo especial llamado `.env`.

Por ejemplo, en vez de tener esto:

CÓDIGO

```
SECRET_KEY = 'clave-super-secreta-escrita-en-el-codigo'
```

podemos tener esto:

CÓDIGO

```
SECRET_KEY = config('SECRET_KEY')
```

Y guardar el valor real en un archivo `.env`.

La idea es simple:

- El código dice qué variable necesita.
- El archivo `.env` guarda el valor real.

- El archivo `.env` no se sube a GitHub.

4. Instalación de `python-decouple`

Para leer variables desde un archivo `.env`, podemos usar una librería llamada `python-decouple`.

Primero, instalá la dependencia dentro del entorno virtual:

CÓDIGO

```
pip install python-decouple
```

Luego actualizá el archivo `requirements.txt`:

CÓDIGO

```
pip freeze > requirements.txt
```

Esto asegura que la dependencia quede registrada para que otra persona pueda instalarla al clonar el proyecto.

5. Crear el archivo `.env`

En la raíz del proyecto, al mismo nivel que `manage.py`, creá un archivo llamado:

CÓDIGO

```
.env
```

Dentro podés guardar configuraciones privadas.

Por ejemplo:

CÓDIGO

```
SECRET_KEY=tu-clave-secreta-de-django
DEBUG=True
```

Este archivo no debería subirse a GitHub.

6. Usar variables de entorno en `settings.py`

En `settings.py`, primero importá `config`:

CÓDIGO

```
from decouple import config
```

Luego podés reemplazar la configuración de `SECRET_KEY` y `DEBUG`.

Por ejemplo:

CÓDIGO

```
SECRET_KEY = config('SECRET_KEY')

DEBUG = config('DEBUG', default=True, cast=bool)
```

Con esto, Django buscará esos valores en el archivo `.env`.

Si `DEBUG` no existe, usará `True` como valor por defecto.

7. ¿Qué significa `DEBUG`?

La variable `DEBUG` indica si Django debe mostrar información detallada cuando ocurre un error.

Durante el desarrollo usamos:

CÓDIGO

```
DEBUG = True
```

Esto ayuda a encontrar problemas porque Django muestra mensajes detallados.

Pero en un sitio publicado, `DEBUG` debería estar en `False`, porque esos mensajes pueden mostrar información interna del proyecto.

En este curso no vamos a hacer deploy, pero es importante conocer esta diferencia.

8. Archivo `.env.example`

Como el archivo `.env` no se sube a GitHub, una buena práctica es crear un archivo de ejemplo llamado:

CÓDIGO

```
.env.example
```

Este archivo sí puede subirse al repositorio porque no contiene valores reales.

Por ejemplo:

CÓDIGO

```
SECRET_KEY=colocar-una-clave-secreta
DEBUG=True
```

La idea es que otra persona pueda ver qué variables necesita configurar para ejecutar el proyecto.

Entonces:

- `.env` tiene los valores reales y no se sube.
- `.env.example` muestra la estructura esperada y sí se puede subir.

9. Revisar `.gitignore`

El archivo `.gitignore` le indica a Git qué archivos o carpetas debe ignorar.

Esto es importante porque no todo lo que existe en nuestra computadora debería subirse al repositorio. Por ejemplo, no queremos subir el entorno virtual, archivos temporales, cachés de Python o configuraciones privadas.

Una forma práctica de generar una base bastante completa para este archivo es usar la página:

CÓDIGO

```
https://www.toptal.com/developers/gitignore/
```

Allí podés escribir tecnologías o herramientas relacionadas con tu proyecto, por ejemplo:

CÓDIGO

```
Python
Django
VisualStudioCode
Windows
macOS
Linux
```

La página genera automáticamente un `.gitignore` con muchas reglas útiles para ese tipo de proyecto.

Luego, podés copiar ese contenido en tu archivo `.gitignore` y ajustarlo según las necesidades del curso.

De todas formas, aunque uses una herramienta para generarlo, es importante revisar qué se está ignorando. La herramienta online sirve como punto de partida, pero el criterio final depende del proyecto y de la consigna de entrega.

En este proyecto, el `.gitignore` debería incluir como mínimo:

CÓDIGO

```
venv/
__pycache__/
*.pyc
.env
db.sqlite3
media/
```

Veamos qué significa cada línea:

CÓDIGO

```
venv/
```

Evita subir el entorno virtual.

CÓDIGO

```
__pycache__/
```

Evita subir archivos internos que Python genera automáticamente.

CÓDIGO

```
*.pyc
```

Evita subir archivos compilados de Python.

CÓDIGO

```
.env
```

Evita subir variables privadas.

CÓDIGO

```
db.sqlite3
```

Evita subir la base de datos local, si la consigna del curso indica que no debe compartirse.

CÓDIGO

```
media/
```

Evita subir imágenes cargadas localmente por usuarios.

10. ¿Siempre hay que ignorar `db.sqlite3` y `media/`?

Depende del criterio de la entrega.

En proyectos reales, normalmente no se sube la base de datos local ni los archivos subidos por usuarios.

Pero en un curso, a veces puede pasar que el docente quiera ver datos de prueba.

Por eso, para este proyecto recomendamos:

- Subir el código.

- Subir las migraciones.
- Subir `requirements.txt`.
- Subir README.
- No subir `venv/`.
- No subir `.env`.
- No subir `__pycache__/`.
- No subir `db.sqlite3`, salvo que la consigna lo pida explícitamente.
- No subir `media/`, salvo que la consigna lo pida explícitamente.

Si otra persona descarga el proyecto, puede recrear la base ejecutando migraciones.

11. Revisar `requirements.txt`

El archivo `requirements.txt` debe contener las dependencias necesarias para ejecutar el proyecto.

En este punto del curso, debería incluir dependencias como:

CÓDIGO

```
Django
Pillow
python-decouple
```

Puede tener versiones específicas según tu entorno.

Para actualizarlo, ejecutá:

CÓDIGO

```
pip freeze > requirements.txt
```

Esto guarda las librerías instaladas en el entorno virtual.

12. Archivos que sí deberían estar en el repositorio

Antes de entregar, asegurate de que el repositorio tenga archivos importantes como:

CÓDIGO

```
manage.py
requirements.txt
README.md
.gitignore
.env.example
```

También deberían estar:

CÓDIGO

```
blog_project/
posts/
accounts/
```

Y dentro de cada app:

CÓDIGO

```
models.py
views.py
urls.py
forms.py
admin.py
templates/
static/
migrations/
```

Las migraciones sí deben subirse, porque son necesarias para reconstruir la estructura de la base de datos.

13. Archivos que no deberían estar en el repositorio

Revisá que no se hayan subido archivos innecesarios como:

CÓDIGO

```
venv/
__pycache__/
*.pyc
.env
```

También revisá si corresponde ignorar:

CÓDIGO

```
db.sqlite3
media/
```

Esto ayuda a mantener el proyecto limpio y evita que el repositorio se vuelva pesado o contenga información local.

14. Limpieza manual del proyecto

Antes de entregar, podés revisar tu carpeta del proyecto y eliminar archivos temporales.

Por ejemplo:

- Carpetas `__pycache__`.

- Archivos `.pyc`.
- Imágenes de prueba que no quieras entregar.
- Archivos duplicados.
- Capturas viejas.
- Archivos que no forman parte del proyecto.

La idea es que el repositorio final sea claro, liviano y fácil de revisar.

15. Checklist de configuración segura

Antes de avanzar con la entrega final, revisá:

- El proyecto tiene `.gitignore`.
- `.env` está en `.gitignore`.
- `venv/` está en `.gitignore`.
- `__pycache__/` está en `.gitignore`.
- `requirements.txt` está actualizado.
- `SECRET_KEY` se lee desde una variable de entorno.
- `DEBUG` se lee desde una variable de entorno.
- Existe un archivo `.env.example`.
- Las migraciones están subidas.
- El proyecto sigue ejecutándose correctamente.
- No hay archivos privados o innecesarios en GitHub.

16. Ejemplo de estructura final limpia

Una estructura posible del proyecto podría verse así:

CÓDIGO

```
blog_django/
│
├── manage.py
├── README.md
├── requirements.txt
├── .gitignore
├── .env.example
│
├── blog_project/
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
│
├── posts/
│   ├── admin.py
│   ├── forms.py
│   ├── models.py
│   ├── urls.py
│   ├── views.py
│   ├── migrations/
│   ├── templates/
│   └── static/
│
└── accounts/
    ├── admin.py
    ├── forms.py
    ├── models.py
    ├── urls.py
    ├── views.py
    ├── migrations/
    └── templates/
```

Archivos como `.env`, `venv/`, `db.sqlite3` y `media/` pueden existir localmente, pero no necesariamente deberían estar subidos al repositorio.

17. Cierre de la unidad

En esta unidad revisaste cómo preparar el proyecto para una entrega más prolija y segura.

Aprendiste a:

- Reconocer información sensible.
- Usar variables de entorno.
- Crear un archivo `.env`.
- Crear un archivo `.env.example`.
- Leer valores desde `settings.py`.
- Revisar `.gitignore`.

- Usar una herramienta para generar una base de `.gitignore`.
- Actualizar `requirements.txt`.
- Limpiar archivos innecesarios.
- Dejar el repositorio listo para revisión.

Estos detalles no cambian la funcionalidad visible del blog, pero hacen que el proyecto sea más ordenado, profesional y fácil de compartir.

Class-Based Views como alternativa a Function-Based Views

Conocer las Class-Based Views de Django como una alternativa a las vistas basadas en funciones, aplicándolas a operaciones comunes del CRUD mediante ListView, DetailView, CreateView, UpdateView y DeleteView, junto con el uso de LoginRequiredMixin para proteger vistas basadas en clases.

Hasta ahora, la mayoría de las vistas del proyecto fueron construidas como funciones.

Por ejemplo:

CÓDIGO

```
def lista_posts(request):
    ...
```

Este enfoque se conoce como **Function-Based Views**, o vistas basadas en funciones.

Las Function-Based Views son claras, directas y muy útiles para aprender cómo funciona Django por dentro. Permiten ver paso a paso qué ocurre cuando llega una petición, cómo se procesa un formulario, cómo se consulta la base de datos y cómo se devuelve un template.

Sin embargo, Django también ofrece otra forma de escribir vistas: las **Class-Based Views**, o vistas basadas en clases.

En esta unidad vas a conocer este enfoque como una alternativa para organizar mejor algunas vistas repetitivas, especialmente en operaciones comunes como listar, ver detalle, crear, editar y eliminar registros.

1. ¿Qué son las Class-Based Views?

Las **Class-Based Views** son vistas escritas usando clases en lugar de funciones.

En vez de escribir una función como esta:

CÓDIGO

```
def lista_posts(request):
    ...
```

podemos escribir una clase como esta:

CÓDIGO

```
class PostListView(ListView):
    ...
```

Django incluye varias clases listas para resolver tareas comunes.

Por ejemplo:

- Mostrar una lista de objetos.
- Mostrar el detalle de un objeto.
- Crear un objeto.
- Editar un objeto.
- Eliminar un objeto.

Estas clases se llaman **vistas genéricas** porque resuelven patrones muy comunes del desarrollo web.

2. ¿Por qué existen las Class-Based Views?

Cuando construimos un CRUD, muchas vistas tienen una estructura repetida.

Por ejemplo, para listar posts normalmente hacemos algo como:

1. Consultar la base de datos.
2. Obtener una lista de objetos.
3. Enviar esos objetos al template.
4. Renderizar una página.

Para ver el detalle:

1. Buscar un objeto por su ID.
2. Enviarlo al template.
3. Renderizar la página.

Para crear:

1. Mostrar un formulario vacío.
2. Recibir datos por POST.
3. Validar.
4. Guardar.
5. Redirigir.

Django detecta que estos patrones se repiten mucho y ofrece clases preparadas para resolverlos con menos código.

3. Function-Based Views vs Class-Based Views

Las **Function-Based Views** suelen ser más explícitas.

Ejemplo:

CÓDIGO

```
def lista_posts(request):
    posts = Post.objects.all()
    return render(request, 'posts/lista_posts.html', {'posts': posts})
```

Se ve claramente qué está pasando.

Las **Class-Based Views** pueden ser más compactas.

Ejemplo:

CÓDIGO

```
class PostListView(ListView):
    model = Post
    template_name = 'posts/lista_posts.html'
    context_object_name = 'posts'
```

Ambos enfoques pueden resolver el mismo problema.

No se trata de que uno sea “bueno” y el otro “malo”. Son herramientas distintas.

4. ¿Cuándo conviene usar Function-Based Views?

Las Function-Based Views son una buena opción cuando:

- Estás aprendiendo Django.
- Querés entender cada paso de la lógica.
- La vista tiene un comportamiento muy personalizado.
- Necesitás controlar manualmente el flujo.
- La lógica es simple y directa.

Por eso fueron una muy buena opción para los módulos anteriores.

Ayudan a entender qué hace Django detrás de escena.

5. ¿Cuándo conviene usar Class-Based Views?

Las Class-Based Views son útiles cuando:

- La vista responde a un patrón común.
- Querés reducir código repetido.
- Estás haciendo un CRUD.
- Querés aprovechar herramientas genéricas de Django.
- Querés organizar mejor vistas similares.

Por ejemplo, para un CRUD de posts, Django ya tiene clases que pueden ayudarte a resolver varias operaciones con menos código.

6. Vistas genéricas principales

Las vistas genéricas más comunes para un CRUD son:

CÓDIGO

```
ListView
DetailView
CreateView
UpdateView
DeleteView
```

Cada una resuelve una parte del CRUD:

CÓDIGO

```
ListView   -> listar objetos
DetailView -> ver el detalle de un objeto
CreateView -> crear un objeto
UpdateView -> editar un objeto
DeleteView -> eliminar un objeto
```

Estas clases se importan desde:

CÓDIGO

```
from django.views.generic import ListView, DetailView, CreateView, UpdateView,
DeleteView
```

ListView

7. ¿Para qué sirve `ListView`?

`ListView` sirve para mostrar una lista de objetos de un modelo.

En nuestro blog, puede usarse para mostrar la lista de posts.

Ejemplo:

CÓDIGO

```
from django.views.generic import ListView
from .models import Post

class PostListView(ListView):
    model = Post
    template_name = 'posts/lista_posts.html'
    context_object_name = 'posts'
```

Veamos qué significa cada parte.

8. Atributo `model`

El atributo `model` indica qué modelo va a usar la vista.

CÓDIGO

```
model = Post
```

En este caso, Django entiende que debe trabajar con objetos del modelo `Post`.

Por defecto, `ListView` buscará todos los registros de ese modelo.

9. Atributo `template_name`

El atributo `template_name` indica qué template debe usarse para mostrar la respuesta.

CÓDIGO

```
template_name = 'posts/lista_posts.html'
```

Esto reemplaza al `render()` que escribíamos en una Function-Based View.

10. Atributo `context_object_name`

El atributo `context_object_name` define con qué nombre va a llegar la lista de objetos al template.

CÓDIGO

```
context_object_name = 'posts'
```

Gracias a esto, en el template podemos seguir usando:

CÓDIGO

```
{% for post in posts %}
    ...
{% endfor %}
```

Si no definimos este nombre, Django usa un nombre automático como `object_list`.

11. ListView con orden

También podemos modificar el orden de los posts.

Por ejemplo:

CÓDIGO

```
class PostListView(ListView):
    model = Post
    template_name = 'posts/lista_posts.html'
    context_object_name = 'posts'
    ordering = ['-fecha_creacion']
```

Esto permite mostrar primero los posts más recientes.

12. ListView con consulta personalizada

Si necesitamos una consulta más específica, podemos usar el método `get_queryset()`.

Por ejemplo, para mostrar solo posts publicados:

CÓDIGO

```
class PostListView(ListView):
    model = Post
    template_name = 'posts/lista_posts.html'
    context_object_name = 'posts'

    def get_queryset(self):
        return Post.objects.filter(estado='publicado').order_by('-fecha_creacion')
```

Este método reemplaza la consulta por defecto.

13. ListView con búsqueda simple

Si ya agregaste búsqueda con `request.GET`, también podés adaptarla a una Class-Based View.

Por ejemplo:

CÓDIGO

```
from django.db.models import Q

class PostListView(ListView):
    model = Post
    template_name = 'posts/lista_posts.html'
    context_object_name = 'posts'

    def get_queryset(self):
        busqueda = self.request.GET.get('q')
        posts = Post.objects.all().order_by('-fecha_creacion')

        if busqueda:
            posts = posts.filter(
                Q(titulo__icontains=busqueda) |
                Q(contenido__icontains=busqueda)
            )

        return posts
```

En una Class-Based View, en lugar de usar `request`, usamos:

CÓDIGO

```
self.request
```

14. Enviar datos extra al template

Si queremos enviar también la búsqueda al template, podemos usar `get_context_data()`.

CÓDIGO

```
class PostListView(ListView):
    model = Post
    template_name = 'posts/lista_posts.html'
    context_object_name = 'posts'

    def get_queryset(self):
        busqueda = self.request.GET.get('q')
        posts = Post.objects.all()

        if busqueda:
            posts = posts.filter(titulo__icontains=busqueda)

        return posts

    def get_context_data(self, **kwargs):
        contexto = super().get_context_data(**kwargs)
        contexto['busqueda'] = self.request.GET.get('q', '')
        return contexto
```

Este método permite agregar información extra al contexto del template.

DetailView

15. ¿Para qué sirve `DetailView`?

`DetailView` sirve para mostrar el detalle de un único objeto.

En nuestro blog, puede usarse para mostrar un post individual.

Ejemplo:

CÓDIGO

```
from django.views.generic import DetailView
from .models import Post

class PostDetailView(DetailView):
    model = Post
    template_name = 'posts/detalle_post.html'
    context_object_name = 'post'
```

Esta vista busca un objeto del modelo `Post` y lo envía al template con el nombre `post`.

16. URL para DetailView

Para usar una Class-Based View en `urls.py`, se utiliza `.as_view()`.

Ejemplo:

CÓDIGO

```
from django.urls import path
from .views import PostDetailView

urlpatterns = [
    path('posts/<slug:slug>/', PostDetailView.as_view(), name='detalle_post'),
]
```

El parámetro `slug` indica el identificador legible del objeto.

Django usa ese valor para buscar el post correspondiente.

CreateView

17. ¿Para qué sirve `CreateView`?

`CreateView` sirve para crear un nuevo objeto desde un formulario.

En nuestro blog, puede usarse para crear un post.

Ejemplo:

CÓDIGO

```
from django.views.generic import CreateView
from django.urls import reverse_lazy
from .models import Post
from .forms import PostForm

class PostCreateView(CreateView):
    model = Post
    form_class = PostForm
    template_name = 'posts/post_form.html'
    success_url = reverse_lazy('lista_posts')
```

Esta vista:

- Muestra el formulario.
- Recibe los datos enviados.
- Valida el formulario.
- Guarda el objeto si todo está bien.
- Redirige al usuario después de guardar.

18. Atributo `form_class`

El atributo `form_class` indica qué formulario debe usar la vista.

CÓDIGO

```
form_class = PostForm
```

Esto permite reutilizar el `ModelForm` que ya habías creado en el módulo de CRUD.

19. Atributo `success_url`

El atributo `success_url` indica a dónde se redirige al usuario después de guardar correctamente.

CÓDIGO

```
success_url = reverse_lazy('lista_posts')
```

Usamos `reverse_lazy()` porque en Class-Based Views es común definir la URL de redirección como atributo de clase.

20. CreateView con imágenes

Si el formulario tiene un `ImageField`, `CreateView` puede trabajar con archivos siempre que el template tenga:

CÓDIGO

```
<form method="post" enctype="multipart/form-data">
```

El atributo `enctype="multipart/form-data"` sigue siendo obligatorio.

Sin eso, la imagen no se envía correctamente.

UpdateView

21. ¿Para qué sirve `UpdateView`?

`UpdateView` sirve para editar un objeto existente.

En nuestro blog, puede usarse para editar un post.

Ejemplo:

CÓDIGO

```
from django.views.generic import UpdateView
from django.urls import reverse_lazy
from .models import Post
from .forms import PostForm

class PostUpdateView(UpdateView):
    model = Post
    form_class = PostForm
    template_name = 'posts/post_form.html'
    success_url = reverse_lazy('lista_posts')
```

Esta vista:

- Busca el post por su identificador.
- Muestra el formulario con los datos actuales.
- Recibe los cambios.
- Valida.
- Guarda.
- Redirige.

22. URL para UpdateView

La URL debe incluir el identificador del post.

Ejemplo:

CÓDIGO

```
path('posts/<slug:slug>/editar/', PostUpdateView.as_view(), name='editar_post')
```

Django usa el `slug` recibido en la URL para saber qué objeto editar.

DeleteView

23. ¿Para qué sirve `DeleteView`?

`DeleteView` sirve para eliminar un objeto.

En nuestro blog, puede usarse para borrar un post.

Ejemplo:

CÓDIGO

```
from django.views.generic import DeleteView
from django.urls import reverse_lazy
from .models import Post

class PostDeleteView(DeleteView):
    model = Post
    template_name = 'posts/post_confirm_delete.html'
    success_url = reverse_lazy('lista_posts')
```

Esta vista muestra una pantalla de confirmación antes de borrar.

24. Template de confirmación

El template puede ser similar al que ya usaste con Function-Based Views.

Por ejemplo:

CÓDIGO

```
{% extends 'posts/base.html' %}

{% block content %}
    <h1>Eliminar post</h1>

    <p>¿Estás seguro de que querés eliminar "{{ post.titulo }}"?</p>

    <form method="post">
        {% csrf_token %}
        <button type="submit">Sí, eliminar</button>
        <a href="{% url 'detalle_post' post.slug %}">Cancelar</a>
    </form>
{% endblock %}
```

La diferencia es que ahora la vista que controla esta página es una clase.

URLs con Class-Based Views

25. Uso de `.as_view()`

Las Class-Based Views no se conectan igual que las funciones.

Con una Function-Based View hacíamos:

CÓDIGO

```
path('', inicio, name='inicio')
```

Con una Class-Based View usamos:

CÓDIGO

```
path('posts/', PostListView.as_view(), name='lista_posts')
```

El método `.as_view()` convierte la clase en una vista que Django puede ejecutar cuando llega una petición.

26. Ejemplo de `urls.py`

Un archivo `posts/urls.py` podría quedar así:

CÓDIGO

```
from django.urls import path
from .views import (
    PostListView,
    PostDetailView,
    PostCreateView,
    PostUpdateView,
    PostDeleteView,
)

urlpatterns = [
    path('posts/', PostListView.as_view(), name='lista_posts'),
    path('posts/<slug:slug>/', PostDetailView.as_view(), name='detalle_post'),
    path('posts/crear/', PostCreateView.as_view(), name='crear_post'),
    path('posts/<slug:slug>/editar/', PostUpdateView.as_view(), name='editar_post'),
    path('posts/<slug:slug>/eliminar/', PostDeleteView.as_view(),
name='eliminar_post'),
]
```

Los nombres de rutas pueden variar según tu proyecto, pero la idea general es la misma.

LoginRequiredMixin

27. Proteger Class-Based Views

En módulos anteriores protegimos vistas con `@login_required`.

Ese decorador funciona muy bien con vistas basadas en funciones.

Por ejemplo:

CÓDIGO

```
@login_required
def crear_post(request):
    ...
```

Pero si la vista está escrita como clase, usamos otra herramienta:

CÓDIGO

```
LoginRequiredMixin
```

Este mixin permite restringir el acceso a usuarios autenticados.

28. Importar `LoginRequiredMixin`

Se importa desde:

CÓDIGO

```
from django.contrib.auth.mixins import LoginRequiredMixin
```

Luego se coloca dentro de la clase.

Ejemplo:

CÓDIGO

```
class PostCreateView(LoginRequiredMixin, CreateView):
    model = Post
    form_class = PostForm
    template_name = 'posts/post_form.html'
    success_url = reverse_lazy('lista_posts')
```

Con esto, si un usuario no autenticado intenta crear un post, Django lo redirige al login.

29. Orden del mixin

El orden importa.

El mixin debe ir primero:

CÓDIGO

```
class PostCreateView(LoginRequiredMixin, CreateView):
    ...
```

No al revés.

Esta forma permite que `LoginRequiredMixin` controle el acceso antes de que se ejecute la lógica de `CreateView`.

30. Proteger creación, edición y borrado

Normalmente, en un blog, conviene proteger las acciones sensibles.

Por ejemplo:

CÓDIGO

```
class PostCreateView(LoginRequiredMixin, CreateView):
    ...

class PostUpdateView(LoginRequiredMixin, UpdateView):
    ...

class PostDeleteView(LoginRequiredMixin, DeleteView):
    ...
```

En cambio, el listado y el detalle pueden quedar públicos:

CÓDIGO

```
class PostListView(ListView):
    ...

class PostDetailView(DetailView):
    ...
```

Esto permite que cualquier visitante pueda leer el blog, pero solo usuarios autenticados puedan modificar contenido.

31. Configurar `LOGIN_URL`

Para que Django sepa a dónde redirigir cuando alguien intenta acceder a una vista protegida, debemos tener configurado `LOGIN_URL`.

En `settings.py` puede existir algo como:

CÓDIGO

```
LOGIN_URL = 'login'
```

El valor debe coincidir con el nombre de la URL de login del proyecto.

¿Tengo que cambiar todo mi proyecto?

32. No es obligatorio reescribir todo

Esta unidad presenta las Class-Based Views como una alternativa.

Si tu proyecto ya funciona correctamente con Function-Based Views, no es obligatorio reescribir todo.

Lo importante es que entiendas que Django ofrece dos formas comunes de construir vistas:

- Function-Based Views.
- Class-Based Views.

Ambas pueden convivir en el mismo proyecto.

33. Una estrategia posible

Si querés practicar sin romper todo, podés migrar solo una parte.

Por ejemplo:

- Convertir el listado a `ListView`.
- Convertir el detalle a `DetailView`.
- Mantener creación, edición y eliminación como Function-Based Views.

O al revés:

- Mantener listado y detalle como funciones.
- Probar `CreateView` en la creación de posts.

No hace falta cambiar todo de una vez.

34. Ventajas de usar Class-Based Views

Algunas ventajas son:

- Reducen código repetido.
- Ordenan vistas comunes.
- Aprovechan herramientas listas de Django.
- Funcionan muy bien para CRUD.
- Facilitan proteger vistas con mixins.
- Permiten personalizar comportamiento sobrescribiendo métodos.

35. Desventajas o cuidados

También tienen algunos puntos a tener en cuenta:

- Al principio pueden parecer menos claras.
- Parte de la lógica está escondida dentro de la clase base.
- Hay que aprender sus atributos y métodos.
- Si se necesita una lógica muy personalizada, a veces una Function-Based View es más directa.

Por eso, para aprender Django, está perfecto empezar con funciones y luego conocer clases.

36. Checklist de la unidad

Antes de cerrar esta unidad, revisá si podés responder:

- Qué es una Function-Based View.
- Qué es una Class-Based View.
- Para qué sirve `ListView`.
- Para qué sirve `DetailView`.
- Para qué sirve `CreateView`.
- Para qué sirve `UpdateView`.
- Para qué sirve `DeleteView`.
- Qué hace `.as_view()`.
- Qué significa `model`.
- Qué significa `template_name`.
- Qué significa `context_object_name`.
- Qué significa `form_class`.
- Qué significa `success_url`.
- Para qué sirve `LoginRequiredMixin`.
- Por qué el mixin se coloca antes de la vista genérica.
- Cuándo conviene usar `@login_required`.
- Cuándo conviene usar `LoginRequiredMixin`.

37. Cierre de la unidad

En esta unidad conociste las Class-Based Views como una alternativa a las Function-Based Views.

Viste que Django incluye vistas genéricas para resolver operaciones comunes como listar, ver detalle, crear, editar y eliminar objetos.

También aprendiste que, cuando una vista está escrita como clase, se puede proteger usando `LoginRequiredMixin`.

No necesitás reemplazar todo tu proyecto para aprovechar este concepto. Lo importante es entender que Django ofrece diferentes herramientas para resolver problemas similares, y que las Class-Based Views son una opción muy utilizada cuando trabajamos con CRUD.

Integración visual con templates externos

Adaptar un template externo de Start Bootstrap al proyecto Django, usando Clean Blog como ejemplo para mejorar la presentación visual del sitio sin perder la lógica de URLs, views, templates dinámicos, archivos estáticos y funcionalidades ya construidas.

Hasta ahora, tu blog ya tiene una estructura funcional: puede mostrar publicaciones, gestionar usuarios, cargar imágenes y proteger rutas. Sin embargo, todavía puede verse bastante simple si todo el diseño fue hecho desde cero con CSS básico.

En esta unidad vas a aprender una forma práctica de mejorar la presentación visual del proyecto: usar un template HTML externo y adaptarlo a Django.

La idea no es copiar y pegar sin entender, sino aprender a tomar una plantilla ya diseñada, identificar sus partes principales y convertirla en templates reutilizables dentro del proyecto.

1. ¿Qué es un template HTML externo?

Un template HTML externo es una plantilla de diseño ya armada.

Suele incluir:

- Archivos `.html`.
- Archivos `.css`.
- Archivos `.js`.
- Imágenes.
- Fuentes.
- Iconos.
- Estructura visual lista para usar.

Estos templates sirven como punto de partida para no tener que diseñar toda la interfaz desde cero.

En lugar de construir cada estilo manualmente, podemos adaptar una base visual ya creada al proyecto Django.

2. ¿Por qué usar un template externo?

Usar un template externo puede ayudarte a mejorar rápidamente la apariencia del sitio.

Algunas ventajas son:

- El sitio se ve más profesional.
- El diseño suele ser responsive.
- Ya trae una estructura visual clara.
- Permite ahorrar tiempo.
- Ayuda a concentrarse en la lógica de Django.
- Sirve para aprender cómo se organiza un sitio real.

Pero hay algo importante: el template externo no reemplaza a Django.

Django sigue siendo el encargado de:

- Definir las URLs.

- Ejecutar las views.
- Consultar la base de datos.
- Enviar datos al template.
- Proteger rutas.
- Manejar usuarios.
- Procesar formularios.

El template externo solo mejora la presentación visual.

3. Página sugerida: Start Bootstrap

Una página útil para encontrar templates gratuitos es:

CÓDIGO

```
https://startbootstrap.com/
```

Start Bootstrap ofrece plantillas basadas en Bootstrap que pueden usarse como punto de partida para distintos tipos de sitios.

Para este proyecto, nos interesa especialmente un template orientado a blogs.

4. Template sugerido: Clean Blog

Para este curso, usaremos como ejemplo el template:

CÓDIGO

```
Clean Blog
```

Podés verlo en:

CÓDIGO

```
https://startbootstrap.com/previews/clean-blog
```

Este template es una buena opción porque ya tiene una estética pensada para blogs.

Incluye páginas como:

- Página principal.
- Página “About”.
- Página de post.
- Página de contacto.

Nosotros no vamos a usar necesariamente todo lo que trae, sino que vamos a adaptar las partes que sirven para nuestro proyecto.

5. Descargar el template

Desde la página del template, descargá el paquete de archivos.

Al descomprimirlo, vas a encontrar una estructura similar a esta:

CÓDIGO

```
clean-blog/
│
├── index.html
├── about.html
├── post.html
├── contact.html
│
├── css/
│   └── styles.css
│
├── js/
│   └── scripts.js
│
└── assets/
    └── img/
```

La estructura puede cambiar según la versión del template, pero generalmente vas a encontrar HTML, CSS, JavaScript e imágenes.

6. Diferencia entre HTML común y template de Django

Un archivo HTML común tiene contenido fijo.

Por ejemplo:

CÓDIGO

```
<h1>Clean Blog</h1>
<p>Un blog creado con Bootstrap</p>
```

En Django, los templates pueden tener contenido dinámico.

Por ejemplo:

CÓDIGO

```
<h1>{{ post.titulo }}</h1>
<p>{{ post.contenido }}</p>
```

También pueden usar etiquetas propias de Django:

CÓDIGO

```
{% extends 'posts/base.html' %}
{% load static %}
{% block content %}
{% endblock %}
```

Por eso, cuando usamos un template externo, necesitamos adaptarlo para que funcione con Django.

7. Mover archivos estáticos

Los archivos CSS, JS e imágenes del template deben moverse a la carpeta `static` de Django.

Por ejemplo, si el template trae:

CÓDIGO

```
css/styles.css
js/scripts.js
assets/img/
```

Podés moverlos a:

CÓDIGO

```
posts/static/posts/
```

Quedando así:

CÓDIGO

```
posts/
└── static/
    └── posts/
        ├── css/
        │   └── styles.css
        ├── js/
        │   └── scripts.js
        └── assets/
            └── img/
```

Esto permite que Django pueda encontrar esos archivos usando `{% static %}`.

8. Convertir el HTML principal en `base.html`

El archivo `index.html` del template suele tener toda la estructura general del sitio:

- `<html>`.
- `<head>`.
- Navbar.

- Header.
- Footer.
- Scripts.
- Enlaces a CSS.

Esa estructura general se puede convertir en el archivo:

CÓDIGO

```
posts/templates/posts/base.html
```

La idea es que `base.html` tenga las partes comunes del sitio y deje un espacio para el contenido específico de cada página.

Ese espacio se define con:

CÓDIGO

```
{% block content %}
{% endblock %}
```

9. Cargar archivos estáticos en Django

En la parte superior de `base.html`, agregá:

CÓDIGO

```
{% load static %}
```

Luego, reemplazá las rutas fijas del template.

Por ejemplo, si el HTML original tiene:

CÓDIGO

```
<link href="css/styles.css" rel="stylesheet" />
```

en Django debería quedar:

CÓDIGO

```
<link href="{% static 'posts/css/styles.css' %}" rel="stylesheet" />
```

Si el HTML original tiene:

CÓDIGO

```
<script src="js/scripts.js"></script>
```

en Django debería quedar:

CÓDIGO

```
<script src="{% static 'posts/js/scripts.js' %}"></script>
```

Si una imagen está en `static/posts/assets/img/`, se puede usar así:

CÓDIGO

```
<img src="{% static 'posts/assets/img/home-bg.jpg' %}" alt="Imagen de fondo" />
```

10. Adaptar el contenido con bloques

Una vez que tenés `base.html`, las demás páginas pueden extenderlo.

Por ejemplo, `lista_posts.html` puede quedar así:

CÓDIGO

```
{% extends 'posts/base.html' %}

{% block content %}
    <h1>Últimas publicaciones</h1>

    {% for post in posts %}
        <article>
            <h2>{{ post.titulo }}</h2>
            <p>{{ post.contenido|truncatewords:30 }}</p>
            <a href="{% url 'detalle_post' post.slug %}">Leer más</a>
        </article>
    {% empty %}
        <p>Todavía no hay publicaciones cargadas.</p>
    {% endfor %}
{% endblock %}
```

De esta forma, el diseño general vive en `base.html`, pero el contenido dinámico sigue viniendo desde Django.

11. Adaptar la navegación

El template externo seguramente trae una navegación fija.

Por ejemplo:

CÓDIGO

```
<a href="index.html">Home</a>
<a href="about.html">About</a>
<a href="contact.html">Contact</a>
```

En Django, no conviene dejar enlaces fijos a archivos `.html`.

En su lugar, usamos el sistema de URLs de Django:

CÓDIGO

```
<a href="{% url 'lista_posts' %}">Inicio</a>
<a href="{% url 'acerca' %}">Acerca de</a>
<a href="{% url 'crear_post' %}">Crear post</a>
```

Si el menú depende del estado del usuario, podemos usar:

CÓDIGO

```
{% if user.is_authenticated %}
    <a href="{% url 'perfil' %}">Mi perfil</a>
    <form action="{% url 'logout' %}" method="post" style="display: inline;">
        {% csrf_token %}
        <button type="submit">Cerrar sesión</button>
    </form>
{% else %}
    <a href="{% url 'login' %}">Iniciar sesión</a>
    <a href="{% url 'registro' %}">Registrarse</a>
{% endif %}
```

Esto permite que la navegación se adapte al usuario.

12. Adaptar el listado de posts al diseño

El template Clean Blog suele mostrar entradas como bloques visuales con título, subtítulo y fecha.

Podés adaptar cada post del blog a esa estructura.

Por ejemplo:

CÓDIGO

```
{% for post in posts %}
    <div class="post-preview">
        <a href="{% url 'detalle_post' post.slug %}">
            <h2 class="post-title">{{ post.titulo }}</h2>
            <h3 class="post-subtitle">{{ post.contenido|truncatewords:20 }}</h3>
        </a>
        <p class="post-meta">
            Publicado por {{ post.autor.username }} - {{ post.fecha_creacion }}
        </p>
    </div>
    <hr class="my-4" />
{% empty %}
    <p>Todavía no hay publicaciones cargadas.</p>
{% endfor %}
```

La clave es conservar las clases CSS del template y reemplazar el contenido fijo por variables de Django.

13. Adaptar la página de detalle

El archivo `post.html` del template puede servir como base para `detalle_post.html`.

En lugar de dejar texto fijo, usá los datos del post.

Por ejemplo:

CÓDIGO

```
{% extends 'posts/base.html' %}

{% block content %}
    <article>
        <h1>{{ post.titulo }}</h1>

        {% if post.imagen %}
            <img src="{{ post.imagen.url }}" alt="{{ post.titulo }}" />
        {% endif %}

        <p>{{ post.contenido }}</p>

        <p>
            Autor: {{ post.autor.username }}
        </p>
    </article>
{% endblock %}
```

Si el post tiene imagen, se muestra. Si no tiene imagen, el template no se rompe.

14. Adaptar formularios al diseño

Los formularios de creación, edición, login, registro y perfil también pueden usar clases del template o de Bootstrap.

Por ejemplo:

CÓDIGO

```
<form method="post" enctype="multipart/form-data">
    {% csrf_token %}

    {{ form.as_p }}

    <button type="submit" class="btn btn-primary">
        Guardar
    </button>
</form>
```

Más adelante, podés mejorar la presentación campo por campo, pero para este curso está bien usar `form.as_p` si el foco está en integrar el diseño sin complicar demasiado.

15. Cuidar los archivos que no usamos

El template descargado puede traer archivos que no necesitás.

Por ejemplo:

- Páginas HTML extra.
- Imágenes de ejemplo.
- Formularios de contacto que no se van a usar.
- Scripts que no cumplen ninguna función en el proyecto.

No es obligatorio usar todo.

Podés conservar solo lo necesario:

- CSS principal.
- JS principal.
- Algunas imágenes.
- Estructura base.
- Clases útiles.

Esto ayuda a que el proyecto no quede lleno de archivos innecesarios.

16. No reemplazar la lógica de Django

Este punto es clave.

Al integrar un template externo, no deberías romper lo que ya funcionaba.

Después de adaptar el diseño, el sitio debe seguir permitiendo:

- Ver listado de posts.
- Ver detalle.
- Crear posts.
- Editar posts.
- Eliminar posts.
- Subir imágenes.
- Registrarse.
- Iniciar sesión.
- Cerrar sesión.
- Ver perfil.
- Editar perfil.
- Proteger rutas.

El template externo mejora la apariencia, pero no debe eliminar la lógica construida en módulos anteriores.

17. Checklist de integración visual

Antes de avanzar, revisá:

- El template externo fue descargado correctamente.
- Los archivos CSS están en `static/`.
- Los archivos JS están en `static/`.
- Las imágenes necesarias están en `static/`.
- `base.html` tiene `{% load static %}`.
- Las rutas de CSS y JS usan `{% static %}`.
- Las páginas hijas usan `{% extends %}`.
- El contenido fijo fue reemplazado por datos dinámicos.
- Los enlaces usan `{% url %}`.
- El listado de posts se ve correctamente.
- El detalle de post se ve correctamente.
- Los formularios siguen funcionando.
- Las imágenes de posts y perfil siguen funcionando.
- El sitio no perdió funcionalidades anteriores.

18. Cierre de la unidad

En esta unidad aprendiste a tomar un template externo y adaptarlo a Django.

Viste cómo:

- Identificar archivos HTML, CSS, JS e imágenes.
- Mover archivos estáticos a la estructura de Django.
- Convertir un HTML en `base.html`.
- Reemplazar rutas fijas por `{% static %}`.
- Reemplazar enlaces fijos por `{% url %}`.
- Adaptar páginas como inicio, detalle y formularios.
- Mantener la lógica dinámica del proyecto.

Esta integración visual ayuda a que el blog se vea más profesional sin perder lo más importante: que siga funcionando como una aplicación Django.

Mejoras de experiencia de usuario

Mejorar la experiencia de usuario del blog agregando mensajes de confirmación, manejo visible de errores, estados vacíos, navegación condicional y validaciones visuales para que el sitio sea más claro, completo y fácil de usar.

Mejoras de experiencia de usuario

Hasta ahora, tu blog ya tiene muchas funcionalidades importantes: publicaciones, imágenes, usuarios, perfiles, rutas protegidas y una estructura visual más completa.

Pero que una aplicación “funcione” no siempre significa que sea cómoda de usar.

En esta unidad vas a mejorar la experiencia del usuario agregando mensajes, estados vacíos, validaciones visibles y pequeños detalles que hacen que el sitio sea más claro y fácil de entender.

1. ¿Qué es la experiencia de usuario?

La experiencia de usuario, también llamada UX, tiene que ver con cómo se siente una persona al usar una aplicación.

Un sitio puede estar técnicamente correcto, pero aun así generar confusión.

Por ejemplo:

- El usuario crea un post, pero no aparece ningún mensaje.
- El usuario borra una publicación, pero no sabe si se eliminó correctamente.
- Un formulario tiene errores, pero no se muestran de forma clara.
- Una página no tiene posts, pero queda vacía.
- Un avatar no existe y el template rompe.
- Un botón importante no se entiende.
- El usuario no sabe dónde hacer clic para volver.

En esta unidad vamos a corregir ese tipo de detalles.

2. ¿Por qué son importantes los mensajes?

Cuando una persona realiza una acción dentro del sitio, necesita recibir una respuesta.

Por ejemplo:

- “El post se creó correctamente”.
- “El perfil fue actualizado”.
- “La publicación fue eliminada”.
- “Debés iniciar sesión para continuar”.
- “No se encontraron resultados”.

Estos mensajes ayudan a que el usuario entienda qué pasó.

Django incluye un sistema de mensajes que permite mostrar este tipo de avisos de manera sencilla.

3. El sistema de mensajes de Django

Django trae una herramienta llamada `messages`.

Sirve para guardar mensajes temporales que se muestran una vez en el template.

Primero, en la view se crea el mensaje.

Luego, en el template se muestra.

Por ejemplo, después de crear un post correctamente, podemos mostrar un mensaje de éxito.

4. Importar messages

En el archivo donde tengas tus views, por ejemplo:

CÓDIGO

```
posts/views.py
```

podés importar `messages`:

CÓDIGO

```
from django.contrib import messages
```

Luego podés usarlo dentro de una view.

5. Mensaje al crear un post

Por ejemplo, en una vista basada en funciones para crear un post:

CÓDIGO

```
from django.shortcuts import render, redirect
from django.contrib import messages
from .forms import PostForm

def crear_post(request):
    if request.method == 'POST':
        form = PostForm(request.POST, request.FILES)

        if form.is_valid():
            form.save()
            messages.success(request, 'El post fue creado correctamente.')
            return redirect('lista_posts')
    else:
        form = PostForm()

    return render(request, 'posts/post_form.html', {'form': form, 'titulo_pagina':
'Crear post', 'texto_boton': 'Crear'})
```

En este caso, si el formulario es válido y el post se guarda, se crea un mensaje de éxito.

6. Tipos comunes de mensajes

Django permite usar distintos tipos de mensajes.

Los más comunes son:

CÓDIGO

```
messages.success(request, 'Operación realizada correctamente.')
messages.error(request, 'Ocurrió un error.')
messages.warning(request, 'Revisá la información ingresada.')
messages.info(request, 'Información importante.')
```

Cada uno representa una intención diferente.

Por ejemplo:

- `success` : algo salió bien.
- `error` : algo salió mal.
- `warning` : algo necesita atención.
- `info` : información general.

7. Mostrar mensajes en `base.html`

Para que los mensajes aparezcan en todo el sitio, conviene agregarlos en el template base.

Por ejemplo, en:

CÓDIGO

```
posts/templates/posts/base.html
```

podés agregar:

CÓDIGO

```
{% if messages %}
    <div class="mensajes">
        {% for message in messages %}
            <div class="mensaje {{ message.tags }}">
                {{ message }}
            </div>
        {% endfor %}
    </div>
{% endif %}
```

Como `base.html` es heredado por las demás páginas, los mensajes podrán mostrarse en todo el sitio.

8. Dar estilo a los mensajes

En tu archivo CSS podés agregar estilos simples.

Por ejemplo:

CÓDIGO

```
.mensajes {
    margin: 20px 0;
}

.mensaje {
    padding: 12px;
    border-radius: 6px;
    margin-bottom: 10px;
}

.mensaje.success {
    background-color: #d1e7dd;
    color: #0f5132;
}

.mensaje.error {
    background-color: #f8d7da;
    color: #842029;
}

.mensaje.warning {
    background-color: #fff3cd;
    color: #664d03;
}

.mensaje.info {
    background-color: #cff4fc;
    color: #055160;
}
```

Estos estilos pueden adaptarse al diseño del template que estés usando.

Si usás Bootstrap, también podés adaptar los mensajes a clases como `alert alert-success`, `alert alert-danger` o similares.

9. Mensajes en edición y eliminación

También podés usar mensajes en otras acciones.

Por ejemplo, al editar un post:

CÓDIGO

```
messages.success(request, 'El post fue actualizado correctamente.')
```

Al eliminar un post:

CÓDIGO

```
messages.success(request, 'El post fue eliminado correctamente.')
```

Al actualizar perfil:

CÓDIGO

```
messages.success(request, 'Tu perfil fue actualizado correctamente.')
```

Estos detalles hacen que el usuario no quede “adivinando” si la acción funcionó.

10. Mostrar errores de formularios

Cuando un formulario tiene errores, Django puede mostrarlos automáticamente.

Si usás:

CÓDIGO

```
{{ form.as_p }}
```

Django suele mostrar los errores cerca de cada campo.

Pero también podés mostrar errores generales del formulario.

Por ejemplo:

CÓDIGO

```
{% if form.errors %}
    <div class="mensaje error">
        Revisá los datos ingresados. Hay errores en el formulario.
    </div>
{% endif %}
```

Esto le avisa al usuario que algo necesita corrección.

11. Mostrar errores campo por campo

Si querés tener más control sobre el diseño, podés recorrer los campos del formulario.

Por ejemplo:

CÓDIGO

```
<form method="post" enctype="multipart/form-data">
    {% csrf_token %}

    {% for field in form %}
        <div class="campo-formulario">
            <label>{{ field.label }}</label>
            {{ field }}

            {% if field.errors %}
                <div class="error-campo">
                    {{ field.errors }}
                </div>
            {% endif %}
        </div>
    {% endfor %}

    <button type="submit">Guardar</button>
</form>
```

Esto permite mostrar los errores justo debajo del campo correspondiente.

12. Estados vacíos

Un estado vacío aparece cuando una sección no tiene datos para mostrar.

Por ejemplo:

- No hay posts cargados.
- No hay resultados de búsqueda.
- El usuario no tiene avatar de perfil.
- El post no tiene imagen destacada.
- El perfil todavía no tiene biografia.

En lugar de dejar un espacio vacío, conviene mostrar un mensaje claro.

13. Estado vacío en listado de posts

En Django templates, el bucle `for` puede tener una sección `{% empty %}`.

Por ejemplo:

CÓDIGO

```
{% for post in posts %}
    <article>
        <h2>{{ post.titulo }}</h2>
        <p>{{ post.contenido|truncatewords:30 }}</p>
    </article>
{% empty %}
    <p>Todavía no hay publicaciones cargadas.</p>
{% endfor %}
```

Esto permite mostrar un mensaje si la lista está vacía.

14. Estado vacío en búsqueda

Si el usuario busca un post y no hay resultados, también conviene mostrar un mensaje.

Por ejemplo:

CÓDIGO

```
{% if busqueda %}
    <p>Resultados para: "{{ busqueda }}"</p>
{% endif %}

{% for post in posts %}
    <article>
        <h2>{{ post.titulo }}</h2>
    </article>
{% empty %}
    <p>No se encontraron publicaciones para esa búsqueda.</p>
{% endfor %}
```

Esto evita que el usuario piense que la página falló.

15. Imagen de post opcional

Si un post puede tener imagen, pero la imagen no es obligatoria, siempre hay que verificar que exista antes de mostrarla.

Por ejemplo:

CÓDIGO

```
{% if post.imagen %}
    <img src="{{ post.imagen.url }}" alt="{{ post.titulo }}" />
{% endif %}
```

Si intentás usar `post.imagen.url` cuando no hay imagen, podés generar un error.

16. Avatar de perfil opcional

Lo mismo ocurre con el avatar de perfil.

Por ejemplo:

CÓDIGO

```
{% if user.perfil.avatar %}
    <img src="{{ user.perfil.avatar.url }}" alt="Avatar de perfil" />
{% else %}
    <p>Este usuario todavía no cargó un avatar de perfil.</p>
{% endif %}
```

Esto mejora la experiencia y evita errores visuales.

17. Biografía vacía

Si el perfil tiene biografia opcional, podés mostrar un texto alternativo.

Por ejemplo:

CÓDIGO

```
{% if user.perfil.biografia %}
    <p>{{ user.perfil.biografia }}</p>
{% else %}
    <p>Este usuario todavía no agregó una biografia.</p>
{% endif %}
```

El objetivo es que ninguna página quede incompleta o confusa.

18. Botones y enlaces claros

La experiencia de usuario también mejora cuando las acciones están bien nombradas.

No es lo mismo mostrar un enlace que diga:

CÓDIGO

```
Click acá
```

que mostrar:

CÓDIGO

```
Crear nuevo post
```

o:

CÓDIGO

```
Editar perfil
```

Los botones deberían indicar claramente qué acción realizan.

Ejemplos:

CÓDIGO

```
<a href="{% url 'crear_post' %}">Crear nuevo post</a>
<a href="{% url 'perfil' %}">Ver mi perfil</a>
<a href="{% url 'editar_perfil' %}">Editar perfil</a>
```

19. Confirmaciones antes de acciones importantes

Algunas acciones no deberían ejecutarse directamente.

Por ejemplo, eliminar un post.

Antes de borrar, conviene mostrar una página de confirmación.

Ejemplo:

CÓDIGO

```
<h1>Eliminar post</h1>

<p>¿Estás seguro de que querés eliminar "{{ post.titulo }}"?</p>

<form method="post">
    {% csrf_token %}
    <button type="submit">Sí, eliminar</button>
    <a href="{% url 'detalle_post' post.slug %}">Cancelar</a>
</form>
```

Esto evita borrados accidentales.

20. Navegación clara

El usuario siempre debería poder moverse por el sitio sin perderse.

En el menú principal conviene incluir enlaces importantes como:

- Inicio.
- Crear post.
- Perfil.
- Login.
- Logout.
- Registro.

Además, dentro de las páginas conviene agregar enlaces de retorno.

Por ejemplo:

CÓDIGO

```
<a href="{% url 'lista_posts' %}">Volver al inicio</a>
```

o:

CÓDIGO

```
<a href="{% url 'detalle_post' post.slug %}">Volver al detalle</a>
```

21. Navegación según usuario autenticado

Si una persona no inició sesión, no tiene sentido mostrarle acciones como “Crear post” o “Mi perfil”.

Podemos usar:

CÓDIGO

```
{% if user.is_authenticated %}
    <a href="{% url 'crear_post' %}">Crear post</a>
    <a href="{% url 'perfil' %}">Mi perfil</a>
    <form action="{% url 'logout' %}" method="post" style="display: inline;">
        {% csrf_token %}
        <button type="submit">Cerrar sesión</button>
    </form>
{% else %}
    <a href="{% url 'login' %}">Iniciar sesión</a>
    <a href="{% url 'registro' %}">Registrarse</a>
{% endif %}
```

Esto hace que el menú sea más coherente.

22. Mostrar acciones solo cuando corresponden

Además de proteger rutas desde las views, conviene ocultar botones que el usuario no puede usar.

Por ejemplo:

CÓDIGO

```
{% if user.is_authenticated %}
    <a href="{% url 'editar_post' post.slug %}">Editar</a>
    <a href="{% url 'eliminar_post' post.slug %}">Eliminar</a>
{% endif %}
```

Esto no reemplaza la protección de rutas, pero mejora la experiencia visual.

La seguridad debe estar en las views. El template solo ayuda a mostrar una interfaz más ordenada.

23. Mejorar formularios sin complicarlos

Si el proyecto ya usa Bootstrap o un template externo, podés mejorar un poco los formularios.

Por ejemplo, usando clases en botones:

CÓDIGO

```
<button type="submit" class="btn btn-primary">Guardar</button>
<a href="{% url 'lista_posts' %}" class="btn btn-secondary">Cancelar</a>
```

No hace falta reescribir todo el formulario manualmente si eso complica demasiado.

Para este curso, está bien usar:

CÓDIGO

```
{{ form.as_p }}
```

y mejorar lo básico con CSS.

24. Checklist de experiencia de usuario

Antes de avanzar, revisá:

- Hay mensajes al crear un post.
- Hay mensajes al editar un post.
- Hay mensajes al eliminar un post.
- Hay mensajes al actualizar perfil.
- Los errores de formularios se muestran.
- Si no hay posts, aparece un mensaje.
- Si no hay resultados de búsqueda, aparece un mensaje.
- Si un post no tiene imagen, el template no rompe.
- Si el perfil no tiene avatar, se muestra un texto alternativo.
- Si el perfil no tiene biografía, se muestra un texto alternativo.
- El usuario puede volver al inicio fácilmente.
- Los botones tienen textos claros.
- El menú cambia según si el usuario inició sesión.
- Las acciones sensibles no se muestran a usuarios anónimos.
- Las rutas sensibles siguen protegidas desde las views.

25. Cierre de la unidad

En esta unidad aprendiste a mejorar la experiencia de usuario del blog.

Viste cómo:

- Mostrar mensajes con `messages`.
- Informar acciones exitosas.
- Mostrar errores de formularios.
- Usar estados vacíos.
- Evitar errores con imágenes opcionales.
- Mejorar botones y enlaces.
- Mostrar navegación condicional.
- Confirmar acciones importantes antes de ejecutarlas.

Estos cambios parecen pequeños, pero hacen que el proyecto se sienta mucho más completo, claro y fácil de usar.

Búsqueda, filtros y revisión del flujo completo

Agregar una búsqueda simple de publicaciones usando request.GET y filtros con el ORM, revisando además el flujo completo del blog para comprobar que listado, detalle, usuarios, perfiles, imágenes, formularios y rutas protegidas funcionen correctamente.

Hasta ahora, tu blog ya tiene una base bastante completa: publicaciones, imágenes, usuarios, perfiles, rutas protegidas, mensajes y una mejor experiencia visual.

En esta unidad vas a sumar una mejora funcional muy común en cualquier sitio de contenido: la búsqueda de publicaciones.

Además, vas a revisar el flujo completo del proyecto para comprobar que todas las partes trabajadas durante el curso funcionen conectadas entre sí.

1. ¿Por qué agregar búsqueda?

Cuando un blog empieza a tener varias publicaciones, mostrar todos los posts en una sola página puede volverse poco práctico.

Una búsqueda permite que el usuario encuentre publicaciones específicas de forma más rápida.

Por ejemplo, podría buscar:

- Posts sobre Django.
- Posts sobre Python.
- Una palabra incluida en el título.
- Una palabra incluida en el contenido.

En esta unidad vamos a implementar una búsqueda simple, sin instalar librerías externas y sin usar herramientas avanzadas.

La idea es practicar cómo recibir datos desde la URL y usarlos para filtrar resultados con el ORM de Django.

2. ¿Qué vamos a construir?

Vamos a agregar un formulario de búsqueda en el blog.

El usuario escribirá una palabra o frase y Django mostrará los posts que coincidan con esa búsqueda.

El flujo será:

CÓDIGO

```
Usuario escribe una búsqueda
↓
El navegador envía el dato por la URL
↓
La view recibe ese dato con request.GET
↓
Django filtra los posts con el ORM
↓
El template muestra los resultados
```

3. ¿Qué es `request.GET`?

En Django, `request.GET` permite leer información enviada por la URL.

Por ejemplo, si el usuario busca “django”, la URL podría quedar así:

CÓDIGO

```
http://127.0.0.1:8000/?q=django
```

En esa URL, el dato importante es:

CÓDIGO

```
q=django
```

La letra `q` es el nombre del parámetro y `django` es el valor buscado.

Desde la view podemos leer ese valor usando:

CÓDIGO

```
busqueda = request.GET.get('q')
```

Si el usuario no buscó nada, `busqueda` puede quedar vacío o como `None`.

4. Agregar un formulario de búsqueda

Podés agregar un pequeño formulario en el template donde mostrás el listado de posts.

Por ejemplo, en:

CÓDIGO

```
posts/templates/posts/lista_posts.html
```

podés incluir:

CÓDIGO

```
<form method="get">
    <input type="text" name="q" placeholder="Buscar posts..."
value="{{ busqueda }}" />
    <button type="submit">Buscar</button>
</form>
```

Este formulario usa:

CÓDIGO

```
method="get"
```

Eso significa que los datos se enviarán en la URL.

El campo:

CÓDIGO

```
name="q"
```

indica que la búsqueda se enviará con el nombre `q`.

5. Actualizar la view de listado

Ahora necesitamos modificar la view que muestra los posts.

La idea es:

1. Traer todos los posts.
2. Revisar si llegó una búsqueda.
3. Si hay búsqueda, filtrar los posts.
4. Enviar los resultados al template.

Ejemplo con vista basada en funciones:

CÓDIGO

```
from django.shortcuts import render
from .models import Post


def lista_posts(request):
    busqueda = request.GET.get('q')

    posts = Post.objects.all()

    if busqueda:
        posts = posts.filter(titulo__icontains=busqueda)

    return render(request, 'posts/lista_posts.html', {
        'posts': posts,
        'busqueda': busqueda,
    })
```

En este ejemplo usamos:

CÓDIGO

```
titulo__icontains=busqueda
```

Esto significa:

- Buscar dentro del campo `titulo`.
- No distinguir entre mayúsculas y minúsculas.
- Encontrar coincidencias parciales.

Por ejemplo, si buscás “djan”, puede encontrar “Django”.

6. Buscar en más de un campo

También podés buscar en el título y en el contenido.

Para eso podemos usar `Q`, una herramienta de Django que permite combinar condiciones.

Primero importamos:

CÓDIGO

```
from django.db.models import Q
```

Luego usamos:

CÓDIGO

```
posts = posts.filter(
    Q(titulo__icontains=busqueda) |
    Q(contenido__icontains=busqueda)
)
```

La barra vertical `|` significa “o”.

Entonces Django buscará posts donde:

- El título contenga la palabra buscada.
- O el contenido contenga la palabra buscada.

La view completa podría quedar así:

CÓDIGO

```
from django.shortcuts import render
from django.db.models import Q
from .models import Post

def lista_posts(request):
    busqueda = request.GET.get('q')

    posts = Post.objects.all()

    if busqueda:
        posts = posts.filter(
            Q(titulo__icontains=busqueda) |
            Q(contenido__icontains=busqueda)
        )

    return render(request, 'posts/lista_posts.html', {
        'posts': posts,
        'busqueda': busqueda,
    })
```

7. Mostrar la búsqueda realizada

Es útil mostrarle al usuario qué buscó.

En el template, podés agregar:

CÓDIGO

```
{% if busqueda %}
    <p>Resultados para: "{{ busqueda }}"</p>
{% endif %}
```

Esto mejora la claridad de la página.

8. Mostrar mensaje si no hay resultados

Si el usuario busca algo y no existe ningún post que coincida, la página no debería quedar vacía.

Podés usar `{% empty %}` dentro del bucle:

CÓDIGO

```
{% for post in posts %}
    <article>
        <h2>{{ post.titulo }}</h2>
        <p>{{ post.contenido|truncatewords:30 }}</p>
        <a href="{% url 'detalle_post' post.slug %}">Leer más</a>
    </article>
{% empty %}
    {% if busqueda %}
        <p>No se encontraron publicaciones para esa búsqueda.</p>
    {% else %}
        <p>Todavía no hay publicaciones cargadas.</p>
    {% endif %}
{% endfor %}
```

Así el mensaje cambia según el caso.

9. Botón para limpiar búsqueda

Después de buscar, puede ser útil ofrecer un enlace para volver al listado completo.

Por ejemplo:

CÓDIGO

```
{% if busqueda %}
    <a href="{% url 'lista_posts' %}">Limpiar búsqueda</a>
{% endif %}
```

Esto elimina el parámetro `q` de la URL y vuelve a mostrar todos los posts.

10. Filtro simple por estado

Si tu modelo `Post` tiene un campo `estado`, podés usarlo para mostrar solo publicaciones publicadas.

Por ejemplo:

CÓDIGO

```
posts = Post.objects.filter(estado='publicado')
```

Esto puede ser útil para que en la página principal no aparezcan posts en borrador.

Si en tu proyecto los estados están definidos con otros valores, usá el valor correspondiente.

Por ejemplo:

CÓDIGO

```
posts = Post.objects.filter(estado='PUBLICADO')
```

o:

CÓDIGO

```
posts = Post.objects.filter(estado='pub')
```

según cómo hayas definido las opciones del modelo.

11. Combinar estado y búsqueda

También podés combinar el filtro de estado con la búsqueda.

Por ejemplo:

CÓDIGO

```
posts = Post.objects.filter(estado='publicado')

if busqueda:
    posts = posts.filter(
        Q(titulo__icontains=busqueda) |
        Q(contenido__icontains=busqueda)
    )
```

En este caso, primero se traen solo los posts publicados y luego se busca dentro de esos resultados.

12. ¿Conviene mostrar borradores?

Depende de cómo quieras que funcione el blog.

Una opción simple es:

- Visitantes anónimos ven solo posts publicados.
- El panel admin puede ver todos.
- Más adelante, el autor podría ver sus propios borradores.

Para este curso, no hace falta hacer permisos avanzados.

Pero sí podés dejar la página principal mostrando solo publicaciones con estado “publicado”, si querés que el blog se vea más realista.

13. Ordenar publicaciones

También es común mostrar primero los posts más recientes.

Si tu modelo tiene `fecha_creacion`, podés ordenar así:

CÓDIGO

```
posts = Post.objects.all().order_by('-fecha_creacion')
```

El signo `-` indica orden descendente.

Es decir: del más nuevo al más viejo.

También podés combinarlo con filtros:

CÓDIGO

```
posts = Post.objects.filter(estado='publicado').order_by('-fecha_creacion')
```

14. View completa con búsqueda, filtro y orden

Una versión más completa de la view podría ser:

CÓDIGO

```
from django.shortcuts import render
from django.db.models import Q
from .models import Post

def lista_posts(request):
    busqueda = request.GET.get('q')

    posts = Post.objects.filter(estado='publicado').order_by('-fecha_creacion')

    if busqueda:
        posts = posts.filter(
            Q(titulo__icontains=busqueda) |
            Q(contenido__icontains=busqueda)
        )

    return render(request, 'posts/lista_posts.html', {
        'posts': posts,
        'busqueda': busqueda,
    })
```

Esta versión:

- Muestra solo posts publicados.
- Ordena del más nuevo al más viejo.
- Permite buscar por título o contenido.

15. Adaptar la búsqueda al diseño

Si estás usando un template externo como Clean Blog o Bootstrap, podés mejorar el formulario visualmente.

Por ejemplo:

CÓDIGO

```
<form method="get" class="mb-4">
    <div class="input-group">
        <input
            type="text"
            name="q"
            class="form-control"
            placeholder="Buscar publicaciones..."
            value="{{ busqueda }}" />
        <button class="btn btn-primary" type="submit">
            Buscar
        </button>
    </div>
</form>
```

Si no estás usando Bootstrap, podés usar clases propias y darles estilo en CSS.

Revisión del flujo completo

Después de agregar búsqueda y filtros, es importante revisar que todo el proyecto siga funcionando.

A veces, cuando modificamos templates o views, podemos romper algo que antes andaba bien.

Por eso, conviene hacer una prueba completa del sitio.

16. Flujo del visitante anónimo

Primero probá el sitio sin iniciar sesión.

Un visitante anónimo debería poder:

- Entrar a la página de inicio.
- Ver el listado de posts publicados.
- Usar la búsqueda.
- Ver resultados.
- Ver mensaje si no hay resultados.
- Entrar al detalle de un post.
- Ver imágenes de posts.
- Ver enlaces de login y registro.

Un visitante anónimo no debería poder:

- Crear posts.
- Editar posts.
- Borrar posts.
- Editar perfil.
- Acceder a páginas protegidas.

Si intenta entrar escribiendo una URL protegida, debería ser redirigido al login.

17. Flujo del usuario registrado

Después probá con un usuario registrado.

Un usuario registrado debería poder:

- Iniciar sesión.
- Ver su perfil.
- Editar su perfil.
- Subir o cambiar su avatar de perfil.
- Crear un post.
- Subir una imagen al post.
- Editar un post.
- Borrar un post.
- Cerrar sesión.

También debería poder:

- Ver el listado.
- Usar la búsqueda.
- Entrar al detalle de posts.

18. Flujo de publicaciones

Revisá que las publicaciones funcionen de principio a fin.

Probá:

- Crear un post publicado.
- Crear un post en borrador, si tu modelo tiene estados.
- Ver si aparece o no en la página principal según el filtro elegido.
- Editar título.
- Editar contenido.
- Cambiar imagen.
- Borrar publicación.
- Confirmar que desaparece del listado.

19. Flujo de imágenes

Como el proyecto maneja imágenes de posts y de perfil, también hay que probar esos casos.

Probá:

- Crear post con imagen.
- Crear post sin imagen.

- Ver detalle de post con imagen.
- Ver detalle de post sin imagen.
- Editar post y cambiar imagen.
- Editar perfil y subir avatar.
- Ver perfil sin avatar.
- Ver perfil con avatar.

El sitio no debería romperse si falta una imagen.

20. Flujo de formularios

Probá los formularios con datos correctos e incorrectos.

Por ejemplo:

- Enviar formulario vacío.
- Enviar formulario incompleto.
- Enviar formulario correcto.
- Ver si aparecen errores.
- Ver si se muestran mensajes de éxito.
- Ver si redirige a la página esperada.

Esto aplica a:

- Crear post.
- Editar post.
- Registro.
- Login.
- Editar perfil.

21. Flujo de navegación

Revisá que el menú y los enlaces funcionen bien.

Probá:

- Ir al inicio.
- Ir al detalle de un post.
- Volver al inicio.
- Ir a crear post.
- Ir a perfil.
- Ir a login.
- Ir a registro.
- Cerrar sesión.
- Limpiar búsqueda.

También revisá que no haya enlaces rotos o URLs escritas manualmente de forma incorrecta.

22. Prueba con datos reales

Para revisar mejor el proyecto, conviene tener algunos datos de prueba.

Por ejemplo:

- 3 posts publicados.
- 1 post borrador, si usás estados.
- 1 usuario con perfil completo.
- 1 usuario sin avatar de perfil.
- 1 post con imagen.
- 1 post sin imagen.

Esto permite comprobar distintos escenarios.

23. Checklist de búsqueda y filtros

Antes de cerrar esta parte, revisá:

- El formulario de búsqueda existe.
- El formulario usa `method="get"`.
- El input tiene `name="q"`.
- La view lee `request.GET.get('q')`.
- La búsqueda filtra por título o contenido.
- Se muestra el texto buscado.
- Si no hay resultados, aparece un mensaje.
- Existe forma de limpiar la búsqueda.
- Los posts se ordenan de forma coherente.
- Si aplicás filtro por estado, funciona correctamente.

24. Checklist del flujo completo

Antes de pasar a la entrega final, revisá:

- El sitio carga correctamente.
- El listado de posts funciona.
- El detalle de posts funciona.
- La búsqueda funciona.
- El mensaje sin resultados funciona.
- Las imágenes de posts se muestran.
- El registro funciona.
- El login funciona.
- El logout funciona.

- El perfil funciona.
- La edición de perfil funciona.
- Los avatares de perfil se muestran.
- Crear post funciona.
- Editar post funciona.
- Borrar post funciona.
- Las rutas protegidas redirigen al login.
- Los mensajes de éxito se muestran.
- Los errores de formularios se muestran.
- La navegación es clara.
- No hay errores visibles en consola o terminal.

25. Cierre de la unidad

En esta unidad agregaste búsqueda y filtros simples al blog.

También revisaste el flujo completo del proyecto para asegurarte de que todas las partes trabajadas durante el curso funcionen bien juntas.

Viste cómo:

- Crear un formulario de búsqueda.
- Enviar datos por `GET`.
- Leer datos con `request.GET`.
- Filtrar resultados con el ORM.
- Usar `Q` para buscar en más de un campo.
- Mostrar mensajes cuando no hay resultados.
- Ordenar publicaciones.
- Revisar el flujo de usuario anónimo.
- Revisar el flujo de usuario registrado.
- Probar posts, imágenes, formularios y rutas protegidas.

Esta unidad ayuda a cerrar el proyecto como una aplicación más completa, integrada y fácil de probar.

MÓDULO 13

Entrega Final del Blog Web

Consolidar todos los componentes del sistema para entregar un blog web funcional y documentado mediante un repositorio de GitHub.

ENTREGA FINAL DEL PROYECTO

Actividad evaluable · proyecto integrador final

Consolidación del blog web en GitHub

Guía completa para integrar todas las partes del blog web, documentar el repositorio en GitHub y realizar la entrega definitiva del programa.

¡Felicitaciones! Llegaste al cierre del recorrido.

Durante el curso construiste un proyecto completo desde cero. Empezaste escribiendo tus primeras instrucciones en Python, trabajaste con datos en memoria, organizaste código en funciones y módulos, incorporaste Programación Orientada a Objetos, guardaste información en JSON y luego diste el salto a Django para construir una aplicación web real.

Esta entrega final reúne todo ese recorrido en un único proyecto: un blog web desarrollado con Django, organizado en un repositorio de GitHub y preparado para ser revisado por otra persona.

El objetivo de esta instancia no es agregar una funcionalidad nueva, sino consolidar, revisar, documentar y presentar correctamente el trabajo realizado.Objetivo

Presentar el blog web hecho con Django en un repositorio público de GitHub que otra persona pueda clonar, instalar y ejecutar.

Qué entregar

La **URL directa de un repositorio público de GitHub** con el código completo y funcional del blog. El repo debe contener: código del proyecto Django, apps, templates, archivos estáticos, migraciones, `requirements.txt`, `README.md`, `.gitignore` y `.env.example` (solo si usaste variables de entorno).

Criterios de aceptación

Publicaciones (desde la interfaz web, no solo el admin)

- Listar posts, ver detalle, crear, editar y eliminar.
- Cargar y mostrar imagen del post; el template no se rompe si falta la imagen.

Usuarios

- Registro, login y logout funcionando.
- Perfil accesible (editar perfil e imagen de perfil solo si los implementaste).
- El sistema distingue usuario anónimo de autenticado.

Rutas protegidas

- Crear/editar/eliminar posts y editar perfil requieren sesión iniciada.

- La protección está en la view con `@login_required` o `LoginRequiredMixin` (ocultar el botón en el template no alcanza).

Media

- `MEDIA_URL` y `MEDIA_ROOT` configurados; las imágenes se ven en desarrollo.

Búsqueda (solo si la implementaste)

- Busca por título o contenido, muestra resultados y avisa cuando no hay coincidencias.

Repositorio

- Es público y abre correctamente en el navegador.
- Existen `requirements.txt` (con Django, Pillow y python-decouple si corresponde, actualizado), `README.md` y `.gitignore`.
- El historial de commits muestra progreso (no todo en un único commit final).
- No hay rutas absolutas; la config se apoya en `BASE_DIR`.

README — debe incluir como mínimo:

- Título, descripción breve, tecnologías, funcionalidades principales.
- Instrucciones de instalación, migraciones y arranque del servidor.
- Información del autor.
- Sin contraseñas ni claves reales.

Pasos

Antes de entregar, verificá localmente y contra GitHub que: el servidor inicia sin errores, las migraciones aplican, el CRUD funciona, las imágenes cargan (y el template resiste sin imagen), registro/login/logout funcionan, las rutas protegidas redirigen, y los cambios están **sincronizados con GitHub** (no basta con guardar en local).

Cómo entregar

Enviá el **enlace directo al repositorio del proyecto**. No entregues un `.zip`, ni una captura, ni el link a tu perfil general de GitHub.

Qué evitar

- Repositorio privado (los evaluadores no lo verían).
- Subir `venv/`, `.env`, `__pycache__/`, `.pyc`, archivos temporales o config local del editor → usá `.gitignore`.
- Subir claves o contraseñas reales (ni en el código ni en el README).
- Dejar rutas absolutas de tu computadora.
- Faltar `requirements.txt` o las migraciones.

- Probar todo con sesión iniciada y olvidar al visitante anónimo.
- README que solo tenga el nombre del proyecto.

(Si decidís no subir `db.sqlite3` o `media/`, el README debe explicar cómo reconstruir/probar el proyecto desde cero).

Pasos para la entrega

Presentar el blog web consolidado en un repositorio público de GitHub, listo para ser revisado, instalado y ejecutado por un tercero.

El proyecto debe integrar las funcionalidades trabajadas durante el curso: gestión de posts, imágenes, usuarios, autenticación, rutas protegidas y documentación básica del repositorio.

Instrucciones paso a paso

1. Preparación del código

Antes de entregar, revisá que el proyecto funcione correctamente.

Asegurate de que:

- `settings.py` tenga configurados correctamente `STATIC_URL`, `MEDIA_URL` y `MEDIA_ROOT`.
- Las migraciones finales estén creadas.
- El proyecto pueda preparar la base de datos ejecutando `python manage.py migrate`.
- El proyecto pueda ejecutarse con `python manage.py runserver`.
- El CRUD de posts funcione desde la interfaz web.
- Las imágenes de los posts se carguen y se visualicen correctamente.
- El registro, login y logout de usuarios funcionen correctamente.
- Las rutas protegidas no sean accesibles para usuarios no autenticados.
- No queden `print()` de prueba, comentarios innecesarios o código temporal.

2. Gestión de dependencias

Con el entorno virtual activo, generá o actualizá el archivo `requirements.txt` ejecutando:

CÓDIGO

```
pip freeze > requirements.txt
```

Verificá que el archivo se haya creado en la raíz del proyecto.

Este archivo es necesario para que otra persona pueda instalar las dependencias del proyecto.

3. Archivo README.md

Creá o actualizá un archivo `README.md` en la raíz del proyecto.

Debe incluir, como mínimo:

- Título del proyecto.
- Descripción breve del blog.
- Tecnologías utilizadas, por ejemplo Python, Django, Bootstrap, SQLite, Pillow.
- Funcionalidades principales.
- Instrucciones de instalación.
- Pasos para crear y activar el entorno virtual.
- Comando para instalar dependencias.
- Comando para ejecutar migraciones.
- Comando para iniciar el servidor.
- Instrucciones para crear un superusuario, si es necesario probar el panel administrativo.
- Autor del proyecto.

El README debe permitir que otra persona entienda qué hace el proyecto y cómo ejecutarlo localmente.

Si el proyecto no incluye `db.sqlite3` o la carpeta `media/`, el README debe permitir reconstruir y probar el proyecto desde cero.

4. Variables de entorno

Si el proyecto utiliza variables de entorno, no subas el archivo `.env` al repositorio.

En su lugar, incluí un archivo `.env.example` con las variables necesarias para ejecutar el proyecto, sin valores privados reales.

Por ejemplo, puede indicar que el proyecto necesita configurar valores como:

- `SECRET_KEY`
- `DEBUG`

No incluyas contraseñas personales, claves secretas reales ni credenciales privadas en el código o en el README.

5. Limpieza con .gitignore

Asegurate de tener un archivo `.gitignore` correctamente configurado.

Debe excluir archivos y carpetas que no deberían subirse al repositorio, por ejemplo:

- `venv/`
- `.env`
- `__pycache__/`
- `*.pyc`
- `db.sqlite3`, opcional pero recomendado si querés que el evaluador empiece con una base limpia.

- `media/`, opcional según el criterio de entrega.
- `.DS_Store`, si usás macOS.

También verificá que no se suban claves secretas, contraseñas personales, archivos temporales o configuraciones locales innecesarias.

6. Commit y push final

Realizá un último commit con un mensaje claro, por ejemplo:

CÓDIGO

```
git commit -m "Final: Proyecto consolidado y documentado"
```

Luego subí los cambios a GitHub:

CÓDIGO

```
git push origin main
```

Antes de enviar, ingresá al repositorio desde el navegador y verificá que todos los archivos necesarios estén subidos correctamente.

Formato de entrega

La entrega consiste en enviar la URL directa del repositorio público de GitHub.

No envíes una carpeta comprimida ni una captura de pantalla. El enlace debe llevar directamente al repositorio del proyecto.

Criterios de evaluación

Antes de entregar, revisá el siguiente checklist:

- El repositorio es público.
- El proyecto inicia correctamente con `python manage.py runserver`.
- El proyecto puede preparar la base de datos con `python manage.py migrate`.
- Las migraciones del proyecto están incluidas en el repositorio.
- Existe el archivo `requirements.txt`.
- Existe el archivo `README.md`.
- El README explica cómo instalar y ejecutar el proyecto.
- El README explica cómo crear un superusuario, si es necesario para probar el admin.
- Si se usan variables de entorno, existe un archivo `.env.example`.
- El archivo `.gitignore` evita subir archivos innecesarios.
- El CRUD de posts funciona desde la interfaz web.
- Los posts permiten cargar y mostrar imágenes.
- El registro de usuarios funciona correctamente.

- El login y logout funcionan correctamente.
- Las rutas protegidas no son accesibles para usuarios no autenticados.
- No se subieron carpetas como `venv/` o `__pycache__/`.
- No se subieron contraseñas, claves secretas o credenciales personales.
- Si no se incluye `db.sqlite3` o `media/`, el proyecto igualmente puede ejecutarse siguiendo las instrucciones del README.

Entregable

Formato: repositorio

URL de un repositorio de GitHub público que contenga el código completo del Blog Web, incluyendo el archivo README.md con instrucciones de instalación y uso.

Objetivos de aprendizaje

- Consolidar todas las funcionalidades (CRUD, Autenticación, Multimedia) en un proyecto Django funcional.
- Generar una documentación técnica profesional mediante un archivo README.md.
- Gestionar un repositorio de GitHub limpio siguiendo las mejores prácticas de la industria (.gitignore, commits claros).
- Validar la reproducibilidad del entorno de desarrollo mediante un archivo de requerimientos.

Rúbrica de evaluación

Esta rúbrica evalúa la entrega final del blog web en Django como un repositorio público de GitHub listo para ser revisado y ejecutado por un tercero. Se prioriza la consolidación del proyecto completo, la correcta documentación, la preparación del entorno para instalación y ejecución, y la calidad de la entrega final como producto profesional.

Criterio	Descripción	Peso
Repositorio público y entrega en GitHub	Evalúa que la entrega esté publicada en un repositorio público de GitHub, con acceso claro para revisión y con el proyecto consolidado en la rama principal indicada.	25%
Documentación del proyecto en README.md	Evalúa la calidad y utilidad del archivo README.md como guía de instalación, ejecución y comprensión general del blog web.	20%
Preparación técnica para instalación y ejecución	Evalúa que el proyecto esté preparado para ser instalado y ejecutado correctamente por otra persona, incluyendo dependencias, migraciones finales y configuración básica de estáticos y media.	25%
Funcionamiento del CRUD de posts con imágenes	Evalúa que el núcleo funcional del blog final siga operando correctamente, especialmente la creación, lectura, edición y eliminación de posts con soporte de imágenes.	10%
Autenticación, acceso de usuarios y protección de rutas	Evalúa que el login y el registro de usuarios funcionen como parte del blog consolidado y que las rutas sensibles (crear, editar y borrar posts, y editar perfil) no sean accesibles para usuarios no autenticados, manteniendo visibles el listado y el detalle para visitantes anónimos.	10%
Limpieza del repositorio y control de archivos no versionados	Evalúa que el repositorio incluya una configuración adecuada de .gitignore para evitar subir archivos innecesarios o sensibles y para mantener una estructura profesional.	10%

Total: 100 pts | Aprobación: 70 pts

Material complementario

Palabras clave

- **Entorno Virtual (venv):** Es un espacio aislado dentro de tu computadora donde puedes instalar herramientas y librerías específicas para un proyecto sin afectar a los demás. Imagina que es una caja de herramientas propia para cada trabajo, evitando que las versiones de los programas se mezclen o causen errores entre sí.
- **Variable:** Es un nombre que usamos para guardar un dato en la memoria de la computadora y poder usarlo más tarde en nuestro código. Por ejemplo, puedes crear una variable llamada

'titulo_post' para almacenar el texto 'Mi primer día aprendiendo Python' y mostrarla en pantalla cuando el usuario la necesite.

- **Diccionario:** Es una estructura de datos que organiza la información mediante pares de 'clave' y 'valor', similar a un diccionario real donde buscas una palabra para encontrar su significado. En programación, se usa para agrupar datos relacionados; por ejemplo, un diccionario de 'Autor' puede tener las claves 'nombre', 'email' y 'biografía'.
- **Función:** Es un bloque de código diseñado para realizar una tarea específica que puedes reutilizar cuantas veces quieras sin tener que escribir las mismas instrucciones de nuevo. Por ejemplo, puedes crear una función llamada 'saludar_usuario' que imprima un mensaje de bienvenida cada vez que alguien entra al blog.
- **Programación Orientada a Objetos (POO):** Es una forma de programar que intenta modelar elementos del mundo real como 'objetos' que tienen sus propias características (atributos) y acciones (métodos). En el curso, usaremos esto para definir qué es un 'Post' (con título y contenido) y qué puede hacer (como publicarse o editarse).
- **Git y GitHub:** Git es una herramienta que registra todos los cambios que haces en tu código para que puedas volver atrás si algo falla, mientras que GitHub es el sitio web donde guardas esos registros en la nube. Funcionan como una 'máquina del tiempo' y una 'copia de seguridad' que te permite compartir tu trabajo con otros.
- **Framework (Django):** Es un conjunto de herramientas y reglas ya creadas que ayudan a los programadores a construir aplicaciones web de forma más rápida y segura. En lugar de escribir todo desde cero, Django te ofrece piezas listas para usar, como el sistema de usuarios o la conexión con la base de datos.
- **Modelo (Model):** En Django, un modelo es la representación de una tabla en tu base de datos escrita en lenguaje Python. Define qué tipo de información vamos a guardar; por ejemplo, el modelo de un 'Post' indicará que necesitamos un espacio para el texto, otro para la fecha y otro para la imagen.
- **Vista (View):** Es la parte del código que recibe las peticiones de los usuarios (cuando hacen clic en un enlace) y decide qué información mostrarles. La vista actúa como un intermediario que busca los datos necesarios en la base de datos y los envía a la pantalla del navegador.
- **Plantilla (Template):** Es un archivo de texto (generalmente HTML) que define cómo se verá visualmente tu blog. La ventaja de Django es que permite usar 'herencia de plantillas', lo que significa que puedes crear un diseño base con el menú y el pie de página, y reutilizarlo en todas las secciones del sitio.
- **ORM (Object-Relational Mapping):** Es una herramienta de Django que permite interactuar con la base de datos usando código Python sencillo en lugar de lenguaje SQL complejo. Gracias al ORM, puedes buscar todos los posts de un autor escribiendo algo como 'Post.objects.filter(autor="Juan")'.
- **CRUD:** Es el acrónimo de Crear, Leer, Actualizar y Borrar (Create, Read, Update, Delete). Son las cuatro operaciones básicas que cualquier aplicación de datos debe permitir realizar, como cuando creas un nuevo post en el blog, lo lees, corriges un error o decides eliminarlo.

- **Migraciones:** Son como un historial de cambios para tu base de datos. Cada vez que modificas la estructura de tus modelos (por ejemplo, al añadir un campo para 'votos'), Django crea un archivo de migración que actualiza la base de datos de forma segura para que coincida con tu código.
- **JSON:** Es un formato ligero para guardar e intercambiar datos que es fácil de leer tanto para humanos como para máquinas. En el curso lo usaremos para guardar la información del blog en archivos de texto antes de pasar a usar bases de datos profesionales.
- **Archivos Estáticos y Media:** Los archivos estáticos son los que no cambian (como el logo o el estilo visual CSS), mientras que los archivos Media son los que suben los usuarios (como las fotos de sus posts o sus avatares). Django maneja ambos de forma distinta para que el sitio cargue rápido y sea organizado.

Análisis profundo

La Lógica del Control de Flujo: Por qué el 'Menú Interactivo' es el Cerebro de tu Aplicación

Cuando empiezas a programar, es fácil ver el código como una lista estéril de instrucciones. Sin embargo, el verdadero 'poder' surge cuando el programa decide qué hacer basándose en lo que el usuario quiere. Imagina que entras a un cajero automático y, en lugar de darte opciones, el cajero simplemente te entrega dinero, te muestra tu saldo y luego se apaga. Sería inútil. El control de flujo mediante el bucle `while` y las estructuras `if-elif-else` resuelve precisamente esto: transforma un script lineal en una herramienta interactiva.

En este programa, el menú de consola no es solo una práctica; es la base funcional de cualquier software. Un error común es pensar que el programa 'adivina' cuándo detenerse. Sin un bucle infinito controlado por una condición de salida (como `opcion != 'salir'`), tu blog de consola se cerraría tras crear un solo post. En la industria real, esta lógica es la misma que mantiene funcionando a servidores de aplicaciones gigantes como Spotify o sistemas de trading: un bucle 'event loop' que espera instrucciones, las procesa y vuelve a esperar.

Para visualizarlo, piensa en un restaurante. El `while` es el camarero que se queda junto a tu mesa hasta que terminas de pedir. El `if` es la decisión de traerte postre o la cuenta según lo que digas. Sin esta estructura, el programa no tiene 'memoria' operativa mientras corre. Entender esto te permite pasar de escribir 'recetas de cocina' fijas a crear sistemas dinámicos que responden al mundo real.

Del Diccionario al Objeto (POO): Organizando el Caos de Datos

Al inicio del curso usamos diccionarios para guardar datos de un post: `{'titulo': 'Hola', 'autor': 'Ana'}`. Es sencillo y rápido. Pero, ¿qué pasa cuando el blog crece? Imagina que tienes 1.000 posts y quieres asegurarte de que ningún título esté vacío o que cada autor tenga un formato de correo válido. Con diccionarios, tendrías que validar esto manualmente en cada rincón de tu código. Aquí es donde la Programación Orientada a Objetos (POO) actúa como un 'superpoder' de organización.

Una `Clase` no es más que un molde de galletas. Si el molde dice que cada galleta (objeto) debe tener chispas de chocolate, no puedes crear una galleta sin ellas. En el módulo 6, al crear la clase `Post`,

estamos definiendo un contrato. Esto evita el error típico de principiante: intentar acceder a una clave de diccionario que no existe (`KeyError`), lo que rompe todo el programa. Con objetos, el código 'sabe' qué atributos tiene un post.

Empresas como Instagram están construidas sobre este principio. En sus servidores de Python, no manejan 'carpetas de datos' sueltas, sino objetos `User` y `Photo` que tienen métodos específicos (como `.publicar()` o `.dar_like()`). Al dominar la POO, dejas de ver datos como texto suelto y empiezas a verlos como entidades con reglas propias, lo que hace que tu código sea escalable y mucho más fácil de depurar cuando algo falla.

El Patrón MVT en Django: El Mapa para no Perderse en la Web

Muchos estudiantes se sienten abrumados al pasar de un solo archivo de Python a la estructura de carpetas de Django. ¿Por qué hay tantos archivos? La respuesta es el patrón Model-View-Template (MVT). Es la arquitectura que permite que sitios web masivos como Pinterest o The Washington Post funcionen sin que sus desarrolladores se vuelvan locos.

- **El Modelo (Model)** es el bibliotecario: su único trabajo es saber dónde están los datos y cómo guardarlos en la base de datos (SQLite en nuestro caso). No sabe nada de diseño.
- **La Plantilla (Template)** es el decorador: solo le importa cómo se ve la página. Usa HTML y CSS para que el usuario esté cómodo.
- **La Vista (View)** es el director de orquesta: es el intermediario. Cuando un usuario pide ver un post, la Vista le pregunta al Modelo '¿Tienes este post?' y, cuando lo recibe, se lo entrega a la Plantilla para que lo dibuje.

El error más común es intentar poner lógica de base de datos dentro del HTML. Django separa estas capas para que, si mañana quieres cambiar tu diseño, no rompas tu base de datos. Esta separación es un estándar de la industria. Al entender el flujo MVT, dejas de copiar y pegar código y empiezas a entender el 'viaje' que hace un clic desde que el usuario presiona un botón hasta que la información aparece en pantalla. Es la diferencia entre construir una choza y diseñar un edificio que puede crecer infinitamente.

Preguntas frecuentes

P: ¿Por qué empezamos aprendiendo Python por consola si mi objetivo final es crear una página web con Django?

R: Muchos estudiantes sienten la urgencia de ver colores y botones desde el primer día, asumiendo que la consola es algo 'anticuado'. Sin embargo, la lógica de programación es el motor oculto de cualquier sitio web. Aprender variables, bucles y funciones por consola te permite concentrarte en 'cómo piensa el programa' sin la complejidad añadida de HTML o CSS. Imagina que quieres ser piloto: primero practicas en un simulador para entender los controles (Python básico) antes de despegar un avión real con pasajeros (Django). Si saltas directo a Django sin entender la lógica, te sentirás perdido cuando algo falle en el servidor, porque no sabrás si el error está en la visualización o en el procesamiento de los datos.

P: ¿Cuál es la diferencia real entre un 'entorno virtual' (venv) y simplemente instalar todo en mi computadora?

R: Esta es una duda muy común porque al principio parece un paso extra innecesario. La realidad es que, en el mundo profesional, cada proyecto es un ecosistema aislado. Imagina que tienes dos proyectos: uno usa una versión antigua de Django y otro la más reciente. Si instalas todo de forma global en tu sistema, las versiones entrarán en conflicto y nada funcionará. Un entorno virtual es como una 'caja de herramientas' específica para un solo proyecto. Lo que instalas ahí dentro no afecta al resto de tu computadora. Usar `venv` desde el inicio te ahorra el dolor de cabeza de romper proyectos anteriores al intentar actualizar uno nuevo, una práctica estándar en la industria.

P: ¿Es estrictamente necesario usar Git y GitHub si estoy trabajando solo en mi proyecto final?

R: Muchos asumen que Git es solo para trabajar en equipo, pero en realidad es tu 'máquina del tiempo' personal. Programar implica experimentar y, a veces, romper cosas que funcionaban. Sin Git, si cometes un error grave, podrías perder horas de trabajo intentando volver atrás. Con Git, puedes hacer un 'commit' (un punto de guardado) cada vez que completes una funcionalidad. Si algo sale mal, regresas al estado anterior en segundos. Además, GitHub actúa como tu portafolio profesional: para un reclutador, un repositorio bien organizado con commits frecuentes es la prueba real de que sabes construir software, mucho más que cualquier certificado.

P: Me confundo entre lo que hace un Diccionario en Python y lo que hace un Modelo en Django, ¿son lo mismo?

R: Es normal confundirlos porque ambos guardan datos relacionados. Un diccionario es una estructura temporal que vive en la memoria de tu programa mientras se ejecuta; si apagas la computadora, los datos desaparecen. Un Modelo de Django es mucho más potente: es el plano arquitectónico que define cómo se guardarán los datos de forma permanente en una base de datos (SQLite en nuestro caso). Mientras que el diccionario es como una nota adhesiva rápida en tu escritorio, el Modelo es el diseño de un archivador físico donde la información se queda guardada para siempre, permitiendo que tu blog tenga miles de posts grabados en el disco duro.

P: ¿Por qué Django usa plantillas (templates) con herencia en lugar de escribir cada página HTML completa?

R: Al principio, copiar y pegar el código del menú de navegación en cada archivo parece fácil, pero ¿qué pasa si decides cambiar una opción del menú y tienes 50 páginas? Tendrías que editarlas una por una. La herencia de plantillas con un archivo `base.html` sigue el principio DRY (*Don't Repeat Yourself* o 'No te repitas'). Definimos la estructura común (cabecera, menú, pie de página) en un solo lugar y las demás páginas solo 'rellenan' el contenido específico. Es como usar un sello: creas el diseño una vez y lo aplicas donde lo necesites. Esto no solo ahorra tiempo, sino que reduce drásticamente la posibilidad de errores tipográficos u olvidos en partes críticas del sitio.

P: Siento que me cuesta recordar la sintaxis de los bucles y condicionales, ¿debería memorizarlos antes de seguir?

R: ¡Definitivamente no! Una de las mayores barreras para los principiantes es creer que programar es memorizar comandos. La programación es resolver problemas lógicos; la sintaxis se consulta en la documentación o en tus propios ejemplos anteriores. Incluso los desarrolladores senior buscan en Google cómo escribir un bucle específico o cómo abrir un archivo. No te detengas por no recordar si va un punto o un paréntesis; lo importante es que entiendas el concepto de 'necesito repetir esta acción' o 'necesito tomar una decisión'. Con la práctica constante en los módulos del curso, tu cerebro automatizará la escritura por repetición, no por memoria forzada.

Conclusiones

Del Pensamiento Lógico a la Arquitectura Web: Una Transformación Integral

Has recorrido un camino que comenzó con simples variables en una consola y culminó en una plataforma web dinámica. El mayor aprendizaje no es solo sintaxis de Python, sino la capacidad de descomponer problemas complejos en componentes manejables (módulos, funciones y clases). Esta metodología de 'divide y vencerás' es el estándar de oro en la ingeniería de software profesional. Ahora comprendes que una aplicación no es un bloque monolítico, sino una orquestación de lógica de negocio, persistencia de datos y presentación visual, permitiéndote abordar proyectos técnicos con una estructura mental organizada.

El Patrón de Persistencia: De Archivos Planos al ORM de Django

A lo largo del programa, evolucionaste la forma de guardar información: desde listas en memoria y archivos JSON hasta una base de datos relacional gestionada por el ORM de Django. Este progreso te ha enseñado un principio fundamental: la integridad de los datos. Al usar modelos y migraciones, has aprendido a definir 'contratos' para la información, asegurando que tu sistema sea robusto y escalable. En el mundo laboral, esta habilidad para estructurar modelos de datos es lo que separa a un programador de scripts de un desarrollador de aplicaciones backend fiable.

Seguridad y Gestión de Usuarios como Pilar Profesional

La transición de un script abierto a un sistema con autenticación (Login, Logout, Registro) marca tu entrada al desarrollo de software responsable. Al integrar el sistema de usuarios de Django y proteger vistas con decoradores, has aprendido a gestionar la privacidad y la autorización. Entender cómo fluye la sesión de un usuario y cómo proteger formularios contra entradas maliciosas son competencias críticas en cualquier entorno corporativo, donde la seguridad de la información es la prioridad máxima. Ahora puedes construir herramientas que no solo funcionan, sino que son seguras para el usuario final.

El Ciclo de Vida del Software y Control de Versiones con Git

No solo aprendiste a escribir código, sino a gestionarlo profesionalmente usando Git y GitHub. Has pasado de crear archivos sueltos a mantener un repositorio con historial de cambios, ramas y documentación clara en el README. Esta es la 'fábrica' del desarrollador moderno. El control de versiones te permite experimentar sin miedo al error y colaborar en equipos multidisciplinarios. Al

finalizar este curso, posees un portafolio tangible y profesional que demuestra no solo tus habilidades técnicas, sino tu capacidad para seguir flujos de trabajo estándar de la industria.

Dominio de la Abstracción: Templates, Herencia y Modularidad

El uso de la herencia de plantillas (`base.html`) y la organización en paquetes refleja un dominio de la abstracción. Has aprendido a no repetirte (principio DRY: Don't Repeat Yourself), creando componentes reutilizables tanto en la lógica de Python como en el diseño visual con Django. Esta eficiencia es vital en el desarrollo profesional para reducir la deuda técnica y facilitar el mantenimiento de sitios grandes. Tu capacidad para separar la lógica del servidor de la interfaz de usuario te permitirá colaborar eficazmente con diseñadores y otros desarrolladores en el futuro.